BALLUFF

A brand of Balluff MATRIX VISION



mvBlueFOX3

Technical Manual

English - Version 3.05

1.1 About this manual . . . 1
1.1.1 Goal of the manual . . . 1
1.1.2 Contents of the manual . . . 1
1.2 Imprint . . . 2
1.2.1 Introduction . . . 3
1.2.2 wxWidgets . . . 3
1.2.3 Sarissa . . . 3
1.2.4 GenICam . . . 3
1.2.5 libusb . . . 3
1.2.6 libusbK . . . 3
1.2.6.1 libusbK license . . . 3
1.2.7 Doxygen . . . 4
1.2.7.1 Doxygen license . . . 4
1.2.8 SHA1 algorithm . . . 4
1.2.9 Expat . . . 5
1.2.9.1 Expat Copyright . . . 5
1.2.10 CppUnit . . . 5
1.2.11 NUnit . . . 5
1.2.11.1 NUnit License . . . 5
1.3 Legal notice . . . 6
1.3.1 Introduction . . . 6
1.3.2 cJSON . . . 6
1.3.3 Unity . . . 6
1.4 Revisions . . . 7
1.5 Symbols and Conventions . . . 8
1.5.1 Explanation of the warnings . . . 8
1.6 Important Information . . . 9
1.6.1 Important Safety Instructions . . . 9
1.6.2 Operating considerations . . . 9
1.6.2.1 Important Safety Notes . . . 9
1.6.2.2 Handling And Cleaning . . . 10
1.6.2.3 Installing . . . 10
1.6.2.4 Optimizing performance and life time . . . 11
1.6.2.5 Connectors . . . 11
1.6.2.6 Cleaning . . . 11
1.6.3 Conformity . . . 13
1.6.4 Additional notices . . . 20
1.6.4.1 For customers in the U.S.A. . . . 20
1.6.4.2 For customers in Canada . . . 20
1.6.4.3 Pour utilisateurs au Canada . . . 20
1.7 Introduction . . . 21
1.7.1 Software concept . . . 22

1.7.2 Order code nomenclatures . . . 23
1.7.2.1 mvBlueFOX3-1 . . . 23
1.7.2.2 mvBlueFOX3-M1 . . . 24
1.7.2.3 mvBlueFOX3-2 . . . 25
1.7.2.4 mvBlueFOX3-M2 . . . 27
1.7.2.5 mvBlueFOX3-3M . . . 29
1.7.2.6 mvBlueFOX3-4 . . . 30
1.7.2.7 mvBlueFOX3-5M . . . 31
1.7.2.8 Ordering code samples . . . 32
1.7.3 What's inside and accessories . . . 33
1.7.3.1 Accessories for the mvBlueFOX3 . . . 34
1.8 Quickstart . . . 36
1.8.1 System Requirements . . . 36
1.8.1.1 Host System . . . 36
1.8.1.2 Supported Operating Systems . . . 36
1.8.2 Installing The mvGenTL-Acquire Package . . . 37
1.8.2.1 Windows . . . 38
1.8.2.2 Linux . . . 41
1.8.3 Connecting The Camera . . . 44
1.8.3.1 Communicating With The Camera . . . 44
1.8.3.2 Setting Up The Camera . . . 44
1.8.3.3 About Settings . . . 45
1.8.4 Driver concept . . . 47
1.8.4.1 NeuroCheck Support . . . 48
1.8.4.2 VisionPro Support . . . 49
1.8.4.3 HALCON Support . . . 49
1.8.4.4 LabVIEW Support . . . 49
1.8.4.5 DirectShow Support . . . 49
1.8.4.6 Micro-Manager Support . . . 49
1.8.5 Relationship Between Driver, Firmware And SDK . . . 50
1.8.6 Optimizing USB Performance . . . 53
1.8.6.1 Checklist for Windows . . . 53
1.8.6.2 Checklist for Linux . . . 53
1.8.7 Using USB3 Vision™ Devices In A Docker Container . . . 55
1.8.7.1 Host Preparation . . . 56
1.8.7.2 Building A Docker Image . . . 56
1.8.7.3 Starting The Docker Container . . . 57
1.8.7.4 Validation . . . 58
1.9 Technical Data . . . 58
1.9.1 Dimensions . . . 58
1.9.1.1 Standard model (mvBlueFOX3-1) . . . 58
1.9.1.2 Standard model (mvBlueFOX3-2) . . . 59

1.9.1.3 Model without housing (mvBlueFOX3-M1) . . . 61
1.9.1.4 Model without housing (mvBlueFOX3-M2) . . . 63
1.9.1.5 Single-board Model for Embedded Vision (mvBlueFOX3-3M) . . . 66
1.9.1.6 Hi-res model (mvBlueFOX3-4) . . . 66
1.9.1.7 Board-level Model for Embedded Vision (mvBlueFOX3-5M) . . . 67
1.9.2 Camera interfaces (mvBlueFOX3-1,mvBlueFOX3-2,mvBlueFOX3-M1,mvBlueFOX3-M2,mvBlueFOX3-4) . . . 68
1.9.2.1 Circular connector male (Power / Digital I/O) . . . 68
1.9.2.2 Characteristics of the digital inputs . . . 69
1.9.2.3 Characteristics of the digital outputs . . . 70
1.9.3 Status / Power LED . . . 71
1.9.3.1 Standard model (mvBlueFOX3-1) . . . 71
1.9.3.2 Standard model (mvBlueFOX3-2) . . . 72
1.9.4 BFembedded interface (mvBlueFOX3-3M,mvBlueFOX3-5M) . . . 72
1.9.4.1 Pin assignment . . . 73
1.9.4.2 Boards for the BFembedded interface (mvBlueFOX3-3M,mvBlueFOX3-5M) . . . 75
1.9.5 Components . . . 81
1.10 Sensor Overview . . . 83
1.10.1 Image data flow . . . 83
1.10.2 Output sequence of color sensors (RGB Bayer) . . . 83
1.10.3 Bilinear interpolation of color sensors (RGB Bayer) . . . 84
1.10.4 CMOS sensors . . . 84
1.10.4.1 Details of operation . . . 84
1.10.4.2 Models . . . 86
1.10.5 Supported image formats . . . 95
1.11 Filters and lenses . . . 95
1.11.1 Hot Mirror Filter . . . 95
1.11.2 Cold mirror filter . . . 97
1.11.3 Glass filter . . . 98
1.11.4 Lenses . . . 98
1.12 GUI tools . . . 99
1.12.1 Introduction . . . 99
1.12.2 wxPropView . . . 99
1.12.3 mvDeviceConfigure . . . 99
1.13 GenICam and advanced features . . . 100
1.13.1 Introduction . . . 100
1.13.2 Device Control . . . 101
1.13.3 Image Format Control . . . 102
1.13.4 Acquisition Control . . . 103
1.13.5 Counter And Timer Control . . . 107
1.13.6 Analog Control . . . 109
1.13.7 Color Transformation Control . . . 111

1.13.8 Event Control . . . 112
1.13.9 Chunk Data Control . . . 113
1.13.10 File Access Control . . . 114
1.13.11 Digital I/O Control . . . 114
1.13.12 Encoder Control . . . 115
1.13.13 Sequencer Control . . . 116
1.13.13.1 Sequencer overview . . . 116
1.13.13.2 Configuration of a sequencer set . . . 116
1.13.14 Transport Layer Control . . . 121
1.13.15 User Set Control . . . 122
1.13.16 mv Logic Gate Control . . . 123
1.13.17 mv Flat Field Correction Control . . . 124
1.13.18 mv Serial Interface Control . . . 125
1.13.19 mv I2C Interface Control . . . 125
1.13.20 mv Defective Pixel Correction Control . . . 126
1.13.21 mv Frame Average Control (only with specific models) . . . 126
1.13.22 mv Auto Feature Control . . . 126
1.13.23 mv High Dynamic Range Control (only with specific sensor models) . . . 127
1.13.24 LUT Control . . . 127
1.13.24.1 mvLUTType . . . 129
1.13.24.2 mvLUTInputData . . . 129
1.13.24.3 mvLUTMapping . . . 129
1.13.24.4 LUT support in MATRIX VISION cameras . . . 130
1.14 Developing applications using the mvIMPACT Acquire SDK . . . 133
1.15 DirectShow interface . . . 134
1.15.1 Supported interfaces . . . 134
1.15.1.1 IAMCameraControl . . . 134
1.15.1.2 IAMDroppedFrames . . . 134
1.15.1.3 IAMStreamConfig . . . 134
1.15.1.4 IAMVideoProcAmp . . . 134
1.15.1.5 IKsPropertySet . . . 134
1.15.1.6 ISpecifyPropertyPages . . . 134
1.15.2 Logging . . . 134
1.15.3 Setting up devices for DirectShow usage . . . 134
1.15.3.1 Registering devices . . . 135
1.15.3.2 Renaming devices . . . 137
1.15.3.3 Using regsvr32 . . . 138
1.16 Troubleshooting . . . 139
1.16.1 Error code list . . . 139
1.16.2 Accessing log files . . . 152
1.16.2.1 Windows . . . 152
1.16.2.2 Linux . . . 152

1.16.3 General Issues . . . 153
1.16.3.1 The error counter increases . . . 154
1.16.3.2 I get an oscillating frame rate . . . 155
1.16.3.3 Why does updating the device list take so long . . . 156
1.16.4 Windows . . . 157
1.16.4.1 Calling AcquisitionStart For The First Time In A Process Takes A Longer Time . . . 157
1.16.4.2 mvBlueFOX3 or other USB3 Vision devices connected to the system are not detected by mvIMPACT Acquire or cannot be initialised . . . 158
1.16.4.3 After installing a new driver version or after initially installing the mvGenTL_Acquire package USB3 Vision devices are not accessible . . . 162
1.16.5 Linux . . . 162
1.16.5.1 No GenICam devices are detected on a Linux system . . . 163
1.16.5.2 Image transfer From USB3 Vision™ Devices Stops Randomly On A Linux System 164
1.17 Glossary . . . 165
1.18 Use Cases . . . 174
1.18.1 Introducing acquisition / recording possibilities . . . 175
1.18.1.1 Acquiring a number of images . . . 175
1.18.1.2 Recording sequences with pre-trigger . . . 176
1.18.1.3 Creating acquisition sequences (Sequencer Control) . . . 178
1.18.1.4 Generating very long exposure times . . . 191
1.18.1.5 Working with multiple AOIs (mv Multi Area Mode) . . . 193
1.18.1.6 Working with burst mode buffer . . . 197
1.18.1.7 Using the SmartFrameRecall feature . . . 201
1.18.1.8 Using The Linescan mode . . . 204
1.18.1.9 Using the mvBlockscan feature . . . 208
1.18.1.10 Working with Event Control . . . 210
1.18.1.11 Polarized Data Extraction . . . 213
1.18.1.12 Using Video Stream Recording . . . 217
1.18.2 Improving the acquisition / image quality . . . 228
1.18.2.1 Correcting image errors of a sensor . . . 228
1.18.2.2 Optimizing the color/luminance fidelity of the camera . . . 236
1.18.2.3 Reducing noise by frame averaging . . . 250
1.18.2.4 Optimizing the bandwidth . . . 253
1.18.2.5 Setting a flicker-free auto expose and auto gain . . . 255
1.18.2.6 Working with binning / decimation . . . 258
1.18.2.7 Minimizing sensor pattern of mvBlueFOX3-1100G . . . 261
1.18.2.8 Working with the dual gain feature of mvBlueFOX3-2071/2071a . . . 263
1.18.2.9 Working With Gain And Black-Level Values Per Color Channel . . . 267
1.18.3 Improving the communication . . . 275
1.18.3.1 Optimizing the bandwidth . . . 275
1.18.4 Working with triggers . . . 276
1.18.4.1 Processing triggers from an incremental encoder . . . 277
1.18.4.2 Generating a pulse width modulation (PWM) . . . 280

1.18.4.3 Outputting a pulse at every other external trigger . . . 282
1.18.4.4 Creating different exposure times for consecutive images . . . 284
1.18.4.5 Detecting overtriggering . . . 287
1.18.4.6 Triggering of an indefinite sequence with precise starting time . . . 291
1.18.4.7 Low latency triggering . . . 294
1.18.5 Working with I/Os . . . 295
1.18.5.1 Controlling strobe or flash at the outputs . . . 296
1.18.5.2 Creating a debouncing filter at the inputs . . . 299
1.18.6 Working with HDR (High Dynamic Range Control) . . . 302
1.18.6.1 Adjusting sensor of camera models -x02d (-1012d) . . . 302
1.18.6.2 Adjusting sensor of camera models -x02e (-1013) / -x04e (-1020) . . . 303
1.18.6.3 Adjusting sensor of camera models -1031C . . . 306
1.18.7 Working with LUTs . . . 308
1.18.7.1 Introducing LUTs . . . 308
1.18.7.2 Working with LUTValueAll . . . 312
1.18.7.3 Implementing a hardware-based binarization . . . 314
1.18.8 Saving data on the device . . . 316
1.18.8.1 Creating user data entries . . . 316
1.18.8.2 Creating user set entries . . . 318
1.18.8.3 Working with the UserFile section (Flash memory) . . . 321
1.18.9 Working with device features . . . 325
1.18.9.1 Reset timestamp by hardware . . . 325
1.18.9.2 Synchronizing camera timestamps . . . 326
1.18.9.3 Using the standby mode . . . 330
1.18.9.4 Working With The Serial Interface (mv Serial Interface Control) . . . 333
1.18.9.5 Working with the I2C interface (mv I2C Interface Control) . . . 337
1.18.10 Working with several cameras simultaneously . . . 339
1.18.10.1 Creating synchronized acquisitions using timers . . . 339
1.18.11 Working with 3rd party tools . . . 345
1.18.11.1 Using VLC Media Player . . . 345
1.18.11.2 Working with ROS (Robot Operating System) . . . 347
1.18.11.3 Using USB3 Vision™ Devices In A Docker Container . . . 350
1.19 Working with the temperature sensors . . . 353
1.20 Appendix A. Specific Camera / Sensor Data . . . 354
1.20.1 A.1 Pregius CMOS . . . 355
1.20.1.1 Pregius S . . . 355
1.20.1.2 Pregius . . . 376
1.20.2 A.2 Starvis CMOS . . . 445
1.20.2.1 mvBlueFOX3-2064 / BF3-3M-0064Z / BF3-5M-0064Z (6.4 Mpix [3096 x 2080]) . . . 445
1.20.2.2 mvBlueFOX3-2124r / BF3-5M-0124R (12.4 Mpix [4064 x 3044]) . . . 450
1.20.2.3 mvBlueFOX3-2205 / BF3-5M-0205Z (20.5 Mpix [5544 x 3692]) . . . 454
1.20.3 A.3 Polarsens CMOS . . . 459

1.20.3.1 mvBlueFOX3-2051p (5.1 Mpix [2464 x 2056]) . . . 459
1.20.4 A.4 CMOS . . . 463
1.20.4.1 mvBlueFOX3-1012b (1.2 Mpix [1280 x 960]) . . . 463
1.20.4.2 mvBlueFOX3-1012d (1.2 Mpix [1280 x 960]) . . . 466
1.20.4.3 mvBlueFOX3-1013 (1.3 Mpix [1280 x 1024]) . . . 469
1.20.4.4 mvBlueFOX3-1020 (1.9 Mpix [1600 x 1200]) . . . 473
1.20.4.5 mvBlueFOX3-1020a (1.9 Mpix [1600 x 1200]) . . . 476
1.20.4.6 mvBlueFOX3-1031 (3.2 Mpix [2048 x 1536]) . . . 479
1.20.4.7 mvBlueFOX3-1100 (11 Mpix [3856 x 2764]) . . . 481
1.20.4.8 mvBlueFOX3-1140 (14 Mpix [4384 x 3288]) . . . 485
1.21 Appendix C. Tested ARM platforms . . . 489
1.21.1 C.1 ARM64 based devices . . . 490
1.21.1.1 NVIDIA Jetson AGX Xavier . . . 490
1.21.1.2 NVIDIA Jetson Xavier NX . . . 493
1.21.1.3 NVIDIA Jetson Nano . . . 496
1.21.1.4 NVIDIA Jetson TX2 . . . 499
1.21.1.5 i.MX8M Mini . . . 502
1.21.2 C.2 ARMhf based devices . . . 504
1.21.2.1 Raspberry Pi 4 . . . 504

1.1 About this manual

1.1.1 Goal of the manual

This manual gives you an overview of the mvBlueFOX3, MATRIX VISION's compact USB3 industrial camera family compliant to USB3 Vision, its technical data and basic operation of the mvBlueFOX3. Programming the device is detailed in a separate documentation, which will be available in an online format.

1.1.2 Contents of the manual

At the beginning of the manual, you will get an **introduction** (p. 21) to the possible usages of the camera. The following chapters contain general information about the camera including:

- **Quickstart** (p. 36) followed by
- **Technical Data** (p. 58)
- **Sensor Overview** (p. 83)
- **Filters and lenses** (p. 95)

The general information is followed by the description of the

- **Software tools for the camera** (p. 99) including the tools
- **GenICam and advanced features** (p. 100) introduces the GenICam and the advanced features of the camera (the cameras are GenICam compliant devices).
- **Developing applications using the mvIMPACT Acquire SDK** (p. 133)
- **DirectShow developers** (p. 134) documents MATRIX VISION's mvIMPACT Acquire to DirectShow interface(DirectShow_acquire).
- **Troubleshooting** (p. 139) shows how to detect damages and other inconveniences.
- **Use Cases** (p. 174) describes solutions for general tasks and
- A **Glossary** (p. 165) explains abbreviations and technical terms.
  - **A.1 Pregius CMOS** (p. 355) contains all data of the Pregius CMOS sensors like timings, details of operation, etc.
  - **A.2 Starvis CMOS** (p. 445) contains all data of the Starvis CMOS sensors like timings, details of operation, etc.
  - **A.3 Polarsens CMOS** (p. 459) contains all data of the Polarsens CMOS sensors like timings, details of operation, etc.
  - **A.4 CMOS** (p. 463) contains all data of the other CMOS sensors like timings, details of operation, etc.
- **Appendix C. Tested ARM platforms** (p. 489) contains a list of ARM platforms tested with this product and information on how to setup these systems for achieving optimal results

1.2 Imprint

MATRIX VISION GmbH
Talstrasse 16
DE - 71570 Oppenweiler
Telephone: +49-7191-9432-0
Fax: +49-7191-9432-288
Website: http://www.matrix-vision.de
E-Mail:

- info@matrix-vision.de
- jobs@matrix-vision.de

Author

U. Lansche
H. Mattfeldt
S. Battmer
U. Hagmaier
D. Neuholz

Date

2020

This document assumes a general knowledge of PCs and programming.

Since the documentation is published electronically, an updated version may be available online. For this reason we recommend checking for updates on the MATRIX VISION website.

MATRIX VISION cannot guarantee that the data is free of errors or is accurate and complete and, therefore, assumes no liability for loss or damage of any kind incurred directly or indirectly through the use of the information of this document.

MATRIX VISION reserves the right to change technical data and design and specifications of the described products at any time without notice.

Copyright

MATRIX VISION GmbH. All rights reserved. The text, images and graphical content are protected by copyright and other laws which protect intellectual property. It is not permitted to copy or modify them for trade use or transfer. They may not be used on websites.

- Windows® XP, Windows® Vista, Windows® 7, 8, 10 are trademarks of Microsoft, Corp.
- Linux® is a trademark of Linus Torvalds.
- Jetson is a registered trademark of NVIDIA Corporation.
- NVIDIA and Jetson are trademarks and/or registered trademarks of NVIDIA Corporation in the U.S. and other countries.
- Arm and Cortex are registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere.
- GenICam™ is a trademark of the GenICam™ standard group.
- USB3 Vision™ and the distinctive logo are trademarks owned by the Automated Imaging Association and may only be used under license for compliant products registered with the AIA.

1.2.1 Introduction

The mvIMPACT Acuire SDK and its underlying libraries and drivers as well as some of the applications shipped with the mvIMPACT Acquire packages make use of a couple of third party software packages that come with various licenses. This section is meant to list all these packages and to give credit to those whose code helped in the creation of the mvIMPACT Acquire SDK.

1.2.2 wxWidgets

Most of the applications offering a graphical user interface have been written using wxWidgets (http://www.↩ wxwidgets.org/).

wxWidgets is a C++ library that lets developers create applications for Windows, OS X, Linux and Unix on 32-bit and 64-bit architectures as well as several mobile platforms including Windows Mobile, iPhone SDK and embedded GTK+. Please refer to the wxWidgets website for detailed license information.

The source code of the applications provided by MATRIX VISION GmbH (http://www.matrix-vision.↩ com) using wxWidgets is either part of the packet this document was taken from or can be obtained by contacting MATRIX VISON GmbH.

1.2.3 Sarissa

Parts of the log file creation and the log file display make use of Sarissa (Website: http://dev.abiss.↩ gr/sarissa) which is distributed under the GNU GPL version 2 or higher, GNU LGPL version 2.1 or higher and Apache Software License 2.0 or higher. The Apache Software License 2.0 is part of this driver package.

1.2.4 GenICam

At least one driver package shipped under the product family name mvIMPACT Acquire makes use of the **GenICam** (p. 165) reference implementation, which is hosted by the EVMA and can be downloaded from their website: http://www.emva.org. All license files belonging to the **GenICam** (p. 165) reference implementation are shipped with the libraries belonging to the **GenICam** (p. 165) runtime.

1.2.5 libusb

The Linux version of the **mvBlueFOX** driver package makes use of a modified version of libusb (http://www.↩ libusb.org/), which comes under LGPL 2.1. The full license text is included in the Linux distribution of the **mvBlueFOX** driver package. The source code for the modified version of libusb can be obtained by contacting MATRIX VISION GmbH or it can be downloaded from here: http://gpl.matrix-vision.com (navigate to *others/libusb*).

1.2.6 libusbK

The USB3 Vision implementation currently makes use of libusbK (http://libusbk.sourceforge.net) written by Travis Lee Robinson who owns all rights for the source code of all modules belonging to the libusbK framework.

1.2.6.1 libusbK license

```
APPLICABLE FOR ALL LIBUSBK BINARIES AND SOURCE CODE UNLESS OTHERWISE SPECIFIED.
PLEASE SEE INDIVIDUAL COMPONENTS LICENSING TERMS FOR DETAILS.
```

Note

Portions of dpscat use source code from libwdi which is licensed for LGPL use only. (See `dpscat.c`)
`libusbK-inf-wizard.exe` is linked to libwdi which is licensed for LGPL use only.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Travis Lee Robinson nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

```
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES,
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF
MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL TRAVIS ROBINSON BE LIABLE FOR
ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF
USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE
OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

1.2.7 Doxygen

All the documentation belonging to the mvIMPACT Acquire framework has been generated using Doxygen (`http://www.doxygen.org/`) written by Dimitri van Heesch.

1.2.7.1 Doxygen license

Copyright © 1997-2013 by Dimitri van Heesch.

Permission to use, copy, modify, and distribute this software and its documentation under the terms of the GNU General Public License is hereby granted. No representations are made about the suitability of this software for any purpose. It is provided "as is" without express or implied warranty. See the GNU General Public License for more details.

Documents produced by Doxygen are derivative works derived from the input used in their production; they are not affected by this license.

1.2.8 SHA1 algorithm

Parts of this framework make use of an open source implementation of the SHA1 algorithm written by Dominik Reichl (`http://www.dominik-reichl.de`).

1.2.9 Expat

Expat is used to parse XML strings within the SDK.

1.2.9.1 Expat Copyright

Copyright (c) 1998, 1999, 2000 Thai Open Source Software Center Ltd

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

```
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

1.2.10 CppUnit

The C and C++ code is tested using the CppUnit (`http://cppunit.sourceforge.net`) framework, which come under GNU LESSER GENERAL PUBLIC LICENSE Version 2.1, February 1999.

1.2.11 NUnit

The .NET code is tested using the NUnit (`http://www.nunit.org/`) framework.

1.2.11.1 NUnit License

Copyright © 2002-2014, 2018 Charlie Poole Copyright © 2002-2004 James W. Newkirk, Michael C. Two, Alexei A. Vorontsov Copyright © 2000-2002 Philip A. Craig

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment (see the following) in the product documentation is required. Portions Copyright © 2002-2014, 2018 Charlie Poole or Copyright © 2002-2004 James W. Newkirk, Michael C. Two, Alexei A. Vorontsov or Copyright © 2000-2002 Philip A. Craig
2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

1.3 Legal notice

1.3.1 Introduction

The firmware running on MATRIX VISION devices make use of a couple of third party software packages that come with various licenses. This section is meant to list all these packages and to give credit to those whose code helped in the creation of this software:

1.3.2 cJSON

A slightly modified version of cJSON is used inside some of the modules that eventually build up the firmware.


```
Copyright (c) 2009 Dave Gamble

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
THE SOFTWARE.
```


1.3.3 Unity

A slightly modified version of Unity (`https://github.com/ThrowTheSwitch/Unity`) is used for unit testing various modules that eventually build up the firmware.

1.4 Revisions

Date	Rev.	Author	Description	Driver / Firmware version
17. May 2022	V3.05	LAN	Added **mvBlueFOX3-2017 / BF3-5M-0017Z (1.8 Mpix [1600 x 1104])** (p. 384) .	
17. September 2021	V3.04	LAN	New control **mv Auto Feature Control** (p. 126) .	FW Revision 2.40.↩ 2546.0
24. August 2021	V3.03	LAN	Updated t_{ON} of "digital input delay".	
19. August 2021	V3.02	LAN	Updated power consumption information.	
01. July 2021	V3.01	LAN	Added use case **Low latency triggering** (p. 294).	
03. May 2021	V3.00	LAN	Corrected **Symbols and Conventions** (p. 8).	
24. March 2021	V2.02	LAN	Added use case section **Improving the communication** (p. 275).	
08. March 2021	V2.01	LAN	Added sensors **mvBlueFOX3-2051d / BF3-5M-0051D (5.1 Mpix [2472 x 2064])** (p. 355) **mvBlueFOX3-2081a / BF3-5M-0081A (8.1 Mpix [2856 x 2848])** (p. 359) **mvBlueFOX3-2124d / BF3-5M-0124D (12.4 Mpix [4128 x 3008])** (p. 362).	
13. January 2021	V2.00	LAN	Seperated **GUI tools** (p. 99).	

1.5 Symbols and Conventions

Note

This symbol indicates general notes.

1.5.1 Explanation of the warnings

Always observe the warnings in these instructions and the measures described to avoid hazards. The warnings used here contain various signal words and are structured as follows:

Attention

SIGNAL WORD
"Type and source of the hazard"

Consequences if not complied with

→ Measures to avoid hazards.

The individual signal words mean:

Attention

Indicates a **danger** that can lead to **damage** or **destruction** of the product.

All due care and attention has been taken in preparing this manual. In view of our policy of continuous product improvement, however, we can accept no liability for completeness and correctness of the information contained in this manual. We make every effort to provide you with a flawless product.

In the context of the applicable statutory regulations, we shall accept no liability for direct damage, indirect damage or third-party damage resulting from the acquisition or operation of a MATRIX VISION product. Our liability for intent and gross negligence is unaffected. In any case, the extend of our liability shall be limited to the purchase price.

1.6 Important Information



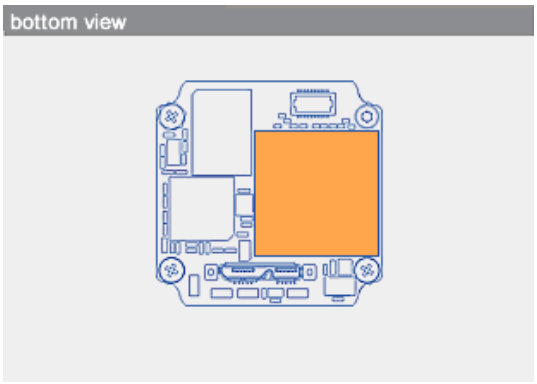
1.6.1 Important Safety Instructions

- We cannot and do not take any responsibility for the damage caused to you or to any other equipment connected to a MATRIX VISION device. Similarly, warranty will be void, if a damage is caused by not following the manual.
- Handle your MATRIX VISION device with care. Do not misuse it. Avoid shaking, striking, etc.. Your MATRIX VISION device could be damaged by faulty handling or shortage.
- Do not use accessories not recommended by the product manufacturer as they may cause hazards.
- The product should be situated away from heat sources such as radiators, heat registers, stoves, or other products (including amplifiers) that produce heat.
- Using the board-level version:
  - Provide sufficient cooling because single components can reach high temperatures.
  - Handle with care and avoid damage of electrical components by electrostatic discharge (ESD):
    - Discharge body static (contact a grounded surface and maintain contact).
    - Avoid all plastic, vinyl, and Styrofoam (except anti-static versions) around printed circuit boards.
    - Do not touch components on the printed circuit board with your hands or with conductive devices.

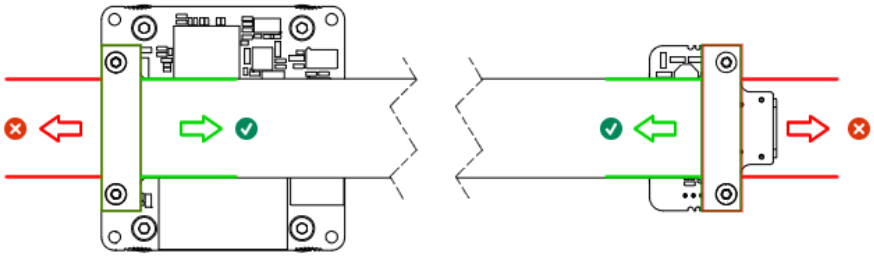
1.6.2 Operating considerations

1.6.2.1 Important Safety Notes

- Use this camera with a suitable power supply with following specifications: 12V, 2.5A $\pm$ 5% or 24V, 1.25A, $\pm$ 5%.
- Using the board-level cameras like **mvBlueFOX3-M1** (p. 61), **mvBlueFOX3-M2** (p. 63), **mvBlueFOX3-3M** (p. 66), or **mvBlueFOX3-5M** (p. 67):
  - Handle with care and avoid damage of electrical components by electrostatic discharge (ESD):
    - Discharge body static (contact a grounded surface and maintain contact).
    - Avoid all plastic, vinyl, and Styrofoam (except anti-static versions) around printed circuit boards.
    - Do not touch components on the printed circuit board with your hands or with conductive devices.
  - Be careful when bending the flex cable of the **mvBlueFOX3-M1** (p. 61). The minimum bending radius is approx. 3 mm.
  - Provide sufficient cooling because single components can reach high temperatures. Inadequate and incorrect cooling invalidate the guarantee. For heat dissipation of the **mvBlueFOX3-M2xxx-1111**, we recommend the surface of the FPGA (orange area of the following figure):



– Because the connector of the rigid-flex extension cable (BFE-FLEX) of the **mvBlueFOX3-3M** (p. 66) or **mvBlueFOX3-5M** (p. 67) can be connected in both ways you have to check if it is connected the right way and that is: **The cable must be lead away over each board**



Attention

"Overheating"

The mainboards temperature may not exceed 80 °C otherwise the device can be damaged.

1.6.2.2 Handling And Cleaning

- Do not attempt to disassemble camera.
- When installing or removing a lens, take care that water or dust does not enter the inside of the camera.

1.6.2.3 Installing

Avoid installing or storing the camera in the following environments:

- Environments exposed to direct sunlight, rain or snow.
- Environments where combustible or corrosive gas exists.
- Excessively warm or cold environment (Operating ambient temperature: 0 to 45 °C)
- Humid or dusty environment.
- Place subjected to excessive vibration or shock.
- Environment exposed to strong electric or magnetic field.
- It is recommended to mount the camera on a thermoconducting surface such as aluminum or other metals rather than plastic or wood.
- Please contact manufacturer or local distributor if you want to use additional enclosures for higher ingress protection.
- Do not aim the camera at the sun or other very strong light sources, otherwise you can destroy the image sensor.

1.6.2.4 Optimizing performance and life time

When the camera is used continuously for long time under high ambient temperature, the inside electrical parts may deteriorate, resulting in shorter life span. Additional cooling by e.g. air convection is recommended.

1.6.2.5 Connectors

Confirm the power is off before connecting or disconnecting a signal cable. Grap connectors by the body, not the attached wires.

1.6.2.6 Cleaning

- Use a blower or a lens brush to remove dust on the lens or the optical filter.
- Do not disassemble front flange.
- Clean case with dry soft cloth. Use neutral detergent liquid if needed; wipe the cover with dry cloth.
- Do not use benzene, thinner, alcohol, liquid cleaner or spray-type cleaner.

1.6.2.6.1 Adjusting the C-mount (mvBlueFOX3-2xxx-1xxx) The mvBlueFOX3-2xxx-1xxx does not support back focus adjustment. However, with the four screw locks at the front of the lens holder, it is possible to rotate the C-mount ring.

> **Note**
>
> In combination with mvBlueFOX3-2089 and mvBlueFOX3-2124 the C-mount lens holder has to look upwards during the adjusting. Otherwise the aperture can jump out of the guide.

- Loosen the screw locks with an Allen key (2.5 mm).
- With it, you can adjust the position of the lens, for example, to have the scale or the locking screws of the lens at a specific position.

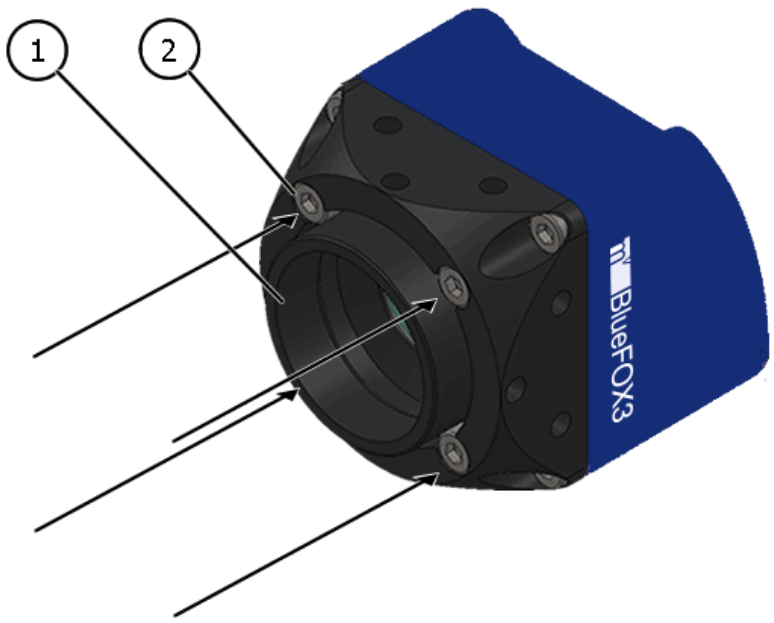


Figure 1: mvBlueFOX3-2xxx-1xxx Lensholder with C-mount ring (1) and screw locks (2)

Note

Always tighten the screws in a diagonal sequence first slightly and then little by little to a torque of 0.9 Nm.

1.6.2.6.2 Adjusting the C-mount (mvBlueFOX3-2xxx-2xxx) The mvBlueFOX3-2xxx-2xxx cameras allow a precise adjustment of the back focus of the C-mount by means of a back focus ring which is threaded into the C-mount and is secured by a lock nut ring which itself is secured by two screws. The mechanical adjustment of the imaging device is important in order to achieve a perfect alignment with the focal point of the lens. This adjustment is made before leaving the factory to conform to the standard of 17.526 mm (in air) and should normally not require adjustment in the field. However, if the back focal plane of your lens does not conform to the C-mount back focus specification or if you have e.g. removed the **IR-CUT filter** (p. 95), renewed adjustment may be required.

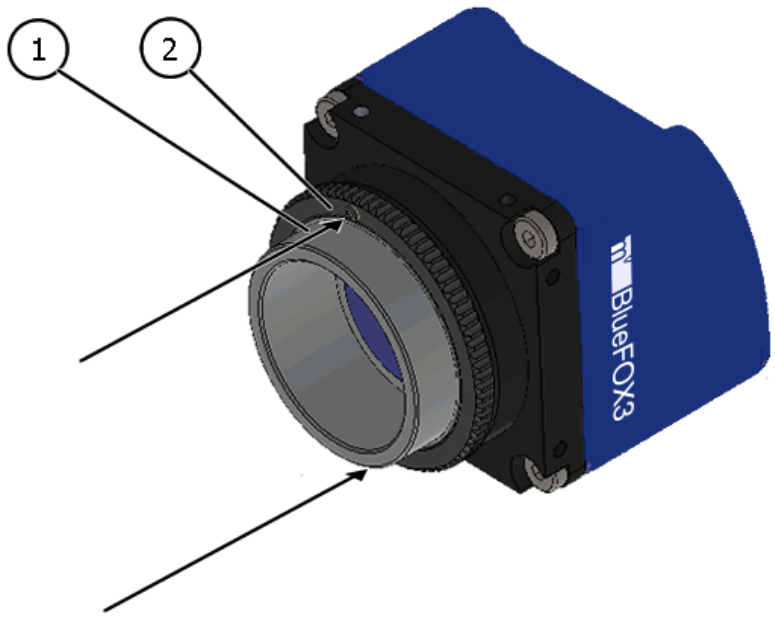


Figure 2: mvBlueFOX3-2xxx-2xxx Lensholder with C-mount ring (1) and lock nut ring (2)

How to proceed:

- Loosen screws (location as shown above by arrows) of the lock nut ring with an Allen key (0.9 x 50).
- Loosen the lock nut ring.
- With the lens set to infinity or a known focus distance, set the camera to view an object located at "infinity" or the known distance.
- Rotate the C-mount ring and lens forward or backwards on its thread until the object is in sharp focus.

  Note

  Be careful that the lens remains seated in the C-mount.

- Once focus is achieved, tighten the lock nut ring, then tighten the two locking screws of the lock ring without applying excessive torque.

1.6.3 Conformity

CE	The mvBlueFOX3 is in conformity with all applicable essential requirements necessary for CE marking. It corresponds to the EU EMC guideline 2014/30/EU based on the following harmonized standards Electromagnetic compatibility (EMC) • Interference emission EN 61000-6-3 / 2007 • Interference immunity EN 61000-6-2 / 2005
UK CA	
	MATRIX VISION corresponds to the EU guideline WEEE 2002/96/EG on waste electrical and electronic equipment and is registered under WEEE-Reg.-No. DE 25244305.
RoHS	All units delivered are RoHS compliant.
IP30 [1]	mvBlueFOX3

[1] not evaluated by UL

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueFOX3-1**
Type: ***mvBlueFOX3-1***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

A brand of Balluff
MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueFOX3-M1**
Type: ***mvBlueFOX3-M1***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung eines geeigneten EMV-Gehäuses sowie hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of a suitable EMC-case, as well as the use of a high-quality shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueFOX3-2 (BVS CA-SF2)**
Type: ***mvBlueFOX3-2 (BVS CA-SF2)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueFOX3-3M/5M (BVS CA-SF3/SF5)**
Type: ***mvBlueFOX3-3M/5M (BVS CA-SF3/SF5)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung eines geeigneten EMV-Gehäuses sowie hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of a suitable EMC-case, as well as the use of a high-quality shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueFOX3-M2 (BVS CA-SF2)**
Type: ***mvBlueFOX3-M2 (BVS CA-SF2)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung eines geeigneten EMV-Gehäuses sowie hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of a suitable EMC-case, as well as the use of a high-quality shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueFOX3-4 (BVS CA-SF4)**
Type: ***mvBlueFOX3-4 (BVS CA-SF4)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

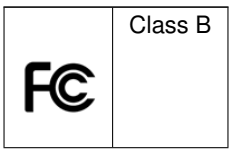
Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

1.6.4 Additional notices

1.6.4.1 For customers in the U.S.A.



This equipment has been tested and found to comply with the limits for a Class B digital device, pursuant to Part 15 of the FCC Rules. These limits are designed to provide reasonable protection against harmful interference when the equipment is operated in a residential environment. This equipment generates, uses, and can radiate radio frequency energy and, if not installed and used in accordance with the instruction manual, may cause harmful interference to radio communications. However there is no guarantee that interferences will not occur in a particular installation. If the equipment does cause harmful interference to radio or television reception, the user is encouraged to try to correct the interference by one or more of the following measures:

- Reorient or relocate the receiving antenna.
- Increase the distance between the equipment and the receiver.
- Use a different line outlet for the receiver.
- Consult a radio or TV technician for help.

You are cautioned that any changes or modifications not expressly approved in this manual could void your authority to operate this equipment. The shielded interface cable recommended in this manual must be used with this equipment in order to comply with the limits for a computing device pursuant to Subpart B of Part 15 of FCC Rules.

1.6.4.2 For customers in Canada

This apparatus complies with the Class B limits for radio noise emissions set out in the Radio Interference Regulations.

1.6.4.3 Pour utilisateurs au Canada

Cet appareil est conforme aux normes classe B pour bruits radioélectriques, spécifiées dans le Règlement sur le brouillage radioélectrique.

1.7 Introduction



Figure 1: mvBlueFOX3-1

The mvBlueFOX3 is a compact USB 3 camera which is compliant to the brand new vision standard **USB3 Vision** (p. 173). The mvBlueFOX3 offers

- a wide range of CMOS sensors,
- high frame rates,
- I/Os suitable for industrial applications, and
- wide range of resolutions.

The image memory of the camera enables a high-speed-buffer mode (a.k.a. burst mode) which writes images faster in the camera's memory as they are transferred. With this mode, image losses are a thing of the past.

The mvBlueFOX3 is ideally suited for all classical areas of machine vision and especially for applications in the medicine and microscopy area. The hardware capabilities can be teamed with MATRIX VISION's machine vision library mvIMPACT or any other third party libraries which are compliant to USB3 Vision.

1.7.1 Software concept

The mvBlueFOX3 is a **USB3 Vision** (p. 173) compliant device, using a **GenICam** (p. 165) XML-file describing the device capabilities. Within this XML-file it uses the names, units, data types etc. recommended by the **SFNC** (p. 173) to describe the devices features. Whenever custom features have been added to the mvBlueFOX3 firmware these can clearly be spotted by the leading 'mv' in the features name. The device can communicate with every third party compliant **USB3 Vision** (p. 173) capture driver that can interpret **GenICam** (p. 165) XML-files.

Note

Given that the mvBlueFOX USB 2.0 camera family is not a **GenICam** (p. 165) based device, it is not compliant to the mvBlueFOX3. I.e. software written for the mvBlueFOX USB 2.0 camera family has to be adapted concerning the initialization of the camera (minor changes necessary) and the changing of settings. However, the driver a.k.a **mvIMPACT_Acquire** (p. 171) interface remains the same.

The following figure shows the software concept of MATRIX VISION's camera devices:

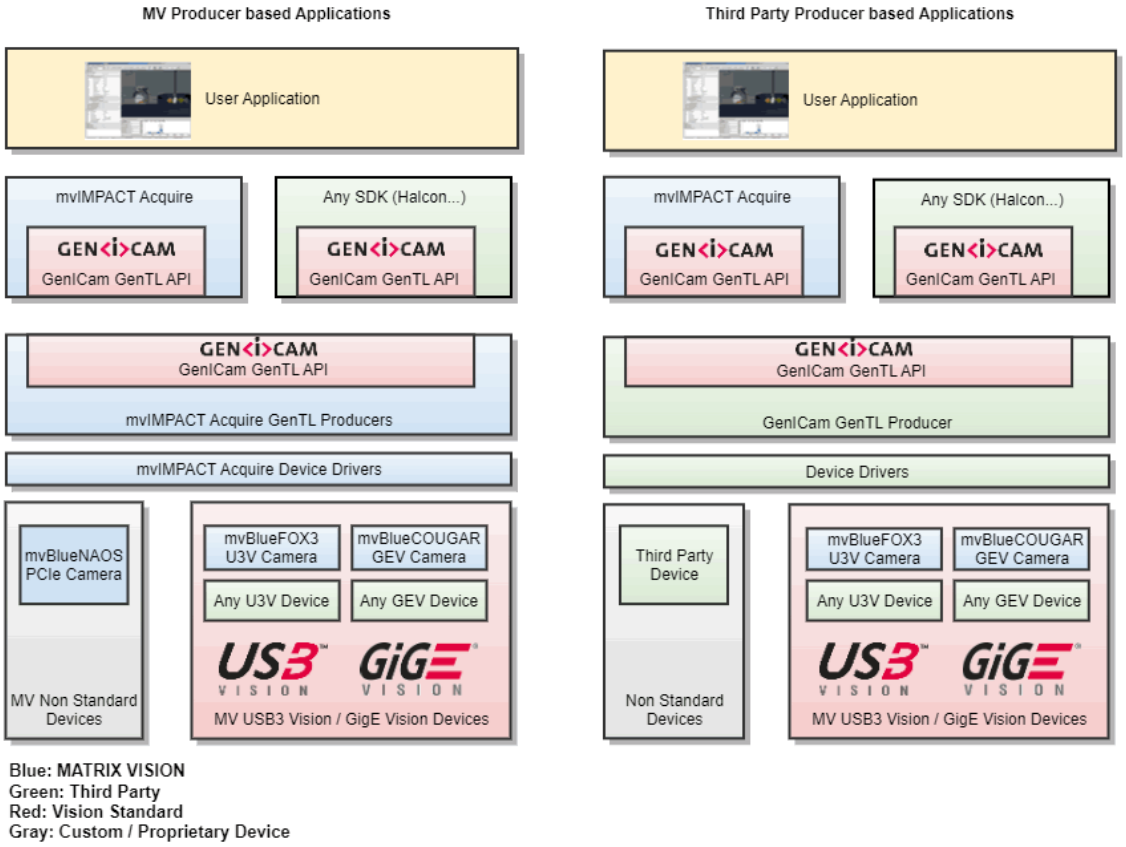


Figure 2: Software concept

As shown in figure 2, for the mvBlueFOX3 the **mvIMPACT_Acquire** (p. 171) interface is stacked on the **USB3 Vision** (p. 173) and **Genicam** (p. 165) layers. The **mvIMPACT_Acquire** (p. 171) interface internally uses the **GenICam** (p. 165) runtime libraries, so that it can be considered as a user application written with the **GenICam** (p. 165) interface.

1.7.2 Order code nomenclatures

1.7.2.1 mvBlueFOX3-1

The mvBlueFOX3 nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
1.2 Mpix, 1280 x 960, 1/3", MT9M031, CMOS	BVS CA-SF1-0012BG/C	mvBlueFOX3-1012bG/C
1.2 Mpix, 1280 x 960, 1/3", MT9M034, CMOS	BVS CA-SF1-0012DG/C	mvBlueFOX3-1012dG/C
1.3 Mpix, 1280 x 1024, 1/1.8", EV76C560, CMOS	BVS CA-SF1-0013ZG/C	mvBlueFOX3-1013G/C
2.0 Mpix, 1600 x 1200, 1/1.8", EV76C570, CMOS	BVS CA-SF1-0020ZG/C	mvBlueFOX3-1020G/C
2.0 Mpix, 1600 x 1200, 1/1.8", EV76C570 _AxT6, CMOS	BVS CA-SF1-0020AG/C	mvBlueFOX3-1020aG/C
3.2 Mpix, 2064 x 1544, 1/1.8", AR0331, CMOS	BVS CA-SF1-0031ZC	mvBlueFOX3-1031C
10.7 Mpix, 3856 x 2764, 1/2.35", MT9J003, CMOS	BVS CA-SF1-0107ZG/C	mvBlueFOX3-1100G/C

	Series	Sensor	Color		HW Variant		CUP/STD
Balluff model name	BVS CA-SF1-	xxxxx	G/C	-	see legend below	-	see legend below
Model name	mvBlueFOX3-1	xxxx	G/C	-	see legend below	-	see legend below

Legend	Balluff model name	Model name
HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling (2): Lensholder **0**: None **1**: CS-Mount, (no adjustment) **2**: C-Mount (CS-mount, same as 1 + 5 mm spacer) (3): Filter **0**: None **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (only without backfocus adjustment) (4): Housing **1**: Color blue (standard) **2**: Color black, no logo, no label MATRIX VISION (5): I/O **1**: None (standard) **2**: With I/Os (standard) (6): Software Adjustments **0**: None	(1)(2)(3)(4) (1): Lensholder **0**: None **1**: CS-Mount, (no adjustment) **2**: C-Mount (CS-mount, same as 1 + 5 mm spacer) (2): Filter **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (only without backfocus adjustment) **9**: None (3): Housing **1**: Color blue (standard) **2**: Color black, no logo, no label MATRIX VISION (4): I/O **1**: None (standard) **2**: With I/Os (standard)
CUP/STD	**001**	

1.7.2.2 mvBlueFOX3-M1

The mvBlueFOX3-M1 nomenclature scheme is as follows:

```
mvBlueFOX3-M1(A)(B)(C) - (1)(2)(3)(4)

  - (A): Sensor model
      012b: 1.2 Mpix, 1280 x 960, 1/3"
      012d: 1.2 Mpix, 1280 x 960, 1/3"
      013: 1.3 Mpix, 1280 x 1024, 1/1.8"
      020: 2.0 Mpix, 1600 x 1200, 1/1.8"
      020a: 2.0 Mpix, 1600 x 1200, 1/1.8"
      031: 3.1 Mpix, 2052 x 1536, 1/3"
      100: 10 Mpix, 3856 x 2764, 1/2.35"

  - (B): Sensor color
      G: Gray scale
      C: Color

  - (C): Infrared enhanced (for -013G)
      E: Infrared enhanced

  - (1): Lensholder
      1: None
      2: S-Mount 13mm

  - (2): Filter
      1: none

  - (3): Case
      1: standard

  - (4): I/O
      1: standard; if I/O is needed, use separate article: mvBlueFOX3-IO
```

1.7.2.3 mvBlueFOX3-2

The mvBlueFOX3-2 nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
0.4 Mpix, 728 x 544, 1/2.9", IMX287, CMOS	BVS CA-SF2-0004FG/C	mvBlueFOX3-2004G/C
1.6 Mpix, 1456 x 1088, 1/2.9", IMX273, CMOS	BVS CA-SF2-0016ZG/C	mvBlueFOX3-2016G/C
1.7 Mpix, 1600 x 1104, 1/1", IMX425, CMOS	BVS CA-SF2-0017ZG/C	mvBlueFOX3-2017G/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX174, CMOS	BVS CA-SF2-0024ZG/C	mvBlueFOX3-2024G/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX249, CMOS	BVS CA-SF2-0024AG/C	mvBlueFOX3-2024aG/C
3.2 Mpix, 2064 x 1544, 1/1.8", IMX252, CMOS	BVS CA-SF2-0032ZG/C	mvBlueFOX3-2032G/C
3.2 Mpix, 2064 x 1544, 1/1.8", IMX265, CMOS	BVS CA-SF2-0032AG/C	mvBlueFOX3-2032aG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX250, CMOS	BVS CA-SF2-0051ZG/C	mvBlueFOX3-2051G/C
5.1 Mpix, 2464 x 2056, 2/3", IMX264, CMOS	BVS CA-SF2-0051AG/C	mvBlueFOX3-2051aG/C
5.1 Mpix, 2472 x 2064, 1/1.8", IMX547, CMOS	BVS CA-SF2-0051DG/C	mvBlueFOX3-2051dG/C
6.4 Mpix, 3096 x 2080, 1/1.8", IMX178, CMOS	BVS CA-SF2-0064ZG/C	mvBlueFOX3-2064G/C
7.1 Mpix, 3216 x 2208, 1/1", IMX420, CMOS	BVS CA-SF2-0071ZG/C	mvBlueFOX3-2071G/C
7.1 Mpix, 3216 x 2208, 1/1", IMX428, CMOS	BVS CA-SF2-0071AG/C	mvBlueFOX3-2071aG/C
8.1 Mpix, 2856 x 2848, 2/3", IMX546, CMOS	BVS CA-SF2-0081AG/C	mvBlueFOX3-2081aG/C
8.9 Mpix, 4112 x 2176, 1", IMX255, CMOS	BVS CA-SF2-0089ZG/C	mvBlueFOX3-2089G/C
8.9 Mpix, 4112 x 2176, 1", IMX267, CMOS	BVS CA-SF2-0089AG/C	mvBlueFOX3-2089aG/C
12.4 Mpix, 4112 x 3008, 1.1", IMX253, CMOS	BVS CA-SF2-0124ZG/C	mvBlueFOX3-2124G/C
12.4 Mpix, 4112 x 3008, 1.1", IMX304, CMOS	BVS CA-SF2-0124AG/C	mvBlueFOX3-2124aG/C
12.4 Mpix, 4128 x 3008, 1/1.1", IMX545, CMOS	BVS CA-SF2-0124DG/C	mvBlueFOX3-2124dG/C
12.4 Mpix, 4064 x 3044, 1/1.7", IMX226, CMOS	BVS CA-SF2-0124RG/C	mvBlueFOX3-2124rG/C
16.2 Mpix, 5328 x 3040, 1.1", IMX542, CMOS	BVS CA-SF2-0162AG/C	mvBlueFOX3-2162G/C
20.4 Mpix, 4512 x 4512, 1.1", IMX541, CMOS	BVS CA-SF2-0204AG/C	mvBlueFOX3-2204G/C
20.5 Mpix, 5544 x 3692, 1", IMX183, CMOS	BVS CA-SF2-0205ZG/C	mvBlueFOX3-2205G/C
24.6 Mpix, 5328 x 4608, 1.2", IMX540, CMOS	BVS CA-SF2-0246A/C	mvBlueFOX3-2246G/C

	Series	Sensor	Color		HW Variant		CUP/STD
Balluff model name	BVS CA-SF2-	xxxxx	G/C	-	see legend below	-	see legend below
Model name	mvBlueFOX3-2	xxxx	G/C	-	see legend below	-	see legend below

Legend	Balluff model name	Model name

HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling (2): Lensholder **1**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **2**: C-Mount, Type1, with adjustable backfocus, round filter with screw ring, Ø23.2 mm **4**: CS-mount Type1, with adjustable backfocus, round filter glued, Ø20 mm **A:** C-Mount, Type 2, with high durability and factory-set backfocus, fixed square filter, adjustable center position **B:** C-Mount Type 2 + D23.2mm Filter with screwring (3): Filter (only valid with lensholder) **0**: None **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (4): Housing **1**: Color blue (standard) **2**: Color black, no logo (5): I/O **1**: None (standard) **2**: With I/O (6): Software Adjustments **0**: None	(1)(2)(3)(4) (1): Lensholder **1**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **2**: C-Mount, Type1, with adjustable backfocus, round filter with screw ring, Ø23.2 mm **4**: CS-mount Type1, with adjustable backfocus, round filter glued, Ø20 mm **A:** C-Mount, Type 2, with high durability and factory-set backfocus, fixed square filter, adjustable center position **B:** C-Mount Type 2 + D23.2mm Filter with screwring (2): Filter (only valid with lensholder) **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut **9**: None (3): Housing **1**: Color blue (standard) **2**: Color black, no logo (4): I/O **1**: None (standard) **2**: With I/O
CUP/STD	**001**	

1.7.2.4 mvBlueFOX3-M2

The mvBlueFOX3-M2 nomenclature scheme is as follows:

Note

Model name is the same as mvBlueFOX3-2 with the additional order option 0 for "without housing".
E.g. BVS CA-SF2-0004FG
BVS CA-SF2-0004FG-xxx0xx (x is any value which can be combined with 0)

Sensor	Balluff model name	Model name
0.4 Mpix, 728 x 544, 1/2.9", IMX287, CMOS	BVS CA-SF2-0004FG/C-xxx0xx	mvBlueFOX3-M2004G/C
1.6 Mpix, 1456 x 1088, 1/2.9", IMX273, CMOS	BVS CA-SF2-0016ZG/C-xxx0xx	mvBlueFOX3-M2016G/C
1.7 Mpix, 1600 x 1104, 1/1", IMX425, CMOS	BVS CA-SF2-0017ZG/C-xxx0xx	mvBlueFOX3-M2017G/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX174, CMOS	BVS CA-SF2-0024ZG/C-xxx0xx	mvBlueFOX3-M2024G/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX249, CMOS	BVS CA-SF2-0024AG/C-xxx0xx	mvBlueFOX3-M2024aG/C
3.2 Mpix, 2048 x 1544, 1/1.8", IMX252, CMOS	BVS CA-SF2-0032ZG/C-xxx0xx	mvBlueFOX3-M2032G/C
3.2 Mpix, 2064 x 1544, 1/1.8", IMX265, CMOS	BVS CA-SF2-0032AG/C-xxx0xx	mvBlueFOX3-M2032aG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX250, CMOS	BVS CA-SF2-0051ZG/C-xxx0xx	mvBlueFOX3-M2051G/C
5.1 Mpix, 2464 x 2056, 2/3", IMX264, CMOS	BVS CA-SF2-0051AG/C-xxx0xx	mvBlueFOX3-M2051aG/C
5.1 Mpix, 2472 x 2064, 1/1.8", IMX547, CMOS	BVS CA-SF2-0051DG/C-xxx0xx	mvBlueFOX3-M2051dG/C
6.4 Mpix, 3096 x 2080, 1/1.8", IMX178, CMOS	BVS CA-SF2-0064ZG/C-xxx0xx	mvBlueFOX3-M2064G/C
7.1 Mpix, 3216 x 2208, 1/1", IMX420, CMOS	BVS CA-SF2-0071ZG/C-xxx0xx	mvBlueFOX3-M2071G/C
7.1 Mpix, 3216 x 2208, 1/1", IMX428, CMOS	BVS CA-SF2-0071AG/C-xxx0xx	mvBlueFOX3-M2071aG/C
8.1 Mpix, 2856 x 2848, 2/3", IMX546, CMOS	BVS CA-SF2-0081AG/C-xxx0xx	mvBlueFOX3-M2081aG/C
8.9 Mpix, 4112 x 2176, 1", IMX255, CMOS	BVS CA-SF2-0089ZG/C-xxx0xx	mvBlueFOX3-M2089G/C
8.9 Mpix, 4112 x 2176, 1", IMX267, CMOS	BVS CA-SF2-0089AG/C-xxx0xx	mvBlueFOX3-M2089aG/C
12.4 Mpix, 4112 x 3008, 1.1", IMX253, CMOS	BVS CA-SF2-0124ZG/C-xxx0xx	mvBlueFOX3-M2124G/C
12.4 Mpix, 4112 x 3008, 1.1", IMX304, CMOS	BVS CA-SF2-0124AG/C-xxx0xx	mvBlueFOX3-M2124aG/C
12.4 Mpix, 4128 x 3008, 1/1.1", IMX545, CMOS	BVS CA-SF2-0124DG/C-xxx0xx	mvBlueFOX3-M2124dG/C
16.2 Mpix, 5328 x 3040, 1.1", IMX542, CMOS	BVS CA-SF2-0162AG/C-xxx0xx	mvBlueFOX3-M2162G/C
20.4 Mpix, 4512 x 4512, 1.1", IMX541, CMOS	BVS CA-SF2-0204AG/C-xxx0xx	mvBlueFOX3-M2204G/C
24.6 Mpix, 5328 x 4608, 1.2", IMX540, CMOS	BVS CA-SF2-0246A/C-xxx0xx	mvBlueFOX3-M2246G/C

	Series	Sensor	Color		HW Variant		CUP/STD
Balluff model name	BVS CA-SF2-	xxxxx	G/C	-	see legend below	-	see legend below
Model name	mvBlueFOX3-M2	xxxx	G/C	-	see legend below	-	see legend below

Legend	Balluff model name	Model name

HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling (2): Lensholder **0**: None **1**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **2**: C-Mount, Type1, with adjustable backfocus, round filter with screw ring, Ø23.2 mm **A:** C-Mount, Type 2, with high durability and factory-set backfocus, fixed square filter, adjustable center position **B:** C-Mount Type 2 + D23.2mm Filter with screwring **Y:** S-Mount, 13 mm length for sensors with image circule <= 2/3 inch (3): Filter (only valid with lensholder) **0**: None **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (4): Housing **0**: Without housing (5): I/O **1**: None (standard) **2**: With I/O (6): Software Adjustments **0**: None	(1)(2)(3)(4) (1): Lensholder **1**: None **6**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **2**: C-Mount, Type1, with adjustable backfocus, round filter with screw ring, Ø23.2 mm **A:** C-Mount, Type 2, with high durability and factory-set backfocus, fixed square filter, adjustable center position **B:** C-Mount Type 2 + D23.2mm Filter with screwring **4**: S-Mount, 13 mm length for sensors with image circule <= 2/3 inch (2): Filter (only valid with lensholder) **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut **9**: No filter (3): Housing **1**: Standard **5**: Mounting and thermal conduction plate for board level camera (4): I/O **1**: None (standard) **2**: With I/O
CUP/STD	**001**	

1.7.2.5 mvBlueFOX3-3M

The mvBlueFOX3-3M nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
6.4 Mpix, 3096 x 2080, 1/1.8", IMX178, CMOS	BVS CA-SF3-0064ZG/C	BF3-3M-0064ZG/C

	Series	Sensor	Color		HW Variant		CUP/STD
Balluff model name	BVS CA-SF3-	xxxxx	G/C	-	see legend below	-	see legend below
Model name	BF3-3M	xxxxx	G/C	-	see legend below	-	see legend below

Legend	Balluff model name / Model name
HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling (2): Lensholder **0**: No lensholder **Y:** S-Mount, 13 mm length for sensors with image circule <= 2/3 inch (3): Filter (only valid with lensholder) **0**: No filter **1**: IR cut (standard) **2**: Glass **3**: Daylight cut filter **6**: Dualband filter DB850 (4): Housing **0**: None (standard) (5): I/O **Note:** only available via BF Embedded Interface boards which are available as accessories (6): Software Adjustments **0**: None
CUP/STD	**001**

1.7.2.6 mvBlueFOX3-4

The mvBlueFOX3-4 nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
16.9 Mpix, 5472 x 3080, 4/3", IMX387, CMOS	BVS CA-SF4-0169ZG/C	BF3-4-0169ZG/C
19.6 Mpix, 4432 x 4432, 4/3", IMX367, CMOS	BVS CA-SF4-0196ZG/C	BF3-4-0196ZG/C
31.5 Mpix, 6480 x 4856, APS-C", IMX342, CMOS	BVS CA-SF4-0315ZG/C	BF3-4-0315ZG/C

	Series	Sensor	Color		HW Variant		CUP/STD
Balluff model name	BVS CA-SF4-	xxxxx	G/C	-	see legend below	-	see legend below
Model name	BF3-4	xxxxx	G/C	-	see legend below	-	see legend below

Legend	Balluff model name / Model name
HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling (2): Lensholder **N:** M42x1 mount, 12mm factory-set backfocus, fixed square filter **P:** M42x1 mount, 45.5mm factory-set backfocus, fixed square filter **Q:** F-Mount, 46mm factory-set backfocus, fixed square filter **T:** Lensholder with TFL-Mount or Adapter for M42 to TFL-Mount (3): Filter (only valid with lensholder) **0**: No filter **1**: IR cut (standard) **2**: Glass (4): Housing **1**: Blue housing (standard) (5): I/O **0**: None (standard) **2**: Standard I/Os (6): Software Adjustments **0**: None
CUP/STD	**001**

1.7.2.7 mvBlueFOX3-5M

The mvBlueFOX3-5M nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
0.4 Mpix, 728 x 544, 1/2.9", IMX287, CMOS	BVS CA-SF5-0004FG/C	mvBlueFOX3-5M-0004FG/C
1.6 Mpix, 1456 x 1088, 1/2.9", IMX273, CMOS	BVS CA-SF5-0016ZG/C	mvBlueFOX3-5M-0016ZG/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX174, CMOS	BVS CA-SF5-0024ZG/C	mvBlueFOX3-5M-0024ZG/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX249, CMOS	BVS CA-SF5-0024AG/C	mvBlueFOX3-5M-0024AG/C
2.4 Mpix, 1936 x 1216, 1/2.3", IMX392, CMOS	BVS CA-SF5-0024BG/C	mvBlueFOX3-5M-0024BG/C
3.2 Mpix, 2048 x 1544, 1/1.8", IMX252, CMOS	BVS CA-SF5-0032ZG/C	mvBlueFOX3-5M-0032ZG/C
3.2 Mpix, 2064 x 1544, 1/1.8", IMX265, CMOS	BVS CA-SF5-0032AG/C	mvBlueFOX3-5M-0032AG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX250, CMOS	BVS CA-SF5-0051ZG/C	mvBlueFOX3-5M-0051ZG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX250_POL, CMOS	BVS CA-SF5-0051PG/C	mvBlueFOX3-5M-0051PG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX264, CMOS	BVS CA-SF5-0051AG/C	mvBlueFOX3-5M-0051AG/C
5.1 Mpix, 2472 x 2064, 1/1.8", IMX547, CMOS	BVS CA-SF5-0051DG/C	mvBlueFOX3-5M-0051dG/C
6.4 Mpix, 3096 x 2080, 1/1.8", IMX178, CMOS	BVS CA-SF5-0064ZG/C	mvBlueFOX3-5M-0064ZG/C
8.1 Mpix, 2856 x 2848, 2/3", IMX546, CMOS	BVS CA-SF5-0081AG/C	mvBlueFOX3-5M-0081aG/C
8.9 Mpix, 4112 x 2176, 1", IMX255, CMOS	BVS CA-SF5-0089ZG/C	mvBlueFOX3-5M-0089ZG/C
8.9 Mpix, 4112 x 2176, 1", IMX267, CMOS	BVS CA-SF5-0089AG/C	mvBlueFOX3-5M-0089AG/C
12.4 Mpix, 4112 x 3008, 1.1", IMX253, CMOS	BVS CA-SF5-0124ZG/C	mvBlueFOX3-5M-0124ZG/C
12.4 Mpix, 4064 x 3044, 1/1.7", IMX226, CMOS	BVS CA-SF5-0124RG/C	mvBlueFOX3-5M-0124RG/C
12.4 Mpix, 4112 x 3008, 1.1", IMX304, CMOS	BVS CA-SF5-0124AG/C	mvBlueFOX3-5M-0124AG/C
12.4 Mpix, 4128 x 3008, 1/1.1", IMX545, CMOS	BVS CA-SF5-0124DG/C	mvBlueFOX3-5M-0124dG/C
16.2 Mpix, 5328 x 3040, 1.1", IMX542, CMOS	BVS CA-SF5-0162AG/C	mvBlueFOX3-5M-0162AG/C
16.9 Mpix, 5472 x 3080, 4/3", IMX387, CMOS	BVS CA-SF5-0169ZG/C	mvBlueFOX3-5M-0169ZG/C
19.6 Mpix, 4432 x 4432, 4/3", IMX367, CMOS	BVS CA-SF5-0196ZG/C	mvBlueFOX3-5M-0196ZG/C
20.4 Mpix, 4512 x 4512, 1.1", IMX541, CMOS	BVS CA-SF5-0204AG/C	mvBlueFOX3-5M-0204AG/C
20.5 Mpix, 5544 x 3692, 1", IMX183, CMOS	BVS CA-SF5-0205ZG/C	mvBlueFOX3-5M-0205ZG/C
24.6 Mpix, 5328 x 4608, 1.2", IMX540, CMOS	BVS CA-SF5-0246AG/C	mvBlueFOX3-5M-0246AG/C
31.5 Mpix, 6480 x 4856, APS-C", IMX342, CMOS	BVS CA-SF5-0315Z/C	mvBlueFOX3-5M-0315ZG/C

	Series	Sensor	Color		HW Variant		CUP/STD
Balluff model name	BVS CA-SF5-	xxxxx	G/C	-	see legend below	-	see legend below
Model name	BF3-5M	xxxxx	G/C	-	see legend below	-	see legend below

Legend	Balluff model name / Model name

HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling **2**: Extended Temperature Range **3**: Extended cleanliness **8**: Customized handling (2): Lensholder **0**: None **1**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **A:** C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter, adjustable center position **B:** C-Mount Type 2 + D23.2mm Filter with screwring **N:** M42x1 mount, 12mm factory-set backfocus, fixed square filter **Y:** S-Mount, 13 mm length for sensors with image circule <= 2/3 inch (3): Filter (only valid with lensholder) **0**: No filter **1**: IR cut **2**: Glass **3**: Daylight cut filter **5**: Bandpass filter BP680 **6**: Dualband filter DB850 **8**: Customized filter type (4): Housing **0**: No housing **5**: Mounting and thermal transfer plate for board level camera **8**: Customized housing (5): I/O **Note:** only available via BF Embedded Interface boards which are available as accessories (6): Software Adjustments **0**: None
CUP/STD	**001**

1.7.2.8 Ordering code samples

mvBlueFOX3-1013G[1]	CMOS, 1.3 Mpix, 1280 x 1024, 1/1.8", gray scale.
mvBlueFOX3-2024C-1212	CMOS, 2.4 Mpix, 1936 x 1214, 1/1.2", color, CS-mount without back focus adjustment, glass filter, etc.
mvBlueFOX3-1100G-2312	CMOS, 10 Mpix, 3856 x 2764, 1/2.35", gray scale, C-mount without back focus adjustment, daylight cut filter, with I/O

[1]: `-1111` is the standard delivery variant and for this reason it is not mentioned.

1.7.3 What's inside and accessories

The mvBlueFOX3 is shipped without any accessories:



Figure 3: mvBlueFOX3 - scope of supply

For this reason, you will need at least

- a lens (by default, the mvBlueFOX3 is shipped without lens) and
- a USB 3 cable

to use the mvBlueFOX3.

Note

Although maximum cable length is not specified in the USB 3 standard, the electrical properties of the cable and signal quality limitations may define the practical maximum length.
There are different USB 3 cable qualities available on the market. If you want to use your own USB 3 cables, you have to ensure that the data quality and shielding of the cable is sufficient.
As a rule of thumb thin cables will only work for short distances up to max. 3m.
Better cable qualities which go alongside with thicker cable diameter will allow longer distances such as 5m or even 8m.
We recommend to use the cables we supply to be on the safe side.

1.7.3.1 Accessories for the mvBlueFOX3

Part code	Description
Cables	
KS-MICUSB3B-A LS	USB 3 cable lockable, length up to 5 m
KS-BCX-HR12	I/O cable, length up to 20 m
KS-USB3A-A EXT OPT 10.0 REV2	USB 3.0 active optical cable, A plug to A plug, Shield not connected to connector housing, additional device for power needed (Hub USB3 HUB U3H01AR); length 10 meters
KS-USB3A-A EXT OPT 30.0 REV2	USB 3.0 active optical cable, A plug to A plug, Shield not connected to connector housing, additional device for power needed (Hub USB3 HUB U3H01AR); length 30 meters
KS-TUSB3C-A LS 03.0 000	USB3 connecting cable Type C USB3 to USB3-A, Ø6.2mm, with lock screws on both sides, for mvBlueFOX3-3M/-5M, length 3m
KS-TUSB3C-A LS 05.0 000	USB3 connecting cable Type C USB3 to USB3-A, Ø6.2mm, with lock screws on both sides, for mvBlueFOX3-3M/-5M, length 5m
KS-MICUSB3B-A LS AC12 05.0	USB3 connecting cable micro USB-B angled to USB-A, with lock screws, length 5m
KS-MICUSB3B-A LS AC14 05.0	USB3 connecting cable micro USB-B angled to USB-A, with lock screws, length 5m
KS-MICUSB3B-A LS 08.0	USB3 connecting cable micro USB-B to USB-A, with lock screws on both sides, for mvBlueFOX3, length 8m
KS-MICUSB3B-A LS 03.0 R2	USB3 connecting cable micro USB-B to USB-A, Ø5.8mm, with lock screws on both sides, for mvBlueFOX3, length 3m
KS-MICUSB3B-A LS 05.0 R2	USB3 connecting cable micro USB-B to USB-A, Ø5.8mm, with lock screws on both sides, for mvBlueFOX3, length 5m
BFE-FLEX-100	Flex cable extension for **BF3embedded interface** (p. 72), length 100mm, weight 1g, for use with mvBlueFOX3-3M and mvBlueFOX3-5M, lock screws included
BFE-FLEX-300	Flex cable extension for **BF3embedded interface** (p. 72), length 300mm, for use with mvBlueFOX3-3M and mvBlueFOX3-5M, lock screws included
KS-12pPicoBlade-OPEN W0.30	Cable set periphery to mvBlueFOX3-3M/-5M; design board-to-wire; MOLEX plug to 12 x cable ends
KS-03pPicoBlade-OPEN W0.30	Cable set periphery to mvBlueFOX3-3M/-5M; design board-to-wire; MOLEX plug to 3 x cable ends
Optics	
MV-ZWISCHENRING 5MM CMOUNT	Spacer for C-Mount lenses
Mounting	
MV-Tripod Adapter BF3	1/4" tripod adapter including three suitable screws
TRIPOD ADAPT BF3-2	Tripod adapter for mvBlueFOX3-2
Boards	
mvBlueFOX3-IO	**I/O board for mvBlueFOX3-M (mvBlueFOX3-IO)** (p. 62)
mvBlueFOX3-IO NC	**I/O board for mvBlueFOX3-M (mvBlueFOX3-IO)** (p. 62) without Hirose connector
BFE-IF-MICUSB3B-H-IO12	**BF3embedded interface** (p. 72) board, USB3 Micro B connector, horizontal, 12pin connector for I/O, weight 2g, for use with mvBlueFOX3-3M and mvBlueFOX3-5M
BFE-IF-USB3C-H-IO12	**BF3embedded interface** (p. 72) board, USB3 typ C connector, horizontal, 12pin connector for I/O, weight 2g, for use with mvBlueFOX3-3M and mvBlueFOX3-5M
Peripherals	
USB3 HUB U3H01AR	4-Port USB 3.0 HUB Upstream Port USB3 A; Power connector Mini-Combicon; Power Input Range: DC 5 - 12V (max. 5.0A)

BF3-3M-AP-00	Thermal transfer plate for use with mvBlueFOX3-3M and mvBlueFOX3-5M, weight 5g

1.8 Quickstart

- **System Requirements** (p. 36)
- **Installing The mvGenTL-Acquire Package** (p. 37)
- **Connecting The Camera** (p. 44)
- **Driver concept** (p. 47)
- **Relationship Between Driver, Firmware And SDK** (p. 50)
- **Optimizing USB Performance** (p. 53)
- **Using USB3 Vision™ Devices In A Docker Container** (p. 350)

1.8.1 System Requirements

1.8.1.1 Host System

The mvBlueFOX3 is a high-performance camera which requires a high data throughput on the host system, for example, when processing large amount of image data and high CPU resources if processing color images on the host PC.

For this reason we recommend the following components:

Component	Recommendation
Processor	Preferably multi core Intel or ARM CPUs
RAM	4 GB in 32-bit OS; 8 GB in 64-bit OS
Mainboard	Latest PC architecture and chipset

Several USB3 extension cards or adapters for PCs and Notebooks have been successfully tested by MATRIX VISION. Please ask our support team.

See also

`https://servicedesk.matrix-vision.com`

There is a huge variety of ARM based devices available in the market. Some suitable platforms have been tested by MATRIX VISION and a summery of the results of this test can be found here: **Appendix C. Tested ARM platforms** (p. 489)

Please ask your system vendor for further advice and consult our technical documentation.

For more information about USB3 related settings on Linux, please have a look at the troubleshooting chapter **Optimizing USB Performance** (p. 53) .

1.8.1.2 Supported Operating Systems

The following operating system are supported officially:

1.8.1.2.1 Windows

- Microsoft Windows 7 (32-bit, 64-bit)
- Microsoft Windows 8.1 (32-bit, 64-bit)
- Microsoft Windows 10 (32-bit, 64-bit)
- Microsoft Windows 11

Other Windows versions might work as well but will not be tested on a regular basis.

Note

Since mvIMPACT Acquire version 2.8.0 it could be possible that you have to update your Windows installer at least using Windows XP. The necessary packages are available from Microsoft's website: `http://www.microsoft.com/en-US/download/details.aspx?id=8483`

1.8.1.2.2 Linux Please check the the 'Support' section of the MATRIX VISION website for the availability of the latest Linux driver package.

See also

`https://matrix-vision.com/software-drivers-en.html`

Currently supported Kernel versions are:

- Kernel 3.5.x or greater

Note

Linux kernels prior to 3.5.x do not inherently support USB3!

In case the target system runs an older Linux kernel, **it is absolutely necessary to update the kernel to at least version 3.5.0** . Please refer to the documentation of your Linux distribution for information on how to update your system's Linux kernel.

All Kernels starting from version 3.5.0 should work without problems.

1.8.2 Installing The mvGenTL-Acquire Package

All necessary drivers are available from the MATRIX VISION website at `https://www.matrix-vision.de`, section "`Products -> Cameras -> your interface -> your product -> Downloads`".

1.8.2.1 Windows

Starting the installer application

- **mvGenTL_Acquire-x86-n.n.n.msi** (for 32-bit systems) or
- **mvGenTL_Acquire-x86_64-n.n.n.msi** (for 64-bit systems): will display the following dialog:

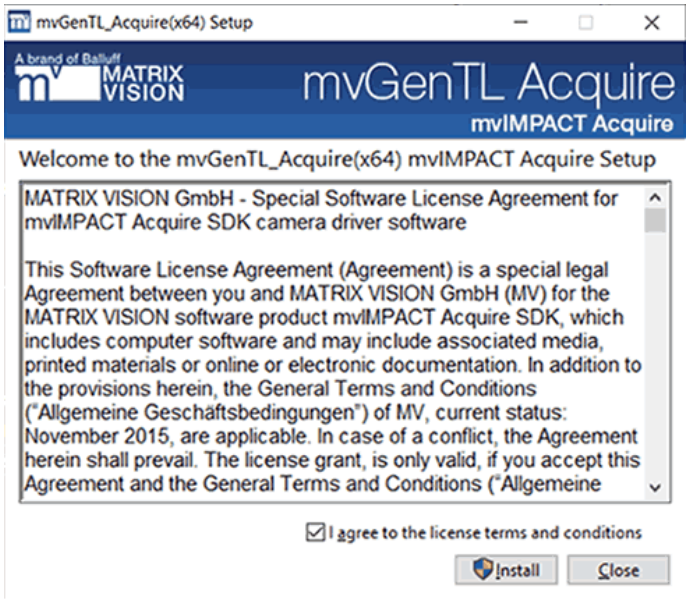


Figure 1:Driver installation - Start Window

- Now, follow the instructions of installation program and adjust the settings to your needs:

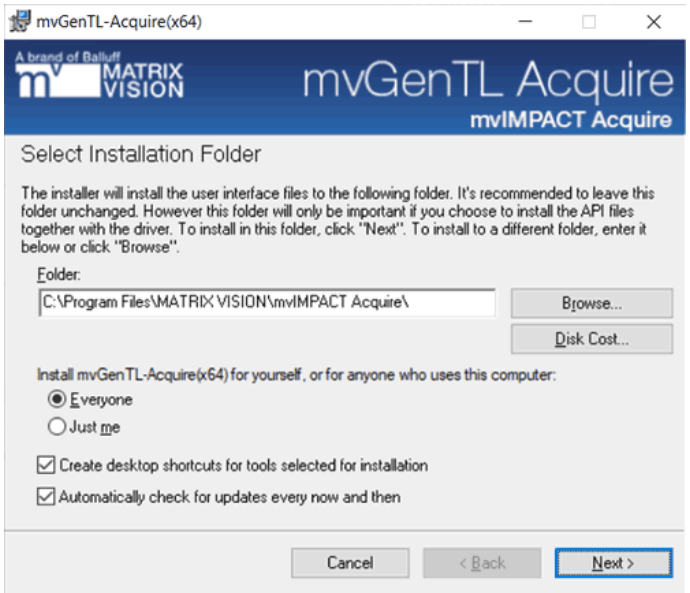


Figure 2:Driver installation - Select Installation Folder

Since

mvIMPACT Acquire 2.25.0

wxPropView (p. 99) is able to check if new driver version became available on a weekly basis. Deactivate the check box if wxPropView should not check for updates. You can activate this again in **wxPropView** (p. 99) via the help menu.

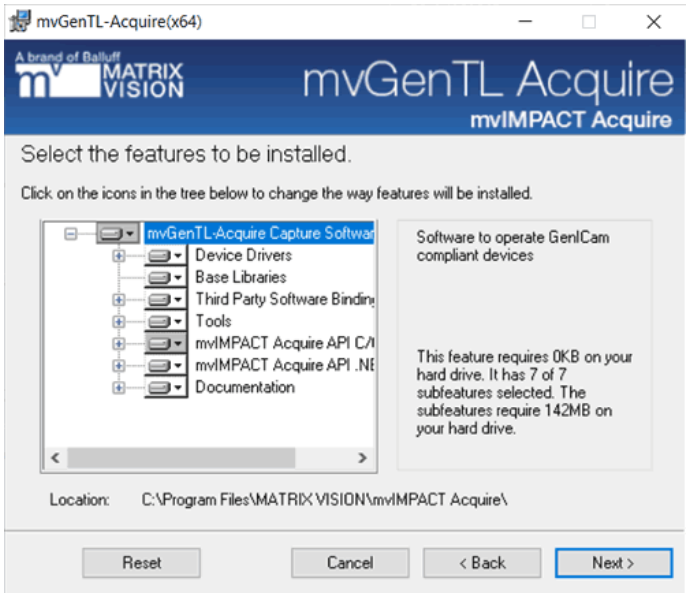


Figure 3:Driver installation - Select Features

- After confirmation, the installation will start and copy files and install device drivers.

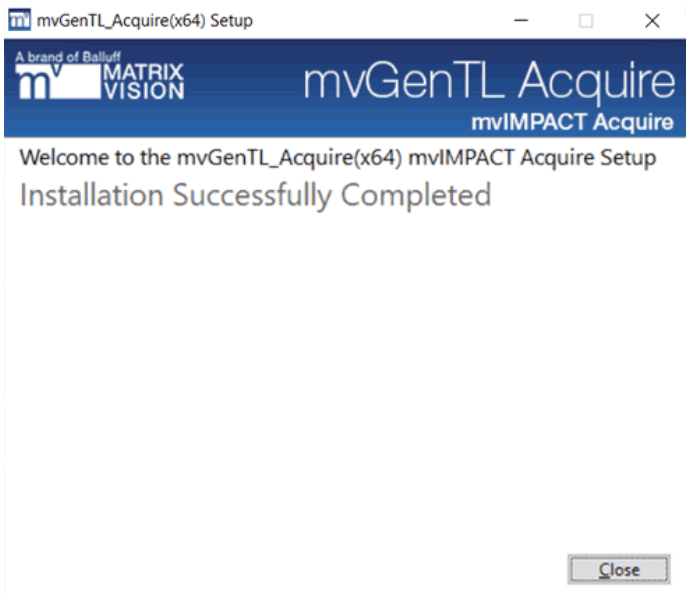


Figure 4:Driver installation - Installation Completed

You will find all tools like

- **wxPropView** (p. 99) and
- **mvDeviceConfigure** (p. 99)

either as shortcuts on the desktop or in the Windows start menu under **"MATRIX VISION ->** **mvIMPACT Acquire"**.

Note

You can ignore the other tools mvIPConfigure and mvGigEConfigure, because they are only necessary in combination with GigE Vision™ devices like the mvBlueCOUGAR-X.

Afterwards, you can use **mvDeviceConfigure** (p. 99) to update the firmware if needed. The latest firmware image is available on the web - please check for updates. The current firmware version can be read out using **wxPropView** (p. 99).

1.8.2.2 Linux

The following (additional) packages will be needed to use all features of mvIMPACT Acquire:

- libwxbase3.0-0v5
- libwxbase3.0-dev
- libwxgtk3.0-gtk3-0v5
- libwxgtk3.0-gtk3-dev
- libwxgtk-webview3.0-gtk3-0v5
- libwxgtk-webview3.0-gtk3-dev
- wx3.0-headers
- build-essential (meta package)
- libgtk2.0-dev
- gcc 4.8.5 (4.9.4 for ARM versions) environment or newer

Note

The names of the packages as mentioned above are the Ubuntu specific ones. Other distributions (e.g. Debian, Arch, Redhat, ...) might use different names.

The installation script will ask if the packages should be downloaded during the installation process. If some of the packages are not installed some features might not be available. If the e.g. wxWidgets related packages are missing on the target system then all GUI application coming as part of the mvIMPACT Acquire installation won't be available.

Note

If you are going to install the mvGenTL-Acquire package on an ARM device, please read **this** (p. 53) section first.

To use a (camera) device in Linux (capture images from it and change its settings), a driver is needed, consisting of several libraries and several configuration files. These files are required during run time.

To develop applications that can use the device an API is needed, containing header files, makefiles, samples, and a few libraries. These files are required at compile time.

Both file collections are distributed in a single package which is available in the `Support` section of the MATRIX VISION website. In addition to that an installation script is provided which can be downloaded from the same location. Using this script makes installing the driver package a lot easier.

Note

The following table shows the supported platforms and the corresponding package and installation script name:

Architecture	Package	Installation Script
ARM64	mvGenTL_Acquire-ARM64_gnu-n.n.n.tgz	install_mvGenTL_Acquire_ARM.sh
ARMhf	mvGenTL_Acquire-ARMhf_gnueabi-n.n.n.tgz	install_mvGenTL_Acquire_ARM.sh
x86_64	mvGenTL_Acquire-x86_64_ABI2-n.n.n.tgz	install_mvGenTL_Acquire.sh

The following example explains the installation process for the x86_64 package. **The installation process for other packages will work almost identical except different names as mentioned in the previous table.**

- Please start a console and change into the directory where the installation script and the installation package are located e.g. `/home/username/Downloads` :

```
cd /home/username/Downloads
```

> **Note**
>
> If root permissions are needed, the script will ask for the permissions. **There is no need to call it with root permissions.**

- You might need to enable the execute flag with:

```
chmod a+x install_mvGenTL_Acquire.sh
```

- Run the install script:

```
./install_mvGenTL_Acquire.sh
```

During installation the script will ask, if it should build all tools and samples.

> **Note**
>
> The installation scripts is developed for Ubuntu/Debian, SUSE Linux and Red Hat Linux based distributions. On other distributions some features of the installation script may or may not work. Get in touch with us if you encounter any problems!

The installation script checks for package dependencies described above and installs them with the respective standard package manager (e.g. **apt-get**) if necessary. So an Internet connection is recommended.

> **Note**
>
> The installation script (`install_mvGenTL_Acquire.sh`) and the archive (`mvGenTL_Acquire-x86_64_ABI2-n.n.n.tgz`) must reside in the same directory. Nothing is written to this directory during script execution, so no write access to the directory is needed in order to execute the script.

You need Internet access in case one or more of the packages on which the **GenICam** (p. 165) libs depend are not yet installed on your system. In this case, the script will install these packages, and for that, Internet access is required.

The script supports various arguments, which allow to customize the installation, the desired functionalities and the installation process itself. All arguments are optional:

Argument	Function
`-h` or `-help`	Display the help.
`-p` or `-path`	Define a custom installation directory.
`-gev` or `-gev_support`	Install the GigE Vision related features of the driver. Default: **yes**
`-u3v` or `-u3v_support`	Install the USB3 Vision related features of the driver. Default: **yes**
`-pcie` or `-pcie_support`	Install the PCI Express related features of the driver. Default: **yes**
`-ogev` or `-only_gev`	Install only the GigE Vision related features of the driver (deprecated, favor flavors without the 'only' instead). Default: **no**

`-ou3v` or `-only_u3v`	Install only the USB3 Vision related features of the driver (deprecated, favor flavors without the 'only' instead). Default: **no**
`-onaos` or `-only_naos`	Install only the PCI Express related features of the driver (deprecated, favor flavors without the 'only' instead). Default: **no**
`-u` or `-unattended`	Unattended installation with default settings. By using this parameter you explicitly accept the EULA.
`-m` or `-minimal`	Minimal installation. No tools or samples will be built, and no automatic configuration and/or optimizations will be done. By using this parameter you explicitly accept the EULA.

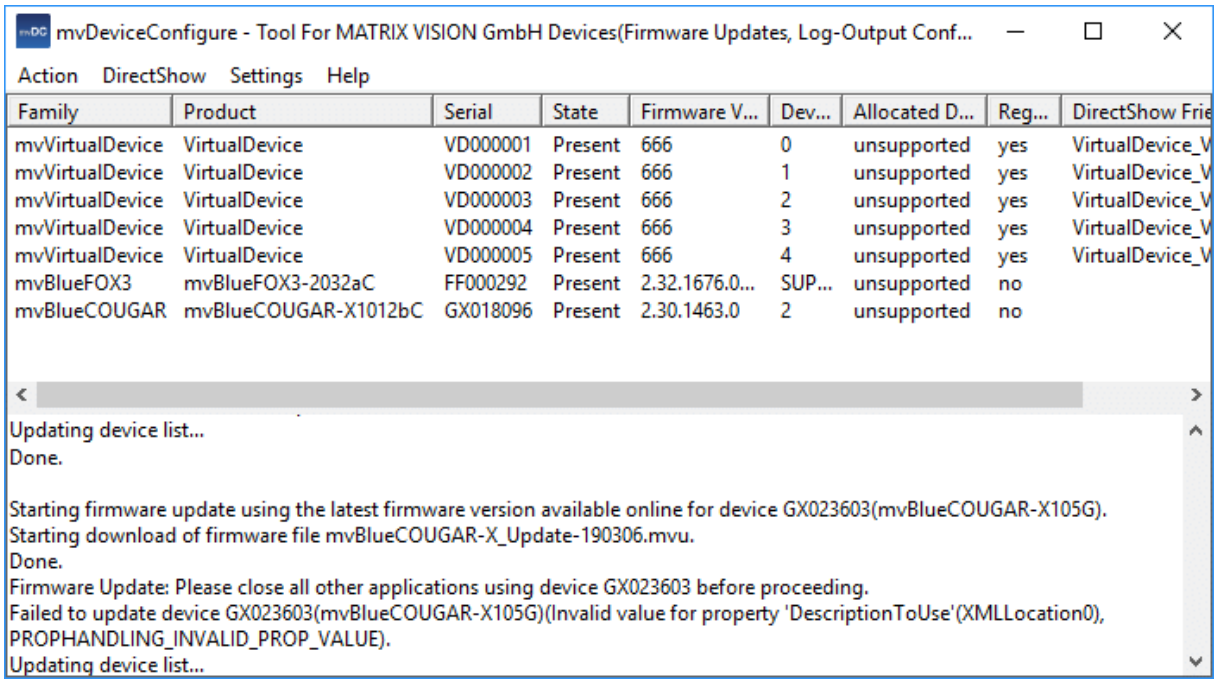
The target directory name specifies where to place the driver. If the directory does not yet exist, it will be created. The path can be either absolute or relative; i.e. the name may but need not start with `/` .

If no path is specified, the package will be installed to **/opt/mvIMPACT_Acquire**.

1.8.3 Connecting The Camera

After the driver installation you have to connect the mvBlueFOX3 using a USB3 cable.

You can check if the driver installation was successful by using **mvDeviceConfigure** (p. 99). Supported device with an installed and running driver should be listed:



Connected mvBlueFOX3

Afterwards, you can start **wxPropView** (p. 99) to configure the mvBlueFOX3.

Since driver version 2.11.3, starting **wxPropView** (p. 99) the first time, the so called Quick Setup Wizard will be started. Read more about how to make optimal use of it in the mvIMPACT Acquire GUI manual: `https://www.matrix-vision.com/manuals/SDK_GUI_Tools/index.html`

1.8.3.1 Communicating With The Camera

You can communicate with the camera the following way

1. **Via wxPropView** (p. 99) :
   Since driver version 2.11.3, starting **wxPropView** (p. 99) the first time, the so called Quick Setup Wizard will be launched.

   Read more about how to make optimal use of it in the mvIMPACT Acquire GUI manual: `https://www.matrix-vision.com/manuals/SDK_GUI_Tools/index.html`

1.8.3.2 Setting Up The Camera

MATRIX VISION offers several **GUI tools** (p. 99) to work with the camera. Please have a look at the specific chapter.

1.8.3.3 About Settings

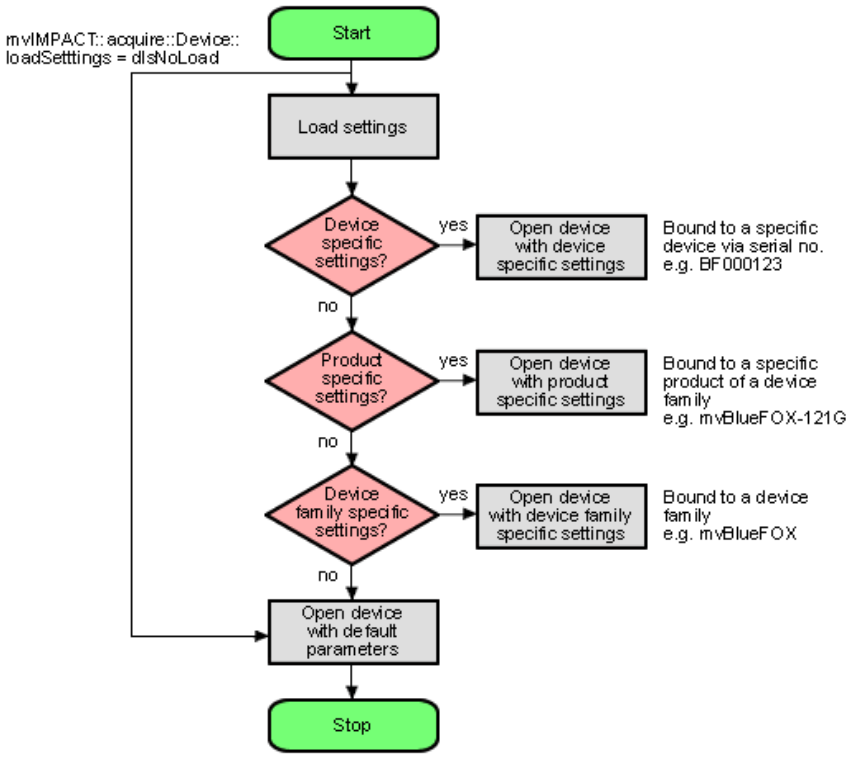
A setting contains all parameters that are needed to configure the device to a state it was in when the setting was created. Every image can be captured with a completely different set of parameters. In almost every case, these parameters are accessible via a property offered by the device driver. A setting e.g. might contain

- The gain to be applied to the analog to digital conversion process for analog video sources or
- The AOI to be captured from the incoming image data.

So for the user a setting is the one and only place where all the necessary modifications can be applied to achieve the desired data acquisition mode. There is however an important difference in behaviour between different interface layouts. Please have a look at the **"mvIMPACT Acquire SDK GUI Applications"** chapter **"wxPropView ->** **Device Configuration ->** **General Device Configuration ->** **Changing The Interface Layout To GenICam Or Device↩ Specific"** to find out how to modify the interface layout or check in the API documentation for the **interfaceLayout** property of the class **Device**.

- When working with the **DeviceSpecific** interface layout, each frame will be captured with the settings as present when requesting the image. Every parameter can be modified at any time. When requesting another image the settings valid at that moment will be used to fill this buffer with data
- For the **GenICam** interface layout all device properties modified during a continuous acquisition will be applied at once so might affect this or the next image transmitted by the device. Depending on various parameters (the number of buffer already captured but not collected by the application, the way the device internally operates(e.g. has already captured a couple of images that await transmission), etc.) this will have impact on that captured images somewhere in the near future thus when a precise moment to change settings is needed, continuous acquisition must be stopped and then restarted after modifying the features. Certain features (typically those affecting the buffer layout/size) cannot be changed while a continuous acquisition is running in GenICam interface layout anyway.

Now, whenever a device is opened, the driver will execute following procedure:



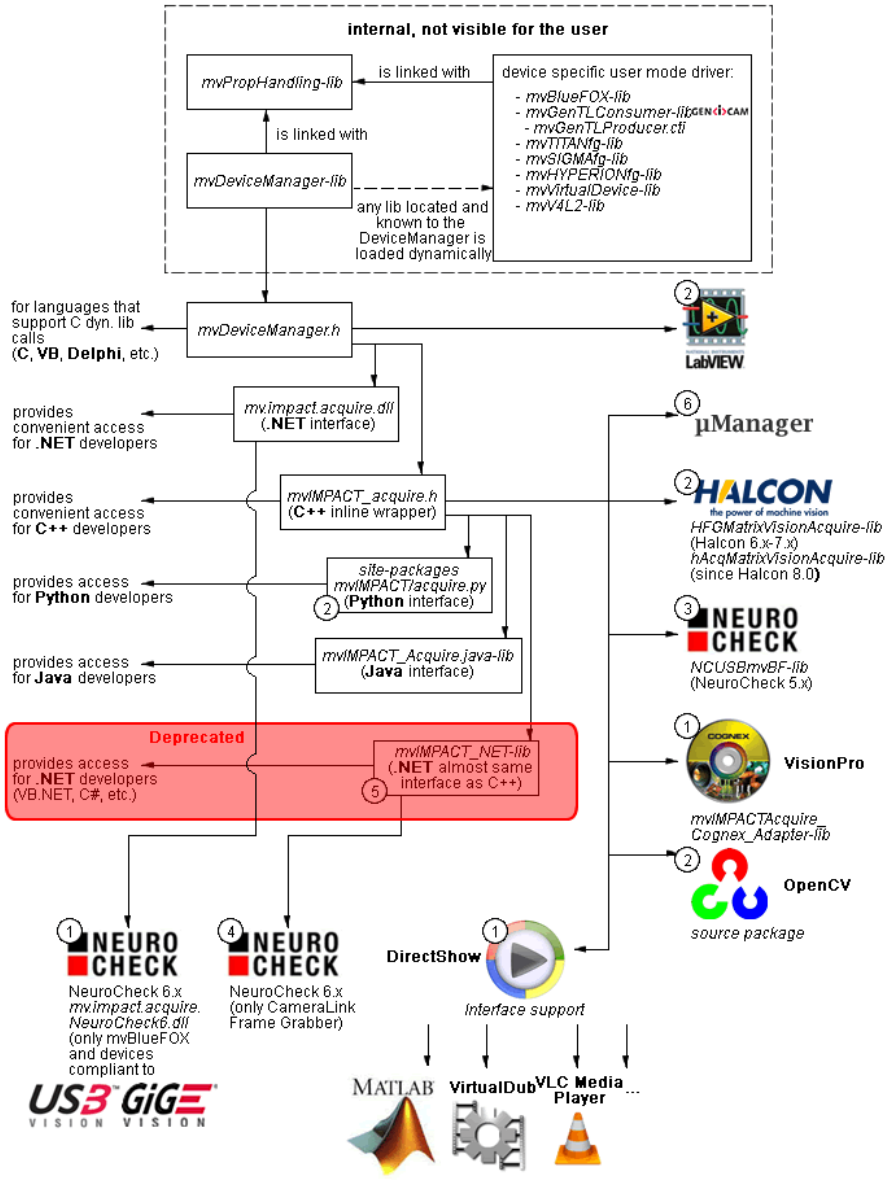
wxPropView - Device setting start procedure

- Please note that each setting location step in the figure from above internally contains two search steps. First the framework will try to locate a setting with user scope and if this can't be located, the same setting will be searched with global (system-wide) scope. On Windows this e.g. will access either the **HKEY_CURRENT_USER** or (in the second step) the **HKEY_LOCAL_MACHINE** branch in the Registry.
- Whenever storing a product specific setting, the device specific setting of the device used for storing will be deleted (if existing). E.g. you have a device "VD000001" which belongs to the product group "VirtualDevice" with a setting exclusively for "VD000001". As soon as you store a product specific setting **using THIS device**, the (device specific) setting for "VD000001" will be deleted. Otherwise a product specific setting would never be loaded as a device specific setting will always be found first. Storing a product specific setting with a different device belonging to the same family however will **NOT** delete device specific settings for other devices.
- The very same thing will also happen when opening a device from any other application! mvPropView does not behave in a special way but only acts as an arbitrary user application.
- Whenever storing a device family specific setting, the device specific or product specific setting of the device used for storing will be deleted (if existing). See above to find out why.
- On Windows the driver will **not** look for a matching XML file during start-up automatically as the native storage location for settings is the Windows Registry. This must be loaded explicitly by the user by using the appropriate API function offered by the SDK. However, under **Linux** XML files are the only setting formats understood by the driver framework thus here the driver will also look for them at start-up. The device specific setting will be an XML file with the serial number of the device as the file name, the product specific setting will be an XML file with the product string as the filename, the device family specific setting will be an XML file with the device family name as the file name. All other XML files containing settings will be ignored!
- Restoring of settings previously stored works in a similar way. After a device has been opened the settings will be loaded automatically as described above.
- A detailed description of the individual properties offered by a device will **not** be provided here but can be found in the C++ API reference, where descriptions for all properties relevant for the user (grouped together in classes sorted by topic) can be found. As mvPropView doesn't introduce new functionality but simply evaluates the list of features offered by the device driver and lists them any modification made using the GUI controls just calls the underlying function needed to write to the selected component. mvPropView also doesn't know about the type of component or e.g. the list of allowed values for a property. This again is information delivered by the driver and therefore can be queried by the user as well without the need to have special *inside information*. One version of the tool will always be delivered in source so it can be used as a reference to find out how to get the desired information from the device driver.

1.8.4 Driver concept

The driver supplied with the MATRIX VISION product represents the port between the programmer and the hardware. The driver concept of MATRIX VISION provides a standardized programming interface to all image processing products made by MATRIX VISION GmbH.
The advantage of this concept for the programmer is that a developed application runs without the need for any major modifications to the various image processing products made by MATRIX VISION GmbH. You can also incorporate new driver versions, which are available for download free of charge on our website:
`https://www.matrix-vision.com`.

The following diagram shows a schematic structure of the driver concept:



Driver concept

- **1** Part of any mvIMPACT Acquire driver installation package (Windows).
- **2** Separately available for 32 bit and 64 bit. Requires at least one installed driver package.
- **3** See 2, but requires an installed version of the mvBlueFOX driver.

- **4** Part of the NeuroCheck installer but requires at least one installed frame grabber driver.
- **5** Part of the mvIMPACT SDK installation. However, new designs should use the .NET libs that are now part of mvIMPACT Acquire ("mv.impact.acquire.dll"). The namespace "mv.impact.acquire" of "mv.impact.acquire.dll" provides a more natural and more efficient access to the same features as contained in the namespace "mvIMPACT_NET.acquire" of "mvIMPACT_NET.dll", which is why the latter one should only be used for backward compatibility but **NOT** when developing a new application.
- **6** Part of Micro-Manager.

1.8.4.1 NeuroCheck Support

A couple of devices are supported by NeuroCheck. However between NeuroCheck 5.x and NeuroCheck 6.x there has been a breaking change in the internal interfaces. Therefore also the list of supported devices differs from one version to another and some additional libraries might be required.

For NeuroCheck 5.x the following devices are supported:

Device	Additional software needed
mvTITAN-G1	mvSDK driver for mvTITAN/mvGAMMA devices
mvTITAN-CL	mvSDK driver for mvTITAN/mvGAMMA devices
mvGAMMA-CL	mvSDK driver for mvTITAN/mvGAMMA devices
mvBlueFOX	mvIMPACT Acquire driver for mvBlueFOX devices, "NCUSBmvBF.dll"

For NeuroCheck 6.0 the following devices are supported:

Device	Additional software needed
mvTITAN-G1	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvTITAN-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvGAMMA-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvHYPERION-CLb	mvIMPACT Acquire driver for mvHYPERION devices
Every other mvIMPACT Acquire compliant device	mvIMPACT Acquire driver for the corresponding device family, "mv.impact.acquire.NeuroCheck6.↩ dll" (comes with the driver package, but the driver package must be installed **AFTER** installing NeuroCheck 6

For NeuroCheck 6.1 the following devices are supported:

Device	Additional software needed
mvTITAN-G1	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvTITAN-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvGAMMA-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvHYPERION-CLb	mvIMPACT Acquire driver for mvHYPERION devices
Every other mvIMPACT Acquire compliant device	mvIMPACT Acquire driver for the corresponding device family, "mv.impact.acquire.NeuroCheck6_1.dll" (comes with the driver package, but the driver package must be installed **AFTER** installing NeuroCheck 6.1

1.8.4.2 VisionPro Support

Every *mvIMPACT Acquire* driver package on Windows comes with an adapter to VisionPro from Cognex. The installation order does not matter. After the driver package and VisionPro has been installed, the next time VisionPro is started it will allow selecting the *mvIMPACT Acquire* device. No additional steps are needed.

MATRIX VISION devices that also comply with the GigE Vision or USB3 Vision standard don't need any software at all, but can also use VisionPro's built-in GigE Vision or USB3 Vision support.

1.8.4.3 HALCON Support

HALCON comes with built-in support for *mvIMPACT Acquire* compliant devices, so once a device driver has been installed for the *mvIMPACT Acquire* device, it can also be operated from a HALCON environment using the corresponding acquisition interface. No additional steps are needed.

MATRIX VISION devices that also comply with the GigE Vision standard don't need any software at all, but can also use HALCON's built-in GigE Vision support.

As some *mvIMPACT Acquire* device driver packages also come with a GenTL compliant interface, these can also be operated through HALCON's built-in GenTL acquisition interface.

1.8.4.4 LabVIEW Support

Every *mvIMPACT Acquire* compliant device can be operated under LabVIEW through an additional set of VIs which is shipped by MATRIX VISION as a separate installation (`"mvLabVIEW Acquire"`).

MATRIX VISION devices that also comply with the GigE Vision or USB3 Vision standard don't need any additional software at all, but can also be operated through LabVIEW's GigE Vision or USB3 Vision driver packages.

1.8.4.5 DirectShow Support

Every *mvIMPACT Acquire* compliant device driver package comes with an interface to DirectShow. In order to be usable from a DirectShow compliant application, devices must first be registered for DirectShow support. How to this is explained **here** (p. 134).

1.8.4.6 Micro-Manager Support

Every *mvIMPACT Acquire* compliant device can be operated under `https://micro-manager.org` when using mvIMPACT Acquire 2.18.0 or later and at least *Micro-Manager* 1.4.23 build **AFTER** 15.12.2016. The adapter needed is part of the *Micro-Manager* release. Additional information can be found here: `https://micro-manager.org/wiki/MatrixVision`.

1.8.5 Relationship Between Driver, Firmware And SDK

To operate a **GenICam** (p. 165) based device apart from the physical hardware itself 2 pieces of software are needed:

- A firmware running on the device. This firmware consists of
  - A **GenICam** (p. 165) compliant XML file exposing the features in a generic and standard compliant way
  - A **FPGA** file
  - The actual **micro-code** making the device operational
- A **device driver** (this is the `mvGenTLConsumer.dll` and the `mvGenTLProducer.cti` on Windows and the `libmvGenTLConsumer.so` and the `libmvGenTLProducer.so` on Linux when using mv↩ IMPACT Acquire, but can be any other **USB3 Vision** (p. 173) / **GigE Vision** (p. 167) compliant driver package from a third party vendor) running of the host system (provides control over the device from an application running on the host system).

The physical **GenICam** (p. 165) compliant device has a firmware programmed into the device's non-volatile memory, thus allowing the device to boot to a fully functional state without the need of any additional software running on the host. The firmware version that will be used when operating the device does NOT depend on the driver version that is used to communicate with the device. This will allow any piece of compliant third party software to operate the device without the need to have special knowledge about the firmware structure. This shall be illustrated by the following figure:

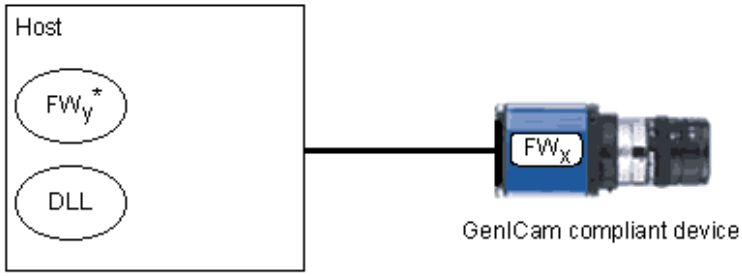


Figure 8: The firmware is not a part of the device driver

Note

As it can be seen in the image the firmware file is NOT part of the device driver but comes as a separate archive. It is important to notice that a firmware file that may be present on the host system will *not* be used automatically but only when the user or an application explicitly updates the firmware on the device and will only become active after power-cycling the device.

The name of the firmware update archive ($*$ in the figure above) is:

- **mvBlueFOX3_Update.mvu**

Only during a firmware update the firmware file that has been selected from the file system of the host system will be downloaded permanently into the device's non-volatile memory.

Attention

"Wrong firmware"

Each firmware archive might contain multiple firmware files (e.g. for different device types or device hardware revisions). Installing an unsuitable firmware version can damage the device.

→ In order to make sure suitable firmware versions for a specific device are installed appropriate tools such as **mvDeviceConfigure** (p. 99) should be used.

So assume a device with a certain firmware version is connected to a host system.

During an explicit firmware update, the firmware file will be downloaded onto the device. In order to become active the device must be power-cycled:

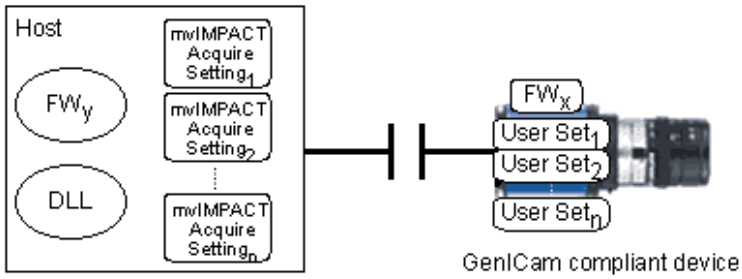


Figure 9: Firmware file will be downloaded during an firmware update...

This can either be done by unplugging the device and then by plugging it back in or (for devices supporting this feature) by resetting/rebooting the device by a certain software command (DeviceControl/DeviceReset). When using **mvDeviceConfigure** (p. 99) to update devices the latter mechanism will be used by the tool thus it is NOT necessary to unplug the device.

When the device has completed rebooting the new firmware version will become active:

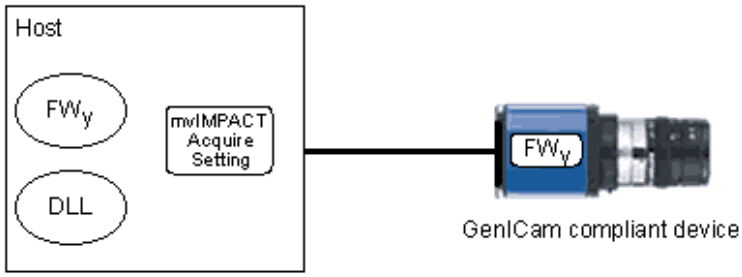


Figure 10: ... after re-powering the device it will be active

- The current firmware version of the device can be obtained either by using one of the applications which are part of the SDK such as **mvDeviceConfigure** (p. 99) or by reading the value of the property `Device/↩FirmwareVersion` or `DeviceControl/DeviceFirmwareVersion` using the API
- The current FPGA file version used by the device can be obtained by reading the value of the property `DeviceControl/mvDeviceFPGAVersion`

Note

The FPGA file is a part of the firmware and cannot be updated independently thus reading it's version just provides some additional information.

Using **wxPropView** (p. 99) the same information is available as indicated by the following figure:

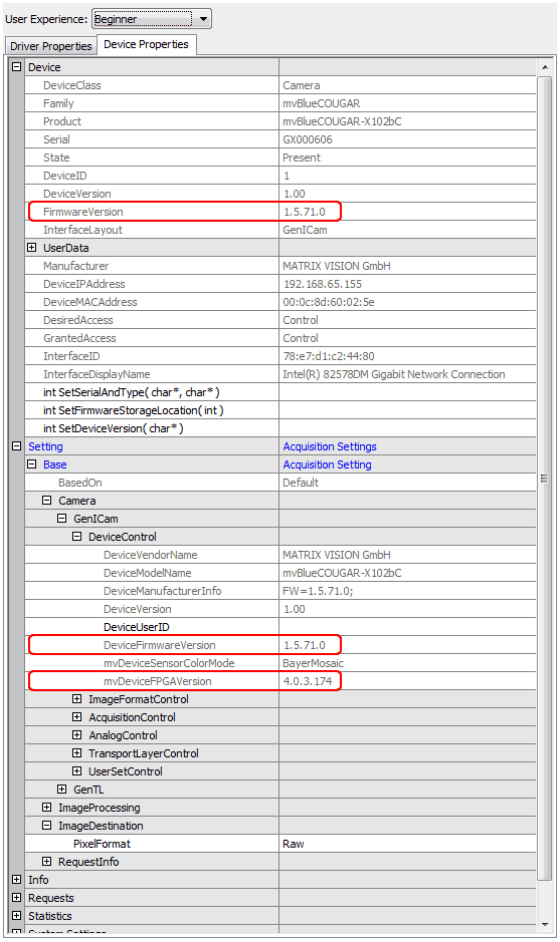


Figure 11: wxPropView - FPGA and Firmware version numbers

Apart from the device driver and firmware relationship there are certain places where a device configuration can be stored when dealing with **GenICam** (p. 165) compliant devices:

- There may be User Sets which are stored in the device's non-volatile memory. User Sets contain all the features, which affect the device's behaviour such as transfer pixel format, exposure time etc. User Sets are bound to major **GenICam** (p. 165) XML file releases, thus these settings will be lost whenever a firmware contains a different major version of a devices **GenICam** (p. 165) XML file
- mvIMPACT Acquire settings which contain the state of all the features also stored in a User Set as well as other features added by the device driver. These settings will be stored on the host system either as a XML file or (on Windows only) in the Registry

Both methods can be used to pre-configure a device. Using the first method, the state of the features will travel with the physical device, using the mvIMPACT Acquire settings, feature states can be copied from host to host as a file.

1.8.6 Optimizing USB Performance

Note

This section is only relevant for applications working with USB3 Vision™ or MATRIX VISION USB 2.0 devices!

Another important aspect when dealing with USB3 Vision™ devices is described **here** (p. 275).

1.8.6.1 Checklist for Windows

1.8.6.1.1 Host Controller Driver Also the USB host controller manufacturers provide driver updates for their cards/chips every now and then. Using the latest drivers is always recommended and might improve the overall performance of the system dramatically!

1.8.6.2 Checklist for Linux

1.8.6.2.1 udev rules Most Linux system nowadays use the **udev** device manager, which is responsible for dynamically managing the `/dev` tree. In order to be able to use the MATRIX VISION mvBlueFOX3 **"USB3 Vision"** camera as non-root user, a special set of rules has to be handed to the udev device manager.

On older systems this could be done by directly editing the contents of a "`/etc/udev/rules`" file, however nowadays a "`/etc/udev/rules.d`" directory exists, which may contain several different files, each defining the behavior of a system device.

In the specific case of mvBlueFOX3 device or any **"USB3 Vision"** device actually, if the camera has been installed through the respective installation script `install_mvGenTL_Acquire.sh` , a suitable set of rules has been installed automatically. However if for some reason these rules have to be created manually or must be changed at later time it should be done like this:

1. Create a file in the "`/etc/udev/rules.d`" directory with name `52-U3V.rules` if this doesn't exist already. The content of the file should be something like this:

```
SUBSYSTEM!="usb|usb_device|plugdev", GOTO="u3v_rules_end"
ACTION!="add", GOTO="u3v_rules_end"

ATTRS{bDeviceClass}=="ef", ATTRS{bDeviceSubClass}=="02", ATTRS{bDeviceProtocol}=="01",
        ENV{ID_USB_INTERFACES}=="*:ef0500:*", MODE="0664", GROUP="plugdev"

LABEL="u3v_rules_end"
```

2. OPTIONAL: Create another file in the "`/etc/udev/rules.d`" directory with name `52-mvbf3.rules` . This step is only necessary if a mvBlueFOX3 in the "mvbootloader" state should be recognised by the system. This might happen if for any reason a camera has no valid firmware running e.g. due to a power failure during a firmware update. The content of the file should be something like this:

```
SUBSYSTEM!="usb|usb_device|plugdev", GOTO="mvbf_rules_end"
ACTION!="add", GOTO="mvbf_rules_end"

ATTRS{idVendor}=="164c", ATTRS{idProduct}=="5531", MODE="0664", GROUP="plugdev"

LABEL="mvbf_rules_end"
```

Note

The above `52-U3V.rules` file provides the necessary access privileges not only for mvBlueFOX cameras, but also for any **"USB3 Vision"-compliant** device of other vendors.

As soon as this file is into place, each time the camera is plugged to the system it acquires the set of rights that allows the user to use it without having root privileges.

1.8.6.2.2 Increasing Kernel memory On most modern Linux systems, support for USB3 functionality has been moved from separate kernel modules to the kernel itself (usbcore). On such systems, the kernel memory which is allocated for the use with USB is predefined and set to a relatively small value (e.g. 16 MB on a typical 64-bit Ubuntu system). This value is usually enough for reading from an external HDD, operating a mouse or a keyboard or similar applications, but in the case of machine vision applications, with Megapixel sensors and ultra-fast transfer speeds it is way too low since this memory is needed as a temporary buffer for image data and especially with high resolution sensors not even a single image might fit into this memory segment then.

Note

You may inquire the value (in Megabytes) on your system by typing:
`cat /sys/module/usbcore/parameters/usbfs_memory_mb`

We recommend to increase this value to at least 256 MB or even more depending on your application (number of cameras, number of image buffers needed per camera etc.). For example, a 5 Mpix camera, capturing RGB data with a default request/image buffer count of 10, needs about 5M $*$ 3 $*$ 10 $\sim$= 150MB of the usbfs memory alone. Always bear in mind though, that mvIMPACT Acquire is probably not the only system component using usbcore memory, so you should always reserve several MB more than the number you get from the above calculation. As a general rule of thumb, for a single-camera application with a medium resolution and a default request setting of 10, a value of 256 MB should be adequate.

Note

If systemd is supported on the system, the installation script will ask if a systemd service should be created which modifies the value on each boot. Once this option is used, which is **highly recommended**, the following Kernel memory chapters will no longer be relevant and can be skipped.

1.8.6.2.2.1 Increasing Kernel memory at boot time To change the value of the `usbfs_memory_mb` system parameter, one has to **invoke the kernel at boot time with an argument that sets the parameter to the desired value**. Trying to modify this parameter after the system has booted (e.g. with `modprobe`), will have no effect since `usbcore` is a system module integrated in the Linux kernel, and not a separate kernel module which can be loaded and unloaded on demand.

Passing parameters to the kernel at boot time is usually done by typing: `systemModuleName.↩ parameter=value` . Therefore to reserve 256MB of USB memory we need:

```
usbcore.usbfs_memory_mb=256
```

How this can be done depends on the system bootloader. For systems using the `GRUB2` bootloader the `"/etc/default/grub"` (or in some distributions `"/etc/default/grub/c.cfg"`) file has to be modified.

Attention

Always modify configuration files with **extreme caution**, since the slightest syntax error may render the file invalid, or even the system no longer bootable!

After opening this file, the GRUB_CMDLINE_LINUX_DEFAULT entry must be located. It usually has the following value:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

In order to invoke the Kernel with the usbfs_memory_mb parameter, it should be modified like:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash usbcore.usbfs_memory_mb=256"
```

As a final step, GRUB has to be updated by executing the update-grub script:

```
sudo update-grub
```

If all went well, after rebooting the system, the kernel will utilize the requested amount of memory for the usbcore subsystem. See **Increasing Kernel memory** (p. 54) again for how to check this!

Note

On systems with GRUB Legacy support the bootloader settings are controlled by other files (e.g. "/boot/grub/menu/c.lst" etc.). In this case it is recommended to upgrade your bootloader to GRUB2 :

```
sudo apt-get install grub2
```

If, for some reason, this is not an option, then the menu.lst itself could be directly modified as an absolutely last resort. Always keep in mind that **it is strongly recommended not to tamper directly with files in the /boot/grub directory!** The smallest typing error can render the system no longer bootable!

1.8.6.2.2.2 Increasing Kernel memory at runtime If just a temporary change of the usbfs_memory_mb parameter is needed or there is no way to pass the parameter at boot time to the kernel, it will be sufficient to modify the usbfs.usbfs_memory_mb value at runtime. The following command will be sufficient to change the usbfs_memory_mb value to 256 MB until the system is restarted:

```
sudo sh -c 'echo 256 > /sys/module/usbcore/parameters/usbfs_memory_mb'
```

Note

root permissions will be necessary to change the parameter at runtime.

1.8.6.2.3 Disabling The Auto-Suspend Mode Usually the Linux kernel suspends USB devices when they are not in use for a certain time. In some cases this might cause unsuspected behaviour of USB devices. To avoid this kind of issues it is a good idea to disable the USB autosupend mode.

```
sudo sh -c 'echo -1 > /sys/module/usbcore/parameters/autosuspend'
```

1.8.7 Using USB3 Vision™ Devices In A Docker Container

When developing machine vision applications using Docker containers, chances are that you would like to access USB3 Vision™ devices inside the container. With the mvIMPACT Acquire driver stack this can be achieved fairly easy and this chapter will show you how to build a basic Docker container where you can use USB3 Vision™ devices. The current sample Docker container runs on a native Linux machine.

Note

The following chapter is documented only for a native Linux host system.

1.8.7.1 Host Preparation

Note

For this demo Docker container the operating system of the host machine is Linux.

Since Docker uses the kernel of the host machine, we first have to **Increasing Kernel memory** (p. 54) of the USB filesystem to make sure that there will be enough temporary buffer for image data transmission at USB3 speed.

1.8.7.2 Building A Docker Image

The following demo Dockerfile builds a basic Docker image based on a slim version of Debian, where the mv↩ IMPACT Acquire GenTL driver package and its sample programs are installed. This Dockerfile can be used in many ways:

- Use it directly to test your device in a Docker container.
- Use it as a base image for your device applications.
- Use it as an inspiration for building your own Dockerfile.

Before building the Dockerfile, please download the mvIMPACT Acquire GenTL driver installation files from MATRIX VISION GmbH website (`https://www.matrix-vision.com/treiber-software.html`) (user login is required):

- The installation script: *install_mvGenTL_Acquire.sh*
- The installation package: *mvGenTL_Acquire-x86_64_ABI2-*.tgz* (* should be replaced by the version number)

Create a directory called *mvIMPACT_Acquire* (as used in this demo Dockerfile) and move both installation files into this directory. In this example, both files are downloaded into the *Downloads* directory and the *mvIMPACT_Acquire* directory is created inside the *Downloads* directory:

```
$ cd ~/Downloads
$ mkdir mvIMPACT_Acquire
$ mv install_mvGenTL_Acquire.sh mvGenTL_Acquire-x86_64_ABI2-*.tgz mvIMPACT_Acquire/
```

Make the installation script *install_mvGenTL_Acquire.sh* executable:

```
$ cd mvIMPACT_Acquire
$ chmod a+x install_mvGenTL_Acquire.sh
```

Navigate back into the directory where *mvIMPACT_Acquire* resides (e.g. *Downloads*) and create your Dockerfile:

```
$ cd ~/Downloads
$ touch Dockerfile
```

Create the content of your Dockerfile. Our demo Dockerfile (for Linux x86_64) looks as follows:

```
# start with slim version of actual Debian
FROM debian:9-slim

ENV LC_ALL C
ENV DEBIAN_FRONTEND noninteractive

# entrypoint of Docker
CMD ["/bin/bash"]

# set environment variables
ENV TERM linux
ENV MVIMPACT_ACQUIRE_DIR /opt/mvIMPACT_Acquire
ENV MVIMPACT_ACQUIRE_DATA_DIR /opt/mvIMPACT_Acquire/data
ENV GENICAM_GENTL64_PATH /opt/mvIMPACT_Acquire/lib/x86_64
ENV GENICAM_ROOT /opt/mvIMPACT_Acquire/runtime
ENV container docker

# update packets and install minimal requirements
# after installation it will clean apt packet cache
RUN apt-get update && apt-get -y install build-essential && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

# move the directory mvIMPACT_Acquire with *.tgz and *.sh files to the container
COPY mvIMPACT_Acquire /var/lib/mvIMPACT_Acquire

# execute the setup script in an unattended mode
RUN cd /var/lib/mvIMPACT_Acquire && \
    ./install_mvGenTL_Acquire.sh -u && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

Note

In case of ARM architectures, all occurrences of "x86_64" in this demo dockerfile have to be replaced by the correct platform e.g. "arm64" and the install script to use will be *install_mvGenTL_Acquire_ARM.sh* then.

Finally, build a Docker image using this Dockerfile:

```
$ sudo docker build -t [image_name] .
```

Note

Please make sure to call *docker build* from within the directory where the Dockerfile resides. Note that Internet access is required for the *docker build*.

If built successfully, you will be able to see [image_name] being listed when calling:

```
$ sudo docker images
```

1.8.7.3 Starting The Docker Container

Since the Docker container is isolated from the host system, we have to run it with certain volume mounts and cgroup permissions for it to access USB3 Vision™ devices. In order to avoid running the container in privileged mode, which is not secure, it can be started like this:

```
$ sudo docker run -ti -v /dev:/dev -v /run/udev:/run/udev:ro --device-cgroup-rule 'a 189:* rwm'
 [image_name] /bin/bash
```

Where:

- **-v /dev:/dev**: use volume mount to map the host */dev* directory to the container, so the container will be able to always detect devices also when they get unplugged and re-plugged at any time.
- **-v /run/udev:/run/udev:ro**: volume mount the udev database with read-only permission, so the USB3 Vision™ interfaces can be enumerated correctly in the container.
- **–device-cgroup-rule 'a 189:* rwm'**: with the *–device-cgroup-rule* flag we can add specific permission rules to a device list that is allowed by the container's cgroup. Here in this example, *189* is the major number of the USB bus, * means all minor numbers, and *rwm* are respectively read, write, mknod accesses. By doing so, we will have read, write, mknod accesses to all the USB devices. USB3 Vision™ devices can thus be enumerated successfully.

1.8.7.4 Validation

After starting the container the correct operation of USB3 Vision™ devices can be validated by running one of the sample programs provided by the mvIMPACT Acquire (e.g. *SingleCapture*):

```
$ cd /opt/mvIMPACT_Acquire/apps/SingleCapture/x86_64
$ ./SingleCapture
```

If the attached USB3 Vision™ device appears in the device list of the program's output, congratulations, you've managed to access USB3 Vision™ devices in the container by using the mvIMPACT Acquire. Now you can use them inside the Docker container for your machine vision applications.

1.9 Technical Data

1.9.1 Dimensions

1.9.1.1 Standard model (mvBlueFOX3-1)

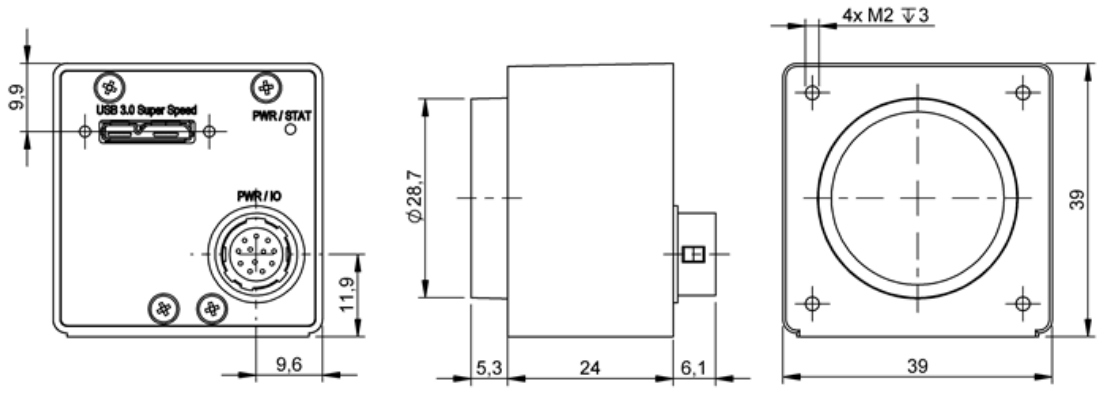


Figure 1: mvBlueFOX3 -xx1x dimensions and connectors

mvBlueFOX3	
Size of body (w x h x l)	39 x 39 x 24 mm

Lens protrusion	C-Mount	CS-Mount
	12.5 mm	7.5 mm

Mounting holes

On the bottom-side, the mvBlueFOX3 provides integrated tripod mounting holes.

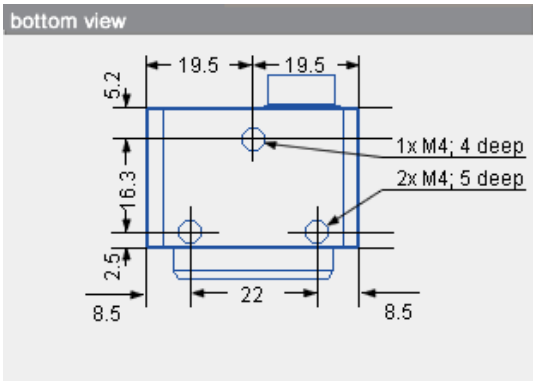


Figure 2: mvBlueFOX3 mounting holes

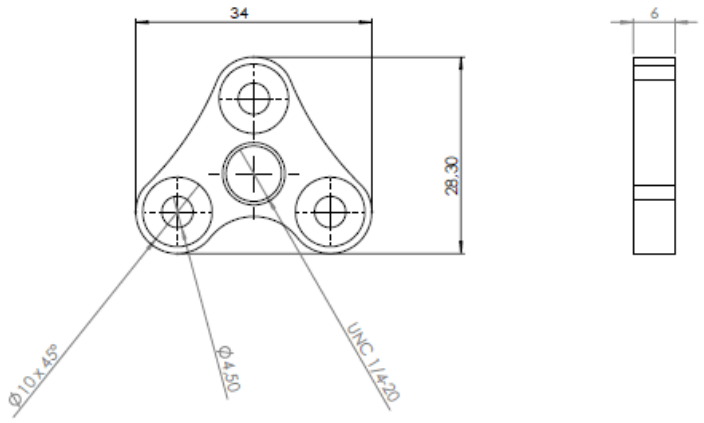


Figure 3: Dimensional drawing of tripod adapter

1.9.1.2 Standard model (mvBlueFOX3-2)

1.9.1.2.1 Option -1xxx (lensholder without back focus adjustment)

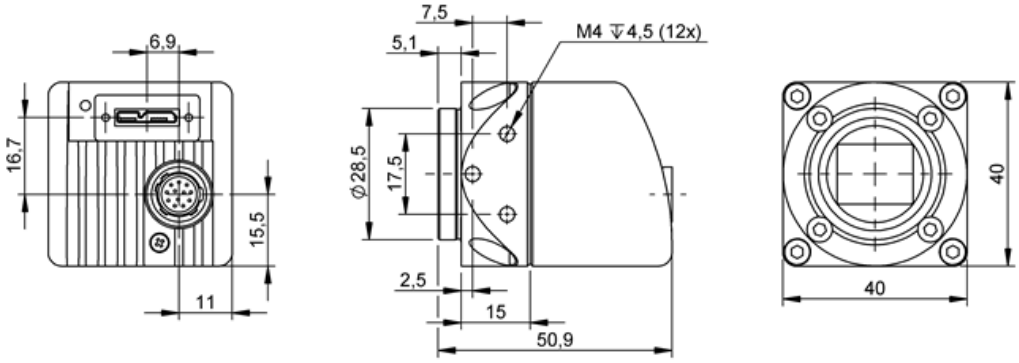


Figure 4: mvBlueFOX3-2xxx-1xxx dimensions and connectors

mvBlueFOX3-2xxx-1xxx	
Size of body (w x h x l)	40 x 40 x 50.9 mm

Lens protrusion	
	10.7 mm with 1" lenses

Mounting holes

The mvBlueFOX3-2xxx-1xxx provides integrated mounting holes.

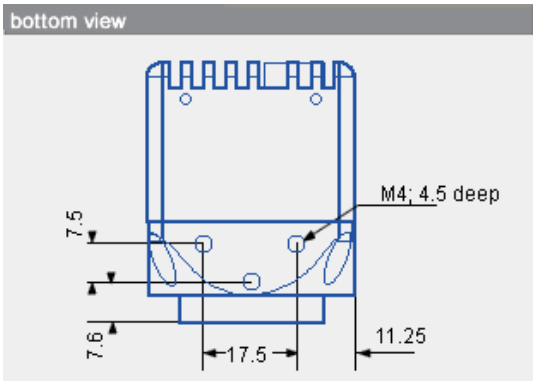


Figure 5: mvBlueFOX3-2xxx-1xxx mounting holes

1.9.1.2.2 Option -2xxx (lensholder with back focus adjustment)

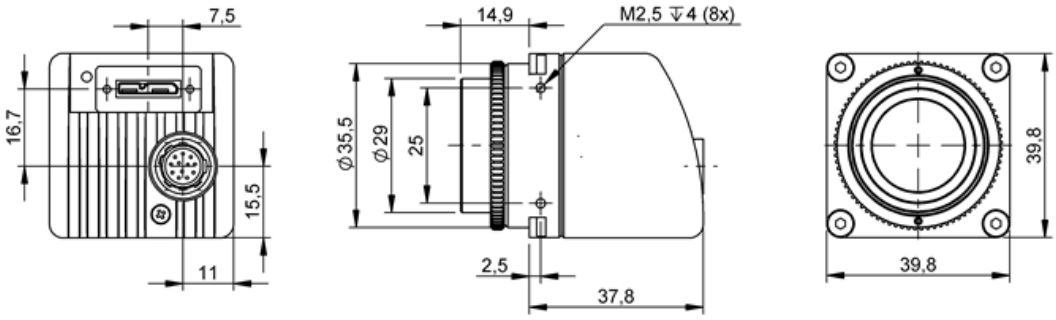


Figure 6: mvBlueFOX3-2xxx-2xxx dimensions and connectors

mvBlueFOX3-2xxx-2xxx	
Size of body (w x h x l)	39.8 x 39.8 x 37.7 mm

Lens protrusion	C-Mount	CS-Mount

X	8 mm	6 mm

	C-Mount	CS-Mount
W	approx. 15 mm (C-mount)	approx. 10 mm (CS-mount)
Z	17.526 mm (in air)	12.5 mm (in air)

Mounting holes

The mvBlueFOX3-2xxx-2xxx provides integrated mounting holes.

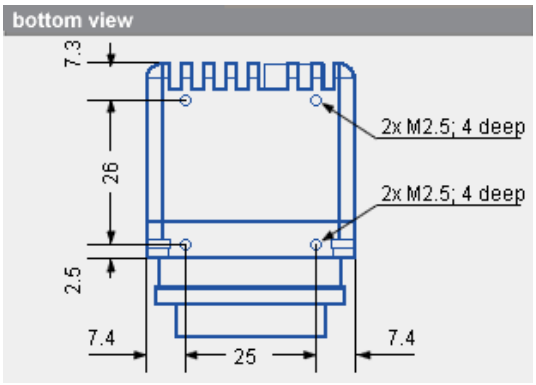


Figure 7: mvBlueFOX3-2xxx-2xxx mounting holes

1.9.1.3 Model without housing (mvBlueFOX3-M1)

Attention

"Broken connectors"

Handle the connectors with care otherwise you can damage the device.

→ In combination with the connectors, please limit the mechanical stress.

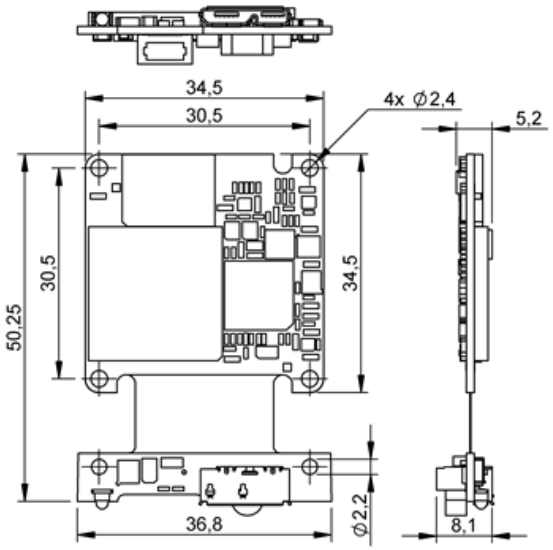


Figure 8: mvBlueFOX3-M dimensions and connectors

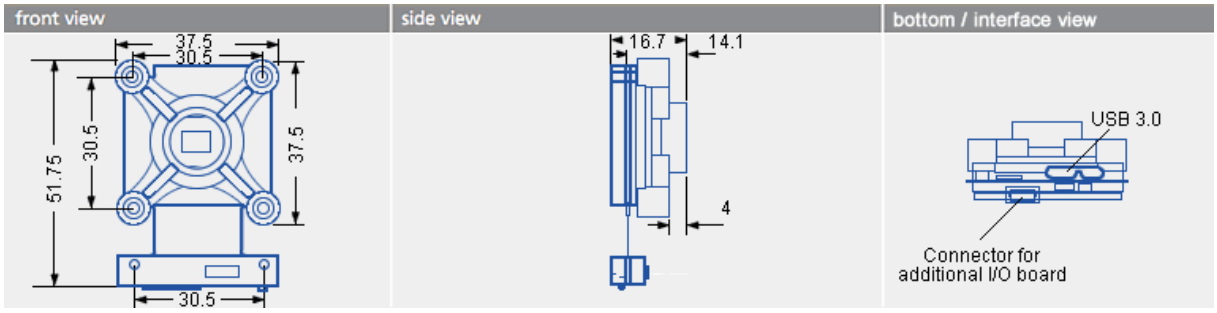


Figure 9: mvBlueFOX3-M1xxx-2xxx with S-Mount lensholder BF3-LH-SMNT 13

1.9.1.3.1 I/O board for mvBlueFOX3-M (mvBlueFOX3-IO)

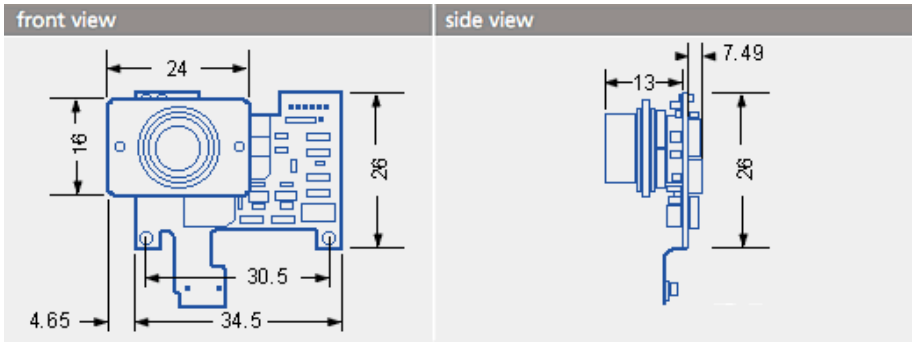


Figure 10: mvBlueFOX3-M dimensions of additional I/O board

The following figure shows, how the additional I/O board gets connected correctly.

Attention

"Short circuit"

Since the connector of the I/O board will also fit upside down, you have to be careful while connecting. Otherwise you can destroy the camera and / or the I/O board.

→ As show in the figure, if the I/O board was connected correctly, you can bend the I/O board on the back of the sensor board. Then the I/O board connector will point to the opposite direction as the sensor.



Figure 11: mvBlueFOX3-M connected I/O board

The pinning of the mvBlueFOX3-I/O is described in the chapter **Circular connector male (Power / Digital I/O)** (p. 68).

Note

There is also a version of the I/O board without connector as **"mvBlueFOX3-IO NC"** (NC = not connected). The pinning is provided in the figure:

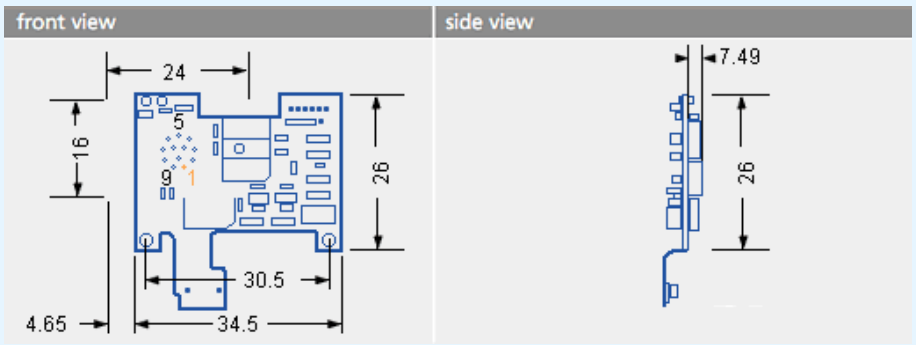


Figure 12: mvBlueFOX3-M dimensions of additional I/O board without connector.

1.9.1.4 Model without housing (mvBlueFOX3-M2)

Attention

"Broken connectors"

Handle the connectors with care otherwise you can damage the device.

→ In combination with the connectors, please limit the mechanical stress.

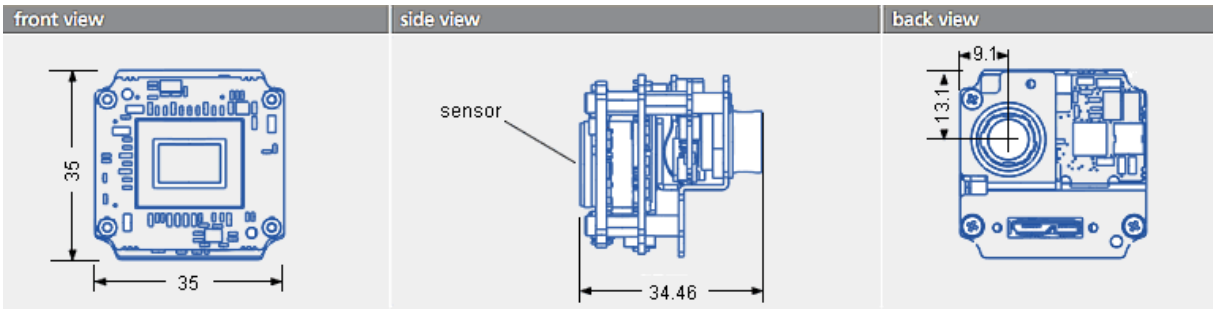


Figure 13: mvBlueFOX3-M2xxx-1xx2 dimensions and connectors

S-mount lensholder and heat sink backplate option (-4951)

The heat sink backplate is connected to the GND potential of the camera's power. The connection itself takes place via the fixing points. Both, the mounting holes on the lensholder (4x M2) as well as the mounting holes at the heat sink plate (2x M3) can be used to mount the camera.

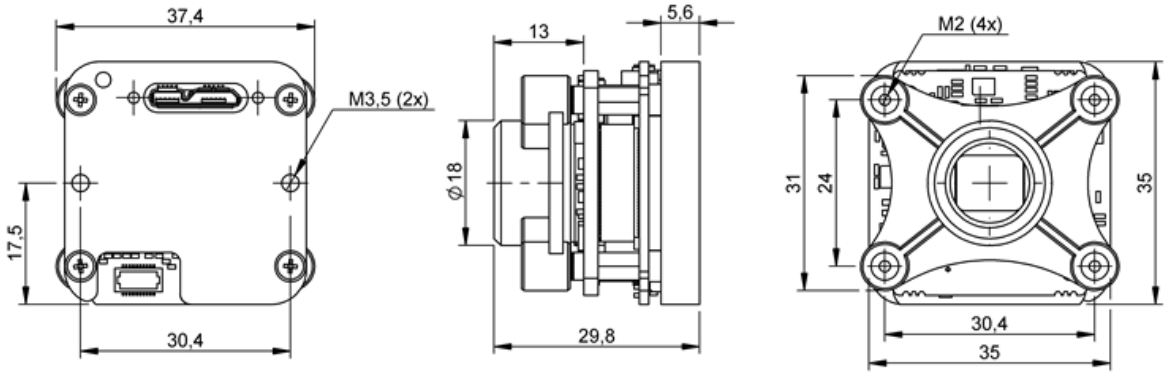


Figure 14: mvBlueFOX3-M2xxx-495x dimensions and connectors

Attention

"Overheating"

Wihtout a heat sink the device can be damaged.

→ Pay attention to the **Important Safety Notes** (p. 9) .

C-mount lensholder and heat sink backplate option (-6151)

The heat sink backplate is connected to the GND potential of the camera's power. The connection itself takes place via the fixing points. Both, the mounting holes on the lensholder (12x M4) as well as the mounting holes at the heat sink plate (2x M3) can be used to mount the camera.

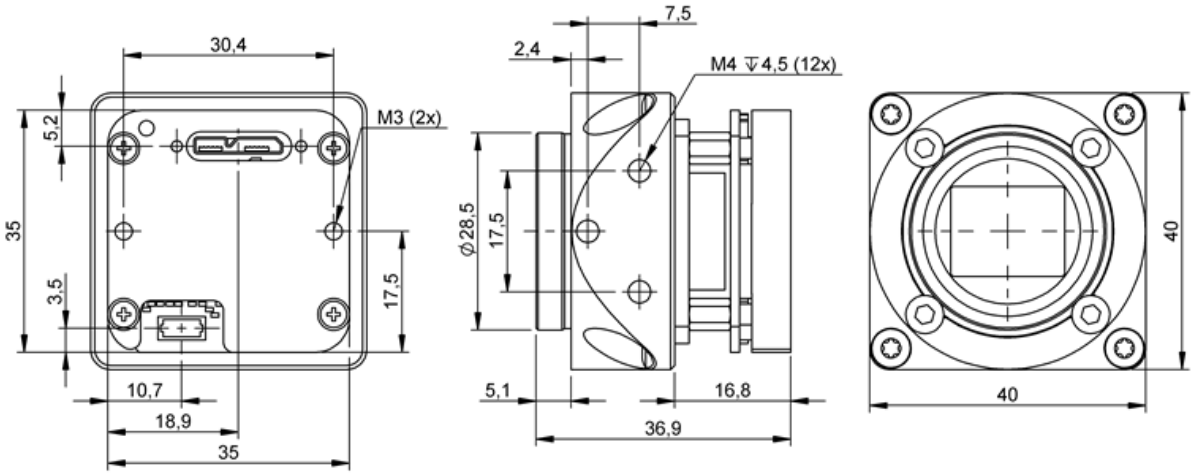


Figure 15: mvBlueFOX3-M2xxx-615x dimensions and connectors

Attention

"Overheating"

Wihtout a heat sink the device can be damaged.

→ Pay attention to the **Important Safety Notes** (p. 9) .

Lens protrusion	
X	10.7 mm with 1" lenses

	C-Mount	CS-Mount
Z	17.526 mm (in air)	12.5 mm (in air)

Fixed and compact C-mount lensholder (-7111)

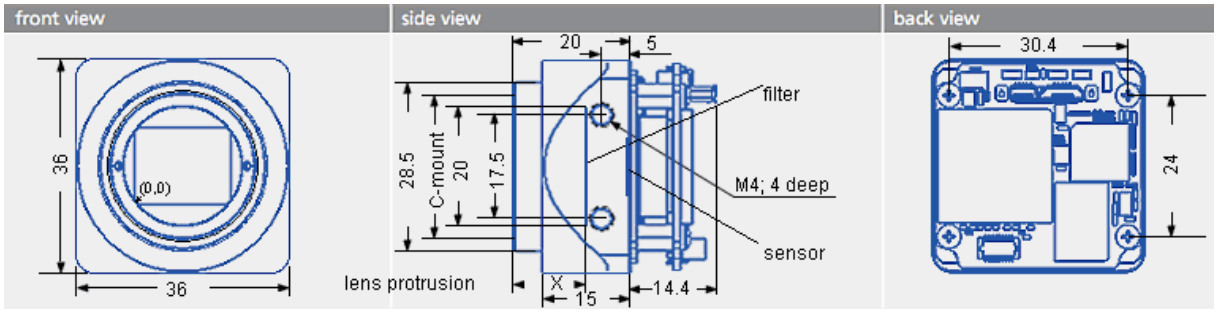


Figure 16: mvBlueFOX3-M2xxx-7111 dimensions and connectors

Lens protrusion	
X	13.1 +- 0.5

	C-Mount
Z	17.526 mm (in air)

1.9.1.5 Single-board Model for Embedded Vision (mvBlueFOX3-3M)

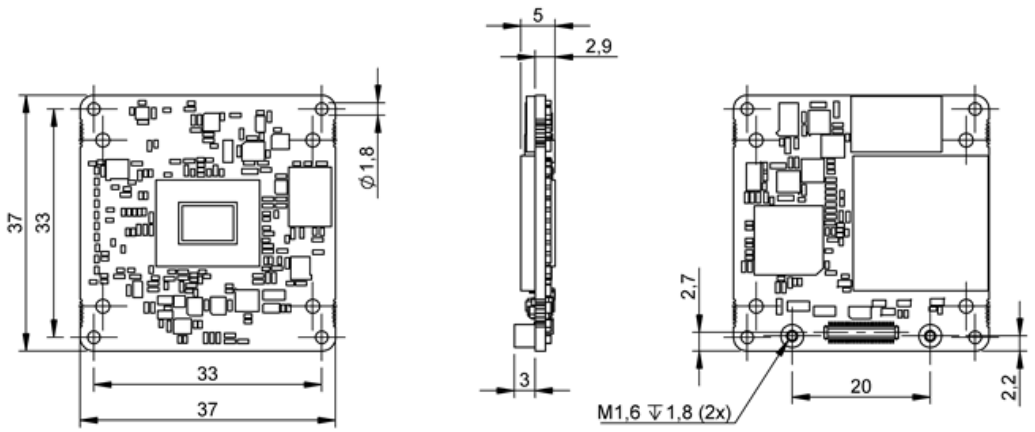


Figure 17: BF3-3M-xxxxxx-1xxx2x dimensions and connectors

See also

BFembedded interface (mvBlueFOX3-3M,mvBlueFOX3-5M) (p. 72)

1.9.1.6 Hi-res model (mvBlueFOX3-4)

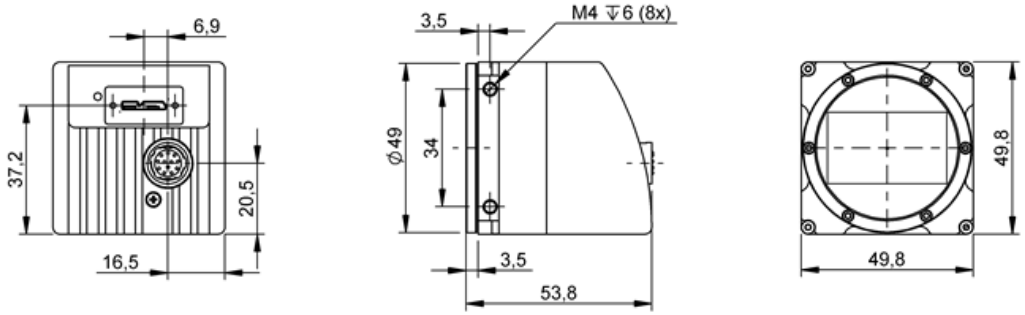


Figure 18: mvBlueFOX3-2xxx-1xxx dimensions and connectors

mvBlueFOX3-4xxx	
Size of body (w x h x l)	49.8 x 49.8 x 54.215 mm

Mounting holes

The mvBlueFOX3-4xxx provides integrated mounting holes.

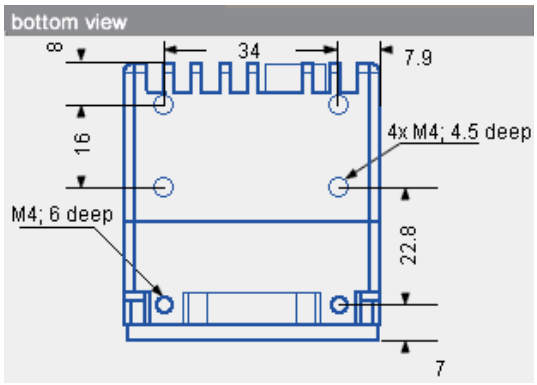


Figure 19: mvBlueFOX3-4xxx mounting holes

1.9.1.7 Board-level Model for Embedded Vision (mvBlueFOX3-5M)

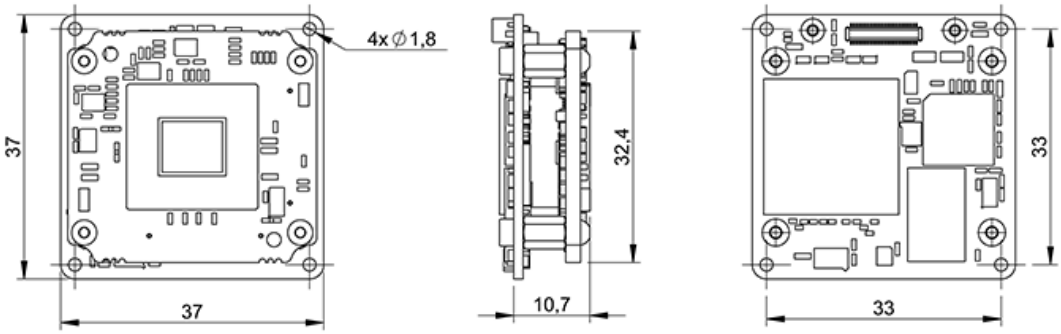


Figure 20: BF3-5M-xxxxxx-1xxx2x dimensions and connectors

Note

The red dot marks pin 1.

See also

BFembedded interface (mvBlueFOX3-3M,mvBlueFOX3-5M) (p. 72)

1.9.2 Camera interfaces (mvBlueFOX3-1,mvBlueFOX3-2,mvBlueFOX3-M1,mvBlueFOX3-M2,mvBlueFOX3-4)

1.9.2.1 Circular connector male (Power / Digital I/O)

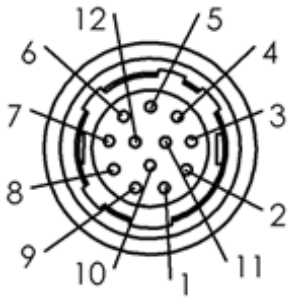


Figure 21: 12-pin (male; top view), digital I/O, power

Pin.	mvBlueFOX3-1xxx (Signal)	mvBlueFOX3-2xxx / -4xxx (Signal)	Line in wxPropView	Cable KS-BCX-HR12 color scheme
1	GND (for PWR_IN)			black
2	not connected, leave open	**PWR_IN** (p. 82)		brown
3	Opto DigOut3		Line3	red
4	Opto DigIn0		Line4	orange
5	Opto DigOut2		Line2	yellow
6	Opto DigOut0		Line0	green
7	Opto DigIn_GND			blue
8	RS232 RX			violet
9	RS232 TX			gray
10	Opto DigOut_PWR_IN			white
11	Opto DigIn1		Line5	white-black
12	Opto DigOut1		Line1	white-brown
	Main connector shield		Main connector shield	Main shield

Connector (camera side): SAMWOO SNH-10-12 (RPCB) or equivalent
Plug (matching cable plug): Hirose HR10A-10P-12S (01) or equivalent

Pinning of KS-BCX-HR12

Pin.	CON 1 mvBlueFOX3-1xxx | Signal	CON 1 mvBlueFOX3-2xxx / -4xxx | Signal	CON 2 open ended cable | Color
1	GND (for PWR_IN)		black
2	not connected	**PWR_IN** (p. 82)	brown
3	DigOut3 (**wxPropView** (p. 99) numbering: line3)		red
4	Opto DigIn0 (line4)		orange
5	DigOut2 (line2)		yellow
6	DigOut0 (line0)		green
7	Opto DigIn GND		blue
8	RS232_RX		violet

9	RS232_TX	gray
10	Opto DigOut_PWR_IN	white
11	Opto DigIn1 (line5)	white-black
12	Opto DigOut1 (line1)	white-brown
	Main connector shield	main shield

Color assignment following international code for UL wiring.

Power Supply

The mvBlueFOX3 is bus powered. Anyway, it is possible to power the mvBlueFOX3-2 externally with following specs:

- Input voltage range:
  - 12 .. 24 V DC (typical)
  - min. 10V
  - max. 28V
- The power supply is protected against
  - burst (EN 61000-4-4)
  - surge (EN 61000-4-5) and
  - polarity inversion
- internal short circuit protection by 1.5 A slow blow fuse

The USB power cannot be accessed via the I/O connector (this is prevented by a diode).

Note

The mvBlueFOX3-2 will reboot whenever you connect or disconnect the power at pin 2.

Attention

For mvBlueFOX3-4 an external power supply is obligatory (USB power supply may exceed 900 mA).

1.9.2.2 Characteristics of the digital inputs

Electrical characteristics

Delay

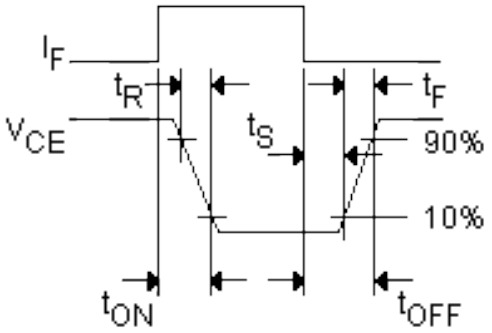


Figure 22: Input switching times

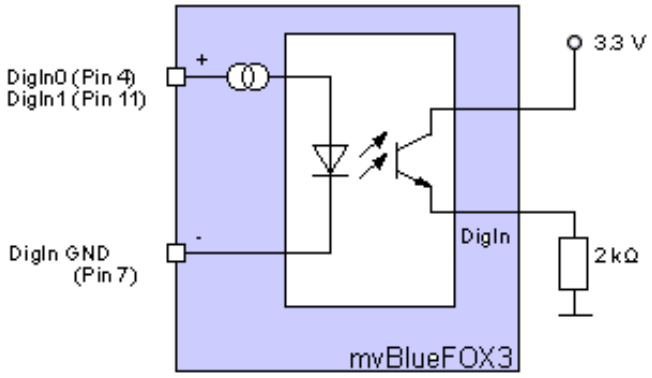
	Standard
Notes	
High level	+3 to +24 V (max. 30 V)
Low level	0 V (min. -30 V) to +0.7 V
Threshold (Low --> High / High --> Low)	2 V +- 1 V
I_{max}	5 mA
	

Figure 23: Opto DigIn mvBlueFOX3-1/-2/-M2/-4

Switching characteristics

Characteristics	Symbol	Test conditions	Typ.	Unit
Minimum trigger pulse width			5	us
Turn-On time	t_{ON}		10	
Storage time	t_S	R = 2 kOhm (Figure 7), internal output voltage 5 V, I_F = 16 mA	25	
Turn-Off time	t_{OFF}		40	

1.9.2.3 Characteristics of the digital outputs

Electrical characteristics

	Comment	Min.	Typ.	Max.	Unit
I_C	load current			15	mA
$V_{CE(sat)}$	@ I_C = 7 mA		0.4		V
V_{OUT}	Output Voltage			30	V

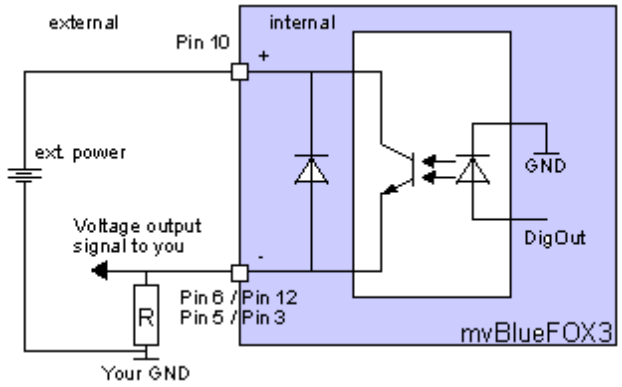


Figure 24: Opto DigOut mvBlueFOX3-1/-2/-M2/-4

Switching characteristics

Characteristics	Symbol	Test conditions	Typ.	Unit
Turn-On time	t_{ON}	R_L = 100 Ohm, V_{CC} 10 V, I_C = 2 mA	3	us
Storage time	t_S		3	
Turn-Off time	t_{OFF}		3	
Turn-On time	t_{ON}	R_L = 1.9 kOhm, V_{CC} 5 V, I_C = 16 mA	10	us
Storage time	t_S		25	
Turn-Off time	t_{OFF}		40	

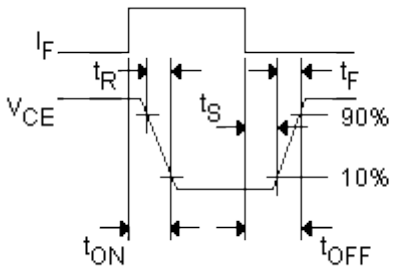


Figure 25: Switching time

1.9.3 Status / Power LED

1.9.3.1 Standard model (mvBlueFOX3-1)

States	Description
1. Off	No power or no bootloader found.
2. Red	Bootloader was recognized and FPGA is booting-up or device is in standby mode.
3. Green	mvBlueFOX3 is running.
4. Green blink	mvBlueFOX3 is busy (e.g. file upload).

1.9.3.2 Standard model (mvBlueFOX3-2)

States	Description
1. Off	No power or no bootloader found.
2. White	Bootloader was recognized and FPGA is booting-up.
3. Yellow	mvBlueFOX3 is running.
4. Green	mvBlueFOX3 is streaming images.
5. Yellow blink	mvBlueFOX3 is busy (e.g. file upload).
6. White blink	Waiting for USB connection (external power is connected)
7. Red	Error or if you put the device into standby

1.9.4 BFembedded interface (mvBlueFOX3-3M,mvBlueFOX3-5M)

The BFembedded interface provides USB3.2 Gen.1 - SuperSpeed (5Gbps) including USB2.0 - Hi-Speed (480Mbps) and various IO functionality on a single 48-pin Board-to-Board connector. The main features of the user I/O interface are

- 4 digital inputs,
- 4 digital outputs,
- a UART interface for serial communication, and
- a I2C two wire serial interface.

Attention

"Disconnections"

When using **mvBlueFOX3-3M** (p. 66) or **mvBlueFOX3-5M** (p. 67) with the rigid-flex extension cable (BFE-FLEX), please ensure **you have connected it correctly** (p. 9).

"Camera connector"

- Hirose DF40GB-48DP-0.4V

"Mating Connector"

Used for accessory IO Boards or customer implementation.

- Hirose DF40GB(3.0)-48DS-0.4V

"Mechanical characteristics"

- 48-pin, 0.4 mm pitch, shielded, stacking height: 3.0 mm
- 2 steel spacers (height 3mm). Thread M1.6, max. screw depth 1.8 mm.

1.9.4.1 Pin assignment

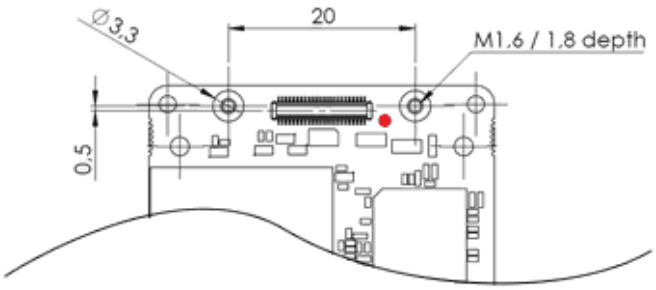


Figure 26: BFembedded interface - pin 1

Note

The red dot marks pin 1 of Hirose DF40GB-48DP-0.4V.

Pin	Signal	Description
1	VBUS_5V	USB Power
2	VBUS_5V	USB Power
3	VBUS_5V	USB Power
4	POWER_DOWN_N	- Complete power shut down • Pull to GND for power down • Internal 10K pull up to VBUS_5V
5	VBUS_5V	USB Power
6	VAUX_PRESENT↩ _N	- External power supply indication • Connect to VBUS if USB supplied • Otherwise leave open
7	D-	USB 2.0 Data - (differential)
8	-	Do not connect, internal use
9	D+	USB 2.0 Data + (differential)
10	-	Do not connect, internal use
11	ID	OTG-Identification (OTG = On-The-Go)
12	-	Do not connect, internal use
13	GND	Ground
14	-	Do not connect, internal use
15	GND	Ground
16	GND	Ground
17	GND	Ground
18	GND	Ground
19	SSTX-	USB3 Super Speed Transmitter (differential), polarity inversion allowed
20	-	Do not connect, internal use
21	SSTX+	USB3 Super Speed Transmitter (differential), polarity inversion allowed
22	-	Do not connect, internal use
23	GND	Ground

24	-	Do not connect, internal use
25	SSRX+	USB3 Super Speed Receiver (differential), polarity inversion allowed
26	-	Do not connect, internal use
27	SSRX-	USB3 Super Speed Receiver (differential), polarity inversion allowed
28	-	Do not connect, internal use
29	GND	Ground
30	-	Do not connect, internal use
31	GND	Ground
32	GND	Ground
33	GND	Ground
34	GND	Ground
35	DigOut0	Digital Output (with level shifter), voltage reference VCC_IO
36	VCC_IO	I/O voltage reference input 1.8 V...5.0 V, abs. min max: 1.65 V...5.5 V **Note** Leave not unconnected! Connect to VBUS_5V if I/O not used.
37	DigOut1	Digital Output (with level shifter), voltage reference VCC_IO
38	DigIn2	Digital Input (with level shifter), voltage reference VCC_IO
39	DigOut2	Digital Output (with level shifter), voltage reference VCC_IO
40	DigIn3	Digital Input (with level shifter), voltage reference VCC_IO
41	DigOut3	Digital Output (with level shifter), voltage reference VCC_IO
42	UART_RX	Serial interface (see details below), LVCMOS 3.3 V IO level
43	DigIn0	Digital Input (with level shifter), voltage reference VCC_IO
44	UART_TX	Serial interface (see details below), LVCMOS 3.3 V IO level
45	DigIn1	Digital Input (with level shifter), voltage reference VCC_IO
46	I2C_SCL	I2C two wire serial bus (see details below), LVCMOS 3.3 V IO level
47	-	Do not connect, internal use
48	I2C_SDA	I2C two wire serial bus (see details below), LVCMOS 3.3 V IO level

Note

Additional shield pins are connected to GND.

Electrical characteristics of signals

USB Power

Signal	Parameter	min	nom	max	Unit
VBUS_5V		4.45	5	5.25	V
I_{VBUS_5V*}				900	mA
I per pin				300	mA

*Limit for USB3 high-power SuperSpeed devices (Power consumption of the camera will be within this limit).

The rated current limit for the connector is 300 mA per pin and therefore the BF3-Embedded Interface would allow up to 1200 mA.

VCC_IO / DIGIN / DIGOUT

Signal	Parameter	min	nom	max	Unit
VCC_IO	I/O voltage power	1.65		5.5	V
$U_{DIG_IN_LOW}$	V_{IL} (low level input voltage)			VCC_IO x 0.3	V
$U_{DIG_IN_HIGH}$	V_{IH} (high level input voltage)	VCC_IO x 0.7			V
I_{OH}	High-level output current VCC_IO: 4.5 V to 5.5 V			-32	mA
I_{OL}	Low-level output current VCC_IO: 4.5 V to 5.5 V			32	mA
$U_{DIG_OUT_HIGH}$	Digital output (VCC_IO = 4.5 V / I_{OUT}= -32 mA)	3.8			V
$U_{DIG_OUT_LOW}$	Digital output (VCC_IO = 4.5 V / I_{OUT}= 32 mA)			0.55	V

Digital I/Os include level shifters to allow customized IO Levels for both inputs and outputs. VCC_IO (Pin 36) is the input for the user defined IO voltage.

Note

If no I/O functionality is needed, connect VCC_IO to VBUS_5V.

For other output characteristics, see datasheet SN74LVC2T45.

Serial Interface, I2C and Power Down

Signal	Parameters / Properties	min	nom	max	Unit
POWER_DOWN_N	Complete power shut down, internal 10K pull up to VBUS_5V				
	POWER_DOWN_N Input Low Voltage	0		0.75	V
	POWER_DOWN_N Input High Voltage	1.↩ 8		5	V
UART_RX / UART_TX	Voltage level • The camera provides a UART Interface for serial communication (internally protected by 100 Ohm series resistor)		3.3	3.4	V
I2C_SCL / I2C_SDA	Voltage level, internal 2K pull up to 3.3 V • Camera provides a I2C master to control devices connected to the bus • Interface clock rate 400 kHz • Reserved I2C addresses (8 Bit) of shared devices: 0x0, 0x30...0x36, 0x48, 0x60...0x66, 0x↩ A0...0xA6, 0xB0...0xBE, 0xF8 • Access to the I2C interface has to be enabled • By default the access to the I2C bus is disabled to prevent unintended interference		3.3	3.4	V

1.9.4.2 Boards for the BFembedded interface (mvBlueFOX3-3M,mvBlueFOX3-5M)

1.9.4.2.1 USB Micro B (horizontal) The board includes a USB3 Micro B connector (CON3) in horizontal alignment, a 12-pin header (CON2) for IO signals and a 3-pin header (CON4) for I2C, weight 2g.

"Order Code": BFE-IF-MICUSB3B-H-IO12

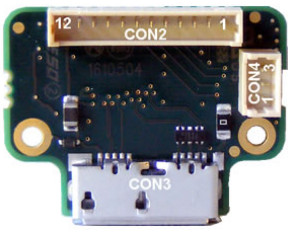


Figure 27: Connectors of the USB3 Micro B board (horizontal)

1.9.4.2.2 USB Micro B (vertical) The board includes a USB3 Micro B connector (CON3) in vertical alignment, a 12-pin header (CON2) for IO signals and a 3-pin header (CON4) for I2C, weight 2g.

"Order Code": BFE-IF-MICUSB3B-V-IO12

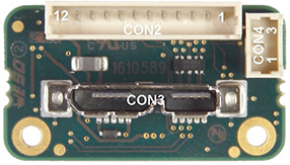


Figure 28: Connectors of the USB3 Micro B board (vertical)

1.9.4.2.3 USB Type C The board includes a USB Type C connector (CON3) in horizontal alignment, a 12-pin header (CON2) for IO signals and a 3-pin header (CON4) for I2C, weight 2g.

"Order Code": BFE-IF-USB3C-H-IO12

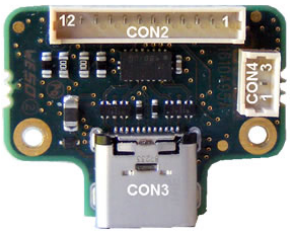


Figure 29: Connectors of the USB Type C board

1.9.4.2.4 Pin assignments 12-pin I/O connector (CON2)

- Part type:
  - Wire-to-Bord Connector 1.25 mm, WUERTH_653012114822 or MOLEX PicoBlade 53047_1210
- Connecting part:
  - Würth: 653 012 113 322 1.25 MM female terminal housing with female crimp terminal WR-WTB 653 001 137 22 or
  - Molex 0510211200, 1.25 Wire to Board Connection Receptacle Housing, Applicable Terminal 50058-8000, 50079-8000 Series

Pin	I/O	Signal	Description
1	Out	DigOut0	Digital Output
2	Out	DigOut1	Digital Output
3	Out	DigOut2	Digital Output
4	Out	DigOut3	Digital Output
5	In	DigIn0	Digital Input
6	In	DigIn1	Digital Input
7	In	DigIn2	Digital Input
8	In	DigIn3	Digital Input
9	Out	RS232_TXD	Serial Interface RS232_TXD
10	In	RS232_RXD	Serial Interface RS232_RXD
11	GND	GND	Ground
12	POWER_OUT	VBUS_OUT	Directly connected to USB Power

"Electrical characteristics of signals"

"VBUS_OUT / DIGIN / DIGOUT"

Signal	Parameter	min	nom	max	Unit
VBUS_OUT		4.45	5.00	5.25	V
I_{VBUS_OUT}				10	mA
$U_{DIG_IN_LOW}$	V_{IL} (low level inputvoltage)			0.3	V
$U_{DIG_IN_HIGH}$	V_{IH} (high level inputvoltage)	0.7		5.5	V
I_{OH}	High-level output current			-32	mA
I_{OL}	Low-level output current			32	mA
$U_{DIG_OUT_HIGH}$	Digital output (I_{OUT}= -32 mA)	3.8			V
$U_{DIG_OUT_LOW}$	Digital output (I_{OUT}= 32 mA)			0.45	V

"Serial interface RS232"

Signal	Test Properties	min	nom	max	Unit
EIA/TIA-232E Input Voltage Range*		-30		+30	V
EIA/TIA-232E Input Threshold Low		0.6	1.3		V
EIA/TIA-232E Input Threshold High			1.6	2.4	V
EIA/TIA-232E Input Hysteresis			0.4		mA
EIA/TIA-232E Input Resistance		3	5	7	V
Output Voltage Swing		±5.↩ 0	±5.↩ 7		V
RS-232 Output Short-Circuit Current			±15		mA
ESD PROTECTION	Human body model air discharge Human body model contact discharge		±15 ±8		kV kV
TIMING CHARACTERISTICS Maximum Data Rate		460			kbps

* Guaranteed by design.

USB Micro B Connector (CON3)

- Standard USB3.0 Micro-B Connector

- Part Type: WUERTH_ 692 622 030 100

Pin	I/O	Signal	Description
1	Power_IN	VBUS_IN	Digital Output
2	In/Out	D-	USB 2.0 Data −(differential)
3	In/Out	D+	USB 2.0 Data +(differential)
4	IN	ID	OTG-Identification (OTG= On-The-Go)
5	GND	GND	Ground to Pin 1
6	In/Out	SSTX-	Super Speed Transmitter(differential)
7	In/Out	SSTX+	Super Speed Transmitter(differential)
8	GND	GND	Ground for Super SpeedSignale
9	In/Out	SSRX+	Super Speed Receiver(differential)
10	In/Out	SSRX-	Super Speed Receiver (differential)
Shell		Shield	Shield

"Electrical characteristics of signals"

Signal	Parameter	min	nom	max	Unit
VBUS_IN		4.45	5.00	5.25	V
I_{VBUS_IN}				900	mA

USB Type C Connector (CON3)

- Standard USB3.2 Gen.1 TYPE-C, Connector Type: JAE_DX07S024JJ2

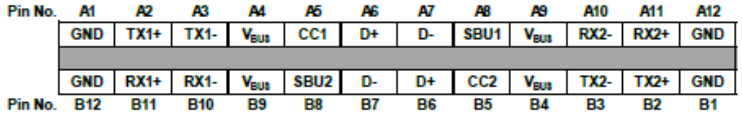


Figure 30: Pinning USB Type C

Pin	I/O	Signal	Description
A1	GND	GND	Ground for Super Speed Signals
A2	In/Out	SSTX1+	Super Speed Transmitter (differential)
A3	In/Out	SSTX1-	Super Speed Transmitter (differential)
A4	Power_IN	VBUS_IN	Digital Output
A5	IN	CC1	Configuration Channel
A6	In/Out	D+	USB 2.0 Data + (differential)
A7	In/Out	D-	USB 2.0 Data - (differential)
A8	–	NC	Sideband Use -> Alternate Mode
A9	Power_IN	VBUS_IN	Digital Output
A10	In/Out	SSRX2-	Super Speed Receiver (differential)
A11	In/Out	SSRX2+	Super Speed Receiver (differential)
A12	GND	GND	Ground for Super Speed Signals
B1	GND	GND	Ground for Super Speed Signals
B2	In/Out	SSTX1+	Super Speed Transmitter (differential)

B3	In/Out	SSTX1-	Super Speed Transmitter (differential)
B4	Power_IN	VBUS_IN	Digital Output
B5	IN	CC1	Configuration Channel
B6	In/Out	D+	USB 2.0 Data + (differential)
B7	In/Out	D-	USB 2.0 Data - (differential)
B8	–	NC	Sideband Use > Alternate Mode
B9	Power_IN	VBUS_IN	Digital Output
B10	In/Out	SSRX1-	Super Speed Receiver (differential)
B11	In/Out	SSRX1+	Super Speed Receiver (differential)
B12	GND	GND	Ground for Super Speed Signals
Shell	Shield	GND	Shield

"Electrical characteristics of signals"

Signal	Parameter	min	nom	max	Unit
VBUS_IN		4.45	5.00	5.25	V
I_{VBUS_IN}				900	mA

3-pin I2C connector (CON4)

- Part type:
  - Wire to Bord Connector 1.25 mm, WUERTH_653003114822 or MOLEX PicoBlade 53047_0310
- Mating part:
  - Würth: 653 003 113 322 1.25 MM female terminal housing with female crimp terminal WR-WTB 653 001 137 22 or
  - Molex 0510210300, 1.25 Wire to Board Connection Receptacle Housing, Applicable Terminal 50058-8000, 50079-8000 Series

Pin	I/O	Signal	Description
1	-	GND	Camera GND
2	Out	SCL	I2C Clock
3	Bi	SDA	I2C data

"Electrical characteristics of signals"

- Camera provides a I2C master to control devices connected to the bus
- Interface clock rate 400kHz
- Reserved I2C addresses (8Bit) of shared devices: 0x0, 0x30...0x36, 0x48, 0x60...0x66, 0xA0...0xA6, 0x↩ B0...0xBE, 0xF8
- Access to the I2C interface has to be enabled
- By default the access to the I2C bus is disabled to prevent unintended interference

See also

API documentation: `https://www.matrix-vision.com/manuals/SDK_NET/classmv_1↩ _1impact_1_1acquire_1_1GenICam_1_1mvI2cInterfaceControl.html`

Signal	Parameters / Properties	min	nom	max	Unit
I2C_SCL / I2C_SDA	Voltage level, internal 2K pull up to 3.3 V		3.3	3.4	V

1.9.5 Components

Features		mv↩ Blue↩ FOX3	mv↩ Blue↩ FOX3-↩ M1	mv↩ Blue↩ FOX3-2	mv↩ Blue↩ FOX3-3M	mv↩ Blue↩ FOX3-↩ M2	mv↩ Blue↩ FOX3-4	mv↩ Blue↩ FOX3-5M
Interface		USB 3.2 Gen 1 / USB 2.0 (up to 5 Gbit/s / up to 480 Mbit/s)						
Image Memory		256 MBytes						
Digital Inputs		2	as an option	2	4	as an option	2	4
	Type	opto-isolated with current limiters		opto-isolated with current limiters	CMOS/↩ TTL with adjustable I/O level		opto-isolated with current limiters	CMOS/↩ TTL with adjustable I/O level
Digital Outputs		4	as an option	4	4	as an option	4	4
	Type	opto-isolated		opto-isolated	CMOS/↩ TTL with adjustable I/O level		opto-isolated	CMOS/↩ TTL with adjustable I/O level
Lens Mount (Focal Distance)		C-mount (17.526 mm in air), CS-mount (12.526 mm in air)	C-mount (17.526 mm in air)	C-mount (17.526 mm in air), CS-mount (12.526 mm in air)	S-mount	C-mount (17.526 mm in air)	several options available	several options available
Environment	Ambient Temperature							
	Operation	0..45 deg C / 30 to 80% RH [1]						
	Storage	-20..60 deg C / 20 to 90% RH						
	Protection class [2]	IP30	-	IP30	-	-	IP30	-
Weight without lens		approx. 58.5 g	without lensholder, lens, I/O: approx. 7 g	approx. 94 g (option -2xxx) & approx. 110 g (option -1xxx)	approx. 9 g (base module)	without lensholder, lens, I/O: 21 g I/O board: 14 g lensholder (-4xxx): 6 g heat sink plate (-xx51): 16 g	approx. 175 g	approx. 17 g (camera board)

Power supply (PWR_↩ IN)								
	Consumption via Vbus	< 4.5 W					< 4.8 W	< 4.5 W
	Option 10 to 28 V			DC 10 to 28 V Pmax 5.5 W			DC 10 to 28 V Pmax 5.5 W	

[1] for board level cameras follow cooling recommendations in **"mvBlueFOX3-3M** / **-5M: Cooling instructions"** and **"mvBlueFOX3-M2xxx: Cooling instructions"** which are part of the scope of delivery
[2] not evaluated by UL

1.10 Sensor Overview

1.10.1 Image data flow

The following block diagrams show the data flow of the image data after being read from the sensor chip in the camera.

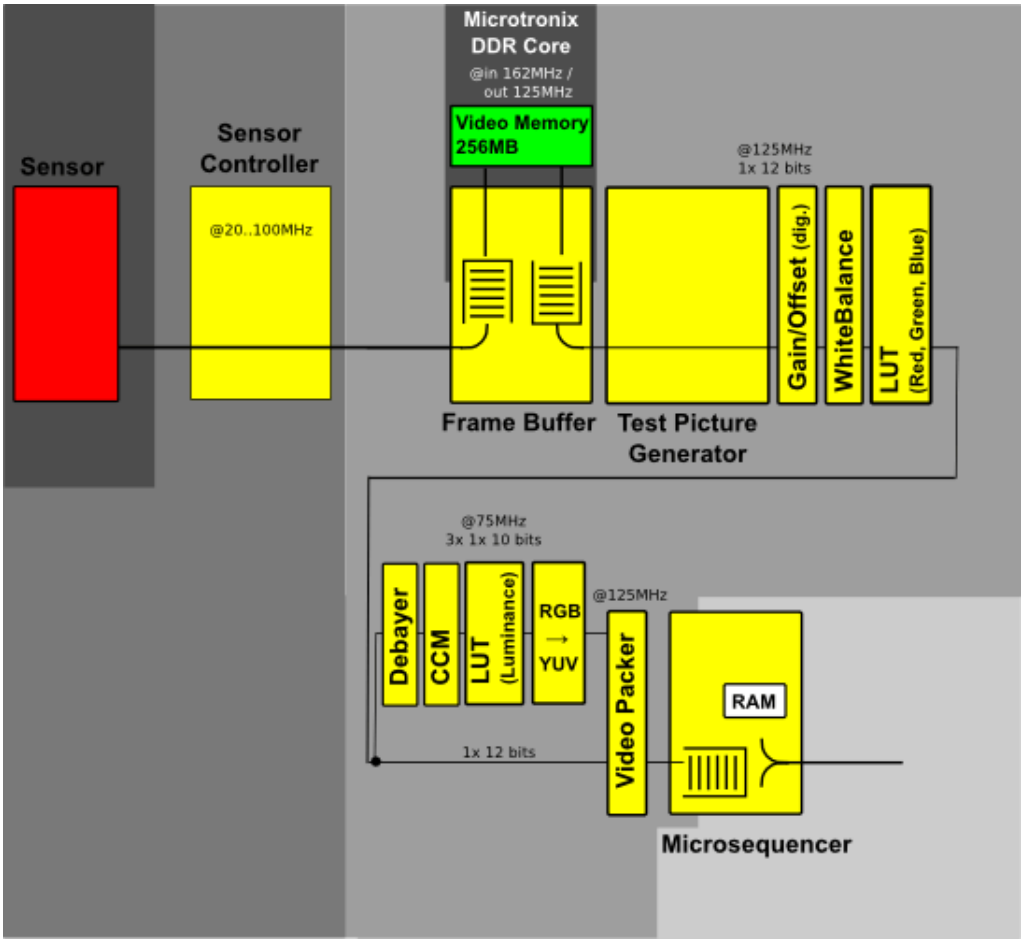


Figure 1: Block diagram

1.10.2 Output sequence of color sensors (RGB Bayer)

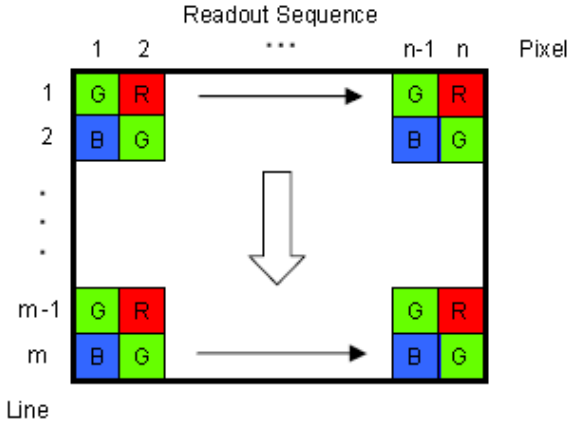


Figure 2: Output sequence of RAW data

1.10.3 Bilinear interpolation of color sensors (RGB Bayer)

For Bayer demosaicing in the camera, we use bilinear interpolation:

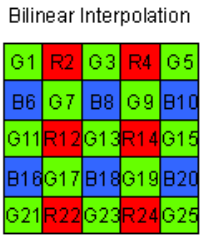


Figure 5: Bilinear interpolation

1. Interpolation of green pixels: the average of the upper, lower, left and right pixel values is assigned as the G value of the interpolated pixel.
   For example:

$$G8 = \frac{(G3+G7+G9+G13)}{4}$$

   For G7:

$$G7_{new} = 0.5 * G7 + 0.5 * \frac{(G1+G3+G11+G13)}{4}$$

2. Interpolation of red/blue pixels:
   Interpolation of a red/blue pixel at a green position: the average of two adjacent pixel values in corresponding color is assigned to the interpolated pixel.
   For example:

$$B7 = \frac{(B6+B8)}{2} \; ; \; R7 = \frac{(R2+R12)}{2}$$

   Interpolation of a red/blue pixel at a blue/red position: the average of four adjacent diagonal pixel values is assigned to the interpolated pixel.
   For example:

$$R8 = \frac{(R2+R4+R12+R14)}{4} \; ; \; B12 = \frac{(B6+B8+B16+B18)}{4}$$

Any colored edge which might appear is due to Bayer false color artifacts.

Note

There are more advanced and adaptive methods (like edge sensitive ones) available if the host is doing this debayering.

1.10.4 CMOS sensors

1.10.4.1 Details of operation

The CMOS sensors offer two different modes of operation:

- Free running mode (Overlapping integration and readout)
- Snapshot mode (Sequential integration and readout)

1.10.4.1.1 Free running mode In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, integration and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode `Continuous`.

1.10.4.1.2 Snapshot mode In snapshot mode, the image acquisition process consists off several sequential phases:

- Trigger
- Erase, exposure and readout

1.10.4.1.2.1 Trigger Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The CMOS sensors support the following trigger modes:

Description	Setting in GenICam
Free running, no external trigger signal needed (formerly known as **Continuous**).	`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`

Image acquisition triggered by command (software trigger; formerly known as **OnDemand**).	`"TriggerSelector = FrameStart"`
	`"TriggerMode = On"`
	`"TriggerSource = Software"`
	`"ExposureMode = Timed"`
	
	To trigger one frame execute the *TriggerSoftware@i* command then.

Start an exposure of a frame as long as the trigger input is below the trigger threshold (formerly known as **OnLowLevel**).	`"TriggerSelector = Acquisition↩ Active"`
	`"TriggerMode = On"`
	`"TriggerSource = <desired Line>"`
	`"TriggerActivation = LevelLow"`
	`"ExposureMode = Timed"`

Start an exposure of a frame as long as the trigger input is above the trigger threshold (formerly known as **OnHighLevel**).	`"TriggerSelector = Acquisition↩ Active"`
	`"TriggerMode = On"`
	`"TriggerSource = <desired Line>"`
	`"TriggerActivation = LevelHigh"`
	`"ExposureMode = Timed"`

If an external trigger signal occurs (e.g. high or low), the sensor will start to expose and readout one image. Now, if the trigger signal is still available, the sensor will start to expose and readout the next image (see figure 12, upper part). This will lead to an acquisition just like using continuous trigger.

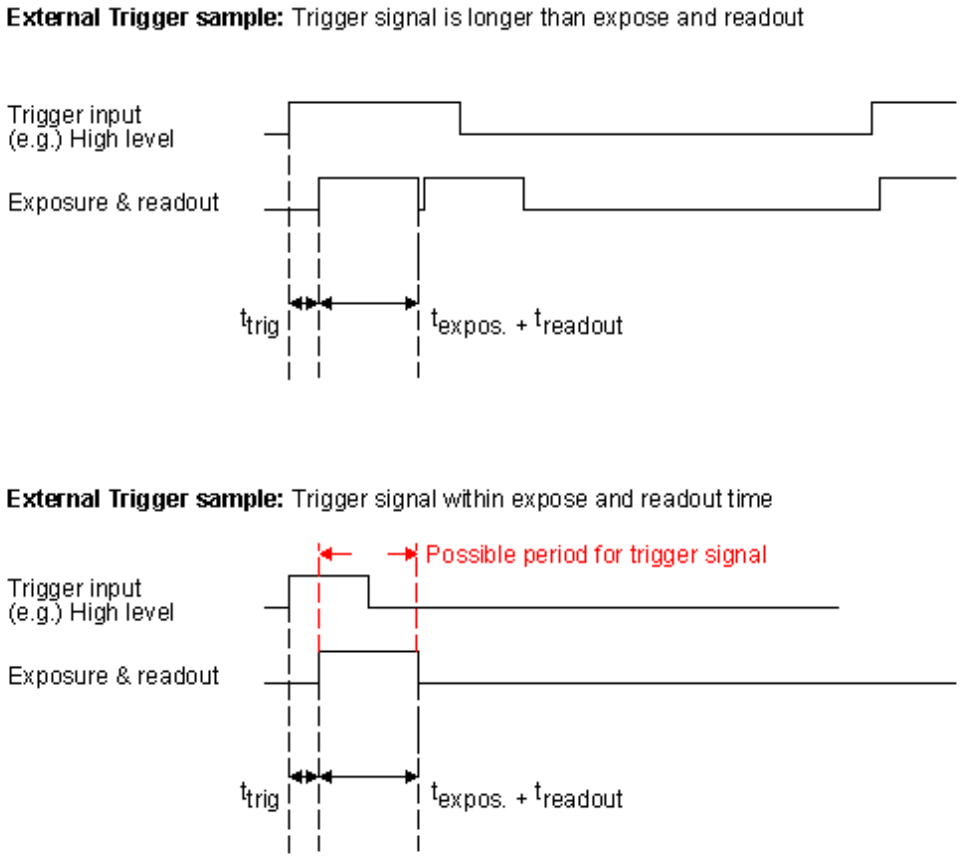


Figure 3: External Trigger with CMOS sensors

If you want to avoid this effect, you have to adjust the trigger signal. As you can see in figure 3 (lower part), the possible period is small.

1.10.4.1.2.2 Example External synchronized image acquisition (high active)

- Trigger modes
  - *OnHighLevel:*
    The high level of the trigger has to be shorter than the frame time. In this case, the sensor will make one image exactly. If the high time is longer, there will be images with the possible frequency of the sensor as long as the high level takes. The first image will start with the low-high edge of the signal. The integration time of the exposure register will be used.
  - *OnLowLevel:*
    The first image will start with the high-low edge of the signal.

1.10.4.1.2.3 Erase, exposure and readout All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after integration.

1.10.4.2 Models

The CMOS sensor modules incorporate the following features:

Sony Pregius S

$>$ 5 Mpix $<$ 10 Mpix

Sensors	5.1 Mpix (-2051d)	8.1 Mpix (-2081a)
Sensor supplier	Sony	Sony
Sensor name	IMX547	IMX546
Res.	2472 x 2064 gray scale / RGB	2856 x 2848 gray scale / RGB
Sensor size	1/1.8"	2/3"
Max. FPS (in free-running full frame mode)	74.0	46.7
Frame rate exactness (p. 104)	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	40.5	40.4
DR (normal / **HDR** (p. 302)) [dB][1/]	70.3 /	70.2 /
Rolling shutter	-	-
Global shutter	X	X
Global Reset	-	-
Trigger (HW / SW)	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	X	X
Linescan mode	-	-
High color reproductivity (for color version)	X	X
Power consumption (since FW 2.↩ 5.146) [W]	approx. 2.7	approx. 2.7
More specific data	**mvBlueFOX3-2051d / BF3-5M-0051D (5.1 Mpix [2472 x 2064])** (p. 355)	**mvBlueFOX3-2081a / BF3-5M-0081A (8.1 Mpix [2856 x 2848])** (p. 359)

> 10 Mpix < 20 Mpix

Sensors	12.4 Mpix (-2124d)	16.2 Mpix (-2162)
Sensor supplier	Sony	Sony
Sensor name	IMX545	IMX542
Res.	4128 x 3008 gray scale / RGB	5328 x 3040 gray scale / RGB
Sensor size	1/1.1"	1.1"
Max. FPS (in free-running full frame mode)	30.6	23.4
Frame rate exactness (p. 104)	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	40.3	39.8
DR (normal / **HDR** (p. 302)) [dB][1/]	70.1	70.6
Rolling shutter	-	-
Global shutter	X	X
Global Reset	-	-
Trigger (HW / SW)	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	X	X
Linescan mode	-	-
High color reproductivity (for color version)	X	X

Power consumption (since FW 2.↩ 5.146) [W]	approx. 2.7	approx. 3.0
More specific data	**mvBlueFOX3-2124d / BF3-5M-0124D (12.4 Mpix [4128 x 3008])** (p. 362)	**mvBlueFOX3-2162 / BF3-5M-0162A (16.2 Mpix [5328 x 3040])** (p. 366)

> 20 Mpix

Sensors	20.4 Mpix (-2204)	24.6 Mpix (-2246)
Sensor supplier	Sony	Sony
Sensor name	IMX541	IMX540
Res.	4512 x 4512 gray scale / RGB	5328 x 4608 gray scale / RGB
Sensor size	1.1"	1.2"
Max. FPS (in free-running full frame mode)	18.7	15.5
Frame rate exactness (p. 104)	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	39.8	39.7
DR (normal / **HDR** (p. 302)) [dB][1/]	70.7	70.4
Rolling shutter	-	-
Global shutter	X	X
Global Reset	-	-
Trigger (HW / SW)	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	X	X
Linescan mode	-	-
High color reproductivity (for color version)	X	X
Power consumption (since FW 2.↩ 5.146) [W]	approx. 3.2	approx. 4.0
More specific data	**mvBlueFOX3-2204 / BF3-5M-0204A (20.5 Mpix [4512 x 4512])** (p. 369)	**mvBlueFOX3-2246 / BF3-5M-0246A (24.6 Mpix [5328 x 4608])** (p. 373)

Sony Pregius

< 5 Mpix

Sensors	0.4 Mpix (-2004)	1.6 Mpix (-2016)	1.7 Mpix (-2017)	2.4 Mpix (-2024)	2.4 Mpix (-2024a)	2.4 Mpix (-0024b)	3.2 Mpix (-2032)	3.2 Mpix (-2032a)
Sensor supplier	Sony	Sony	Sony	Sony	Sony	Sony	Sony	Sony
Sensor name	IMX287	IMX273	IMX425	IMX174	IMX249	IMX392	IMX252	IMX265
Res.	728 x 544 gray scale / RGB	1456 x 1088 gray scale / RGB	1600 x 1104 gray scale / RGB	1936 x 1216 gray scale / RGB	1936 x 1216 gray scale / RGB	1936 x 1216 gray scale / RGB	2064 x 1544 gray scale / RGB	2064 x 1544 gray scale / RGB

Sensor size	1/2.9"	1/2.9"	1/1"	1/1.2"	1/1.2"	1/2.3"	1/1.8"	1/1.8"
Max. FPS (in free-running full frame mode)	436.9	226.5	212.9	164	166	46.9	123	55
Frame rate exactness (p. 104)	-	-	-	-	-	-	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	43.3	40.2	49.9	45.1	45.1	40.2	40.3	40.2
DR (normal / **HDR** (p. 302)) [dB][1/]	74.2 /	71.4 /	72.4 /	66.4 /	73.0 /	71.4 /	71.1 /	71.3 /
Rolling shutter	-	-	-	-	-	-	-	-
Global shutter	X	X	X	X	X	X	X	X
Global Reset	-	-	-	-	-	-	-	-
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	X	X	X	X	X	X	X	X
Linescan mode	-	-	-	-	-	-	-	-
High color reproductivity (for color version)	X	X	X	X	X	X	X	X
Power consumption (since FW 2.↩ 5.146) [W]	approx. 3.2	approx. 3.5	approx. 4.5	approx. 3.4	approx. 2.8	approx. 2.8	approx. 3.6	approx. 3.0

More specific data	**mv Blue FOX3-2004 / BF3-5 M-0004F (0.4 Mpix [728 x 544])** (p. 377)	**mv Blue FOX3-2016 / BF3-5 M-0016Z (1.6 Mpix [1456 x 1088])** (p. 381)	**mv Blue FOX3-2017 / BF3-5 M-0017Z (1.8 Mpix [1600 x 1104])** (p. 384)	**mv Blue FOX3-2024 / BF3-5M-0024ZG (2.4 Mpix [1936 x 1216])** (p. 387)	**mv Blue FOX3-2024a / BF3-5 M-0024A (2.4 Mpix [1936 x 1216])** (p. 391)	**BF3-5 M-0024B (2.4 Mpix [1936 x 1216])** (p. 395)	**mv Blue FOX3-2032 / BF3-5 M-0032Z (3.2 Mpix [2064 x 1544])** (p. 398)	**mv Blue FOX3-2032a / BF3-5 M-0032A (3.2 Mpix [2064 x 1544])** (p. 402)

> 5 Mpix < 10 Mpix

Sensors	5.1 Mpix (-2051)	5.1 Mpix (-2051a)	7.1 Mpix (-2071)	7.1 Mpix (-2071a)	8.9 Mpix (-2089)	8.9 Mpix (-2089a)
Sensor supplier	Sony	Sony	Sony	Sony	Sony	Sony
Sensor name	IMX250	IMX264	IMX420	IMX428	IMX255	IMX267
Res.	2464 x 2056 gray scale / RGB	2464 x 2056 gray scale / RGB	3216 x 2208 gray scale / RGB	3216 x 2208 gray scale / RGB	4112 x 2176 gray scale / RGB	4112 x 2176 gray scale / RGB
Sensor size	2/3"	2/3"	1.1"	1.1"	1"	1"
Max. FPS (in free-running full frame mode)	80	35	53.5	50.9	47	32
Frame rate exactness (p. 104)	-	-	-	-	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	40.3	40.1	43.8	43.9	40.2	40.2
DR (normal / **HDR** (p. 302)) [dB][1/]	71.2 /	71.3 /	71.7	71.8	71.1 /	71.0
Rolling shutter	-	-	-	-	-	-
Global shutter	X	X	X	X	X	X
Global Reset	-	-	-	-	-	-
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	X	X	X	X	X	X
Linescan mode	-	-	-	-	-	-
High color reproductivity (for color version)	X	X	X	X	X	X

Power consumption (since FW 2.5.146) [W]	approx. 3.6	approx. 3.0	approx. 4.4	approx. 4.0	approx. 4.0	approx. 4.0
More specific data	**mvBlue↩ FOX3-2051 / BF3-5M-0024Z (5.1 Mpix [2464 x 2056])** (p. 405)	**mvBlue↩ FOX3-2051a / BF3-5M-0051A (5.1 Mpix [2464 x 2056])** (p. 409)	**mvBlue↩ FOX3-2071 (7.1 Mpix [3216 x 2208])** (p. 412)	**mvBlue↩ FOX3-2071a (7.1 Mpix [3216 x 2208])** (p. 416)	**mvBlue↩ FOX3-2089 / BF3-5M-0089Z (8.9 Mpix [4112 x 2176])** (p. 419)	**mvBlue↩ FOX3-2089a / BF3-5M-0089A (8.9 Mpix [4112 x 2176])** (p. 423)

> 10 Mpix < 20 Mpix

Sensors	12.4 Mpix (-2124)	12.4 Mpix (-2124a)	16.9 Mpix (-0169Z)	19.6 Mpix (-0196Z)
Sensor supplier	Sony	Sony	Sony	Sony
Sensor name	IMX253	IMX304	IMX387	IMX367
Res.	4112 x 3008 gray scale / RGB	4112 x 3008 gray scale / RGB	5472 x 3084 gray scale / RGB	4432 x 4432 gray scale / RGB
Sensor size	1.1"	1.1"	4/3"	4/3"
Max. FPS (in free-running full frame mode)	34	23	22.5	19.3
Frame rate exactness (p. 104)	-	-	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	40.2	40.2	40.2	40.1
DR (normal / **HDR** (p. 302)) [dB][1/]	70.9 /	71.0	70.8	70.7
Rolling shutter	-	-	-	-
Global shutter	X	X	X	X
Global Reset	-	-	-	-
Trigger (HW / SW)	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	X	X	X	X
Linescan mode	-	-	-	-
High color reproductivity (for color version)	X	X	X	X
Power consumption (since FW 2.5.146) [W]	approx. 4.0	approx. 4.0	approx. 4.9	approx. 4.9
More specific data	**mvBlueFOX3-2124 / BF3-5M-0124Z (12.4 Mpix [4112 x 3008])** (p. 427)	**mvBlueFOX3-2124a / BF3-5M-0124A (12.4 Mpix [4112 x 3008])** (p. 430)	**BF3-4-0169Z / BF3-5M-0169Z (16.9 Mpix [5472 x 3080])** (p. 434)	**BF3-4-0196Z / BF3-5M-0196Z (19.6 Mpix [4432 x 4432])** (p. 438)

> 20 Mpix

Sensors	31.5 Mpix (-0315Z)
Sensor supplier	Sony
Sensor name	IMX342
Res.	6480 x 4856 gray scale / RGB
Sensor size	APS-C
Max. FPS (in free-running full frame mode)	12.1
Frame rate exactness (p. 104)	-
ADC resolution / Out	12 / 12, 10, 8
SNR_{max} [dB][1/]	39.7
DR (normal / **HDR** (p. 302)) [dB][1/]	69.8
Rolling shutter	-
Global shutter	X
Global Reset	-
Trigger (HW / SW)	X / X
Pipelined global shutter in trigger mode (p. 172)	X
Linescan mode	-
High color reproductivity (for color version)	X
Power consumption (since FW 2.5.146) [W]	approx. 4.9 (mvBlueFOX3-4) | 5 (mvBlueFOX3-5M)[2]
More specific data	**BF3-4-0315Z / BF3-5M-0315Z (31.5 Mpix [6480 x 4856])** (p. 441)

[1] Measured accord. to EMVA1288 with gray scale version of the camera
[2] Connection via Type C recommended VBUS>900mA

Sony Starvis

Sensors	6.4 Mpix (-064)	12.4 Mpix (-124r)	20.5 Mpix (-205)
Sensor supplier	Sony	Sony	Sony
Sensor name	IMX178	IMX226	IMX183
Res.	3096 x 2080 gray scale / RGB	4064 x 3044 gray scale / RGB	5544 x 3692 gray scale / RGB
Sensor size	1/1.8"	1/1.7"	1"
Max. FPS (in free-running full frame mode)	59	30.7	18.6
Frame rate exactness (p. 104)	-	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	41.6	40.3	41.6
DR (normal / **HDR** (p. 302)) [dB][1/]	71.6	69.2	71.5
Rolling shutter	X	X	X
Global shutter	-	-	-
Global Reset	-	-	-
Trigger (HW / SW)	- / X	- / X	- / X
Pipelined global shutter in trigger mode (p. 172)	-	-	-

Linescan mode	-	-	-
High color reproductivity (for color version)	X	X	X
Power consumption (since FW 2.5.146) [W]	approx. 2.9	approx. 2.9	approx. 2.9
More specific data	**mvBlueFOX3-2064 / BF3-3M-0064Z / BF3-5M-0064Z (6.4 Mpix [3096 x 2080])** (p. 445)	**mvBlueFOX3-2124r / BF3-5M-0124R (12.4 Mpix [4064 x 3044])** (p. 450)	**mvBlueFOX3-2205 / BF3-5M-0205Z (20.5 Mpix [5544 x 3692])** (p. 454)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Sony Polarsens

Sensors	5.1 Mpix (-2051p)
Sensor supplier	Sony
Sensor name	IMX250_POL
Res.	2464 x 2056 gray scale / RGB
Sensor size	2/3"
Max. FPS (in free-running full frame mode)	80
Frame rate exactness (p. 104)	-
ADC resolution / Out	12 / 12, 10, 8
SNR_{max} [dB][1/]	40.2
DR (normal / **HDR** (p. 302)) [dB][1/]	71.2
Rolling shutter	-
Global shutter	X
Global Reset	-
Trigger (HW / SW)	X / X
Pipelined global shutter in trigger mode (p. 172)	X
Linescan mode	-
High color reproductivity (for color version)	X
Power consumption (since FW 2.5.146) [W]	approx. 3.6
More specific data	**mvBlueFOX3-2051p (5.1 Mpix [2464 x 2056])** (p. 459)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Aptina, CMOSIS, e2v

Sensors	1.2 Mpix (-x012b)	1.2 Mpix (-x012d)	1.3 Mpix (-x013)	2 Mpix (-x020)	2 Mpix (-x020a)	3.1 Mpix (-x031)	10 Mpix (-x100)	14 Mpix (-x140)
Sensor supplier	Aptina	Aptina	e2v	e2v	e2v	Aptina	Aptina	Aptina
Sensor name	MT9↩ M031	MT9↩ M034	EV76↩ C560	EV76↩ C570	EV76↩ C570	AR0331	MT9J003	MT9↩ F002
Res.	1280 x 960 gray scale / RGB	1280 x 960 gray scale / RGB	1280 x 1024 gray scale / RGB	1600 x 1200 gray scale / RGB	1600 x 1200 gray scale / RGB	2048 x 1536 RGB	3856 x 2764 gray scale / RGB	4384 x 3288 RGB Bayer mosaic

Sensor size	1/3"	1/3"	1/1.8"	1/1.8"	1/1.8"	1/3"	1/2.3"	1/2.3"
Pixel clock [MHz]	40 / 66 / 74.25	40 / 66 / 74.25	85	85	85	25 / 50	81.25	96.88
Max. FPS (in free-running full frame mode)	45.6	45.6	60	51	60	22.2	7	6
Frame rate exactness (p. 104)	-	-	-	-	-	-	-	-
ADC resolution / Out [bit]	12 / 12, 10, 8	12 / 12, 10, 8	10 / (12), 10, 8	10 / (12)(HW / SW), 10, 8	10 / (12), 10, 8	12 / 12, 10, 8	12 / 12, 10, 8	12 / (12), 10, 8
SNR_{max} [dB][1/]	37.4	37.7	39	38.9	38.9		37.2	35.1
DR (normal / **HDR** (p. 302)) [dB][1/]	54.3 /	63.4 /	50.5 /	50.7 /	50.5 /	/	56 /	57.3 /
Rolling shutter	-	X	-	-	-	X	X	X
Global shutter	X	-	X	X	X	-	-	-
Global Reset	-	-	-	-	-	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 172)	-	-	-	-	-	-	-	-
Linescan mode	-	-	**available** (p. 472)	**available** (p. 475)	**available** (p. 478)	-	-	-
High color reproductivity (for color version)	X	X	X	X	X	X	X	X

Power consumption (since FW 2.↩ 5.146) [W]	approx. 2.25	approx. 2.3	approx. 2.3	approx. 2.25	approx. 2.25	approx. 2.45	approx. 2.55	approx. 2.25
More specific data	**mv↩ Blue↩ FOX3-1012b (1.2 Mpix [1280 x 960])** (p. 463)	**mv↩ Blue↩ FOX3-1012d (1.2 Mpix [1280 x 960])** (p. 466)	**mv↩ Blue↩ FOX3-1013 (1.↩ 3 Mpix [1280 x 1024])** (p. 469)	**mv↩ Blue↩ FOX3-1020 (1.↩ 9 Mpix [1600 x 1200])** (p. 473)	**mv↩ Blue↩ FOX3-1020a (1.9 Mpix [1600 x 1200])** (p. 476)	**mv↩ Blue↩ FOX3-1031 (3.↩ 2 Mpix [2048 x 1536])** (p. 479)	**mv↩ Blue↩ FOX3-1100 (11 Mpix [3856 x 2764])** (p. 481)	**mv↩ Blue↩ FOX3-1140 (14 Mpix [4384 x 3288])** (p. 485)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.10.5 Supported image formats

Gray scale version	Color version
Mono8	
Mono10	
Mono12	
Mono14	
Mono16	
	RGB8Packed
	BGR8Packed
	BGRA8Packed
	BGR10V2Packed
	YUV422Packed
	YUV422_YUYVPacked
	YUV444Packed

See also

For more details about the image formats, please have a look at the enums **"TImageDestinationPixel↩ Format"** and **"TImageBufferPixelFormat"** in the `C++ developers` section.
An example application about the `pixel formats is also available`.

1.11 Filters and lenses

MATRIX VISION offers several specific filters for cameras. The **hot mirror filter** (p. 95) is part of the standard delivery condition.

1.11.1 Hot Mirror Filter

The hot mirror filter has great transmission in the visible spectrum and blocks out a significant portion of the IR energy.

Technical data	
Diameter	Varies depending on the camera
Thickness	1.0 mm
Material	Borofloat
Characteristics	T = 50% @ 650 +/- 10 nm
	T $>$ 92% 390-620 nm
	R_{avg} $>$ 95% 700-1150 nm
	AOI = 0 degrees
Surface quality	Polished on both sides P4
Surface irregularity	5/3x0.06 on both sides

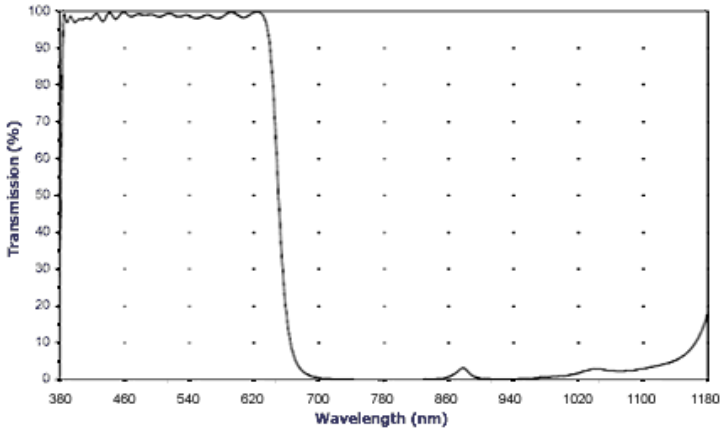


Figure 1: IR-CUT wavelengths and transmission diagram

1.11.2 Cold mirror filter

The high-quality daylight cut filter has optically polished surfaces. The polished surface allows the use of the filter directly in the path of rays in image processing applications. The filter is protected against scratches during the transport by a protection film that has to be removed before the installing the filter.

Technical data	
Diameter	Varies depending on the camera
Thickness	1.0 mm
Material	Solaris S 306
Characteristics	$T_{avg} > 80\% > 780$ nm
	AOI = 0 degrees
	Protective foil on both sides
	Without antireflexion
	Without bezel

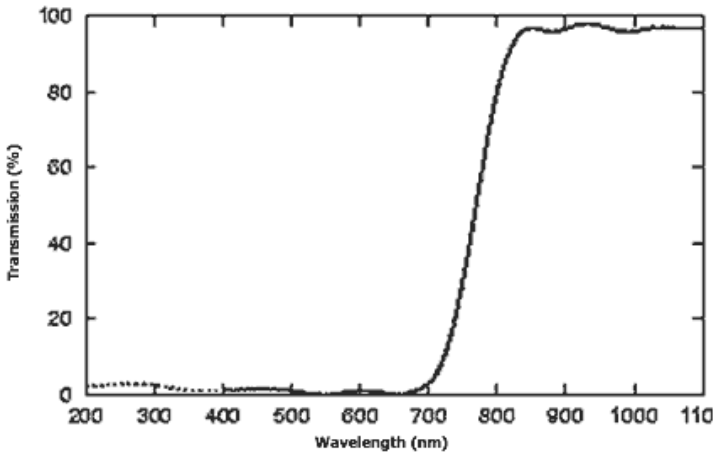


Figure 2: DL-CUT wavelengths and transmission diagram

1.11.3 Glass filter

It is also possible to choose a glass filter with following characteristics:

Technical data	
Glass thickness	1.0 mm
Material	Borofloat without coating
	ground with protection chamfer
Surface quality	polished on both sides P4
Surface irregularity	5/3x0.06 on both sides

1.11.4 Lenses

MATRIX VISION offers a high-quality selection of lenses. If you have questions about our accessories, please contact our sales team: info@matrix-vision.com.

1.12 GUI tools

1.12.1 Introduction

MATRIX VISION provides several convenient tools with graphical user interface to set up and work with their devices. Please find a short list and description below:

1.12.2 wxPropView

With wxPropView it is possible

- to acquire images
- to configure the device, and
- to display and modify the device properties.

1.12.3 mvDeviceConfigure

mvDeviceConfigure can be used to

- to set the device ID
- to update firmware- to disable CPU sleep states(some version of Windows only)

See also

For further information about the tools, please follow the link to the separate manual describing the GUI tools in great detail on our website: `https://www.matrix-vision.com/manuals/`

1.13 GenICam and advanced features

1.13.1 Introduction

For new applications or to set the device via **wxPropView** (p. 99) we recommend to use the **GenICam** (p. 165) interface layout as it allows the most flexible access to the device features.

After you've set the interface layout to **GenICam** (p. 165) (either programmed or using **wxPropView** (p. 99)), all **GenICam** (p. 165) controls of the device are available.

> **Note**
>
> It depends on the device, which controls are supported.

In **wxPropView** (p. 99), you can see them in *"Setting -> Base -> Camera -> GenICam"*:

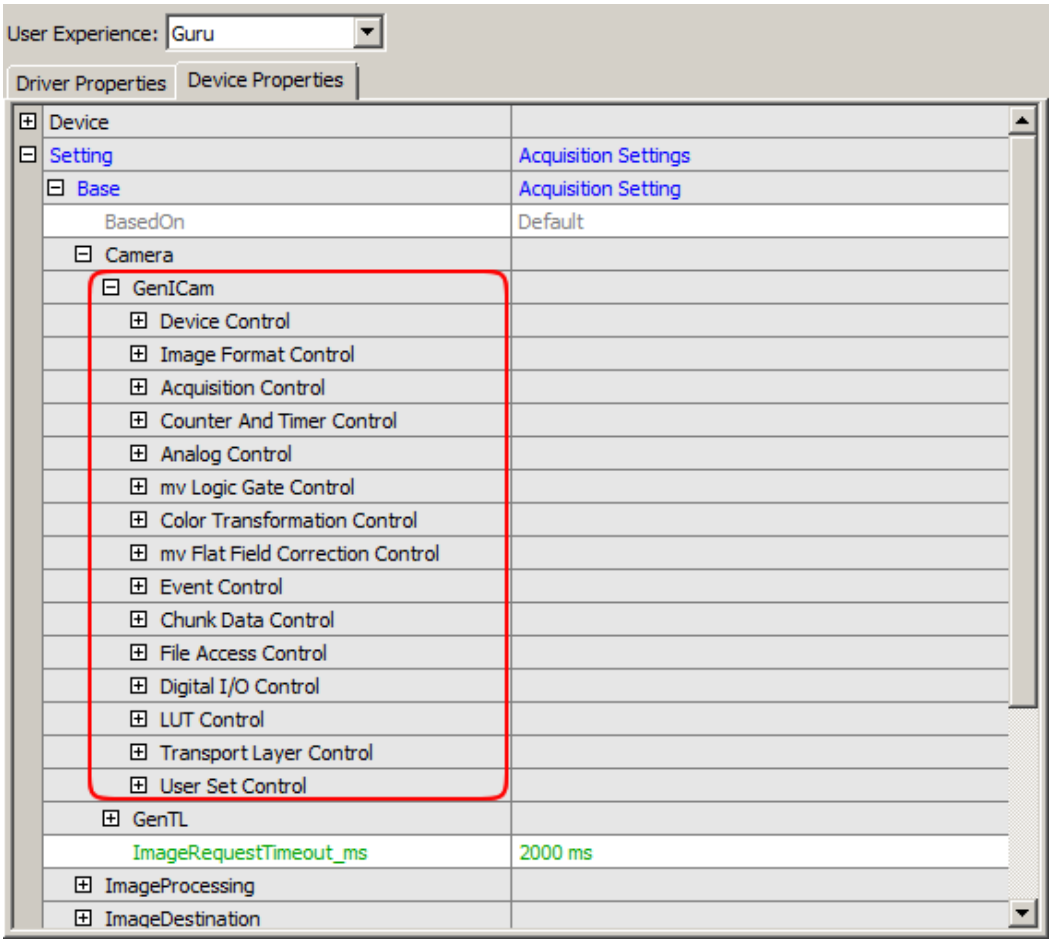


Figure 1: wxPropView - GenICam controls (depends on device and FW version)

As you can see, there are some controls with and without the prefix **"mv"**.

- **"mv"** prefix features are unique non-standard features developed by MATRIX VISION.
- Without **"mv"** are standard features as known from the **Standard Feature Naming Convention of GenICam properties** (p. 173) (SFNC).

All those features are **"camera based** / **device based"** features which can also be accessed using the camera with other **GenICam** (p. 165) / **USB3 Vision** (p. 173) compliant third-party software.

Note

In USB3 Vision timestamps are denoted in nanoseconds.

Do not mix up the camera based / device based features with the features available in *"Setting -> Base -> Image Processing"*. Theses features are driver based features which are processed by the software and therefore need CPU load.

1.13.2 Device Control

The **"Device Control"** contains the features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
DeviceType	*deviceType*	Returns the device type.
DeviceScanType	*deviceScanType*	Scan type of the sensor of the device.
DeviceVendorName	*deviceVendorName*	Name of the manufacturer of the device.
DeviceModelName	*deviceModelName*	Name of the device model.
DeviceManufacturerInfo	*deviceManufacturerInfo*	Manufacturer information about the device.
DeviceVersion	*deviceVersion*	Version of the device.
DeviceFirmwareVersion	*deviceFirmwareVersion*	Firmware version of the device.
DeviceSerialNumber	*deviceSerialNumber*	Serial number of the device.
DeviceUserID	*deviceUserID*	User-programmable device identifier.
DeviceTLVersionMajor	*deviceTLVersionMajor*	Major version of the transport layer of the device.
DeviceTLVersionMinor	*deviceTLVersionMinor*	Minor version of the transport layer of the device.
DeviceLinkSpeed	*deviceLinkSpeed*	Indicates the speed of transmission negotiated on the specified Link.
DeviceTemperature	*deviceTemperature*	Device temperature.
etc.		

related to the device and its sensor.

This control provides a bandwidth control feature . You have to select **DeviceLinkThroughputLimit**. Here you can set the maximum bandwidth in KBps.

MATRIX VISION offers also some information properties about the

- FPGA
  - **mvDeviceFPGAVersion**
- and the image sensor
  - **mvDeviceSensorColorMode**

1.13.3 Image Format Control

The **"Image Format Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
SensorWidth	*sensorWidth*	Effective width of the sensor in pixels.
SensorHeight	*sensorHeight*	Effective height of the sensor in pixels.
SensorName	*sensorName*	Name of the sensor.
Width	*width*	Width of the image provided by the device (in pixels).
Height	*height*	Height of the image provided by the device (in pixels).
BinningHorizontal, BinningVertical	*binningHorizontal*, *binningVertical*	Number of horizontal/vertical photo-sensitive cells to combine together.
DecimationHorizontal, DecimationVertical	*decimationHorizontal*, *decimationVertical*	Sub-sampling of the image. This reduces the resolution (width) of the image by the specified decimation factor.
TestPattern	*testPattern*	Selects the type of test image that is sent by the device.
etc.		

related to the format of the transmitted image.

Note

Binning Horizontal and Decimation Horizontal should **never** be used together! The same applies to Binning Vertical and Decimation Vertical. If e.g. Binning Horizontal and Decimation Horizontal are used together this may cause artifacts and/or loss of image data!

With **TestPattern**, for example, you can select the type of test image that is send by the device. Here, MATRIX VISION offers two special types:

- **mvBayerRaw** (the Bayer mosaic raw image)
- **mvFFCImage** (the **flat-field correction image** (p. 233))

Additionally, MATRIX VISION offers numerous additional features like:

- **mvMultiAreaMode**
  which can be used to define multiple AOIs (Areas of Interests) in one image.

  See also

  The use case **Working with multiple AOIs (mv Multi Area Mode)** (p. 193) shows how this feature works.

1.13.4 Acquisition Control

The **"Acquisition Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
AcquisitionMode	*acquisitionMode*	Sets the acquisition mode of the device. The different modes configures a device to send • exactly **one** image (**"SingleFrame"**), • exactly the **set number** of frames (**"MultiFrame"**) or • images constantly until an application explicitly stops the acquisition again (**"Continuous"**). and can be used for asynchronously grabbing and sending image(s). It works with internal and external hardware trigger where the edge is selectable. The external trigger uses ImageRequestTimeout (ms) to time out. • The chapter **"How To See The First Image"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual shows how to acquire images with **wxPropView** (p. 99). • The use case **Acquiring a number of images** (p. 175) shows how to acquire a number of images; also triggered externally.
AcquisitionStart	*acquisitionStart*	Starts the acquisition of the device.
AcquisitionStop	*acquisitionStop*	Stops the acquisition of the device at the end of the current Frame.
AcquisitionAbort	*acquisitionAbort*	Aborts the acquisition immediately.

AcquisitionFrameRate	*acquisitionFrameRate*	Controls the acquisition rate (in Hertz) at which the frames are captured. Some cameras support a special internal trigger mode that allows more exact frame rates. This feature keeps the frame rate constant to an accuracy of +/-0.005 fps at 200 fps. This is achieved using frames with a length difference of up to 1 us. Please check in the **sensor summary** (p. 83) if this feature exists for the requested sensor.
TriggerSelector	*triggerSelector*	Selects the type of trigger to configure. A possible option is **mv↩ TimestampReset**. The use case about **mvTimestampReset is available** (p. 325).
TriggerOverlap[TriggerSelector]	*triggerOverlap*	Specifies the type trigger overlap permitted with the previous frame. *TriggerOverlap* is only intended for external trigger (which is usually non-overlapped: i.e. exposure and readout are sequentially). This leads to minimal latency / jitter between trigger and exposure. However, the camera accepts a new trigger (the exposure time earlier) before the end of the transmission of the current image. `Maximum frame rate in triggered mode = frame rate of continuous mode` This however leads to higher latency / jitter between trigger and exposure. A trigger will be **not** latched if it occurs before this moment (trigger is accurate in time). **See also** **Principles of overlapped and pipelined trigger** (p. 172).
ExposureMode	*exposureMode*	Sets the operation mode of the exposure (or shutter).
ExposureTime	*exposureTime*	Sets the exposure time (in microseconds) when *ExposureMode* is **Timed** and *ExposureAuto* is **Off**.
ExposureAuto	*exposureAuto*	Sets the automatic exposure mode when *ExposureMode* is **Timed**.
etc.		

related to the image acquisition, including the triggering mode.

Additionally, MATRIX VISION offers numerous additional features like:

- **mvShutterMode**
  which selects the shutter mode of the CMOS sensors like rolling shutter or global shutter.
- **mvDefectivePixelEnable**
  which activates the sensor's defective pixel correction.

  See also

  `https://www.matrix-vision.com/files/mv11/Glossary/art_image_errors_sensors_en.pdf`
- **mvExposureAutoAverageGrey**
  common desired average grey value (in percent) used for Auto Gain Control (AGC) and Auto Exposure Control (AEC).
- **mvExposureAutoAOIMode**
  common AutoControl AOI used for Auto Gain Control (AGC), Auto Exposure Control (AEC) and Auto White Balance (AWB).
- **mvAcquisitionMemoryMaxFrameCount**
  which shows the maximum of frames the internal memory can save.

  See also

  The use case **Working with burst mode buffer** (p. 197) lists some maximum frame counts of some camera models.
- **mvSmearReduction**
  smear reduction in triggered and non-overlapped mode.
- **mvSmartFrameRecallEnable**
  which configures the internal memory to store each frame (thats gets transmitted to the host) in full resolution.

  See also

  The use case **SmartFrameRecall** (p. 201) shows how this feature works.

  Since

  FW Revision 2.40.2546.0

  Common properties for *AutoExposureControl*, *AutoGainControl* , and *AutoWhiteBalance* are available via **mv Auto Feature Control** (p. 126).

For **"Exposure Auto Mode"**, in **wxPropView** (p. 99) just select in *"Exposure Auto"* **"Continuous"**. Afterwards, you have the possibility to set lower and upper limit, average gray combined with AOI setting:

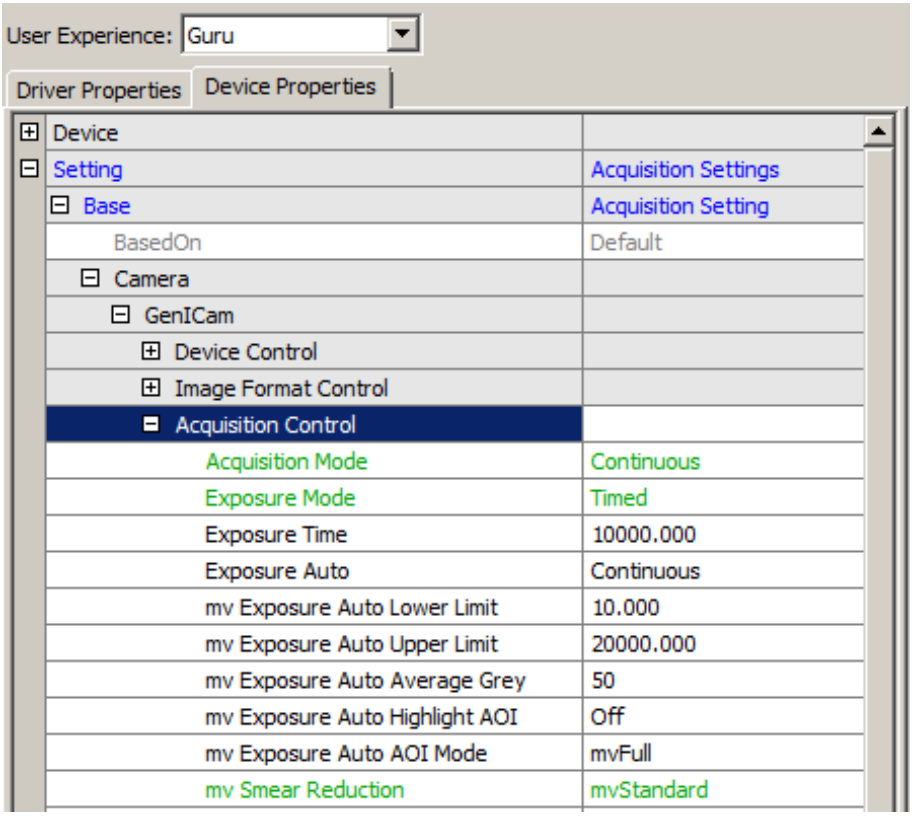


Figure 2: Acquire Control -> Exposure Auto

- **mvSmartFrameRecallFrameSkipRatio**
  When set to a value != 0, the smaller frames get thinned out. AOI requests can still be done for all frames.
- **mvSmartFrameRecallTimestampLookupAccuracy**
  is needed for the SkipRatio feature since you **don't know** the timestamps of the **internal** frames. This value defines the strictness of the timestamp-check for the recalled image (given in us).

1.13.5 Counter And Timer Control

The **"Counter And Timer Control"** is a powerful feature which MATRIX VISION customers already know under the name Hardware Real-Time Controller (HRTC). MATRIX VISION cameras provide:

- 4 counters for counting events or external signals (compare number of triggers vs. number of frames; over-trigger) and
- 2 timers.

Counter and Timers can be used, for example,

- for **pulse width modulation (PWM)** (p. 280) and
- to generate output signals of variable length, depending on conditions in camera.

This achieves complete HRTC functionality which supports following applications:

- frame rate by timer
- exposure time by timer
- pulse width at input

The **"Counter And Timer Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
CounterSelector	*counterSelector*	Selects which counter to configure.
CounterEventSource[CounterSelector]	*counterEventSource*	Selects the events that will be the source to increment the counter.
CounterEventActivation[CounterSelector]	*counterEventActivation*	Selects the activation mode event source signal.
etc.		
TimerSelector	*timerSelector*	Selects which timer to configure.
TimerDuration[TimerSelector]	*timerDuration*	Sets the duration (in microseconds) of the timer pulse.
TimerDelay[TimerSelector]	*timerDelay*	Sets the duration (in microseconds) of the delay.
etc.		

related to the usage of programmable counters and timers.

Because there are many ways to use this feature, the list of use cases is long and not finished yet:

- **Processing triggers from an incremental encoder** (p. 277)
- **Creating different exposure times for consecutive images** (p. 284)
- **Creating synchronized acquisitions using timers** (p. 339)
- **Generating a pulse width modulation (PWM)** (p. 280)

- **Outputting a pulse at every other external trigger** (p. 282)
- **Generating very long exposure times** (p. 191)

1.13.6 Analog Control

The **"Analog Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
GainSelector	*gainSelector*	Selects which gain is controlled by the various gain features.
Gain[GainSelector]	*gain*	Controls the selected gain as an absolute physical value [in dB].
GainAuto[GainSelector]	*gainAuto*	Sets the automatic gain control (AGC) mode.
GainAutoBalance	*gainAutoBalance*	Sets the mode for automatic gain balancing between the sensor color channels or taps.
BlackLevelSelector	*blackLevelSelector*	Selects which black level is controlled by the various BlackLevel features.
BlackLevel[BlackLevelSelector]	*blackLevel*	Controls the selected BlackLevel as an absolute physical value.
BalanceWhiteAuto	*balanceWhiteAuto*	Controls the mode for automatic white balancing between the color channels.
Gamma	*gamma*	Controls the gamma correction of pixel intensity.
etc.		

related to the video signal conditioning in the analog domain.

Additionally, MATRIX VISION offers:

- **mvBalanceWhiteAuto** functions and
- **mvGainAuto** functions.

In **wxPropView** (p. 99) just select in *"Gain Auto"* (AGC) **"Continuous"**. Afterwards, you have the possibility to set minimum and maximum limit combined with AOI setting:

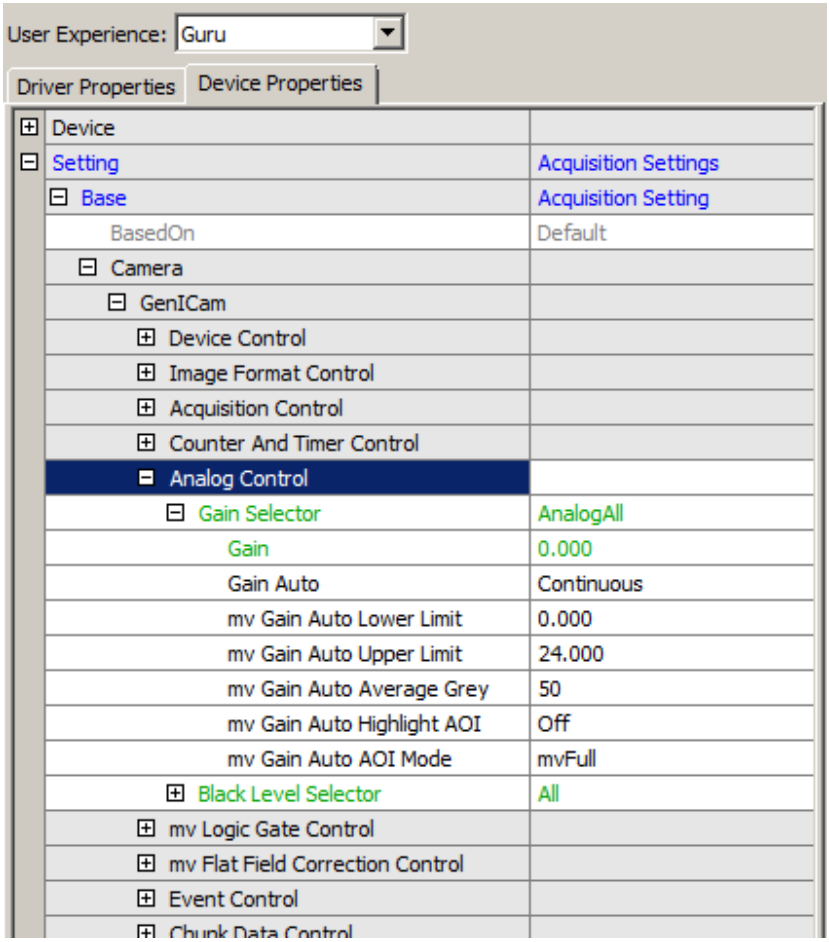


Figure 3: Analog Control -> Gain Auto

See also

Optimizing the color/luminance fidelity of the camera (p. 236)

1.13.7 Color Transformation Control

The **"Color Transformation Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
ColorTransformationSelector	*colorTransformationEnable*	Activates the selected color transformation module.
ColorTransformationSelector	*colorTransformationSelector*	Selects which color transformation module is controlled by the various color transformation features.
ColorTransformationValue	*colorTransformationValue*	Represents the value of the selected Gain factor or Offset inside the transformation matrix.
ColorTransformationValueSelector	*colorTransformationValueSelector*	Selects the gain factor or Offset of the Transformation matrix to access in the selected color transformation module.

related to the control of the color transformation.

This control offers an enhanced color processing for optimum color fidelity using a **color correction matrix** (CCM) and enables

- 9 coefficients values ($Gain_{00}$ .. $Gain_{22}$) and
- 3 offset values ($Offset_0$ .. $Offset_2$)

to be entered for RGB_{IN} -> RGB_{OUT} transformation. This can be used to optimize specific colors or specific color temperatures.

$$\begin{pmatrix} R\\ G\\ B \end{pmatrix}C0_{out} = \begin{pmatrix} Gain00 & Gain01 & Gain02\\ Gain10 & Gain11 & Gain12\\ Gain20 & Gain21 & Gain22 \end{pmatrix}\begin{pmatrix} C0_{in}\\ C1_{in}\\ C2_{in} \end{pmatrix} + \begin{pmatrix} Offset0\\ Offset1\\ Offset2 \end{pmatrix}$$

$$\text{Equivalent: } \begin{pmatrix} R_{out}\\ G_{out}\\ B_{out} \end{pmatrix} = \begin{pmatrix} RR & RG & RB\\ GR & GG & GB\\ BR & BG & BB \end{pmatrix}\begin{pmatrix} R_{in}\\ G_{in}\\ B_{in} \end{pmatrix} + \begin{pmatrix} R_{offset}\\ G_{offset}\\ B_{offset} \end{pmatrix}$$

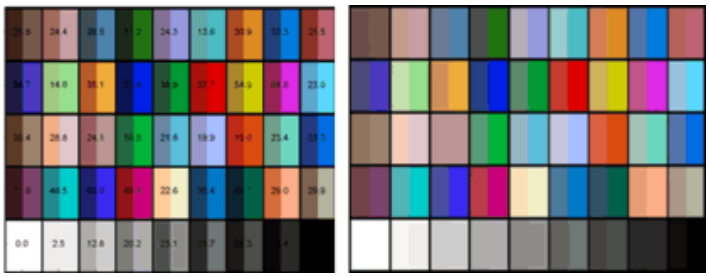


Figure 4: Color correction sample

Coefficients will be made available for sensor models and special requirements on demand.

See also

Optimizing the color/luminance fidelity of the camera (p. 236)

1.13.8 Event Control

The **"Event Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
EventSelector	*eventSelector*	Selects which Event to signal to the host application.
EventNotification[EventSelector]	*eventNotification*	Activate or deactivate the notification to the host application of the occurrence of the selected Event.
EventFrameTriggerData	*eventFrameTriggerData*	Category that contains all the data features related to the *FrameTrigger* event.
EventFrameTrigger	*eventFrameTrigger*	Returns the unique identifier of the *FrameTrigger* type of event.
EventFrameTriggerTimestamp	*eventFrameTriggerTimestamp*	Returns the timestamp of the *AcquisitionTrigger* event.
EventFrameTriggerFrameID	*eventFrameTriggerFrameID*	Returns the unique identifier of the frame (or image) that generated the *FrameTrigger* event.
EventExposureEndData	*eventExposureEndData*	Category that contains all the data features related to the *ExposureEnd* event.
EventExposureEnd	*eventExposureEnd*	Returns the unique identifier of the *ExposureEnd* type of event.
EventExposureEndTimestamp	*eventExposureEndTimestamp*	Returns the timestamp of the *ExposureEnd* Event.
EventExposureEndFrameID	*eventExposureEndFrameID*	Returns the unique identifier of the frame (or image) that generated the *ExposureEnd* event.
EventErrorData	*eventErrorData*	Category that contains all the data features related to the error event.
EventError	*eventError*	Returns the unique identifier of the error type of event.
EventErrorTimestamp	*eventErrorTimestamp*	Returns the timestamp of the error event.
EventErrorFrameID	*eventErrorFrameID*	If applicable, returns the unique identifier of the frame (or image) that generated the error event.
EventErrorCode	*eventErrorCode*	Returns an error code for the error(s) that happened.
etc.		

related to the generation of Event notifications by the device.

The use case **Working with Event Control** (p. 210) shows how this control can be used.

1.13.9 Chunk Data Control

The **"Chunk Data Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
ChunkModeActive	*chunkModeActive*	Activates the inclusion of chunk data in the payload of the image.
ChunkSelector	*chunkSelector*	Selects which chunk to enable or control.
ChunkEnable[ChunkSelector]	*chunkEnable*	Enables the inclusion of the selected chunk data in the payload of the image.
ChunkImage	*chunkImage*	Returns the entire image data included in the payload.
ChunkOffsetX	*ChunkOffsetX*	Returns the offset x of the image included in the payload.
ChunkOffsetY	*chunkOffsetY*	Returns the offset y of the image included in the payload.
ChunkWidth	*chunkWidth*	Returns the width of the image included in the payload.
ChunkHeight	*chunkHeight*	Returns the height of the image included in the payload.
ChunkPixelFormat	*chunkPixelFormat*	Returns the pixel format of the image included in the payload.
ChunkTimestamp	*chunkTimestamp*	Returns the timestamp of the image included in the payload at the time of the *FrameStart* internal event.
etc.		

related to the Chunk Data Control.

A description can be found in the image acquisition section of the **"mvIMPACT Acquire API"** manuals.

1.13.10 File Access Control

The **"File Access Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
FileSelector	*fileSelector*	Selects the target file in the device.
FileOperationSelector[File↩ Selector]	*fileOperationSelector*	Selects the target operation for the selected file in the device.
FileOperationExecute[File↩ Selector][FileOperationSelector]	*fileOperationExecute*	Executes the operation selected by *FileOperationSelector* on the selected file.
FileOpenMode[FileSelector]	*fileOpenMode*	Selects the access mode in which a file is opened in the device.
FileAccessBuffer	*fileAccessBuffer*	Defines the intermediate access buffer that allows the exchange.
etc.		

related to the File Access Control that provides all the services necessary for generic file access of a device.

The use case **Working with the UserFile section (Flash memory)** (p. 321) shows how this control can be used.

1.13.11 Digital I/O Control

The **"Digital I/O Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
LineSelector	*lineSelector*	Selects the physical line (or pin) of the external device connector to configure.
LineMode[LineSelector]	*lineMode*	Controls if the physical Line is used to Input or Output a signal.
UserOutputSelector	*userOutputSelector*	Selects which bit of the user output register will be set by *UserOutput↩ Value*.
UserOutputValue[UserOutput↩ Selector]	*userOutputValue*	Sets the value of the bit selected by *UserOutputSelector*.
etc.		

related to the control of the general input and output pins of the device.

Additionally, MATRIX VISION offers:

- **mvLineDebounceTimeRisingEdge** and
- **mvLineDebounceTimeFallingEdge** functionality.

A description of these functions can be found in the use case **Creating a debouncing filter at the inputs** (p. 299).

How you can test the digital inputs and outputs is described in **"Testing The Digital Inputs"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual. The use case **Creating synchronized acquisitions using timers** (p. 339) is a further example which shows you how to work with digital inputs and outputs.

1.13.12 Encoder Control

The **"Encoder Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
EncoderSourceA	*encoderSourceA*	Selection of the A input line.
EncoderSourceB	*encoderSourceB*	Selection of the B input line.
EncoderMode [FourPhase]	*encoderMode*	The counter increments or decrements 1 for every full quadrature cycle.
EncoderDivider	*encoderDivider*	Sets how many Encoder increment/decrements that are needed generate an encoder output signal.
EncoderOutputMode	*encoderOutputMode*	Output signals are generated at all new positions in one direction. If the encoder reverses no output pulse are generated until it has again passed the position where the reversal started.
EncoderValue	*encoderValue*	Reads or writes the current value of the position counter of the selected Encoder. Writing to EncoderValue is typically used to set the start value of the position counter.

related to the usage if quadrature encoders.

The following figure explains the different **EncoderOutputModes** :

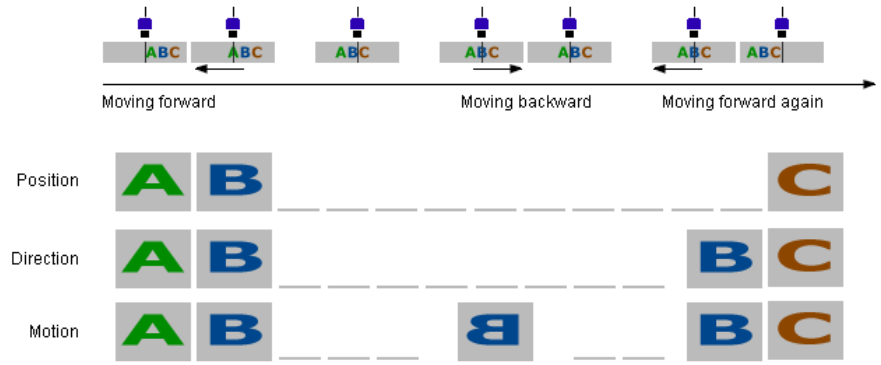


Figure 5: EncoderOutputModes

Additionally, the **Encoder** is also available as **TriggerSource** and as an **EventSource**.

A description of incremental encoder's principle can be found in the use case **Processing triggers from an incremental encoder** (p. 277).

1.13.13 Sequencer Control

1.13.13.1 Sequencer overview

The purpose of a sequencer is to allow the user of a camera to define a series of feature sets for image acquisition which can consecutively be activated during the acquisition by the camera. Accordingly, the proposed sequence is configured by a list of parameter sets. Each of these sequencer sets contains the settings for a number of camera features. Similar to user sets, the actual settings of the camera are overwritten when one of these sequencer sets is loaded. The order in which the features are applied to the camera depends on the design of the vendor. It is recommended to apply all the image related settings to the camera, before the first frame of this sequence is captured. The sequencer sets can be loaded and saved by selecting them using SequencerSetSelector. The Execution of the sequencer is completely controlled by the device.

(quoted from the `GenICam SFNC 2.3`)

1.13.13.2 Configuration of a sequencer set

The index of the adjustable sequencer set is given by the SequencerSetSelector. The number of available sequencer sets is directly given by the range of this feature. The features which are actually part of a sequencer set are defined by the camera manufacturer. These features can be read by **SequencerFeatureSelector** and activated by **SequencerFeatureEnable**[SequencerFeatureSelector]. This configuration is the same for all Sequencer Sets. To configure a sequencer set the camera has to be switched into configuration mode by SequencerConfigurationMode. Then the user has to select the desired sequencer set he wants to modify with the SequencerSetSelector. After the user has changed all the needed camera settings it is possible to store all these settings within a selected sequencer set by executing **SequencerSetSave**[SequencerSetSelector]. The user can also read back these settings by executing **SequencerSetLoad**[SequencerSetSelector]. To permit a flexible usage, more than one possibility to go from one sequencer set to another can exist. Such a path is selected by **SequencerPathSelector**[SequencerSetSelector]. Each path and therefore the transition between different sequencer sets is based on a defined trigger and an aimed next sequencer set which is selectable by **SequencerSetNext**[SequencerSetSelector][SequencerPathSelector]. After the trigger occurs the settings of the next set are active. The trigger is defined by the features **SequencerTriggerSource**[SequencerSetSelector][SequencerPathSelector] and **SequencerTriggerActivation**[SequencerSetSelector][SequencerPathSelector]. The functions of these features are the same as **TriggerSource** and **TriggerActivation**. For a flexible sequencer implementation, the **SequencerPathSelector**[SequencerSetSelector] should be part of the sequencer sets.

(quoted from the `GenICam SFNC 2.3`)

The **"Sequencer Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
SequencerMode	*sequencerMode*	Controls if the sequencer mechanism is active. Possible values are: • Off: Disables the sequencer. • On: Enables the sequencer.

SequencerConfigurationMode	*sequencerConfigurationMode*	Controls if the sequencer configuration mode is active. Possible values are: • Off: Disables the sequencer configuration mode. • On: Enables the sequencer configuration mode.
SequencerFeatureSelector	*sequencerFeatureSelector*	Selects which sequencer features to control. The feature lists all the features that can be part of a device sequencer set. All the device's sequencer sets have the same features. Note that the name used in the enumeration must match exactly the device's feature name.
SequencerFeature↩ Enable[SequencerFeature↩ Selector]	*sequencerFeatureEnable*	Enables the selected feature and make it active in all the sequencer sets.
SequencerSetSelector	*sequencerSetSelector*	Selects the sequencer set to which further settings applies.
SequencerSetSave	*sequencerSetSave*	Saves the current device state to the selected sequencer set selected by *SequencerSetSelector*.
SequencerSetLoad	*sequencerSetLoad*	Loads the sequencer set selected by *SequencerSetSelector* in the device. Even if *SequencerMode* is **Off**, this will change the device state to the configuration of the selected set.
SequencerSetActive	*sequencerSetActive*	Contains the currently active sequencer set.
SequencerSetStart	*sequencerSetStart*	Sets the initial/start sequencer set, which is the first set used within a sequencer.
SequencerPath↩ Selector[SequencerSetSelector]	*sequencerPathSelector*	Selects to which branching path further path settings apply.
SequencerSetNext	*sequencerSetNext*	Select the next sequencer set.

SequencerTriggerSource	*sequencerTriggerSource*	Specifies the internal signal or physical input line to use as the sequencer trigger source. Values supported by MATRIX VISION devices are: • Off: Disables the sequencer trigger. • ExposureEnd: Starts with the reception of the *ExposureEnd* event. • Counter1End: Starts with the reception of the *Counter1End* event. • UserOutput0: Specifies *UserOutput0* bit signal to use as internal source for the trigger. Other possible values that might be supported by third party devices are: • AcquisitionTrigger: Starts with the reception of the Acquisition Trigger. • AcquisitionTriggerMissed: Starts with the reception of the missed Acquisition Trigger. • AcquisitionStart: Starts with the reception of the Acquisition Start. • AcquisitionEnd: Starts with the reception of the Acquisition End. • FrameTrigger: Starts with the reception of the Frame Start Trigger. • FrameTriggerMissed: Starts with the reception of the missed Frame Trigger.
		• FrameStart: Starts with the reception of the Frame Start.

SequencerTriggerActivation	*sequencerTriggerActivation*	Specifies the activation mode of the sequencer trigger. Supported values for UserOutput0 are: • RisingEdge: Specifies that the trigger is considered valid on the rising edge of the source signal. • FallingEdge: Specifies that the trigger is considered valid on the falling edge of the source signal. • AnyEdge: Specifies that the trigger is considered valid on the falling or rising edge of the source signal. • LevelHigh: Specifies that the trigger is considered valid as long as the level of the source signal is high. • LevelLow: Specifies that the trigger is considered valid as long as the level of the source signal is low.

The sequencer mode can be used to set a series of feature sets for image acquisition. The sets can consecutively be activated during the acquisition by the camera. The sequence is configured by a list of parameters sets.

Note

At the moment, the Sequencer Mode is only available for MATRIX VISION cameras with CCD sensors and Sony's CMOS sensors. Please consult the **"Device Feature and Property List"s** to get a summary of the actually supported features of each sensor.

The following features are currently available for using them inside the sequencer control:

Feature	Note	Changeable during runtime
BinningHorizontal		-
BinningVertical		-
CounterDuration	Can be used to configure a certain set of sequencer parameters to be applied for the next `Counter↩ Duration` frames.	since Firmware version 2.15
DecimationHorizontal		-
DecimationVertical		-
ExposureTime		since Firmware version 2.15
Gain		since Firmware version 2.15

Height		since Firmware version 2.36
OffsetX		since Firmware version 2.35
OffsetY		since Firmware version 2.35
Width		since Firmware version 2.36
mvUserOutput		-
UserOutputValueAll		-
UserOutputValueAllMask		-
Multiple conditional sequencer paths		-

Note

Configured sequencer programs are stored as part of the **User Sets** (p. 318) like any other feature.

Actual settings of the camera are overwritten when a sequencer set is loaded.

See also

- **Define multiple exposure times using the Sequencer Control** (p. 178)
- There are 3 C++ examples called **GenICamSequencerUsage**, **GenICamSequencerUsageWithPaths** and **GenICamSequencerParameterChangeAtRuntime** that show how to control the sequencer from an application. They can be found in the `Examples` section of the `mvIMPACT Acquire C++ API`

1.13.14 Transport Layer Control

The **"Transport Layer Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
PayloadSize	*payloadSize*	Provides the number of bytes transferred for each image or chunk on the stream channel.
etc.		

related to the Transport Layer Control.

1.13.15 User Set Control

The **"User Set Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
UserSetSelector	*userSetSelector*	Selects the feature user set to load, save or configure.
UserSetLoad[UserSetSelector]	*userSetLoad*	Loads the user set specified by *UserSetSelector* to the device and makes it active.
UserSetSave[UserSetSelector]	*userSetSave*	Endianess of the device registers.
UserSetDefault	*userSetDefault*	Selects the feature user set to load and make active when the device is reset.

related to the User Set Control to save and load the user device settings.

The camera allows the storage of up to four configuration sets in the camera. This feature is similar to storing settings in the registry but this way in the camera. It is possible to store

- exposure,
- gain,
- AOI,
- frame rate,
- **LUT** (p. 127),
- one **Flat-Field Correction** (p. 233)
- etc.

permanently. You can select, which user set comes up after hard reset.

Use case related to the **"User Set Control"** is:

- **Creating user set entries** (p. 318)

Another way to create user data is described here: **Creating user data entries** (p. 316)

1.13.16 mv Logic Gate Control

The **"mv Logic Gate Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvLogicGateANDSelector	Selects the AND gate to configure.
	mvLogicGateANDSource1	Selects the first input signal of the AND gate selected by *mvLogicGateANDSelector*.
	mvLogicGateANDSource2	Selects the second input signal of the AND gate selected by *mvLogicGateANDSelector*.
	mvLogicGateORSelector	Selects the OR gate to configure.
	mvLogicGateORSource1	Selects the first input signal of the OR gate selected by *mvLogicGateORSelector*.
	mvLogicGateORSource2	Selects the second input signal of the OR gate selected by *mvLogicGateORSelector*.
	mvLogicGateORSource3	Selects the third input signal of the OR gate selected by *mvLogicGateORSelector*.
	mvLogicGateORSource4	Selects the fourth input signal of the OR gate selected by *mvLogicGateORSelector*.

related to control the devices Logic Gate Control parameters. It performs a logical operation on one or more logic inputs and produces a single logic output.

The use case **Creating different exposure times for consecutive images** (p. 284) shows how you can create different exposure times with timers, counters and the logic gate functionality.

See also

Triggering of an indefinite sequence with precise starting time (p. 291)

1.13.17 mv Flat Field Correction Control

The **"mv Flat Field Correction Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvFFCEnable	Enables the flat field correction.
	mvFFCCalibrationImageCount	The number of images to use for the calculation of the correction image.
	mvFFCCalibrate	Starts the calibration of the flat field correction.

related to control the devices Flat Field Correction parameters.

The use case shows how this control can be used.

1.13.18 mv Serial Interface Control

The **"mv Serial Interface Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvSerialInterfaceMode	States the interface mode of the serial interface.
	mvSerialInterfaceEnable	Controls whether the serial interface is enabled or not.
	mvSerialInterfaceBaudRate	Serial interface clock frequency.
	mvSerialInterfaceASCIIBuffer	Buffer for exchanging ASCII data over serial interface.
	mvSerialInterfaceWrite	Command to write data from the serial interface.
	mvSerialInterfaceRead	Command to read data from the serial interface.
etc.		

related to control the devices Serial Interface Control parameters. It enables the camera to be controlled via serial interface.

The use case **Working With The Serial Interface (mv Serial Interface Control)** (p. 333) shows how you can work with the serial interface control.

See also

Working With The Serial Interface (mv Serial Interface Control) (p. 333)

1.13.19 mv I2C Interface Control

The **"mv I2C Interface Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvI2CInterfaceEnable	Controls whether the I2C interface is enabled or not.
	mvI2CInterfaceDeviceAddress	I2C interface device address.
	mvI2CInterfaceDeviceSubAddress	I2C interface device sub address.
	mvI2CInterfaceASCIIBuffer	Buffer for exchanging ASCII data over I2C interface.
	mvI2CInterfaceWrite	Command to write data to the I2C interface.
	mvI2CInterfaceRead	Command to read data from the I2C interface.
etc.		

related to control the devices I2C Interface Control parameters. It enables the camera to be controlled via I2C interface.

The use case **Working with the I2C interface (mv I2C Interface Control)** (p. 337) shows how you can work with the serial interface control.

See also

Working with the I2C interface (mv I2C Interface Control) (p. 337)

1.13.20 mv Defective Pixel Correction Control

The **"mv Defective Pixel Correction Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvDefectivePixelCount	Contains the number of valid defective pixels.
	mvDefectivePixelSelector	Controls the index of the defective pixel to access.
	mvDefectivePixelDataLoad	Loads the defective pixels from the device non volatile memory.
	mvDefectivePixelDataSave	Saves the defective pixels to the device non volatile memory.

related to control the devices Defective Pixel data.

See also

Correcting image errors of a sensor (p. 228)

1.13.21 mv Frame Average Control (only with specific models)

The **"mv Frame Average Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvFrameAverageEnable	Enables the frame averaging engine.
	mvFrameAverageSlope	The slope in full range of register.

related to the frame averaging engine.

1.13.22 mv Auto Feature Control

The **"mv Auto Feature Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvAutoFeatureAOIMode	Common AutoControl AOI used for Auto Gain Control(AGC), Auto Exposure Control(AEC) and Auto White Balancing.

	mvAutoFeatureSensitivity	The controllers sensitivity of brightness deviations. This parameter influences the gain as well as the exposure controller.
	mvAutoFeatureCharacteristic	Selects the prioritization between Auto Exposure Control(AEC) and Auto Gain Control(AGC) controller.
	mvAutoFeatureBrightness↩ Tolerance	The error input hysteresis width of the controller. If the brightness error exceeds the half of the value in positive or negative direction, the controller restarts to control the brightness.

related to the automatic control of exposure, gain, and white balance.

With this control you can influence the characteristic of the controller depending on certain light situations. AEC/AGC can be controlled with the new additional properties **mvAutoFeatureSensitivity**, **mvAutoFeatureCharacteristic** and **mvAutoFeatureBrightnessTolerance**.

1.13.23 mv High Dynamic Range Control (only with specific sensor models)

The **"mv High Dynamic Range Control"** contains features like

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
	mvHDREnable	Enables the high dynamic range feature.
	mvHDRPreset	Selects the HDR parameter set.
	mvHDRSelector	Selects the HDR parameter set to configure.
	mvHDRVoltage1	First HDR voltage in mV.
	mvHDRVoltage2	Second HDR voltage in mV.
	mvHDRExposure1	First HDR exposure in ppm.
	mvHDRExposure2	Second HDR exposure in ppm.

related to the control of the device High Dynamic Range parameters.

The use case **Adjusting sensor of camera models -x02d (-1012d)** (p. 302) shows the principle of the HDR.

1.13.24 LUT Control

The **"LUT Control"** contains features like

Since

mvIMPACT Acquire 2.35.2054.0

A 12 to 9 RawLUT for color cameras was added. The RawLUT works identically for all colors but with a higher resolution. This is useful if higher dynamic is needed.

Feature name (acc. to **SFNC** (p. 173))	Property name (acc. to **mvIMPACT Acquire** (p. 171))	Description
LUTSelector	*LUTSelector*	Selects which LUT to control.
LUTEnable[LUTSelector]	*LUTEnable*	Activates the selected LUT.
LUTIndex[LUTSelector]	*LUTIndex*	Controls the index (offset) of the coefficient to access in the selected LUT.
LUTValue[LUTSelector][LUTIndex]	*LUTValue*	Returns the value at entry *LUTIndex* of the LUT selected by *LUTSelector*.
LUTValueAll[LUTSelector]	*LUTValueAll*	Allows access to all the LUT coefficients with a single read/write operation.
	mvLUTType	Describes which type of LUT is used for the current *LUTSelector*
	mvLUTInputData	Describes the data the LUT is applied to (e.g bayer, RGB, or gray data)
	mvLUTMapping	Describes the LUT mapping (e.g. 10 bit -> 12 bit)

related to the look-up table (LUT) control.

The look-up table (LUT) is a part of the signal path in the camera and maps data of the ADC into signal values. The LUT can be used e.g. for:

- High precision gamma
- Non linear enhancement (e.g. S-Shaped)
- Inversion (default)
- Negative offset
- Threshold
- Level windows
- Binarization

This saves (approx. 5%) CPU load, works on the fly in the FPGA of camera, is less noisy and there are no missing codes after Gamma stretching.

See also

Working with LUTValueAll (p. 312)
Implementing a hardware-based binarization (p. 314)
Optimizing the color/luminance fidelity of the camera (p. 236)

Three read-only registers describe the LUT that is selected using the LUTSelector register:

1.13.24.1 mvLUTType

There are two different types of LUTs available in MATRIX VISION cameras:

- Direct LUTs define a mapping for each possible input value, for example a 12 -> 10 bit direct LUT has 2^12 entries and each entry has 10 bit.
- Interpolated LUTs do not define a mapping for every possible input value, instead the user defines an output value for equidistant nodes. In between the nodes linear interpolation is used to calculate the correct output value.
  Considering a 10 -> 10 bit interpolated LUT with 256 nodes (as usually used in MATRIX VISION cameras), the user defines a 10 bit output value for 256 equidistant nodes beginning at input value 0, 4, 8, 12, 16 and so on. For input values in between the nodes linear interpolation is used.

1.13.24.2 mvLUTInputData

This register describes on which data the LUT is applied to:

- Bayer means that the LUT is applied to raw Bayer data, thus (depending on the de-Bayer algorithm) a manipulation of one pixel may also affect other pixels in its neighborhood.
- Gray means that the LUT is applied to gray data.
- RGB means that the LUT is applied to RGB data (i.e. after de-Bayering). Normally this is used to change the luminance on an RGB image and the LUT is applied to all three channels.

1.13.24.3 mvLUTMapping

This register describes the mapping of the currently selected LUT, e.g "map_10To10" means that a 10 bit input value is mapped to a 10 bit output values whereas "map_12To10" means that a 12 bit input value is mapped to a 10 bit output value.

1.13.24.4 LUT support in MATRIX VISION cameras

mvBlueFOX3	LUTSelector	LUT type	LUT mapping	LUT input data
-1100G -1140GW -1031G -1012dG -1012bG -1013G -1013GE -1020G -1020aG	Luminance	Direct	map_10To12	Gray
-1012bC -1012dC -1013C -1020C -1031C -1100C -1140C	Luminance Red Green Blue	Interpolated Direct Direct Direct	map_10To10 map_8To12 map_8To12 map_8To12	RGB Bayer Bayer Bayer

-2004G -2016G -2017G -2024G -2024aG -2032G -2032aG -2051G -2051aG -2051dG -2051pG -2064G -2071G -2071aG -2081aG -2089G -2089aG -2124G -2124aG -2124dG -2124rG -2162G -2204G -2205G -2246G -4315G BF3-5M-0004FG BF3-5M-0016ZG BF3-5M-0024AG BF3-5M-0024ZG BF3-5M-0032AG BF3-5M-0032ZG BF3-5M-0051AG BF3-5M-0051ZG BF3-5M-0064ZG BF3-5M-0089AG BF3-5M-0089ZG BF3-5M-0124AG BF3-5M-0124RG BF3-5M-0124ZG BF3-5M-0205ZG BF3-4-0169ZG BF3-4-0196ZG	Luminance	Direct	map_12To9	Gray

-2004C	Luminance	Interpolated	map_10To10	RGB
-2016C	Red	Direct	map_10To9	Bayer
-2017C	Green	Direct	map_10To9	Bayer
-2024C	Blue	Direct	map_10To9	Bayer
-2024aC	mvRaw	Direct	map_12To9	Bayer
-2032C				
-2032aC				
-2051C				
-2051aC				
-2051dC				
-2051pC				
-2064C				
-2071C				
-2071aC				
-2081aC				
-2089C				
-2089aC				
-2124C				
-2124aC				
-2124dC				
-2124rC				
-2162C				
-2204C				
-2205C				
-2246C				
-4315C				
BF3-5M-0004FC				
BF3-5M-0016ZC				
BF3-5M-0024AC				
BF3-5M-0024ZC				
BF3-5M-0032AC				
BF3-5M-0032ZC				
BF3-5M-0051AC				
BF3-5M-0051ZC				
BF3-5M-0064ZC				
BF3-5M-0089AC				
BF3-5M-0089ZC				
BF3-5M-0124AC				
BF3-5M-0124RC				
BF3-5M-0124ZC				
BF3-5M-0205ZC				
BF3-4-0169ZC				
BF3-4-0196ZC				

1.14 Developing applications using the mvIMPACT Acquire SDK

The mvIMPACT Acquire SDK is a comprehensive software library that can be used to develop applications using the devices described in this manual. A wide variety of programming languages is supported.

For C, C++, .NET, Python or Java developers separate API descriptions can be found on the MATRIX VISION website:

- mvIMPACT Acquire C API
- mvIMPACT Acquire C++ API
- mvIMPACT Acquire Java API
- mvIMPACT Acquire .NET API
- mvIMPACT Acquire Python API

Compiled versions (CHM format) might already be installed on your system. These manuals contain chapters on

- how to link and build applications using mvIMPACT Acquire
- how the log output for **"mvIMPACT Acquire"** devices is configured and how it works in general
- how to create your own installation packages for Windows and Linux
- a detailed API documentation
- etc.

1.15 DirectShow interface

Note

DirectShow can only be used in combination with the Microsoft Windows operating system.

Since Windows Vista, **Movie Maker** does not support capturing from a device registered for DirectShow anymore.

This is the documentation of the MATRIX VISION DirectShow_acquire interface. A MATRIX VISION specific property interface based on the IKsPropertySet has been added. All other features are related to standard DirectShow programming.

- **Supported interfaces** (p. 134)
- **Logging** (p. 134)
- **Setting up devices for DirectShow usage** (p. 134)

1.15.1 Supported interfaces

1.15.1.1 IAMCameraControl

1.15.1.2 IAMDroppedFrames

1.15.1.3 IAMStreamConfig

1.15.1.4 IAMVideoProcAmp

1.15.1.5 IKsPropertySet

The DirectShow_acquire supports the IKsPropertySet Interface. For further information please refer to the Microsoft DirectX 9.0 Programmer's Reference.

Supported property set GUID's:

- AMPROPERTY_PIN_CATEGORY
- DIRECT_SHOW_ACQUIRE_PROPERTYSET

1.15.1.6 ISpecifyPropertyPages

1.15.2 Logging

The DirectShow_acquire logging procedure is equal to the logging of the MATRIX VISION products which uses mvIMPACT Acquire. The log output itself is based on **XML**.

If you want more information about the logging please have a look at the **Logging** chapter of the respective **"mvIMPACT Acquire API"** manual or read on how to configure the log-output using **mvDeviceConfigure** in the **"mvIMPACT Acquire GUI Applications"** manual.

1.15.3 Setting up devices for DirectShow usage

In order to be able to access a device through the mvIMPACT Acquire driver stack from an application through DirectShow a registration procedure is needed. This can either be done using mvDeviceConfigure or by a command line tool that is part of the Windows operating system.

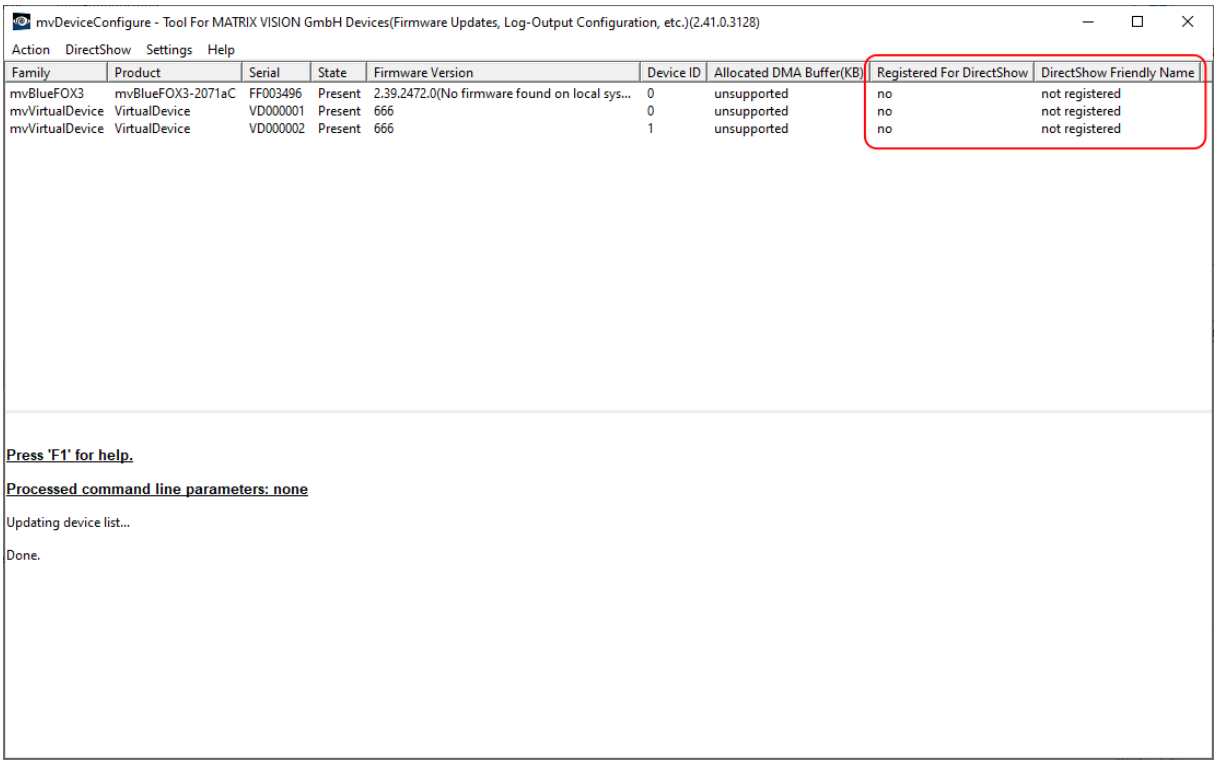
Note

Please be sure to register the MV device for DirectShow with a matching version of **mvDeviceConfigure**. I.e. if you have installed the 32-bit version of the VLC Media Player, Virtual Dub, etc., you have to register devices with the 32-bit version of mvDeviceConfigure (*"C:\Program Files\MATRIX VISION\mvIMPACT Acquire\bin"*, the 64-bit version resides in *"C:\Program Files\MATRIX VISION\mvIMPACT Acquire\bin\x64"*)!

1.15.3.1 Registering devices

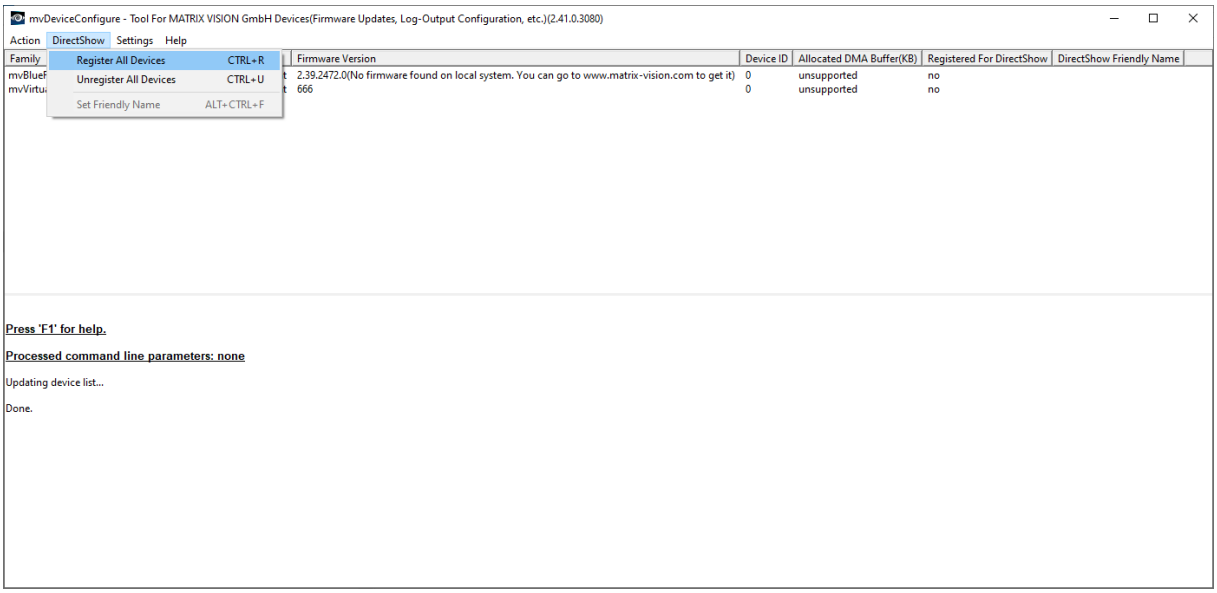
To register all devices currently recognized by the mvIMPACT Acquire driver stack for access with DirectShow the following registration procedure is needed:

1. >mvDeviceConfigure needs to be started(with elevated rights).
   If no device has been registered the application will more or less (depending on the installed devices) look like this.



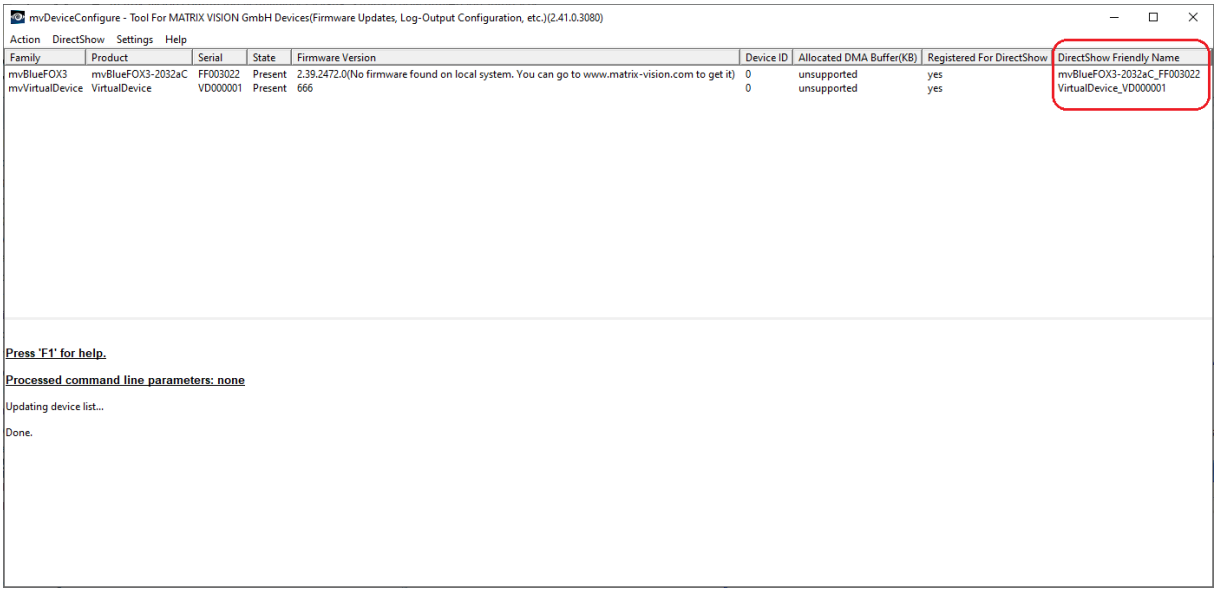
mvDeviceConfigure - After Start

2. To register every installed device for DirectShow access click on the menu item **"DirectShow"** → **"Register All Devices"**.



mvDeviceConfigure - Register All Devices

3. After a successful registration the column "Registered For DirectShow" will display **"yes"** for every device and the devices will be registered with a default DirectShow friendly name which is displayed in the **"DirectShow Friendly Name"** column.

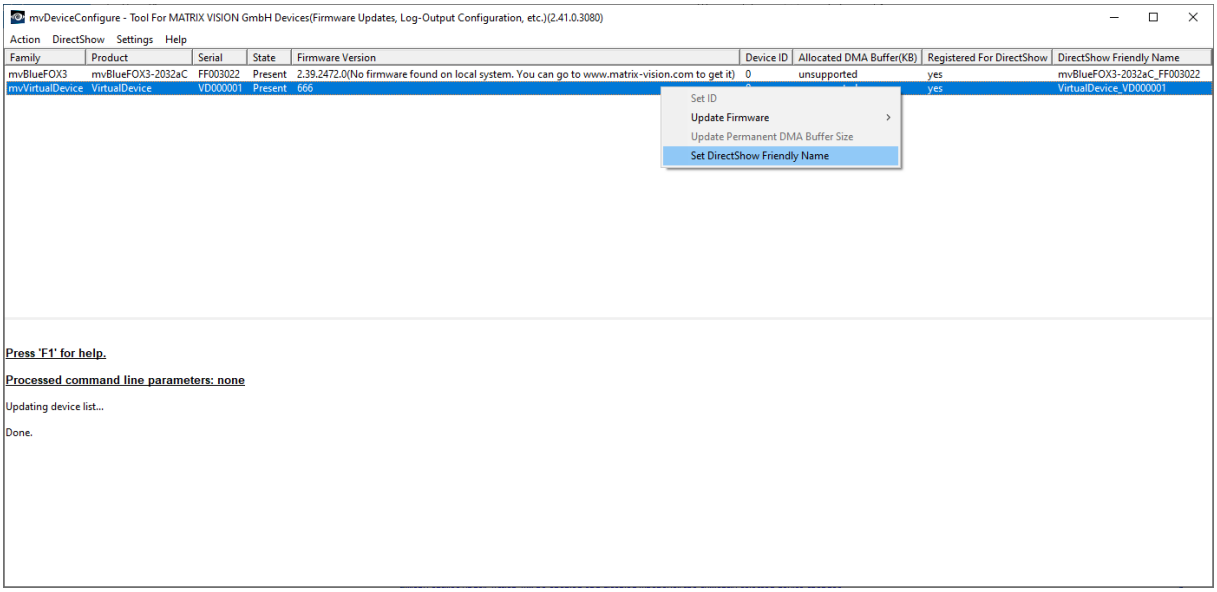


mvDeviceConfigure - All Devices Registered For DirectShow Access

1.15.3.2 Renaming devices

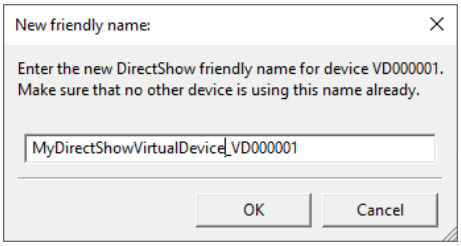
To modify the DirectShow friendly name of a device:

1. mvDeviceConfigure needs to be started(with elevated rights).
2. right-click on the device to rename and select **"Set DirectShow Friendly Name"**:



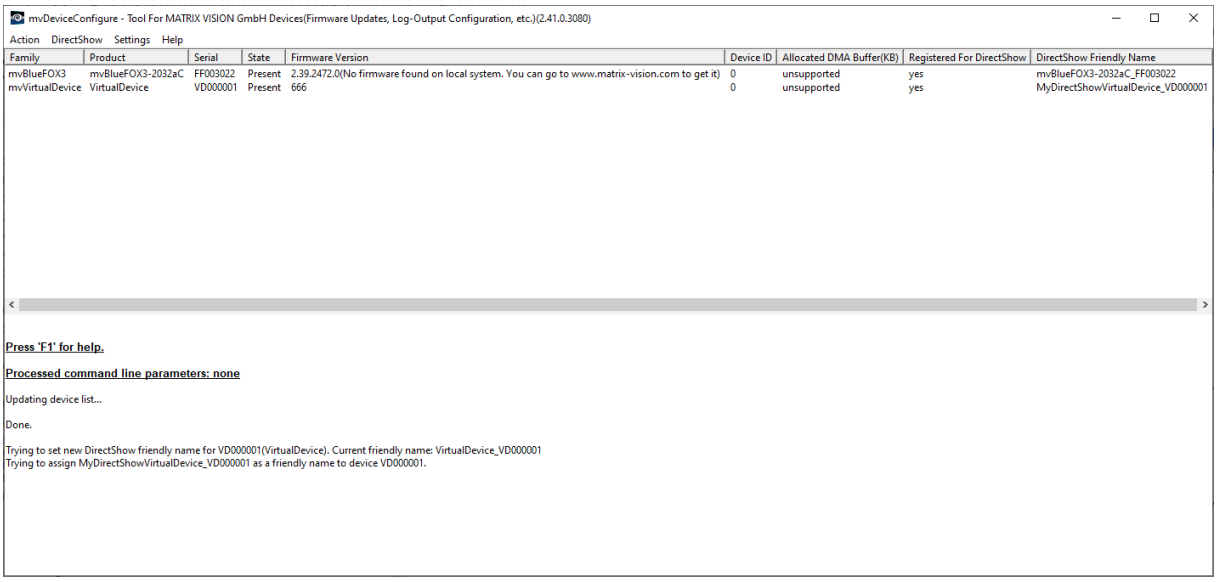
mvDeviceConfigure - Set DirectShow Friendly Name

3. Then, a dialog will appear. Please enter the new name and confirm it with **"OK"**.



mvDeviceConfigure - Dialog For New Name

4. Afterwards the column "DirectShow friendly name" will display the newly assigned friendly name.



mvDeviceConfigure - Renamed Device

Note

Please do not select the same friendly name for two different devices. In theory this is possible, however the mvDeviceConfigure GUI will not allow this to avoid confusion.

1.15.3.3 Using regsvr32

To register all devices currently recognized by the mvIMPACT Acquire driver stack with auto-assigned names, the Windows tool "`regsvr32`" can be used from an elevated command shell.

The following command line options are available and can be passed during the silent registration:

EXAMPLES:

Register ALL devices that are recognized by mvIMPACT Acquire (this will only register devices which have drivers installed) without any user interaction:

```
regsvr32 <path>\DirectShow_acquire.ax /s
```

Unregister ALL devices that have been registered before without any user interaction:

```
regsvr32 <path>\DirectShow_acquire.ax /u /s
```

1.16 Troubleshooting

- **Error code list** (p. 139)
- **Accessing log files** (p. 152)
- **General Issues** (p. 153)
- **Windows** (p. 157)
- **Linux** (p. 162)

1.16.1 Error code list

Numerical Value	String Representation	Brief Description	Detailed Description
-2000	PROPHANDLING_NOT_↩ A_LIST	This component is not a list.	A list operation for this component has been called but this component does not reference a list. [-2000]
-2001	PROPHANDLING_NOT_↩ A_PROPERTY	This component is not a property.	A property operation for this component has been called but this component does not reference a property. [-2001]
-2002	PROPHANDLING_NOT_↩ A_METHOD	This component is not a method.	A method operation for this component has been called but this component does not reference a method. [-2002]
-2003	PROPHANDLING_NO_↩ READ_RIGHTS	The caller has no read rights for this component.	It has been tried to read data from this component, but the caller has no read rights for this component. [-2003]
-2004	PROPHANDLING_NO_↩ WRITE_RIGHTS	The caller has no write rights for this component.	It has been tried to modify data of this component, but the caller has no write rights for this component. [-2004]

-2005	PROPHANDLING_NO_↩ MODIFY_SIZE_RIGHTS	The caller can't modify the size of this component.	It has been tried to modify the size of this list or the number of values stored by a property, but the caller doesn't have the required right to do this. This error will also be reported if the user tried to increase the number of values handled by a property above the maximum number of values it can handle. Therefore before resizing a property check if the new size might exceeds this maximum value by calling the appropriate function. [-2005]
-2006	PROPHANDLING↩ _INCOMPATIBLE_↩ COMPONENTS	The two involved components are not compatible.	An operation requiring two compatible components has been called with two components, which are not compatible. [-2006]
-2008	PROPHANDLING↩ _UNSUPPORTED_↩ PARAMETER	One or more of the specified parameters are not supported by the function.	This error might also be generated if a certain feature is not available on the current platform. [-2008]
-2009	PROPHANDLING_SIZE_↩ MISMATCH	Different sized value buffers have been passed.	While trying to read value pairs the caller passed two different sized value buffers to a function while one is too small to hold all the information. [-2009]
-2010	PROPHANDLING_↩ IMPLEMENTATION_↩ MISSING	A feature that is not implemented so far has been requested.	The caller requested a feature, that hasn't been implemented so far. This error code is only provided for compatibility and will be set in very rare cases only. [-2010]
-2011	PROPHANDLING↩ _ACCESSTOKEN_↩ CREATION_FAILED	An access token object couldn't be created.	This can either happen, because the caller has not the rights required to create an access token or because the system runs very low on memory. DeprecatedThis error code currently is not used anywhere within this framework. It might be removed in a future version. [-2011]

-2012	PROPHANDLING_↩ INVALID_PROP_VALUE	It has been tried to assign an invalid value to a property.	This can either happen if the value lies above or below the min. or max. value for a property or when it has been tried to write a value to a property, which is not in the properties translation dictionary (if it defines one). To find out, which values are allowed for the property in question the user should Check if the property defines a translation dictionary. Check the allowed values within a translation dictionary if one is defined. Check the min and max value for properties, that define limits. [-2012]
-2013	PROPHANDLING_PROP↩ _TRANSLATION_TABLE↩ _CORRUPTED	The properties translation table has been corrupted.	The properties translation table has been corrupted for an unknown reason and can't be used anymore. [-2013]
-2014	PROPHANDLING_PROP↩ _VAL_ID_OUT_OF_↩ BOUNDS	Invalid value index.	The caller tried to read a value from an invalid index from a property. Most properties store one value only, thus the only valid positive value index will be 0 (some negative index values are reserved for special values like e.g. the min/max value of a property). However some properties might store more than one value, thus the max. allowed index might be higher. The highest index allowed will always be the value count of a property minus one for properties with the cfFixedSize flag set. Other properties will automatically adjust the size once the user writes to an index out of bounds. [-2014]
-2015	PROPHANDLING_PROP↩ _TRANSLATION_TABLE↩ _NOT_DEFINED	This property doesn't define a translation table.	The caller tried to modify a translation table, that hasn't been defined for this property. [-2015]

-2016	PROPHANDLING_↩ INVALID_PROP_VALUE_↩ TYPE	An invalid value has been passed to the property.	Although properties are quite tolerant regarding the allowed assignment for them some value types can't be used to write all properties. As an example assigning a float value to an integer property would result in this error. Another reason for this error might be when a user tried to access e.g. a float property with functions meant to be used for int properties. [-2016]
-2017	PROPHANDLING_PROP↩ _VAL_TOO_LARGE	A too large value has been passed.	One or more of the values the caller tried to write to the property are larger than the max. allowed value for this property. [-2017]
-2018	PROPHANDLING_PROP↩ _VAL_TOO_SMALL	A too small value has been passed.	One or more of the values the caller tried to write to the property are smaller than the min. allowed value for this property. [-2018]
-2019	PROPHANDLING_↩ COMPONENT_NOT_↩ FOUND	The specified component could not be found.	[-2019]
-2020	PROPHANDLING_LIST_↩ ID_INVALID	An invalid list has been referenced.	[-2020]
-2021	PROPHANDLING_↩ COMPONENT_ID_INVALID	An invalid component within a list has been referenced.	[-2021]
-2022	PROPHANDLING_LIST_↩ ENTRY_OCCUPIED	The specified list index is occupied.	During the creation of a new component the caller tried the insert the newly created component into a list at a position already used to store another component. [-2022]
-2023	PROPHANDLING_↩ COMPONENT_HAS_↩ OWNER_ALREADY	The specified component already has an owner.	The caller tried to assign an owner to a component that already has an owner. An owner once defined can't be modified anymore. [-2023]
-2024	PROPHANDLING↩ _COMPONENT_↩ ALREADY_REGISTERED	It has been tried to register the same component at twice in the same list.	[-2024]

-2025	PROPHANDLING_LIST_↩ CANT_ACCESS_DATA	The desired data can't be accessed or found.	During loading or saving data this error can occur e.g. if it has been tried to import a setting from a location where the desired setting couldn't be found. Another reason for this error might be that the current user is not allowed to perform a certain operation on the desired data (e.g. a user tries to delete a setting that is stored with global scope but does not have elevated access rights). [-2025]
-2026	PROPHANDLING_↩ METHOD_PTR_INVALID	The function pointer of the referenced method object is invalid.	[-2026]
-2027	PROPHANDLING_↩ METHOD_INVALID_↩ PARAM_LIST	A method object has an invalid parameter list.	[-2027]
-2028	PROPHANDLING_SWIG↩ _ERROR	This indicates an internal error occurred within the SWIG generated wrapper code, when working under Python.	[-2028]
-2029	PROPHANDLING↩ _INVALID_INPUT_↩ PARAMETER	A invalid input parameter has been passed to a function of this module.	In most cases this might be a unassigned pointer, where a valid pointer to a user defined storage location was expected. [-2029]
-2030	PROPHANDLING_↩ COMPONENT_NO_↩ CALLBACK_REGISTERED	The user tried to modify a registered callback, but no callback has been registered for this component.	[-2030]
-2031	PROPHANDLING_INPUT↩ _BUFFER_TOO_SMALL	The user tried to read data into a user supplied storage location, but the buffer was too small to accommodate the result.	[-2031]
-2032	PROPHANDLING_↩ WRONG_PARAM_COUNT	The number of parameters is incorrect.	This error might occur if the user called a function with a variable number of input or output parameters and the number of parameters passed to the function does not match the number of required parameters. [-2032]
-2033	PROPHANDLING↩ _UNSUPPORTED_↩ OPERATION	The user tried to execute an operation, which is not supported by the component he is referring to.	[-2033]

-2034	PROPHANDLING_CANT↩ _SERIALIZE_DATA	The user tried to save(serialize) a property list without having the right to do this.	[-2034]
-2035	PROPHANDLING_↩ INVALID_FILE_CONTENT	The user tried to use a file to update or create a component list, that does not contain valid data for this operation.	This e.g. might happen, if the file does not contain valid XML data or XML data that is not well formed. [-2035]
-2036	PROPHANDLING_CANT↩ _ALLOCATE_LIST	This error will occur when the modules internal representation of the tree structure does not allow the allocation of a new list.	In this case either new list can't be allocated. The only way to solve this problem is to delete another list. [-2036]
-2037	PROPHANDLING_↩ CANT_REGISTER_↩ COMPONENT	The referenced list has no space left to register this component at the desired position.	There might however be an empty space within the list where this element could be registered, but no more components can be registered at the end of this list. [-2037]
-2038	PROPHANDLING_PROP↩ _VALIDATION_FAILED	The user tried to assign a value to a property, that is invalid.	This will result in a detailed error message in the logfile. This error might arise e.g. when a string property doesn't allow the string to contain numbers. In this case trying to set the properties value to 'blabla7bla' would cause this error. [-2038]
-2099	PROPHANDLING_LAST↩ _VALID_ERROR_CODE	Defines the last valid error code value for the property module.	[-2099]
-2100	DMR_DEV_NOT_FOUND	The specified device can't be found.	This error occurs either if an invalid device ID has been passed to the device manager or if the caller tried to close a device which currently isn't initialized. [-2100]
-2101	DMR_INIT_FAILED	The device manager couldn't be initialized.	This is an internal error. [-2101]
-2102	DMR_DRV_ALREADY_IN↩ _USE	The device is already in use.	This error e.g. will occur if this or another process has initialized this device already and an application tries to open the device once more or if a certain resource is available only once but shall be used twice. [-2102]

-2103	DMR_DEV_CANNOT_↩ OPEN	The specified device couldn't be initialized.	[-2103]
-2104	DMR_NOT_INITIALIZED	The device manager or another module hasn't been initialized properly.	This error occurs if the user tries e.g. to close the device manager without having initialized it before or if a library used internally or a module or device associated with that library has not been initialized properly or if [-2104]
-2105	DMR_DRV_CANNOT_↩ OPEN	A device could not be initialized.	In this case the log-file will contain detailed information about the source of the problem. [-2105]
-2106	DMR_DEV_REQUEST_↩ QUEUE_EMPTY	The devices request queue is empty.	This error e.g. occurs if the user waits for an image request to become available at a result queue without having send an image request to the device before. It might also arise when trying to trigger an image with a software trigger mechanism before the acquisition engine has been completely started. In this case a small delay and then again calling the software trigger function will succeed. [-2106]
-2107	DMR_DEV_REQUEST_↩ CREATION_FAILED	A request object couldn't be created.	The creation of a request object failed. This might e.g. happen, if the system runs extremely low on memory. [-2107]
-2108	DMR_INVALID_↩ PARAMETER	An invalid parameter has been passed to a function.	This might e.g. happen if a function requiring a pointer to a structure has been passed an unassigned pointer or if a value has been passed, that is either too large or too small in that context. [-2108]

-2109	DMR_EXPORTED_↩ SYMBOL_NOT_FOUND	One or more symbols needed in a detected driver library couldn't be resolved.	In most cases this is an error handled internally. So the user will not receive this error code as a result of a call to an API function. However when the user tries to get access to an IMPACT buffer type while the needed IMPACT Base libraries are not installed on the target system this error code also might be returned to the user. [-2109]
-2110	DEV_UNKNOWN_ERROR	An unknown error occurred while processing a user called driver function.	[-2110]
-2111	DEV_HANDLE_INVALID	A driver function has been called with an invalid device handle.	[-2111]
-2112	DEV_INPUT_PARAM_↩ INVALID	A driver function has been called but one or more of the input parameters are invalid.	There are several possible reasons for this error↩ : an unassigned pointer has been passed to a function, that requires a valid pointer one or more of the passed parameters are of an incorrect type one or more parameters contain an invalid value (e.g. a filename that points to a file that can't be found, a value, that is larger or smaller than the allowed values. within the current setup one or more parameters impose restrictions on the requested operation that don't allow its execution. [-2112]
-2113	DEV_WRONG_INPUT_↩ PARAM_COUNT	A function has been called with an invalid number of input parameters.	[-2113]
-2114	DEV_CREATE_SETTING↩ _FAILED	The creation of a setting failed.	This can either happen, when a setting with the same name as the one the user tried to create already exists or if the system can't allocate memory for the new setting. [-2114]

-2115	DEV_REQUEST_CANT_↩ BE_UNLOCKED	The unlock for a Request object failed.	This might happen, if the Request is not locked at the time of calling the unlock function. It either has been unlocked by the user already or this request has never been locked as the request so far has not been used to capture image data into its buffer. Another reason for this error might be that the user tries to unlock a request that is currently processed by the device driver. [-2115]
-2116	DEV_INVALID_↩ REQUEST_NUMBER	The number for the Request object is invalid.	The max. number for a Request object is the value of the property RequestCount in the SystemSettings list - 1. [-2116]
-2117	DEV_LOCKED_↩ REQUEST_IN_QUEUE	A Request that hasn't been unlocked has been passed back to the driver.	This error might occur if the user requested an image from the driver but hasn't unlocked the Request that will be used for this new image. [-2117]
-2118	DEV_NO_FREE_↩ REQUEST_AVAILABLE	The user requested a new image, but no free Request object is available to process this request.	[-2118]
-2119	DEV_WAIT_FOR_↩ REQUEST_FAILED	The wait for a request failed.	This might have several reasons: The user waited for an image, but no image has been requested before. The user waited for a requested image, but the image is still not ready(e.g. because of a short timeout and a long exposure time). A triggered image has been requested but no trigger signal has been detected within the wait period. A plug and play device(e.↩ g. an USB device) has been unplugged and therefore can't deliver images anymore. In this case the 'state' property should be checked to find out if the device is still present or not. [-2119]
-2120	DEV_UNSUPPORTED_↩ PARAMETER	The user tried to get/set a parameter, which is not supported by this device.	[-2120]

-2121	DEV_INVALID_RTC_↩ NUMBER	The requested real time controller is not available for this device.	[-2121]
-2122	DMR_INTERNAL_ERROR	Some kind of internal error occurred.	More information can be found in the ∗.log-file or the debug output. [-2122]
-2123	DMR_INPUT_BUFFER_↩ TOO_SMALL	The user allocated input buffer is too small to accommodate the result.	[-2123]
-2124	DEV_INTERNAL_ERROR	Some kind of internal error occurred in the device driver.	More information can be found in the ∗.log-file or the debug output. [-2124]
-2125	DMR_LIBRARY_NOT_↩ FOUND	One or more needed libraries are not installed on the system.	[-2125]
-2126	DMR_FUNCTION_NOT_↩ IMPLEMENTED	A called function or accessed feature is not available for this device.	[-2126]
-2127	DMR_FEATURE_NOT_↩ AVAILABLE	The feature in question is (currently) not available for this device or driver.	This might be because another feature currently blocks the one in question from being accessible. More information can be found in the ∗.log-file or the debug output. [-2127]
-2128	DMR_EXECUTION_↩ PROHIBITED	The user is not permitted to perform the requested operation.	This e.g. might happen if the user tried to delete user data without specifying the required password. [-2128]
-2129	DMR_FILE_NOT_FOUND	The specified file can't be found.	This might e.g. happen if the current working directory doesn't contain the file specified. [-2129]
-2130	DMR_INVALID_LICENCE	The licence doesn't match the device it has been assigned to.	When e.g. upgrading a device feature each licence file is bound to a certain device. If the device this file has been assigned to has a different serial number then the one used to create the licence this error will occur. [-2130]
-2131	DEV_SENSOR_TYPE_↩ ERROR	There is no sensor found or the found sensor type is wrong or not supported.	[-2131]

-2132	DMR_CAMERA_↩ DESCRIPTION_INVALID	A function call was associated with a camera description, that is invalid.	One possible reason might be, that the camera description has been deleted(driver closed?). 1.5.0 [-2132]
-2133	DMR_NEWER_LIBRARY↩ _REQUIRED	A suitable driver library to work with the device manager has been detected, but it is too old to work with this version of the mvDevice↩ Manager library.	This might happen if two different drivers have been installed on the target system and one introduces a newer version of the device manager that is not compatible with the older driver installed on the system. In this case this error message will be written into the logfile together with the name of the library that is considered to be too old. The latest drivers will always be available online under www.↩ matrix-vision.de. There will always be an updated version of the library considered to be too old for download from here. 1.6.6 [-2133]
-2134	DMR_TIMEOUT	A general timeout occurred.	This is the typical result of functions that wait for some condition to be met with a timeout among their parameters. More information can be found in the ∗.log-file or the debug output. 1.7.2 [-2134]
-2135	DMR_WAIT_ABANDONED	A wait operation has been aborted.	This e.g. might occur if the user waited for some message to be returned by the driver and the device driver has been closed within another thread. In order to inform the user that this waiting operation terminated in an unusual wait, DMR_↩ WAIT_ABANDONED will be returned then. 1.7.2 [-2135]
-2136	DMR_EXECUTION_↩ FAILED	The execution of a method object or reading/writing to a feature failed.	More information can be found in the log-file. 1.9.0 [-2136]

-2137	DEV_REQUEST_↩ ALREADY_IN_USE	This request is currently used by the driver.	This error may occur if the user tries to send a certain request object to the driver by a call to the corresponding image request function. 1.10.31 [-2137]
-2138	DEV_REQUEST_↩ BUFFER_INVALID	A request has been configured to use a user supplied buffer, but the buffer pointer associated with the request is invalid.	1.10.31 [-2138]
-2139	DEV_REQUEST_↩ BUFFER_MISALIGNED	A request has been configured to use a user supplied buffer, but the buffer pointer associated with the request has an incorrect alignment.	Certain devices need aligned memory to perform efficiently thus when a user supplied buffer shall be used to capture data into this buffer must follow these alignment constraints 1.10.31 [-2139]
-2140	DEV_ACCESS_DENIED	The requested access to a device could not be granted.	There are multiple reasons for this error code. Detailed information can be found in the ∗.log-file. POSSIBLE CAUSES: an application tries to access a device exclusively that is already open in another process a network device has already been opened with control access from another system and the current system also tries to establish control access to the device an application tried to execute a function that is currently not available an application tries to write to a read-only location. 1.↩ 10.39 [-2140]

-2141	DMR_PRELOAD_↩ CHECK_FAILED	A pre-load condition for loading a device driver failed.	Certain device drivers may depend on certain changes applied to the system in order to operate correctly. E.g. a device driver might need a certain environment variable to exist. When the device manager tries to load a device driver it performs some basic checks to detect problems like this. When one of these checks fails the device manager will not try to load the device driver and an error message will be written to the selected log outputs. 1.10.52 [-2141]
-2142	DMR_CAMERA_↩ DESCRIPTION_INVALID↩ _PARAMETER	One or more of the camera descriptions parameters are invalid for the grabber it is used with.	There are multiple reasons for this error code. Detailed information can be found in the ∗.log-file. POSSIBLE CAUSES: The TapsXGeometry or Taps↩ YGeometry parameter of the selected camera description cannot be used with a user defined AOI. A scan standard has been selected, that is not supported by this device. An invalid scan rate has been selected. ... This error code will be returned by frame grabbers only. 1.10.57 [-2142]
-2143	DMR_FILE_ACCESS_↩ ERROR	A general error returned whenever there has been a problem with accessing a file.	There can be multiple reasons for this error and a detailed error message will be sent to the log-output whenever this error code is returned. POSSIBLE CAUSES: The driver tried to modify a file, for which it has no write access. The driver tried to read from a file, for which it has no read access. ... 1.10.87 [-2143]
-2144	DMR_INVALID_QUEUE_↩ SELECTION	An error returned when the user application attempts to operate on an invalid queue.	1.11.0 [-2144]

-2145	DMR_ACQUISITION_↩ ENGINE_BUSY	An error returned when the user application attempts to start the acquisition engine at a time, where it is already running.	2.5.3 [-2145]
-2146	DMR_BUSY	An error returned when the user application attempts to perform any operation that currently for any reason cannot be started because something else already running.	The log-output will provide additional information. 2.↩ 32.0 [-2146]
-2147	DMR_OUT_OF_MEMORY	An error returned when for any reason internal resources (memory, handles, ...) cannot be allocated.	The log-output will provide additional information. 2.↩ 32.0 [-2147]
-2199	DMR_LAST_VALID_↩ ERROR_CODE	Defines the last valid error code value for device and device manager related errors.	[-2199]

1.16.2 Accessing log files

If you need support using our products, you can shorten response times by sending us your log files. Accessing the log files is different in Windows and Linux:

1.16.2.1 Windows

Since

mvIMPACT Acquire 2.11.9

You can access the log files in Windows using **wxPropView** (p. 99). The way to do this is described in **"Accessing Log Files"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual.

1.16.2.2 Linux

Since

mvIMPACT Acquire 2.24.0

You can access the log files in Linux via `/opt/mvIMPACT_Acquire/data/logs`.

You can also extract the directory using the following command

```
env | grep MVIMPACT_ACQUIRE_DATA_DIR
```

or change the directory directly via

```
cd $MVIMPACT_ACQUIRE_DATA_DIR/logs
```

For older versions:

Like on Windows, log files will be generated, if the activation flag for logging called **mvDebugFlags.mvd** is available in the same folder as the application (however, using Windows log files will be generated automatically, because the applications are started from the same folder). By default, on Linux the **mvDebugFlags.mvd** will be installed in the installation's destination folder in the sub-folder "apps". For example, if the destination folder was "/home/workspace", you can locate the **mvDebugFlags.mvd** like the following way:

```
user@linux-desktop:~$
// <- Starting the console window, you will be in the home directory: /home/
user@linux-desktop:~$ cd workspace/apps/
// <- Change the directory
user@linux-desktop:/home/workspace/apps$ ls -l
// <- List the directory
insgesamt 144
drwxr-xr-x  9 user user  4096 Mai 21 15:08 Callback
drwxr-xr-x  8 user user  4096 Mai 21 15:08 Callback_C
drwxr-xr-x  9 user user  4096 Mai 21 15:08 CaptureToUserMemory_C
drwxr-xr-x  3 user user  4096 Mai 21 15:03 Common
drwxr-xr-x 11 user user  4096 Mai 21 15:09 ContinuousCapture
drwxr-xr-x  9 user user  4096 Mai 21 15:09 ContinuousCaptureAllDevices
drwxr-xr-x  6 user user  4096 Mai 21 15:09 ContinuousCaptureFLTK
drwxr-xr-x  9 user user  4096 Mai 21 15:09 ContinuousCapture_C
drwxr-xr-x 11 user user  4096 Mai 21 15:09 DigitalIOs
drwxr-xr-x  9 user user  4096 Mai 21 15:09 FirmwareUpgrade
drwxr-xr-x 11 user user  4096 Mai 21 15:09 GenericInterfaceLayout
drwxr-xr-x 11 user user  4096 Mai 21 15:09 GenICamInterfaceLayout
-rw-r--r--  1 user user   854 Mai 21 15:03 Makefile
-rw-r--r--  1 user user  7365 Mai 21 15:03 Makefile.samp.inc
-rw-r--r--  1 user user 20713 Mai 21 15:03 mvDebugFlags.mvd
// <- Log activation flag
drwxr-xr-x  7 user user  4096 Mai 21 15:09 mvDeviceConfigure
drwxr-xr-x  6 user user  4096 Mai 21 15:10 mvIPConfigure
drwxr-xr-x  6 user user  4096 Mai 21 15:11 mvPropView
drwxr-xr-x  9 user user  4096 Mai 21 15:11 SingleCapture
drwxr-xr-x  9 user user  4096 Mai 21 15:11 SingleCaptureStorage
```

For log file generation you have to execute your app from the folder where **mvDebugFlags.mvd** is located. E.g. if you want to start wxPropView:

```
user@linux-desktop:/home/workspace/apps$ ./mvPropView/x86/wxPropView
// <- Start the executable from the folder, where \b mvDebugFlags.mvd is located.
```

Another possibility would be, to copy the **mvDebugFlags.mvd** file to the folder of the executable:

```
user@linux-desktop:/home/workspace/apps$ cp mvDebugFlags.mvd ./mvPropView/x86/wxPropView
// <- Copy the log activation flag
user@linux-desktop:/home/workspace/apps$ cd ./mvPropView/x86/
// <- Change the directory
user@linux-desktop:/home/workspace/apps/mvPropView/x86/$ ./wxPropView
// <- Start the executable
```

Afterwards, several log files are generated which are listed in files.mvloglist. The log files have the file extension .mvlog. Please send these files to our support team.

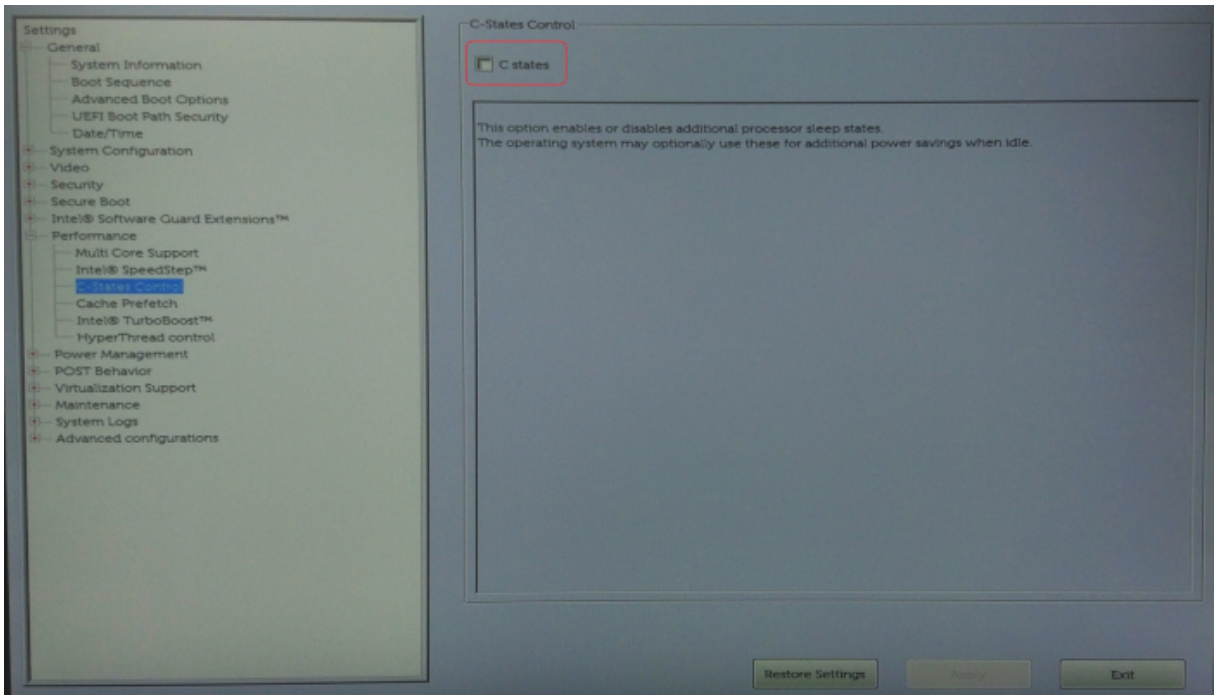
1.16.3 General Issues

- **The error counter increases** (p. 154)
- **I get an oscillating frame rate** (p. 155)
- **Why does updating the device list take so long** (p. 156)

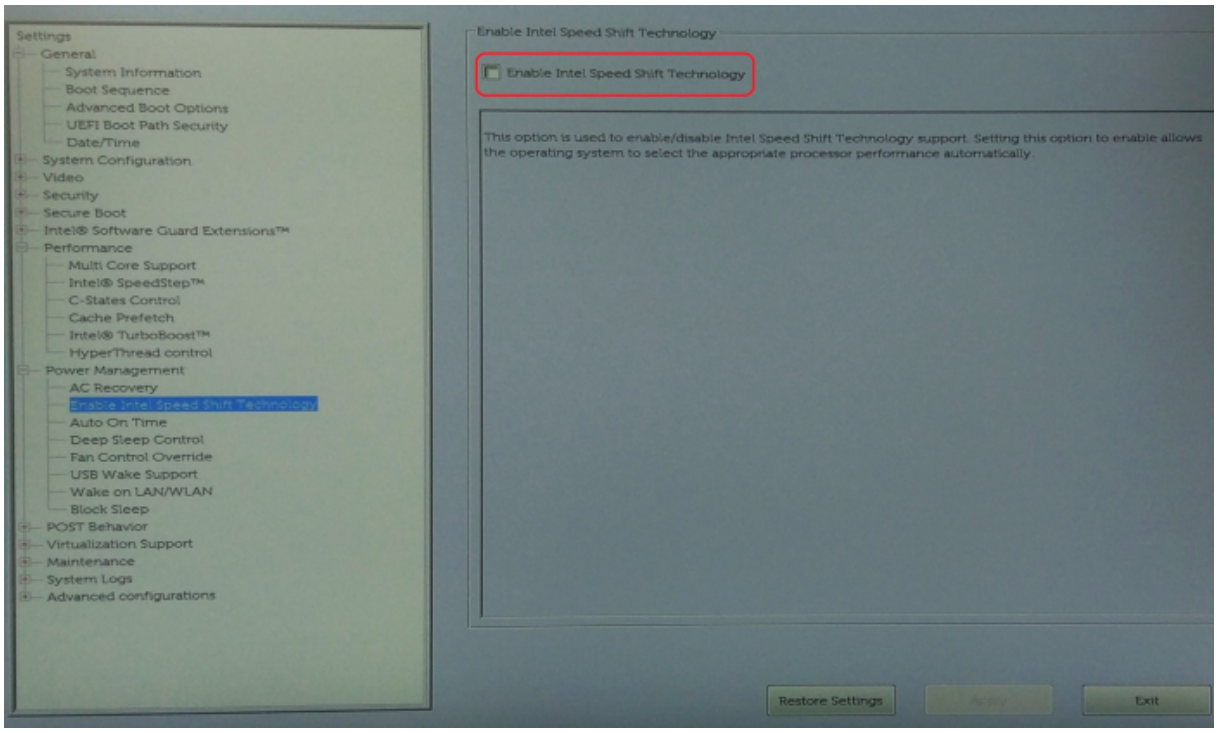
1.16.3.1 The error counter increases

Modern PC's, notebook's, etc. try to save energy by using a smart power management. If processor performance is not needed the processor will change to a power saving (sleep) state automatically and vice versa. Every state change will stop the processor for microseconds. This time is enough to cause image error counts!

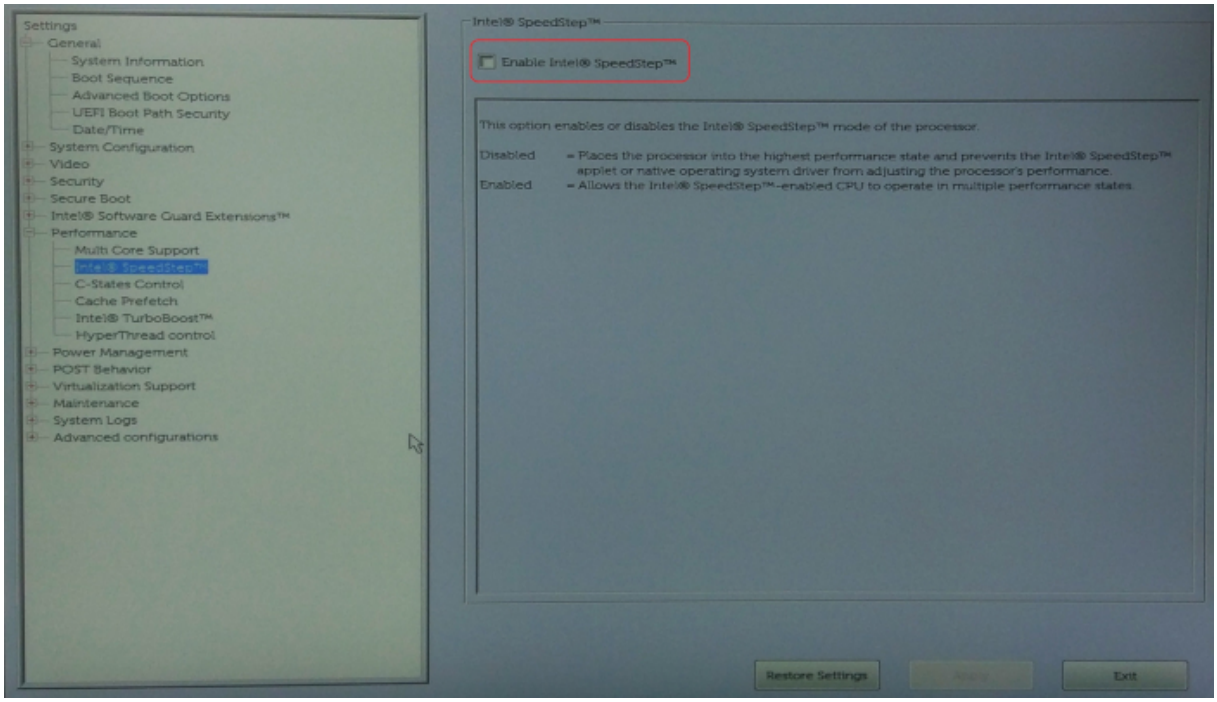
With mvDeviceConfigure running on Windows 7 or below you can disable the CPU sleep states on Windows systems. As the API needed for this has been removed in Windows 8 and above doing the same on Windows 8 or higher or when working with Linux system must be done by using BIOS options. Below are some exemplary screen-shots from BIOS settings. Your system might use different names or look different of course:



Swiching off the C states might improve the situation



Swiching off Intel SpeedShift options might improve the situation



Swiching off Intel SpeedStep options might improve the situation

Note

Another that is always worth considering is to look for an updated version of your systems BIOS! Whenever you encounter problems like described above and you are sure that you have set up the system as perfect as possible (e.g. by configuring network cards of USB interfaces like described in the corresponding sections of the mvBlueFOX3 or mvBlueCOUGAR series) a BIOS update might solve all remaining issues!

1.16.3.2 I get an oscillating frame rate

If your camera supports **FrameRateExactness** it is possible that you may receive an oscillating frame rate. This is due to the fact, that frames of different lengths are used to achieve an overall stable and exact frame rate. We use 10 frames to achieve an more exact frame rate, i.e. if you capture 10 frames, your frame rate will be exact and stable.

Example:

1. Assume we want to achieve a frame rate of 150 fps.
2. This means we need to wait 6666,6667 us until the next frame start.
3. The possible step size is 1 us.
4. If we set 6666 us, the frame rate will be stable, but not exact enough.
5. => 6 frames of length 6666 and 4 frames of length 6667 make the frame rate more exact.

1.16.3.3 Why does updating the device list take so long

Since

mvIMPACT Acquire 2.24.0

The GenTL driver scans both the GigE Vision and the USB3 Vision interfaces for compliant devices. Given that the GigE Vision standard allows detection timeouts of 1000 ms, it could take long, especially for USB3 Vision users which even do not have GigE Vision in their application, until the device update is finished.

For this reason, we upgraded the **MATRIX VISION application tools** (p. 99) to excluded interfaces or single devices to be scanned or detected. The easiest way to do so is as follows:

1. Open **wxPropView** (p. 99) .
2. Open the device list drop down and
3. select **"Missing A Device? Click here..."**.

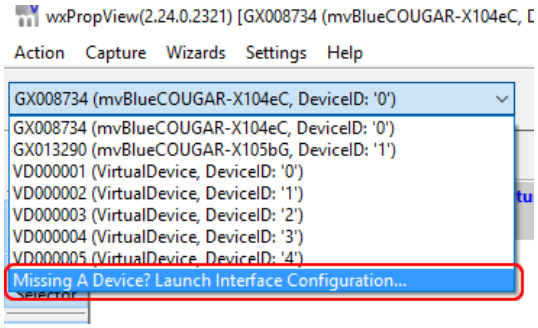


Figure 1: "Missing A Device? Click here..."

Now, you will see the interfaces and the detected device.

4. Finally, you can deselect the interfaces or devices which you do not want to scan for updating the device list. The following figure shows that the GigE Vision interface should be skipped:

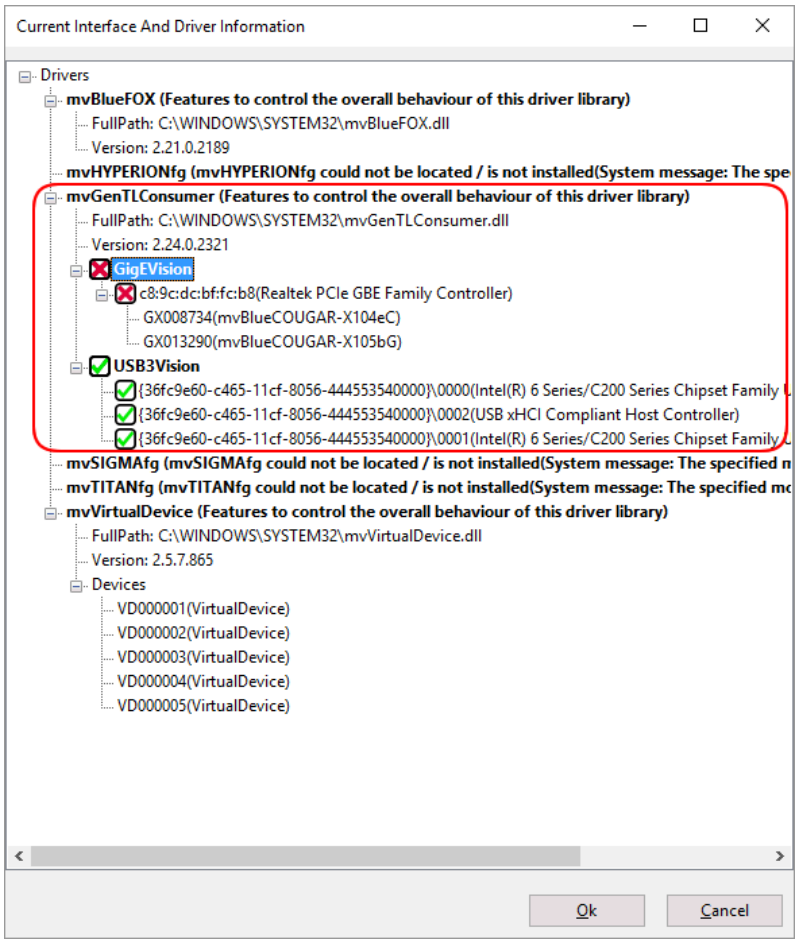


Figure 2: Skip the GigE Vision interface

1.16.4 Windows

- **Calling AcquisitionStart For The First Time In A Process Takes A Longer Time** (p. 157)
- **mvBlueFOX3 or other USB3 Vision devices connected to the system are not detected by mvIMPACT Acquire or cannot be initialised** (p. 158)
- **After installing a new driver version or after initially installing the mvGenTL_Acquire package USB3 Vision devices are not accessible** (p. 162)

1.16.4.1 Calling AcquisitionStart For The First Time In A Process Takes A Longer Time

Symptoms

Calling **the acquisition start function (syntax depending on the used programming language)** takes longer than expected. Especially the first time after the resulting image buffer layout has been changed / the buffer size did increase after applying various changes.

Cause

The internal acquisition engine does not allocate memory until it is actually needed which usually is upon calling `acquisitionStart` for the first time after applying changes affecting (increasing) the resulting buffer e.g. due to AOI changes. This memory allocation time is only consumed when an application doesn't provide it's own custom memory (see the `CaptureToUserMemory` example for the programming language of your choice).

Also the mapping of the virtual addresses of the pages forming the full buffer to physical ones inside the kernel driver does only happen once per buffer if the buffer layout stays the same in between acquisition loops in order to save time.

Both operations are expensive in terms of time consumption for time sensitive applications thus are reduced to a minimum but can still be annoying under certain circumstances.

Resolution

Once `acquisitionStart` is called, the internal buffer management needs to allocate the required memory for all buffers before the acquisition engine can start. Since last minute modifications might change the buffer layout it doesn't make too much sense to do this earlier. Afterwards the buffers must be mapped in kernel space to allow zero copy transfer. Especially this mapping takes time and this time directly relates to the number of request objects queued before starting the acquisition and the size of each request buffer.

Two things can be done to reduce the time needed to start the acquisition engine. Both approaches may be combined to achieve optimal performance:

Pre-Allocation Of Buffers

This will eliminate the extra time needed to start the acquisition the first time after changing the buffer layout!

Configure the camera to the maximum possible AOI and fill the request queue by calling `imageRequestSingle` as many times as request objects are specified by the requestCount property. The acquisition engine will allocate the required memory for each request object. Then start and stop the acquisition engine and unlock all request objects afterwards and adjust all settings as required for the 'real' application.

> **Note**
>
> This procedure will only work in the same process and only until the driver instance is closed within this process. Once the camera driver is closed it has to be repeated!

Don't Queue All Requests Before Starting The Acquisition

Since additional buffers can be queued at any time a significant speedup can be achieved by queueing less requests before starting the acquisition. If not needed it might be a good idea to work with less request objects anyway to reduce the overall memory footprint of the application. However if e.g. 10 request are needed a speedup can be achieved by queueing e.g. 3 requests, then call acquisition start and then queue the remaining requests before starting to wait for the first image to become ready.

1.16.4.2 mvBlueFOX3 or other USB3 Vision devices connected to the system are not detected by mvIMPACT Acquire or cannot be initialised

Symptoms

Tools like wxPropView list mvBlueFOX devices connected to the system but their state is set to *Unreachable* or trying to open the device fails. In rare cases devices are not even displayed.

Cause

A **"USB3 Vision"** camera is part of the bus or system to which it is connected. Like other devices (e.g. PCIe card, mouse, etc.) a **"USB3 Vision"** device is bound to the system via a driver. Because a device can only be bound to one driver in the system, it is possible that the device driver is bound to another **"USB3 Vision"** driver which is available on the system.

In this case it is possible that you cannot see the mvBlueFOX3 or third party USB3 Vision™ device using MATRIX VISION tools like **wxPropView** or you can see it but you cannot use it. A dialog like the following might appear:

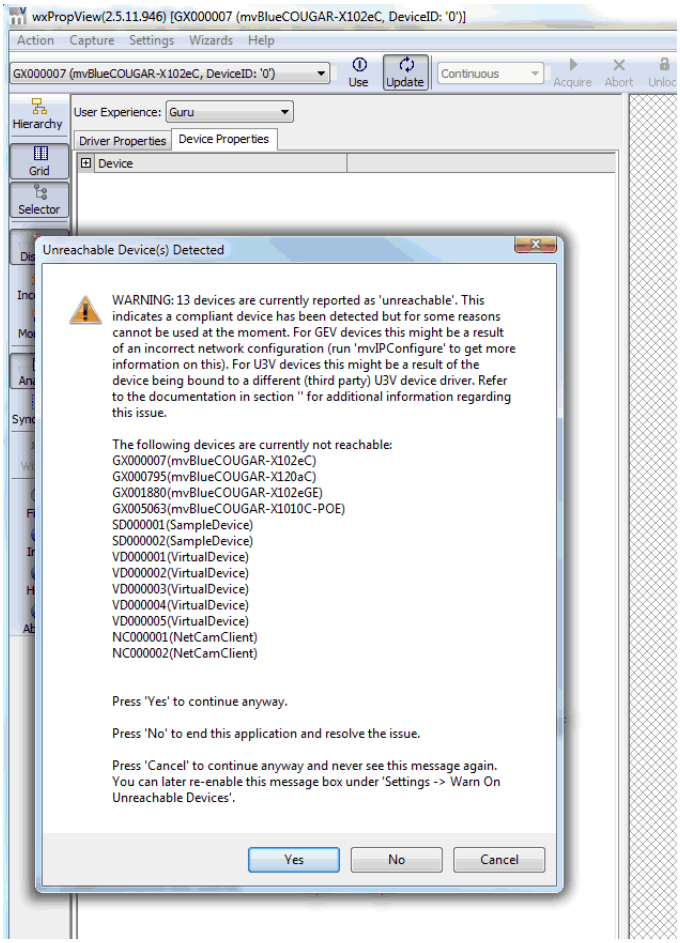


Figure 1: Unreachable devices

Resolution

The driver binding can be changed using the Windows *"Device Manager"*:

Note

Please be sure, that you have already installed the **"mvGenTL_Acquire driver"**.

- Open the *"Device Manager"* from "`Control Panel -> System -> Device Manager`". For example on this system, an mvBlueFOX3 is bound to the "`NI Acquisition Device`" driver

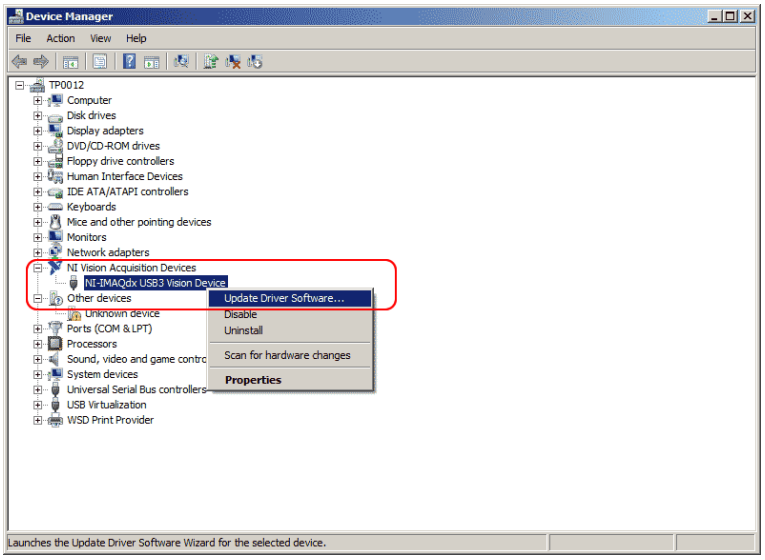


Figure 2: mvBlueFOX3 as a NI Acquisition Device

- Right-click the device and select "`Update Driver Software...`".
- Click on "`Browse my computer for driver software`":

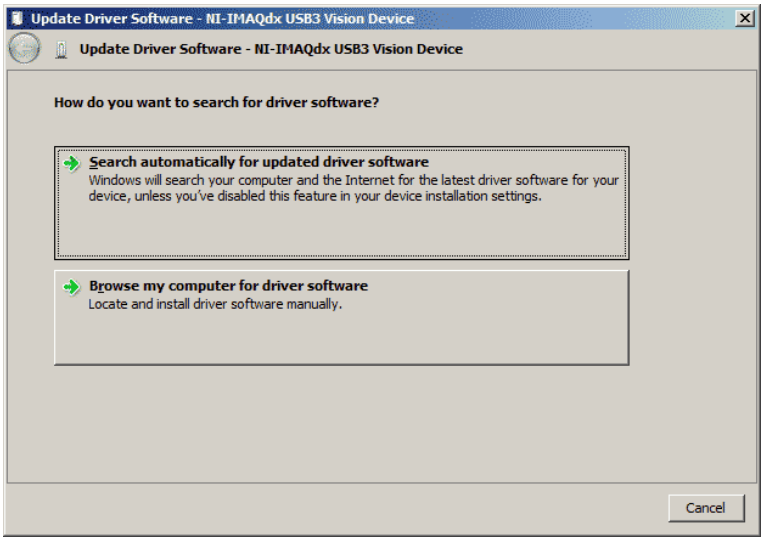


Figure 3: Browse my computer for driver software

- Click on "`Let me pick from a list of device drivers on my computer`":

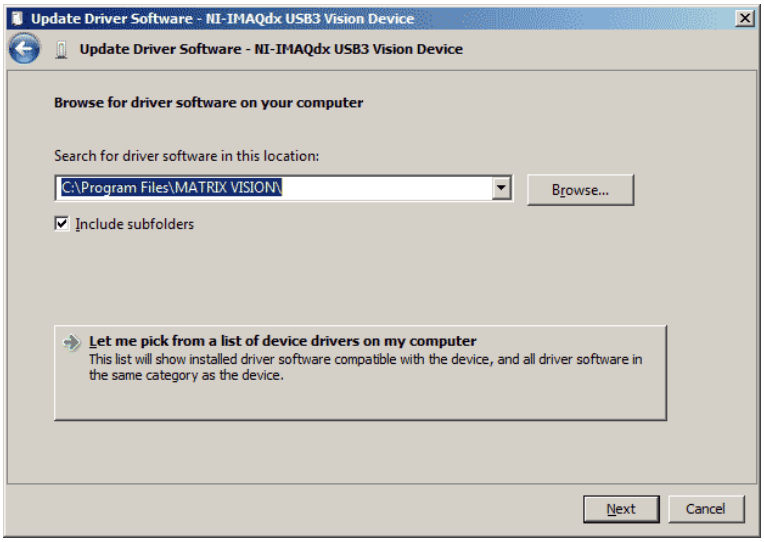


Figure 4: Let me pick from a list of device drivers on my computer

- Select the MATRIX VISION USB3 Vision driver called "`USB3 Vision Device(Bound to MATRIX VISION !GmbH driver using libusbK)`" and click on "`Next`":

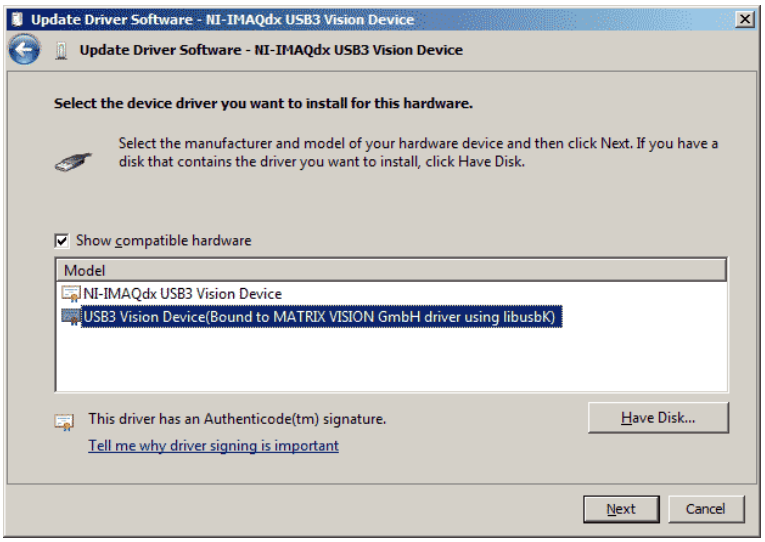


Figure 5: Select the MATRIX VISION driver

Afterwards, the driver binding will be changed.

- Finally, click on "`Close`":

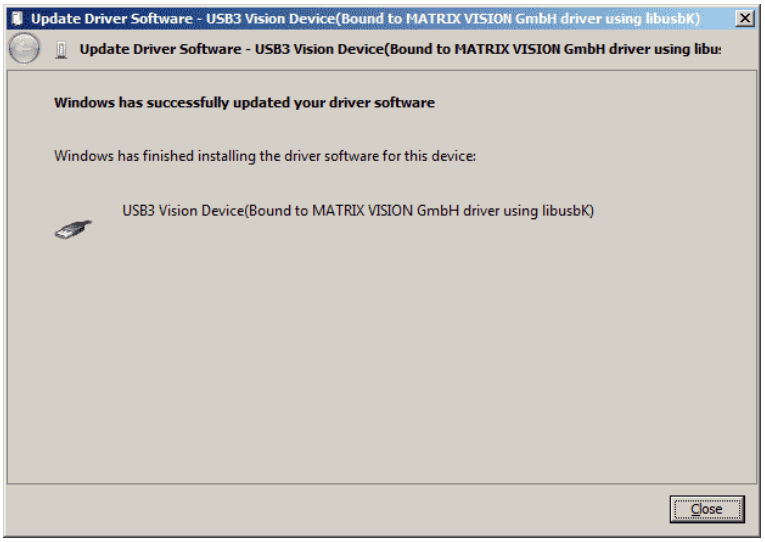


Figure 6: Driver update was successful

- Now the mvBlueFOX3 device should be bound to the MATRIX VISION driver via `libusbK`.

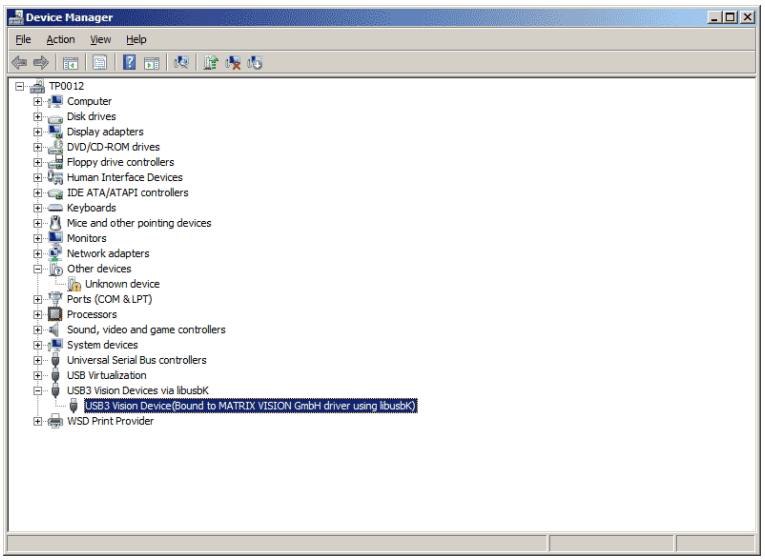


Figure 7: mvBlueFOX3 is bound to the MATRIX VISION driver via libusbK

1.16.4.3 After installing a new driver version or after initially installing the mvGenTL_Acquire package USB3 Vision devices are not accessible

Symptoms

After installing mvGenTL Acquire or Updates on Windows 10 mvBlueFOX3 or any USB3 Vision™ cameras are not recognized as **USB3** Vision Device(Bound to MATRIX VISION GmbH driver using libusbK) device. Once a GUI or example application e.g. wxPropView is used the camera won't show up and the corresponding driver libraries (libusb0.dll, libusbK.dll, libusbK.sys) at *SystemRoot%/system32* and *SystemRoot%/system32* are missing.

Cause

The root cause is that during the installation the digital signatures of the driver files are verified by contacting external servers if an active Internet connection is available.

Resolution

Several things can be done to overcome potential problems:

- Disconnect the system from Internet before starting an application which accesses the camera for the first time and reconnect it afterwards.
- Since the driver installation will be executed in the background once the first application tries to use it you can simply start the application a 2nd time.

1.16.5 Linux

- **No GenICam devices are detected on a Linux system** (p. 163)
- **Image transfer From USB3 Vision™ Devices Stops Randomly On A Linux System** (p. 164)

1.16.5.1 No GenICam devices are detected on a Linux system

Symptoms

After starting any mvIMPACT Acquire based application no GenICam compliant devices are accessible or detected

Cause

The environment variables which are necessary to us the mvGenTL Acquire package are defined by two shell scripts which are set by profile.d. This causes the variables to be available within every login-shell run on the system. Once an application is started from a non-login shell which is using mvIMPACT Acquire libraries the variables from profile.d are not used. This results in a situation that the necessary runtime libraries can not be found which leads to missing interfaces and devices.

Resolution

To avoid this situation there are two possible solutions:

- Use a login-shell to run the application using the MATRIX VISION driver package.
- Insert the variables from: /etc/profile.d/genicam.sh and /etc/profile.d/acquire.sh to a shell configuration file which is read to configure environment variables.

1.16.5.2 Image transfer From USB3 Vision™ Devices Stops Randomly On A Linux System

Symptoms

When operating one or multiple USB3 Vision™ devices on a Linux system after a random time images can no longer be acquired.

Running

```
sudo journalctl
```

with or without additional option afterwards displays messages similar to this:

```
May 05 17:21:30 svc-527521 kernel: xhci_hcd 0000:00:14.0: WARN Set TR Deq Ptr cmd failed due to incorrect slot
May 05 17:21:30 svc-527521 kernel: xhci_hcd 0000:00:14.0: WARN Event TRB for slot 10 ep 6 with no TDs queued?
```

Running commands like

```
uname -srm
```

shows a kernel version of 4.19 or at least smaller than 5.10.

Cause

The exact reasons for this effect are unknown but there is a good chance that a bug in the Linux kernel is responsible for the behaviour:

- `https://bugzilla.kernel.org/show_bug.cgi?id=202541`
- `https://bbs.archlinux.org/viewtopic.php?id=230764` and `https://bbs.archlinux.org/viewtopic.php?id=230362`

Depending on the number of cameras used in parallel, the way these cameras are operated in terms of data throughput and trigger behaviour as well as other conditions like used host controllers it can take anything between 5 minutes and several hours to encounter this effect. In most scenarios it will not show at all. Digging through the Internet seems to show that a combination of kernel version, host controller chip-set and maybe even the USB chip used inside the device have an impact.

- Version 4.19 of the Linux kernel was released in October 2018: Here the issue could be observed, earlier versions might be affected
- Version 5.10 of the Linux kernel was released in December 2020: Here the issue could **NOT** be observed anymore, versions greater than 4.19 but smaller than 5.10 might be fine as well

The issue somehow seems to be related to the `soft retry` mechanism implemented for xHCI controllers.

Note

If you can provide more details regarding this please let us know!

Resolution

Update to a Linux kernel or 5.10 or above!

1.17 Glossary

ADC	Analog-to-digital converter (A/D converter).
API	Application programming interface (API). The standard API for MATRIX VISION products is called **mvIMPACT_Acquire** (p. 171).
BusyBox	Configurable monolithic application including shell and other useful command line tools - often called the "swiss army knife" for embedded systems. Even desktop distributions are sometimes relying on BusyBox due to its robustness. Please see `http://www.busybox.net` for details.
CIFS	Common Internet file system (CIFS) replaced Samba in 2006. It gets rid of NetBIOS packets an introduced Unix features like soft/hard links and allows larger files.
CPU	Central processing unit aka processor.
DAC	Digital to analog converter (D/A converter).
Defaults	Standard system settings.
DHCP	Dynamic Host Configuration Protocol (DHCP). DHCP is a protocol used by networked devices (clients) to obtain various parameters necessary for the clients to operate in an Internet Protocol (IP) network.
Digital I/O	Digital inputs and outputs.
GDB	GDB, the GNU Project debugger.

GenICam	GenICam stands for **GEN**eric programming **I**nterface for **CAM**eras. It's a generic way to access and modify device parameters with a unified interface. A GenICam compliant device either directly provides a GenICam compliant description file (in internal memory) or the description file can be obtained from a local (hard disk etc.) or web location. A GenICam description file is something like a machine readable device manual. It provides a user readable name and value range for **parameters** (p. 173) that are offered by the device for reading and/or writing and instructions on what command must be sent to a device when the user modifies or reads a certain parameter from the device. These description files are written in XML. An excerpt from such a file can be seen in the figure below:
	
	`<Float Name="GainAbs" NameSpace="Standard">`
	`  <ToolTip>Sets the camera's gain in dB</ToolTip>`
	`  <Description>This float value sets the camera's gain in dB.</Description>`
	`  <DisplayName>Gain (Abs)</DisplayName>`
	`  <pIsLocked>TLParamsLocked</pIsLocked>`
	`  <pValue>GainAbs_CtrlValueFao</pValue>`
	`  <Unit>dB</Unit>`
	`  <Representation>Linear</Representation>`
	`</Float>`
	`<FloatReg Name="GainAbs_CtrlValueFao" NameSpace="Custom">`
	`  <Address>0x00000004</Address>`
	`  <pAddress>CamCtrl_AnalogCtrlBaseAddress</pAddress>`
	`  <Length>8</Length>`
	`  <AccessMode>RW</AccessMode>`
	`  <pPort>Device</pPort>`
	`  <Endianess>BigEndian</Endianess>`
	`</FloatReg>`
	
	Excerpt of a GenICam description file (in XML)
	For further information on this topic please have a look at `https://en.wikipedia.org/wiki/GenICam`.

GenTL	GenTL is the transport layer interface for cameras, acquiring images from the camera, and moving them to the user application.

Gigabit Ethernet (GigE)	The term Gigabit Ethernet (defined by the IEEE 802.3-2008 standard) represents various technologies for transmitting Ethernet frames at a rate of a gigabit per second (1,000,000,000 bits per second).

GigE Vision

GigE Vision is a network protocol designed for the communication between an imaging device and an application. This protocol completely describes:

- device discovery
- data transmission
  - image data
  - additional data
- read/write of parameters.

GigE Vision uses UDP for data transmission to reduce overhead introduced by TCP.

Note

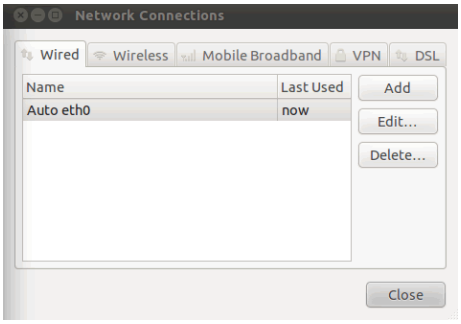
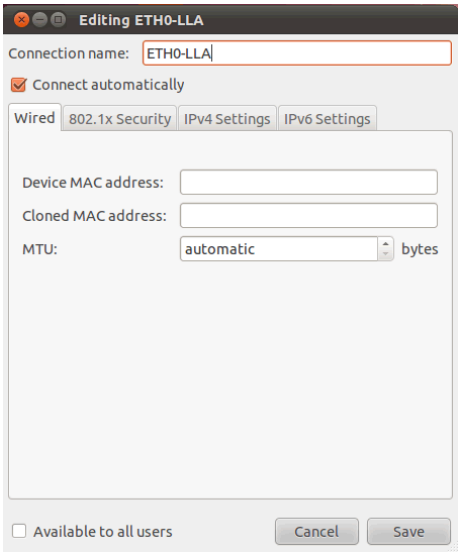
UDP does not guarantee the order in which packets reach the client nor does it guarantee that packets arrive at the client at all. However, GigE Vision defines mechanisms that can detect lost packets. This allows capture driver manufacturers to implement algorithms that can reconstruct images and other data by requesting the device to resend lost data packets until the complete buffer has been assembled. For further information please have a look at `https://en.wikipedia.org/wiki/GigE_Vision` The MATRIX VISION GigE Vision capture filter driver as well as the socket based acquisition driver and all MATRIX VISION GigE Vision compliant devices support resending thus lost data can be detected and in most cases reconstructed. This of course can not enhance the max. bandwidth of the transmission line thus if e.g. parts of the transmission line are overloaded for a longer period of time data will be lost anyway.

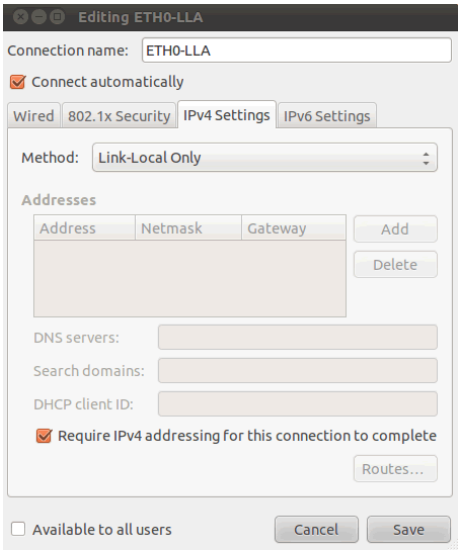
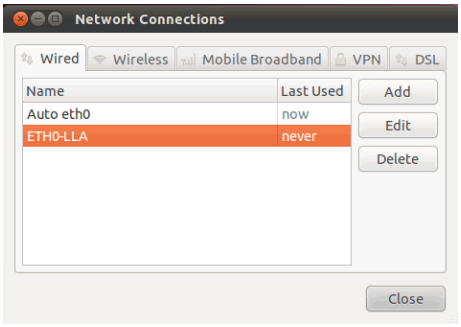
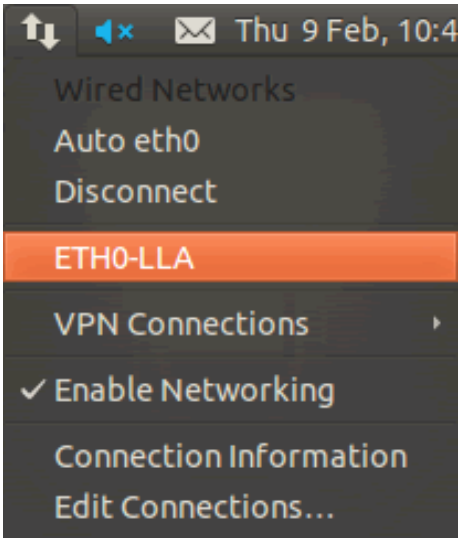
Both capture drivers will allow to fine tune the resend algorithm used internally and both drivers will also provide information about the amount of data lost and the amount of data that was re-requested. This information/configuration will be part of the drivers SDK. More information about it can be found in the corresponding interface description.

Note

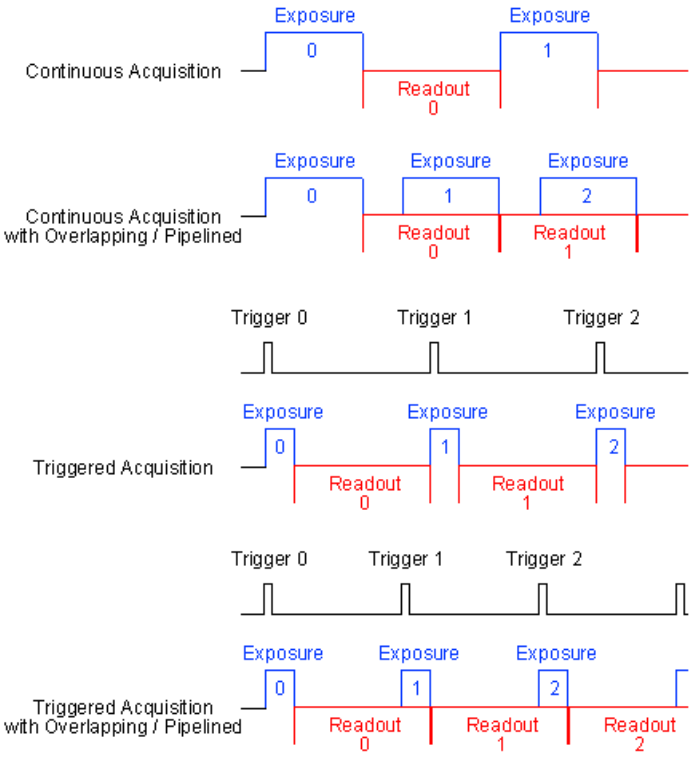
On Windows 2000 the filter driver does not support the **"Resend"** mechanism.

High Dynamic Range (HDR)	The HDR (High Dynamic Range) mode increases the usable contrast range. This is achieved by dividing the integration time in two or three phases. The exposure time proportion of the three phases can be set independently. Furthermore, it can be set, how many signal of each phase is charged.
HRTC	With a *Hardware Real-Time Controller* (HRTC) built inside the FPGA users can define a PLC like sequence of operating steps to control basic time critical functions like exposure time, image trigger and I/O ports. Timing is hard real-time with sub microsecond high resolution.
IDE	a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of a source code editor, a compiler and/or interpreter, build automation tools, and (usually) a debugger.
IPKG	Itsy package management system originally designed for embedded systems. Please have a look at `https://en.wikipedia.org/wiki/Ipkg` or a more sophisticated documentation at `http://buffalo.nas-central.org/index.php` "Overview_of_the_ipkg_package_management_system" MATRIX VISION distributes all non-firmware, i.e. optional software as ipk packages.
JFFS2	JFFS2 is a file system which supports wear leveling. **See also** Sources about the JFFS file system: • `http://sources.redhat.com/jffs2/` • `http://www.linux-mtd.infradead.org/faq/jffs2.html`
Link Aggregation (LAG)	Dual-GigE cameras need a network interface card with two network interfaces. However, both network interfaces have to work as a unit. Link aggregation (LAG) or bonding is the name of the game and has to be supported by the network interface card's driver. With it you can bond the two network interfaces so that the work as one interface.

LLA	Logical link address (LLA) is a type of mechanism to obtain a valid IP address without a DHCP server being present. Whether an IP address is available or not is resolved using address resolution protocol (ARP) packets. If no ARP response is received on a given address it is considered unused and will be assigned to the interface. LLA space is 169.254.x.y, i.e. 16bit netmask yielding 64K possible device addresses. With Linux you have to add LLA as an additional interface. By default, you can find one interface in *Connections:* (This description uses *"Gnome Network Manager"*, however using KDE should be similar)  In *Wired*, you can add interfaces via **Add:** 

	In the tab *"IPv4 Setting"* you have to set **"Link-Local Only"**:  After saving, you will find both connections in the summary:  Now, you can select the wished connection using the left mouse button in the *"Network Manager"* menu. In the LLA case it is just the new created connection: 
MAC address	Media Access Control address (MAC address) is a quasi-unique identifier attached to most network adapters (NICs) in computer networking.

MTU	Maximum transmission unit (MTU) refers to the size (in bytes) of the largest packet that a given layer of a communications protocol can pass onwards. The default MTU for Ethernet is 1500. The optimum for Gigabit Ethernet is 8000 - 12000. Different MTU settings in the same subnet can cause package losses, i.e. **never ever** change the MTU unless you know what you're doing. Changing the MTU to other values than 1500 when using file or web services from other computers are almost always a bug. It's save to increase MTU when working in peer-to-peer mode with both devices sharing the same MTU. Please not that few network cards support 16K, most Gigabit Ethernet cards are limited to 9k, some don't support Jumbo Frames (MTU $>$ 1500) at all.
mvIMPACT Acquire	This driver supplied with MATRIX VISION products represents the port between the programmer and the hardware. The driver concept of MATRIX VISION provides a standardized programming interface to all image processing products made by MATRIX VISION GmbH. The advantage of this concept for the programmer is that a developed application runs without the need for any major modifications to the various image processing products made by MATRIX VISION GmbH. You can also incorporate new driver versions, which are available for download free of charge on our website: `https://www.matrix-vision.com`. The developer interface description of the mvIMPACT Acquire is called • mvIMPACT_Acquire_API_CPP_manual.chm • mvIMPACT_Acquire_API_C_manual.chm • mvIMPACT_Acquire_API_NET_manual.chm and can be downloaded from our website.
Netboot	With netboot you can boot mvBlueCOUGAR over network. This is especially useful when several devices share the same pieces of software, i.e. same root file system, which might be subject to change frequently.
NFS	Network File System (NFS) is a network file system protocol, allowing clients to access files over LAN. Given that you need a NFS server are uncommon on Windows, this protocol best fits for Linux-Linux connections.
NIC	Network interface card - synonym for network controller.

Overlapped / pipelined transfer	By default, the steps exposure and readout out of an image sensor are done one after the other. • By design, CCD sensors support overlap capabilities also combined with trigger (see figure). • In contrast, so-called pipelined CMOS sensors only support the overlapped mode. Even less CMOS sensors support the overlapped mode combined with trigger. Please check the **sensor summary** (p. 83). In overlapping mode, the exposure starts the exposure time earlier during readout.  **Note** In overlapped trigger mode, you have to keep in mind the following formula `interval between two trigger events >= (readout time - exposure time)`
PSE	Power sourcing equipment. The network PoE element that inserts power onto an Ethernet cable.
ROI/AOI	Region/Area of interest.
SDK	Sofware development kit (SDK). The standard image processing software library from MARTIX VISION is **mvIMPACT**.

SFNC	**S**tandard **F**eature **N**aming **C**onvention of **GenICam** (p. 165). **See also** The latest GenICam properties list can be found here: **http://www.emva.org/standards-technology/genicam/genicam-downloads/** The file is called "GenICam Standard Features Naming Convention (PDF)"
Shell	In computing, a shell is a piece of software that provides an interface for users. Command-line shells provide a command-line interface (CLI) to the operating system. The primary purpose of the shell is to invoke or "launch" another program; however, shells frequently have additional capabilities such as viewing the contents of directories.
USB3 Vision	A closed source framework, defined and administered by the Automated Imaging Association (AIA), for transmitting video and related control data over USB 3. Sometimes U3V is used as an acronym.
UDP	The User Datagram Protocol (UDP) is an Internet protocol. It is used by applications to send messages to other hosts on an Internet Protocol (IP) network.
Virtual Network Computing (VNC)	Virtual Network Computing (VNC) is a graphical desktop sharing system that uses the RFB protocol to remotely control another computer. Over a network, it transmits the mouse and keyboard events from one computer to another, relaying the graphical screen updates back in the other direction. To access the camera's desktop from a PC via VNC, • you have to know the IP address of the remote system. • Start a VNC viewer and • point it to the remote system. You won't need a password. Of course, you won't get a very fast live image display via the network with VNC but you should be able to start **wxPropView** (p. 99) and capture images.
wxWidgets	wxWidgets is a cross-platform GUI library. It can be used from languages such as C++, Python, Perl, and C#/.NET. **See also** http://www.wxwidgets.org

1.18 Use Cases

- **Introducing acquisition / recording possibilities** (p. 175)
- **Improving the acquisition / image quality** (p. 228)
- **Improving the communication** (p. 275)
- **Working with triggers** (p. 276)
- **Working with I/Os** (p. 295)
- **Working with HDR (High Dynamic Range Control)** (p. 302)
- **Working with LUTs** (p. 308)
- **Saving data on the device** (p. 316)
- **Working with device features** (p. 325)
- **Working with several cameras simultaneously** (p. 339)
- **Working with 3rd party tools** (p. 345)

1.18.1 Introducing acquisition / recording possibilities

There are several use cases concerning the acquisition / recording possibilities of the camera:

- **Acquiring a number of images** (p. 175)
- **Recording sequences with pre-trigger** (p. 176)
- **Creating acquisition sequences (Sequencer Control)** (p. 178)
- **Generating very long exposure times** (p. 191)
- **Working with multiple AOIs (mv Multi Area Mode)** (p. 193)
- **Working with burst mode buffer** (p. 197)
- **Using the SmartFrameRecall feature** (p. 201)
- **Using The Linescan mode** (p. 204)
- **Using the mvBlockscan feature** (p. 208)
- **Working with Event Control** (p. 210)
- **Polarized Data Extraction** (p. 213)
- **Using Video Stream Recording** (p. 217)

1.18.1.1 Acquiring a number of images

As described in chapter **Acquisition Control** (p. 103), if you want to acquire a number of images, you can use as *"Setting -> Base -> Camera -> Acquisition Control -> Acquisition Mode"* **"MultiFrame"** and you have to set the **"Acquisition Frame Count"**.

Afterwards, if you start the acquisition via the button *"Acquire"*, the camera will deliver the number of images.

The **"MultiFrame"** functionality can be combined with an external signal to start the acquisition.

There are several ways and combinations possible, e.g.:

- A trigger starts the acquisition (Figure 1).
- A trigger starts the acquisition start event and a second trigger starts the grabbing itself (Figure 2).

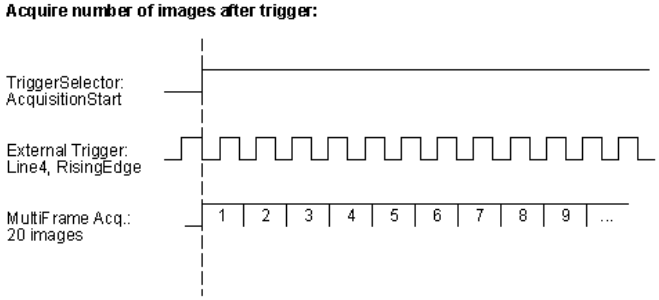


Figure 1: Starting an acquisition after one trigger event

For this scenario, you have to use the *"Setting -> Base -> Camera -> Acquisition Control -> Trigger Selector"* **"AcquisitionStart"**.

The following figure shows, how to set the scenario shown in Figure 1 with **wxPropView** (p. 99)

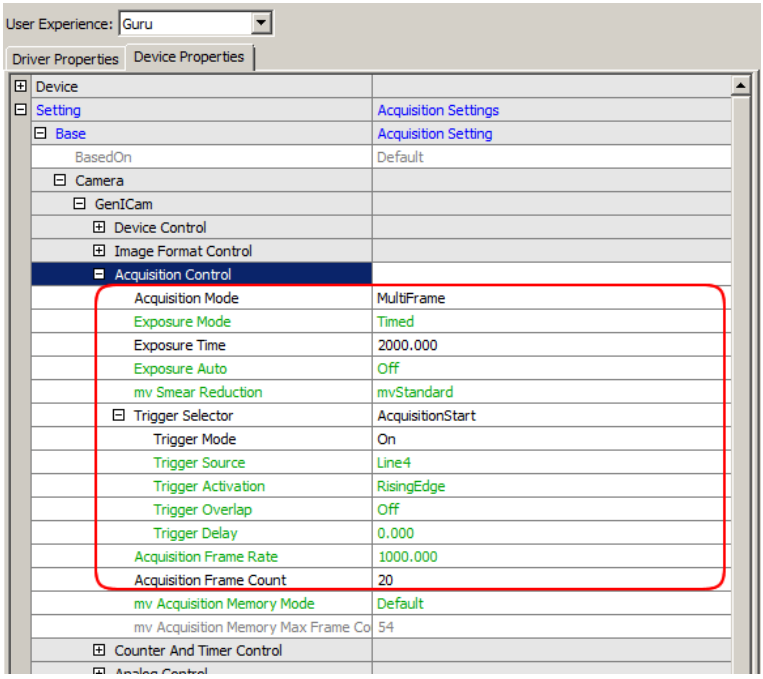


Figure 2: wxPropView - Setting acquisition of a number of images started by an external signal

A rising edge at line 4 will start the acquisition of 20 images.

1.18.1.2 Recording sequences with pre-trigger

1.18.1.2.1 What is pre-trigger?

With pre-trigger it is possible to record frames before and after a trigger event.

1.18.1.2.2 How it works

To use this functionality you have to set the **mv Acquisition Memory Mode** (p. 103) to **"mvPretrigger"**. This can be used to define the number of frames which will be recorded before the trigger event occurs:

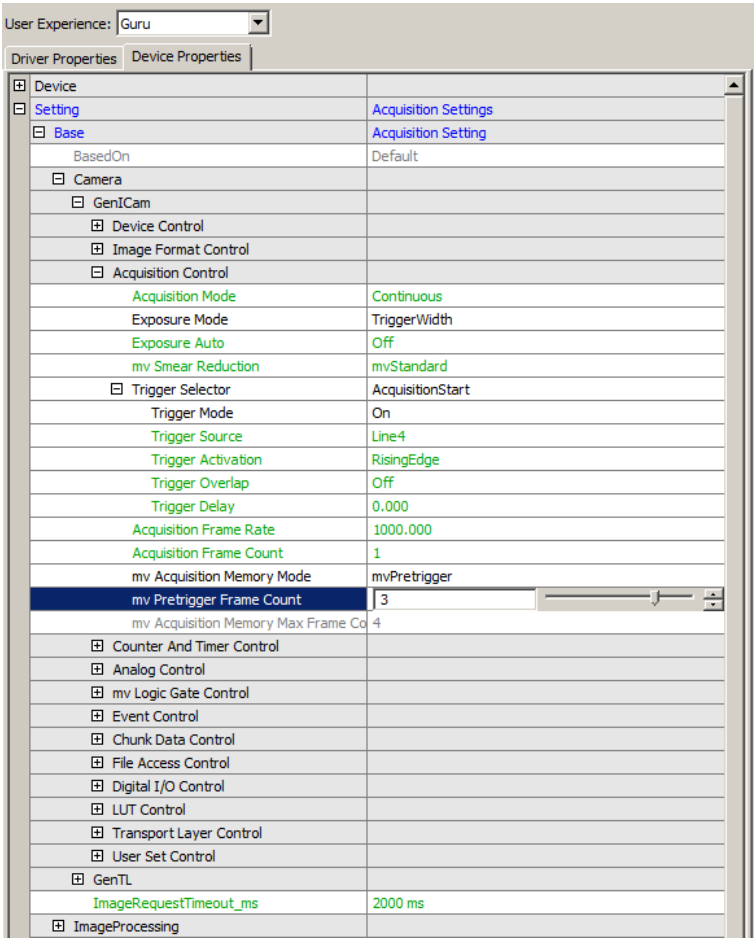


Figure 1: wxPropView - setting pre-trigger

Afterwards, you have to define the **"AcquisitionStart"** or **"AcquisitionActive"** event. In figure 1 this is **Line4** as trigger event, which starts the regular camera streaming.

Now, start the camera by pressing **"Live"** and generate the acquisition event.

The camera will output the number of pre-trigger frames as fast as possible followed by the frames in live mode as fast as possible until the frame rate is in sync:

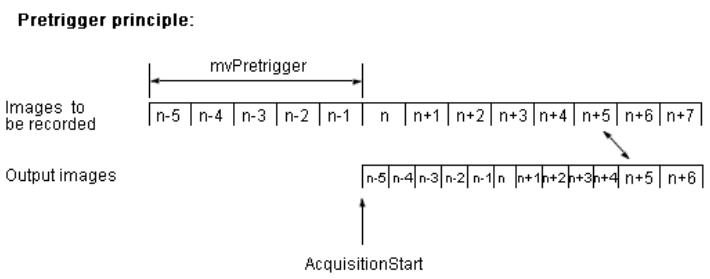


Figure 2: wxPropView - recorded / output images

1.18.1.3 Creating acquisition sequences (Sequencer Control)

1.18.1.3.1 Introduction

As mentioned in **GenICam And Advanced Features** section of this manual, the **Sequencer Mode** (p. 116) is a feature to define feature sets with specific settings. The sets are activated by a user-defined trigger source and event.

Note

At the moment, the Sequencer Mode is only available for MATRIX VISION cameras with CCD sensors and Sony's CMOS sensors. Please consult the **"Device Feature and Property List"s** to get a summary of the actually supported features of each sensor.

The following features are currently available for using them inside the sequencer control:

Feature	Note	Changeable during runtime
BinningHorizontal		-
BinningVertical		-
CounterDuration	Can be used to configure a certain set of sequencer parameters to be applied for the next `CounterDuration` frames.	since Firmware version 2.15
DecimationHorizontal		-
DecimationVertical		-
ExposureTime		since Firmware version 2.15
Gain		since Firmware version 2.15
Height		since Firmware version 2.36
OffsetX		since Firmware version 2.35
OffsetY		since Firmware version 2.35
Width		since Firmware version 2.36
mvUserOutput		-
UserOutputValueAll		-
UserOutputValueAllMask		-
Multiple conditional sequencer paths		-

The **Sequencer Control** (p. 116) uses **Counter And Timer Control** (p. 107) Counter1 to work. If you already preset **Counter1** and you save a new acquisition sequence, the settings of Counter1 will be overwritten.

Note

Configured sequencer programs are stored as part of the **User Sets** (p. 318) like any other feature.

1.18.1.3.2 Creating a sequence using the Sequencer Control in wxPropView

In this sample, we define an acquisition sequence with five different exposure times on the device whereby the last step should be repeated five times. We also activate the digital outputs 0..3 to the sets accordingly - this, for example, can be used as flash signals. All the configuration is done on the device itself so after finishing the configuration and starting the acquisition the device itself will apply the parameter changes when necessary. The host application only needs to acquire the images then. This results in a much faster overall frame rate compared to when applying these changes on a frame to frame basis from the host application.

- 1000 us

- 5000 us
- 10000 us
- 20000 us
- 50000 us (5x)

This will result in the following flow diagram:

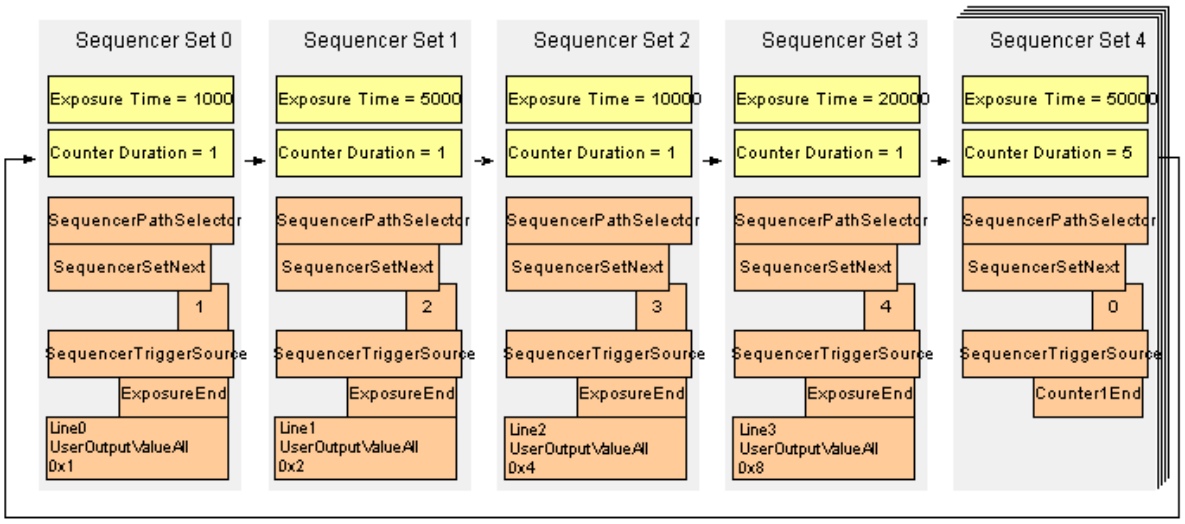


Figure 1: Working diagram of the sample

As a consequence the following exposure times will be used to expose images inside an endless loop once the sequencer has been started:

- Frame 1: 1000 us
- Frame 2: 5000 us
- Frame 3: 10000 us
- Frame 4: 20000 us
- Frame 5: 50000 us
- Frame 6: 50000 us
- Frame 7: 50000 us
- Frame 8: 50000 us
- Frame 9: 50000 us
- Frame 10: 1000 us
- Frame 11: 5000 us
- Frame 12: 10000 us
- ...

So the actual sequence that will be executed on the device later on will be like this

```
while( sequencerMode == On )
{
  take 1 image using set 0
  take 1 image using set 1
  take 1 image using set 2
  take 1 image using set 3
  take 5 images using set 4
}
```

- There are 2 C++ examples called **GenICamSequencerUsage** and **GenICamSequencerParameter↩ ChangeAtRuntime** that show how to control the sequencer from an application. They can be found in the `Examples` section of the `mvIMPACT Acquire C++ API`

The following steps are needed to configure the device as desired:

Note

This is the **SFNC** (p. 173) way how to create an acquisition sequence and consequently how you have to program it. However, **wxPropView offers a wizard** (p. 188) to define an acquisition sequence in a much easier way.

1. First, switch into the *"Configuration Mode"*: *"Sequencer Configuration Mode"* = **"On"**. Only then the sequencer on a device can be configured.

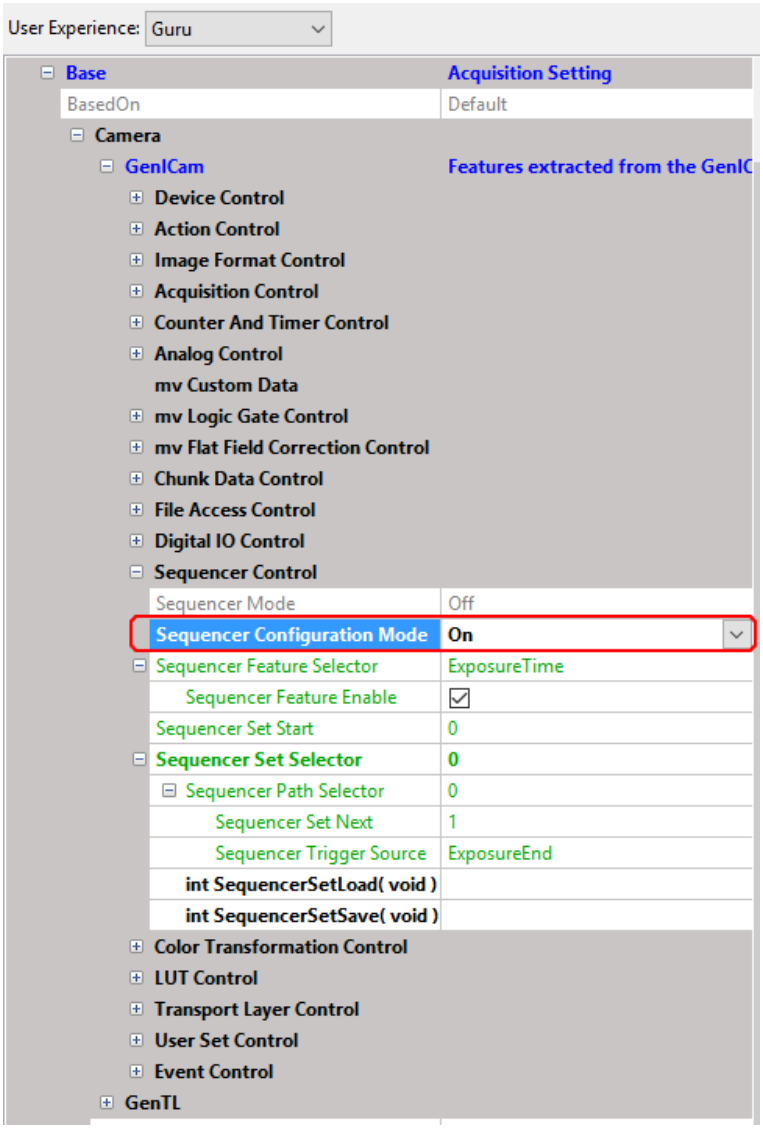


Figure 2: wxPropView - Sequencer Configuration Mode = On

2. Set the *"Sequencer Feature Selector"*, if it is not already active (pink box, figure 3): *"Sequencer Feature Selector"* = **"ExposureTime"** and *"Sequencer Feature Enable"* = **"1"**.

3. Set the *"Sequencer Feature Selector"* for the duration counter (pink box, figure 4): *"Sequencer Feature Selector"* = **"CounterDuration"** and *"Sequencer Feature Enable"* = **"1"**.

4. Set the *"Sequencer Feature Selector"* for using the UserOutputs: *"Sequencer Feature Selector"* = **"mvUser↩ Output"** and *"Sequencer Feature Enable"* = **"1"**.

5. Then, each sequencer set must be selected by the *"Sequencer Set Selector"* (orange box, figure 3): "↩ Sequencer Set Selector" = **"0"**.

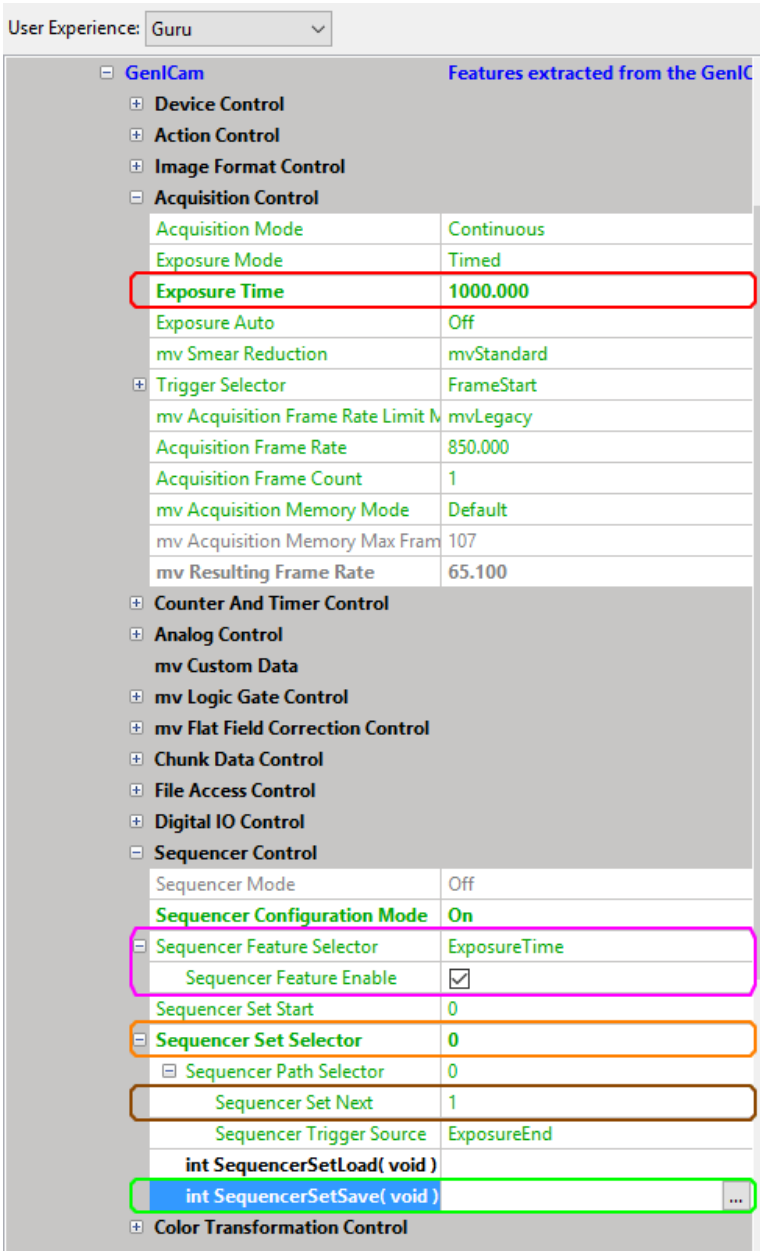


Figure 3: wxPropView - Sequencer set 0

6. Set the following sequencer set using *"Sequencer Set Next"* (brown box, figure 3): *"Sequencer Set Next"* = **"1"**.

7. Set the *"Exposure Time"* (red box, figure 3): *"Exposure Time"* = **"1000"**.

8. Finally, save the *"Sequencer Set"* (green box, figure 3): *"int SequencerSetSave()"*. This ends the configuration of this sequencer set and all the relevant parameters have been stored inside the devices RAM.

9. Set the *"UserOutputValueAllMask"* (purple box, figure 4) to a suitable value. In this case we want to use all UserOutputs, so we set it to **"0xf"**.

10. Set the *"UserOutputValueAll"* (red box, figure 4): *"UserOutputValueAll"* = **"0x1"**.

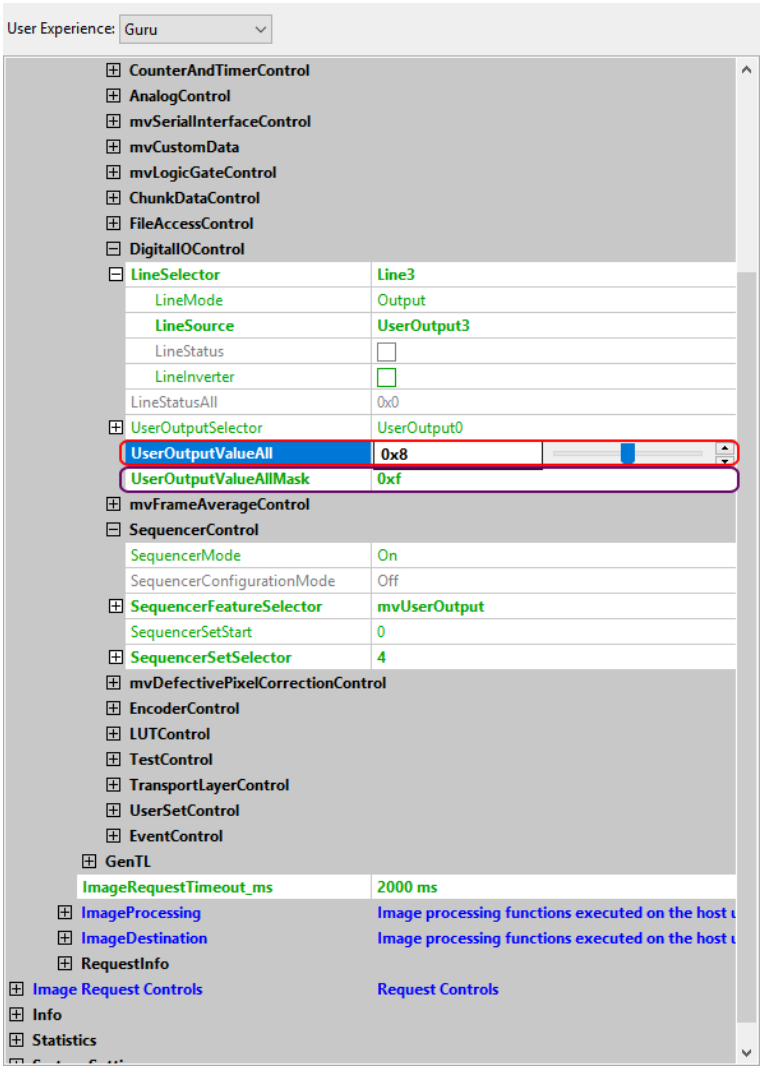


Figure 4: wxPropView - DigitalIOControl - Set UserOutputValueAll of Line3

11. Repeat these steps for the following 3 sequencer sets (Exposure Times 5000, 10000, 20000; UserOutput↩ ValueAll 0x2, 0x4, 0x8).

12. For the last sequencer set, set the desired sequencer set with *"Sequencer Set Selector"* (orange box, figure 5): *"Sequencer Set Selector"* = **"4"**.

13. Set the following sequencer set with *"Sequencer Set Next"* and trigger source with *"Sequencer Trigger Source"* (brown box, figure 5):
"Sequencer Set Next" = **"0"**. This will close the loop of sequencer sets by jumping from here back to the first one.
"Sequencer Trigger Source" = **"Counter1End"**.

14. Set the *"Exposure Time"* (red box, figure 5): *"Exposure Time"* = **"50000"**.

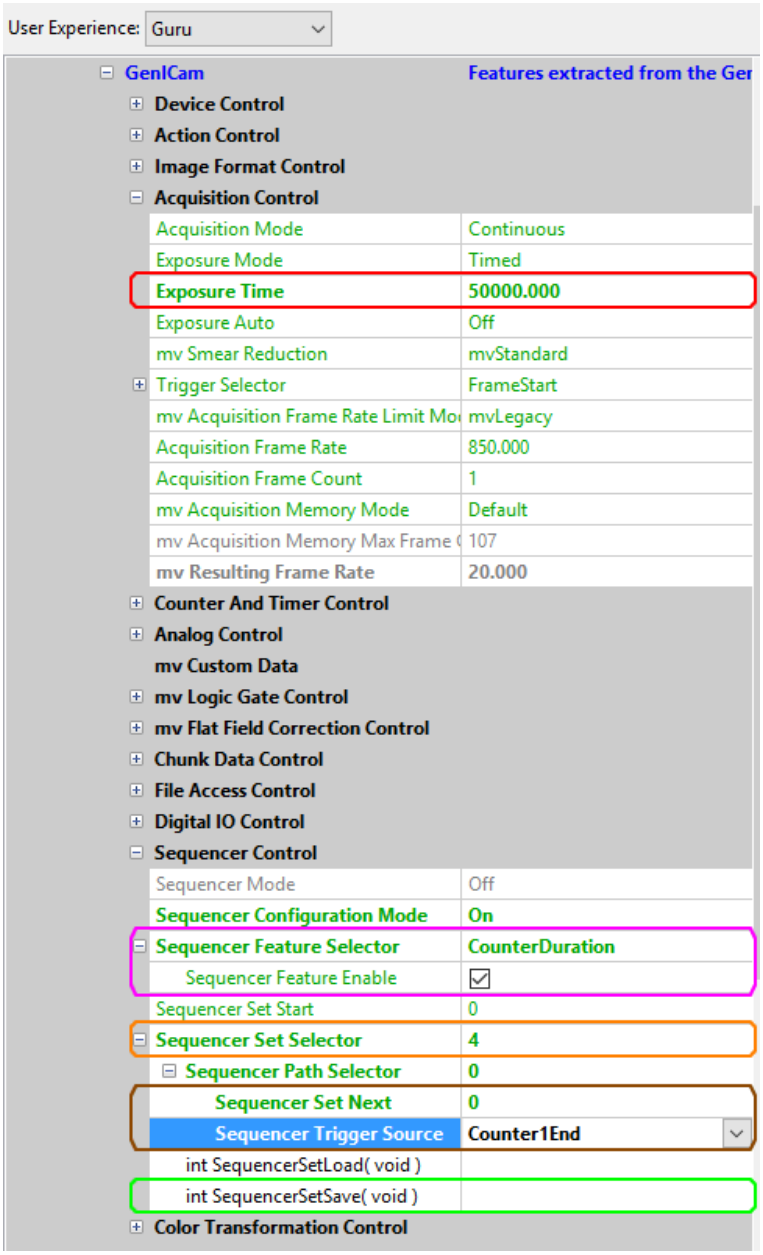


Figure 5: wxPropView - Sequencer set 4

15. Set the *"Counter Duration"* in *"Counter And Timer Control"* (red box, figure 6): *"Counter Duration"* = **"5"**.

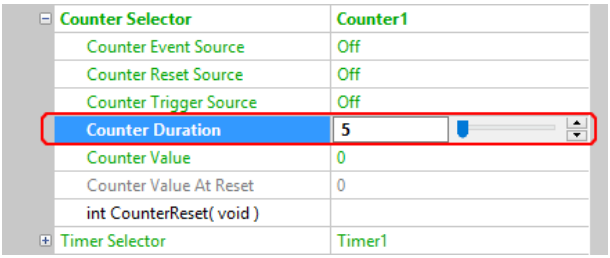


Figure 6: wxPropView - "Sequencer Mode" = "On"

16. As there are only four UserOutputs we decided not to show sequencer set "4" on the output lines.
17. Finally, save the *"Sequencer Set"* (green box, figure 3): *"int SequencerSetSave()"*.
18. Leave the *"Configuration Mode"* (red box, figure 4: *"Sequencer Configuration Mode"* = **"Off"**.
19. Activate the *"Sequencer Mode"* (red box, figure 4): *"Sequencer Mode"* = **"On"**.

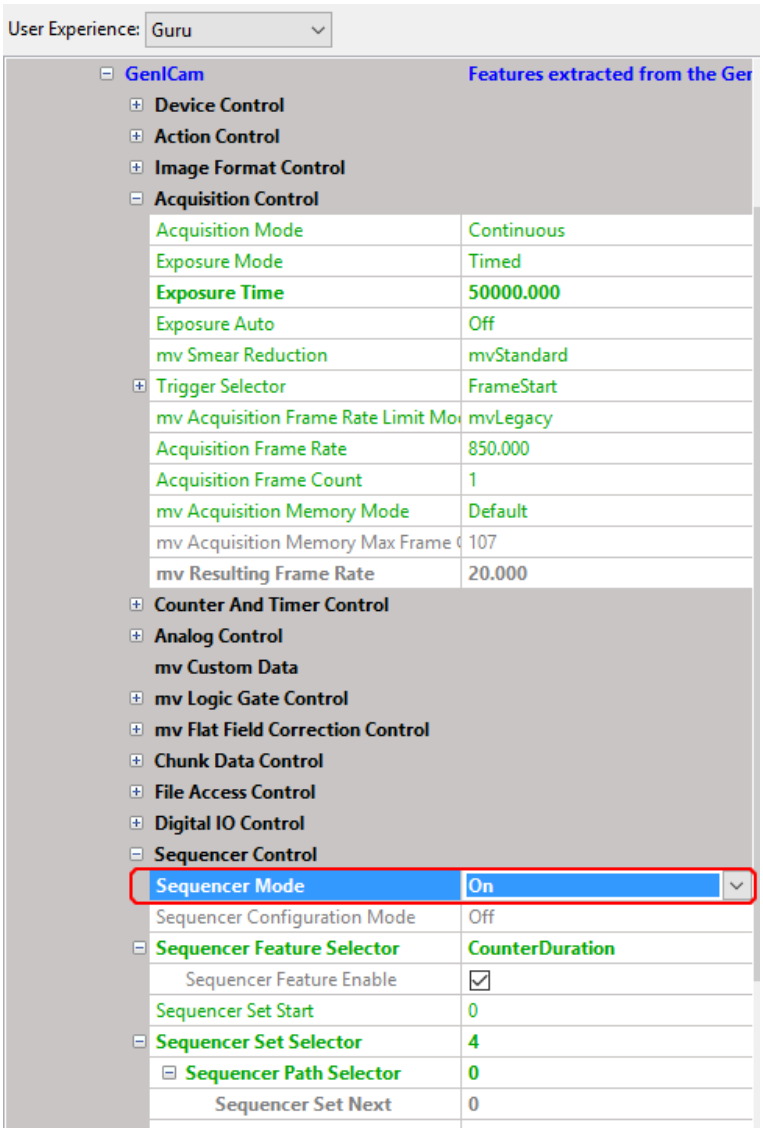


Figure 7: wxPropView - "Sequencer Mode" = "On"

Note

The **"Sequencer Mode"** will overwrite the current device settings.

You will now see that the sequencer sets are processed endlessly. Via the chunk data (activate **chunk data** (p. 113) via *"Setting -> Base -> Camera -> GenICam -> Chunk Data Control"* - activate **"Chunk Mode Active"**), the *"Info Plot"* of the analysis grid can be used to visualize the exposure times:

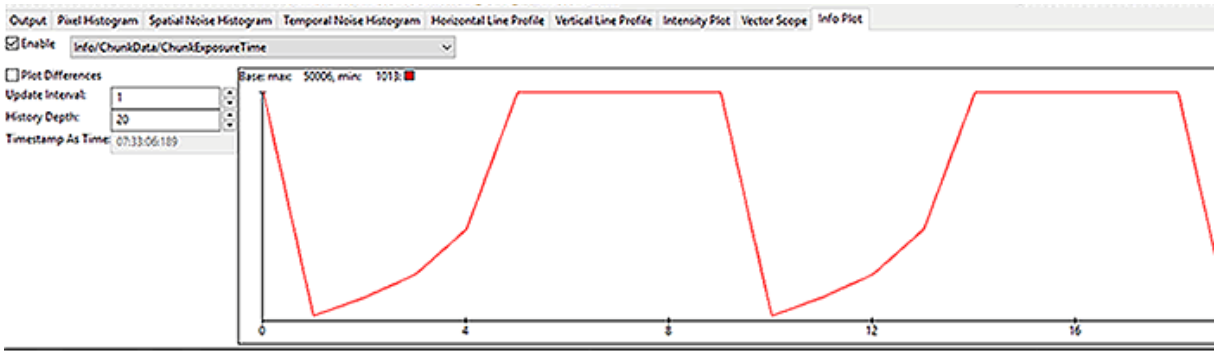


Figure 7: wxPropView - Info Plot shows the exposure times

1.18.1.3.2.1 Adapting the active time on the output lines via logic gates

If you do not want to see the whole active time of a given sequencer set but only the exposure time of a given sequencer set, you can combine your signal with the logic gates in **mvLogicGateControl** (p. 123). Figure 8 shows sample settings of Line3:

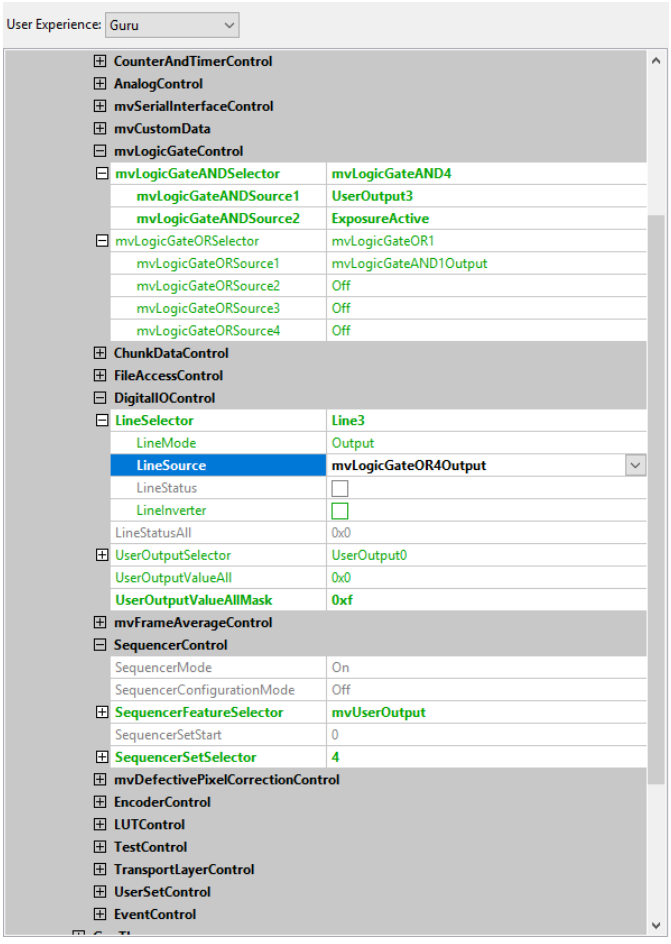


Figure 8: wxPropView - mvLogicGateControl

This produces the following output on the output lines:

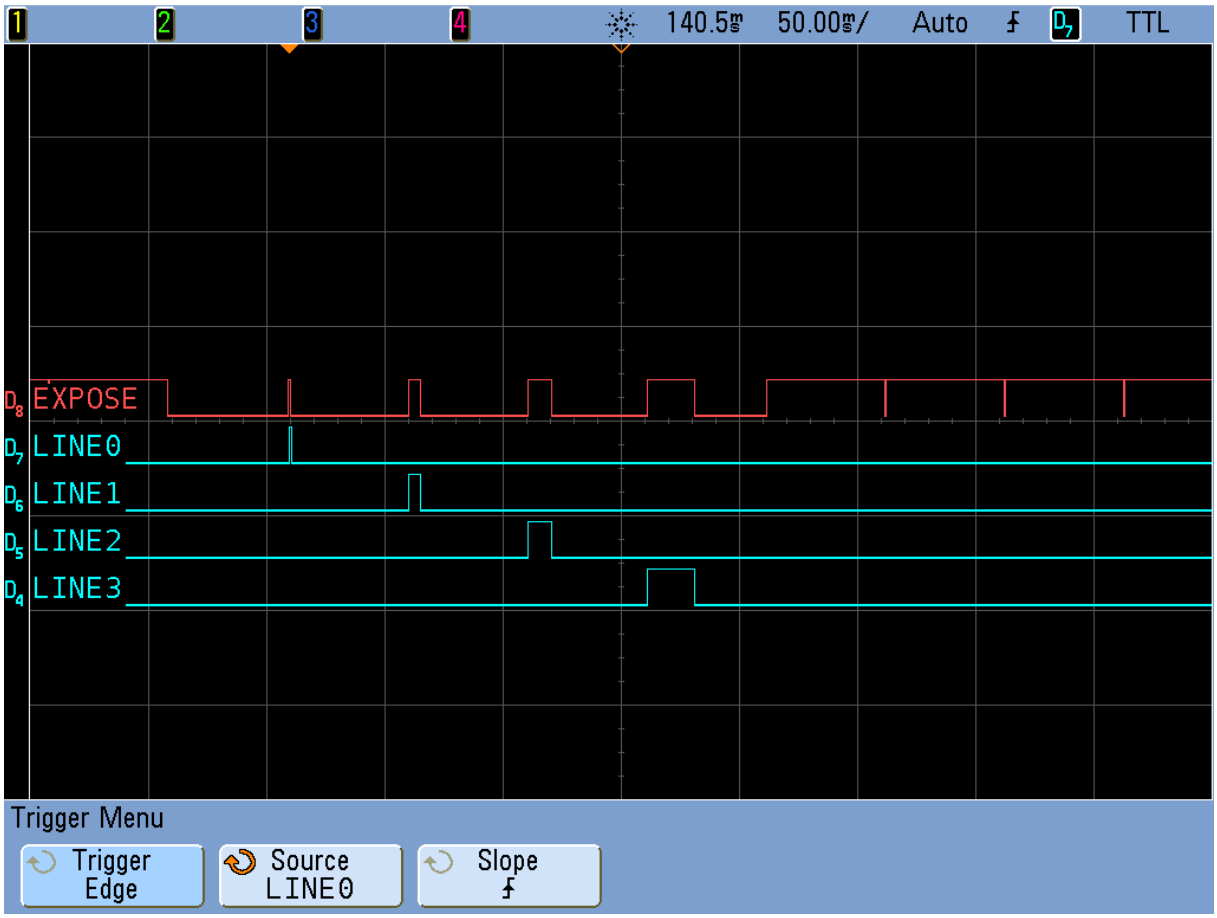


Figure 9: UserOutputs via mvLogicGateControl on output lines

These signals could be used as flash signals for separate sequencer sets.

You can program this as follows:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

GenICam::DigitalIOControl dio = new GenICam::DigitalIOControl(pDev);
dio.userOutputValueAllMask.write( 0xF );
dio.userOutputValueAll.write( 0x1 ); // 0x2, 0x4, 0x8, 0x0

GenICam::mvLogicGateControl mvlgc = new GenICam::mvLogicGateControl(pDev);
mvlgc.mvLogicGateANDSelector.writeS("mvLogicGateAND1");
mvlgc.mvLogicGateANDSource1.writeS("UserOutput0");
mvlgc.mvLogicGateANDSource2.writeS("ExposureActive");
mvlgc.mvLogicGateORSelector.writeS("mvLogicGateOR1");
mvlgc.mvLogicGateORSource1.writeS("mvLogicGateAND1Output");
mvlgc.mvLogicGateORSource2.writeS("Off");
mvlgc.mvLogicGateORSource3.writeS("Off");
mvlgc.mvLogicGateORSource4.writeS("Off");

dio.lineSelector.writeS("Line0");
dio.lineSource.writeS("mvLogicGateOR1Output");

// To output the UserOutputs directly on the output lines would be like this:
// dio.lineSource.writeS("UserOutput0");
```

1.18.1.3.3 Using the Sequencer Control wizard

Since

mvIMPACT Acquire 2.18.0

wxPropView (p. 99) offers a wizard for the **Sequencer Control** (p. 116) usage:

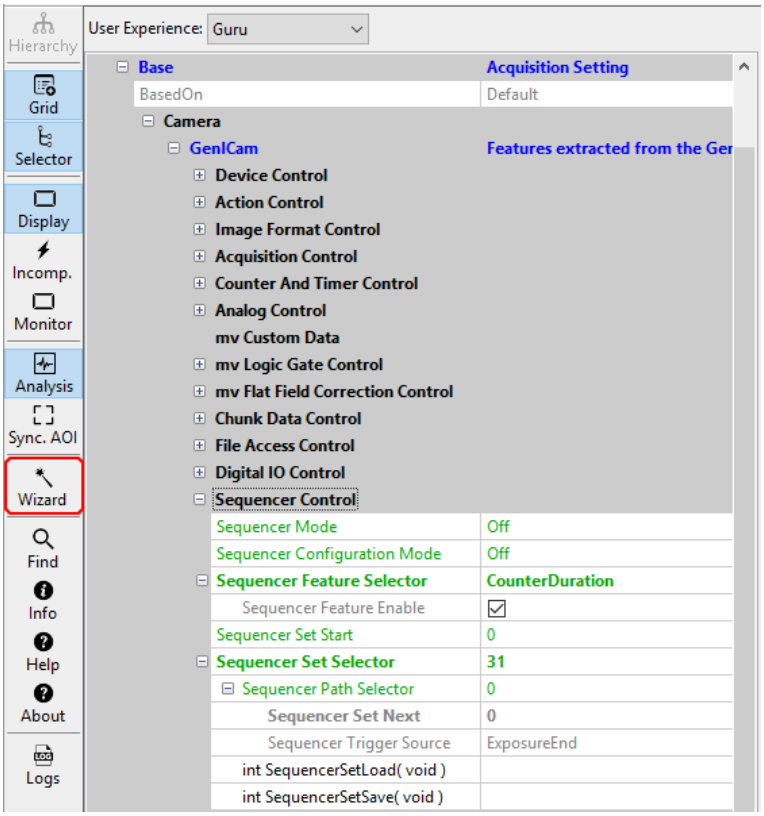


Figure 10: wxPropView - Wizard button

The wizard can be used to get a comfortable overview about the settings of the sequence sets and to create and set a sequence sets in a much easier way:

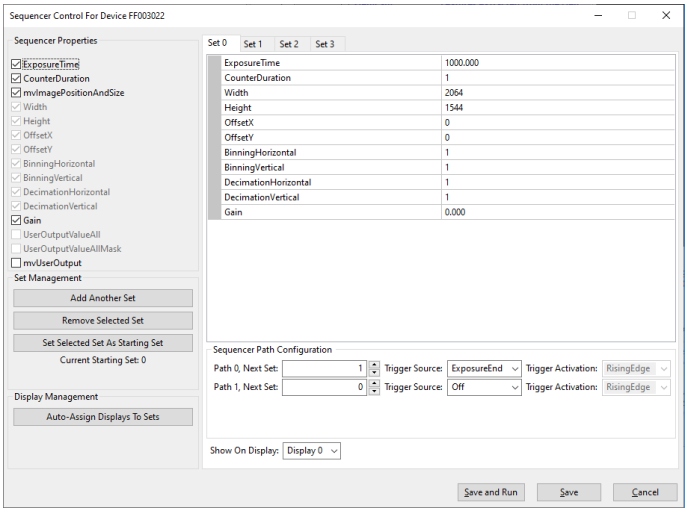


Figure 11: wxPropView - Sequencer Control wizard

Just

- select the desired properties,
- select the desired **Set** tabs,
- set the properties in the table directly, and
- **"Auto-Assign Displays To Sets"**, if you like to show each sequence set in a different display.

Do not forget to save the settings at the end.

1.18.1.3.4 Working with sequencer paths

Since

Firmware version 2.28.0

It is possible to define sets with a maximum of two active paths. The following diagram shows that two paths are defined in **"Set 0"**. **"Path 0"** (*"SequencePathSelector = 0"*) is the "standard" path that in this configuration loops and **"Path 1"** (*"SequencePathSelector = 1"*) will jump to **"Set 1"** after it is activated via a **"RisingEdge"** (*"SequencerTriggerActivation = RisingEdge"*) signal at **"UserOutput0"** (*"SequencerTriggerSource = 0"*). **"UserOutput0"** can be connected, for example, to one of the digital input lines of the camera:

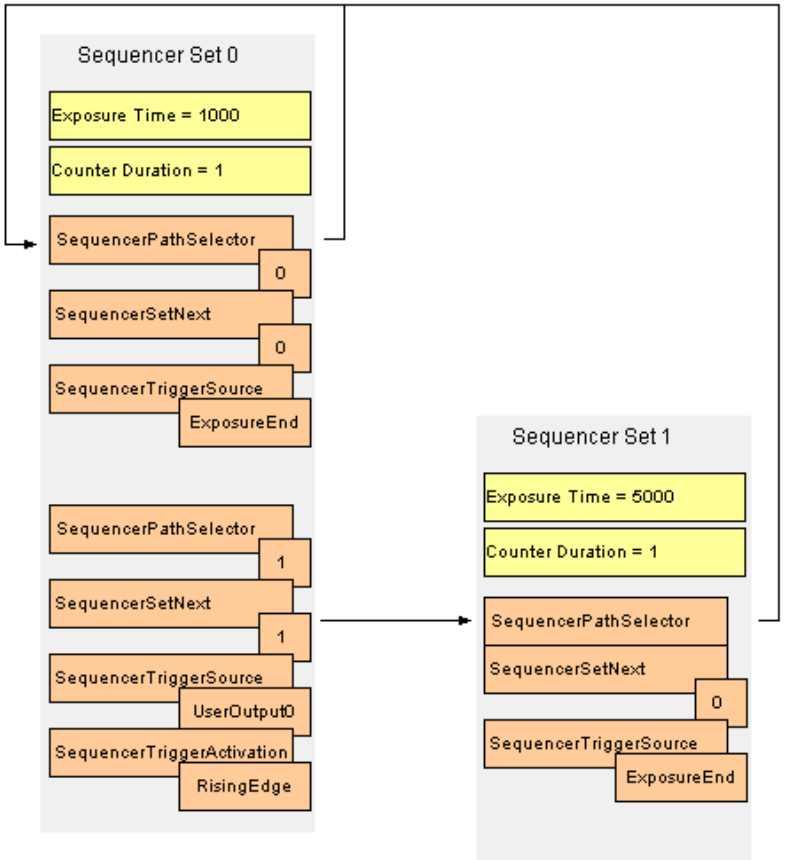


Figure 12: Working diagram of a sample with sequence paths

There are some specifications concerning the sequencer path feature:

- A path is inactive as soon as the **"SequencerTriggerSource"** is **Off**.
- If none of the paths are triggered or both parts are inactive, the sequencer will remain in the current set.
- If both paths were triggered, the path with the trigger that happened first, will be followed.
- As the next sequencer set parameters (like *ExposureTime*) need to be prepared beforehand, the set sequence might not seem straight forward at first glance. The sequencer will always need one frame to switch to the new set; this frame will be the already prepared set.

1.18.1.3.5 Programming a sequence with paths using the Sequencer Control

First the sequencer has to be configured.

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

GenICam::SequencerControl sc = new GenICam::SequencerControl( pDev );
GenICam::AcquisitionControl ac = new GenICam::AcquisitionControl( pDev );
TDMR_ERROR result = DMR_NO_ERROR;

// general sequencer settings
sc.sequencerMode.writeS( "Off" );
sc.sequencerConfigurationMode.writeS( "On" );
sc.sequencerFeatureSelector.writeS( "ExposureTime" );
sc.sequencerFeatureEnable.write( bTrue );
sc.sequencerFeatureSelector.writeS( "CounterDuration" );
sc.sequencerFeatureEnable.write( bFalse );
sc.sequencerFeatureSelector.writeS( "Gain" );
sc.sequencerFeatureEnable.write( bFalse );
sc.sequencerSetStart.write( 0 );

// set0
sc.sequencerSetSelector.write( 0LL );
ac.exposureTime.write( 1000 );
// set0 path0
sc.sequencerPathSelector.write( 0LL );
sc.sequencerTriggerSource.writeS( "ExposureEnd" );
sc.sequencerSetNext.write( 0LL );
// set0 path1
sc.sequencerPathSelector.write( 1LL );
sc.sequencerTriggerSource.writeS( "UserOutput0" );
sc.sequencerTriggerActivation.writeS( "RisingEdge" );
sc.sequencerSetNext.write( 1LL );
// save set
if( ( result = static_cast<TDMR_ERROR>( sc.sequencerSetSave.call() ) ) != DMR_NO_ERROR )
{
  std::cout << "An error was returned while calling function '" << sc.sequencerSetSave.displayName()
            << "' on device " << pDev->serial.read()
            << "(" << pDev->product.read() << "): "
            << ImpactAcquireException::getErrorCodeAsString( result ) << endl;
}

// set1
sc.sequencerSetSelector.write( 1LL );
ac.exposureTime.write( 5000 );
// set1 path0
sc.sequencerPathSelector.write( 0LL );
sc.sequencerTriggerSource.writeS( "ExposureEnd" );
sc.sequencerSetNext.write( 0LL );
// set1 path1
sc.sequencerPathSelector.write( 1LL );
sc.sequencerTriggerSource.writeS( "Off" );
// save set
if( ( result = static_cast<TDMR_ERROR>( sc.sequencerSetSave.call() ) ) != DMR_NO_ERROR )
{
  std::cout << "An error was returned while calling function '" << sc.sequencerSetSave.displayName()
            << "' on device " << pDev->serial.read()
```

```
            << "(" << pDev->product.read() << "): "
            << ImpactAcquireException::getErrorCodeAsString( result ) << endl;
}

// final general sequencer settings
sc.sequencerConfigurationMode.writeS( "Off" );
sc.sequencerMode.writeS( "On" );
```

Then it can later be triggered during runtime.

```
GenICam::DigitalIOControl dic = new GenICam::DigitalIOControl( pDev );
dic.userOutputSelector.write( 0 );
dic.userOutputValue.write( bTrue );
dic.userOutputValue.write( bFalse );
```

This will set an internal event that will cause the sequencer to use set0-path1 at the next possible time, i.e. the next time we are in set0.

There is a C++ example called **GenICamSequencerUsageWithPaths** that shows how to control the sequencer with paths from an application. It can be found in the `Examples` section of the **C++ interface documentation (mvIMPACT Acquire C++ API)** (p. 133)

1.18.1.4 Generating very long exposure times

1.18.1.4.1 Basics

At the moment the exposure time is limited to a maximum of 1 up to 20 seconds depending on certain internal sensor register restrictions. So each device might report a different maximum exposure time.

Since

Firmware version 2.28.0

Firmware version 2.28 did contain a major overhaul here so updating to at least this version can result in a much higher maximum exposure time. However, current sensor controllers can be configured to use even longer exposure times if needed using one of the devices timers to create an external exposure signal that can be fed back into the sensor. This use case will explain how this can be done.

This approach of setting up long exposure times requires the sensor of the camera to allow the configuration of an external signal to define the length of the exposure time, so only devices offering the `ExposureMode` `Trigger↩ Width` can be used for this setup.

Note

The maximum exposure time in microseconds that can be achieved in this configuration is the maximum value offered by the timer used.

With **GenICam** (p. 165) compliant devices that support all the needed features the setup is roughly like this:

1. Select *"Setting -> Base -> Camera -> GenICam -> Counter And Timer Control -> Timer Selector -> Timer 1"* and .
2. set *"Timer Trigger Source"* = *"UserOutput0"*.
3. set *"Timer Trigger Activation"* = *"RisingEdge"*.
   I.e. a rising edge on **UserOutput0** will start **Timer1**.

4. Then set the *"Timer Duration"* property to the desired exposure time in *us*.
5. In *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* set the *Trigger* Selector = *"FrameStart"*. I.e. the acquisition of one frame will start when
6. **Timer1** is active: *"Trigger Source"* = *"Timer1Active"*.
7. **Exposure** time will be the trigger width: *"Exposure Mode"* = *"TriggerWidth"*.

The following diagram illustrates all the signals involved in this configuration:

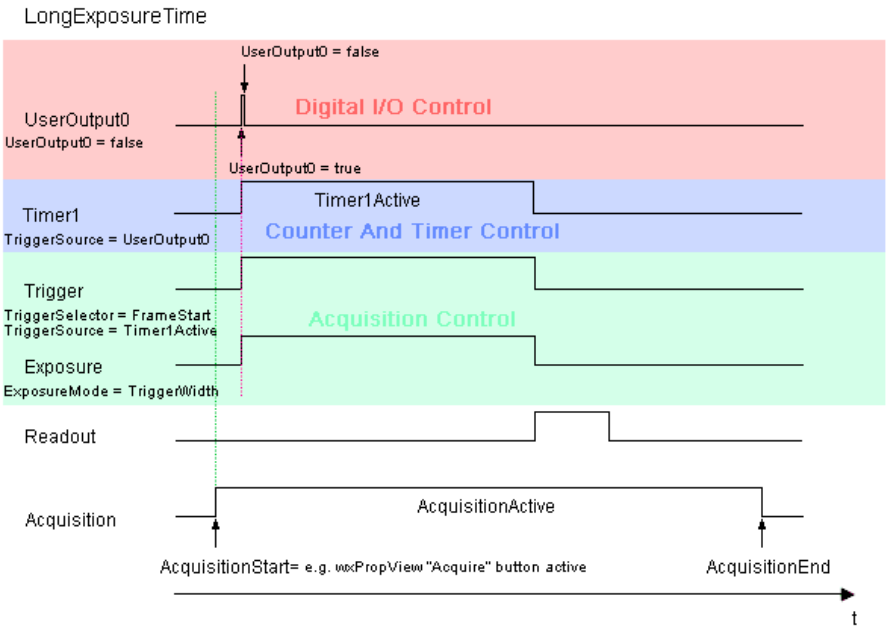


Figure 1: Long exposure times using GenICam

To start the acquisition of one frame a rising edge must be detected on **UserOutput0** in this example but other configurations are possible as well.

1.18.1.4.2 Setting up the device

The easiest way to define a long exposure time would be by using a single timer. The length of the timer active signal is then used as trigger signal and the sensor is configured to expose while the trigger signal is active. This allows to define exposure time with micro-second precision up the the maximum value of the timer register. With a 32 bit timer register this results in a maximum exposure time of roughly 4295 seconds (so roughly 71.5 minutes). When writing code e.g. in C# this could look like this:

```
private static void configureDevice(Device pDev, int exposureSec, GenICam.DigitalIOControl ioc)
{
  try
  {
    // establish access to the CounterAndTimerControl interface
    GenICam.CounterAndTimerControl ctc = new mv.impact.acquire.GenICam.CounterAndTimerControl(pDev);
    // set TimerSelector to Timer1 and TimerTriggerSource to UserOutput0
    ctc.timerSelector.writeS("Timer1");
    ctc.timerTriggerSource.writeS("UserOutput0");
    ctc.timerTriggerActivation.writeS("RisingEdge");

    // Set timer duration for Timer1 to value from user input
    ctc.timerDuration.write(exposureSec * 1000000);
```

```
    // set userOutputSelector to UserOutput0 and set UserOutput0 to inactive.
    // We will later generate a pulse here to initiate the exposure
    ioc.userOutputSelector.writeS("UserOutput0");
    ioc.userOutputValue.write(TBoolean.bFalse);

    // establish access to the AcquisitionCotrol interface
    GenICam.AcquisitionControl ac = new mv.impact.acquire.GenICam.AcquisitionControl(pDev);
    // set TriggerSelector to FrameStart and try to set ExposureMode to TriggerWidth
    ac.triggerSelector.writeS("FrameStart");
    // set TriggerSource for FrameStart to Timer1Active and activate TriggerMode
    ac.triggerSource.writeS("Timer1Active");
    ac.triggerMode.writeS("On");

    // expose as long as we have a high level from Timer1
    ac.exposureMode.writeS("TriggerWidth");
  }
  catch (Exception e)
  {
    Console.WriteLine("ERROR: Selected device does not support all features needed for
    this long time exposure approach: {0}, terminating...", e.Message);
    System.Environment.Exit(1);
  }
}
```

Note

Make sure that you adjust the `ImageRequestTimeout_ms` either to 0 (infinite)(this is the default value) or to a reasonable value that is larger than the actual exposure time in order not to end up with timeouts resulting from the buffer timeout being smaller than the actual time needed for exposing, transferring and capturing the image:

```
ImageRequestTimeout_ms = 0 # or reasonable value
```

See also

Counter And Timer Control (p. 107)

Digital I/O Control (p. 114)

Acquisition Control (p. 103)

1.18.1.5 Working with multiple AOIs (mv Multi Area Mode)

Since

mvIMPACT Acquire 2.18.3

1.18.1.5.1 Introduction

A special feature of Pregius sensors (a.k.a. IMX) from Sony is the possibility to define multiple AOIs (Areas of Interests - a.k.a. ROI - Regions of Interests) and to transfer them at the same time. Because many applications just need one or several specific parts in an image to be checked, this functionality can increase the frame rate.

Once activated, the *"mv Multi Area Mode"* allows you, depending on the sensor, to define up to eight AOIs (*mvArea0* to *mvArea7*) in one image. There are several parameters in combination with the AOIs which are illustrated in the following figure:

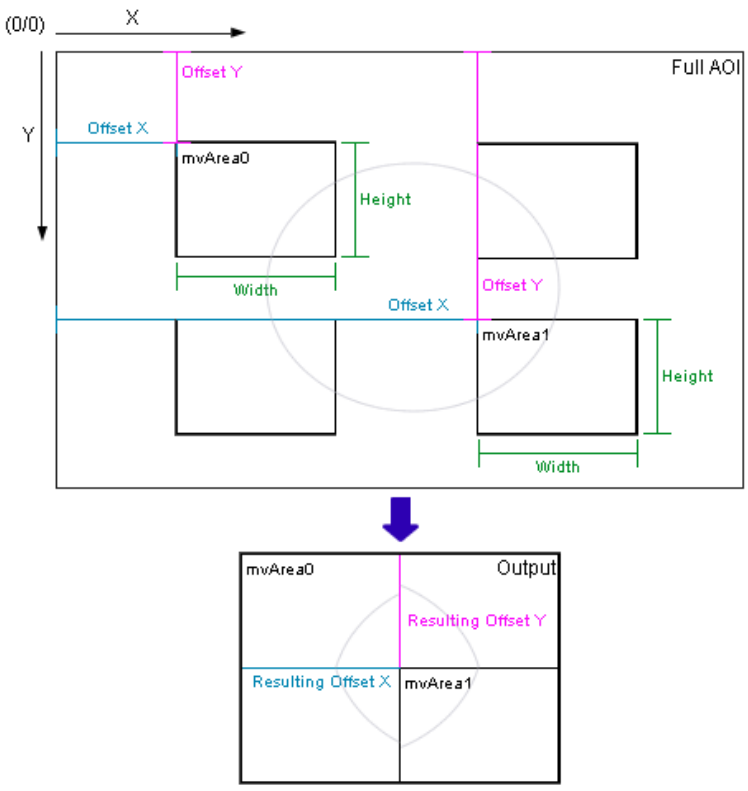


Figure 1: Multiple AOIs principle

The *"Resulting Offset X"* and *"Resulting Offset Y"* indicates the starting point of the specific AOI in the output image. To complete the rectangular output image, the "missing" areas are filled up with the image data horizontally and vertically. We recommend to use the wizard as a starting point - the wizard provides a live preview of the final merged output image.

1.18.1.5.2 Using wxPropView

To create multiple AOIs with **wxPropView** (p. 99), you have to do the following step:

1. Start **wxPropView** (p. 99) and
2. connect to the camera.
3. Then change in *"Setting -> Base -> Camera -> GenICam -> Image Format Control"* the **"mv Multi Area Mode"** to **"mvMultiAreasCombined"**.
   Afterwards, **"mv Area Selector"** is available.
4. Now, select the area a.k.a. AOI you want to create via *"mv Area Selector"*, e.g. **"mvArea3"** and
5. set the parameters **"mv Area Width"**, **"mv Area Height"**, **"mv Area Offset X"**, and **"mv Area Offset Y"** to your needs.
6. Activate the area a.k.a. AOI by checking the box of **"mv Area Enable"**.

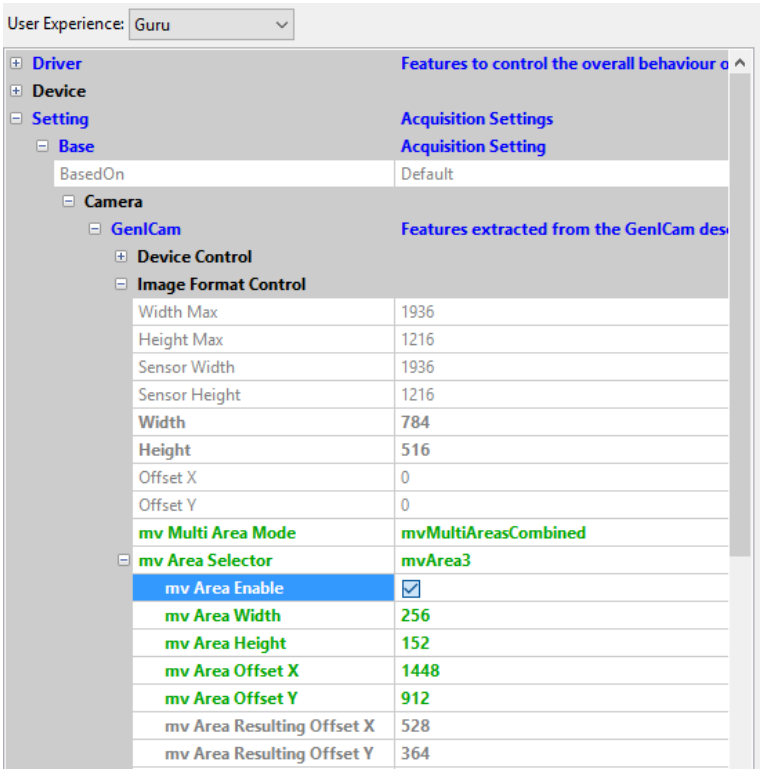


Figure 2: wxPropView - Multiple AOIs

7. Finally, start the acquisition by clicking the button *"Acquire"*.

1.18.1.5.2.1 Using the Multi AOI wizard

Since

mvIMPACT Acquire 2.19.0

wxPropView (p. 99) offers a wizard for the Multi AOI usage:

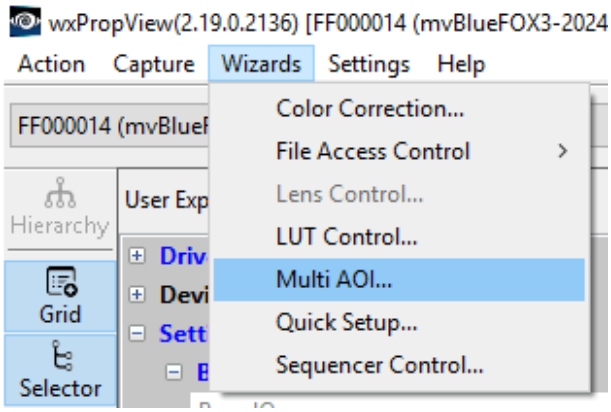


Figure 3: wxPropView - Wizard menu

The wizard can be used to get a comfortable overview about the settings of the AOIs and to create and set the AOIs in a much easier way:

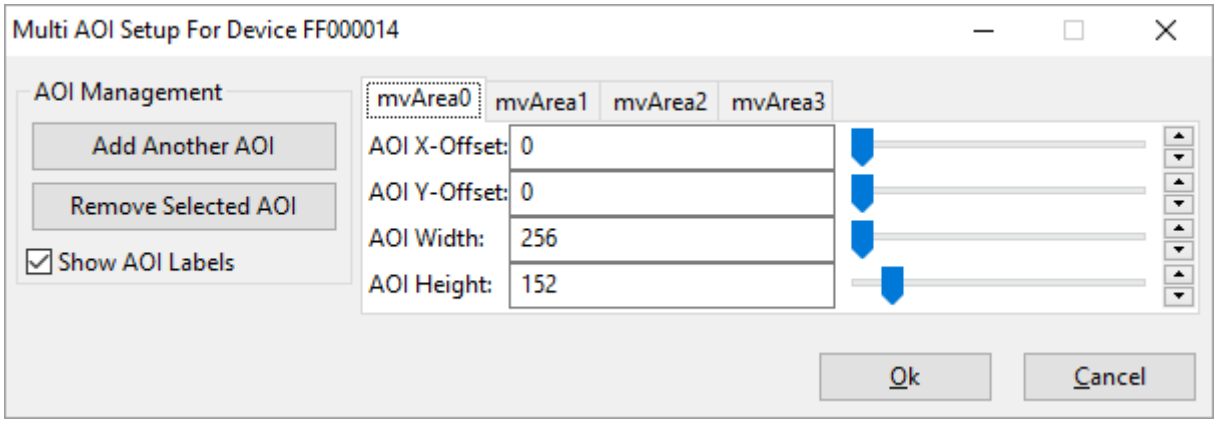


Figure 4: wxPropView - Multi AOI wizard

Just

- select the desired **mvArea** tabs,
- set the properties like offset, width, and height in the table directly, and
- confirm the changes at the end using the **Ok** button.

The live image shows the created AOIs and the merged or "missing" areas which are used to get the final rectangular output image.

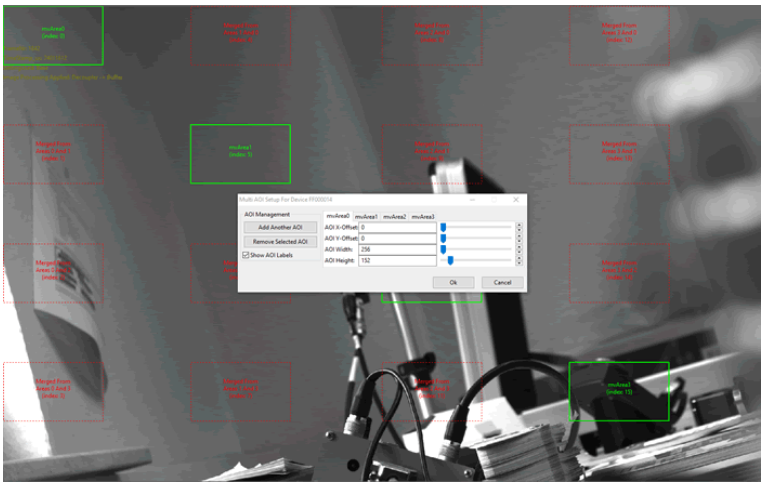


Figure 5: wxPropView - Multi AOI wizard - Live image

1.18.1.5.3 Programming multiple AOIs

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...
    GenICam::ImageFormatControl ifc( pDev );
    ifc.mvMultiAreaMode.writeS( "mvMultiAreasCombined" );
    ifc.mvAreaSelector.writeS( "mvArea0" );
    ifc.mvAreaEnable.write( bTrue );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 0 );
    ifc.mvAreaWidth.write( 256 );
    ifc.mvAreaHeight.write( 152 );
    ifc.mvAreaSelector.writeS( "mvArea1" );
    ifc.mvAreaEnable.write( bFalse );
    ifc.mvAreaSelector.writeS( "mvArea2" );
    ifc.mvAreaEnable.write( bFalse );
    ifc.mvAreaSelector.writeS( "mvArea3" );
    ifc.mvAreaEnable.write( bTrue );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 0 );
    ifc.mvAreaWidth.write( 256 );
    ifc.mvAreaHeight.write( 152 );
    ifc.mvAreaOffsetX.write( 1448 );
    ifc.mvAreaOffsetY.write( 912 );
...
```

1.18.1.6 Working with burst mode buffer

If you want to acquire a number of images at sensor's maximum frame rate while at the same time the image transfer should be at a lower frame rate, you can use the internal memory of the mvBlueFOX3.

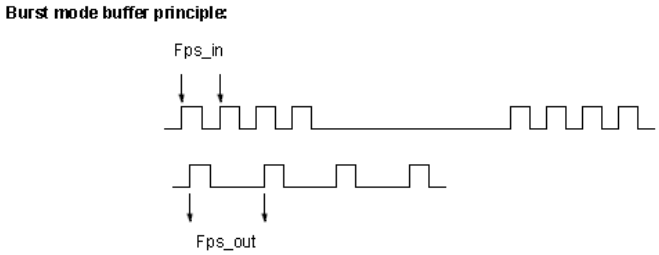


Figure 1: Principle of burst mode buffering of images

Note

The maximum buffer size can be found in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> mv Acquisition Memory Max Frame Count"*.

To create a burst mode buffering of images, please follow these steps:

1. Set image acquisition parameter (*"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> mv Acquisition Frame Rate Limit Mode"*) to **"mvDeviceMaxSensorThroughput"**.
2. Finally, set the acquisition parameter *"mv Acquisition Frame Rate Enable"* to **"Off"**.

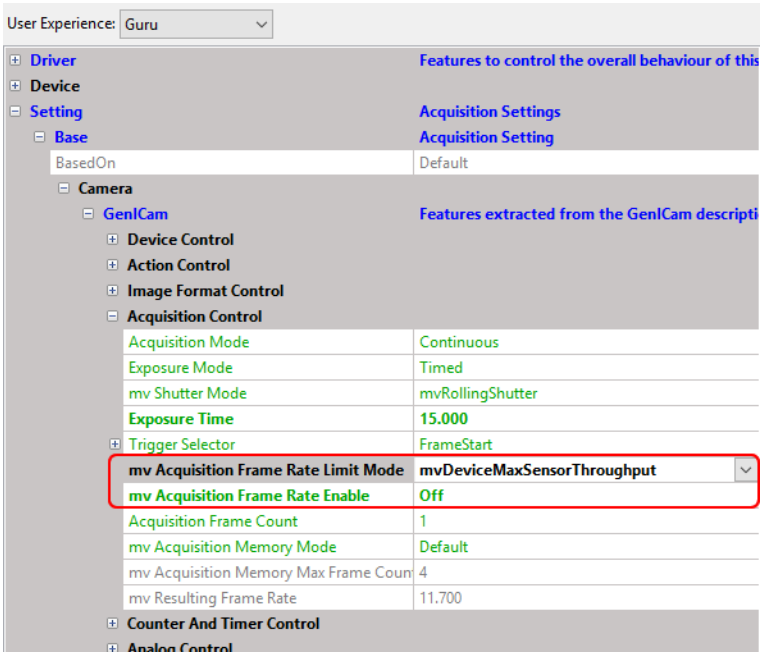


Figure 2: wxPropView - Setting the bandwidth using **"mv Acquisition Frame Rate Limit Mode"**

Alternatively, you can set the burst mode via the desired input frames and the desired output bandwidth:

1. Set image acquisition parameters to the desired input frames per second value (*"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*).

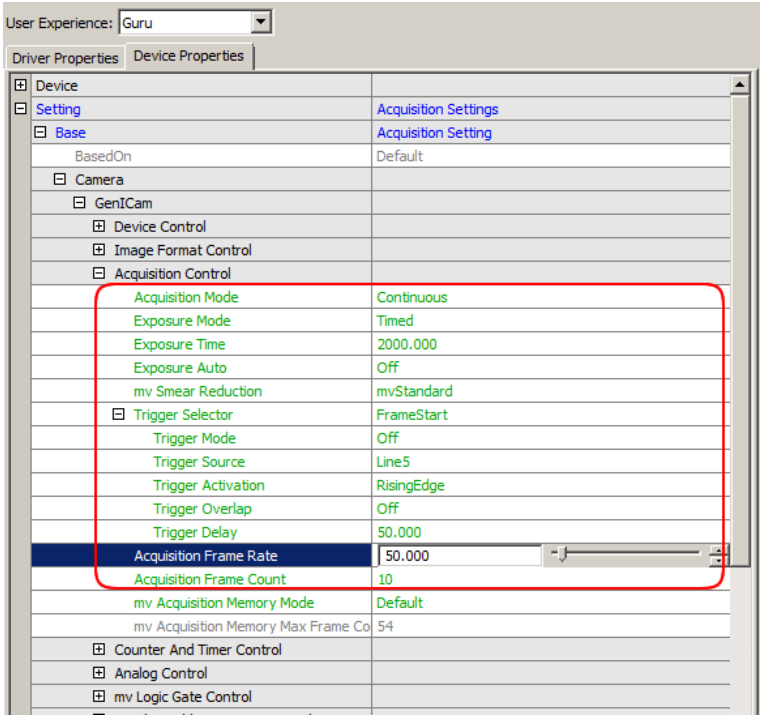


Figure 3: wxPropView - Setting the **"Acquisition Frame Rate"**

2. Set bandwidth control to the desired MByte/s out value in *"Setting -> Base -> Camera -> GenICam -> Device Control -> Device Link Selector -> Device Link Troughput Limit Mode"* to **"On"** and
3. set the desired **"Device Link Troughput Limit"** in Bits per second (Bps).

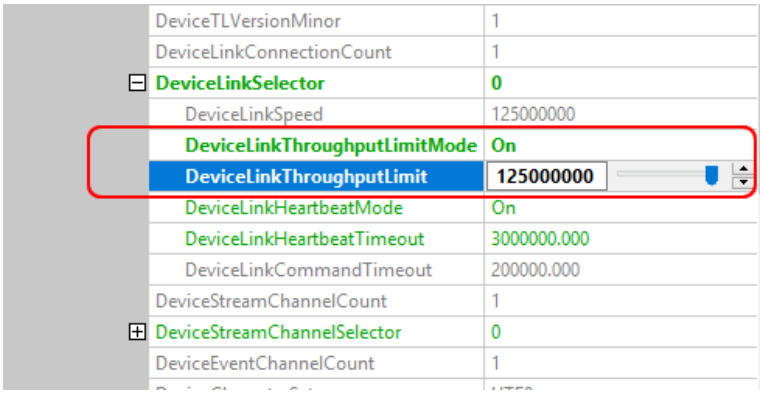


Figure 4: wxPropView - Setting the bandwidth using **"Device Link Troughput Limit Mode"**

Now, the camera will buffer burst number of images in internal memory and readout at frames per second out.

1.18.1.6.1 Triggered frame burst mode

With the **triggerSelector** "FrameBurstStart", you can also start a frame burst acquisition by a trigger. A defined number of images (**"AcquisitionBurstFrameCount"**) will be acquired directly one after the other. With the **"mv Acquisition Frame Rate Limit Mode"** set to **mvDeviceMaxSensorThroughput** , the there won't be hardly any gap between these images.

As shown in figure 5, **"FrameBurstStart"** can be trigger by a software trigger, too.

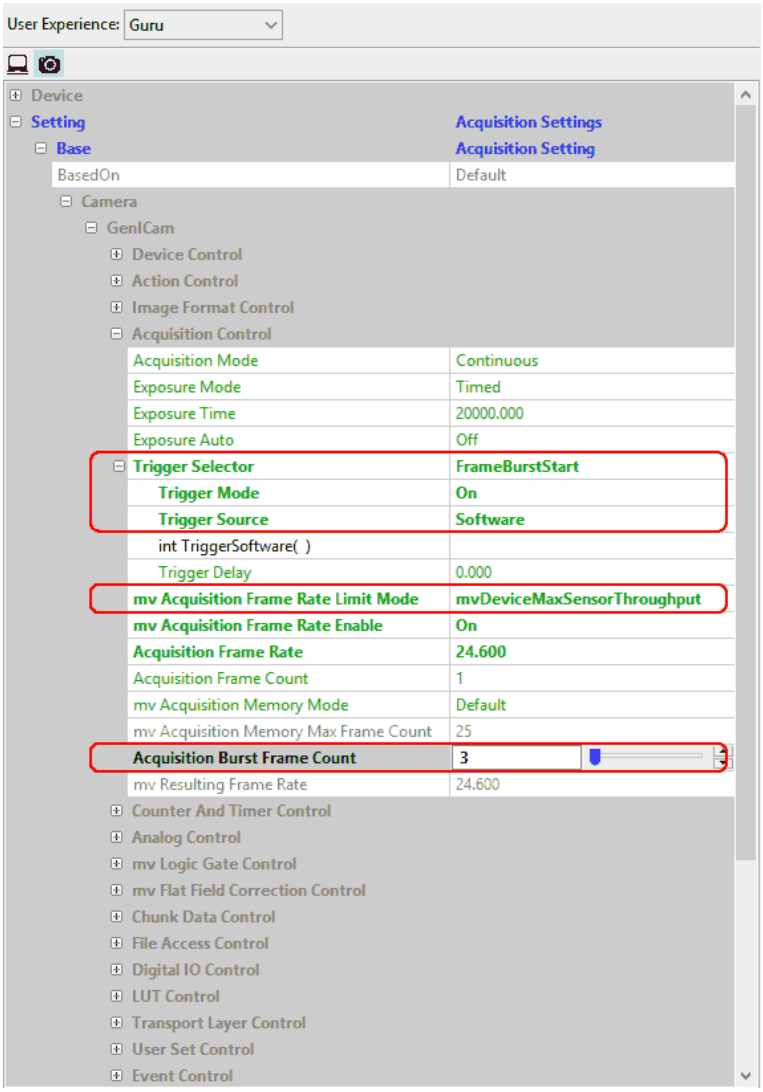


Figure 5: wxPropView - Setting the frame burst mode triggered by software

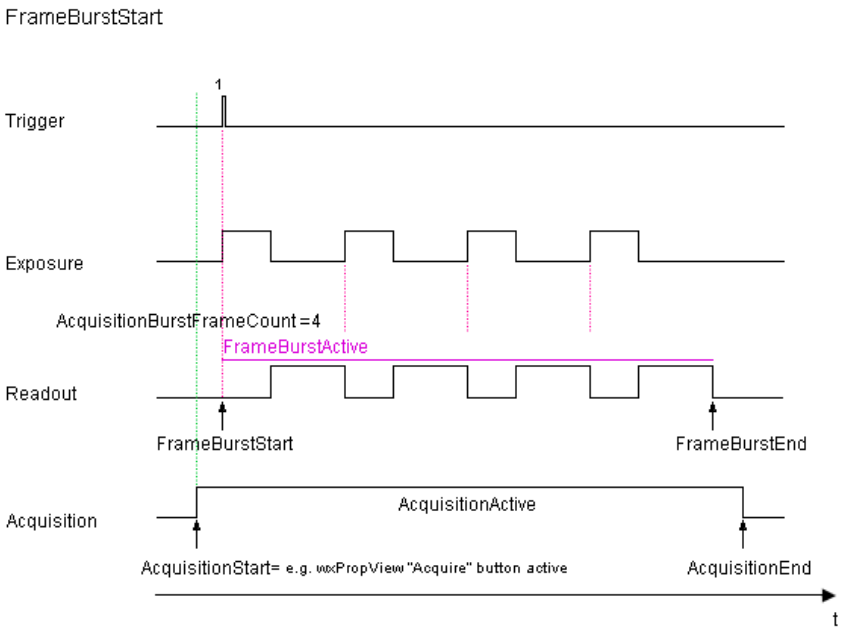


Figure 6: Principle of FrameBurstStart

1.18.1.7 Using the SmartFrameRecall feature

Since

mvIMPACT Acquire 2.18.0

1.18.1.7.1 Introduction

The **SmartFrameRecall** is a new, FPGA based smart feature which takes the data handling of industrial cameras to a new level.

So far, the entire data amount has to transferred to the host PC whereby the packetizer in the camera split the data packages and distributed them to the two Gigabit Ethernet lines. On the host PC, the data was merged again, shrank, and this AOI was possibly processed (Figure 1).

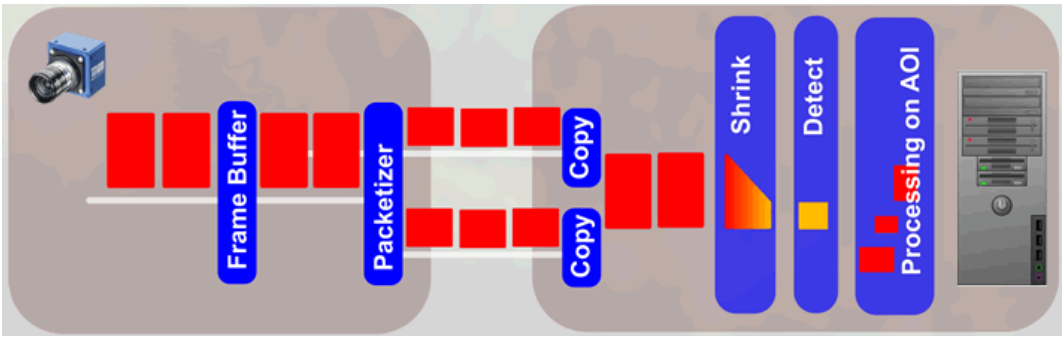


Figure 1: Data handling so far

This procedure has several disadvantages:

- Both data lines (in case of Dual GigE cameras for example) for each camera are required which
  - leads to high cabling efforts.
- A high end PC is needed to process the data which
  - leads to high power consumption.
- In USB 3 multi-camera solutions (depending on the resolution) each camera requires a separate connection line to the host PC which
  - limits the cabling possibilities, the possible distances, and makes an installation more complex (without the possibilities to use hubs, for example).
- Last but not least, the frame rates a limited to the bandwidth.

The **SmartFrameRecall** is a new data handling approach which buffers the hi-res images in the camera and only transfers thumbnails. You or your software decides on the host PC which AOI should be sent to the host PC (Figure 2).

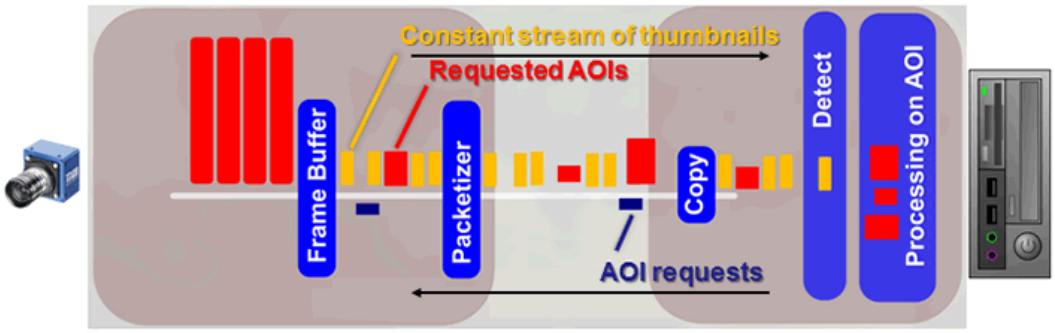


Figure 2: SmartFrameRecall working method

This approach allows

- higher bandwidths,
- less CPU load and power consumption,
- higher frame rates, and
- less complex cabling.

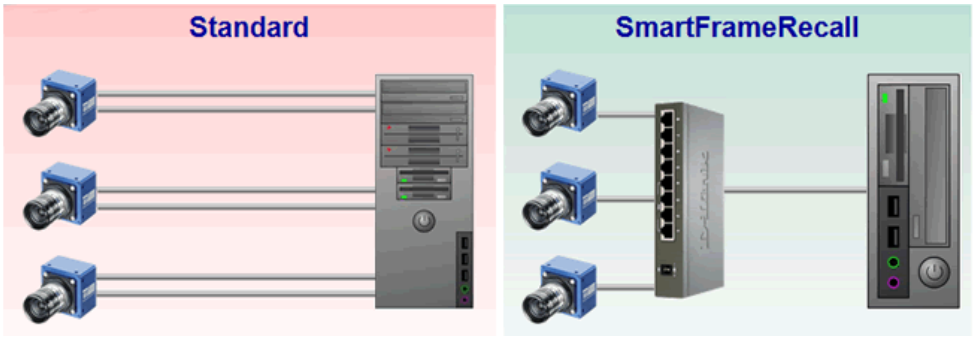


Figure 3: Connection advantages in combination with SmartFrameRecall

1.18.1.7.2 Implementing a SmartFrameRecall application

First of all, clarify if the **SmartFrameRecall** makes sense to be used in your application:

- Taking a short look on the thumbnails, could you specify which frames don't interest you at all or which portions of the frame suffice to you for further processing?

If you can, your application can do so too, and then **SmartFrameRecall** is the right framework for you. To use the **SmartFrameRecall** follow these steps:

- Activate the **Chunk Data Control:**

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...
GenICam::ChunkDataControl cdc( pDev );
cdc.chunkModeActive.write( bTrue );
cdc.chunkSelector.writeS( "Image" );
cdc.chunkEnable.write( bTrue );
cdc.chunkSelector.writeS( "mvCustomIdentifier" );
cdc.chunkEnable.write( bTrue );
...
```

It is necessary to activate the **chunk data** (p. 113) that your application can easily distinguish frames belonging to the normal stream from the ones requested by the host application.

- Reduce the size of the streamed images. This can reduce the size using **Decimation**, both horizontally and vertically. E.g. setting decimation to 16, a normal image will only consume 16∗16 thus 1/256th of the bandwidth:

```
...
GenICam::ImageFormatControl ifc( pDev );
ifc.decimationHorizontal.write( 16 );
ifc.decimationVertical.write( 16 );
...
```

- Make sure that the resulting image width is a multiple of 8! If this is not the case the SmartFrameRecall feature cannot be activated.
- Activate the **SmartFrameRecall** feature:

```
...

GenICam::AcquisitionControl ac( pDev );
ac.mvSmartFrameRecallEnable.write( bTrue );
...
```

This will configure the devices internal memory to store each frame (that gets transmitted to the host) in full resolution. These images can be requested by an application when needed. As soon as the memory is full, the oldest one will be removed from the memory whenever a new one becomes ready (FIFO).

- Analyze the images of the reduced data stream.
- If necessary, request the desired image in full resolution:

```
...
struct ThreadParameter
{
    Device* pDev;
    GenICam::CustomCommandGenerator ccg;
    ...
}
...
unsigned int DMR_CALL liveThread( void* pData )
{
    ThreadParameter* pThreadParameter = reinterpret_cast<ThreadParameter*>( pData );
    ...
    pThreadParameter->ccg.requestTransmission( pRequest, x, y, w, h, rtmFullResolution, cnt );
    ...
}
...
```

The last parameter of **requestTransmission** will be written into the **chunkmvCustomIdentifier** and your application can recognize the request.

- Finally do you analysis/processing with the requested image in full resolution.

We provide

- a basic **SmartFrameRecall** C++ sample called **GenICamSmartFrameRecallUsage.cpp** which will be installed together with **"mvGenTL Acquire"** driver.
  You find the sample's description in the online manual of the mvIMPACT Acquire API C++: https://www.matrix-vision.com/manuals/SDK_CPP/GenICamSmartFrameRecallUsage_8cpp-example.html
- a advanced **SmartFrameRecall** C# sample called **SmartFrameRecall.cs** which will be installed together with **"mvGenTL Acquire"** driver.
  You find the sample's description in the online manual of the mvIMPACT Acquire API .NET: https://www.matrix-vision.com/manuals/SDK_NET/SmartFrameRecall_8cs-example.html

1.18.1.8 Using The Linescan mode

Both CMOS sensors from e2v offer a line scan mode. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1024 or 1200 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

Complete instructions for using the line scan mode are provided here:

- **sensor description of line scan mode (-1013)** (p. 469)
- **sensor description of line scan mode (-1020)** (p. 473)

1.18.1.8.1 System requirements

- mvBlueCOUGAR-X
  - **"firmware version"** at least **"1.6.32.0"**
- mvBlueFOX3
  - **"firmware version"** at least **"1.6.139.0"**

1.18.1.8.2 Initial situation and settings

Generally, line scan cameras are suitable for inspections of moving, continuous materials. In order that the line scan camera acquires the line at the right time, an **incremental encoder** (p. 277), for example, at a conveyor belt triggers the line scan camera. Normally, an **incremental encoder** (p. 277) does this using a specific frequency like 1:1 which means that there is a signal for every line. However, during the adjustment of a line trigger application or while choosing a specific incremental encoder you have to keep the specific frequency in mind.

Note

Using **timers and counters** (p. 107) it is possible to skip trigger signals.

In line scan mode, the camera adds the single lines to one image of the height of max. 1024 or 1200 lines (according to the used sensor). The images are provided with no gap.

Note

While using the line scan mode with a gray scale sensor, one trigger signal will lead to one acquired image line. Using a color sensor, one trigger signal will lead to two acquired image lines.

Due to aberrations at the edges of the lens, you should set an offset in the direction of Y (**"Offset Y"**, see the following figure), generally around the half of the sensor's height (a.k.a. sensor's Y center). With Offset Y you can adjust the scan line in the direction of motion.

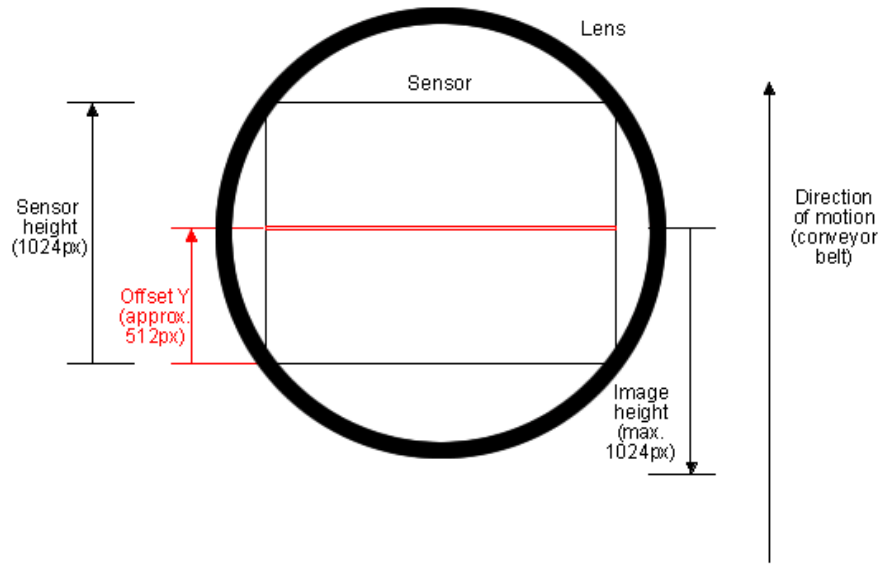


Figure 1: Sensor's view and settings with a sensor with max. height of 1024 pixels/lines (e.g. -x02e / -1013)

1.18.1.8.2.1 Scenarios

With regards to the external trigger signals provided by an **incremental encoder** (p. 277), there are two possible scenarios:

1. A conveyor belt runs continuously and so does the **incremental encoder** (p. 277), or - like in a reverse vending machine,
2. a single item is analyzed and the conveyor belt and so the **incremental encoder** (p. 277) stops after the inspection and restarts for the next item.

In the first scenario you can use the standard settings of the MATRIX VISION devices. Please have a look at the sample **Triggered line scan acquisition with exposure time of 250 us** (p. 205) which shows how you can set the line scan mode with continuous materials and signals from the encoder. However, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible.

In the second scenario, the external trigger stops. If there is a heartbeat functionality in the system (e.g. with GigE Vision), there can be a halt of the system. Please have a look at the sample **Triggered line scan acquisition with a specified number of image blocks and pausing trigger signals** (p. 205) which shows how you can handle pausing trigger signals.

1.18.1.8.3 Sample 1: Triggered linescan acquisition with exposure time of 250 us

This sample will show you how to use the line scan mode of the sensors **-1013** (p. 469) and **-1013GE** (p. 469) using an external trigger provided by an **incremental encoder** (p. 277) which sends a **"trigger for every line"** (1:1)

> **Note**
>
> You can also use the sensor **-1020** (p. 473) . However, the sensor is slower due to the higher number of pixels.

In this sample, we chose an exposure time of **"250 us"** and to ease the calculations we used **"1000 px image height"**.

Note

To get suitable image results, it might be necessary to increase the **gain** or the **illumination**.

These settings result in a max. **"frame rate"** of **"2.5 frames per second"**.

To adjust the opto-mechanics (focus, distance, illumination, etc.), you can use the area mode of the sensor. That's a main advantage of an area sensor with line scan mode compared to a line scan camera!

You will need the following pins of the mvBlueFOX3:

Pin.	Signal (Standard version)	Description
1	GND	Common ground
2	12V .. 24V	Power supply
4	Opto DigIn0 (line4)	The output signal A of the incremental encoder

1.18.1.8.3.1 Setting the application in wxPropView

Summary of our sample:

Property name wxPropView	Setting	**GenICam Control** (p. 100)	Comment
Device Scan Type	`line` scan	Device Control	
Height (in pixels)	`1000`	Image Format Control	
Offset Y (in pixels)	`500`	Image Format Control	
Exposure Time (in microseconds)	`250.000`	Acquisition Control	
Trigger Mode	`On`	Acquisition Control	
Trigger Source	`Line4`	Acquisition Control	
ImageRequestTimeout_ms (in milliseconds)	`0 ms`	-	This is necessary otherwise there will be error counts and no frames are created.

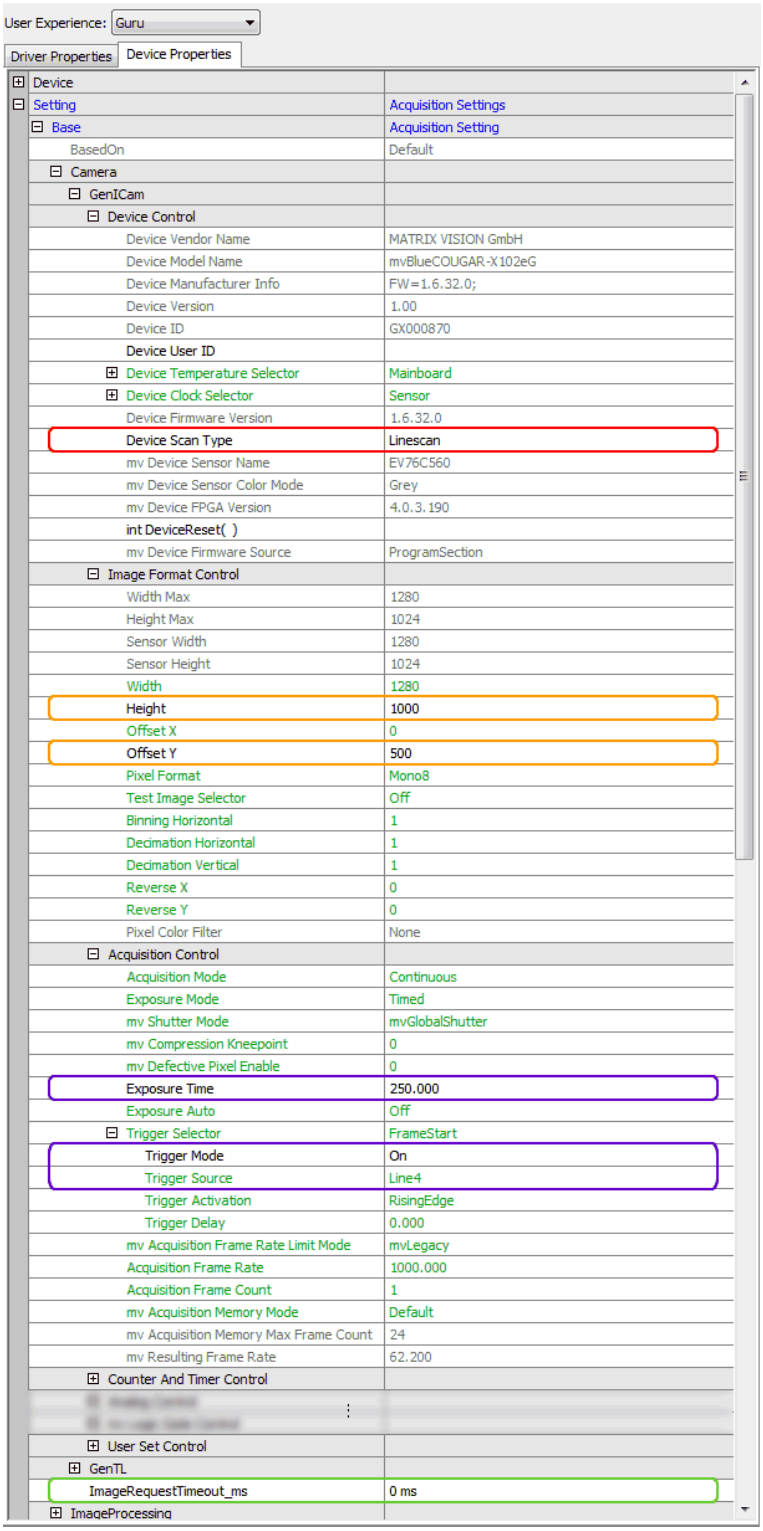


Figure 2: Settings in wxPropView

1.18.1.8.4 Sample 2: Triggered line scan acquisition with a specified number of image blocks and pausing trigger signals

This section will provide you with some information you have to keep in mind while working with pausing triggers and specified number of image blocks.

First of all, using mvBlueCOUGAR-X or mvBlueCOUGAR-XD it is necessary to disable the heartbeat of the GigE Vision control protocol (GVCP) (`"Device Link Heartbeat Mode = On"`) otherwise a paused trigger signal can be misinterpreted as a lost connection:

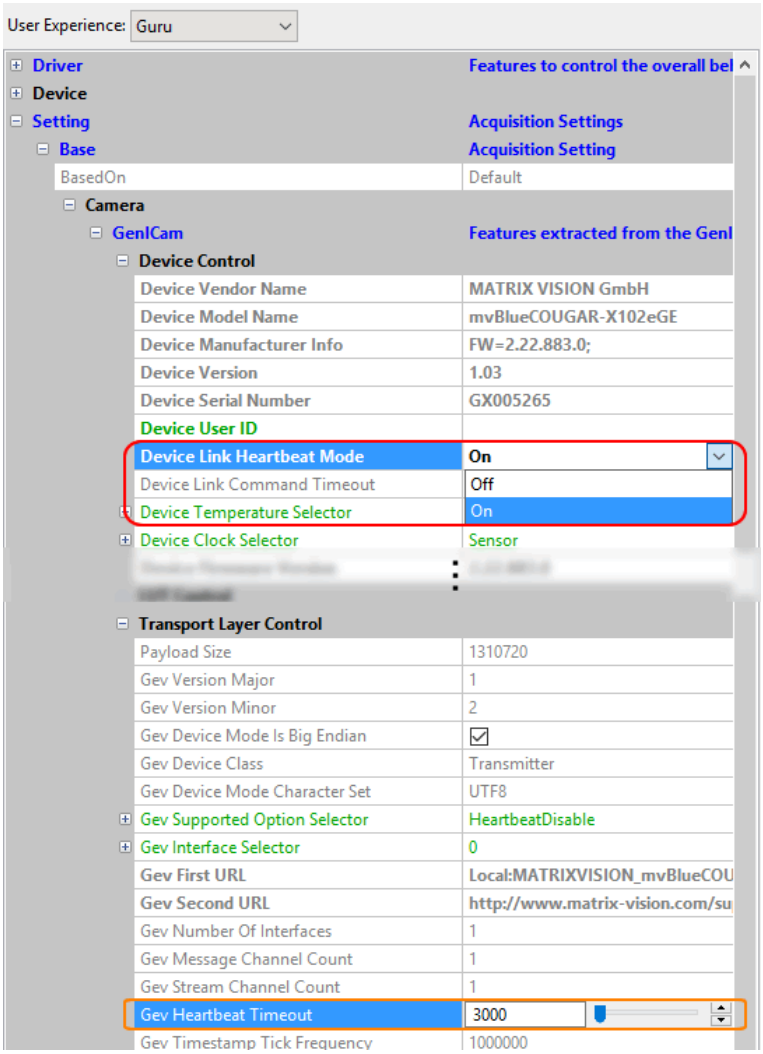


Figure 3: wxPropView - Disabling the heartbeat

Secondly, since the conveyor belt stops sometime, the trigger will do so, too. Be sure, that the trigger signal is available until the last image block was received.

Thirdly, if you know the number of image blocks, you can use the **"MultiFrame functionality"** (in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* `set` "Acquisition Mode = MultiFrame" and "`Acquisition Frame Count`"). This will acquire the number of image blocks and will stop the acquisition afterwards.

1.18.1.9 Using the mvBlockscan feature

Since

Firmware version 2.35.0

1.18.1.9.1 Introduction

The **mvBlockscan** mode can be used for implementing a kind of linescan application. Instead of a single line, an Area of Interest (AOI) with a few lines (**LinesPerBlock**) will be captured. Then a selectable number of these small AOIs will be sent as a single image. This helps to minimize the overhead of a "high frequency" streaming of every single AOI when doing it in the conventional way. This mode is available for all mvBlueFOX3 cameras with Sony Global Shutter CMOS sensors.

The minimum size of **mvBlockscanLinesPerBlock** and the increment/decrement step size is similar the the limitations of the **Height** property for the specific camera model. Typically these are values in the range of 4 - 16 but it is always better to check these limits at runtime.

The advantages of the block scan mode over a line scan camera are:

- Standard interface: USB3 Vision instead of CoaXPress and CameraLink.
- Easier system setup: Since the camera can use area scan too, it is much easier to adjust the focus and get a sharp image than this would be with a line scan sensor.
- Less block loss due to the FPGA inside the camera.
- Less load on the host: Blocks are collected in the camera.
- Less pricy than a line scan camera (with the same line rate).

1.18.1.9.2 Using wxPropView

Let's say we have a rotating drum including an **incremental encoder** (p. 277) connected to the digital inputs (Encoder Source A → Camera Line 4; Encoder Source B → Camera Line 5) and also a light source connected to the digital output, the configuration of the mvBlockscan mode would look like this with wxPropView:

1. Set the **DeviceScanType** in *"Setting → Base → Camera → GenICam → DeviceControl → DeviceScanType"* to **mvBlockscan** .
2. Configure the **Initial situation and settings** (p. 204) **OffSetY**, the **mvBlockscanLinesPerBlock**(>= minimum value of **Height**) and the **mvBlockscanBlockCount**(>= 2) in *"Setting → Base → Camera → GenICam → ImageFormatControl"*.
3. Set the **PixelFormat** in *"Setting → Base → Camera → GenICam → ImageFormatControl"* to e.g. **BayerRG8** for color cameras .
4. Configure an **incremental encoder** (p. 277) in *"Setting → Base → Camera → GenICam → EncoderControl"*, e.g.
   (a) **EncoderSourceA:** Line4
   (b) **EncoderSourceB:** Line5
   (c) **EncoderDivider:** 1
   (d) **EncoderOutputMode:** PositionDown/DirectionDown
5. Configure a trigger in *"Setting → Base → Camera → GenICam → AcquisitionControl"*
   (a) **TriggerSelector:** FrameStart
   (b) **TriggerMode:** On
   (c) **TriggerSource:** Encoder0
6. Adjust the **ExposureTime** to your needs in *"Setting → Base → Camera → GenICam → AcquisitionControl"*.
7. If needed configure the settings for a light source in *"Setting → Base → Camera → GenICam → DigitalIOControl"*.
   (a) **LineSelector:** The output line physically connected with the light source
   (b) **LineSource:** ExposureActive

Optionally you can configure a debounce time for Line4 and Line5 in *"Setting → Base → Camera → GenICam → DigitalIOControl"* if needed.

After setting up the device like this an application will then receive image buffers with a height of *mvBlockscanLinesPerBlock* times `mvBlockscanBlockCount`. When the last of these images shall be terminated early thus with just a portion of the configured `mvBlockscanBlockCount` block in it *"Setting → Base → Camera → GenICam → AcquisitionControl → AcquisitionStop"* can be called. This will terminate the current acquisition and send out an image containing the number of blocks already captured.

1.18.1.10 Working with Event Control

MATRIX VISION devices can generate Event notifications. An Event is a message that is sent to the host application to notify it of the occurrence of an internal event. With *"Setting -> Base -> Camera -> GenICam -> Event Control"* you can handle these notifications.

At the moment, it is possible to handle

- Exposure End (= sensor's exposure end)
- Line 4 (= DigIn0) Rising Edge
- Line 5 (= DigIn1) Rising Edge
- Frame End (= the camera is ready for a new trigger)

1.18.1.10.1 Setting Event notifications using wxPropView

To activate the notifications, just

1. Select the Event via *"Setting -> Base -> Camera -> GenICam -> Event Control -> Event Selector"*, e.g. **ExposureEnd** .
2. Set the *"Event Notification"* to **"On"** .

Afterwards, it is possible to attach a custom callback that gets called whenever the property is modified. E.g. if you want to attach a callback to the Frame ID after the exposure was finished, you have to

1. select *"Setting -> Base -> Camera -> GenICam -> Event Control -> Event Exposure End Data -> Event Exposure End Frame ID"*,
2. right-click on the property, and
3. click on **"Attach Callback"**.

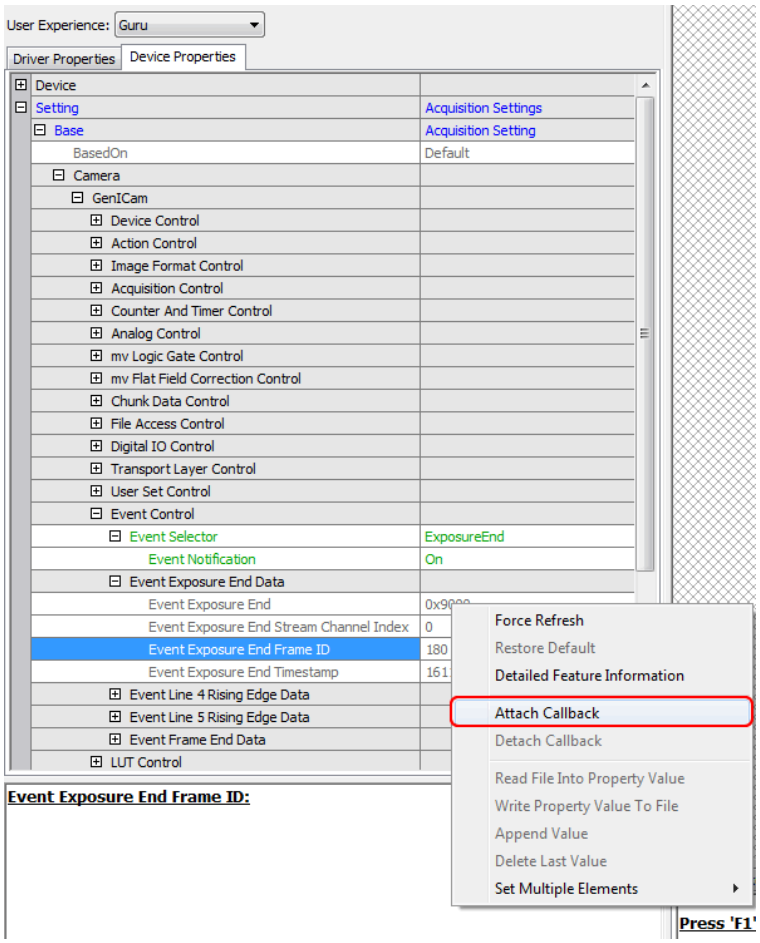


Figure 1: wxPropView - "Attach Callback" to Event Exposure End Frame ID

Now, you can track the property modifications in the output window:

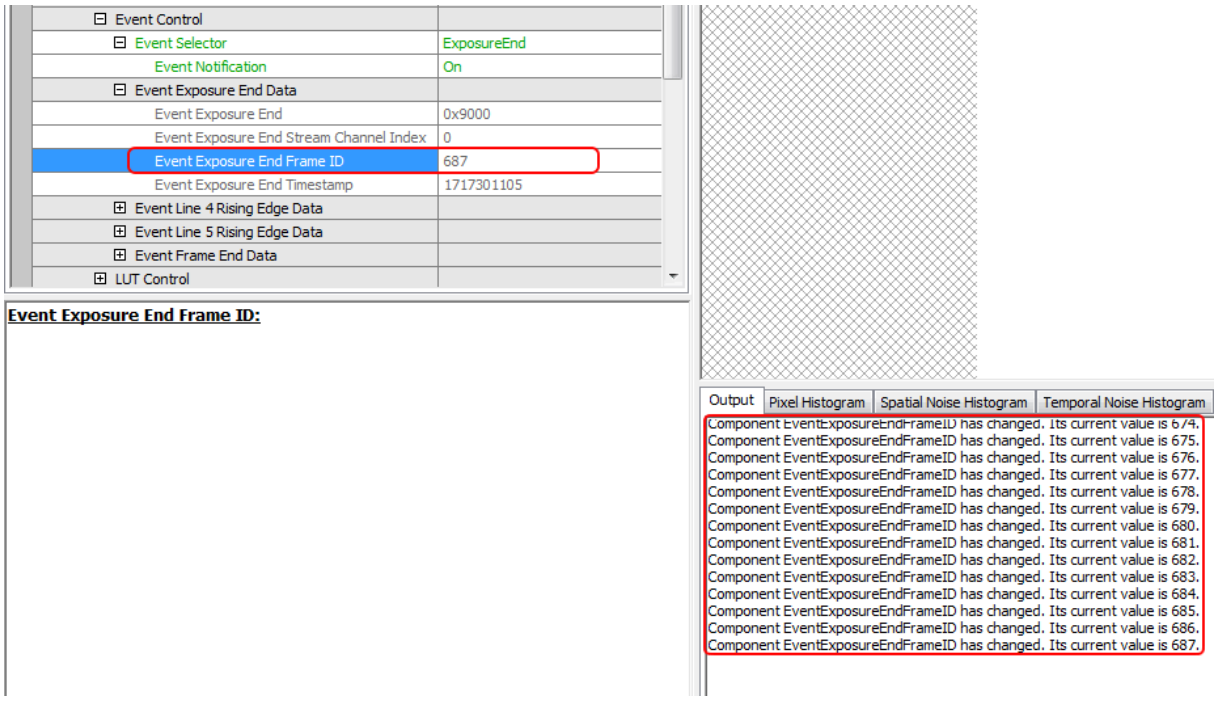


Figure 2: wxPropView - Output window with the Event notifications

You can find a detailed `Callback code example in the C++ API manual`.

1.18.1.11 Polarized Data Extraction

1.18.1.11.1 Introduction

The **mvBlueFOX3-2051p (5.1 Mpix [2464 x 2056])** (p. 459) sensor is equipped with a polarization filter. So every '2 $*$ 2' pixel area is sensitive for polarized light with the angles 0, 90, 135 and 45 degree. In previous versions, you could display the information from these 4 directions in 4 images, each image showing just the one pixel of the '2 $*$ 2' area with the same filtering direction.

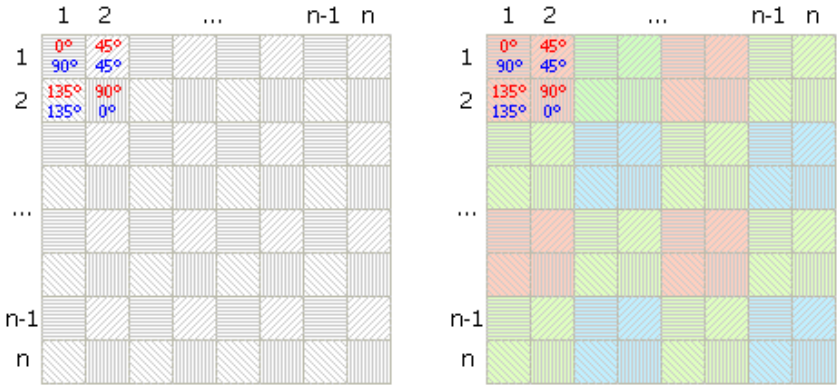


Figure 1: Pixel array mono and color

Since version 2.38.0 we support the calculation of the main polarization angle for each '2 $*$ 2' pixel area. Additionaly it is possible, to display also the degree of the polarized beam. The resulting image will have a width equal to 'input image width / 2' and height equal to 'input image height / 2'. From each 2 by 2 pixel area a single output value will be calculated.

1.18.1.11.2 Polarization value calculation

First we calculate the **Stokes parameters** S_0**,** S_1 **and** S_2 for each 2 by 2 pixel area. Since we just filter for linear polarized light, the circular parameter S_3 will not be used.

- S_0, the total intensity of the beam, is calculated with: $S_0 = P0 + P90$, with $P0$: pixel intensity with 0 degree polarization filter, $P90$: 90 degree polarization filter.
- S_1, the intensity with 0 (horizontal) and 90 (vertical) degree polarization angle. $S_1 = P0 - P90$
- S_2, the intensity with 45 and 135 degree polarization angle. $S_2 = P45 - P135$, with $P45$: pixel intensity with 45 degree polarization filter, $P135$: 135 degree polarization filter.

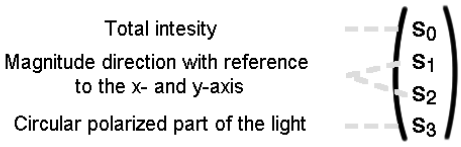


Figure 2: Strokes vector parts.

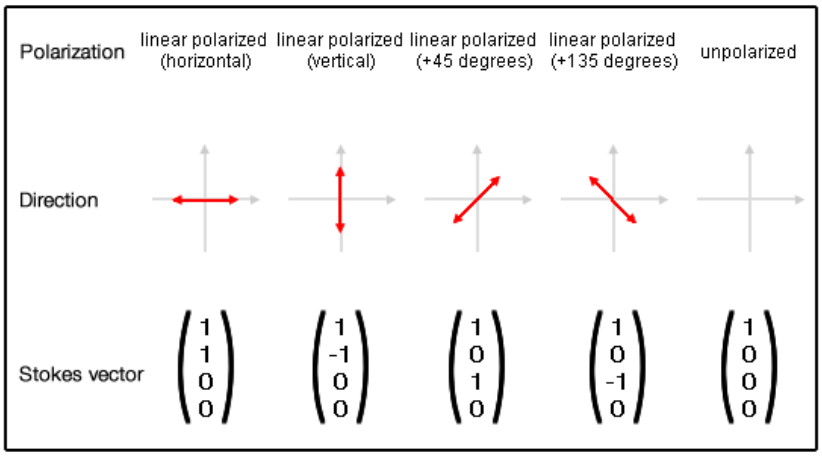


Figure 3: Angel of polarization with corresponding Stokes vector.

1.18.1.11.3 Degree of polarization

The **degree of polarization** Π describes the fraction of the ordered part of the wave. The higher this value, the more polarized is the wave in one single direction. Π is calculated one time for each 2 by 2 pixel area of an image.

$$\Pi = \frac{\sqrt{(P0 - P90)^2 + (P45 - P135)^2}}{(P0 + P90)}$$

1.18.1.11.4 Angle of polarization

The **angle of polarization** Θ describes the angle of the maximum polarization, it ranges from 0 to 180 degree. Like Π, Θ is also calculated one time for each 2 by 2 pixel area.

$$\Theta = \frac{1}{2} * atan\left((P45 - P135)\,;(P0 - P90)\right)$$

1.18.1.11.5 Image Representation

After the calculation of Θ and Π for each 2 by 2 pixel area, there are some ways on how these values can be displayed. The following images show all the same scene, first the raw pixel values and afterwards some pictures with computed values.



Figure 4: Unprocessed image as a reference.

The first possibility is, to display the angle of polarization. Therefore the resulting value for Θ will be mapped to the output pixel format, for example 180 degree would result in a maximum pixel value of 1023 for a 10-bit format.

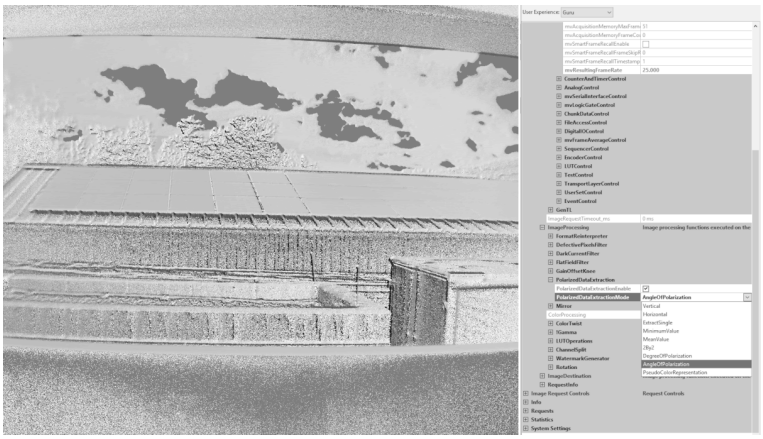


Figure 5: This image shows an example of the representation of the angel of polarization and the corresponding setting in wxPropView.

Another possibility is to display the degree of polarization. The value for Π will be mapped to the output pixel format, for example a maximum polarized beam with Π = 1 would result in a maximum pixel value of 255 for a 8-bit format.

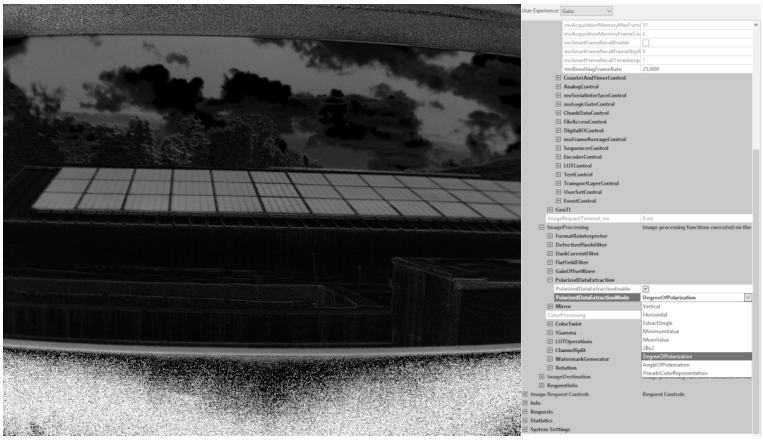


Figure 6: This image shows an example of the representation of the degree of polarization and the corresponding setting in wxPropView.

The third way is to display the angle and the degree of polarization in a single image as a pseudo color representation. Therefore both values will be mapped to a 8-bit HSL image. The angle value Θ represents the hue and the degree Π will be mapped to the saturation value of the image. Finally the HSL image is converted to RGB and displayed.

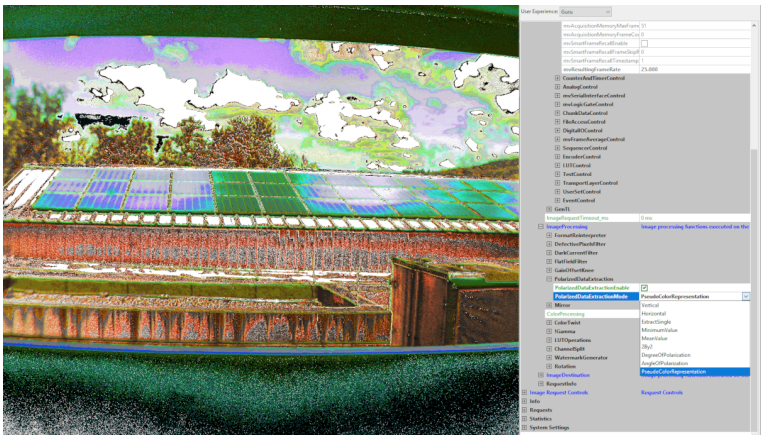


Figure 7: This image shows an example of the representation as pseudo color image and the corresponding setting in wxPropView.

1.18.1.12 Using Video Stream Recording

With the FFmpeg libraries it is possible to record an mvIMPACT Acquire image stream into a compressed video stream.

Since

2.39.0

1.18.1.12.1 Requirements Since the mvIMPACT Acquire API internally uses FFmpeg to record video streams, the FFmpeg libraries need to be present on the target system as well. They can either be built **OR** installed into the systems default library search path **OR** installed somewhere and afterwards an environment variable **MVIMPACT_ACQUIRE_FFMPEG_DIR** can be defined that points to the folder containing the libraries.

Note

Please make sure that you fully understand the license coming with FFmpeg! Have a look at the corresponding `legal section` inside any of the SDK manuals.

At least FFmpeg **4.x** is needed. Older versions of the FFmpeg API are NOT compatible!

1.18.1.12.1.1 Windows

1. Go to `https://ffmpeg.org/download.html` and download the dynamic libraries of FFmpeg (version >= 4.x) according to your operating system (*e.g.* *'ffmpeg-20200809-6e951d0-win64-shared.zip'*)
2. Extract the *.zip file under *'${MVIMPACT_ACQUIRE_DIR}/Toolkits'*.
3. Rename the file to *'ffmpeg-4.2.2-win64-shared'*(64-bit)/*'ffmpeg-4.2.2-win32-shared'*(32-bit) **OR** set an environment variable *e.g.* *'MVIMPACT_ACQUIRE_FFMPEG_DIR'* which points to the folder containing the libraries.

1.18.1.12.2 Recording in wxPropView In wxPropView, a video stream can be recorded by the *'Start'*, *'Pause'* and *'Stop'* buttons at the top right tool-bar. They are however inactive when the video stream recording mode is deactivated OR when the video stream is not correctly set up.



Figure 1: Video stream recording control buttons (inactive)

A video stream needs to be set up first to be able to get recorded. To do so:

1. Select the device to use and open it by clicking on the *'Use'* button.
2. Navigate to the *'Capture'* menu and click on the *'Video Stream Recording...'* option to start a setup dialog.

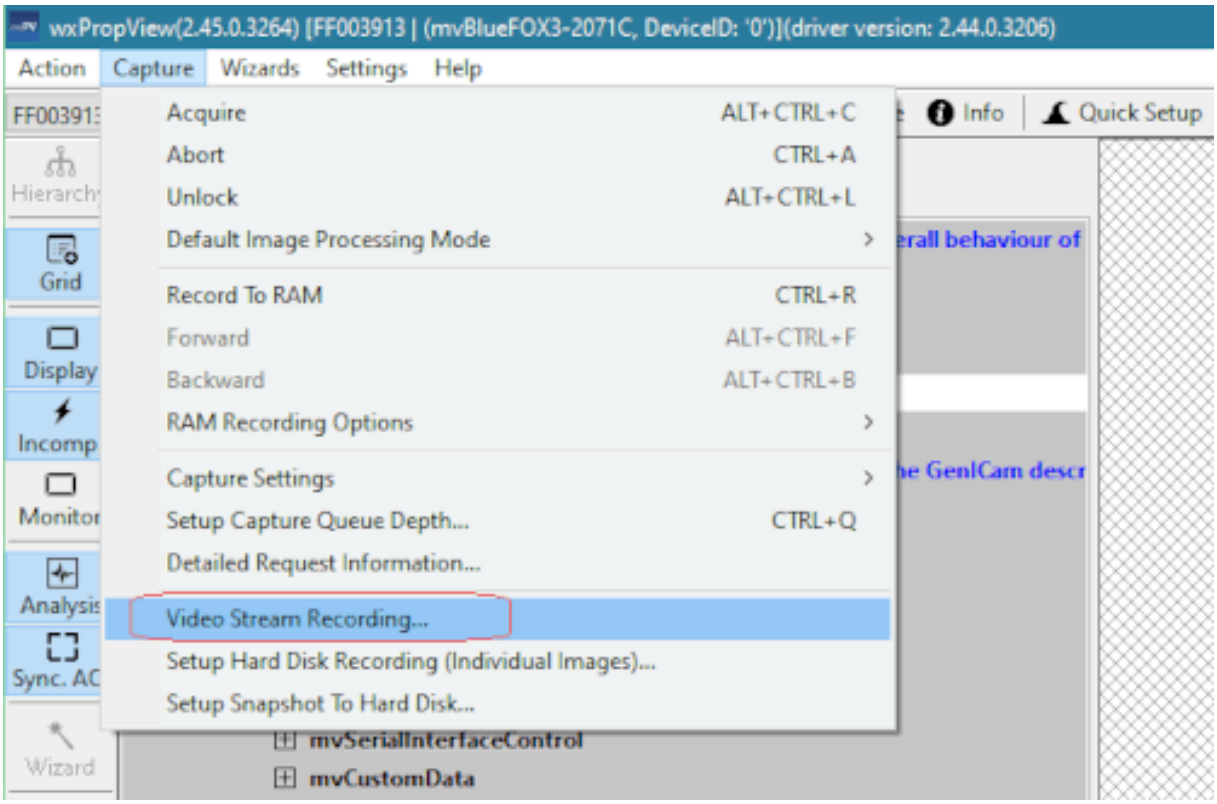


Figure 2: Click 'Video Stream Recording...'

3. A setup dialog will then be initialized as follows. Please read the setup hints in the text box for more information.

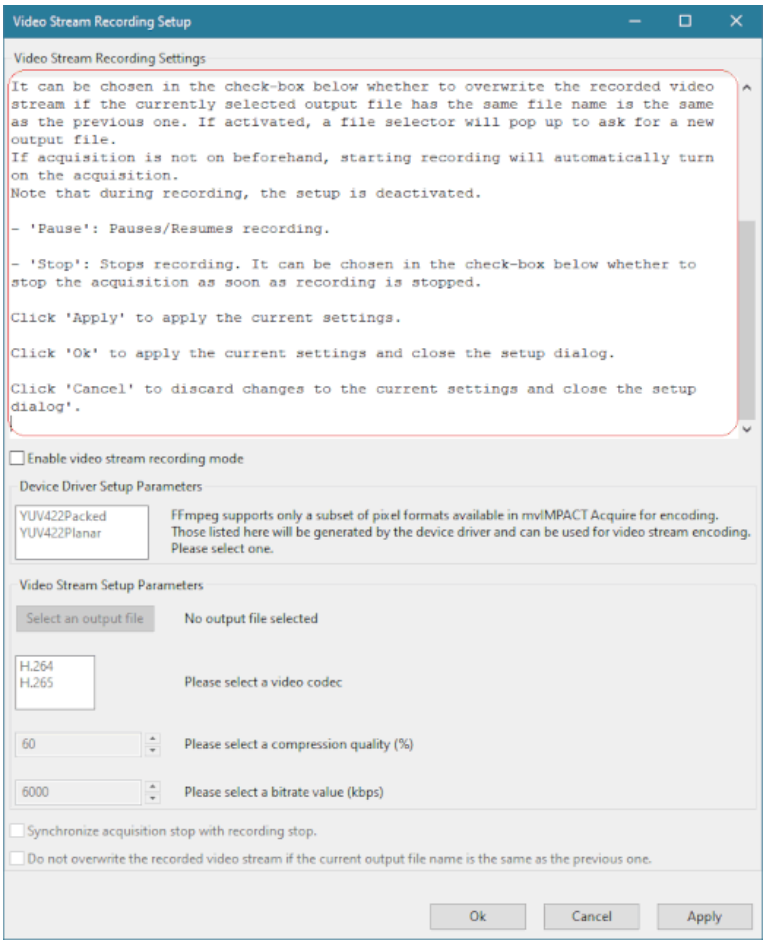


Figure 3: Video stream recording setup dialog

4. Enable the video stream recording mode. Choose a pixel format (e.g. *'YUV422Packed'* or *'YUV422Planar'*) that will be generated by the device driver and used by FFmpeg for video stream encoding. Then click on *'Select an output file'* to create/choose a file to hold the recorded video stream.

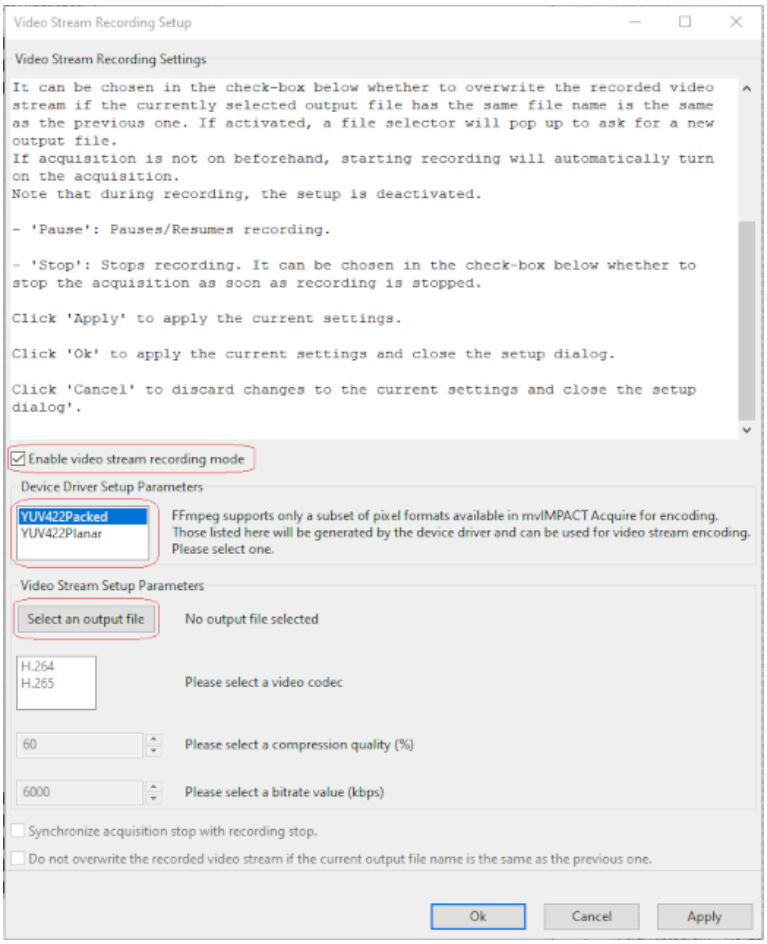


Figure 4: Enable the video stream recording mode and set up device driver related parameters

5. In the file selector, choose a file type (e.g. '∗.mp4' or '∗.m2v') and enter a file name.

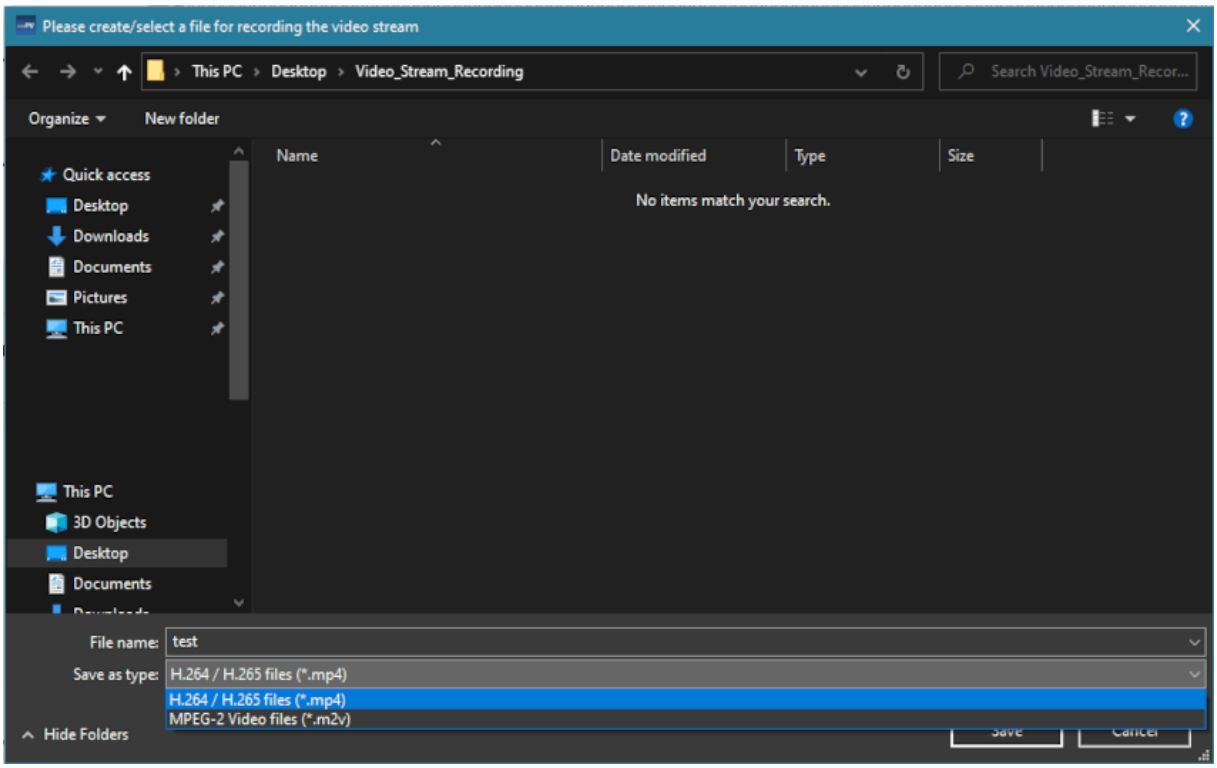


Figure 5: Select an output file

6. Set up video stream related parameters accordingly. In the check boxes below, users are allowed to choose whether to synchronize acquisition stop with recording stop and whether to overwrite the already recorded video stream if the currently selected output file has the same file name as the previous one.

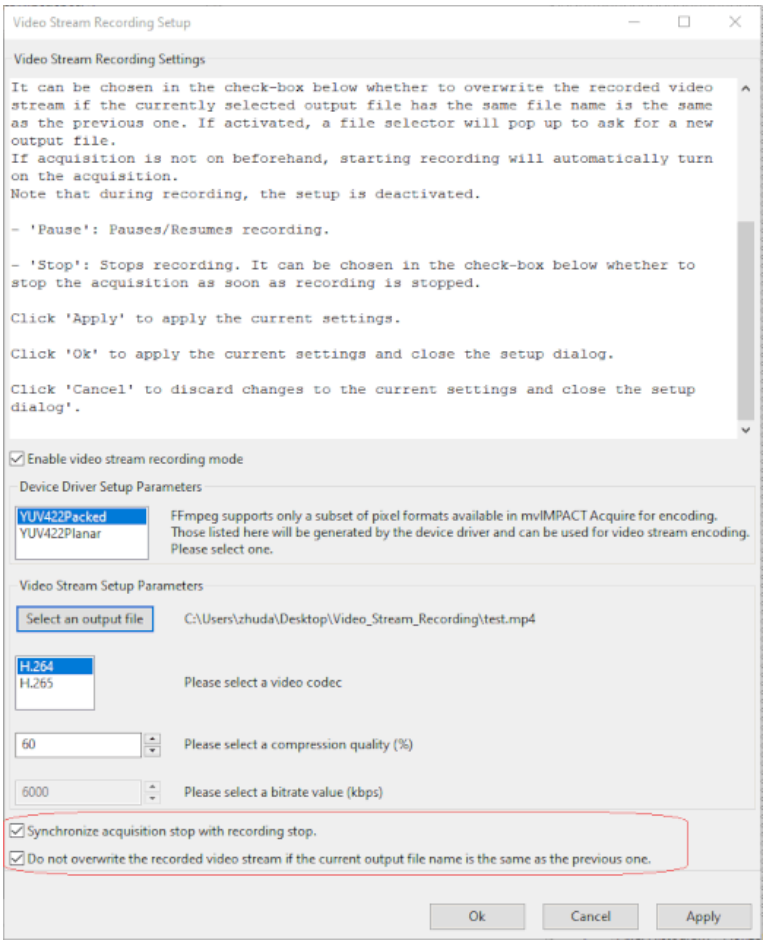


Figure 6: Set up video stream related parameters

7. Once the video stream recording has been set up, click *'Apply'* or *'Ok'* to apply the current settings. Afterwards, a log message in the analysis output will indicate whether the current settings have been applied successfully. If successful, the *'Start'* control button at the top right tool-bar will be enabled.

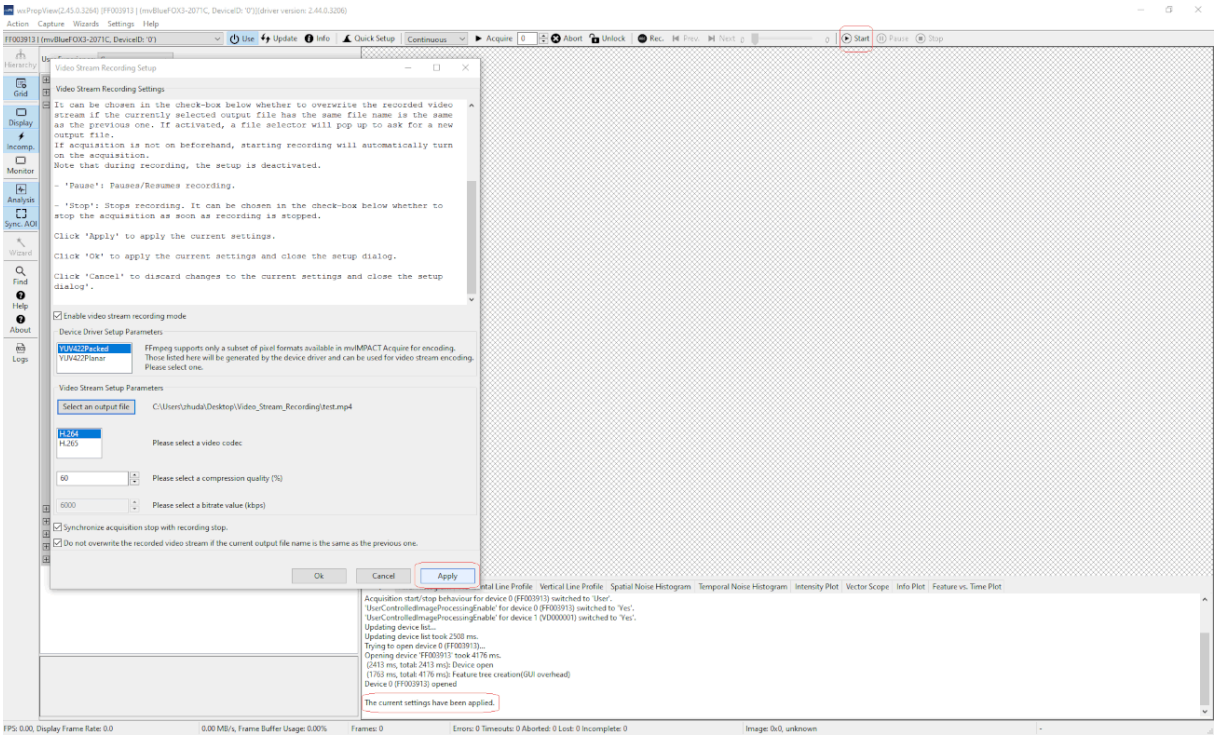


Figure 7: Apply the current settings

Note

When deactivating the video stream recording, uncheck the *'Enable video stream recording mode'* and then click *'Apply'* or *'Ok'* for the settings to take effect.

Once the settings have been applied, users can control the recording process via the *'Start'*, *'Pause'* and *'Stop'* buttons:

- **Start recording**: Click the *'Start'* control button to start recording the video stream. The current recording status and information will be displayed in the analysis output. During recording, the setup dialog as well as the *'Start'* button will be disabled. The *'Pause'* and *'Stop'* buttons will then be enabled.

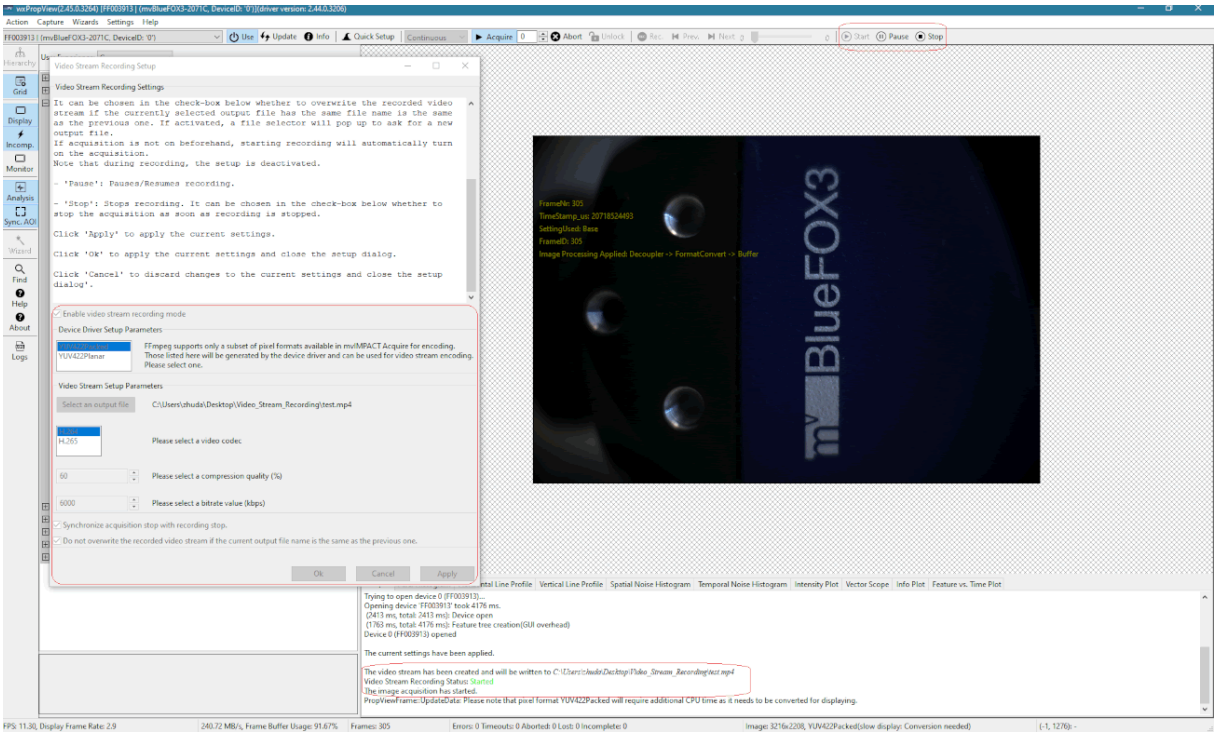


Figure 8: Start recording

- **Pause recording**: Click the *'Pause'* button to pause a running recording. The current recording status will be displayed in the analysis output.

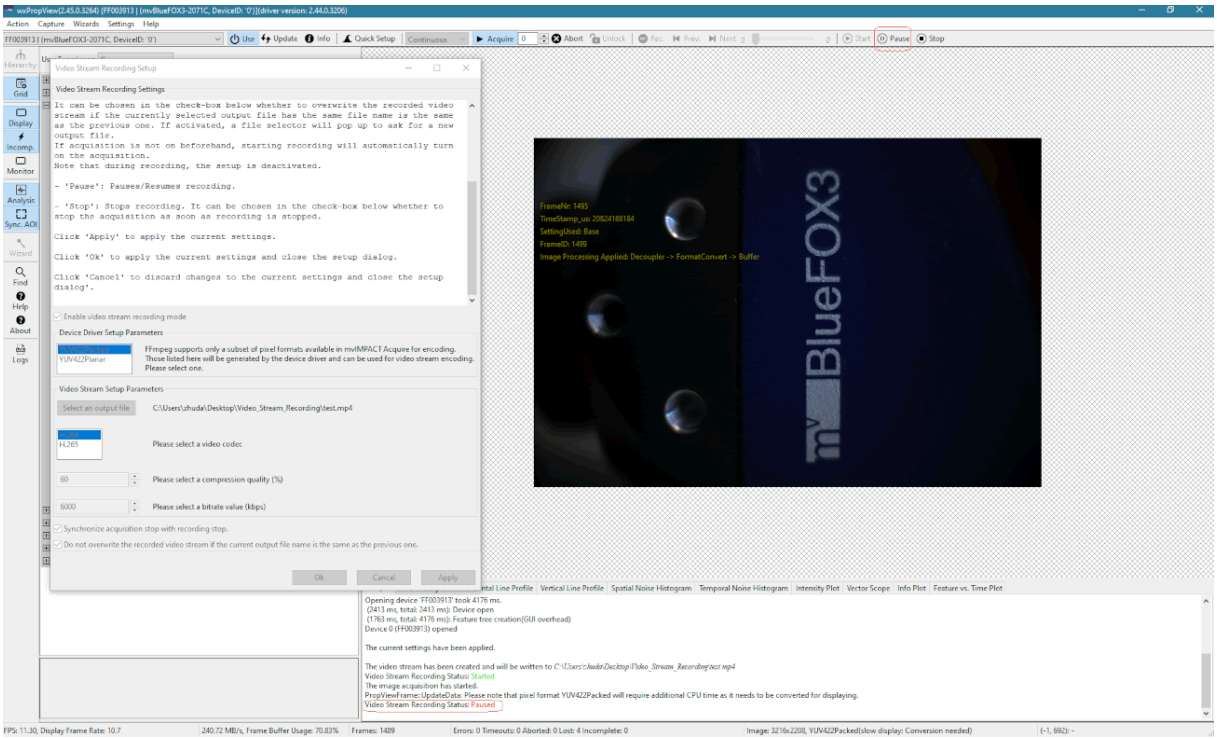


Figure 9: Pause recording

- **Resume recording**: Click the *'Pause'* button to resume a paused recording. The current recording status will be displayed in the analysis output.

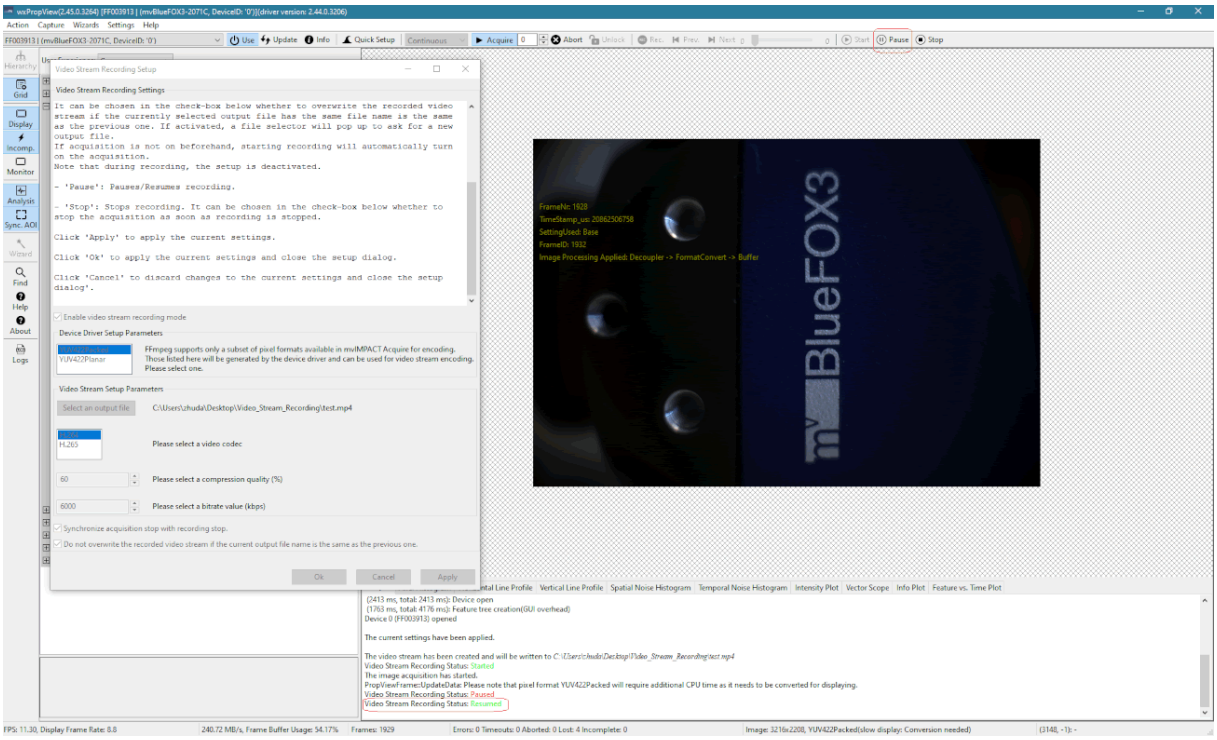


Figure 10: Resume recording

- **Stop recording**: Click the *'Stop'* button to stop recording the video stream. The current recording status and information will be displayed in the analysis output. Once the recording has been stopped, the setup dialog as well as the *'Start'* button will be enabled again. The *'Pause'* and *'Stop'* buttons will then be disabled.

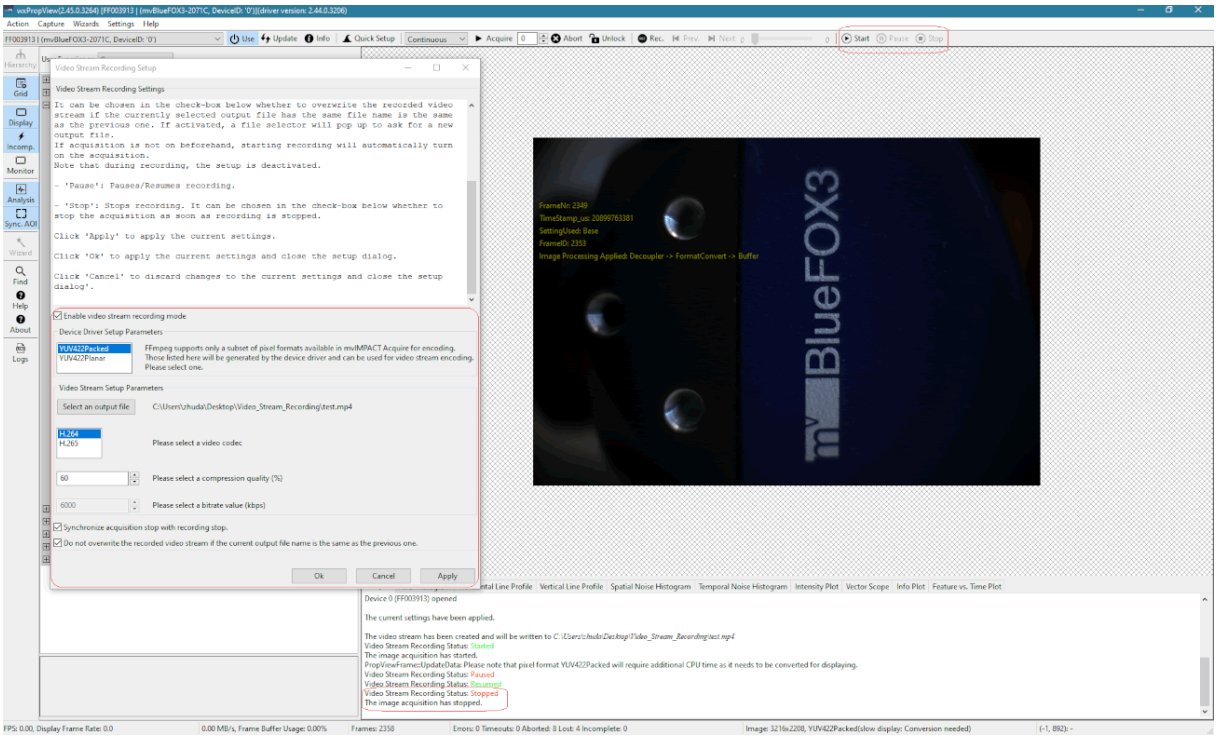


Figure 11: Stop recording

When recording to an output file which has the same file name as the previous one while overwriting the recorded content is not desired:

1. When clicking *'Start'*, a file selector will pop up to ask users to create a new output file with the same file type as the previous one. If a new set of parameters of the video stream recording is needed, please click *'Cancel'* in the file selector and re-configure parameters in the setup dialog.

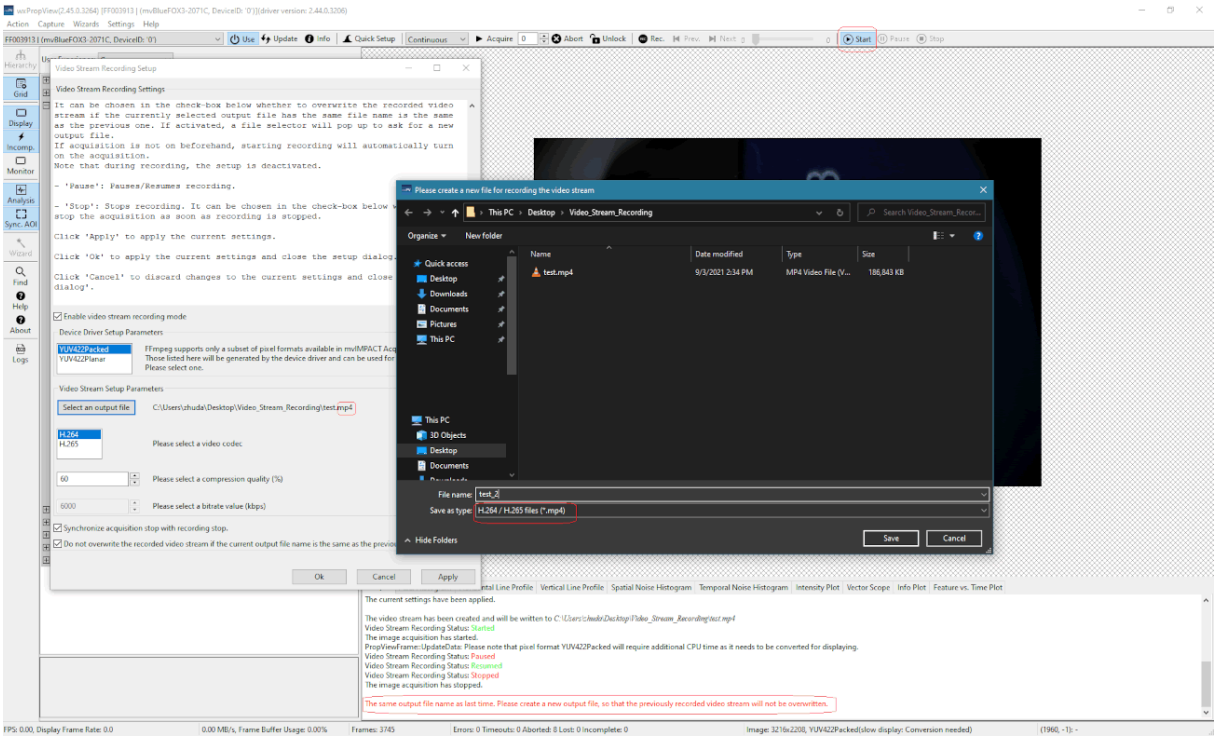


Figure 12: Select a new file when starting to record to an output file with the same file name as the previous one without overwriting

2. Once a new file has been created, the video stream will start to get recorded. The current recording status and information will be displayed in the analysis output. During recording, the setup dialog as well as the *'Start'* button will be disabled. The *'Pause'* and *'Stop'* buttons will then be enabled.

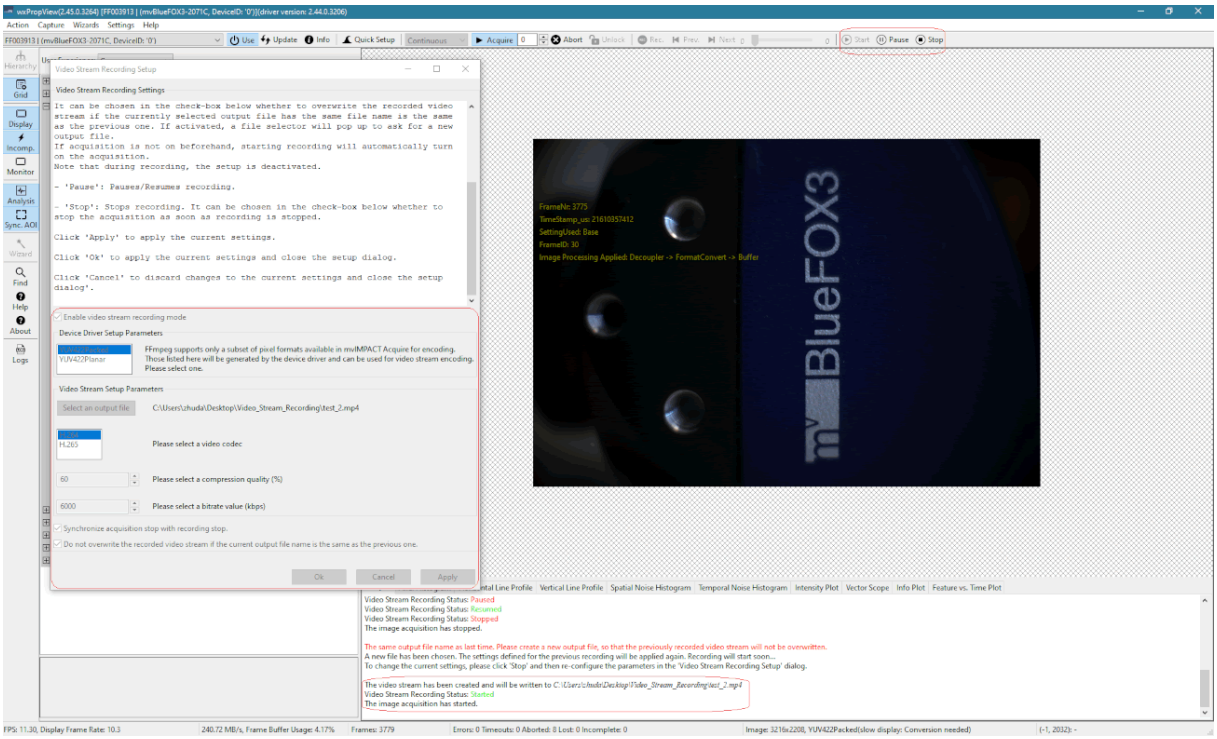


Figure 13: Start recording to an output file with the same file name as the previous one without overwriting

1.18.1.12.3 Recording Using The API

Please refer to the example on how to record a video stream using mvIMPACT Acquire C++ API: `Continuous↩CaptureFFmpeg.cpp` or have a look at the `VideoStream` class.

1.18.2 Improving the acquisition / image quality

There are several use cases concerning the acquisition / image quality of the camera:

- **Correcting image errors of a sensor** (p. 228)
- **Optimizing the color/luminance fidelity of the camera** (p. 236)
- **Reducing noise by frame averaging** (p. 250)
- **Optimizing the bandwidth** (p. 275)
- **Setting a flicker-free auto expose and auto gain** (p. 255)
- **Working with binning / decimation** (p. 258)
- **Minimizing sensor pattern of mvBlueFOX3-1100G** (p. 261)
- **Working with the dual gain feature of mvBlueFOX3-2071/2071a** (p. 263)
- **Working With Gain And Black-Level Values Per Color Channel** (p. 267)

1.18.2.1 Correcting image errors of a sensor

1.18.2.1.1 Introduction Due to random process deviations, technical limitations of the sensors, etc. there are different reasons that image sensors have image errors. MATRIX VISION provides several procedures to correct these errors, by default these are host-based calculations, however some camera families support camera-based corrections, which saves dozens of % CPU load and lowers latency.

Camera Family	Algorithm-↩ Based detection and correction	List-Based correction	Storing facility for defective-pixel list	Flat-Field Correction (Host)	Flat-Field Correction (Camera)
mvBlue↩ COUGAR-X	-	-	X	X	X
mvBlue↩ COUGAR-XD	If binning/decimation is on -> no list is stored	X	X	X	
mvBlue↩ COUGAR-XT	X	-	-	X	X
mvBlueFOX3	If binning/decimation is on -> no list is stored	X	X	X	-

Generally, removing defect pixels requires two sub-tasks:

- Detection of defective pixels
- Correction of defective pixels

Both tasks can performed in different "locations":

- Detection and correction on the host using mvIMPACT Acquire

- Detection on the host using mvIMPACT Acquire, correction on the camera using the camera's `mv↩DefectivePixelCorrectionControl` in the list-based mode
- Detection and correction on the camera using `mvDefectivePixelCorrectionControl` in the algorithm-based mode.

If detection is not happening in real-time, meaning during the image acquisition itself, it is necessary to store the detected defects somewhere. This can be either on the camera or the host or both.

1.18.2.1.2 Defect pixel detection using mvIMPACT Acquire As mentioned, the defect pixel list can be generated using mvIMPACT Acquire. Since there are three types of defects, mvIMPACT Acquire offers three calibration methods for detection:

1. **leaky pixel** (in the dark)
   which indicates pixels that produce a higher read out code than the average
2. **hot pixel** (in standard light conditions)
   which indicates pixels that produce a higher non-proportional read out code when temperatures are rising
3. **cold pixel** (in standard light conditions)
   which indicates pixels that produce a lower read out code than average when the sensor is exposed (e.g. caused by dust particles on the sensor)

Note

Please use either an Mono or RAW Bayer image format when detecting defective pixel data in the image.

1.18.2.1.2.1 Detecting leaky pixels To detect leaky pixels the following steps are necessary:

1. Set gain ("`Setting -> Base -> Camera -> GenICam -> Analog Control -> Gain = 0 dB`") and exposure time "`Setting -> Base -> Camera -> GenICam -> Acquisition Control -> ExposureTime = 360 msec`" to the given operating conditions
   The total number of defective pixels found in the array depend on the gain and the exposure time.
2. Black out the lens completely
3. Set the (Filter-) "`Mode = Calibrate leaky pixel`"
4. Acquire an image (e.g. by pressing **Acquire** in wxPropView with "`Acquisition Mode = Single↩Frame`")

The filter checks:

```
Pixel > LeakyPixelDeviation_ADCLimit // (default value: 50)
```

All pixels above this value are considered as leaky pixel.

1.18.2.1.2.2 Detecting hot or cold pixels

> **Note**
>
> With "Mode = Calibrate Hot And Cold Pixel" you can execute both detections at the same time.

To detect hot or cold pixels the following steps are necessary:

1. You will need a uniform sensor illumination approx. 50 - 70 % saturation (which means an average gray value between 128 and 180)
2. Set the (Filter-) "`Mode = Calibrate Hot Pixel`" or "`Mode = Calibrate Cold Pixel`" or "`Mode = Calibrate Hot And Cold Pixel`"
3. Acquire an image (e.g. by pressing **Acquire** in wxPropView with "`Acquisition Mode = Single↩ Frame`")

The filter checks:

```
Pixel > T[hot] // (default value: 15 %)

// T[hot] = deviation of the average gray value

Pixel < T[cold] // (default value: 15 %)

// T[cold] = deviation of the average gray value
```

> **Note**
>
> Repeating the defective pixel corrections will accumulate the correction data which leads to a higher value in "`DefectivePixelsFound`". If you want to reset the correction data or repeat the correction process you have to set the filter mode to "`Reset Calibration Data`". In oder to limit the amount of defective pixels detected the "`DefectivePixelsMaxDetectionCount`" property can be used.

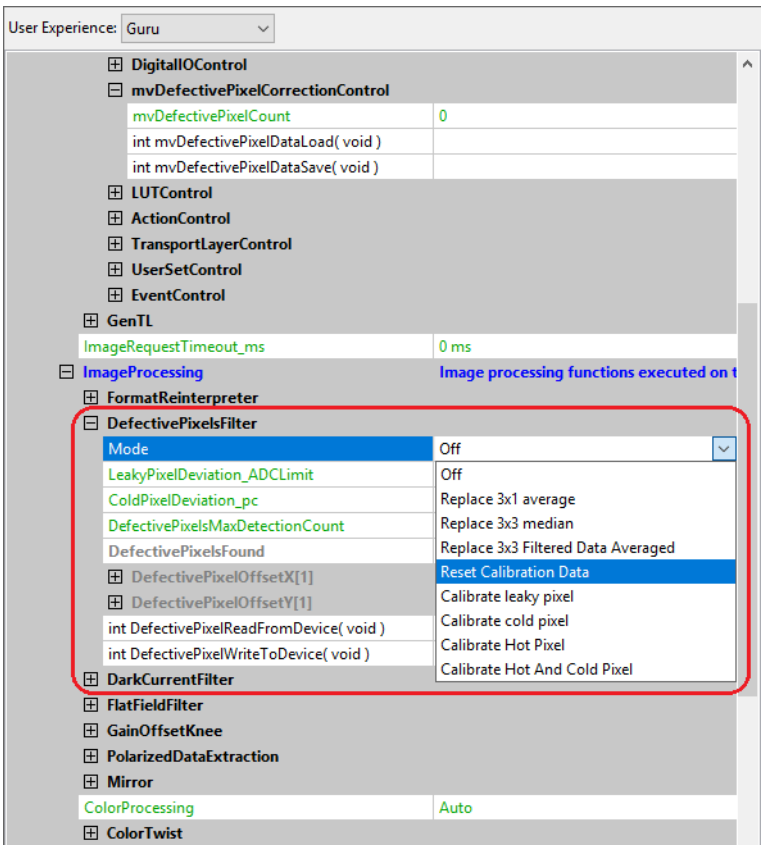


Figure 1: Image corrections: DefectivePixelsFilter

1.18.2.1.3 Storing defective pixel data on the device To save and load the defective pixel data, appropriate functions are available:

- **"int mvDefectivePixelDataLoad(void)"**
- **"int mvDefectivePixelDataSave(void)"**

The section *"Setting -> Base -> ImageProcessing -> DefectivePixelsFilter"* was also extended (see Figure 2). First, the "DefectivePixelsFound" indicates the number of found defective pixels. The coordinates are available through the properties "DefectivePixelOffsetX" and "DefectivePixelOffsetY" now. In addition to that it is possible to edit, add and delete these values manually (via right-click on the "DefectivePixel↩Offset" and select **"Append Value"** or **"Delete Last Value"**). Second, with the functions

- **"int mvDefectivePixelReadFromDevice(void)"**
- **"int mvDefectivePixelWriteToDevice(void)"**

you can exchange the data from the filter with the camera and vice versa.

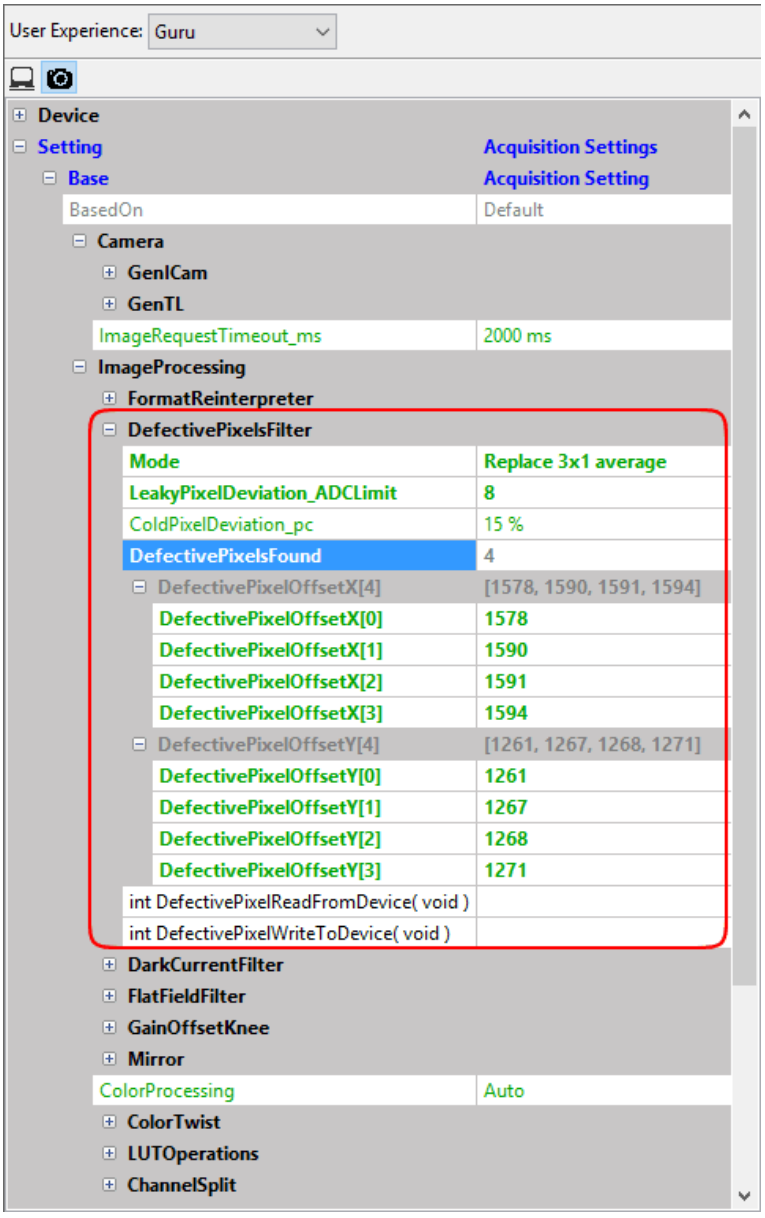


Figure 2: Image corrections: DefectivePixelsFilter (since driver version 2.17.1 and firmware version 2.12.406)

Just right-click on "mvDefectivePixelWriteToDevice" and click on **"Execute"** to write the data to the camera (and hand over the data to the Storing pixel data on the device). To permanently store the data inside the cameras non-volatile memory afterwards "mvDefectivePixelDataSave" must be called as well!

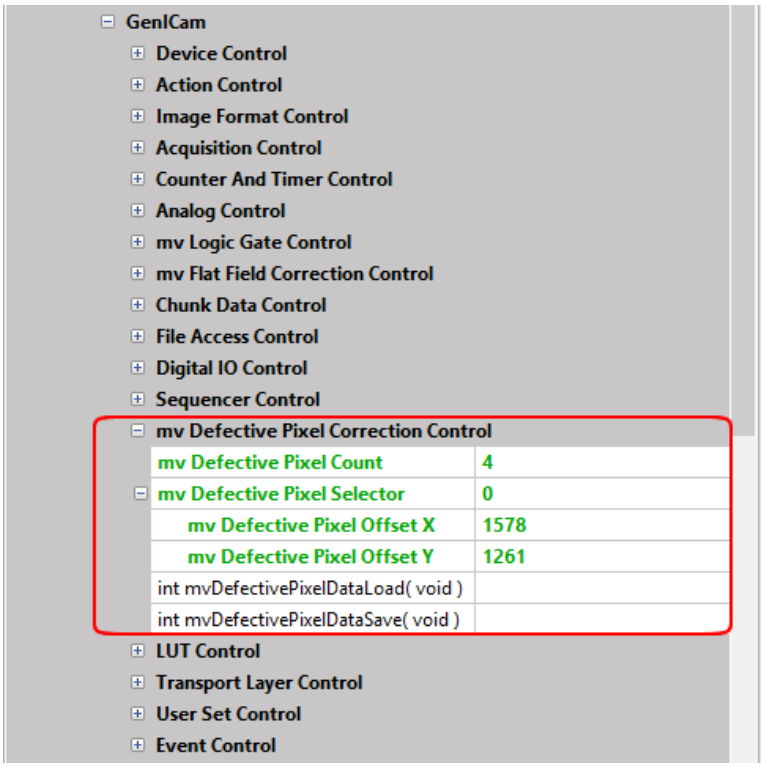


Figure 3: Defective pixel data are written to the camera (since driver version 2.17.1 and firmware version 2.12.406)

While opening the camera, the camera will load the defective pixel data from the camera. If there are pixels in the filter available (via calibration), nevertheless you can load the values from the camera. In this case the values will be merged with the existing ones. I.e., new ones are added and duplicates are removed.

1.18.2.1.4 Defect Pixel correction using mvImpactAcquire

After a defect-list is generated, a correction can be performed using mvIMPACT Acquire.

To correct the defective pixels various substitution methods exist:

1. **"Replace 3x1 average"**
   which substitutes the detected defective pixels with the average value from the left and right neighboring pixel (3x1)
2. **"Replace 3x3 median"**
   which substitutes the detected defective pixels with the median value calculated from the nearest neighboring in a 3 by 3 region
3. **"Replace 3x3 Filtered Data Averaged"**
   which substitutes and treats the detected defective pixels as if they have been processed with a 3 by 3 filter algorithm before reaching this filter
   Only recommended for devices which do not offer a defective pixel compensation; packed RGB or packed YUV444 data is needed. See enumeration value `dpfmReplaceDefectivePixelAfter3x3Filter` in the corresponding API manual for additional details about this algorithm and when and why it is needed

1.18.2.1.5 List-based defect Pixel correction on the camera As described before, it is possible to upload lists of defect pixel onto the camera. Different algorithms can be used to determine whether a pixel is defective or not, which is dependent of how much it is allowed a pixel to deviate, temperature, gain, and exposure time. As described before, the list-based correction is deterministic, meaning it is exactly known which pixels will be corrected.

Anyhow, the list-based correction has some disadvantages:

- A default list is stored in the camera during production, but this might to fit to the target application because of much different temperature / exposure time setting
  → It is necessary to create the list using a detection algorithm (or mvIMPACT Acquire support)
- During time and sensor aging, new defects could/will appear
- It doesn't work in binning/decimation modes
- The memory for storing defective pixels is limited

1.18.2.1.6 Adaptive / algorithm-based correction on the camera In this case, the camera performs detection and correction on-the-fly without using any defect-list.

The adaptive correction addresses the above-mentioned disadvantages of the list-based method. While the correction itself (this is which pixels are used to correct an identified defect) is the same, no static information from a list is used, instead they are detected "on the fly".

To use reasonable thresholds, knowledge of the noise statistics of the sensor is used to detect the outliers. These will be corrected also on the fly. Because this is a dynamic approach, it also works in binning/decimation modes and would also detect new appearing defects.

Nevertheless, there are some disadvantages:

- It is non-deterministic
- Wrong positives can be detected, meaning non-defect pixels could be treated as defect
- If pixels are at the edge of the used thresholds, it could be corrected in one frame, but not in the next

1.18.2.1.7 Flat-Field Correction

Each pixel of a sensor chip is a single detector with its own properties. Particularly, this pertains to the sensitivity as the case may be the spectral sensitivity. To solve this problem (including lens and illumination variations), a plain and equally "colored" calibration plate (e.g. white or gray) as a flat-field is snapped, which will be used to correct the original image. Between flat-field correction and the future application you must not change the optic. To reduce errors while doing the flat-field correction, a saturation between 50 % and 75 % of the flat-field in the histogram is convenient.

Note

Flat-field correction can also be used as a destructive watermark and works for all f-stops.

To make a **flat field correction** following steps are necessary:

1. You need a plain and equally "colored" calibration plate (e.g. white or gray)
2. No single pixel may be saturated - that's why we recommend to set the maximum gray level in the brightest area to max. 75% of the gray scale (i.e., to gray values below 190 when using 8-bit values)
3. Choose a `BayerXY` in *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> PixelFormat"*.

4. Set the (Filter-) "`Mode = Calibrate`" (Figure 4)
5. Start a Live snap (**"Acquire"** with "`Acquisition Mode = Continuous`")
6. Finally, you have to activate the correction: Set the (Filter-) "`Mode = On`"
7. Save the settings including the correction data via "`Action -> Capture Settings -> Save Active Device Settings`"
   (Settings can be saved in the Windows registry or in a file)

Note

After having re-started the camera you have to reload the capture settings vice versa.

The filter snaps a number of images (according to the value of the `CalibrationImageCount`, e.g. `5`) and averages the flat-field images to one correction image.

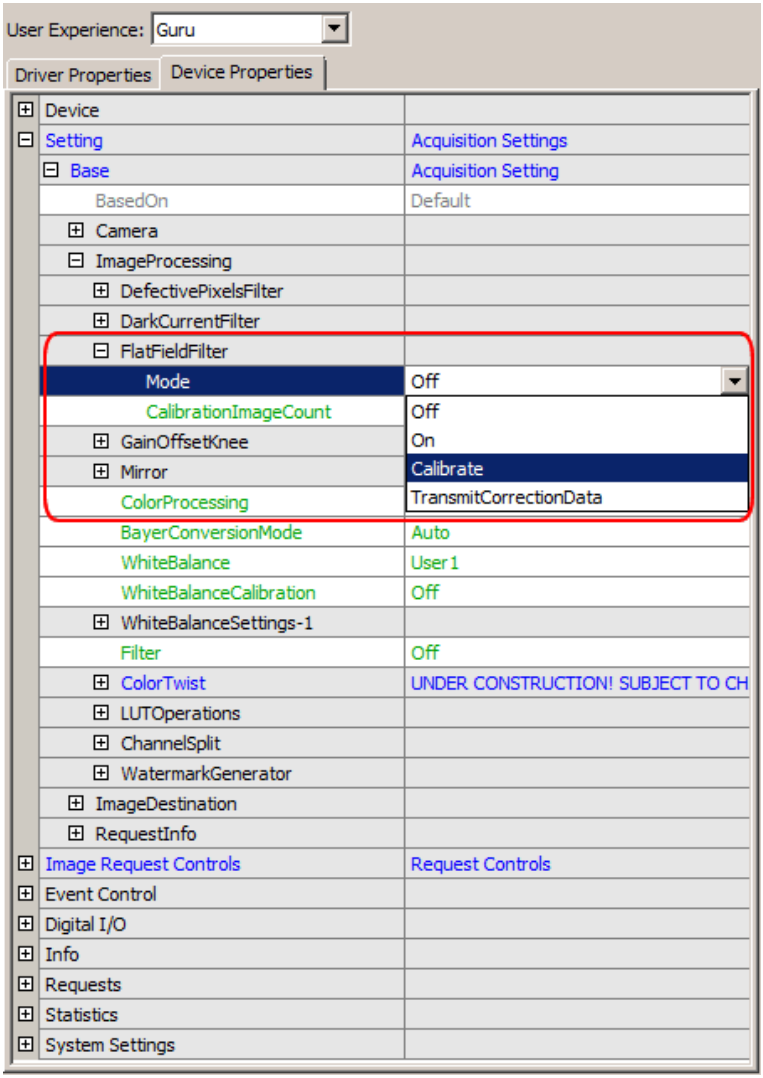


Figure 4: Image corrections: Host-based flat field correction

1.18.2.1.7.1 Host-based Flat-Field Correction With Calibration AOI

In some cases it might be necessary to use just a specific area within the camera's field of view to calculate the correction values. In this case just a specific AOI will be used to calculate the correction factor.

You can set the **"host-based flat field correction"** in the following way:

1. All necessary setting can be found under **"ImageProcessing"->** **"FlatfieldFilter"**.
2. Stop **"Continuous"** acquisition mode.
3. Set **"CalibrationImageCount"** to, for example, 5.
4. Set **"Mode"** to "Calibrate".
5. Set **"CalibrationAoiMode"** to "UseAoi".
6. Set the properties (**"X, Y and W, H"**) appeared under **"CalibrationAOI"** to the desired AOI.
7. Start **"Continuous"** acquisition mode.
8. Finally, you have to activate the correction: Set the **"Mode"** to "On".

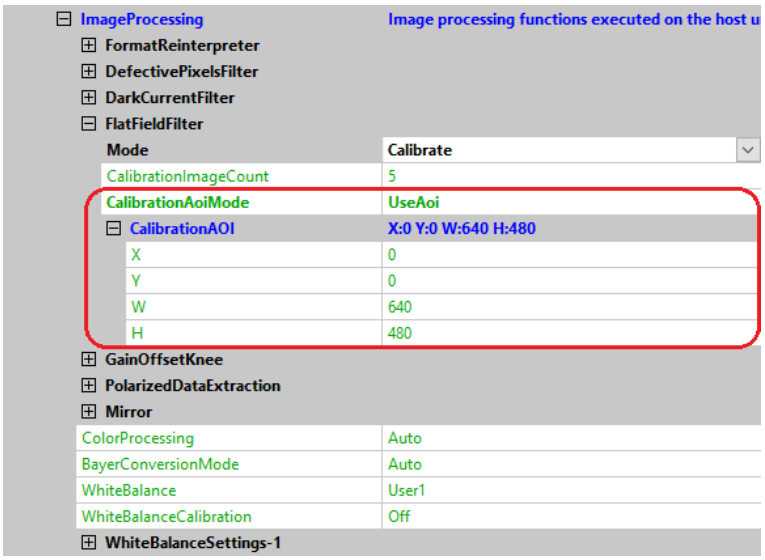


Figure 5: Image corrections: Host-based flat field correction with calibration AOI

1.18.2.1.7.2 Host-based Flat-Field Correction With Correction AOI

In some cases it might be necessary to correct just a specific area in the camera's filed of view. In this case the correction values are only applied to a specific area. For the rest of the image, the correction factor will be just 1.0.

You can set the **"host-based flat field correction"** in the following way:

1. All necessary setting can be found under **"ImageProcessing"** -> **"FlatfieldFilter"**.
2. Stop **"Continuous"** acquisition mode.
3. Set **"CalibrationImageCount"** to, for example, 5.
4. Set **"Mode"** to "Calibrate".
5. Start **"Continuous"** acquisition mode.
6. Now, you have to activate the correction: Set the **"Mode"** to "On".

7. Set **"CorrectionAOIMode"** to "UseAoi".
8. Finally use the properties (**"X, Y and W, H"**) which appeared under **"CorrectionAOI"** to configure the desired AOI.

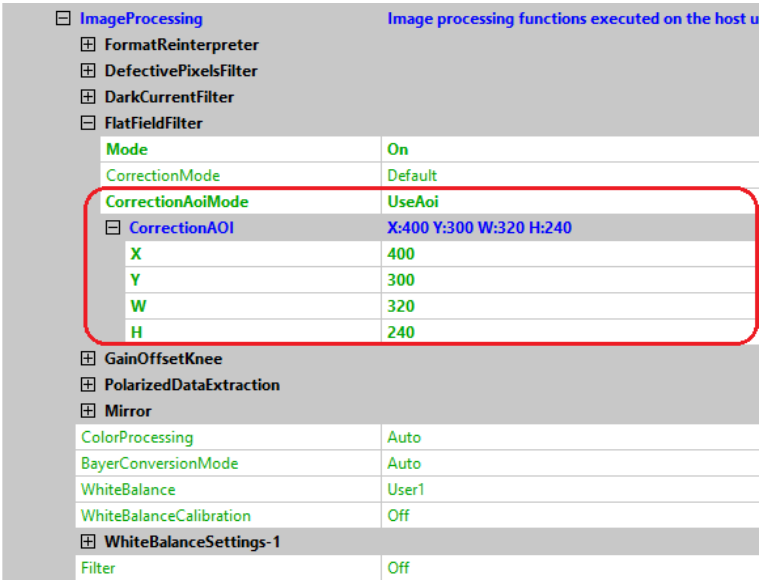


Figure 6: Image corrections: Host-based flat field correction with correction AOI

1.18.2.2 Optimizing the color/luminance fidelity of the camera

Purpose of this chapter is to optimize the color image of a camera, so that it looks as natural as possible on different displays and for human vision.

This implies some linear and nonlinear operations (e.g. display color space or Gamma viewing LUT) which are normally not necessary or recommended for machine vision algorithms. A standard monitor offers, for example, several display modes like *sRGB*, *"Adobe RGB"*, etc., which reproduce the very same color of a camera color differently.

It should also be noted that users can choose for either

- camera based settings and adjustments or
- host based settings and adjustments or
- a combination of both.

Camera based settings are advantageous to achieve highest calculating precision, independent of the transmission bit depth, lowest latency, because all calculations are performed in FPGA on the fly and low CPU load, because the host is not invoked with these tasks. These camera based settings are

- **gamma correction** (p. 239)
- **negative gain / gain** (p. 239)
- **look-up table (LUT)** (p. 239)
- **white balance** (p. 243)

- **offset** (p. 245)
- **saturation and color correction** (p. 247)

Host based settings save transmission bandwidth at the expense of accuracy or latency and CPU load. Especially performing gain, offset, and white balance in the camera while outputting RAW data to the host can be recommended.

Of course host based settings can be used with all families of cameras (e.g. also mvBlueFOX).

Host based settings are:

- look-up table (*LUTOperations*)
- color correction (*ColorTwist*)

To show the different color behaviors, we take a color chart as a starting point:

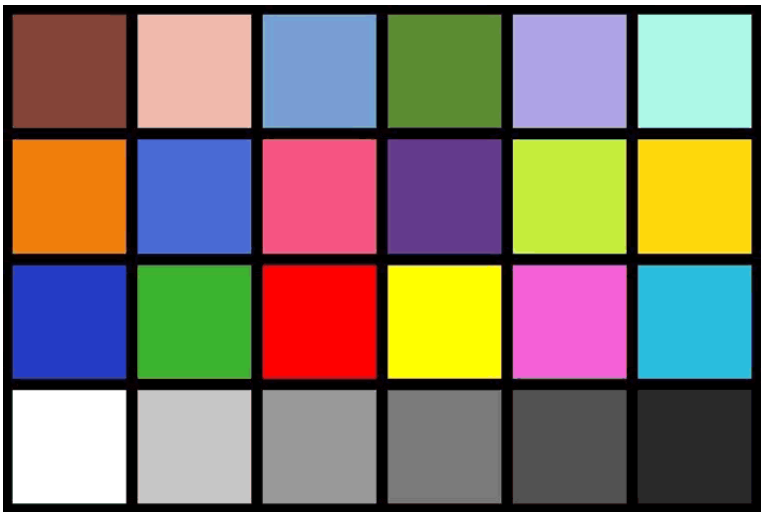


Figure 1: Color chart as a starting point

If we take a *SingleFrame* image without any color optimizations, an image can be like this:



Figure 2: SingleFrame snap without color optimization



Figure 3: Corresponding histogram of the horizontal white to black profile

As you can see,

- saturation is missing,
- white is more light gray,
- black is more dark gray,
- etc.

Note

You have to keep in mind that there are two types of images: the one generated in the camera and the other one displayed on the computer monitor. Up-to-date monitors offer different display modes with different color spaces (e.g. sRGB). According to the chosen color space, the display of the colors is different.

The following figure shows the way to a perfect colored image

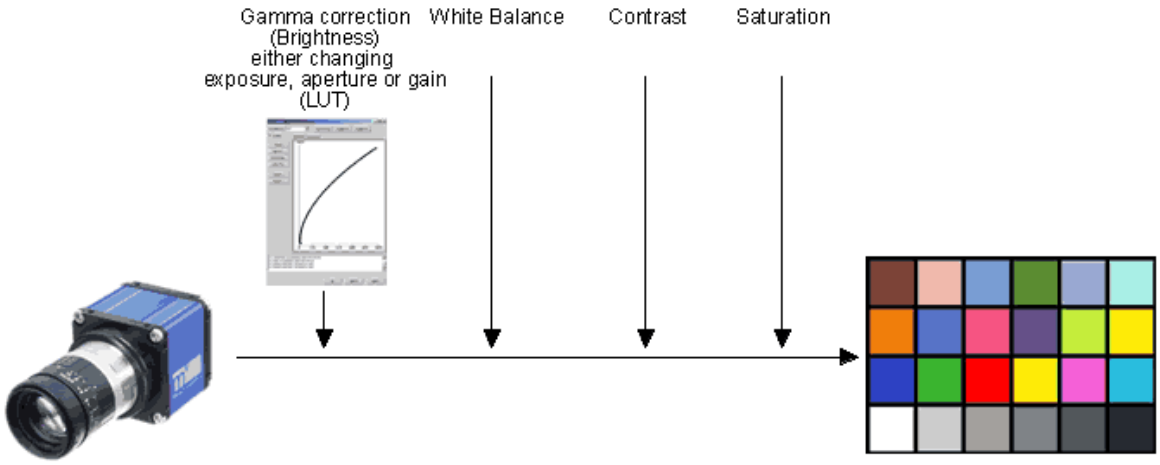


Figure 4: The way to a perfect colored image

including these process steps:

1. Do a **Gamma correction (Luminance)** (p. 239),
2. make a **White balance** (p. 243) and
3. Improve the **Contrast** (p. 245).
4. Improve **Saturation** (p. 247), and use a **"color correction matrix"** for both
   (a) the sensor and / or
   (b) the monitor.

The following sections will describe the single steps in detail.

1.18.2.2.1 Step 1: Gamma correction (Luminance)

First of all, a **Gamma correction** can be performed to change the image in a way how humans perceive light and color.

To do this, you have the following 3 possibilities:

- set Gamma using Gamma feature in AnalogControl,
- set Gamma using a LUT in LUTControl or
- set Gamma using a LUT in ImageProcessing.

Note

The first two possibilities are running on the camera, whereas the third is running on host system

The first and easiest way to set a Gamma is using the gamma feature in AnalogControl (*"Setting -> Base -> Camera -> AnalogControl"*)

mvGammaSelector is used to switch between a fixed mvSRGB gamma curve and mvuser which enables the Gamma property to set gamma between a value of 0.1 and 10.0 The Gamma feature is working on the camera and enabled by setting mvGammaEnable

Note

mvGammaEnable can only be set if no LUT in LUTControl is active

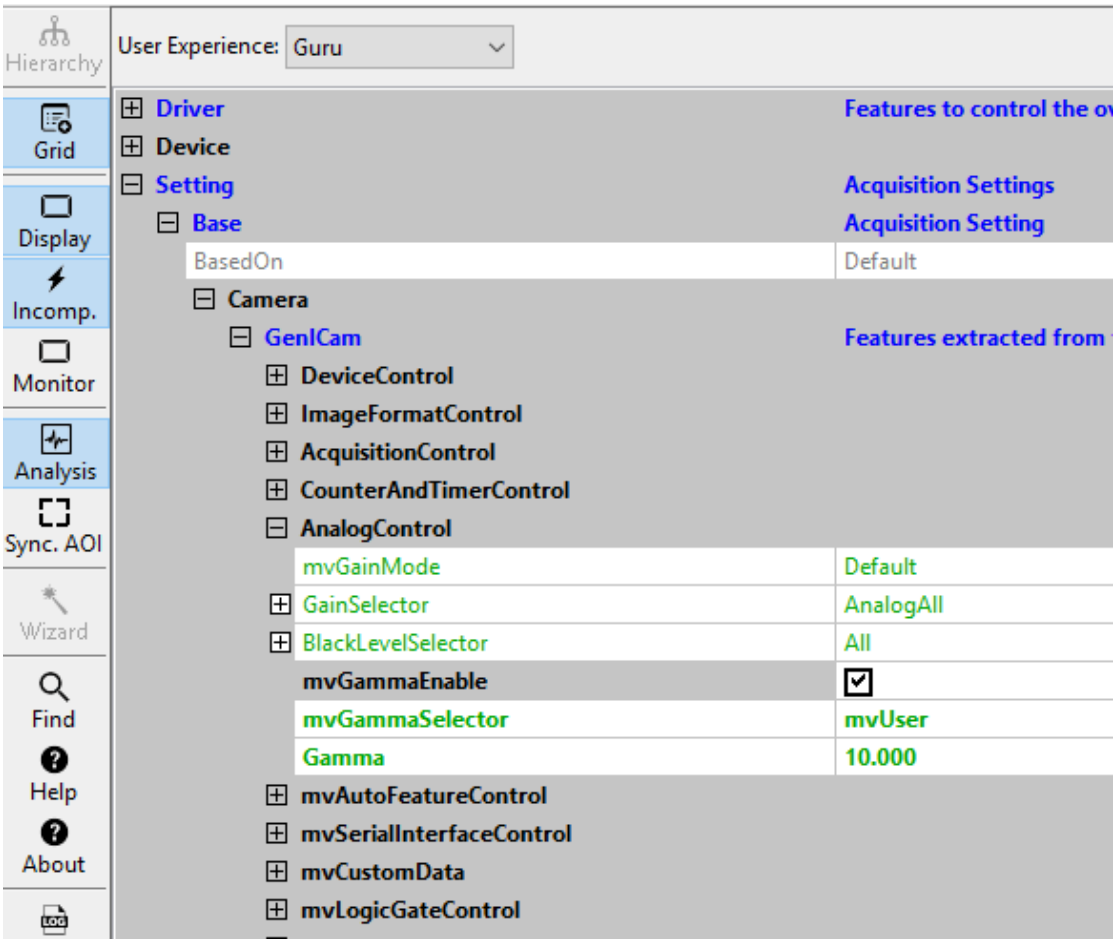


Figure 5: Gamma using AnalogControl

Note

To avoid artifacts while changing the Gamma value, stop Acquisition before changing values and afterwards, start Acquisition

The second way is setting Gamma via LUTControl using **wxPropView** (p. 99), here, the following steps have to be done:

1. Click on *"Setting -> Base -> Camera -> GenICam -> LUT Control -> LUT Selector"*.
2. Afterwards, click on **"Wizard"** to start the LUT Control wizard tool.
   The wizard will load the data from the camera.

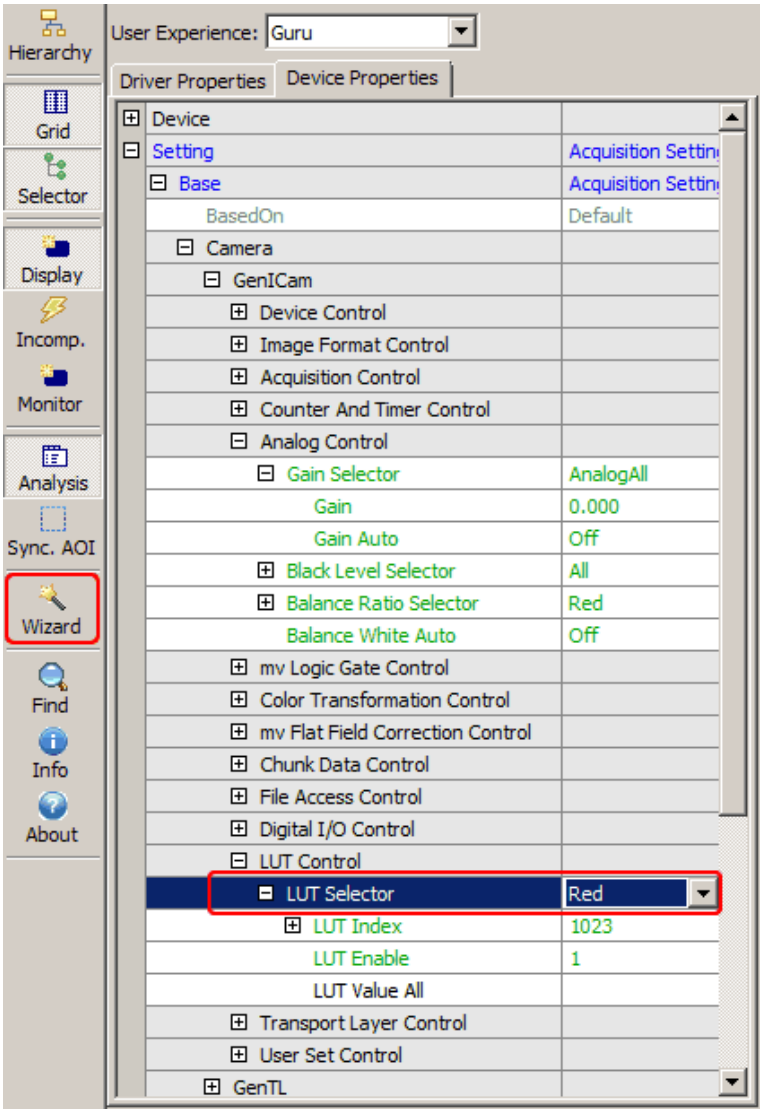


Figure 6: Selected LUT Selector and click on wizard will start wizard tool

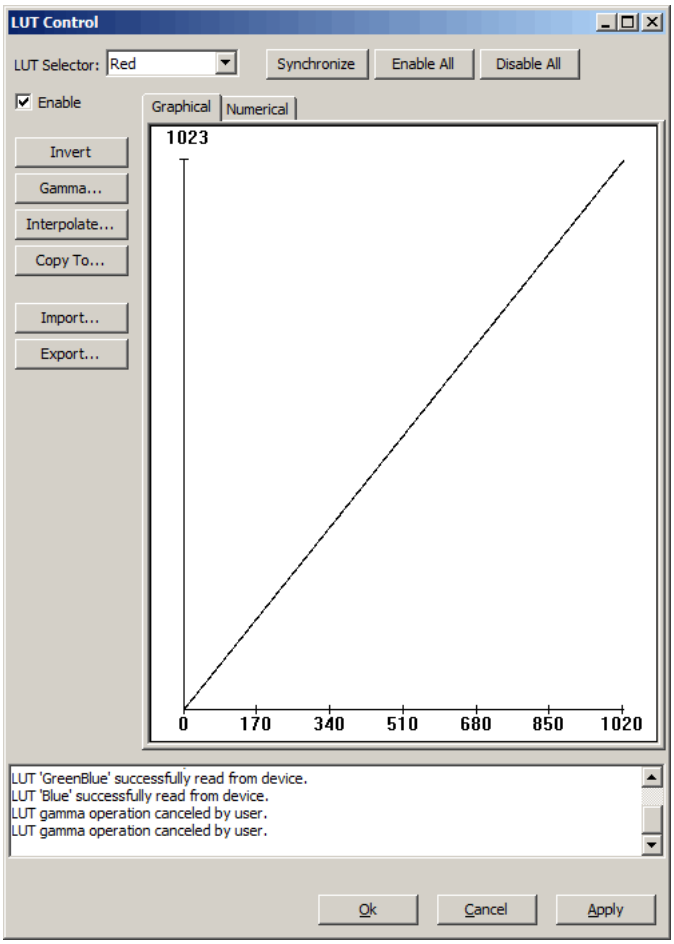


Figure 7: LUT Control

3. Now, click on the **"Gamma..."** button
4. and enter e.g. *"2.2"* as the Gamma value:

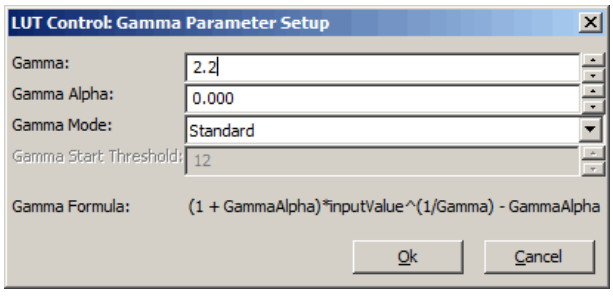


Figure 8: Gamma Parameter Setup

5. Then, click on **"Copy to..."** and select *"All"* and
6. and click on **"Enable All"**.
7. Finally, click on **Synchronize** and play the settings back to the device (via *"Cache -> Device"*).

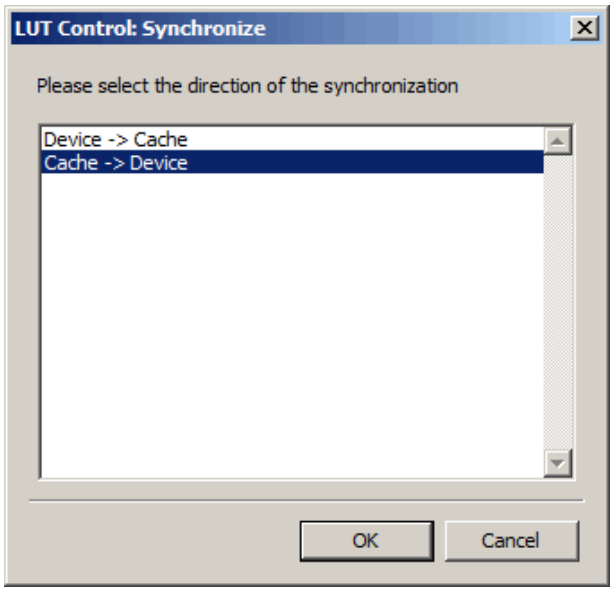


Figure 9: Synchronize

After gamma correction, the image will look like this:

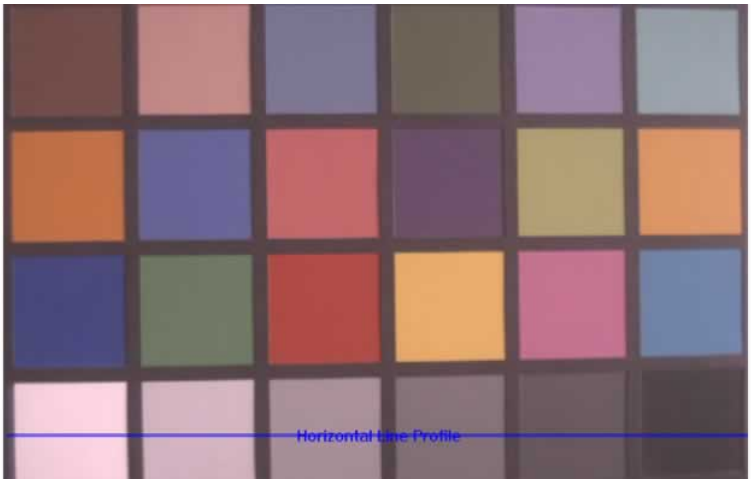


Figure 10: After gamma correction



Figure 11: Corresponding histogram after gamma correction

The third way to set Gamma, is gamma correction via (*"Setting -> Base -> ImageProcessing -> LUTControl"*). Here, the changes will affect the 8 bit image data and the processing is running on the CPU of the host system:

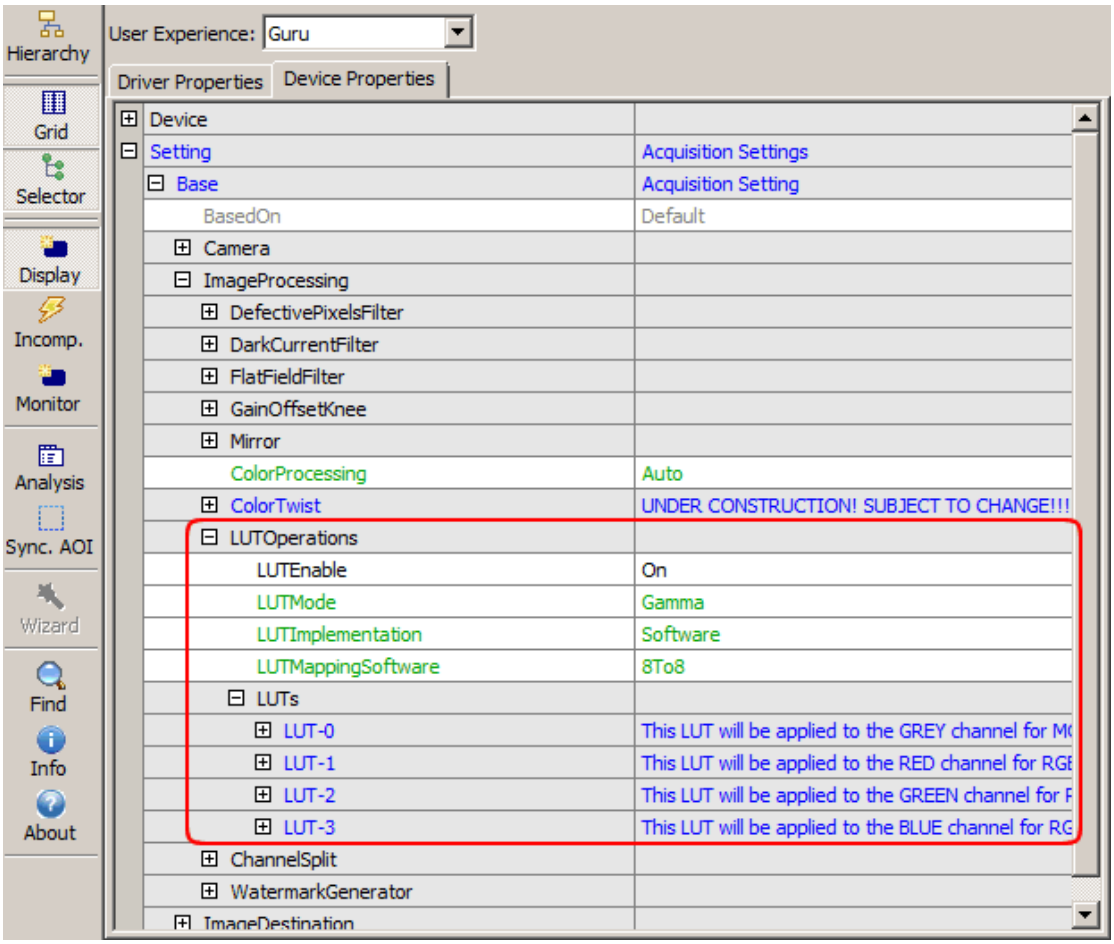


Figure 12: LUTControl dialog

Just set *"LUTEnable"* to **"On"** and adapt the single LUTs like (LUT-0, LUT-1, etc.).

1.18.2.2.2 Step 2: White Balance

As you can see in the histogram, the colors red and blue are above green. Using green as a reference, we can optimize the white balance via *"Setting -> Base -> Camera -> GenICam -> Analog Control -> Balance Ratio Selector"* (*"Balance White Auto"* has to be *"Off"*):

1. Just select *"Blue"* and
2. adjust the *"Balance Ratio"* value until the blue line reaches the green one.

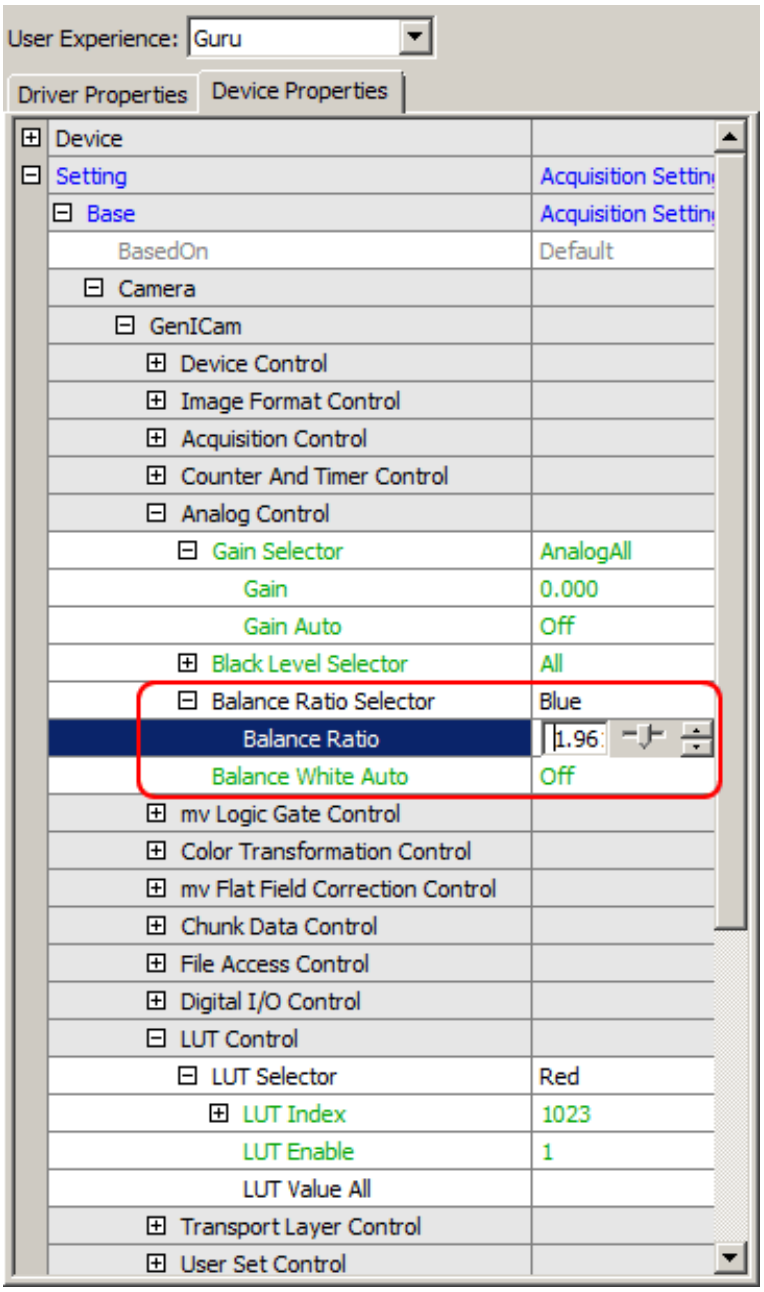


Figure 13: Optimizing white balance

3. Repeat this for *"Red"*.

After optimizing white balance, the image will look like this:

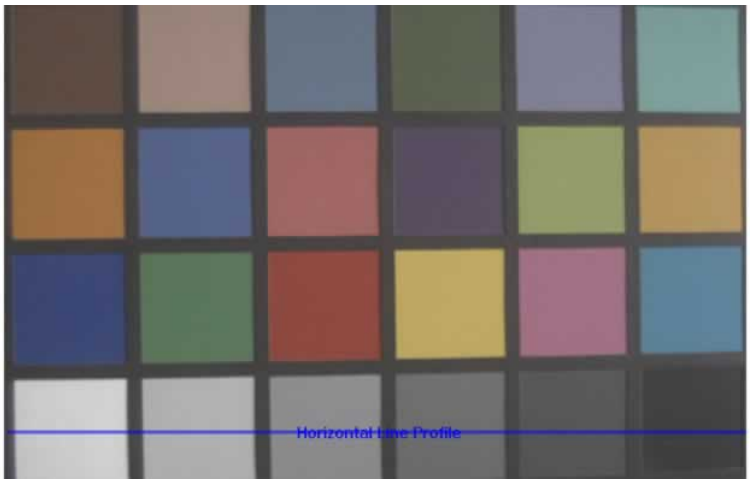


Figure 14: After white balance

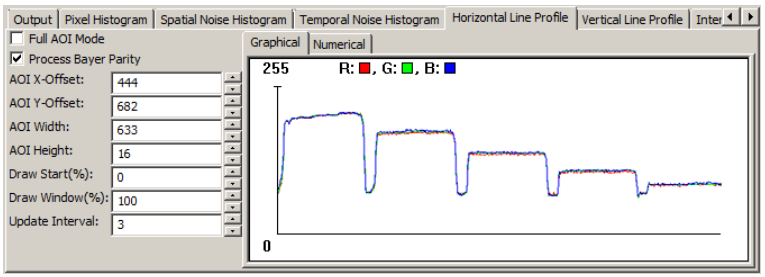


Figure 15: Corresponding histogram after white balance

1.18.2.2.3 Step 3: Contrast

Still, black is more a darker gray. To optimize the contrast you can use *"Setting -> Base -> Camera -> GenICam -> Analog Control -> Black Level Selector"*:

1. Select *"DigitalAll"* and
2. adjust the *"Black Level"* value until black seems to be black.

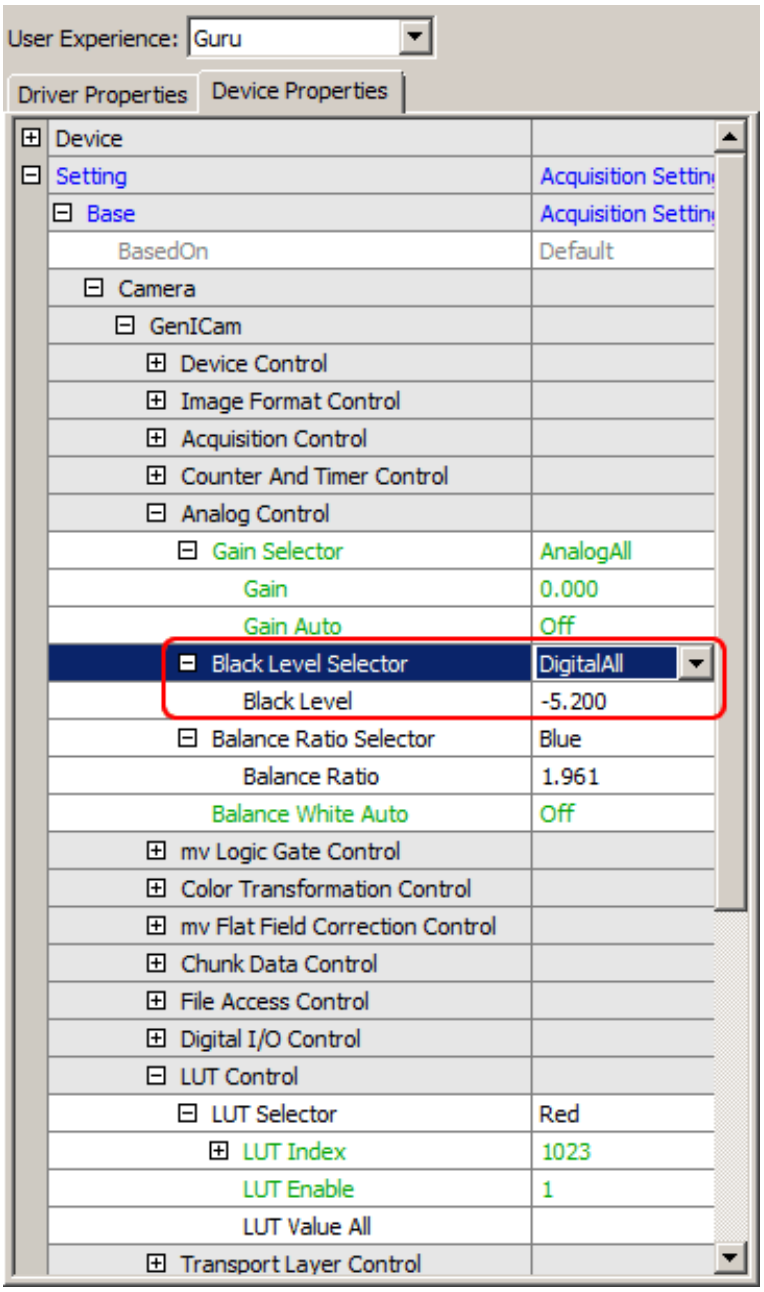


Figure 16: Back level adjustment

The image will look like this now:

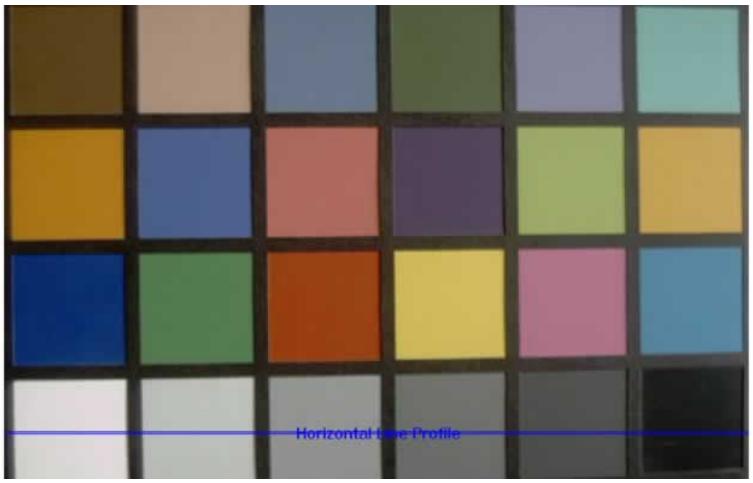


Figure 17: After adapting contrast



Figure 18: Corresponding histogram after adapting contrast

1.18.2.2.4 Step 4: Saturation and Color Correction Matrix (CCM)

Still saturation is missing. To change this, the **"Color Transformation Control"** can be used (*"Setting -> Base -> Camera -> GenICam -> Color Transformation Control"*).

Note

In our optical laboratory we determine the best correction data for the sRGB color space for each sensor type and these parameters are shipped with our cameras. If you want to use these parameters, you have to activate them using the wizard of the Color Transformation Control.

1. Click on **"Color Transformation Enable"** and
2. click on **"Wizard"** to start the saturation via *"Color Transformation Control"* wizard tool (since firmware version 1.4.57).

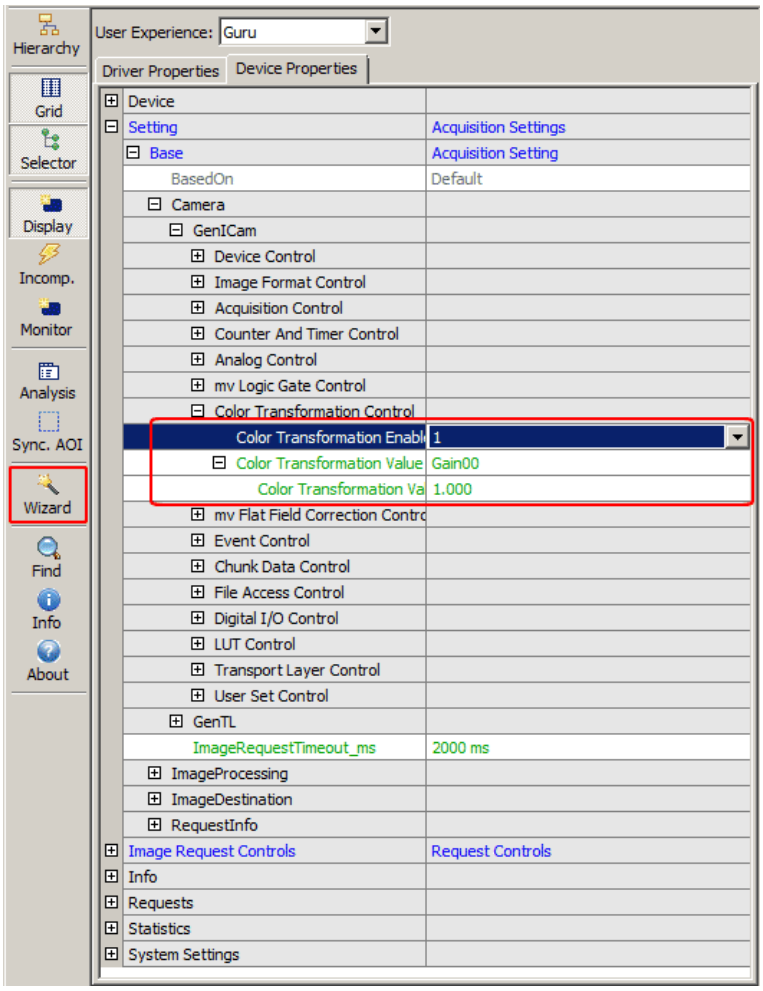


Figure 19: Selected Color Transformation Enable and click on wizard will start wizard tool

3. Now, you can adjust the saturation e.g. *"1.1"*.

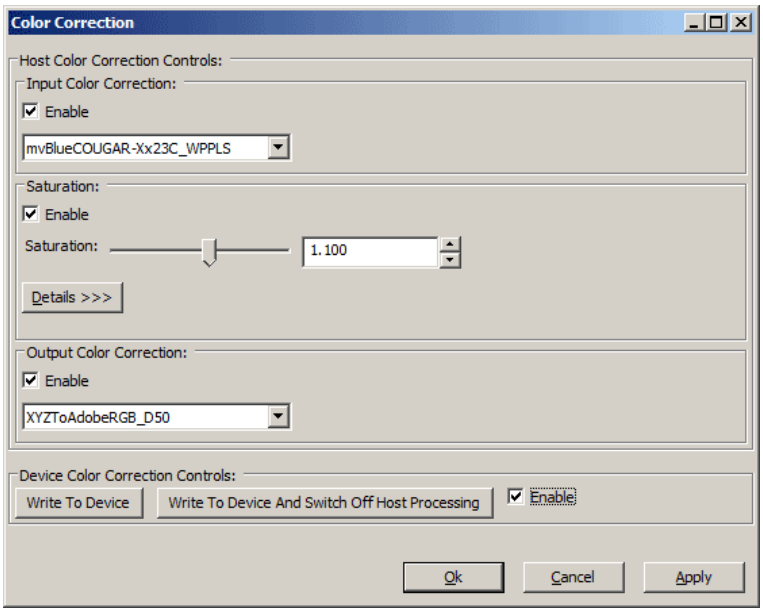


Figure 20: Saturation Via Color Transformation Control dialog

4. Afterwards, click on **"Enable"**.

5. Since driver version 2.2.2, it is possible to set the special color correction matrices at
   (a) the input (sensor),
   (b) the output side (monitor) and
   (c) the saturation itself using this wizard.
6. Select the specific input and output matrix and
7. click on **"Enable"**.
8. As you can see, the correction is done by the host by default (*"Host Color Correction Controls"*). However, you can decide, if the color correction is done by the device by clicking on **"Write To Device And Switch Off Host Processing"**. The wizard will take the settings of the *"Host Color Correction Controls"* and will save it in the device.
9. Finally, click on **"Apply"**.

After the saturation, the image will look like this:

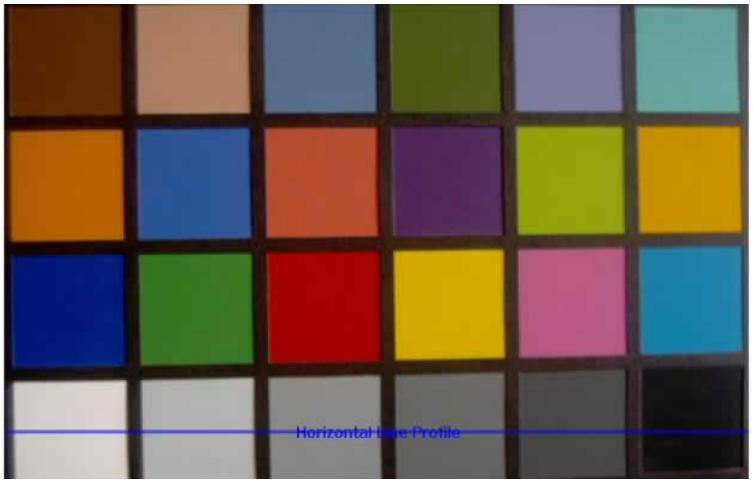


Figure 21: After adapting saturation

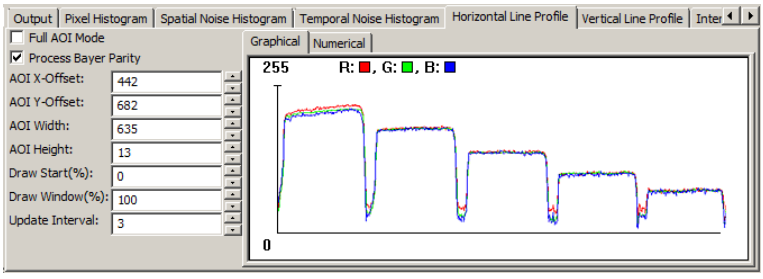


Figure 22: Corresponding histogram after adapting saturation

Note

As mentioned above, you can change the saturation and the color correction matrices via (*"Setting -> Base -> ImageProcessing -> ColorTwist"*). Here, the changes will affect the 8 bit image data and the processing needs the CPU of the host system:

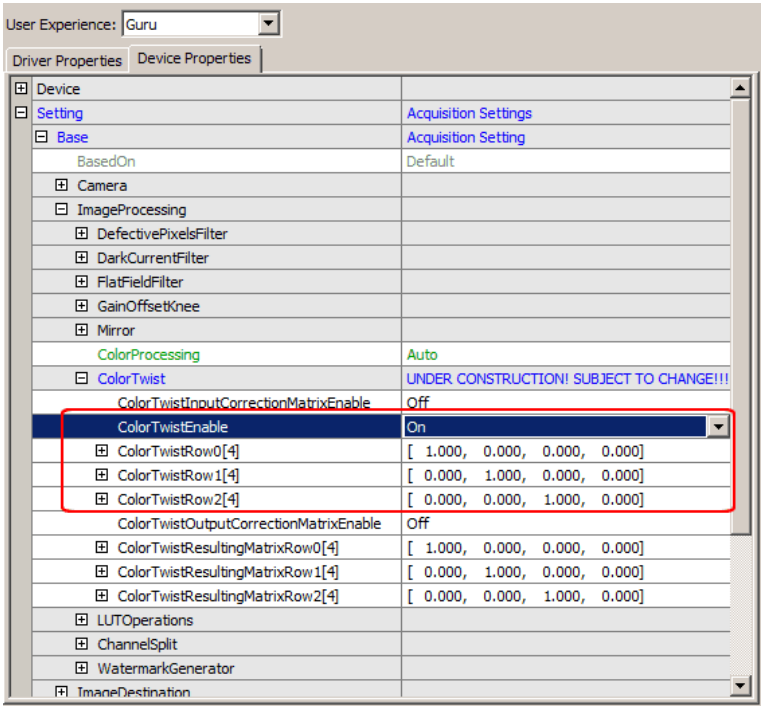


Figure 23: ColorTwist dialog

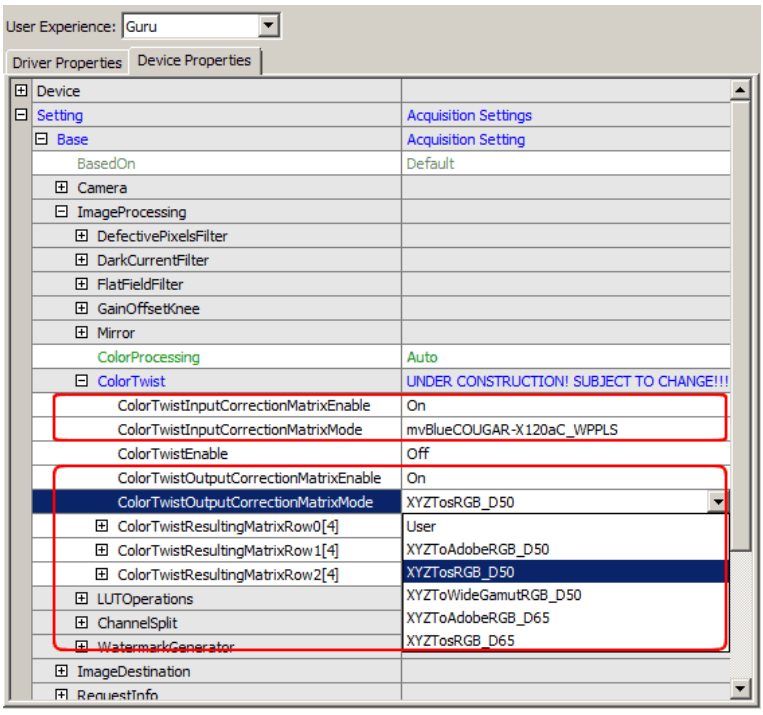


Figure 24: Input and output color correction matrix

1.18.2.3 Reducing noise by frame averaging

1.18.2.3.1 What is frame average?

As the name suggests, the functionality averages the gray values of each pixel using the information of subsequent frames. This can be used to

- reduce the noise in an image and
- compensate motion in an image.

MATRIX VISION implemented two modes of the frame averaging:

- **"mvNTo1"** and
- **"mvNTo1Sum"**.

These modes are a FPGA function which will not need any CPU of the host system. However, these modes are only available for the following cameras:

- mvBlueFOX3-2xxx family.

1.18.2.3.2 mvNTo1

mvNTo1 is the mode, when images have to be averaged. To get an averaged pixel, this mode takes the gray value of each pixel of the specified number of subsequent frames (**"mv Frame Average Frame Count"**) and calculates the average. E.g. Averaging of pixel [0,0] using 8 frames (`"mv Frame Average Frame Count = 8"`) will be as follows:

$$Pixel_avg[0,0] = \frac{Gray_value[0,0,image1] + Gray_value[0,0,image2] + ... + Gray_value[0,0,image8]}{8}$$

Remarks:

- When Pixelformat `"Mono8/BayerRG8"` or `"Mono10/BayerRG10"` is selected input to the averaging process are images with 8 bit pixel values.
  To avoid missing codes when using Mono10/BayerRG10 at least 4 images must be added.(mvFrame↩ AverageFrameCount = 4).
- For other pixel formats input to the averaging process are images with 12 bit pixel values.

1.18.2.3.2.1 Using wxPropView

Using the frame average mode **"mvNTo1"**, you have to do the following step:

1. Start **wxPropView** (p. 99) and
2. connect to the camera.
3. Then specify in *"Setting -> Base -> Camera -> GenICam -> Device Control"* which processing unit of the camera should do the frame averaging, e.g. unit 0 should do the frame averaging
   "mv Device Processing Unit Selector = 0"
   "mv Device Processing Unit = mvFrameAverage".
   Afterwards, **"mv Frame Average Control"** is available.
4. Now open *"Setting -> Base -> Camera -> GenICam -> mv Frame Average Control"* and
5. select **"mv Frame Average Mode = mvNto1"**.

6. Select the number of frames you want to use for averaging (**"mv Frame Average Frame Count"**), e.g. 8. This, of course, reduces the frame rate. If you have a frame rate of 8 frames per second and a "`mv Frame Average Frame Count`" of 8 frames, this will result to a frame rate of 1 Hz.

7. Activate frame averaging by setting **"mv Frame Average Frame Enable = 1"**.

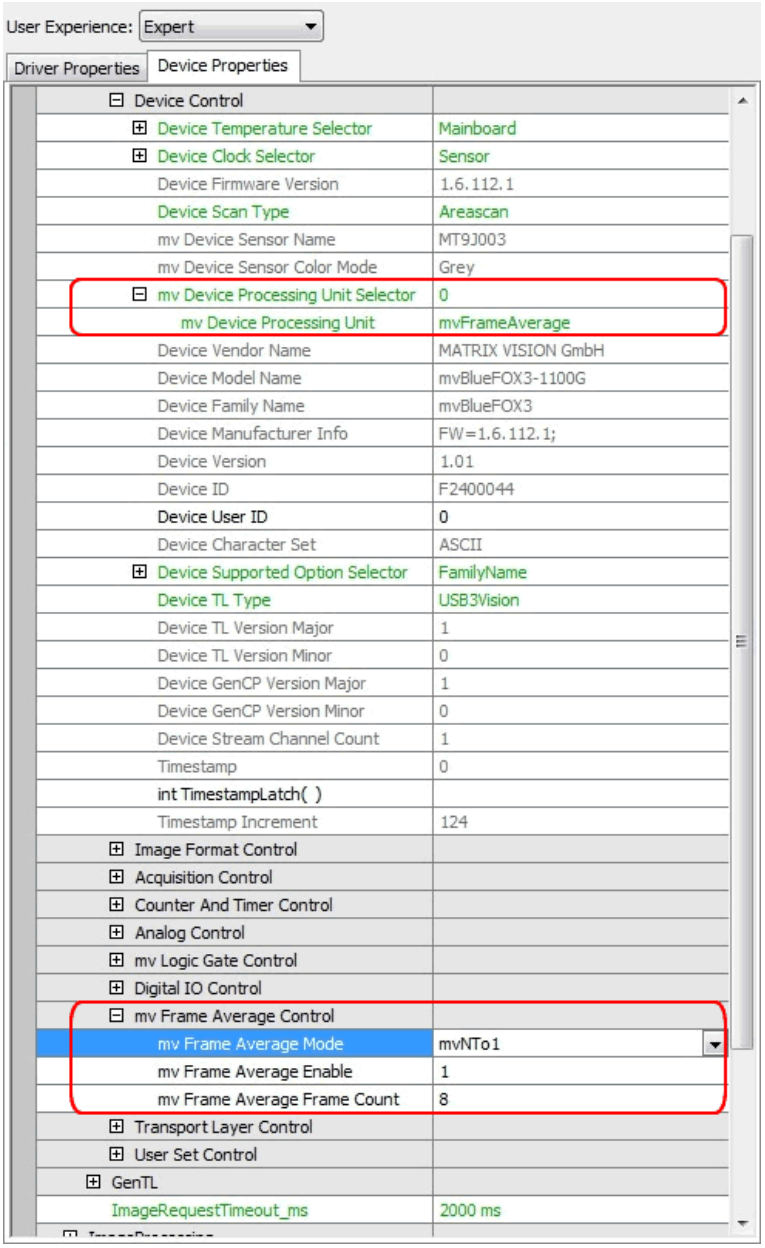


Figure 1: wxPropView: Setting frame average mode "mvNTo1"

1.18.2.3.3 mvNTo1Sum

mvNTo1Sum is a special mode for a defined application. In this mode, the values of the pixel are summed up with the specified number of subsequent frames (**"mv Frame Average Frame Count"**). E.g. Summing up of pixel [0,0] using 8 frames ("`mv Frame Average Frame Count = 8`") will be as follows:

```
Pixel_avg[0,0] = Gray_value[0,0,image1] + Gray_value[0,0,image2] + ... + Gray_value[0,0,image8]
```

Remarks:

- In any case **"PixelFormat"** has to be set to 16 bit
- Captured images are automatically set to 8 bit
- These images are added up and not divided
  - 2 images go to 512 (9 bit)
  - 4 images to 1024 (10 bit)
  - 16 images to 4096 (12 bit)
- Images are always sent with 16 bit, even if the intensity range is smaller.
- This might be interpreted as dark images, but the greyvalues are right within these 16 bit images.
- Standard images viewers can show 8 bit only. So in case of 12 bit result, Bit11 .. Bit4 have to be selected for viewing.

1.18.2.3.3.1 Using wxPropView

Using the frame average mode **"mvNTo1Sum"**, you have to do the following step:

1. Start **wxPropView** (p. 99) and
2. connect to the camera.
3. Then specify in *"Setting -> Base -> Camera -> GenICam -> Device Control"* which processing unit of the camera should do the frame averaging, e.g. unit 0 should do the frame averaging
   "mv Device Processing Unit Selector = 0"
   "mv Device Processing Unit = mvFrameAverage".
   Afterwards, **"mv Frame Average Control"** is available.
4. Now open *"Setting -> Base -> Camera -> GenICam -> mv Frame Average Control"* and
5. select **"mv Frame Average Mode = mvNTo1Sum"**.
6. Select the number of frames you want to use for summing up (**"mv Frame Average Frame Count"**), e.g. 8. This, of course, reduces the frame rate. If you have a frame rate of 8 frames per second and a "`mv Frame Average Frame Count`" of 8 frames, this will result to a frame rate of 1 Hz.
7. Activate the mode by setting **"mv Frame Average Frame Enable = 1"**.

1.18.2.4 Optimizing the bandwidth

1.18.2.4.1 Limiting the bandwidth of the imaging device

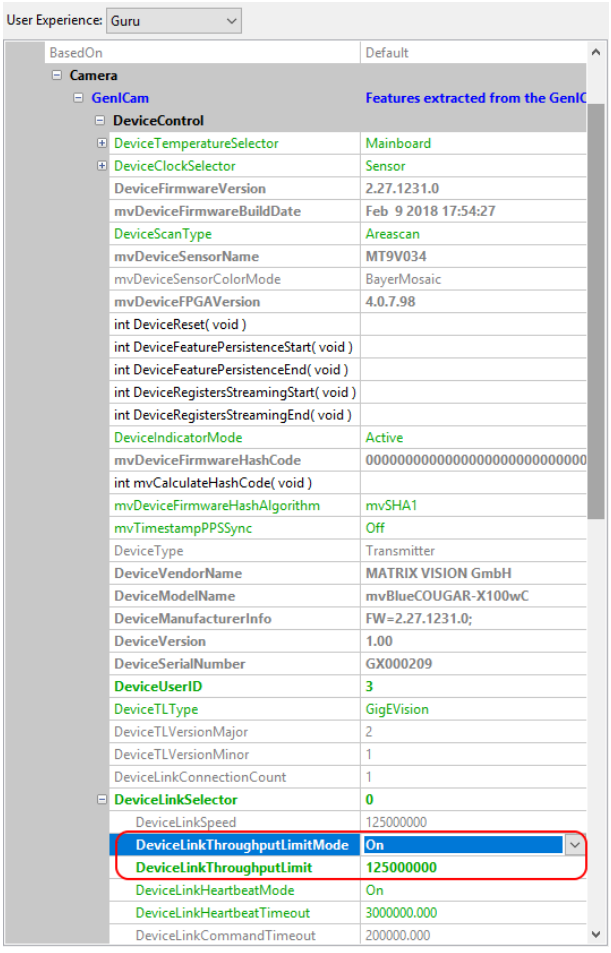
For a setup of multiple streaming devices connected to one host, it is highly recommended to consider the information of this chapter.
Even if the connected links and devices are able to handle the average throughput of a streaming setup situations might be encountered where the data throughput temporarily exceeds the capabilities of involved network components for a very short period of time. This might result in packet loss with GigE Vision™ devices or overflowing buffers with USB3 Vision™ devices.

1.18.2.4.1.1 How the Device Link Throughput Limit works From version 1.5.2 and above, the GenICam™ SFNC defines the features **DeviceLinkThroughputLimit** and **DeviceLinkThroughputLimitMode**, which are meant to provide a standardized way to control the throughput limit for every GenICam™ device.
All MATRIX VISION devices with firmware version 2.25.0 or above support the GenICam™ SFNC features DeviceLinkThroughputLimitMode and DeviceLinkThroughputLimit to limit the bandwidth used by a device in a convenient way. On USB3 Vision™ devices, the effective bandwidth is a combination of image size and acquisition frame rate. If a *DeviceLinkThroughputLimit* is set and the unlimited link speed exceeds the set limit, the acquisition frame rate would be adjusted to fit the DeviceLinkThroughputLimit.

If it is necessary to limit the outgoing link throughput of a device, this can be accomplished the following way:

Available since firmware version 2.25.0

1. In `"Setting -> Base -> Camera -> GenICam -> Device Control -> Device Link Selector"` set property **"Device Link Throughput Limit Mode"** to **"On"**.
2. Now, you can set the bandwidth with **"Device Link Throughput Limit"** to your desired bandwidth in bits per second



wxPropView - Setting Device Link Throughput Limit

1.18.2.5 Setting a flicker-free auto expose and auto gain

1.18.2.5.1 Introduction

In order to prevent oscillations it is important to adapt the camera frequency to the frequency of AC light.

This is, for example, in

- Europe 50 cycles (100 fluctuations/s) whereas in
- USA, Japan and other countries it is 60 Hz.

This means the camera must strictly be coupled to this frequency. In conjunction with auto exposure this can only be maintained by using a timer based generation of external trigger pulses. This is a behavior of both sensor types: CCD and CMOS.

Note

It is not enough to use *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Acquisition Frame Rate"* for this, as there are small fluctuations in this frame rate if the exposure time changes. These fluctuations lead to oscillations (see settings marked with red boxes in Figure 1). The *"Acquisition Frame Rate"* will only provide the exact frame rate if auto exposure is turned off.

As shown in Figure 1, it is possible to set (*"mv Exposure Auto Mode"*) which part of the camera handles the auto expose (device or the sensor itself; pink boxes). Using *"mvSensor"* as *"mv Exposure Auto Mode"*, it is possible to avoid oscillations in some cases. The reason for this behavior is that you can set more parameters like *"mv Exposure Auto Delay Images"* in contrast to *"mvDevice"*. However, as mentioned above it is recommended to use a timer based trigger when using auto expose together with continuous acquisition.

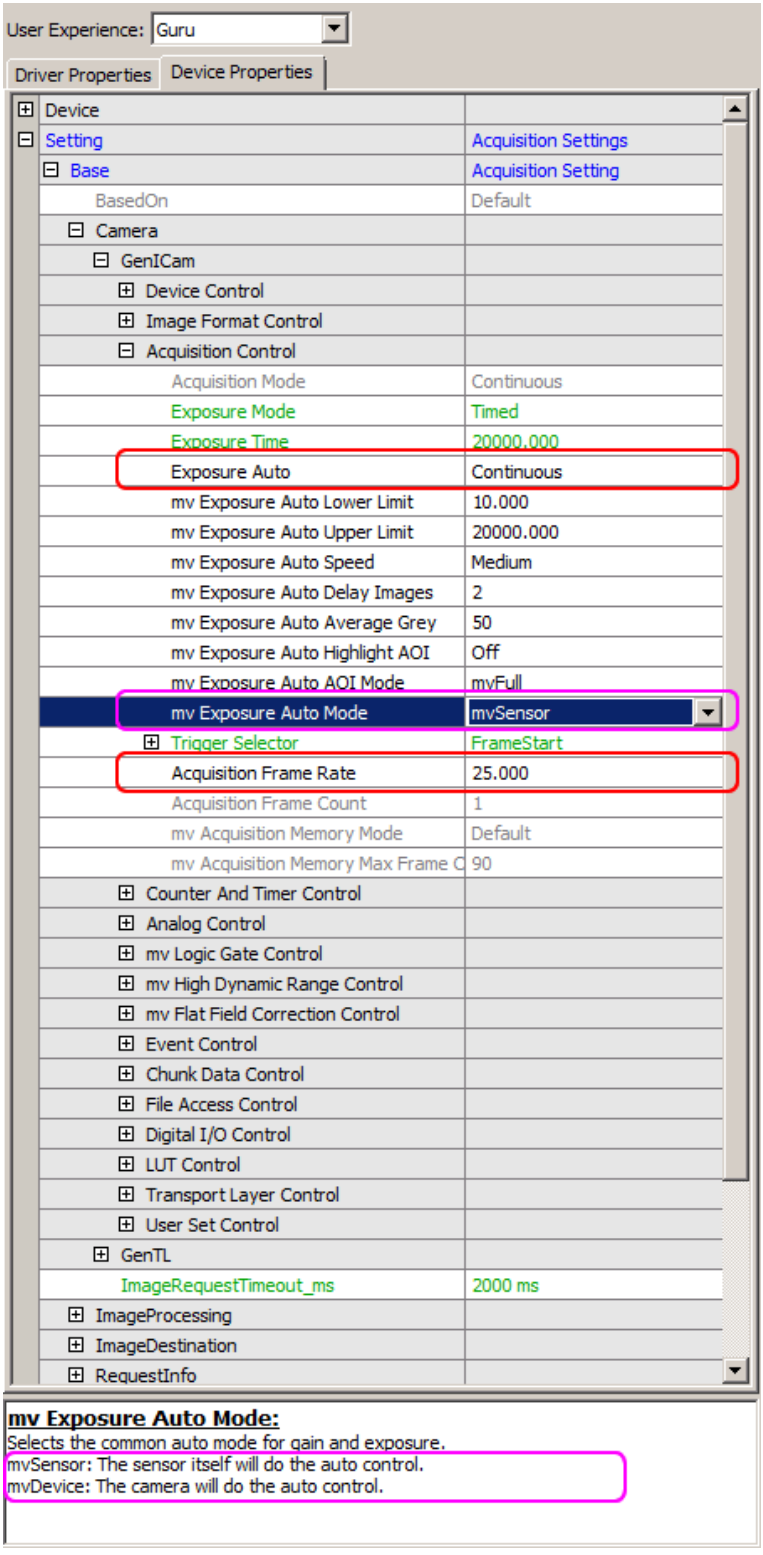


Figure 1: wxPropView - Auto expose is turned on and the frame rate is set to 25 fps

1.18.2.5.2 Example of using a timer for external trigger

Figure 2 shows how to generate a 25 Hz signal, which triggers the camera:

- *"Setting -> Base -> Camera -> GenICam -> Counter & Timer Control -> Timer Selector -> Timer 1"*:

  - *"Timer Trigger Source" = "Timer1End"*
  - *"Timer Duration" = "40000"*

$$FPS_{max} = \frac{1}{40000\ us} = 25$$

- *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector -> FrameStart"*:
  - *"Trigger Mode" = "On"*
  - *"Trigger Source" = "Timer1End"*

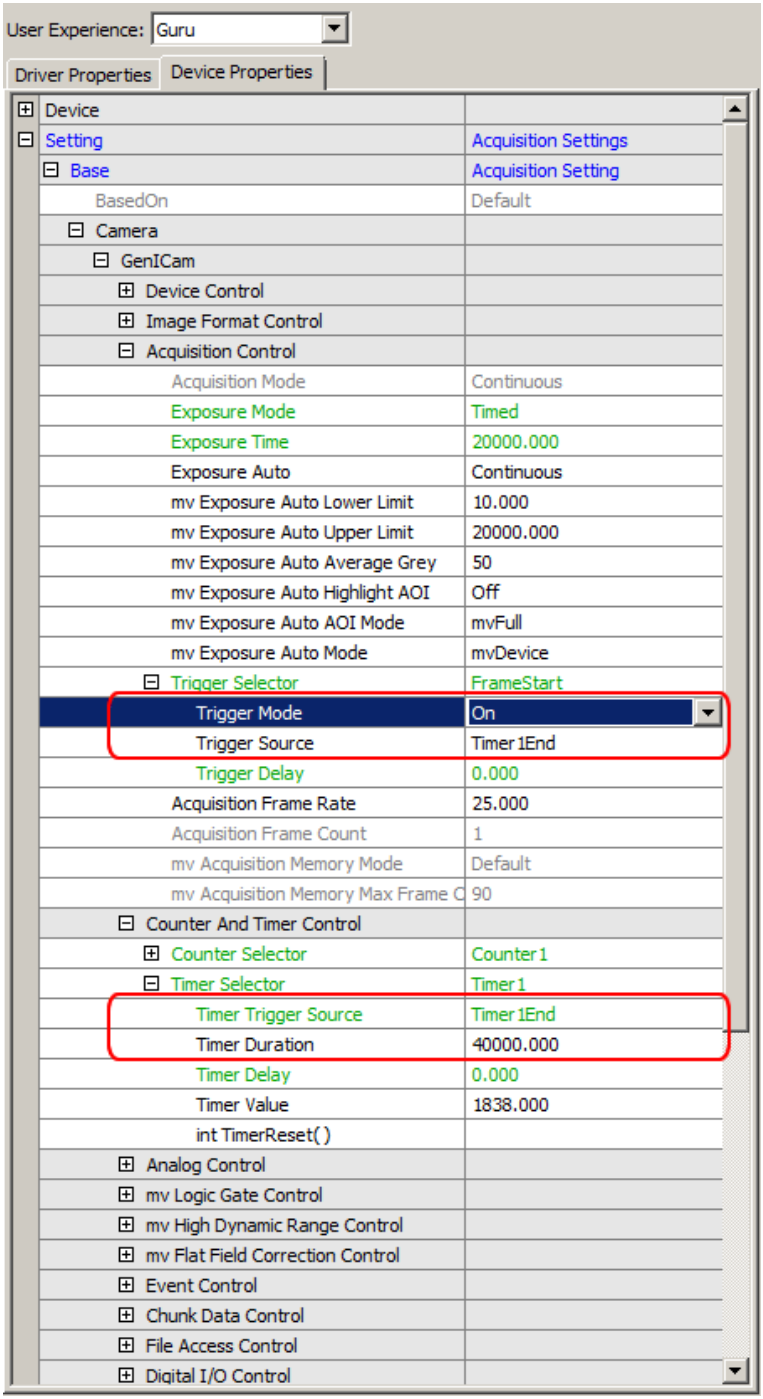


Figure 2: wxPropView - 25 Hz timer for external trigger

No oscillation occurs, regardless of DC ambient vs. AC indoor light.

This operation mode is known as flicker-free or flicker-less operation.

What it mainly does is to adjust the frame frequency to precisely the frequency of the power line. Usually the line frequency is very stable and therefore is the harmonic frequency difference of the two signals are very slow; probably in the range of < 0.1 Hz.

The fact that we do not know the phase relation between the two frequencies means that we scan the alternating ambient light source with our camera. The shorter the exposure time, the more we see a slow change in brightness.

Using AutoExposure/AutoGain can completely eliminate this change because the frequency of change is very low. That means it will be legal if we calculate a brightness difference in one picture and apply it for the next one, because the change is also valid in the next one; as we fulfill the Nyquist theorem.

If we use an arbitrary scanning frequency like 20 fps or whatever your algorithm and data flow is accepting, is wrong in this aspect and leads to oscillations and undesired flicker.

Pointing to a 60 Hz display with flashing backlight an oscillation of 10 Hz can be seen of course.

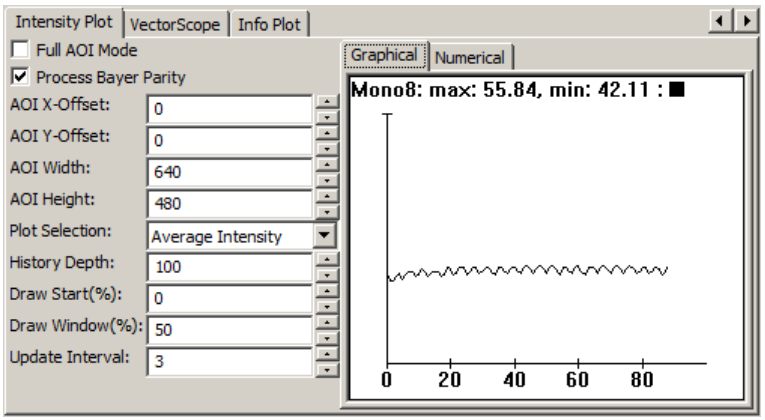


Figure 3: wxPropView - Intensity plot while pointing the camera to a 60 Hz display

1.18.2.5.3 Conclusion

To avoid oscillations, it is necessary to adapt the camera frequency to the frequency of AC light. When using auto expose a flicker-free mode (timer based external trigger) is needed. If the camera is used throughout the world it is necessary that the frequency of AC light can be set in the software and the software adapts the camera to this specific environment.

1.18.2.6 Working with binning / decimation

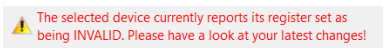
With binning / decimation it is possible to combine / reduce adjacent pixels vertically and/or horizontally. Depending on the sensor, up to 16 adjacent pixels can be combined / reduced.

See also

`https://www.matrix-vision.com/manuals/SDK_CPP/Binning_modes.png`

To optimize the sensor speed, the firmware will check if the binning / decimation functionality is provided by the sensor. If it is available it will use the sensor for binning / decimation, if not, binning / decimation will be processed in the camera's FPGA.

If a binning / decimation combination is not possible, the following note will be displayed:



In this case no acquired images will be displayed until you have changed the settings.

1.18.2.6.1 Binning

Possible binning modes are:

- **Sum:** The response from the combined pixels will be added, resulting in increased sensitivity.
- **Average:** The response from the combined pixels will be averaged, resulting in increased signal/noise ratio.

Binning can be used to lighten the image at the expense of the resolution. This is a neat solution for applications with low light and low noise.

The following results were achieved with the mvBlueFOX3-2124G:

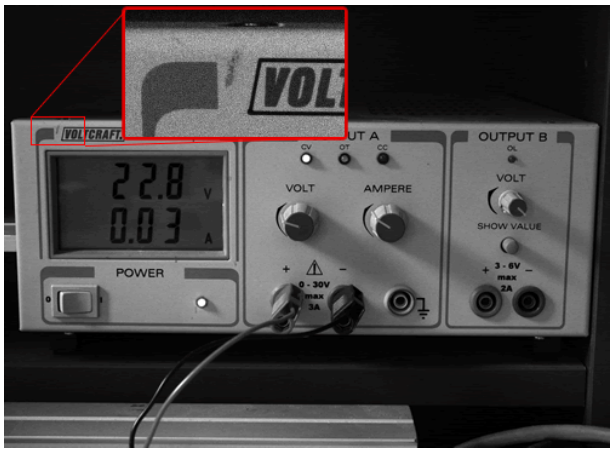
Exposure [in us]	Binning	Gain [in dB]	Averager
2500	-	0	-



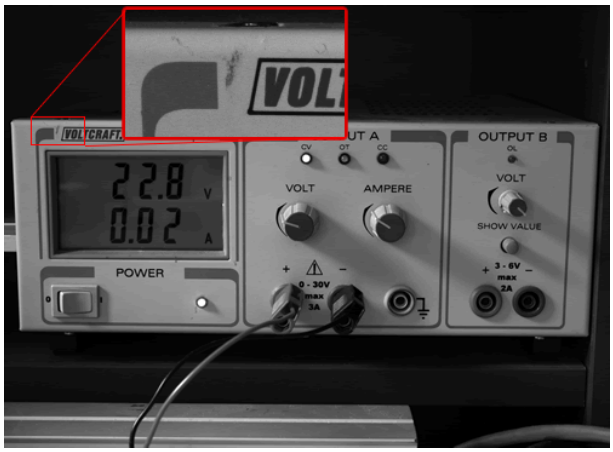
Exposure [in us]	Binning	Gain [in dB]	Averager
2500	-	30	-



Exposure [in us]	Binning	Gain [in dB]	Averager
2500	2H 2V	30	-



Exposure [in us]	Binning	Gain [in dB]	Averager
2500	2H 2V	30	Averaging using 24 frames



The last image shows, that you can reduce the noise, caused by the increased gain, using **frame averaging** (p. 250).

1.18.2.6.2 Decimation

Possible decimation modes are:

- **Discard:** The value of every Nth pixel is kept, others are discarded.
- **Average:** The value of a group of N adjacents pixels are averaged.

1.18.2.7 Minimizing sensor pattern of mvBlueFOX3-1100G

Sometimes the gray scale version of Aptinas sensor MT9J003 shows structures comparable with Bayer patterns of color sensors. This pattern is particularly apparent in scaled images:



Figure 1: Bayer pattern like structures of the MT9J003(gray scale version)

To minimized this pattern, you can balance the sensor patterns since Firmware version 2.3.70.0. This procedure works like the white balancing used with color sensors. For this reason the same terms are used (red, blue, green).

The balance reference is the "green" pixel value from the "blue-green" line of the sensor.

See: **Output sequence of color sensors (RGB Bayer)** (p. 83)

I.e. all gray scale values of the these "green" pixels are averaged.

With *"Setting -> Base -> Camera -> GenICam -> Analog Control -> Balance Ration Selector"* you can select each color to set the *"Balance Ratio"*:

- **"Red"** averages the "red" pixel values from the "red-green" line of the sensor.
- **"Green"** averages the "green" pixel values from the "red-green" line of the sensor, too.
- **"Blue"** averages the "blue" pixel values from the "blue-green" line of the sensor.

I.e. there are 4 average values (reference value, red value, green value, blue value). The lowest value will be unchanged, the other values are increased using each *"Balance Ratio"*.

However by using the property *"Balance White Auto"* you can balance the sensor automatically:

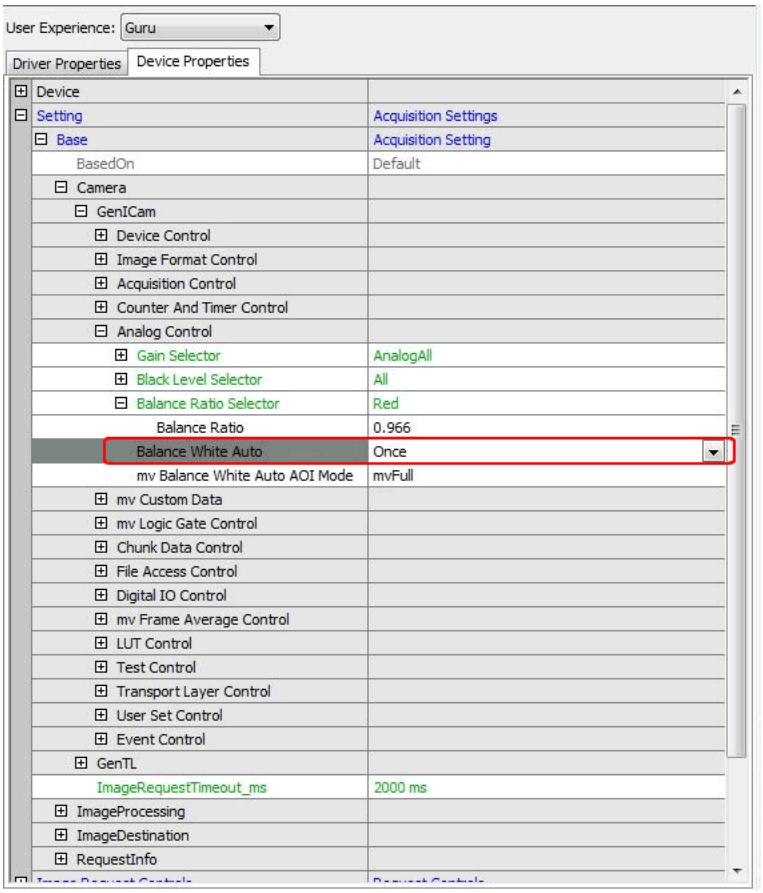


Figure 2: Balance White Auto

After balancing, we recommend to save these settings to a UserSet with **wxPropView** (p. 99) which is described in **"Saving User Settings In The Non-volatile Memory"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual.



Figure 3: Calibrated sensor

1.18.2.8 Working with the dual gain feature of mvBlueFOX3-2071/2071a

1.18.2.8.1 Introduction

The IMX425/428 used in the mvBlueFOX3-2071/2071a are Pregius sensors of the third generation.

Those sensors feature a dual gain mode, i.e. after a trigger event, for example, different image areas can be amplified at the same time differently.

To activate the dual gain mode it is necessary to enable the multi area mode by using the **mvMultiAreaMode** property. At least two different AOIs have to be defined.

Note

Both AOIs must not overlap or touch each other!

Once the AOIs are configured the **GainSelector** has to be set to one of the new implemented options called **mv↩HorizontalZone0** and **mvHorizontalZone1**.

The gain value of the different zones can be specified using the **"Gain Selector"**. And the corresponding gain property. Possible zones are

- **mvHorizontalZone0**
- **mvHorizontalZone1**

If this mode is selected you can set the **"mv Gain Horizontal Zone Divider"**. This property indicates where the zones are divided horizontally once more than two AOIs are configured. In this case the e.g. 25% means that the upper 25% of the image are defined by the gain value of **mvHorizontalZone0** and the lower 75% are defined by the gain value of **mvHorizontalZone1**.

Note

Some sensors may only allow to change the gain at certain positions e.g. the last line of a defined ROI. In this case the first possible switching point above the actual line will be used.

To activate the dual gain, just

1. Use the Multi AOI Wizard to adjust the different AOIs. (They must not overlap or touch each other!)
2. Select **mvMultiZone** in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> mvGainMode"*.
3. Select **mvHorizontalZone0** in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector"*.
4. Adjust the gain value for the first AOI in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector -> Gain"*.
5. Adjust the gain divider position *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector -> mvGainHorizontalZoneDivider"*.
6. Select **mvHorizontalZone1** in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector"*.
7. Adjust the gain value for the second AOI in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector -> Gain"*.

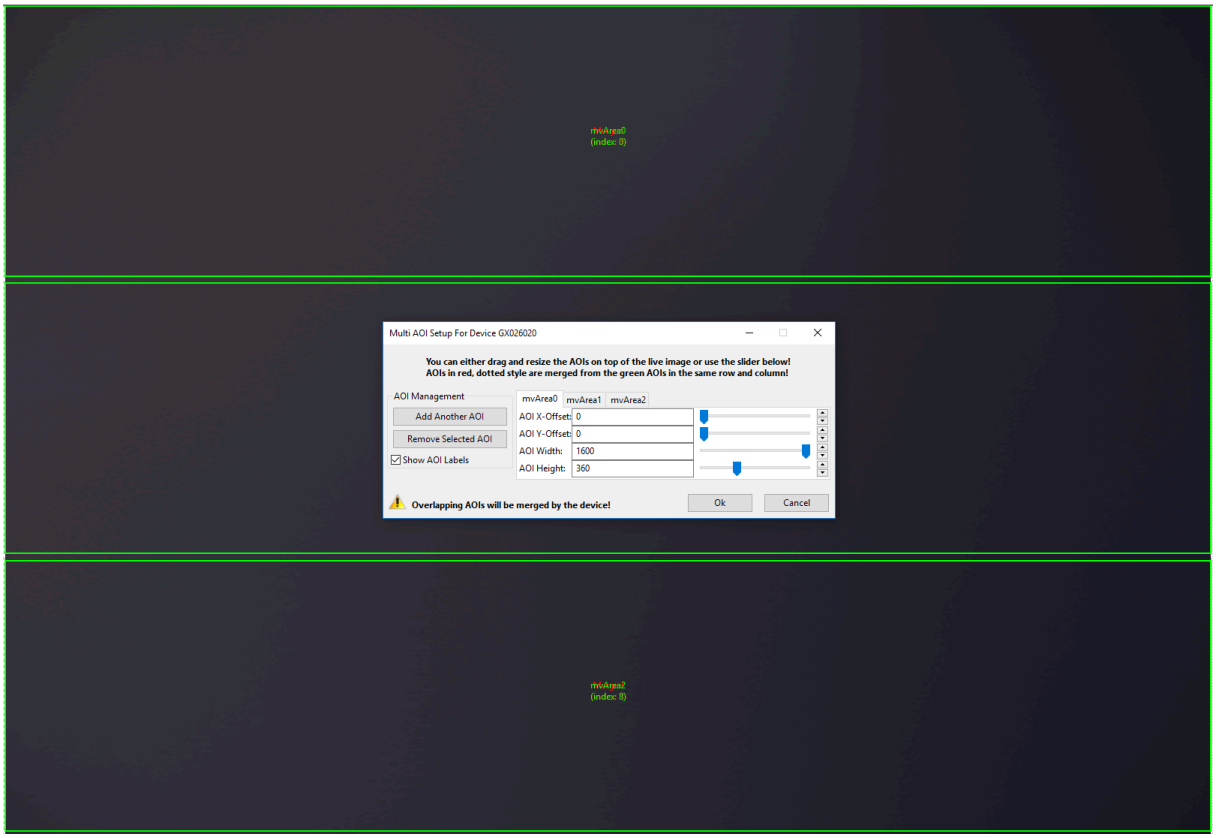


Figure 1: wxPropView - configuring multiple AOIs

AnalogControl	
mvGainMode	mvMultiZone
GainSelector	mvHorizontalZone0
Gain	12.000
mvGainHorizontalZ	50.000
BlackLevelSelector	All
BalanceRatioSelector	Red
BalanceWhiteAuto	Off

Figure 2: wxPropView - configuring the gain of the first zone



Figure 3: Example image with two different amplified zones

1.18.2.8.2 Programming the dual gain mode

As an example the IMX425 sensor is used for the sample. The goal is to configure three AOIs which have a similar height. Since the AOIs must not overlap or touch each other, it is important to increase the offset of the next AOI by the smallest increment size. Which is 8 in this case.

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
    GenICam::ImageFormatControl ifc( pDev );

    ifc.mvMultiAreaMode.writeS( "mvMultiAreasCombined" );

    ifc.mvAreaSelector.writeS( "mvArea0" );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 0 );
    ifc.mvAreaWidth.write( ifc.mvAreaWidth.getMaxValue( ) );
```

```
    ifc.mvAreaHeight.write( 360 );
    ifc.mvAreaEnable.writeS( "1" );

    ifc.mvAreaSelector.writeS( "mvArea1" );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 368 );
    ifc.mvAreaWidth.write( ifc.mvAreaWidth.getMaxValue( ) );
    ifc.mvAreaHeight.write( 360 );
    ifc.mvAreaEnable.writeS( "1" );

    ifc.mvAreaSelector.writeS( "mvArea2" );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 736 );
    ifc.mvAreaWidth.write( ifc.mvAreaWidth.getMaxValue( ) );
    ifc.mvAreaHeight.write( 360 );
    ifc.mvAreaEnable.writeS( "1" );

    GenICam::AnalogControl anc( pDev );
    anc.mvGainMode.writeS( "mvMultiZone" );

    anc.gainSelector.writeS( "mvHorizontalZone0" );
    anc.gain.write( 12 );
    anc.mvGainHorizontalZoneDivider.write( 80 );

    anc.gainSelector.writeS( "mvHorizontalZone1" );
    anc.gain.write( 0 );
  // more code
```

1.18.2.9 Working With Gain And Black-Level Values Per Color Channel

In many low-lighting applications the gain needs to be increased to enhance the brighness of the images. However, while the image brightness is increased, the black-level of the image is also increased, which in many cases should be avoided. With the help of the GainOffsetKnee filter it is possible to correct/adjust the overall black-level as well as the black-level per color channel, even when the gain is applied. *Figure 1* shows the working principle of the GainOffsetKnee filter.

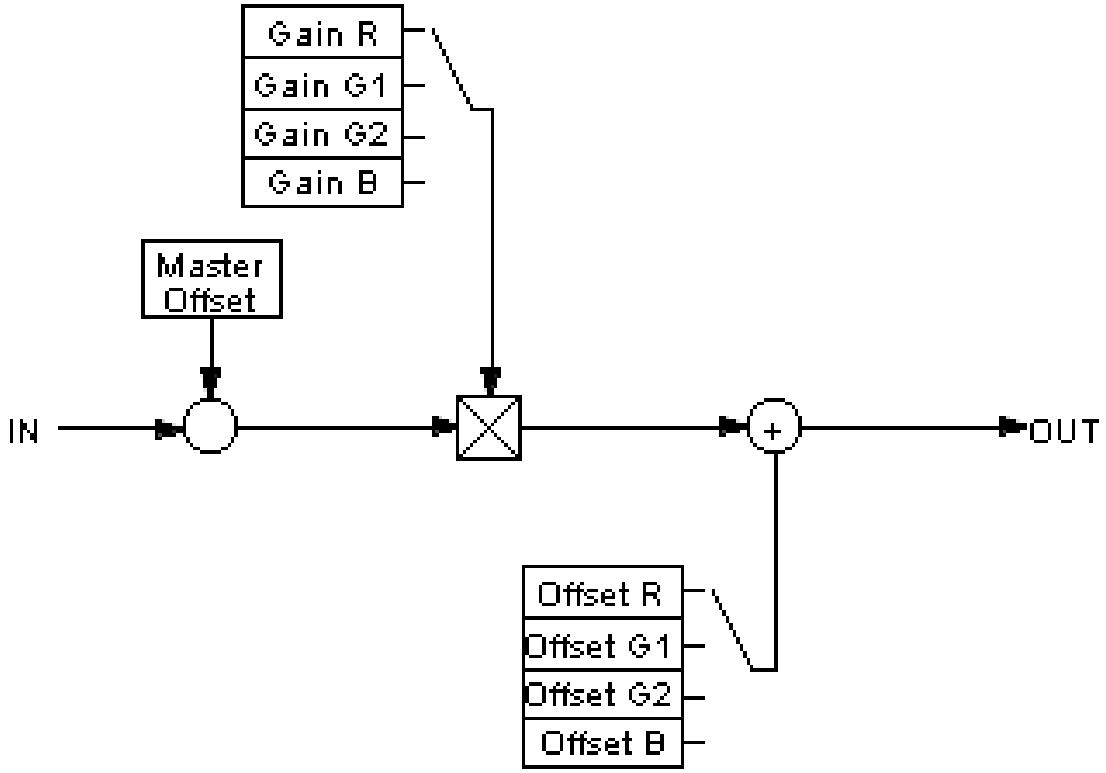


Figure 1: The GainOffsetKnee filter working principle

The GainOffsetKnee filter is one of the image processing methods performed on the host. It allows you to adjust:

- The overall offset (*i.e.* overall black-level) of an image.
- The individual gain per color channel.
- The individual offset (*i.e.* individual black-level) per color channel.

1.18.2.9.1 Configuration in wxPropView Here is how to configure the GainOffsetKnee filter in wxPropView and the impact the filter has on an image:

1. The GainOffsetKnee filter is located under *"Setting -> Base -> ImageProcessing"*.

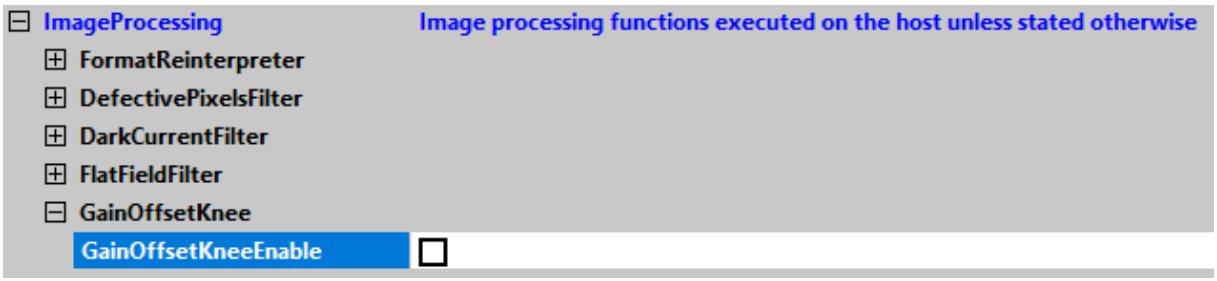


Figure 2: The GainOffsetKnee filter option in wxPropView

2. Once the GainOffsetKnee filter is activated, the configuration field will be displayed (see *Figure 3*). As an example, the current RGB image is shown in *Figure 4* and its histogram in *Figure 5*.



Figure 3: The configuration field for the GainOffsetKnee filter



Figure 4: An image without the GainOffsetKnee filter

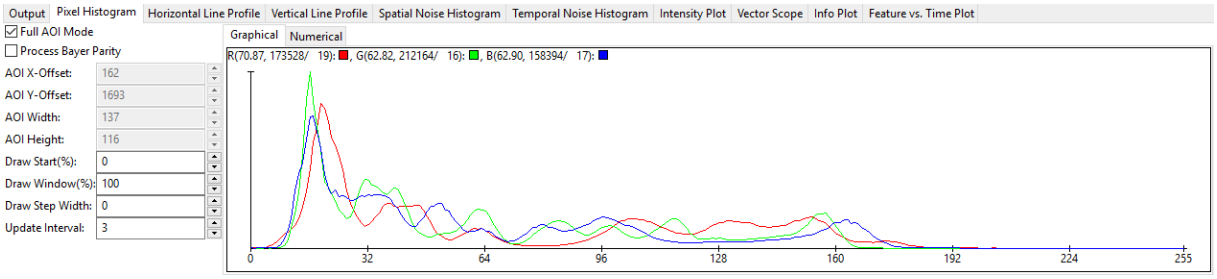


Figure 5: The histogram of *Figure 3*

3. The overall offset can be assigned using the *'GainOffsetKneeMasterOffset_pc'*. A positive offset increases the black-level of the image, whereas a negative offset reduces it. To visualize the effect, an offset of 5% is given as an example, which means that the overall black-level of the image will be increased by 5% of the max. pixel value (*i.e.* 255 in this example). As a result, the overall black-level in the current histogram (see *Figure 8*) has been increase by 12.75 (which is 5% x 255) comparing to the original histogram (see *Figure 5*).

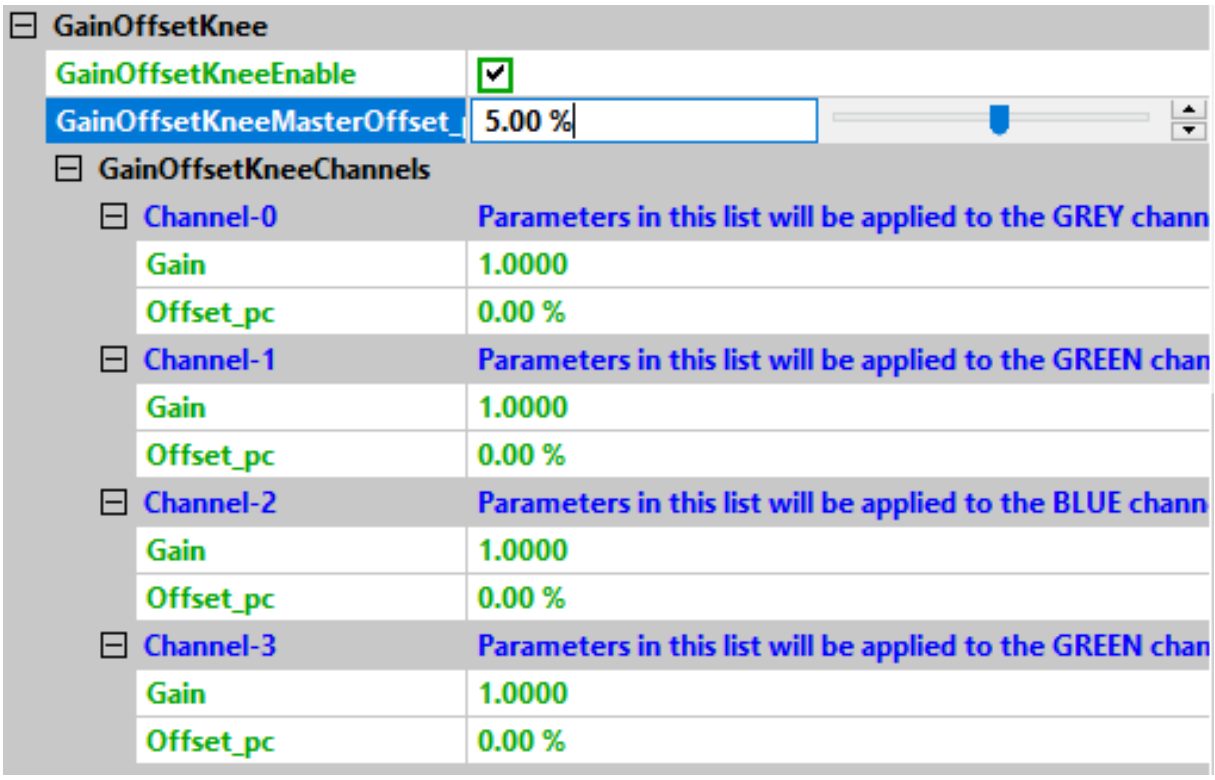


Figure 6: Assign overall/master offset to the image



Figure 7: The image with 5% overall offset

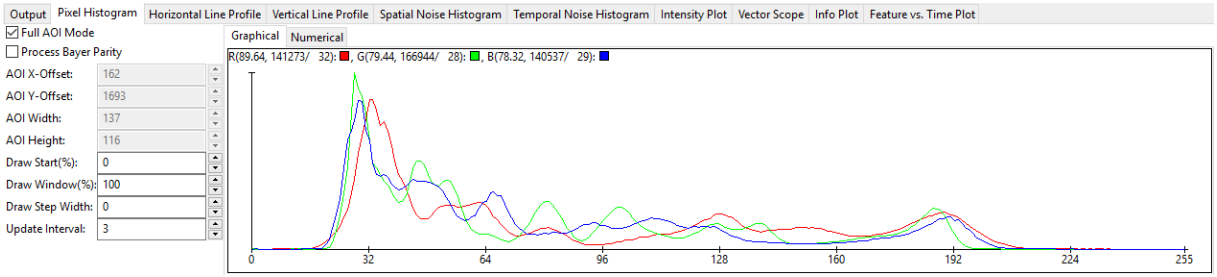


Figure 8: The histogram with 5% overall offset

4. Among the GainOffsetKneeChannels there are 4 channels. For mono images, only channel 0 is used. For RGB images, channel 0-2 are used: red channel, green channel and blue channel respectively. For Bayer images, channel 0-3 are used. For more description please refer to *Figure 3*. As an example, a gain of 1.0625dB is applied to the red channel. As shown in *Figure 10* and *Figure 11*, the grey-level of the red channel is increased while the other two channels remain the same.

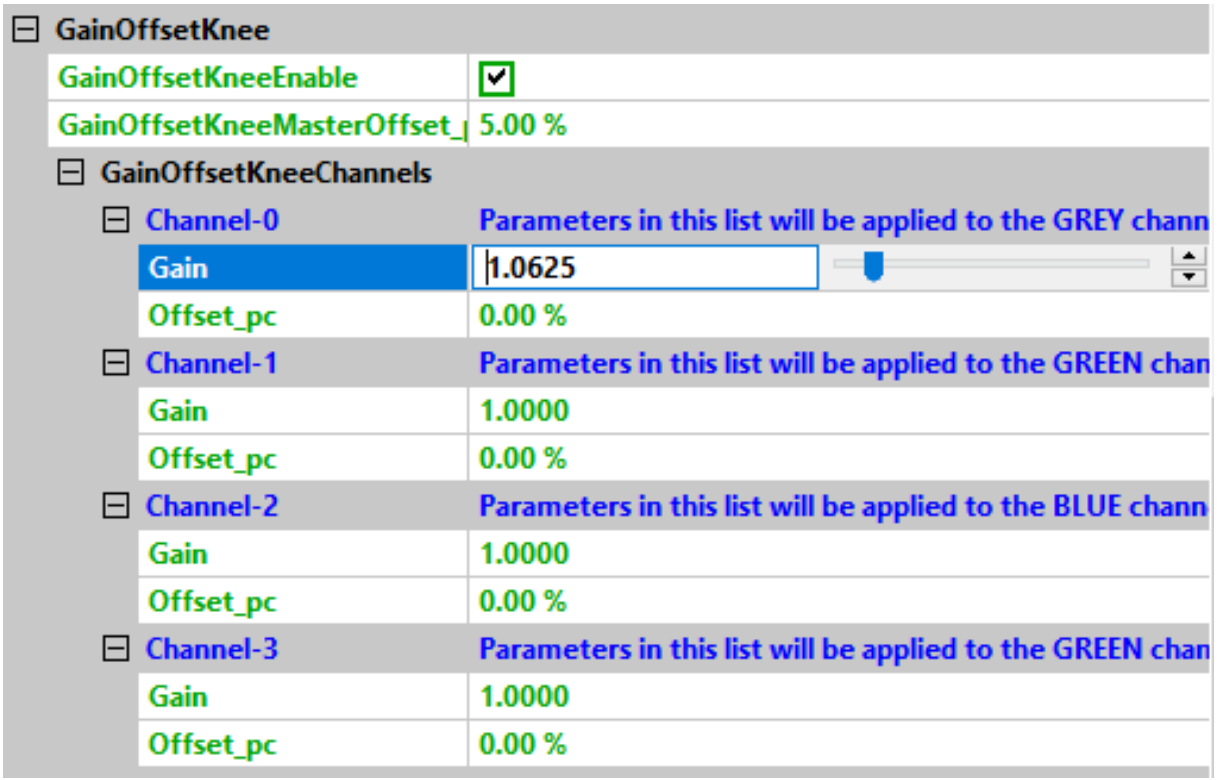


Figure 9: Assign individual gain to the red channel



Figure 10: The image with 1.0625dB gain in the red channel

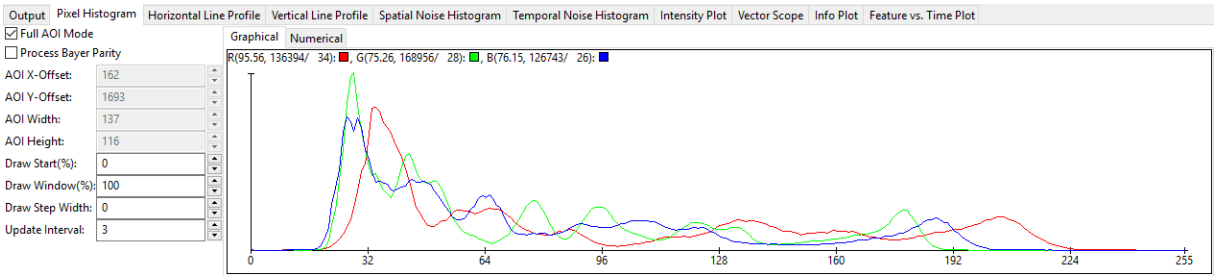


Figure 11: The histogram with 1.0625dB gain in the red channel

5. The individual black-level can be assigned using the channel specific *'Offset_pc'*. Analogous to 'GainOffset↩ KneeMasterOffset_pc', a positive offset increases the black-level of the channel, whereas a negative offset reduces it. To visualize the effect, an offset of 5% is given as an example in the red channel. The histogram (see *Figure 14*) shows therefore a 12.75 (which is 5% x 255) offset increase in the red channel.

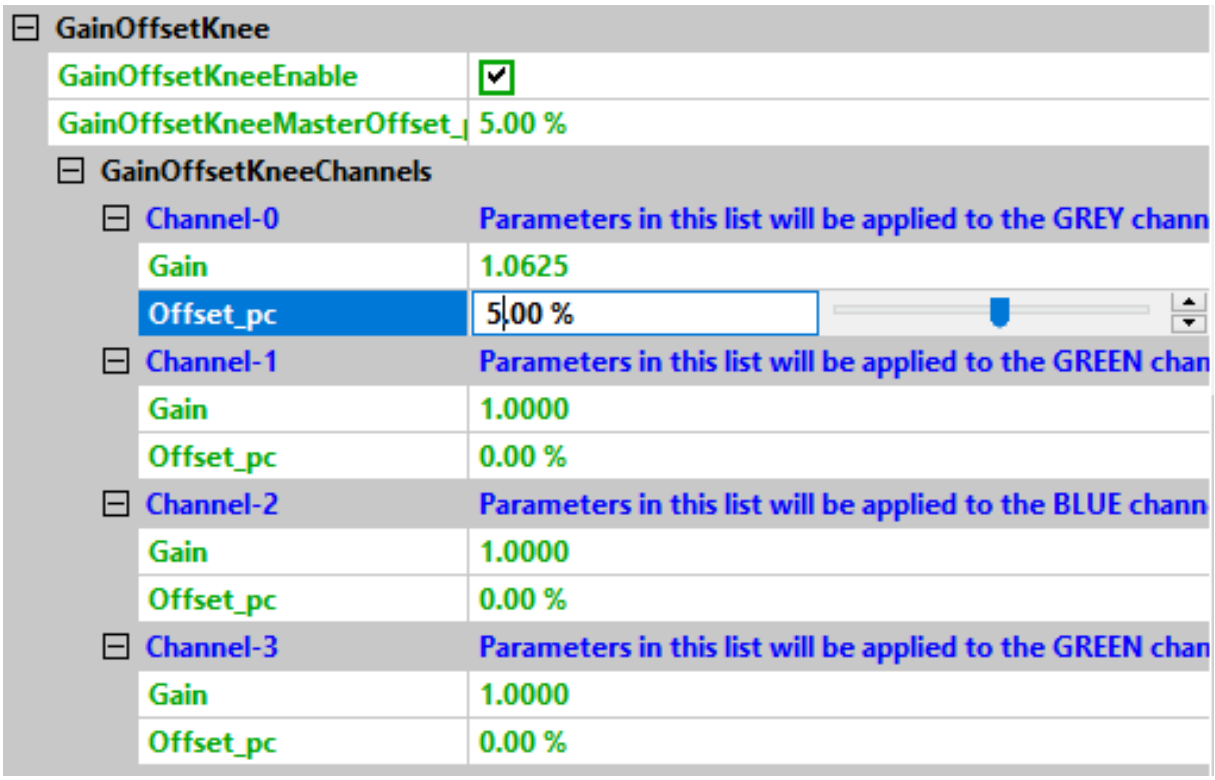


Figure 12: Assign individual offset to the red channel



Figure 13: The image with 5% offset in the red channel

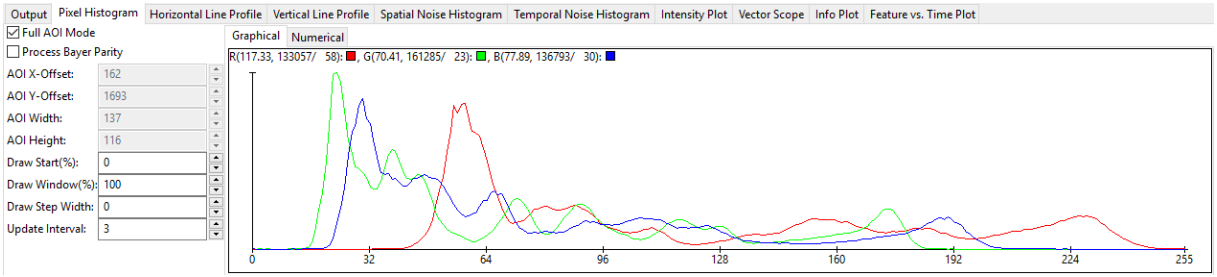


Figure 14: The histogram with 5% offset in the red channel

1.18.2.9.2 Configuration Using The API Depending on the programming language you are working with the names of classes, namespaces and properties might vary slightly. For C++ please refer to the `GainOffset↩KneeChannelParameters` class and the `Image Processing` class for some guidance, for other languages when searching for the *offset* or *knee* properties similar things can be found.

1.18.3 Improving the communication

There are several use cases concerning communication and bandwidth issues:

- **Optimizing the bandwidth** (p. 275)

1.18.3.1 Optimizing the bandwidth

1.18.3.1.1 Limiting the bandwidth of the imaging device

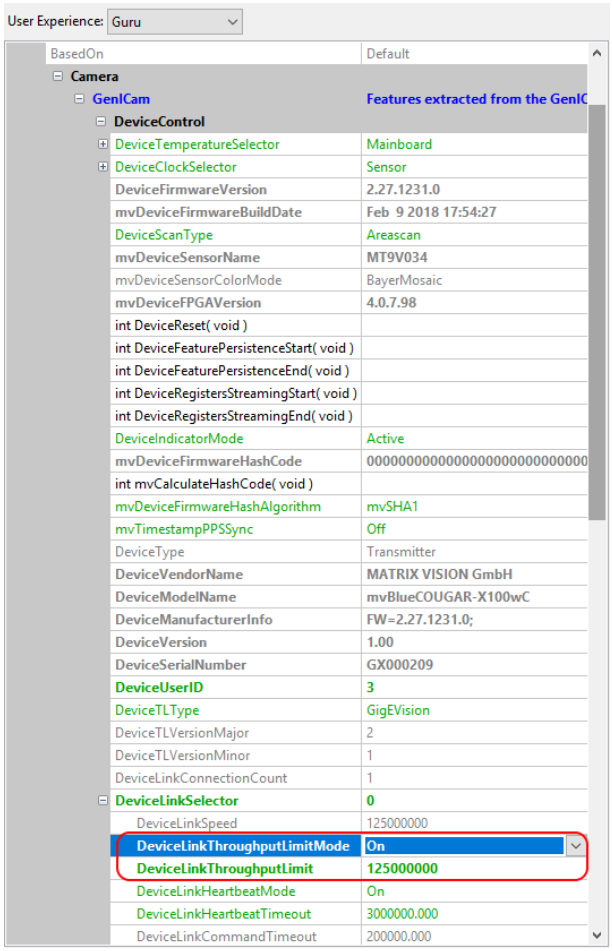
For a setup of multiple streaming devices connected to one host, it is highly recommended to consider the information of this chapter.
Even if the connected links and devices are able to handle the average throughput of a streaming setup situations might be encountered where the data throughput temporarily exceeds the capabilities of involved network components for a very short period of time. This might result in packet loss with GigE Vision™ devices or overflowing buffers with USB3 Vision™ devices.

1.18.3.1.1.1 How the Device Link Throughput Limit works From version 1.5.2 and above, the GenICam™ SFNC defines the features **DeviceLinkThroughputLimit** and **DeviceLinkThroughputLimitMode**, which are meant to provide a standardized way to control the throughput limit for every GenICam™ device.
All MATRIX VISION devices with firmware version 2.25.0 or above support the GenICam™ SFNC features DeviceLinkThroughputLimitMode and DeviceLinkThroughputLimit to limit the bandwidth used by a device in a convenient way. On USB3 Vision™ devices, the effective bandwidth is a combination of image size and acquisition frame rate. If a *DeviceLinkThroughputLimit* is set and the unlimited link speed exceeds the set limit, the acquisition frame rate would be adjusted to fit the DeviceLinkThroughputLimit.

If it is necessary to limit the outgoing link throughput of a device, this can be accomplished the following way:

Available since firmware version 2.25.0

1. In `"Setting -> Base -> Camera -> GenICam -> Device Control -> Device Link Selector"` set property **"Device Link Throughput Limit Mode"** to **"On"**.
2. Now, you can set the bandwidth with **"Device Link Throughput Limit"** to your desired bandwidth in bits per second



wxPropView - Setting Device Link Throughput Limit

1.18.4 Working with triggers

There are several use cases concerning trigger:

- **Processing triggers from an incremental encoder** (p. 277)
- **Generating a pulse width modulation (PWM)** (p. 280)
- **Outputting a pulse at every other external trigger** (p. 282)
- **Creating different exposure times for consecutive images** (p. 284)
- **Detecting overtriggering** (p. 287)
- **Triggering of an indefinite sequence with precise starting time** (p. 291)
- **Low latency triggering** (p. 294)

1.18.4.1 Processing triggers from an incremental encoder

1.18.4.1.1 Basics

The following figure shows the principle of an incremental encoder:

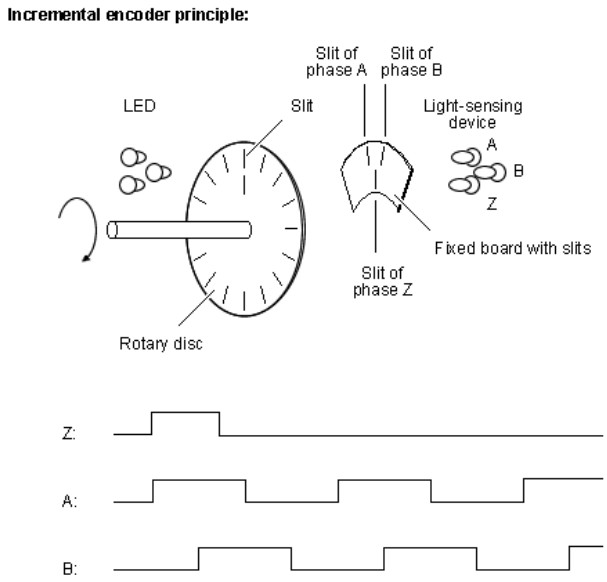


Figure 1: Principle of an incremental encoder

This incremental encoder will send a A, B, and Z pulse. With these pulses there are several ways to synchronize an image with an incremental encoder.

1.18.4.1.2 Using Encoder Control

To create an external trigger event by an incremental encoder, please follow these steps:

1. Connect the incremental encoder output signal A, for example, to the digital input 0 (**"Line4"**) of the mv↩ BlueFOX3 .
   This line counts the forward pulses of the incremental encoder.
2. Depending on the signal quality, it could be necessary to **set a debouncing filter at the input** (p. 299) (red box in Figure 3):
   Adapt in *"Setting -> Base -> Camera -> GenICam -> Digital I/O Control"* the *"Line Selector"* to **"Line4"** and set *"mv Line Debounce Time Falling Edge"* and *"mv Line Debounce Time Rising Edge"* according to your needs.
3. Set the trigger *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector"* **"↩ FrameStart"** to the **"Encoder0"** (*"Trigger Source"*) signal.
4. Then set *"Setting -> Base -> Camera -> GenICam -> Encoder Control -> Encoder Selector"* to **"Encoder0"** and
5. adapt the parameters to your needs.

See also

Encoder Control (p. 115)

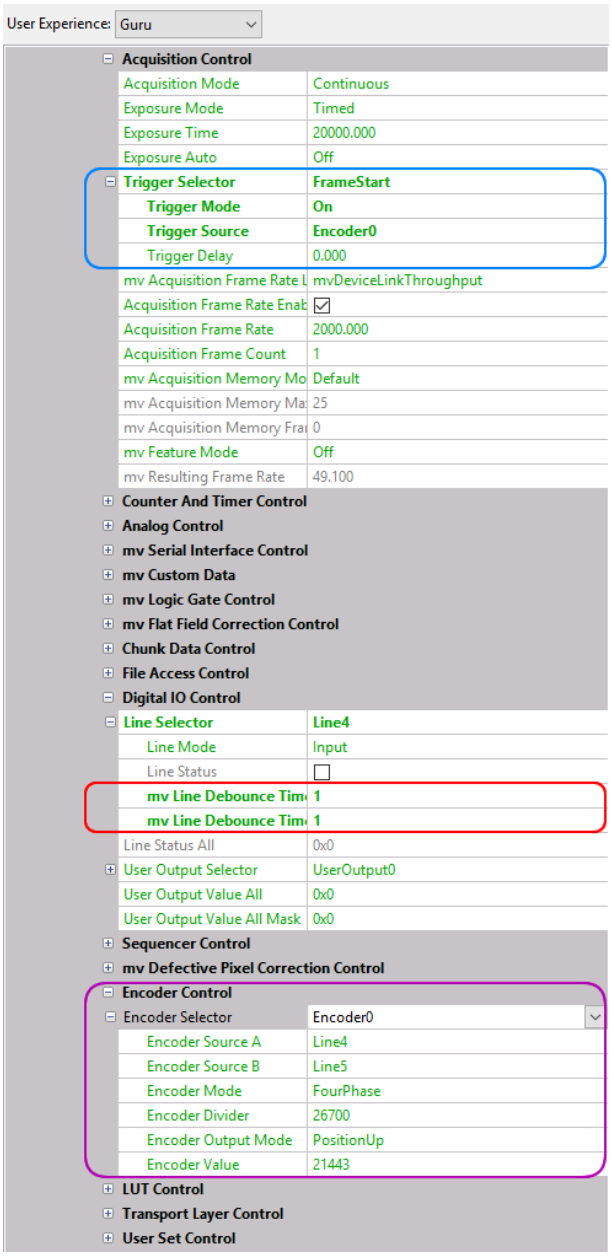


Figure 2: wxPropView settings

Note

The max. possible frequency is 5 KHz.

1.18.4.1.2.1 Programming the Encoder Control

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
  GenICam::EncoderControl ec( pDev );
  ec.encoderSelector.writeS( "Encoder0" );
  ec.encoderSourceA.writeS( "Line4" );
  ec.encoderMode.writeS( "FourPhase" );
  ec.encoderOutputMode.writeS( "PositionUp" );
  // more code
```

1.18.4.1.3 Using Counter

It is also possible to use **Counter** and **CounterEnd** as the trigger event for synchronizing images with an incremental encoder

To create an external trigger event by an incremental encoder, please follow these steps:

1. Connect the incremental encoder output signal A, for example, to the digital input 0 (**"Line4"**) of the mv↩ BlueFOX3 .
   This line counts the forward pulses of the incremental encoder.
2. Set *"Setting -> Base -> Camera -> GenICam -> Counter and Timer Control -> Counter Selector"* to **"↩ Counter1"** and
3. **"Counter Event Source"** to **"Line4"** to count the number of pulses e.g. as per revolution (e.g. **"Counter Duration"** to **3600**).
4. Then set the trigger *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector"* **"FrameStart"** to the **"Counter1End"** (*"Trigger Source"*) signal.

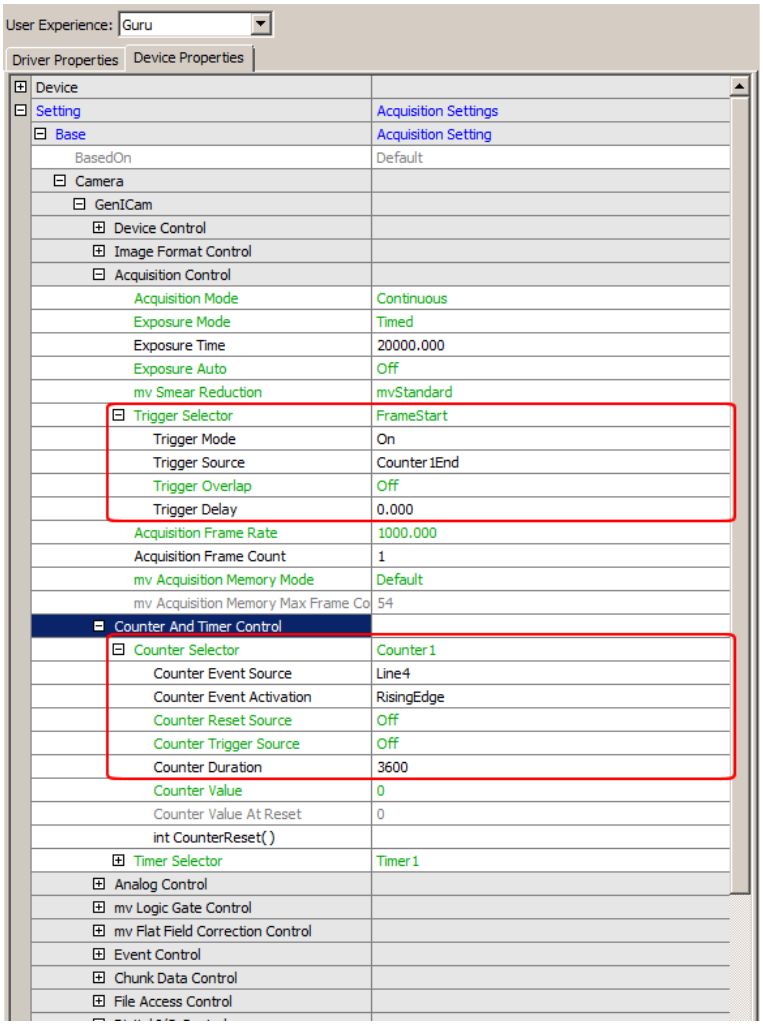


Figure 3: wxPropView setting

To reset **"Counter1"** at Zero degrees, you can connect the digital input 1 (**"Line5"**) to the encoder Z signal.

1.18.4.2 Generating a pulse width modulation (PWM)

1.18.4.2.1 Basics

To dim a laser line generator, for example, you have to generate a pulse width modulation (PWM).

For this, you will need

- 2 timers and
- the active signal of the second timer at an output line

1.18.4.2.2 Programming the pulse width modulation

You will need two timers and you have to set a trigger.

- *Timer1* defines the interval between two triggers.
- *Timer2* generates the trigger pulse at the end of *Timer1*.

The following sample shows a trigger

- which is generated every second and
- the pulse width is 10 ms:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...

// Master: Set timers to trig image: Start after queue is filled
    GenICam::CounterAndTimerControl catcMaster(pDev);
    catcMaster.timerSelector.writeS( "Timer1" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 1000000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );

    catcMaster.timerSelector.writeS( "Timer2" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 10000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );
```

See also

Counter And Timer Control (p. 107)

Note

Make sure the *Timer1* interval must be larger than the processing time. Otherwise, the images are lost.

Now, the two timers will work like the following figure illustrates, which means

- *Timer1* is the trigger event and
- *Timer2* the trigger pulse width:

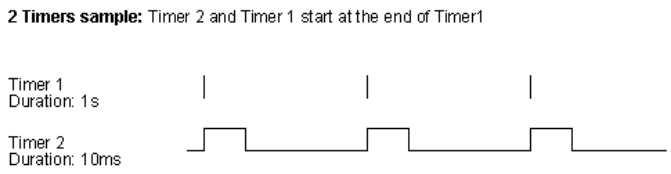


Figure 1: Timers

The timers are defined, now you have to set the digital output, e.g. **"Line 0"**:

```
// Set Digital I/O
    GenICam::DigitalIOControl io(pDev);
    io.lineSelector.writeS( "Line0" );
    io.lineSource.writeS( "Timer2Active" );
```

See also

Digital I/O Control (p. 114)

This signal has to be connected with the digital inputs of the application.

1.18.4.2.3 Programming the pulse width modulation with wxPropView

The following figures show, how you can set the timers using the GUI tool **wxPropView** (p. 99)

1. Setting of *Timer1* (blue box) on the master camera:

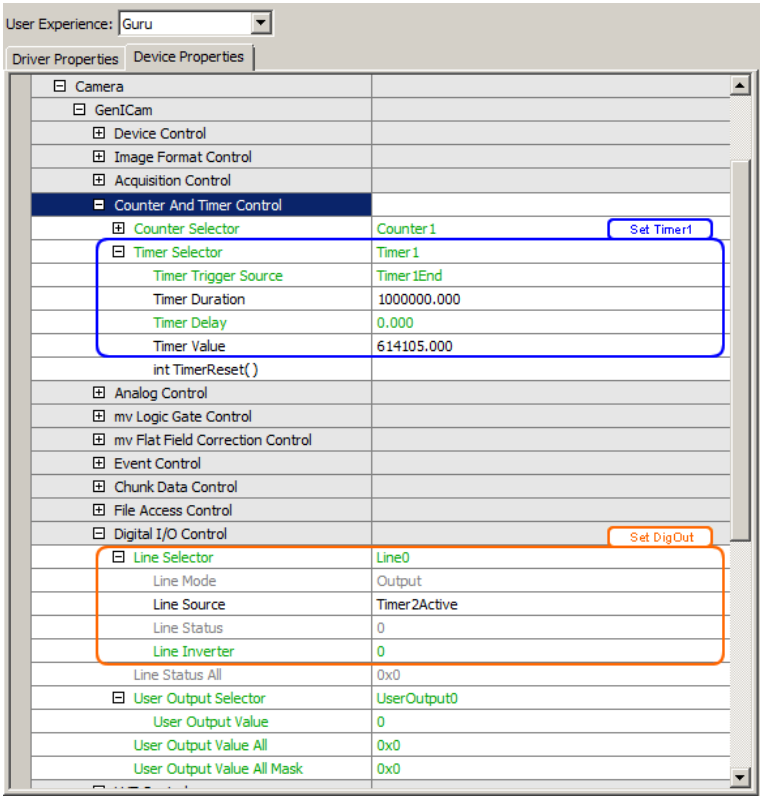


Figure 2: wxPropView - Setting of *Timer1*

2. Setting of *Timer2* (purple on the master camera):

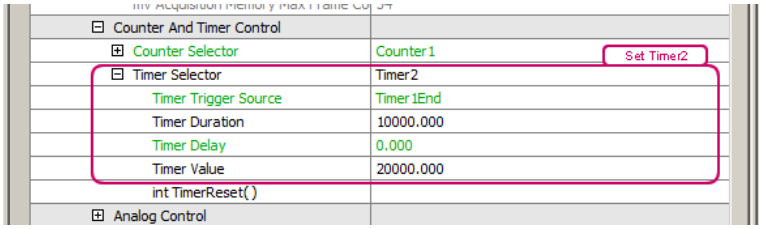


Figure 3: wxPropView - Setting of *Timer2*

3. Assigning timer to DigOut (orange box in Figure 2).

1.18.4.3 Outputting a pulse at every other external trigger

To do this, please follow these steps:

1. Switch *"Trigger Mode"* to **"On"** and
2. Select the *"Trigger Source"*, e.g. **"Line5"**.
3. Use *"Counter1"* and count the number of input trigger by setting the *"Counter Duration"* to **"2"**.
4. Afterwards, start *"Timer1"* at the end of *"Counter1"*:

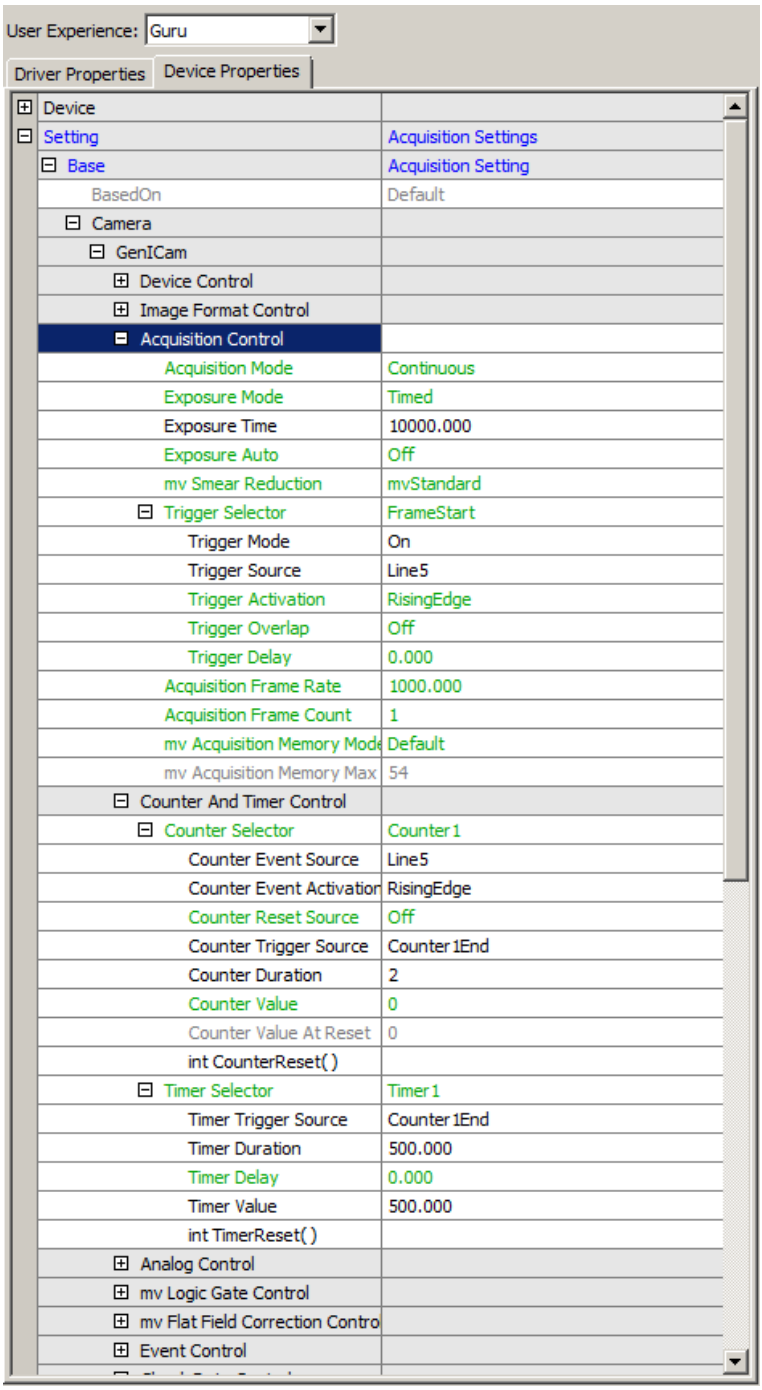


Figure 1: wxPropView - Setting the sample

The *"Timer1"* appears every second image.

Now, you can assign *"Timer1Active"* to a digital output e.g. **"Line3"**:

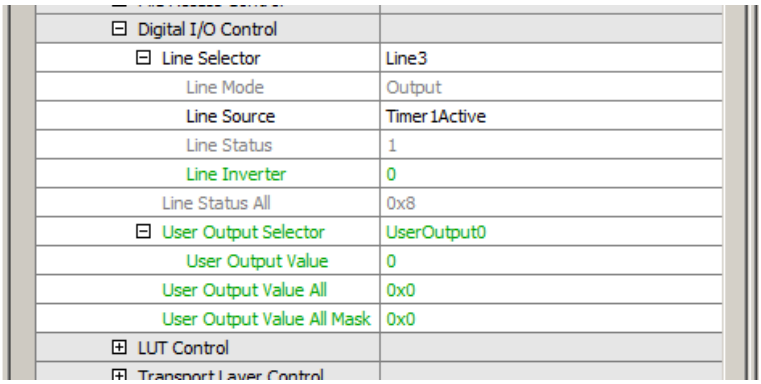


Figure 2: Assigning the digital output

Note

You can delay the pulse if needed.

1.18.4.4 Creating different exposure times for consecutive images

If you want to create a sequence of exposure times, you have to trigger the camera "externally" via pulse width:

1. Use **Timer** and **Counter** to build a sequence of different pulse widths.
2. Use the **Counter** for the time between the exposures (with respect to the readout times).
3. Afterwards, use an **AND** gate followed by **OR** gate to combine different exposure times.

Note

Please be sure that the sensor can output the complete image during **Counter1** or **Counter2**. Otherwise, only one integration time will be used.

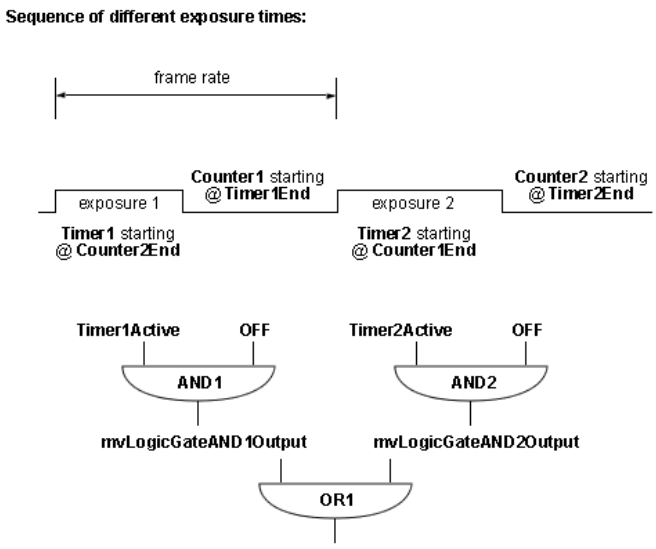


Figure 1: wxPropView - Logic gate principle

You can set this sample in **wxPropView** (p. 99). E.g. the sensor makes 22.7 frames per second in **Continuous Mode** (p. 103). This means that the sensor needs 44 ms to output the complete image.

$$\frac{1}{22.7} = \text{approx. } 44 \text{ ms}$$

We take 55 ms to be sure. Now, as different exposure times we take 1 ms (**Timer1**) and 5 ms (**Timer2**). To get the 55 ms, we have to add 54000 us (**Counter1**) and 50000 us (**Counter2**).

Finally, you have to set the logic gate as shown in the figure:

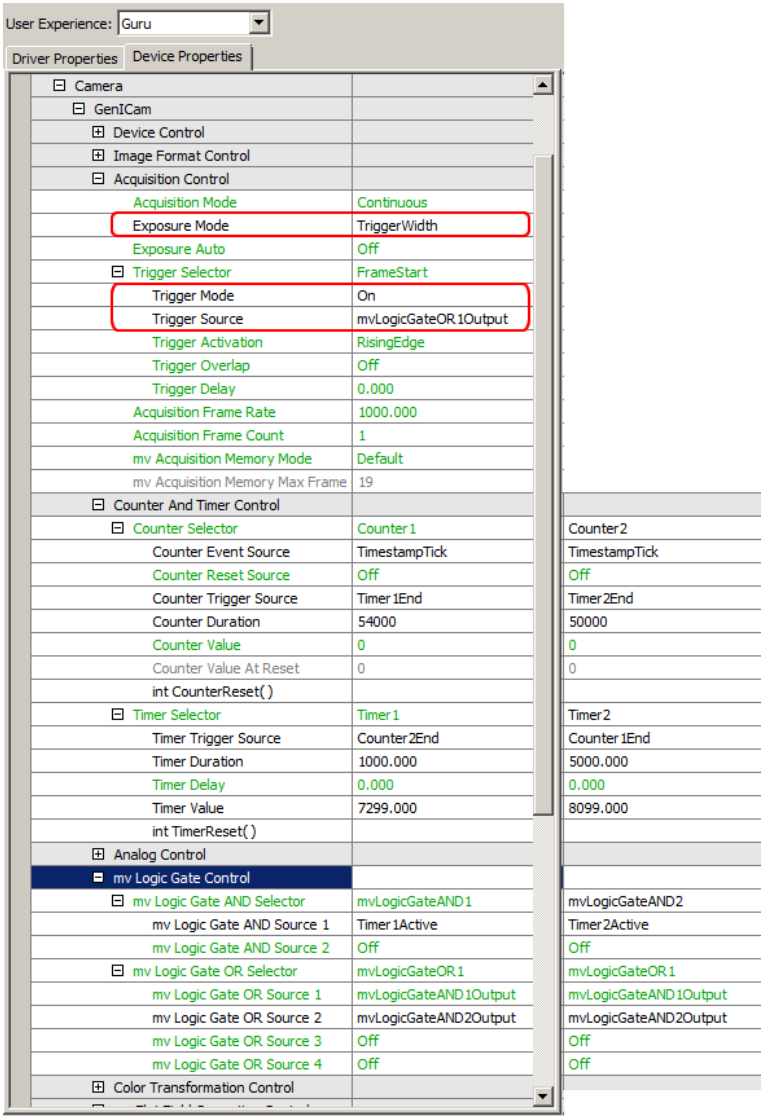


Figure 2: wxPropView - Logic gate setting

Note

Because there are 4 counters and 2 timers you can only add one further exposure time using one counter as a timer.

So if you want other sequences, you have to use the counters and timers in a flexible way as show in the next sample:

1.18.4.4.1 Sequence with 4 times exposure A followed by 1 time exposure B

If you have an external trigger, you can use the counter and timer to create longer exposure sequences.

For example, if you want a sequence with 4 times exposure A followed by 1 time exposure B you can count the trigger events. That means practically:

1. Use **Counter1** to count 5 trigger signals then
2. issuing **Timer2** for the long exposure time (re-triggered by **Counter1End**).
3. Every trigger issues **Timer1** for the short exposure.
4. Afterwards, an **AND** gate followed by **OR** gate combines the different exposure times.

In **wxPropView** (p. 99) it will look like this:

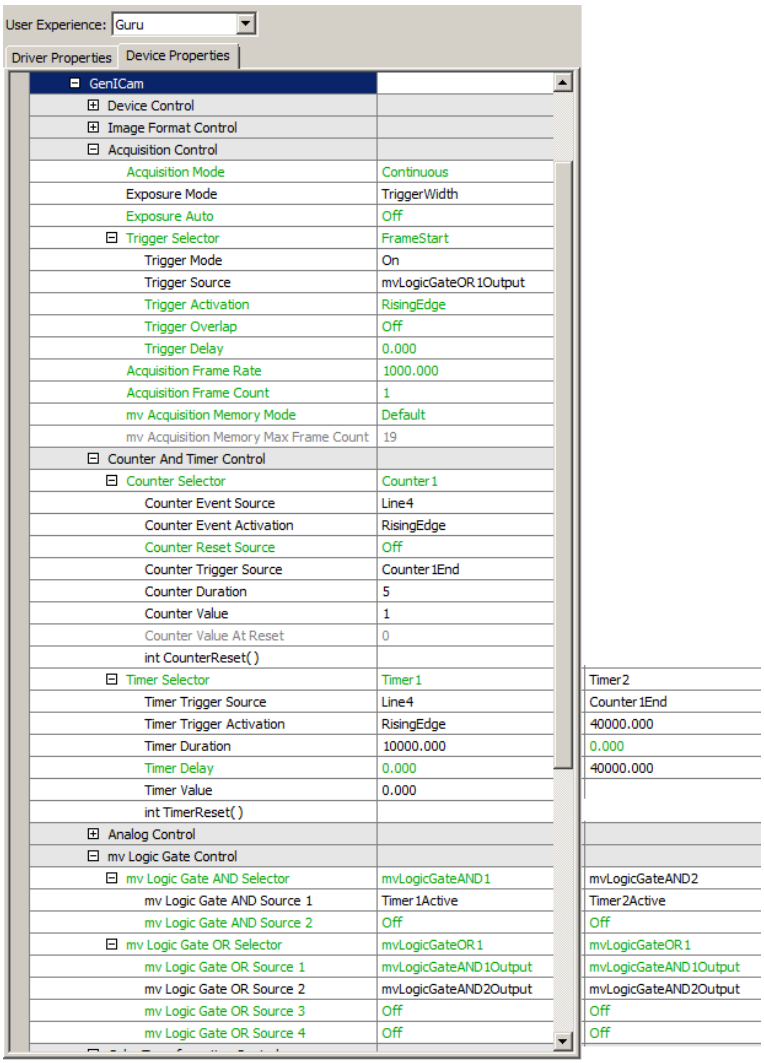


Figure 3: wxPropView - Logic gate setting 2

1.18.4.5 Detecting overtriggering

1.18.4.5.1 Scenario

The image acquisition of a camera consists of two steps:

- exposure of the sensor and
- readout of the sensor data

During these steps, a trigger signal will be skipped:

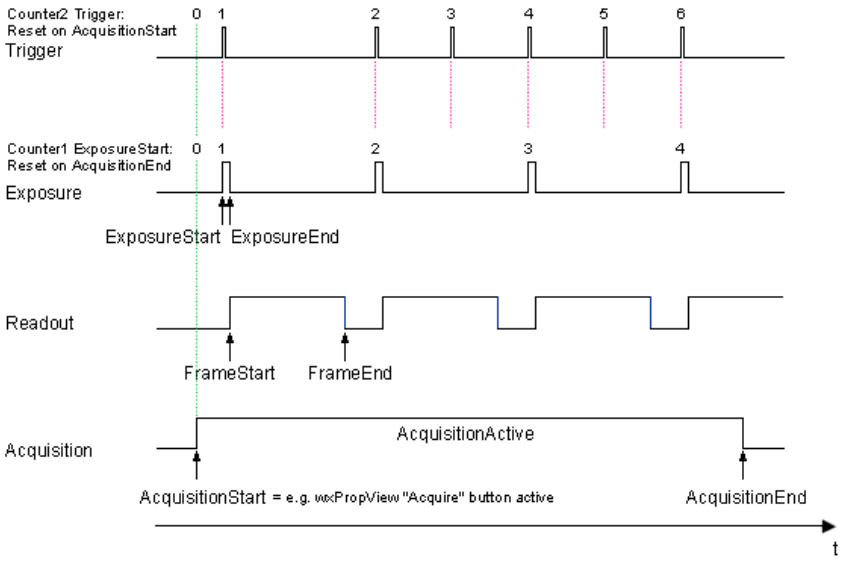


Figure 1: Trigger counter increases but the start exposure counter not

To notice overtriggering, you can use **counters** (p. 107):

- One counter counts the incoming trigger signals, the
- second counter counts the `ExposureStart` signals.

Using the **chunk data** (p. 113) you can overlay the counters in the live image.

1.18.4.5.2 Setting the overtrigger detector using wxPropView

First of all, we have to set the **trigger** in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* with following settings:

Property name wxPropView	Setting
Trigger Selector	`FrameStart`
Trigger Mode	`On`
Trigger Source	`Line4`
Trigger Activation	`RisingEdge`
Exposure Mode	`Timed`

This trigger will start an acquisition after a rising edge signal on line 4 (= DigIn0).

Now, set the two counters. Both counters (`Counter1` and `Counter2`) will be reset and start after the acquisition (`AcquisitionStart`) has started.

While `Counter1` increases with every `ExposureStart` event (see figure above for the event and acquisition details) ...

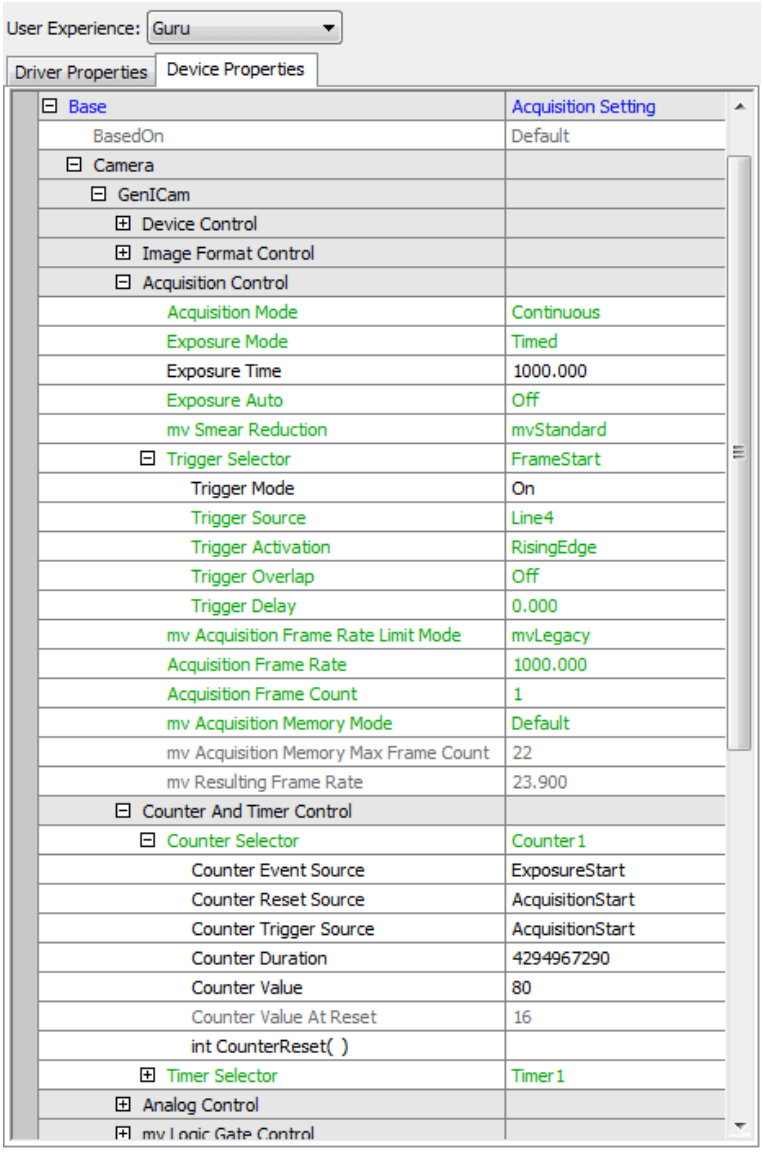


Figure 2: Setting Counter1

... `Counter2` increases with every `RisingEdge` of the trigger signal:

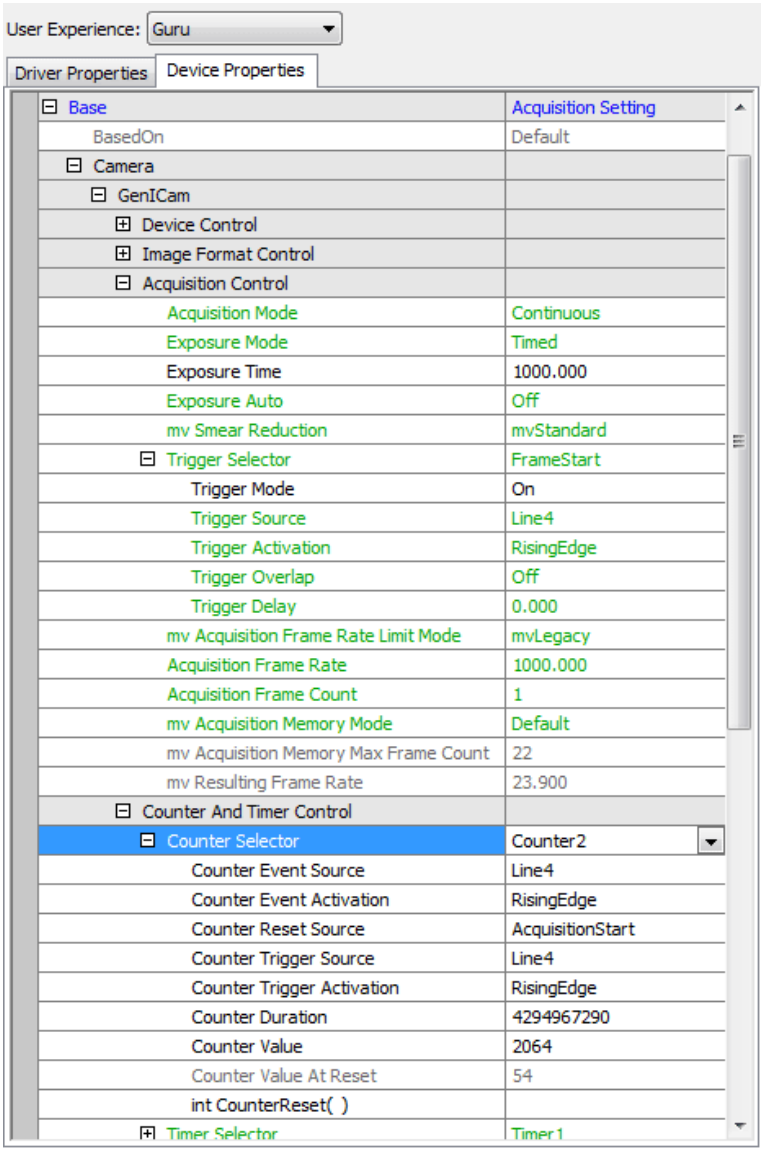


Figure 3: Setting Counter2

Now, you can check if the trigger signal is skipped (when a rising edge signal is active during readout) or not by comparing the two counters.

Enable the inclusion of the selected chunk data ("`Chunk Mode Active = 1`") in the payload of the image in *"Setting -> Base -> Camera -> GenICam -> Chunk Data Control"*:

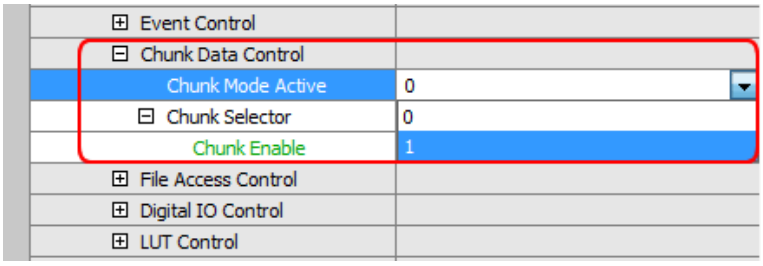


Figure 4: Enable chunk data

Activate the info overlay in the display area. Right-click on the live display and select: *"Request Info Overlay"*

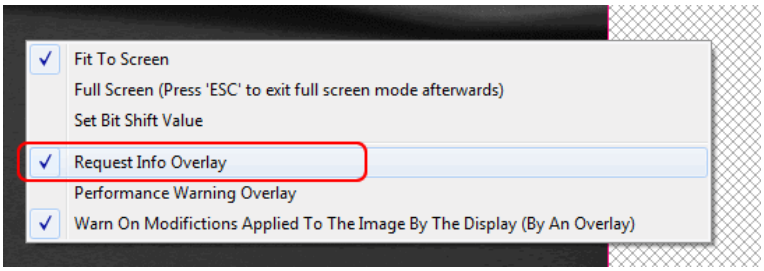


Figure 5: Show chunk data

The following figure shows that no trigger signal is skipped:



Figure 6: Trigger Signal counter equals ExposureStart counter

The following figure shows that the acquisition is overtriggered:



Figure 7: Trigger Signal counter is higher than ExposureStart counter

1.18.4.6 Triggering of an indefinite sequence with precise starting time

1.18.4.6.1 Scenario

Especially in the medical area, there are applications where a triggered acquisition is started, for example, with a foot switch. Following challenges have to be solved in combination with these applications:

- The user wants the acquired image immediately (precise starting time).
- It is not known, when the user stops the acquisition (indefinite sequence).

Using *AcquisitionStart* as the trigger source, it could take between 10 and 40 ms until the camera acquires the first frame. That's not really an immediately acquisition start. It is recommended to use *FrameStart* as the trigger source instead. However, according to the time the trigger event occurs, there will be a timeout during the first frame in nearly all cases.

You can avoid this using a timer which generates a "high" every 100 us and which is connected to the trigger input *Line4* using a logical *AND* gate. I.e. if the timer is "high" and there is a trigger signal at *Line4* then the logical conjunction is true. The AND gate result is then connected as *TriggerSource* of the *FrameStart* trigger using a logical *OR* gate. I.e. as soon as the logical AND conjunction is true, the trigger source is true and the image acquisition will start.

The following figure illustrates the settings:

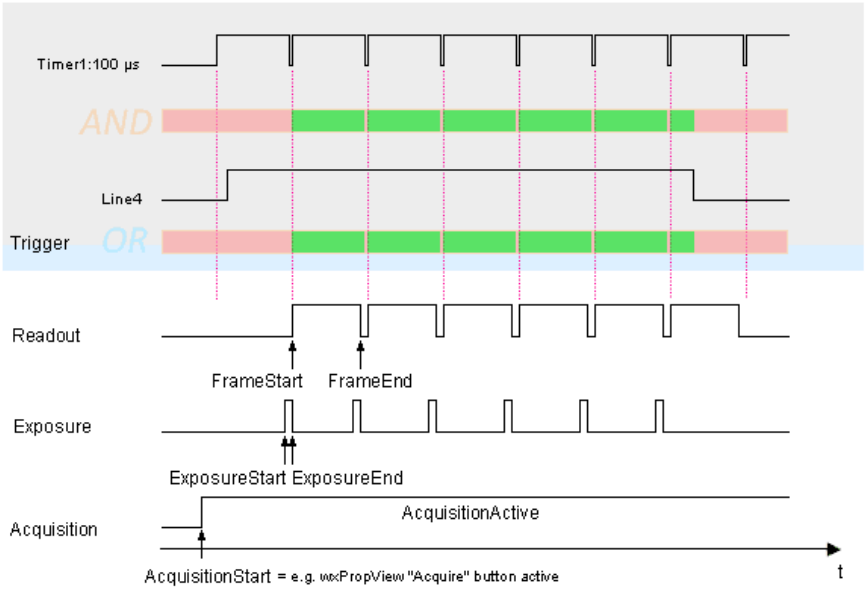


Figure 1: Schematic illustration of the settings

With this setting, there is still an acceptable time delay of approx. 100 to 130 us possible.

1.18.4.6.2 Creating the use case using wxPropView

First of all, we have to set the **timer** in *"Setting -> Base -> Camera -> GenICam -> Counter And Timer Control"* with following settings:

Property name wxPropView	Setting
Timer Selector	`Timer1`
Timer Trigger Source	`Timer1End`
Timer Duration	`100.000`

Afterwards, we have to set the **logical gates** in *"Setting -> Base -> Camera -> GenICam -> mv Logic Gate Control"* with following settings:

Property name wxPropView	Setting
mv Logic Gate AND Selector	`mvLogicGateAND1`
mv Logic Gate AND Source 1	`Line4`
mv Logic Gate AND Source 2	`Time1Active`
mv Logic Gate OR Selector	`mvLogicGateOR1`
mv Logic Gate OR Source 1	`mvLogicGateAND1Output`

Finally, we have to set the **trigger** in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* with following settings:

Property name wxPropView	Setting
Trigger Selector	`FrameStart`
Trigger Mode	`On`
Trigger Source	`mvLogicGateAND1Output`
Trigger Activation	`RisingEdge`
Exposure Mode	`Timed`

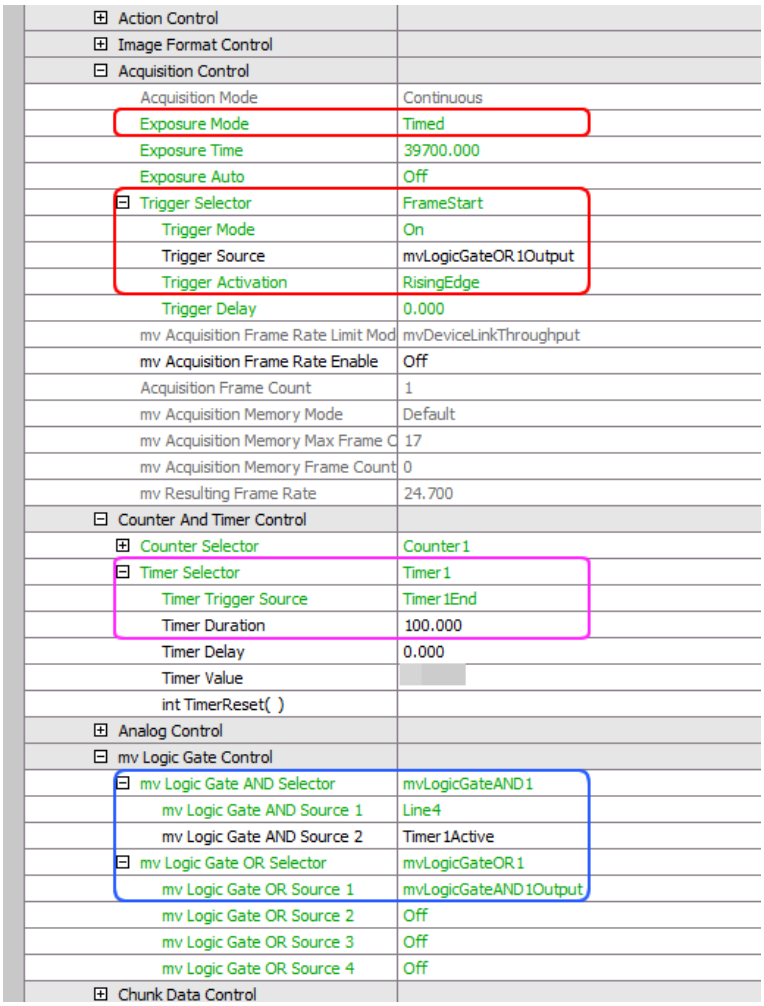


Figure 2: Sample settings

1.18.4.7 Low latency triggering

Since

Firmware revision 2.45

1.18.4.7.1 Introduction

Because the exposure start is synced with the line-period, there is a jitter of the start of up to one line. Additionally, there is a latency of 2 or 4 line-periods (depending on the sensor) until the exposure actually starts. The following image shows this visually:

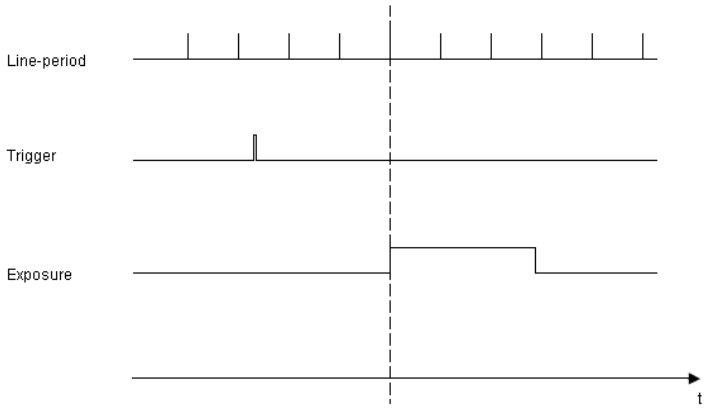


Figure 1: Normal trigger mode

The exposure time increment is a multiple of the line-period. This trigger behavior works also with overlapped exposure and readout.

With the Pregius global shutter sensors of the 2nd, 3rd, and 4th generation from Sony, it is possible to start the exposure on the trigger. There is no latency and no jitter. The length of exposure is also not based on the line-period, because the sensor does not depend on it in this mode. The minimum exposure time is limited by several line periods though. Because during readout the sensor obviously requires the line-periods this mode is only supported in non-overlapped exposure and readout:

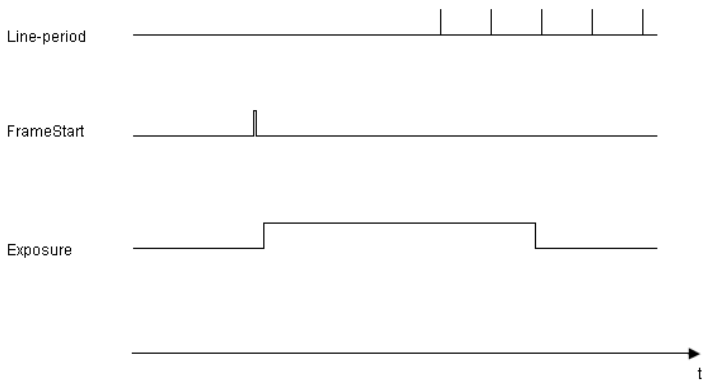


Figure 2: LowLatency trigger mode

1.18.4.7.2 Using wxPropView

To activate the LowLatency trigger mode, please follow these steps:

1. Set the **TriggerSelector** in *"Setting → Base → Camera → GenICam → AcquisitionControl"* to **FrameStart** .
2. Afterwards, change to **TriggerMode** and
3. Select **mvLowLatency**.

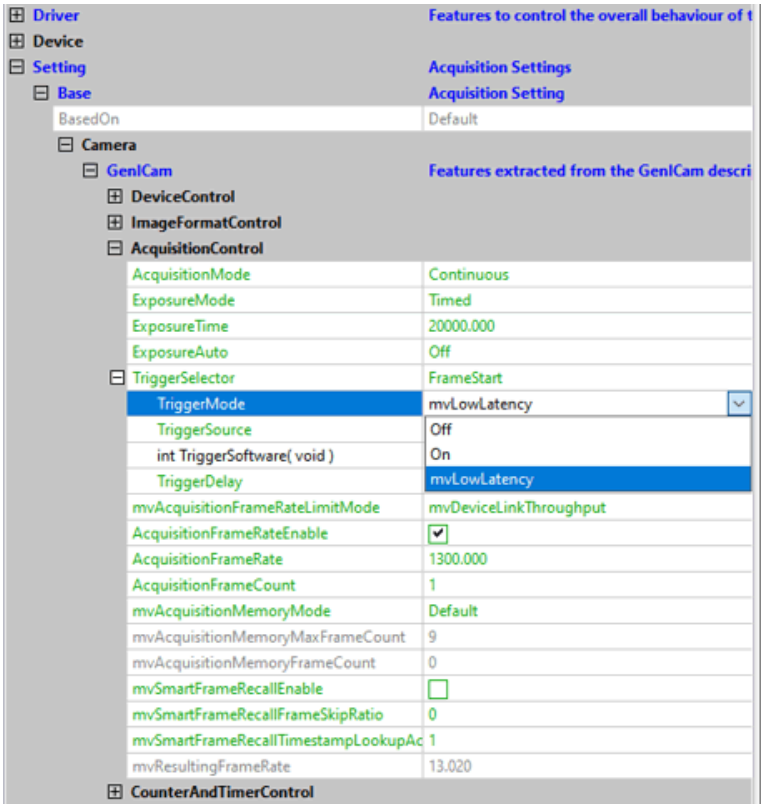


Figure 3: mvLowLatency

1.18.5 Working with I/Os

There are several use cases concerning I/Os:

- **Controlling strobe or flash at the outputs** (p. 296)
- **Creating a debouncing filter at the inputs** (p. 299)

1.18.5.1 Controlling strobe or flash at the outputs

Of course, MATRIX VISION devices support strobe or flash lights. However, there are several things you have to keep in mind when using strobes or flash:

1. Be sure that the illumination fits with the movement of the device under test.
2. Bright illumination and careful control of exposure time are usually required.
3. To compensate blur in the image, short exposure times are needed.

Alternatively, you can use flash with short burn times. For this, you can control the flash using the camera. The following figures show, how you can do this using **wxPropView** (p. 99)

1. Select in *"Setting -> Base -> Camera -> Digital I/O Control"* the output line with the **"Line Selector"** to which the strobe or flash is connected.
2. Now, set the **"Line Source"** to **"mvExposureAndAcquisitionActive"**.
   This means that the signal will be high for the exposure time and only while acquisition of the camera is active.

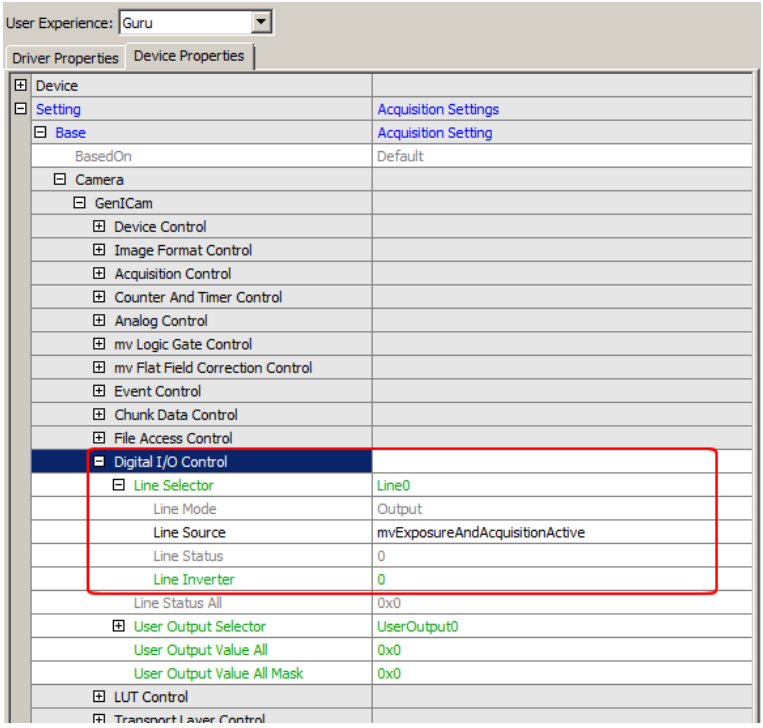


Figure 1: Setting the **"Line Source"** to **"mvExposureAndAcquisitionActive"**

Additionally, you can shift the signal controlling the flash or strobe using the camera. The camera will send the signal at the selected output line either sooner or later. The following figure demonstrates the procedure using **wxPropView** (p. 99)

1. Enable in *"Setting -> Base -> Camera -> AcquisitionControl"* the property **"mvExposureActiveShift↩ Enable"**.
   The property **"mvExposureActiveShift"** will appear.

2. Now you can select a shift value and **"mvExposureAndAcquisitionActive"** will be available either later (with positive values) or sooner (with negative values).

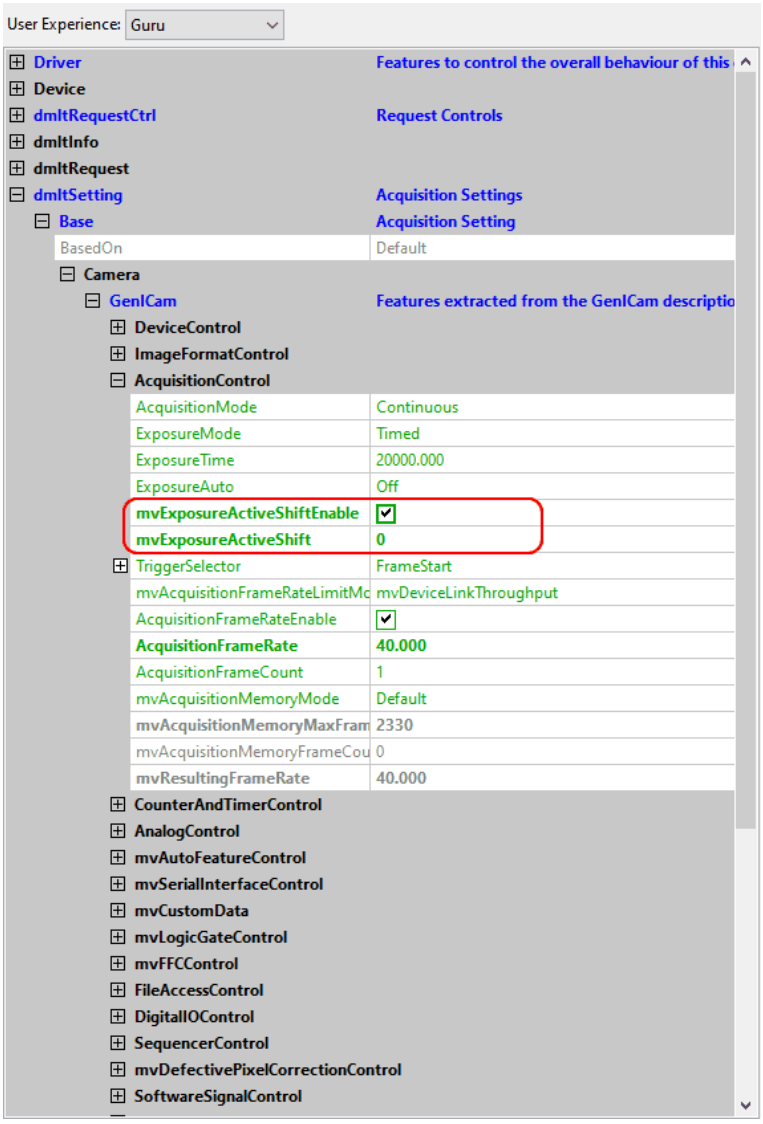


Figure 2: Controlling the shift of **"mvExposureAndAcquisitionActive"** at the output line

Note

This can be combined using an external trigger.

1.18.5.1.1 Special case: Rolling shutter Starvis sensors

Since

Firmware version 2.24.975

With the rolling shutter flash mode, the exposure time which is set corresponds to the time where all lines are exposed simultaneously. The exposure signal corresponds to the time where the last line starts until the first line ends. The rolling shutter flash mode avoids motion effects and is suitable for exposure times $>$ 300 usec.

Additional to the digital output settings in **Controlling strobe or flash at the outputs** (p. 296) you have to set

1. the **"mvShutterMode"** to **"mvRollingShutterFlash"** and
2. the desired **"ExposureTime"**.

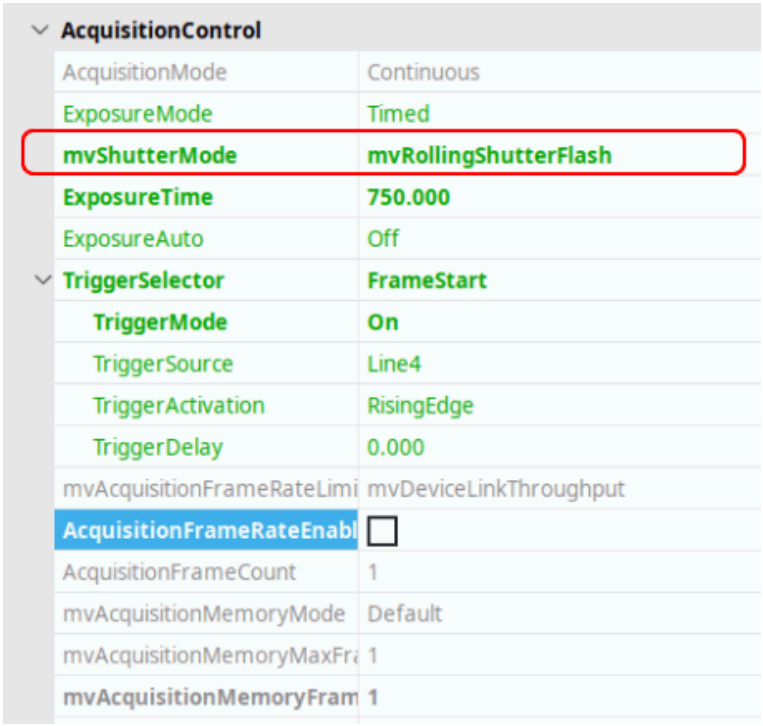


Figure 3: Setting the **"mvShutterMode"** to **"mvRollingShutterFlash"**

Note

If you have an external trigger, please adjust the settings according to **this trigger sample** (p. 287).

1.18.5.1.2 Compensating delay of strobe or flash

Normally, the input circuitry of flash has a delay (e.g. low pass filtering). Using **"ExposureActive"** to fire strobe would actually illuminate delayed with respect to exposure of the sensor. Figure 4 shows the problem:

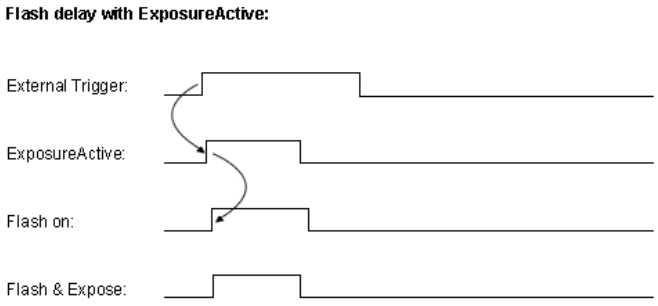


Figure 4: Flash delay with **"ExposureActive"**

To solve this issue, you can use following procedure:

1. Do not use **"ExposureActive"** for triggering strobe.

2. Build flash signal with **Timer**,
3. trigger **Timer** with external trigger (e.g. **"Line5"**).
4. Use **"Trigger Delay"** to delay exposure of the sensor accordingly.

In **wxPropView** (p. 99) it will look like this:

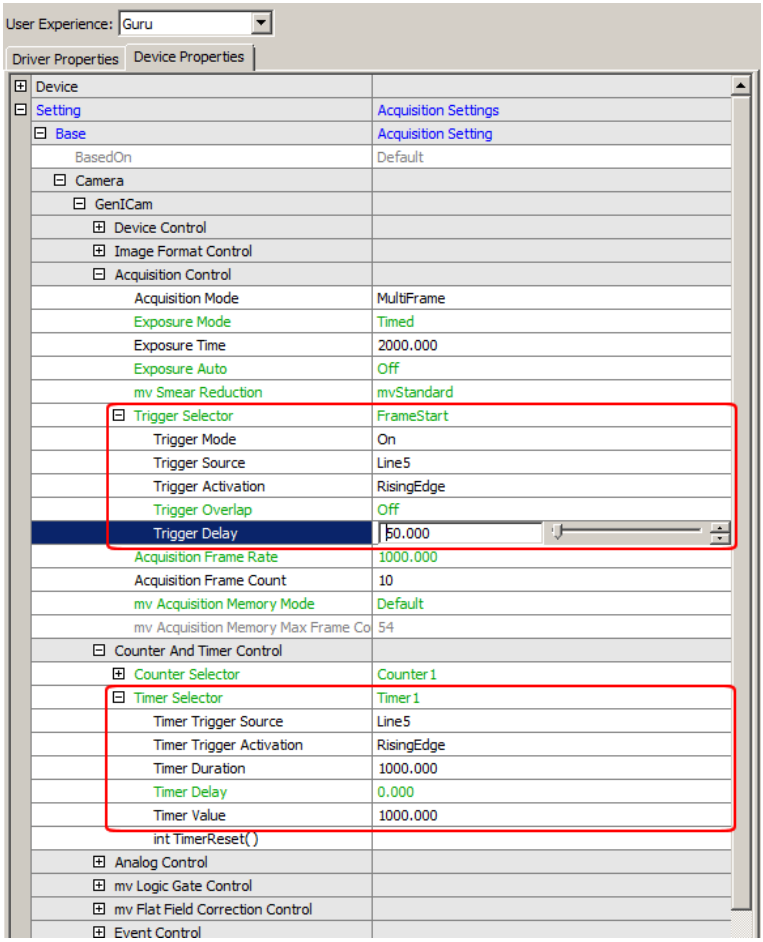


Figure 5: Working with **Timer** and **"Trigger Delay"**

1.18.5.2 Creating a debouncing filter at the inputs

In some cases, it is necessary to eliminate noise on trigger lines. This can become necessary when either

- the edges of a trigger signal are not perfect in terms of slope or
- if, because of the nature of the trigger signal source, multiple trigger edges are generated within a very short period of time even if there has just been a single trigger event.

The latter one is also called *bouncing*.

Bouncing is the tendency of any two metal contacts in an electronic device to generate multiple signals as the contacts close or open; now debouncing is any kind of hardware device or software that ensures that only a single signal will be acted upon for a single opening or closing of a contact.

To address problems that can arise from these kinds of trigger signals MATRIX VISION offers debouncing filters at the digital inputs of a device.

The debouncing filters can be found under *"Setting -> Base -> Camera -> GenICam -> Digital I/O Control -> LineSelector"* (red box in Figure 1) for each digital input:

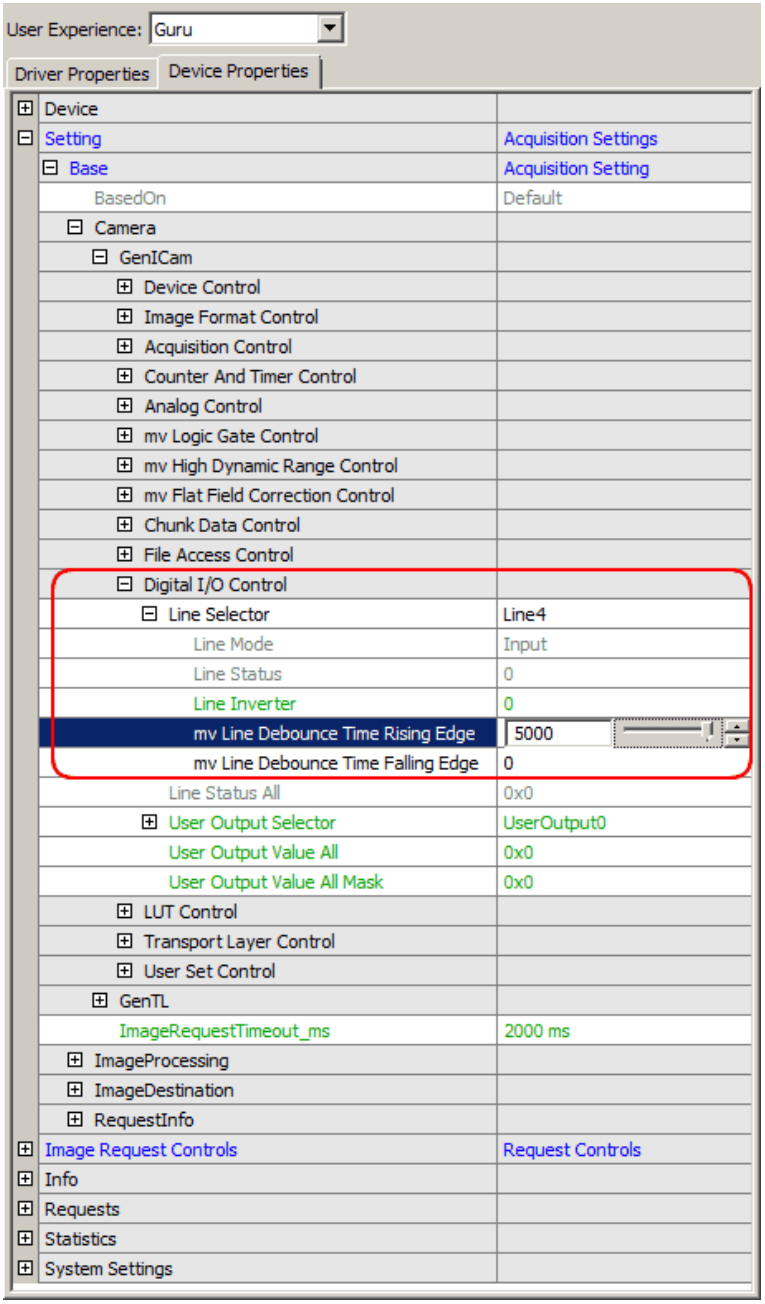


Figure 1: wxPropView - Configuring Digital Input Debounce Times

Each digital input (*LineMode* equals *Input*) that can be selected via the *LineSelector* property will offer its own property to configure the debouncing time for falling edge trigger signals (*"mv Line Debounce Time Falling Edge"*) and rising edge (*"mv Line Debounce Time Rising Edge"*) trigger signals.

The line debounce time can be configured in micro-seconds with a maximum value of up to 5000 micro-seconds. Internally each time an edge is detected at the corresponding digital input a timer will start (orange $*$ in figure 2 and 3), that is reset whenever the signal applied to the input falls below the threshold again. Only if the signal stays at a constant level for a full period of the defined *mvLineDebounceTime* the input signal will be considered as a valid trigger signal.

Note

Of course this mechanism delays the image acquisition by the debounce time.

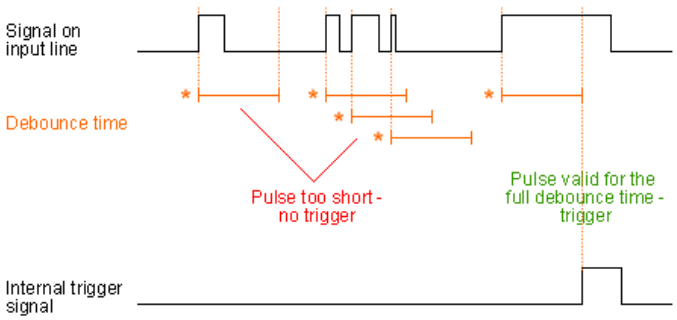


Figure 2: mvLineDebounceTimeRisingEdge Behaviour

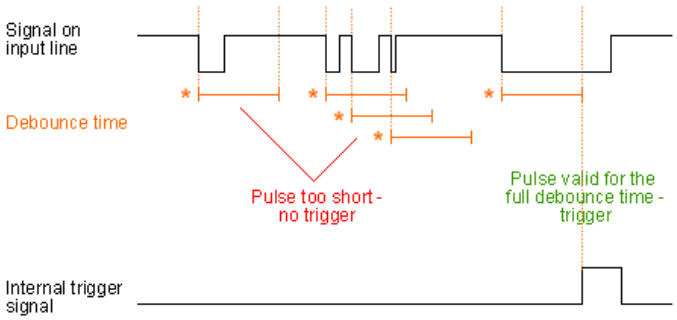


Figure 3: mvLineDebounceTimeFallingEdge Behaviour

1.18.6 Working with HDR (High Dynamic Range Control)

There are several use cases concerning High Dynamic Range Control:

- **Adjusting sensor of camera models -x02d (-1012d)** (p. 302)
- **Adjusting sensor of camera models -x02e (-1013) / -x04e (-1020)** (p. 303)
- **Adjusting sensor of camera models -1031C** (p. 306)

1.18.6.1 Adjusting sensor of camera models -x02d (-1012d)

1.18.6.1.1 Introduction

The HDR (High Dynamic Range) mode of the Aptina sensor increases the usable contrast range. This is achieved by dividing the integration time in three phases. The exposure time proportion of the three phases can be set independently.

1.18.6.1.2 Functionality

To exceed the typical dynamic range, images are captured at 3 exposure times with given ratios for different exposure times. The figure shows a multiple exposure capture using 3 different exposure times.

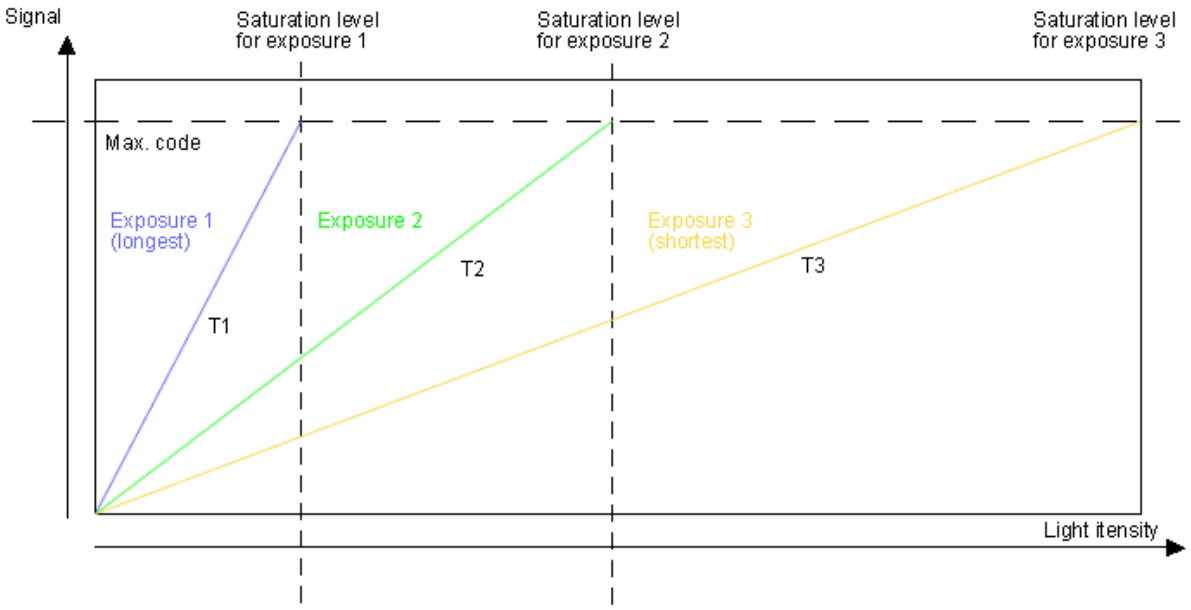


Figure 1: Multiple exposure capture using 3 different exposure times

Note

The longest exposure time (T1) represents the *Exposure_us* parameter you can set in wxPropView.

Afterwards, the signal is fully linearized before going through a compander to be output as a piece-wise linear signal. the next figure shows this.

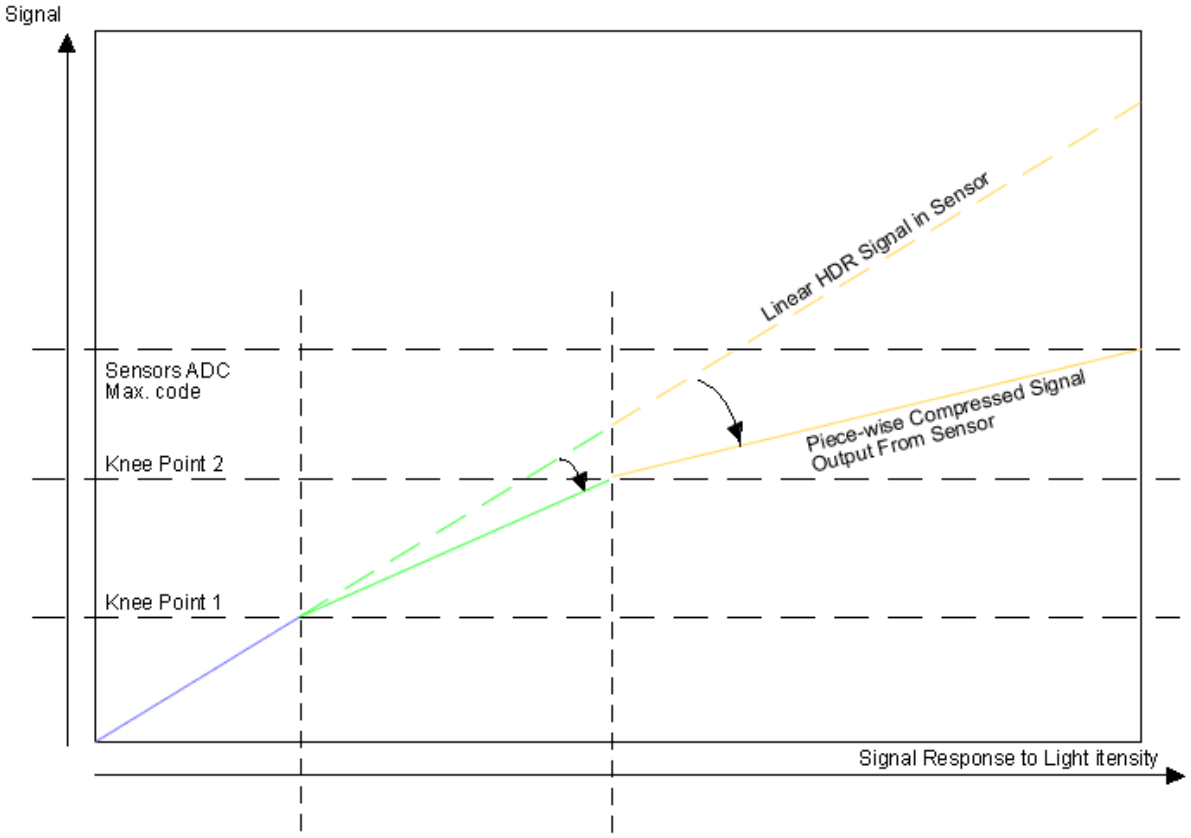


Figure 2: Piece-wise linear signal

1.18.6.1.2.1 Description

Exposure ratios can be controlled by the program. Two rations are used: R1 = T1/T2 and R2 = T2/T3.

Increasing R1 and R2 will increase the dynamic range of the sensor at the cost of lower signal-to-noise ratio (and vice versa).

1.18.6.2 Adjusting sensor of camera models -x02e (-1013) / -x04e (-1020)

1.18.6.2.1 Introduction

The HDR (High Dynamic Range) mode of the e2v sensors increases the usable contrast range. This is achieved by adjusting the logarithmic response of the pixels.

1.18.6.2.2 Functionality

MATRIX VISION offers the *"mv Linear Logarithmic Mode"* to use the HDR mode of the e2v sensors. With this mode you can set the low voltage of the reset signal at pixel level.

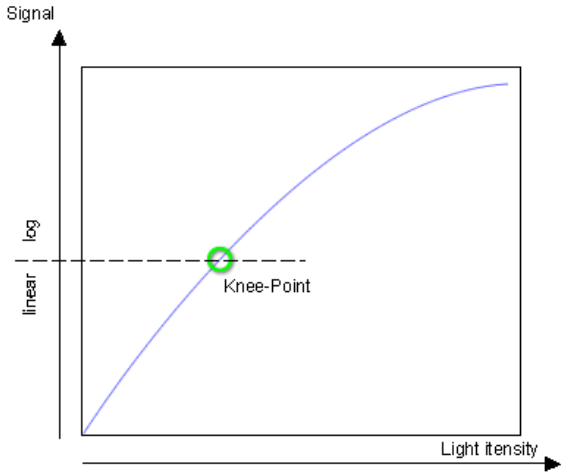


Figure 1: Knee-Point of the e2v HDR mode shifts the linear / logarithmic level

You can find the *"mv Linear Logarithmic Mode"* in "`Setting -> Base -> Camera -> GenICam -> Analog Control`":

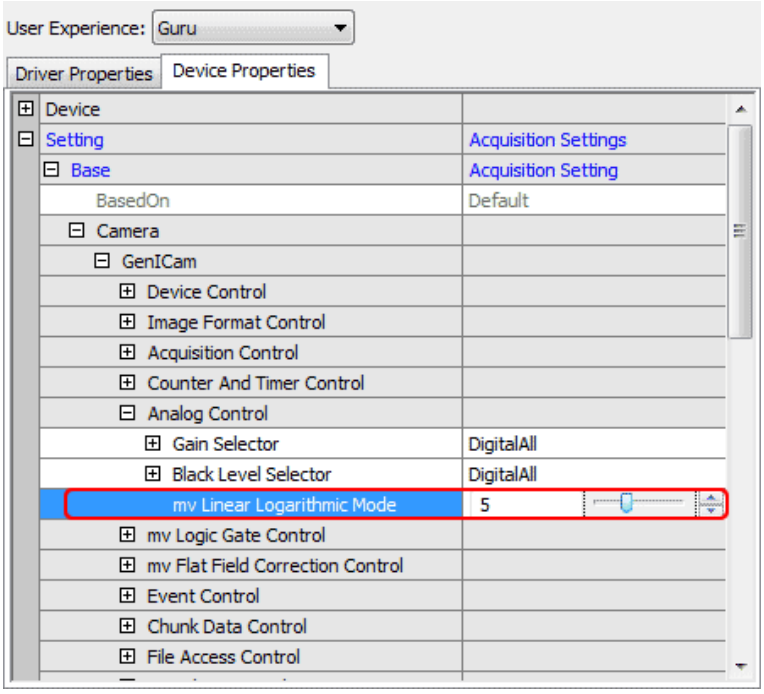


Figure 2: wxPropView - mv Linear Logarithmic Mode

The following figure shows the measured curves at 2 ms and 20 ms exposure time with four different *"mv Linear Logarithmic Mode"* settings:

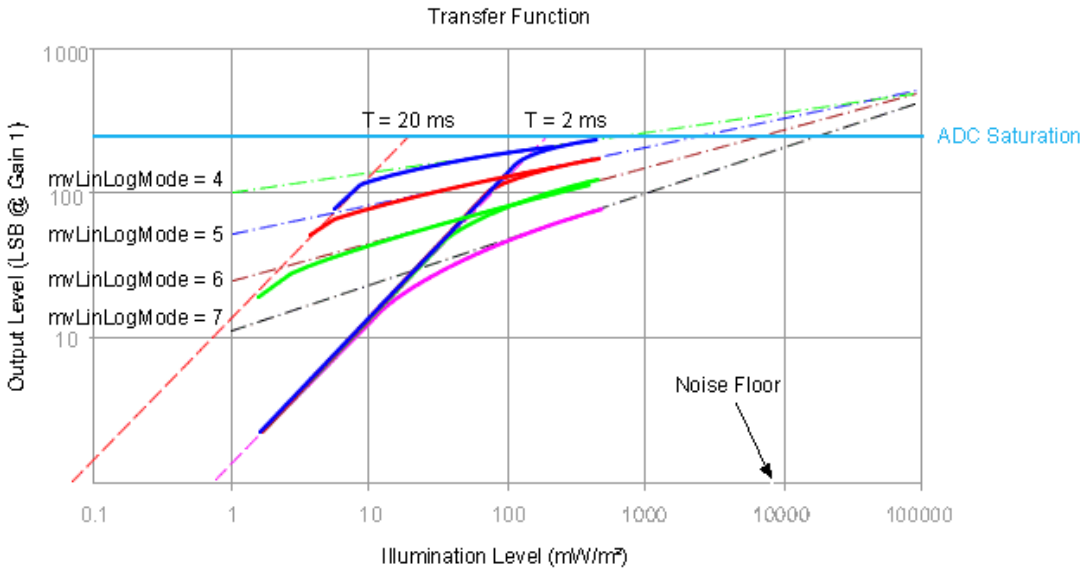


Figure 3: Measured curves

The curves are at Gain 1, lambda = 670 nm (40 nm width), room temperature, nominal power supply values, on a 100 x 100 pixel cantered area.

"mv Linear Logarithmic Mode" value	Dynamic max (dB)	
	T = 2 ms	T = 20 ms
4	47	65
5	74	93
6	85	104
7	92	111

1.18.6.3 Adjusting sensor of camera models -1031C

1.18.6.3.1 Introduction

The AR0331 Aptina sensor supports two dynamic modes:

- **linear dynamic mode** which is activated by default.
  This mode uses two interleaved rest-exposure pointers to create a 16 bit linear dynamic range. The first twelve bits are used for the long exposure (T1) while the remaining four bits are used for the short exposure (T2 = T1 / Ratio). Please have a look at **"Bit-shifting An Image"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual for more information about how you can shift through the bits with **wxPropView** (p. 99).
- **high dynamic mode** (HDR) which compresses the 16 bit value to 12 bits using Adaptive Local Tone Mapping (ALTM) or by companding to 12 or 14 bits (figure 1).

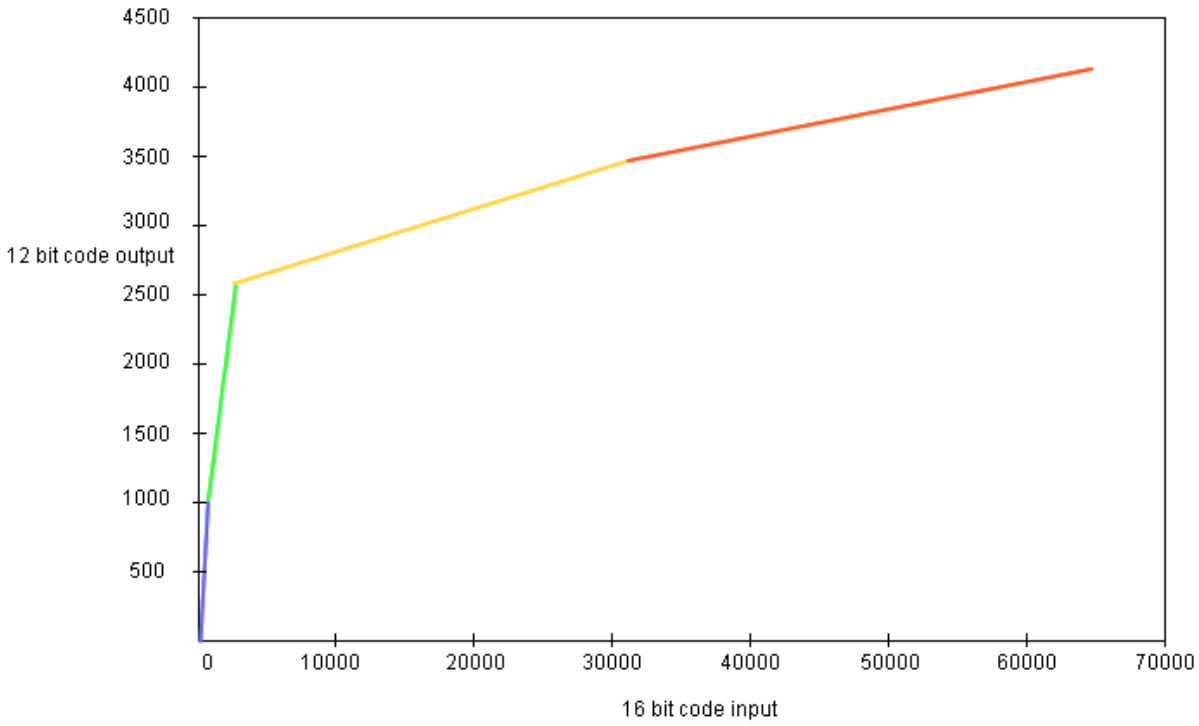


Figure 1: Compression from 16 to 12 bits

The HDR can be combined with

- image average for low noise images,
- the camera's auto exposure, and
- the camera's auto gain.

1.18.6.3.1.1 Adaptive Local Tone Mapping (ALTM)

The Adaptive Local Tone Mapping is used to compress the HDR image so that it can be nicely displayed on a low dynamic range display (i.e. LCD with a contrast ratio about 1000:1). The AR0331 does ALTM internally and fully automatic. The sensor also compensates both motion artifacts which occur because of the two exposures and noise artifacts which occur because of the clipping.

1.18.6.3.2 Enabling the HDR mode with wxPropView

Using the HDR mode, you have to do the following step:

1. Start **wxPropView** (p. 99) and
2. connect to the camera.
3. Then in *"Setting -> Base -> Camera -> GenICam -> mv High Dynamic Range Control"* you can enable the **"mv HDR Enable"**.

In this case

- the Adaptive Local Tone Mapping is on,
- the motion compensation is off (but it can be switched on via **"mv HDR Motion Compensation Enable"**),
- the adaptive color difference noise filtering is on, and
- you can set the exposure ratio:
  - **mvRatio4x** (∼ 84dB)
  - **mvRatio8x** (∼ 90dB)
  - **mvRatio16x** (∼ 96dB, recommended)
  - **mvRatio32x** (∼ 100dB)

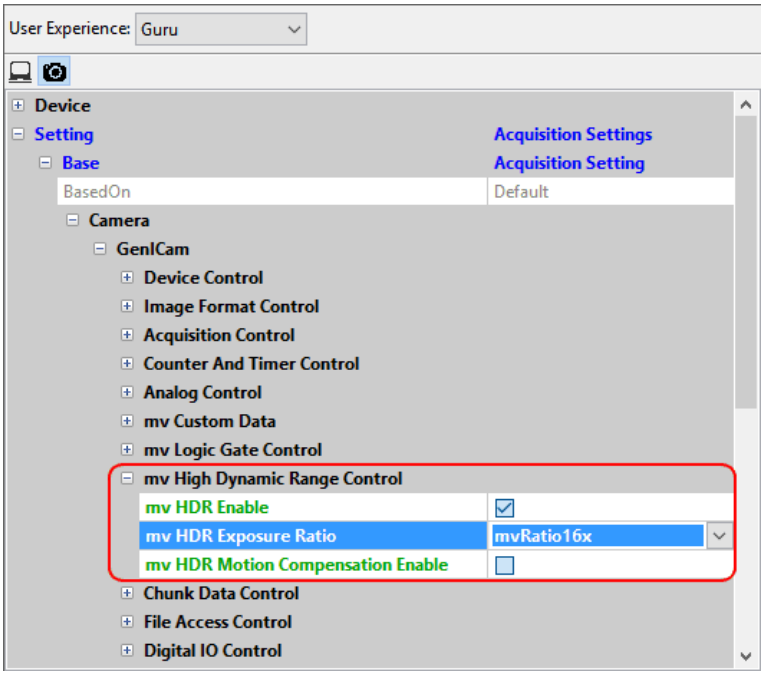


Figure 2: wxPropView - mv High Dynamic Range Control

1.18.7 Working with LUTs

There are several use cases concerning LUTs (Look-Up-Tables):

- **Introducing LUTs** (p. 308)
- **Working with LUTValueAll** (p. 312)
- **Implementing a hardware-based binarization** (p. 314)

1.18.7.1 Introducing LUTs

1.18.7.1.1 Introduction

Look-Up-Tables (LUT) are used to transform input data into a desirable output format. For example, if you want to invert an 8 bit image, a Look-Up-Table will look like the following:

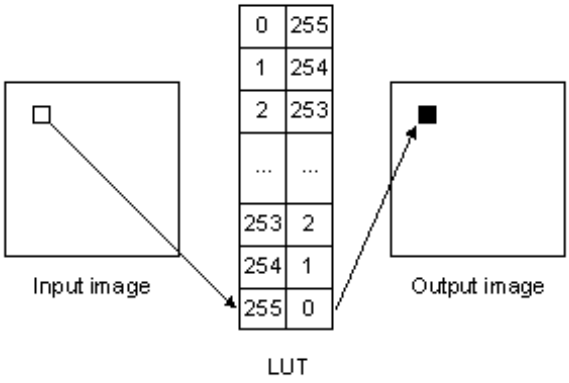


Figure 1: Look-Up-Table which inverts a pixel of an 8 bit mono image

I.e., a pixel which is white in the input image (value 255) will become black (value 0) in the output image.

All MATRIX VISION devices use a hardware based LUT which means that

- no host CPU load is needed and
- the LUT operations are independent of the transmission bit depth.

1.18.7.1.2 Setting the hardware based LUTs via LUT Control

On the mvBlueFOX3 using **wxPropView** (p. 99), you will find the **LUT Control** (p. 127) via *"Setting -> Base -> Camera -> GenICam -> LUT Control"*.

wxPropView (p. 99) offers a wizard for the **LUT Control** (p. 127) usage:

1. Click on *"Setting -> Base -> Camera -> GenICam -> LUT Control"*.
   Now, the *"Wizard"* button becomes active.

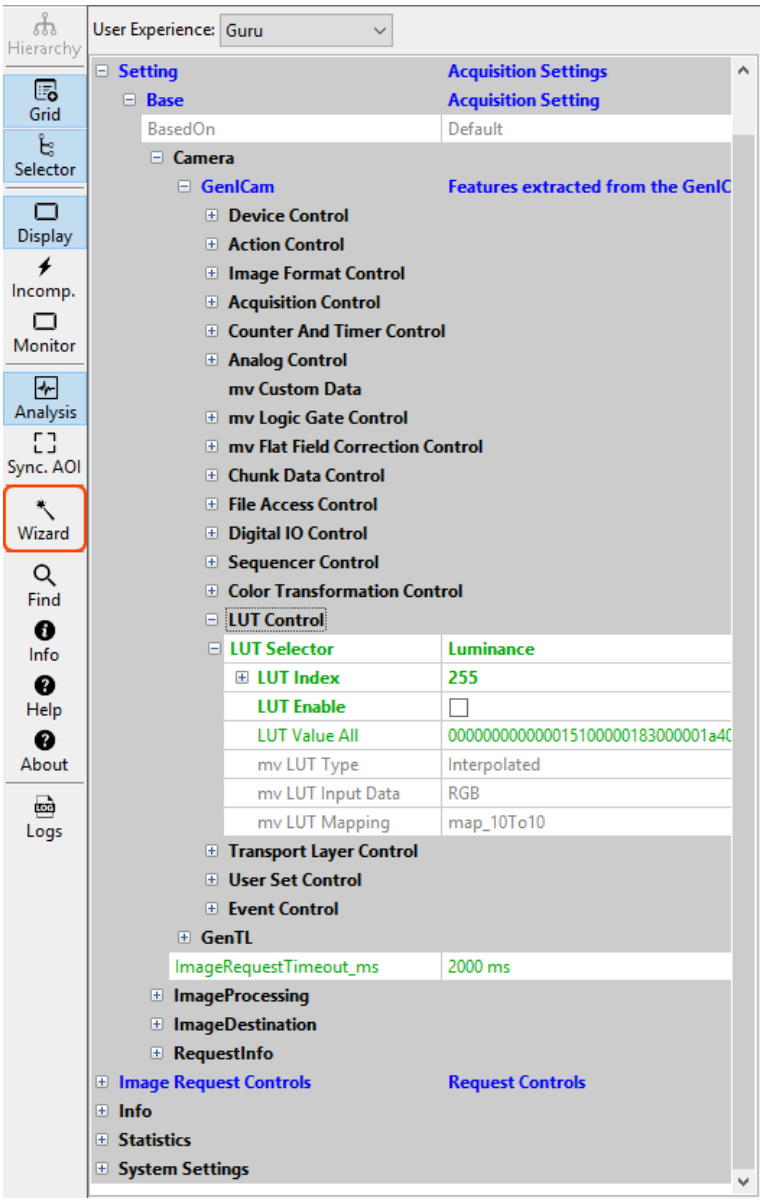


Figure 2: wxPropView - LUT Control wizard button

2. Click on the *"Wizard"* button to start the LUT Control wizard tool.
   The wizard will load the LUT data from the camera.

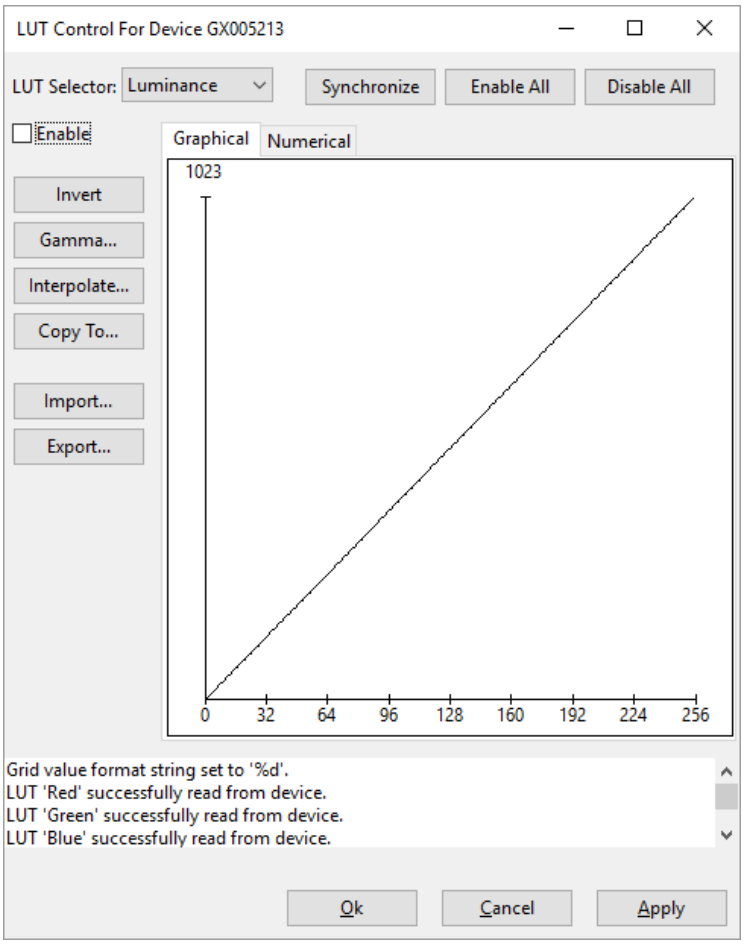


Figure 3: wxPropView - LUT Control wizard dialog

It is easy to change settings like the Gamma value of the Luminance or of each color channel (in combination with a color sensor of course) with the help of the wizard. You can also invert the values of each pixel with the wizard. It is not possible to set a LUT mode and the **"mv LUT Mapping"** is fixed.

Make your changes and do not forget to

1. click on **"Copy to..."** and select *"All"* or the color channel you need, to
2. click on **"Enable All"**, and finally, to
3. click on **Synchronize** and play the settings back to the device (via *"Cache -> Device"*).

Note

If you select *"Enable All"* without entering any value the image will be inverted.

1.18.7.1.3 Setting the Host based LUTs via LUTOperations

Host based LUTs are also available via *"Setting -> Base -> ImageProcessing -> LUTOperations"*. Here, the changes will affect the 8 bit image data and the processing needs the CPU of the host system.

Three *"LUTMode"s* are available:

- **"Gamma"**
  You can use "Gamma" to lift darker image areas and to flatten the brighter ones. This compensates the contrast of the object. `The calculation is described here`. It makes sense to set the *"GammaStartThreshold"* higher than 0 to avoid a too extreme lift or noise in the darker areas.

- **"Interpolated"**
  With "Interpolated" you can set the key points of a characteristic line. You can defined the number of key points. The following figure shows the behavior of all 3 LUTInterpolationModes with 3 key points:

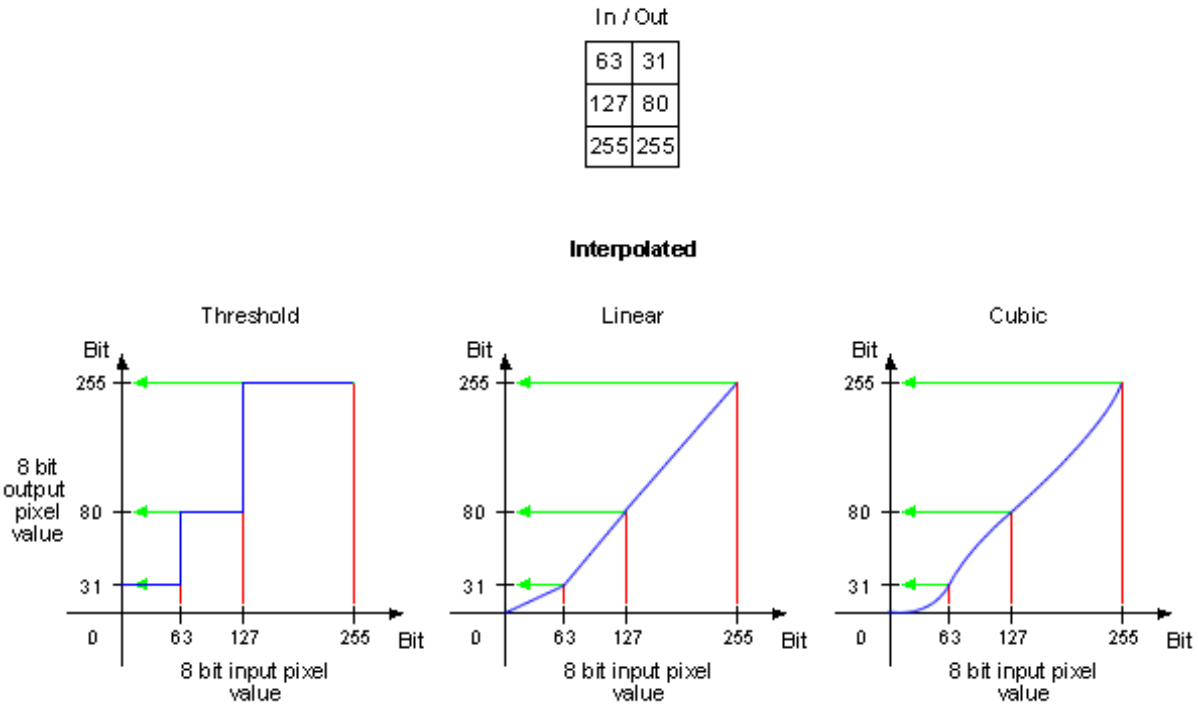


Figure 4: LUTMode "Interpolated" -> LUTInterpolationMode

- **"Direct"**
  With "Direct" you can set the LUT values directly.

1.18.7.1.3.1 Example 1: Inverting an Image

To get an inverted 8 bit mono image like shown in Figure 1, you can set the LUT using **wxPropView** (p. 99). After starting **wxPropView** (p. 99) and using the device,

1. Set *"LUTEnable"* to **"On"** in *"Setting -> Base -> ImageProcessing -> LUTOperations"*.
2. Afterwards, set *"LUTMode"* to **"Direct"**.
3. Right-click on *"LUTs -> LUT-0 -> DirectValues[256]"* and select **"Set Multiple Elements... -> Via A User Defined Value Range"**.
   This is one way to get an inverted result. It is also possible to use the *"LUTMode"* - *"Interpolated"*.
4. Now you can set the range from 0 to 255 and the values from 255 to 0 as shown in Figure 2.

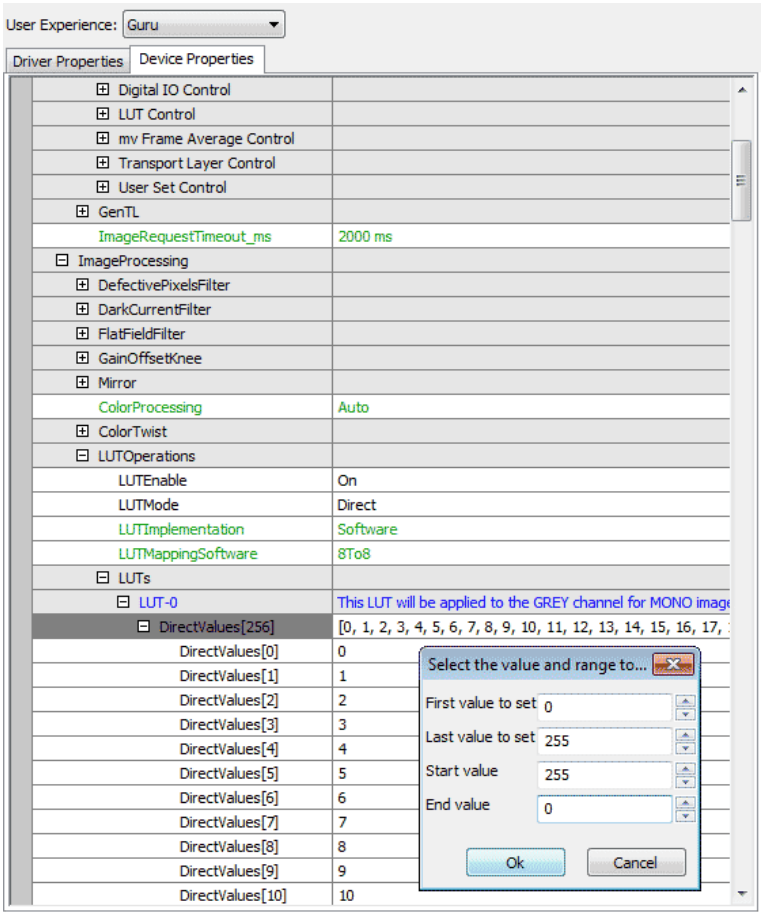


Figure 5: Inverting an image using wxPropView with LUTMode "Direct"

1.18.7.2 Working with LUTValueAll

Working with the **LUTValueAll** feature requires a detailed understanding on both Endianess and the cameras internal format for storing LUT data. **LUTValueAll** typically references the same section in the cameras memory as when accessing the LUT via the features **LUTIndex** and **LUTValue**.

LUT data can either be written to a device like this (C++ syntax):

```
const size_t LUT_VALUE_COUNT = 256;
int64_type LUTData[LUT_VALUE_COUNT] = getLUTDataToWriteToTheDevice();
mvIMPACT::acquire::GenICam::LUTControl lut(getDevicePointerFromSomewhere());
for(int64_type i=0; i< static_cast<int64_type>(LUT_VALUE_COUNT); i++ )
{
  lut.LUTIndex.write( i );
  lut.LUTValue.write( LUTData[i] );
}
```

When using this approach all the Endianess related issues will be handled completely by the **GenICam** (p. 165) runtime library. So this code is straight forward and easy to understand but might be slower than desired as it requires a lot of direct register accesses to the device.

In order to allow a fast efficient way to read/write LUT data from/to a device the **LUTValueAll** feature has been introduced. When using this feature the complete LUT can be written to a device like this:

```
const size_t LUT_VALUE_COUNT = 256;
int LUTData[LUT_VALUE_COUNT] = getLUTDataToWriteToTheDevice();
mvIMPACT::acquire::GenICam::LUTControl lut(getDevicePointerFromSomewhere());
std::string buf(reinterpret_cast<std::string::value_type*>(&LUTData), sizeof(LUTData));
lut.LUTValueAll.writeBinary( buf );
```

BUT as this simply writes a raw block of memory to the device it suddenly becomes important to know exactly how the LUT data is stored inside the camera. This includes:

- The size of one individual LUT entry (this could be anything from 1 up to 8 bytes)
- The Endianess of the device
- The Endianess of the host system used for sending/receiving the LUT data

The first item has impact on how the memory must be allocated for receiving/sending LUT data. For example when the LUT data on the device uses a 'one 32-bit integer per LUT entry with 256 entries' layout then of course this is needed on the host system:

```
const size_t LUT_VALUE_COUNT = 256;
int LUTData[LUT_VALUE_COUNT];
```

When the Endianess of the host system differs from the Endianess used by the device the application communicates with, then before sending data assembled on the host might require Endianess swapping. For the example from above this would e.g. require something like this:

```
#define SWAP_32(l) \
  ((((l) & 0xff000000) >> 24) | \
   (((l) & 0x00ff0000) >> 8)  | \
   (((l) & 0x0000ff00) << 8)  | \
   (((l) & 0x000000ff) << 24))

void fn()
{
 const size_t LUT_VALUE_COUNT = 256;
 int LUTData[LUT_VALUE_COUNT] = getLUTDataToWriteToTheDevice();
 mvIMPACT::acquire::GenICam::LUTControl lut(getDevicePointerFromSomewhere());
 for( size_t i=0; i<LUT_VALUE_COUNT; i++ )
 {
  LUTData[i] = SWAP_32(LUTData[i]);
 }
 std::string buf(reinterpret_cast<std::string::value_type*>(&LUTData), sizeof(LUTData));
 lut.LUTValueAll.writeBinary( buf );
}
```

For details on how the LUT memory is organized for certain sensors please refers to the **Sensor Overview** (p. 83). Please note that all **mvBlueCOUGAR-S**, **mvBlueCOUGAR-X** and **mvBlueCOUGAR-XD** devices are using **Big Endian** while almost any Windows or Linux distribution on the market uses **Little Endian**, thus the swapping of the data will most certainly be necessary when using the **LUTValueAll** feature.

1.18.7.3 Implementing a hardware-based binarization

If you like to have binarized images from the camera, you can use the **hardware-based Look-Up-Tables (LUT)** (p. 127) which you can access via *"Setting -> Base -> Camera -> GenICam -> LUT Control"*.

To get binarized images from the camera, please follow these steps:

1. Set up the camera and the scenery, e.g.



Figure 1: Scenery

2. Open the LUT wizard via the menu *"Wizards -> LUT Control..."*.
3. Export the current LUT as a "*.csv" file.

The "*.csv" file contains just one column for the output gray scale values. Each row of the "*.csv" represents the input gray scale value. In our example, the binarization threshold is 1024 in a 12-to-9 bit LUT. I.e., we have 4096 (= 12 bit) input values (= rows) and 512 (= 9 bit) output values (column values). To binarize the image according to the threshold, you have to

1. set all values below the binarization threshold to 0.
2. Set all values above the binarization threshold to 511:

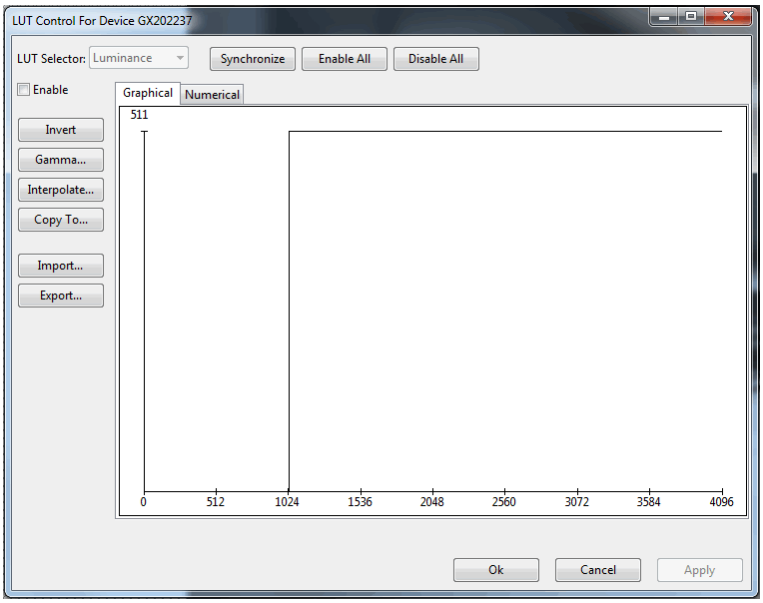


Figure 2: The binarization LUT

3. Now, save the "`*.csv`" file and
4. import it via the LUT Control wizard.
5. Click on synchronize and
6. finally check **"Enable"**.

Afterwards the camera will output binarized images like the following:



Figure 3: Binarized image

1.18.8 Saving data on the device

Note

As described in **"Storing, Restoring And Managing Settings"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual, it is also possible to save the settings as an XML file on the host system. You can find further information about for example the XML compatibilities of the different driver versions in the mvIMPACT Acquire SDK manuals and the according setting classes: `https://www.matrix-vision.com/manuals/SDK_CPP/classmvIMPACT_1_1acquire_1_1FunctionInterface.html` (C++)

- **Creating user data entries** (p. 316)
- **Creating user set entries** (p. 318)
- **Working with the UserFile section (Flash memory)** (p. 321)

1.18.8.1 Creating user data entries

1.18.8.1.1 Basics about user data

It is possible to save arbitrary user specific data on the hardware's non-volatile memory. The amount of possible entries depends on the length of the individual entries as well as the size of the devices non-volatile memory reserved for storing:

- mvBlueFOX,
- mvBlueFOX-M,
- mvBlueFOX-MLC,
- mvBlueFOX3,
- mvBlueCOUGAR-X,
- mvBlueCOUGAR-XD,
- mvBlueCOUGAR-XT and
- mvBlueNAOS

currently offer 512 bytes of user accessible non-volatile memory of which 12 bytes are needed to store header information leaving 500 bytes for user specific data.

```
One entry will currently consume:
1 + <length_of_name (up to 255 chars)> + 2 + <length_of_data (up to 65535 bytes)> + 1 (access mode) bytes

as well as an optional:
1 + <length_of_password> bytes per entry if a password has been defined for this particular entry
```

It is possible to save either **String** or **Binary** data in the *data* property of each entry. When storing binary data please note, that this data internally will be stored in `Base64` format thus the amount of memory required is 4/3 time the binary data size.

The UserData can be accessed and created using **wxPropView** (p. 99) (the device has to be closed). In the section `"UserData"` you will find the entries and following methods:

- `"CreateUserDataEntry"`

- "DeleteUserDataEntry"
- "WriteDataToHardware"

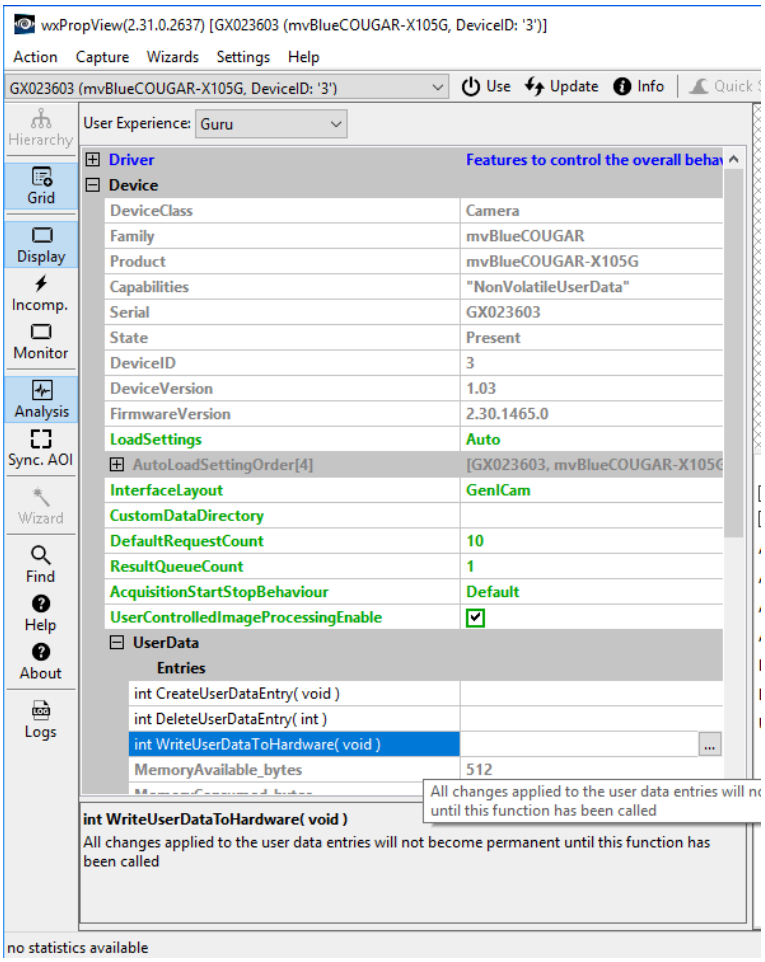


Figure 1: wxPropView - section "UserData -> Entries"

To create a user data entry, you have to

- Right click on "CreateUserDataEntry"
- Select **"Execute"** from the popup menu.
  An entry will be created.
- In "Entries" click on the entry you want to adjust and modify the data fields.
  To permanently commit a modification made with the keyboard the **ENTER** key **must be pressed**.
- To save the data on the device, you have to execute "WriteDataToHardware". Please have a look at the "Output" tab in the lower right section of the screen as shown in Figure 2, to see if the write process returned with no errors. If an error occurs a message box will pop up.

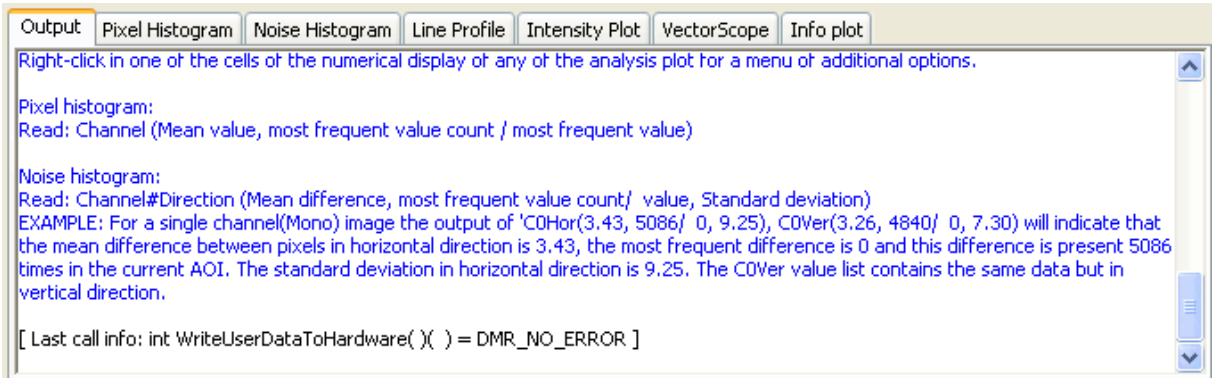


Figure 2: wxPropView - analysis tool "Output"

1.18.8.1.2 Coding sample

If you e.g. want to use the UserData as dongle mechanism (with binary data), it is not suggested to use **wxPropView** (p. 99). In this case you have to program the handling of the user data.

See also

mvIMPACT::acquire::UserDataEntry in mvIMPACT_Acquire_API_CPP_manual.chm.

1.18.8.2 Creating user set entries

With MATRIX VISION GenICam compliant devices it is possible to store up to five configuration sets (4 user plus one factory default) in the camera.

This feature is similar to the **storing settings functionality** (p. 316), which saves the settings in the registry. However, as mentioned before the user sets are stored in the camera.

The user set stores

- exposure,
- gain,
- AOI,
- frame rate,
- **LUT** (p. 127),
  - one **Flat-Field Correction** (p. 233),
- **defective pixels** (p. 229),
  - etc.

permanently and is independent of the computer which in used.

Additionally, you can select, which user set comes up after hard reset.

Note

The storage of user data in the registry can still override user set data!
User sets are cleared after firmware change.

1.18.8.2.1 List of ignored properties

Following properties are not stored in the user set:

- DeviceTLType
- DeviceUserID
- EventExposureEndData
- EventFrameEndData
- EventLine4AnyEdgeData
- EventLine4FallingEdgeData
- EventLine4RisingEdgeData
- EventLine5AnyEdgeData
- EventLine5FallingEdgeData
- EventLine5RisingEdgeData
- EventLine6AnyEdgeData
- EventLine6FallingEdgeData
- EventLine6RisingEdgeData
- EventLine7AnyEdgeData
- EventLine7FallingEdgeData
- EventLine7RisingEdgeData
- EventTestData
- FileAccessBuffer
- FileAccessLength
- FileAccessOffset
- FileOpenMode
- LUTIndex
- LUTValueAll
- mvADCGain
- mvCyclicSignalDelay
- mvCyclicSignalPeriod
- mvDefectivePixelCount
- mvDefectivePixelOffsetX
- mvDefectivePixelOffsetY
- mvDefectPixelSuppressionThreshold
- mvDefectPixelThreshold
- mvDeviceClockPLLPhaseShift
- mvDevicePowerMode
- mvDeviceStandbyTimeoutEnable
- mvDigitalGainOffset
- mvExposureActiveShiftEnable
- mvFFCAutoLoadMode
- mvI2cInterfaceASCIIBuffer
- mvI2cInterfaceBinaryBuffer
- mvI2cInterfaceBytesToRead
- mvI2cInterfaceBytesToWrite
- mvPreGain
- mvSerialInterfaceASCIIBuffer
- mvSerialInterfaceBinaryBuffer
- mvSerialInterfaceBytesToRead
- mvSerialInterfaceBytesToWrite
- mvTimestampPPSTriggerEdge
- mvTimestampResetValue
- mvUserData
- mvVRamp
- UserSetDefault

1.18.8.2.2 Working with the user sets

You can find the user set control in *"Setting -> Base -> Camera -> GenICam -> User Set Control"*:

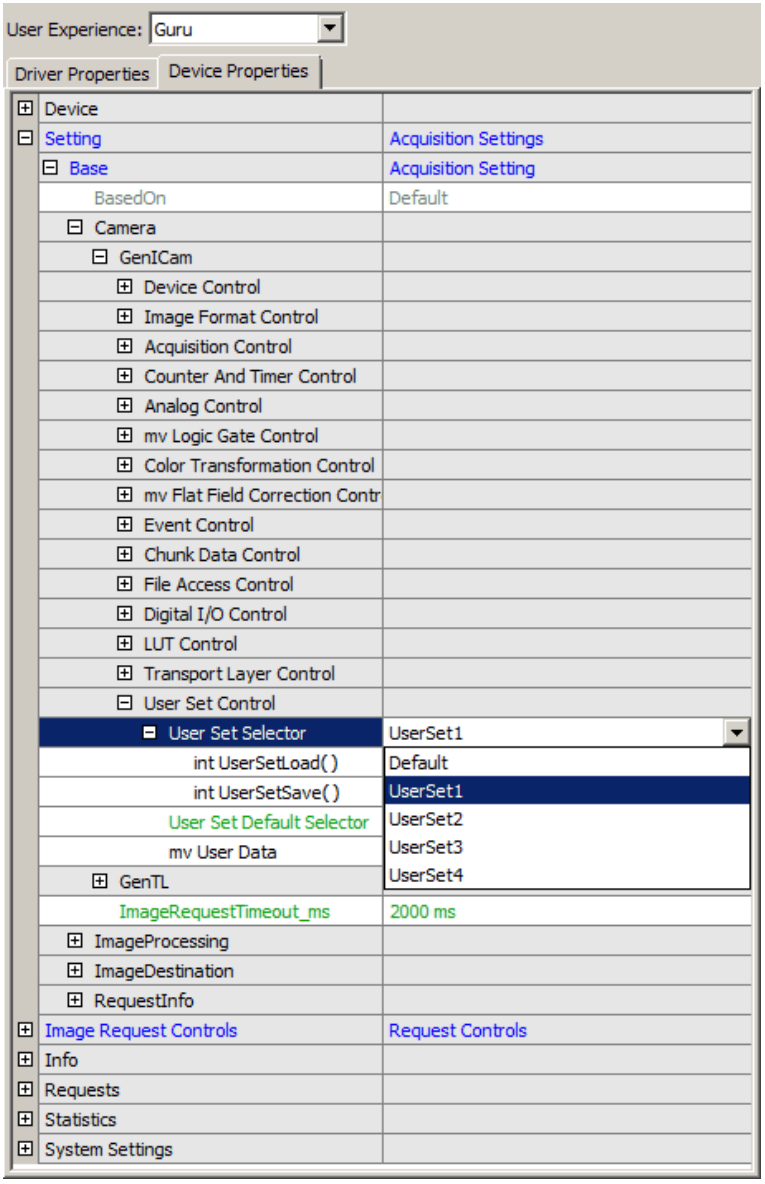


Figure 1: User Set Control

With *"User Set Selector"* you can select the user set (*"Default"*, *"UserSet1 - UserSet4"*). To save or load the specific user set, you have two functions:

- *"int UserSetLoad()"* and
- *"int UserSetSave()"*.

"User Set Default" is the property, where you can select the user set, which comes up after hard reset.

Finally, with *"mv User Data"* you have the possibility to store arbitrary user data.

1.18.8.3 Working with the UserFile section (Flash memory)

The mvBlueFOX3 offers a 64 KByte section in the Flash memory that can be used to upload a custom file to (**UserFile**).

To read or write this file you can use the following GenICam **File Access Control** (p. 114) and its interfaces:

- **IDevFileStream** (read)
- **ODevFileStream** (write)

Note

The **UserFile** is lost each time a firmware update is applied to the device.

1.18.8.3.1 Using wxPropView

wxPropView (p. 99) offers a wizard for the **File Access Control** (p. 114) usage:

1. Click on *"Setting -> Base -> Camera -> GenICam -> File Access Control -> File Selector -> File Operator Selector"*.
   Now, the *"Wizard"* button becomes active.

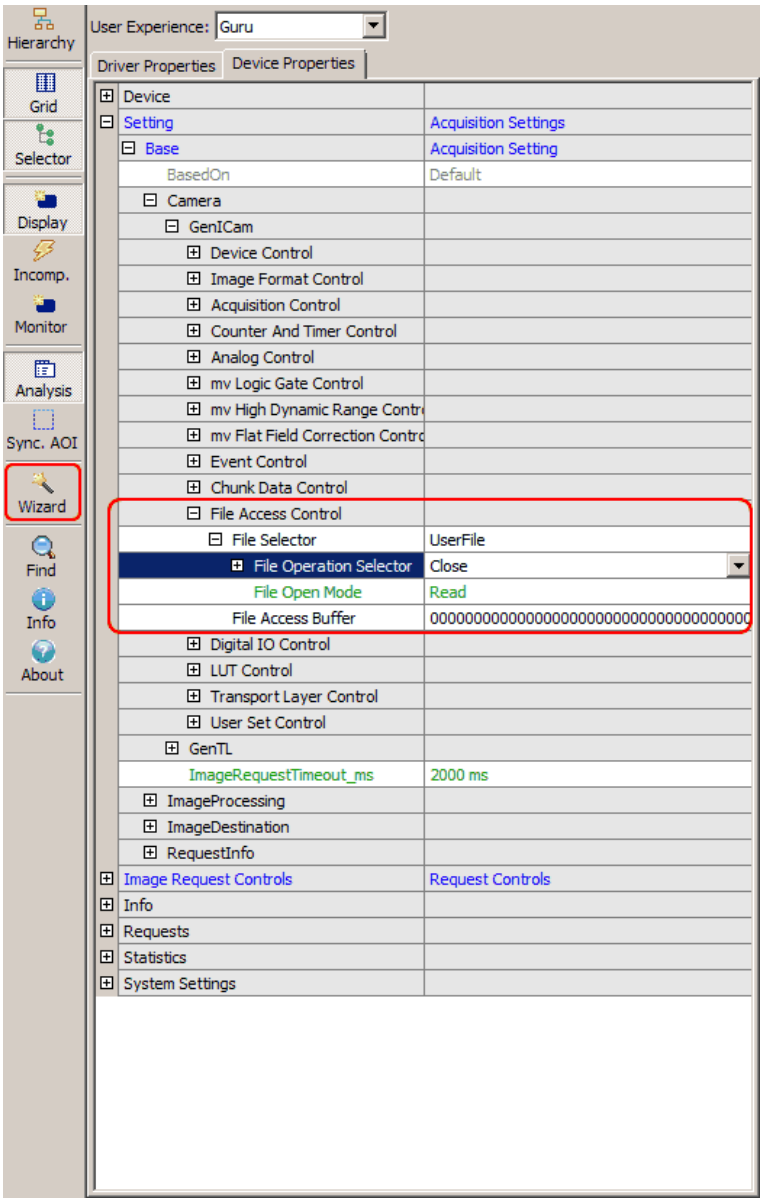


Figure 1: wxPropView - UserFile wizard

2. Click on the *"Wizard"* button.
   Now, a dialog appears where you can choose either to upload or download a file.

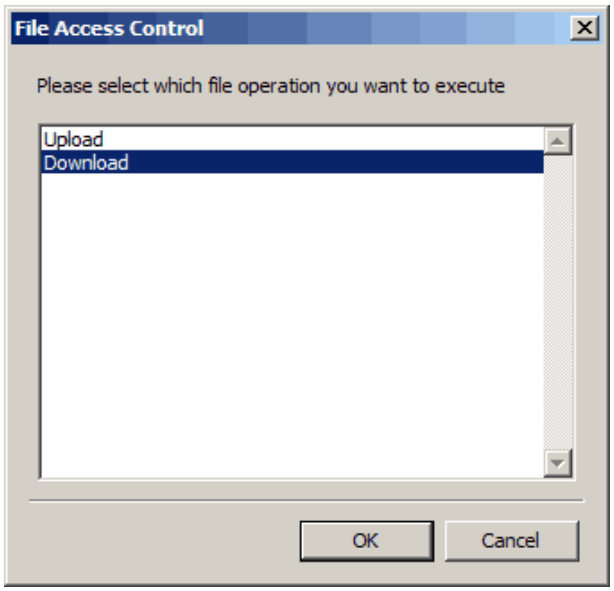


Figure 2: wxPropView - Download / Upload dialog

3. Make your choice and click on *"OK"*.
   Now, a dialog appears where you can select the File.

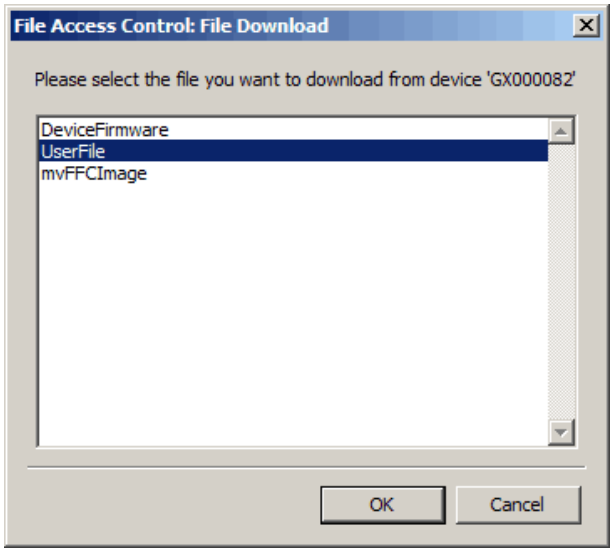


Figure 3: wxPropView - Download / Upload dialog

4. Select *"UserFile"* follow the instructions.

1.18.8.3.2 Manually control the file access from an application (C++)

The header providing the file access related classes must be included into the application:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam_FileStream.h>
```

A write access then will look like:

```
const string fileNameDevice("UserFile");

// uploading a file
mvIMPACT::acquire::GenICam::ODevFileStream file;
file.open( pDev, fileNameDevice.c_str() );

if( !file.fail() )
{
  // Handle the successful upload.
}
else
{
  // Handle the error.
}
```

A read access will look like:

```
const string fileNameDevice("UserFile");

// downloading a file works in a similar way
mvIMPACT::acquire::GenICam::IDevFileStream file;
file.open( pDev, fileNameDevice.c_str() );

if( !file.fail() )
{
  // Handle the successful upload.
}
else
{
  // Handle the error.
}
```

You can find a detailed code example in the C++ API manual in the documentation of the classes **mvIMPACT::acquire::GenICam::IDevFileStream** and **mvIMPACT::acquire::GenICam::ODevFileStream**

1.18.9 Working with device features

- **Reset timestamp by hardware** (p. 325)
- **Synchronizing camera timestamps** (p. 326)
- **Using the standby mode** (p. 330)
- **Working With The Serial Interface (mv Serial Interface Control)** (p. 333)
- **Working with the I2C interface (mv I2C Interface Control)** (p. 337)

1.18.9.1 Reset timestamp by hardware

This feature can be used

- for precise control of timestamp
  - for one camera or
  - to synchronize timestamp of multitude of cameras.

The latter sample, can be achieved by following steps:

1. Define the input line (*"TriggerSource"*) to reset the timestamp, e.g. **"Line5"** and
2. set the *"Trigger Selector"* to **"mvTimestampReset"**.
3. Connect all input lines of all cameras together.
4. Finally, use one output of one camera to generate reset edge:

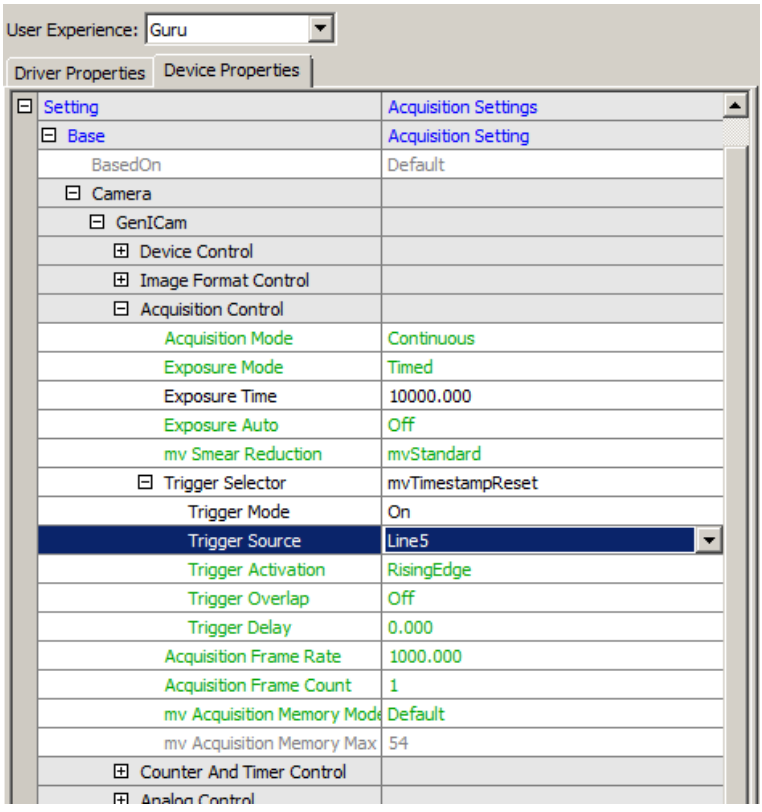


Figure 1: wxPropView - Setting the sample

Note

Be aware of the drift of the individual timestamps.

The timestamp is generated via FPGA in the camera which itself is clocked by a crystal oscillator. This is done independently in each camera and by default not synchronized among cameras or the host system.

Typical stability of crystal oscillators is in the range 100ppm (parts per million).

I.e. for longer operation times (say in excess of hours) there is a tendency that timestamps of individual cameras drift against each other and against the time in the operating system of the host.

Customers wishing to use the individual camera timestamps for synchronization and identification of images via timestamps for multi-camera systems will have in the meantime
- to reset all timestamps either by hardware signal or by command and regularly resynchronize or check the drift algorithmically
- in order to make sure that the drift is less half an image frame time.

1.18.9.2 Synchronizing camera timestamps

1.18.9.2.1 Introduction

Camera timestamps are a recommended **Genicam** (p. 165) / **SFNC** (p. 173) feature to add the information when an image was taken (exactly: when the exposure of the image started).

Without additional synchronization it is merely a camera individual timer with a vendor specific increment and implementation dependent accuracy. Each camera starts its own timestamp beginning with zero and there are no means to adjust or synchronize them among cameras or host PCs. MATRIX VISION cameras offer several ways of synchronizing:

- **mvTimestampReset** (p. 326)
- **Pulse-per-second signal (PPS)** (p. 328)

The usage is described below.

1.18.9.2.2 Without Precision Time Protocol (PTP) IEEE1588

There are many applications which do not or cannot profit from **"IEEE1588"** but have certain synchronization needs. Solutions for these scenarios are describes as follows.

1.18.9.2.3 Synchronizing using mvTimestampReset

First of all the standard does not provide hardware means to reset the timestamp in a camera other than plug off and on again. Therefore MATRIX VISION has created its own mechanism *mvTimestampReset* to reset the timestamp by a hardware input.

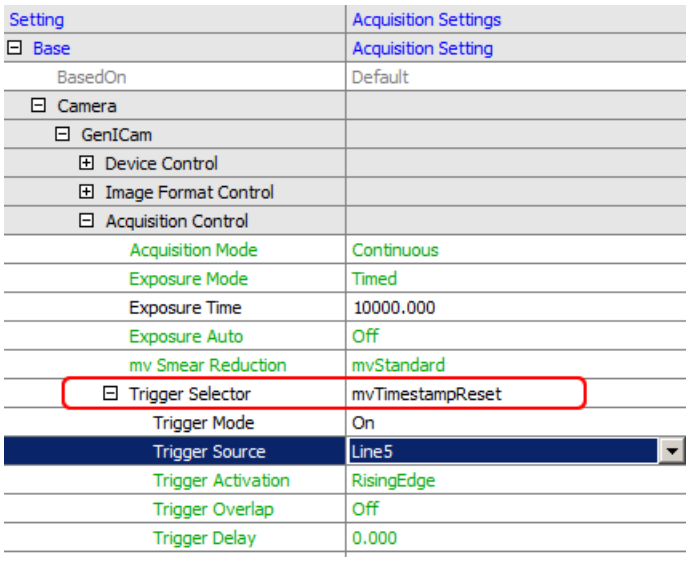


Figure 1: mvTimestampReset

This can be elegantly used for synchronization purposes by means of wiring an input of all cameras together and reset all camera timestamps at the beginning by a defined signal edge from the process. From this reset on all cameras start at zero local time and will increment independently their timestamp so that we achieve a basic accuracy only limited by drift of the clock main frequency (e.g. a 1 MHz oscillator in the FPGA) over time.

In order to compensate for this drift we can in addition reset the timestamp every second or minute or so and count the reset pulse itself by a counter in each camera. Assuming this reset pulse is generated by the master camera itself by means of a timer and output as the hardware reset signal for all cameras, we now can count the reset pulse with all cameras and put both the count and the reset timestamp as so called chunk data in the images.

We thus have achieved a synchronized timestamp with the precision of the master camera among all connected cameras.

Settings required are shown using MATRIX VISION's **wxPropView** (p. 99) tool:

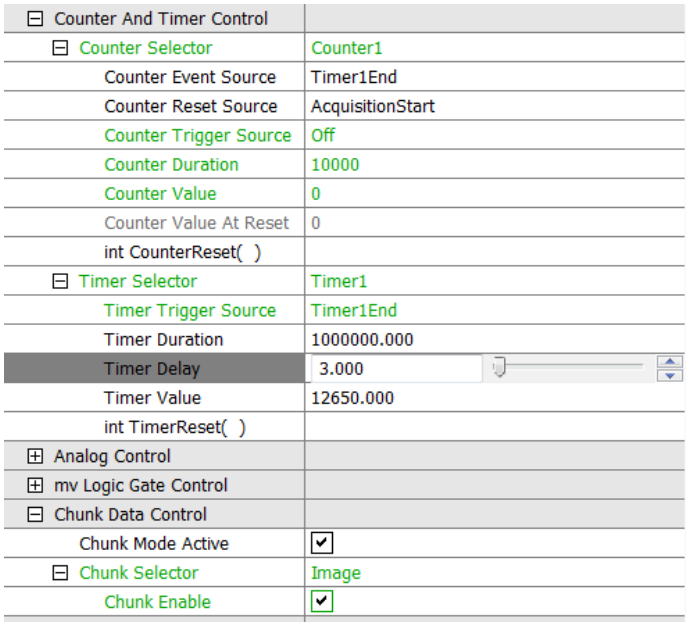


Figure 2: Reset the timestamp every second

An example of the chunk data attached to the image can be seen below. The timestamp is in µs and *Counter1* counts the reset pulses, in this case itself generated by the camera via *Timer1*.

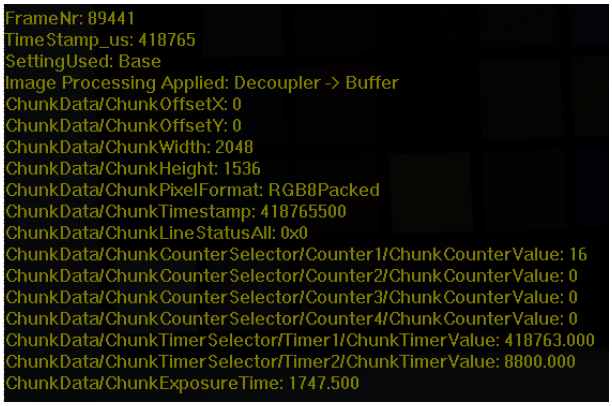


Figure 3: ChunkData

The task of resetting the counter at the beginning of the acquisition can be done by setting the reset property accordingly. Of course is this all independent whether the camera is acquiring images in triggered or continuous mode.

1.18.9.2.4 Synchronizing using a pulse-per-second signal

In order to eliminate the unknown drifts of different devices, a PPS (pulse per second) signal can be fed into each camera using a PC with NTP (network time protocol software), GPS devices or even a looped-back camera timer.

From these pulses the device can extract *how long one second is*. When a device detects that it is no longer running precisely it will adapt its internal clock leading to a "stabilized oscillator".

The device would then maintain their timestamp-differences over long times and stay synchronized. The initial difference between the timers - before the PPS was used - remains however. If you aim to eliminate that as well, you can use the `mvTimestampReset` up front with the same PPS input signal. In an application this can be configured like this (C# syntax):

```
// -----------------------------------------------------------------------
bool waitForNextPulse(Device pDev, String triggerLine)
// -----------------------------------------------------------------------
{
    GenICam.CounterAndTimerControl ctc = new GenICam.CounterAndTimerControl(pDev);
    ctc.counterEventSource.writeS(triggerLine);
    long momentaryValue = ctc.counterValue.read();
    for (int i=0; i<12; i++)
    {
        System.Threading.Thread.Sleep(100);
        if (momentaryValue != ctc.counterValue.read())
        {
            return true;
        }
    }
    return false;
}

// -----------------------------------------------------------------------
void SetupPPS(Device[] pDevs)
// -----------------------------------------------------------------------
{
    string TriggerLine = "Line4";

    if (!waitForNextPulse(pDevs[0],TriggerLine))
    {
```

```
        Console.WriteLine("No pulse seems to be present");
        return;
    }

    // No configure all the devices to reset their timestamp with each pulse coming
    // on the trigger line that the PPS signal is connected to.
    foreach(Device aDevice in pDevs)
    {
        GenICam.AcquisitionControl ac = new GenICam.AcquisitionControl(aDevice);
        ac.triggerSelector.writeS("mvTimestampReset");
        ac.triggerSource.writeS(TriggerLine);
        ac.triggerMode.writeS("On");
    }

    // wait for the next pulse that will then reset the timestamps of all the devices
    if (!waitForNextPulse(pDevs[0],TriggerLine))
    {
        Console.WriteLine("the pulses aren't coming fast enough ...");
        return;
    }

    // Now switch off the reset of the timestamp again. All devices did restart their
    // timestamp counters and will stay in sync using the PPS signal now
    foreach(Device aDevice in pDevs)
    {
        GenICam.AcquisitionControl ac = new GenICam.AcquisitionControl(aDevice);
        ac.triggerMode.writeS("Off");
        GenICam.DeviceControl dc = new GenICam.DeviceControl(aDevice);
        dc.mvTimestampPPSSync.writeS("Line4");
    }
}
```

1.18.9.2.4.1 Using a looped-back camera timer for the PPS signal

To reduce the amount of hardware needed you might want to sacrifice some timestamp validity and use one of the cameras as a master clock. This can be done like this:

- setting *Timer1* to duration 1s
- starting *Timer2* with every *Timer1End* and generating a short pulse (duration = 1000 us)
- placing *Timer2Active* on one of the digital I/O lines and using that as the source for PPS signal

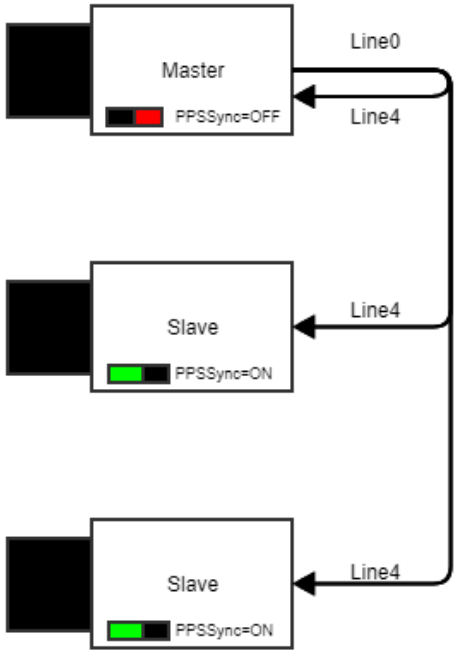


Figure 4: Setup for looped-back Timer

Setting the *Timer1* to 1 s seems like an easy task but due to some internal dependencies you should be carefully here. At the moment two different timer implementations are present in MATRIX VISION products.

- **Type 1:** For cameras with sensors other than the IMX family from Sony please set the duration to the theoretical value of 1000000.
- **Type 2:** For all the other cameras please use 999997 us duration since the self-triggering will consume the other 3 us
- Please refrain from switching on PPSSync inside the Master camera since (at least in Type 1 cameras) since this will lead to an instable feedback loop.

1.18.9.3 Using the standby mode

1.18.9.3.1 System requirements

- **"Firmware version"** at least **"1.6.188.0"**
- **"mvIMPACT Acquire driver version"** at least **"2.10.1"**

Using mvDeviceStandbyTimeout:

- **"Firmware version"** at least **"2.12.406.0"**
- **"mvIMPACT Acquire driver version"** at least **"2.17.1"**

1.18.9.3.2 Introduction

It is possible to switch the mvBlueFOX3 into a power down mode (standby) either

- by changing the property **"mvDevicePowerMode"** to **"mvStandby"** or
- by enabling the automatic power down mode by setting the property **"mvDeviceStandbyTimeoutEnable"** to **"bTrue"** and by specifying the timeout using the property **"mvDeviceStandbyTimeout"**.

The latter one will switch the camera to the power down mode as soon as no register was read (i.e. neither the camera is used nor images were acquired) during a specified time period (in seconds) specified by the property **"mvDeviceStandbyTimeout"**.

Note

As soon as the device stays open by an active driver instance, the driver will periodically read small chunks of data from the device to keep it open when auto standby is active. However if the application terminates or crashes the device will automatically move into standby mode after the specified timeout has elapsed.

The power down mode has following characteristics:

- The power consumption drops to 67 mA (**Standard model (mvBlueFOX3-2)** (p. 59) mvBlueFOX3-2xxx) / 77 mA (**Standard model (mvBlueFOX3-1)** (p. 58) mvBlueFOX3-1xxx) and
- the LED turns red.

If you change state again to power on, it will take about 7 seconds for the camera to wake up. During the wake up, all user settings will to be restored, except for the LUTs. Afterwards, the LED will turn green again.

1.18.9.3.3 Programming the power down mode

You can set the power down mode via the **Device Control** (p. 101) :

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...

    GenICam::DeviceControl device(pDev);
    device.mvDevicePowerMode.writeS( "mvStandby" );
```

Or switch to the power down mode automatically via **"mv Device Standby Timeout"**:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...

    GenICam::DeviceControl device(pDev);
    device.mvDeviceStandbyTimeoutEnable.write( bTrue );
    device.mvDeviceStandbyTimeout.write( 10 );
```

Note

If the power mode of the camera is set to **"mvStandby"** and the process which operates the camera stops for any reason the next time a process detects the camera it will automatically wake up again.

1.18.9.3.4 Changing the power down mode with wxPropView

Using the power down mode, you have to do the following step:

1. Start **wxPropView** (p. 99) and
2. connect to the camera.
3. Then in *"Setting -> Base -> Camera -> GenICam -> Device Control"* you can set the power mode **"mv Device Power Mode"** to **"mvActive"** or **"mvStandby"**.

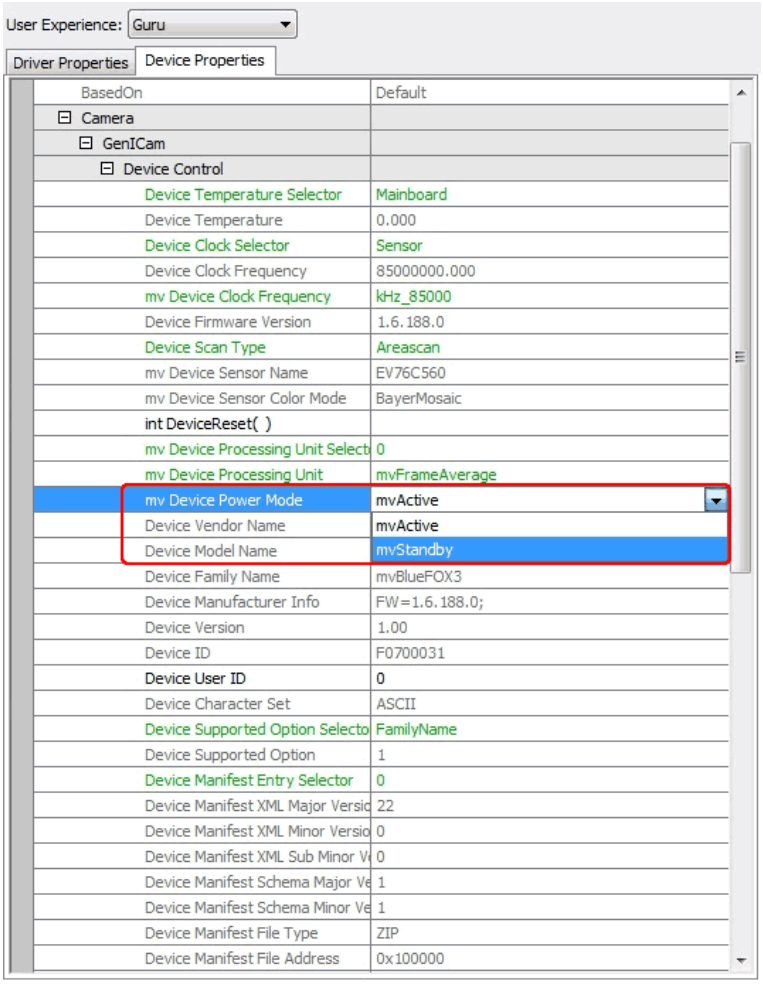


Figure 1: wxPropView: mvDevice Power Mode

Or switch to the power down mode automatically via **"mv Device Standby Timeout"**:

1. Start **wxPropView** (p. 99) and
2. connect to the camera.
3. Then in *"Setting -> Base -> Camera -> GenICam -> Device Control"* you can enable the standby timeout **"mv Device Standby Timeout Enable"** and
4. set the time in seconds via **"mv Device Standby Timeout"** after which the camera switches to standby if no register was read.

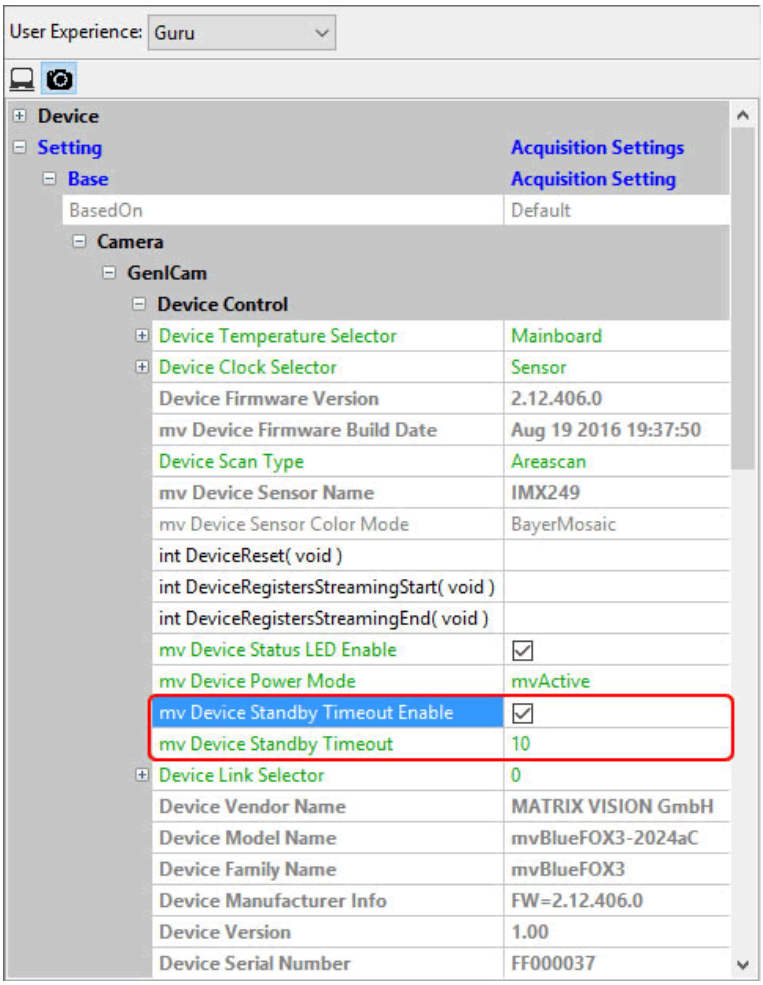


Figure 2: wxPropView: mv Device Standby Timeout Enable

1.18.9.4 Working With The Serial Interface (mv Serial Interface Control)

1.18.9.4.1 Introduction

As mentioned in **GenICam And Advanced Features** section of this manual, the **mv Serial Interface Control** (p. 125) is a feature which allows an easy integration of motor lenses or other peripherals based on RS232.

- Available message buffer size: 128 Bytes

> **Note**
>
> Use the Power GND for the RS232 signal.

1.18.9.4.2 Setting up the device

Follow these steps to prepare the camera for the communication via RS232:

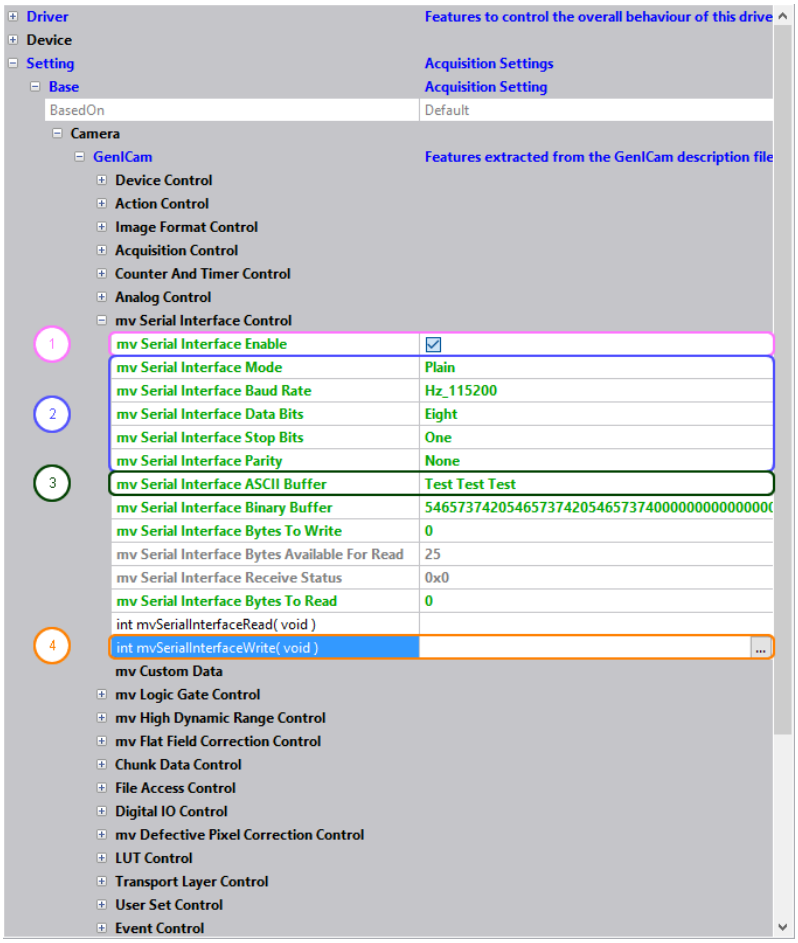


Figure 1: wxPropView - mv Serial Interface Control

1. Start **wxPropView** (p. 99)
2. Connect to the camera
3. Under *"Setting -> Base -> Camera -> GenICam -> mv Serial Interface Control"* activate the serial interface by enabling
   "mv Serial Interface Enable" (1).
   Afterwards **"mv Serial Interface Control"** is available.
4. Set up the connection settings to your needs (2).
5. To test the settings you can send a test message (3).
6. Send messages by executing the function *"int mvSerialInterfaceWrite(void)"* by either clicking on the 3 dots next to the function name or by right-clicking on the command and then selecting *Execute* from the context menu.(4)

Note

Please enter a hexadecimal value in **mvSerialInterfaceBinaryBuffer**.

If you are listening to the RS232 serial line using a tool like *PuTTY* with matching settings...

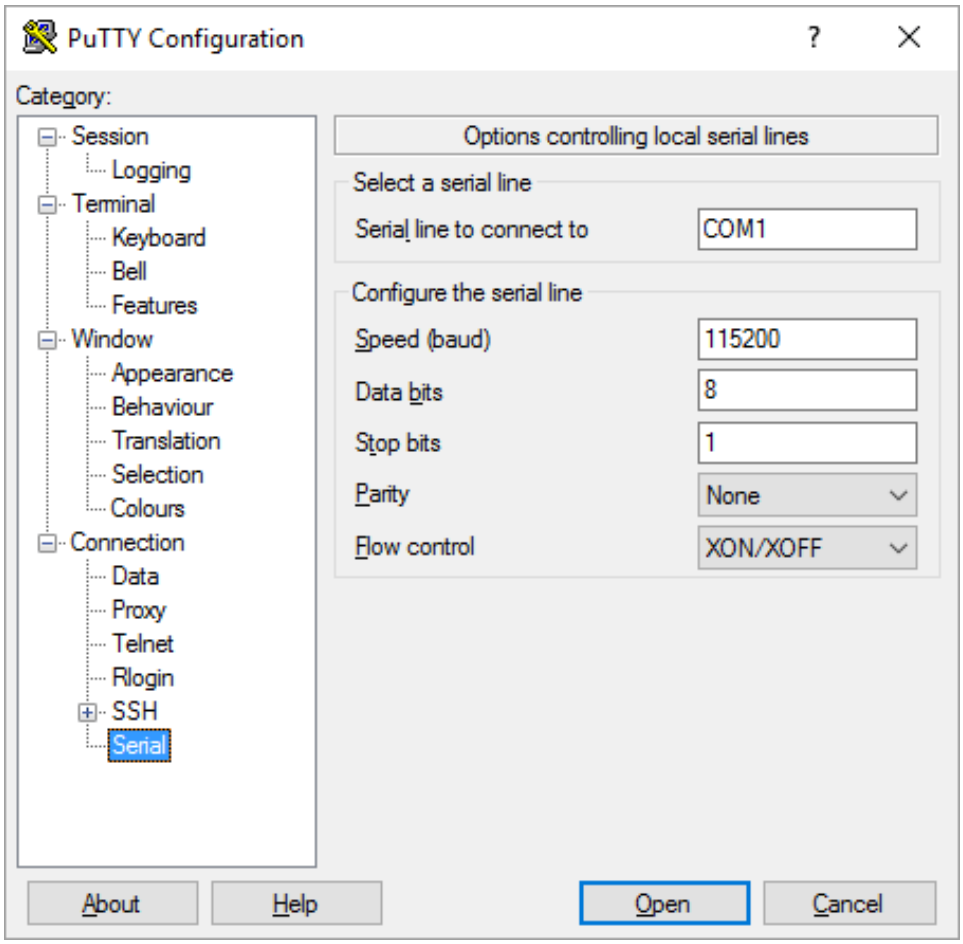


Figure 2: PuTTY - Setting up the serial interface

you will see the test message:



Figure 3: PuTTY - Receiving the test message

1.18.9.4.3 Programming the serial interface

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
  GenICam::mvSerialInterfaceControl sic( pDev );
  sic.mvSerialInterfaceBaudRate.writeS( "Hz_115200" );
  sic.mvSerialInterfaceASCIIBuffer.writeS( "Test Test Test" );
  sic.mvSerialInterfaceWrite();
  // more code
```

1.18.9.5 Working with the I2C interface (mv I2C Interface Control)

1.18.9.5.1 Introduction As mentioned in **GenICam And Advanced Features** section of this manual, the **mv I2C Interface Control** (p. 125) is a feature which allows to communicate with custom-specific peripherals via I2C.

1.18.9.5.2 Setting up the device Follow these steps to prepare the camera for the communication via I2C:

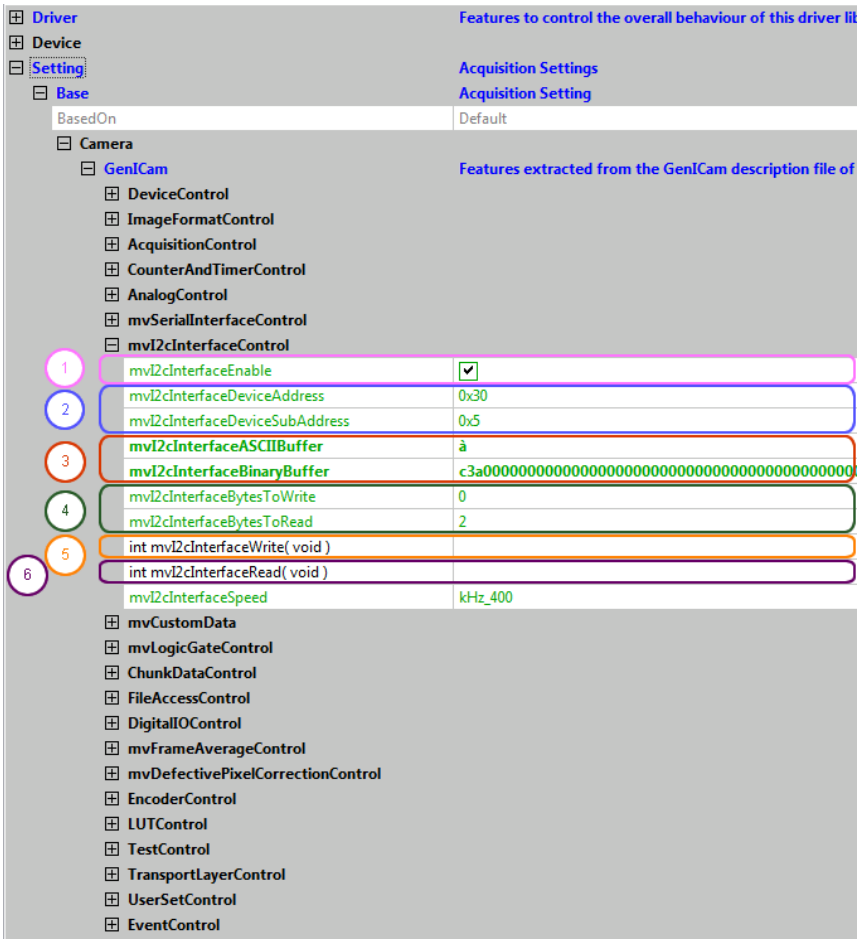


Figure 1: wxPropView - mv I2C Interface Control

1. Start **wxPropView** (p. 99)
2. Connect to the camera
3. Under *"Setting -> Base -> Camera -> GenICam -> mv I2c Interface Control"* activate the serial interface by enabling
   "mv I2C Interface Enable" (1).
   Afterwards **"mv I2C Interface Control"** is available.
4. Set up the connection settings to your needs (2).
   E.g. to get the temperature of the sensor set **"mv I2C Interface Device Address"**, depending on the device, either to **"0x30"** or **"0x32"**.
   Afterwards there are two ways to set the resolution of the temperature value (0.5 °C [0], 0.25 °C [1], 0.125 °C [2], or 0.0625 °C [3]):

   Using the **BinaryBuffer:**

(a) Set **"mv I2C Interface Device Sub Address"** to **"0x08"**.

(b) Set **"mv I2C Interface Binary Buffer"** e.g. to **"1"** i.e. 0.25℃ (3).

(c) Set **"mv I2C Interface Bytes To Write"** to **"1"** to send one byte from the binary buffer (4).

(d) Send messages by executing the function *"int mvI2CInterfaceWrite(void)"* by either clicking on the 3 dots next to the function name or by right-clicking on the command and then selecting *Execute* from the context menu.(5)

Without the **BinaryBuffer** (which is faster):

(a) Set **"mv I2C Interface Device Sub Address"** to **"0x10801"** i.e. 0x1xxxx means 16Bit SubAddress; message: register 8 to 1 (resolution 0.25℃).

(b) Disable the **"mv I2C Interface Bytes To Write"** using **"0"** (4).

(c) Send messages by executing the function *"int mvI2CInterfaceWrite(void)"* by either clicking on the 3 dots next to the function name or by right-clicking on the command and then selecting *Execute* from the context menu.(5)

You can now read the temperature the following way:

1. Set **"mv I2C Interface Device Sub Address"** to **"0x05"**.
2. Set **"mv I2C Interface Bytes To Read"** to **"0x02"**, i.e. two bytes.
3. Send messages by executing the function *"int mvI2CInterfaceRead(void)"* by either clicking on the 3 dots next to the function name or by right-clicking on the command and then selecting *Execute* from the context menu.(6)

1.18.9.5.3 Programming the I2C interface

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code

  GenICam::mvI2cInterfaceControl iic( pDev );
  iic.mvI2cInterfaceEnable.write( bTrue );
  iic.mvI2cInterfaceSpeed.writeS("kHz_400");
  iic.mvI2cInterfaceDeviceAddress.write("0x30");

  // Set the I2C communication

  // Using the BinaryBuffer
  iic.mvI2cInterfaceDeviceSubAddress.write(0x08);
  pWrBuf[0] = 1;
  iic.mvI2cInterfaceBinaryBuffer.writeBinary(pWrBuf);
  iic.mvI2cInterfaceBytesToWrite.write("1");
  iic.mvI2cInterfaceWrite.call();

  // Without BinaryBuffer
  iic.mvI2cInterfaceDeviceSubAddress.write("0x10800");
  iic.mvI2cInterfaceBytesToWrite.write("0");
  iic.mvI2cInterfaceWrite.call();

  // Read the temperature

  iic.mvI2cInterfaceDeviceSubAddress.write("0x05");
  iic.mvI2cInterfaceBytesToRead.write("0x02");
  iic.mvI2cInterfaceRead.call();

  i2cReadBinaryData = iic.mvI2cInterfaceBinaryBuffer.readBinary();

  // more code
```

See also

GenICamI2cUsage.cs sample in the sample folder of the mvIMPACT Acquire SDK installation.

1.18.10 Working with several cameras simultaneously

- **Creating synchronized acquisitions using timers** (p. 339)

1.18.10.1 Creating synchronized acquisitions using timers

1.18.10.1.1 Basics

Getting images from several cameras exactly at the same time is a major task in

- 3D image acquisitions
  (the images must be acquired at the same time using two cameras) or
- acquisitions of larger objects
  (if more than one camera is required to span over the complete image, like in the textile and printing industry).

To solve this task MATRIX VISION devices offer timers that can be used to generate pulse at regular intervals. This pulse can be connected to a digital output. The digital output can be connected digital to the digital input of one or more cameras to use it as a trigger.

1.18.10.1.2 Connecting the hardware

1.18.10.1.2.1 On the master camera

1. Connect power supply GND to **"pin 1"** and **"pin 7"**.
2. Connect power supply +12 V to **"pin 2"** and **"pin 10 (power supply for the outputs)"**.
3. Connect DigOut0 of master (**"pin 6"**) to DigIn0 of master (**"pin 4"**).

1.18.10.1.2.2 On each slave camera

1. Connect power supply GND to **"pin 1"** and **"pin 7"**.
2. Connect power supply +12 V to **"pin 2"** and **"pin 10 (power supply for the outputs)"**.

1.18.10.1.2.3 Between the cameras

1. Connect DigOut0 of master (**"pin 6"**) to DigIn0 of slave (**"pin 4"**).

Note

If more than one slave is used, connect the same **"pin 6"** of master to all **"pin 4"** of the slaves.

If each camera has its own power supply, then connect all grounds (GND) together.

For the master camera, there are 2 possibilities how it is triggered:

1. The master camera triggers itself logically (so called **"Master - Slave"**, see Figure 1), or
2. the master camera uses the external trigger signal, which was created by itself, via digital input (so called **"Slave - Slave"**, see Figure 2).

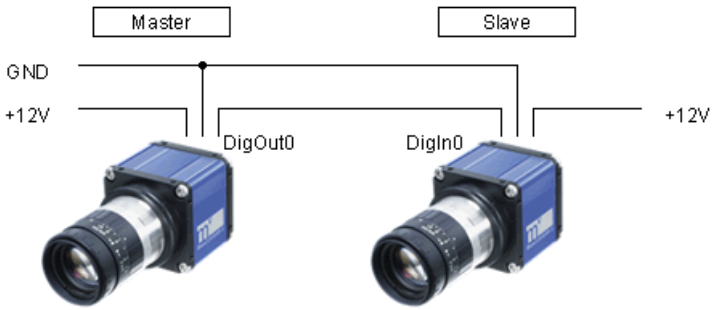


Figure 1: Master - Slave connecting

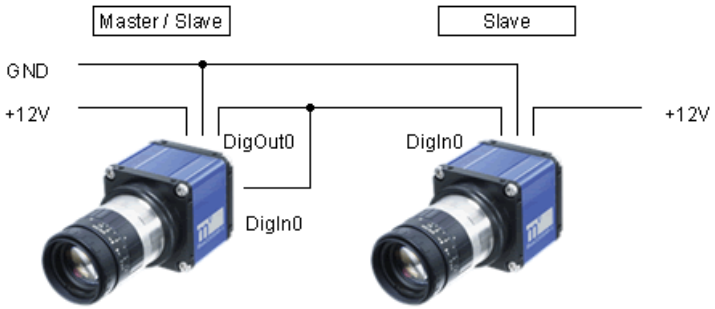


Figure 2: Slave - Slave connecting

Note

With **"Master - Slave"** and according to the delay of the opto-isolated inputs of the slave cameras, you have to adapted the property *"Trigger delay"* of the master camera to synchronize the cameras exactly.

1.18.10.1.3 Programming the acquisition

You will need two timers and you have to set a trigger.

1.18.10.1.3.1 Start timer

Two timers are used for the **"start timer"**. *Timer1* defines the interval between two triggers. *Timer2* generates the trigger pulse at the end of *Timer1*.

The following sample shows a trigger

- which is generated every second and
- the pulse width is 10 ms:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...

// Master: Set timers to trig image: Start after queue is filled
    GenICam::CounterAndTimerControl catcMaster(pDev);
    catcMaster.timerSelector.writeS( "Timer1" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 1000000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );

    catcMaster.timerSelector.writeS( "Timer2" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 10000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );
```

See also

Counter And Timer Control (p. 107)

Note

Make sure the *Timer1* interval must be larger than the processing time. Otherwise, the images are lost.

The timers are defined, now you have to do following steps:

1. Set the digital output, e.g. **"Line 0"**,
2. connect the digital output with the inputs of the slave cameras, and finally
3. set the trigger source to the digital input, e.g. **"Line4"**.

1.18.10.1.3.2 Set digital I/O

In this step, the signal has to be connected to the digital output, e.g. **"Line0"**:

```
// Set Digital I/O
    GenICam::DigitalIOControl io(pDev);
    io.lineSelector.writeS( "Line0" );
    io.lineSource.writeS( "Timer2Active" );
```

See also

Digital I/O Control (p. 114)

This signal has to be connected with the digital inputs of the slave cameras as shown in Figure 1 and 2.

1.18.10.1.3.3 Set trigger

"If you want to use Master - Slave":

```
// Set Trigger of Master camera
    GenICam::AcquisitionControl ac(pDev);
    ac.triggerSelector.writeS( "FrameStart" );
    ac.triggerMode.writeS( "On" );
    ac.triggerSource.writeS( "Timer1Start" );
// or ac.triggerSource.writeS( "Timer1End" );


// Set Trigger of Slave camera
    GenICam::AcquisitionControl ac(pDev);
    ac.triggerSelector.writeS( "FrameStart" );
    ac.triggerMode.writeS( "On" );
    ac.triggerSource.writeS( "Line4" );
    ac.triggerActivation.writeS( "RisingEdge" );
```

See also

Acquisition Control (p. 103)

Now, the two timers will work like the following figure illustrates, which means

- *Timer1* is the trigger event and
- *Timer2* the trigger pulse width:

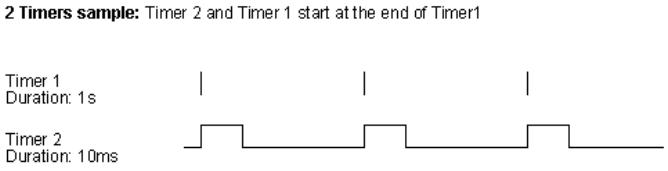


Figure 2: Timers

By the way, this is a simple **"pulse width modulation (PWM)"** example.

1.18.10.1.4 Setting the synchronized acquisition using wxPropView

The following figures show, how you can set the timers and trigger using the GUI tool **wxPropView** (p. 99)

1. Setting of *Timer1* (blue box) on the master camera:

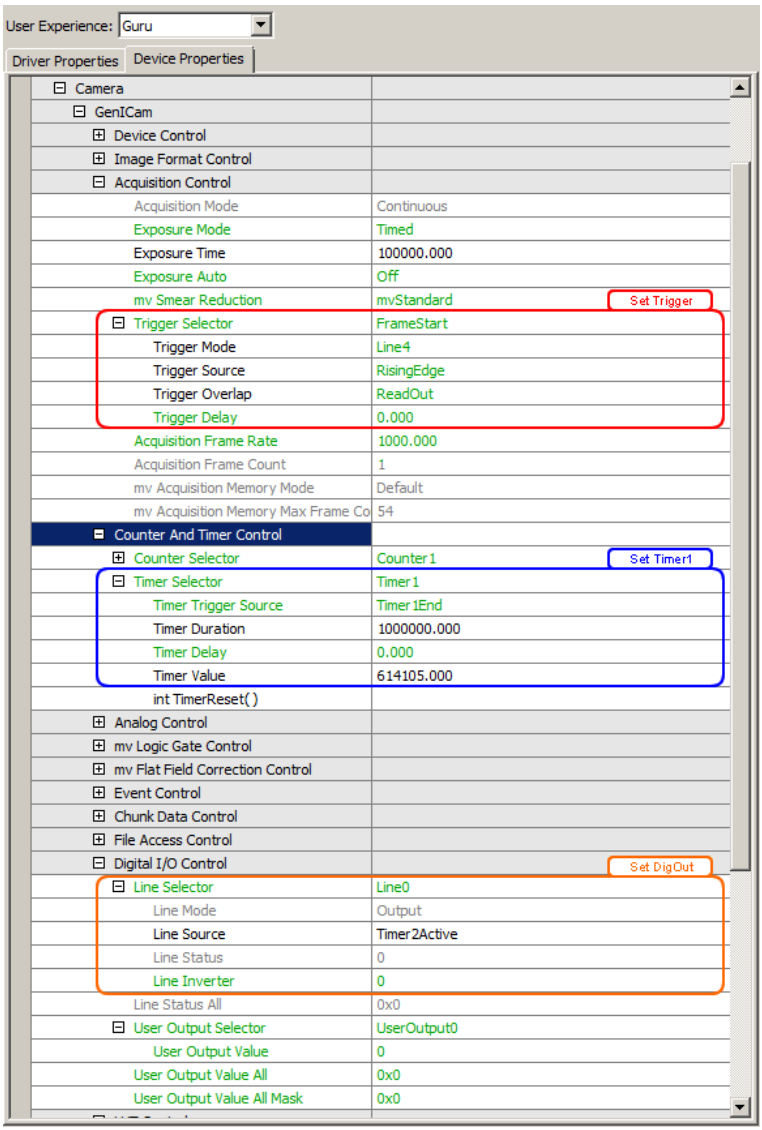


Figure 3: wxPropView - Setting of *Timer1* on the master camera

2. Setting of *Timer2* (purple box) on the master camera:

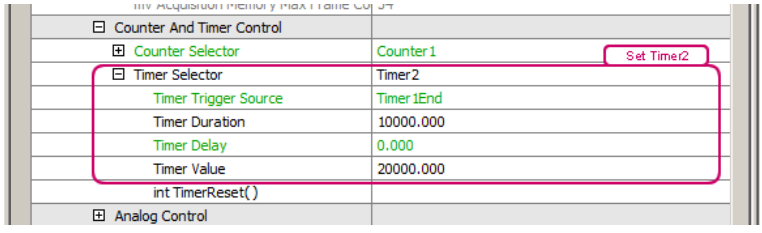


Figure 4: wxPropView - Setting of *Timer2* on the master camera

3. Setting the trigger slave camera(s)
   - The red box in Figure 5 is showing **"Master - Slave"**), which means that the master is triggered internally and the slave camera is set as shown in Figure 4.

4. Assigning timer to DigOut (orange box in Figure 3).

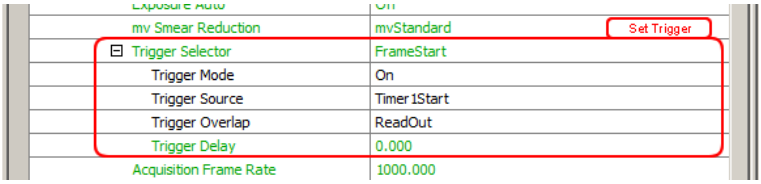


Figure 5: Trigger setting of the master camera using **"Master - Slave"**

1.18.11 Working with 3rd party tools

- **Using VLC Media Player** (p. 345)
- **Working with ROS (Robot Operating System)** (p. 347)
- **Using USB3 Vision™ Devices In A Docker Container** (p. 350)

1.18.11.1 Using VLC Media Player

With the **DirectShow interface** (p. 134) MATRIX VISION devices become a (acquisition) video device for the VLC Media Player.

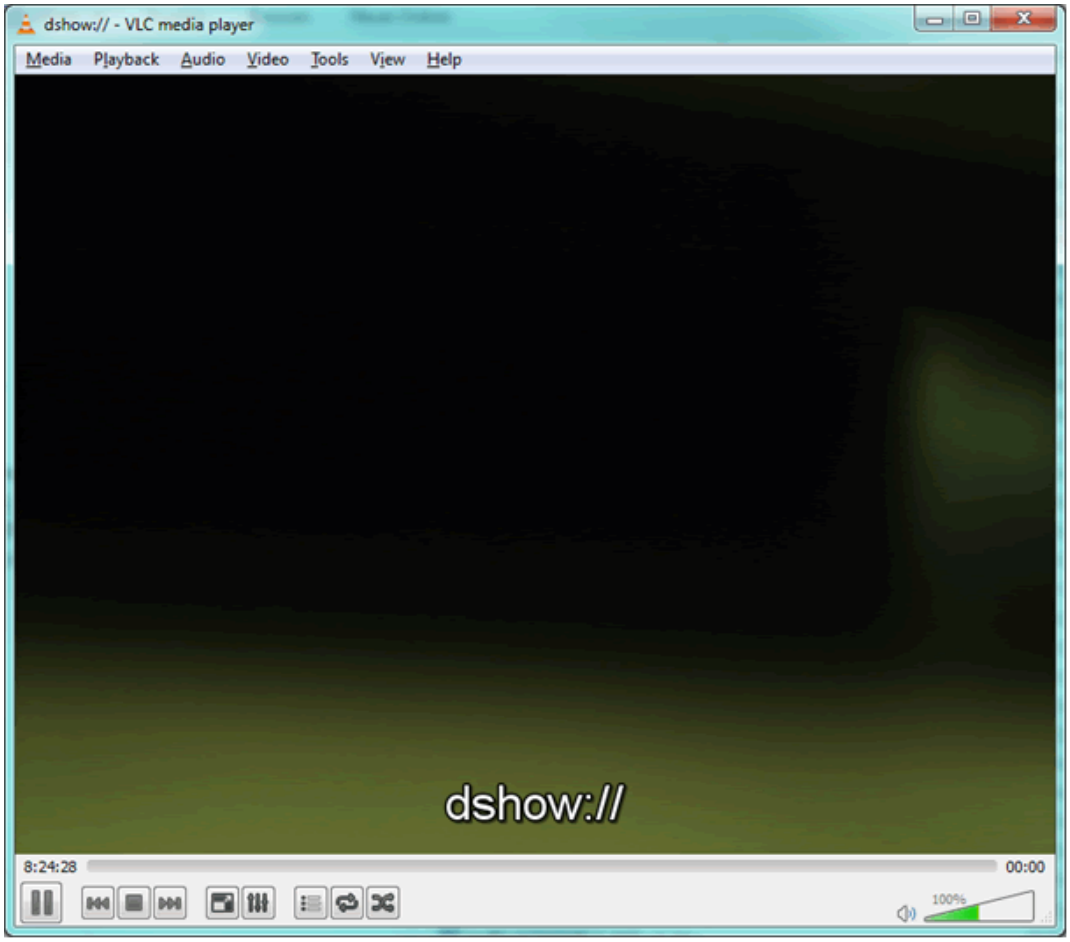


Figure 1: VLC Media Player with a connected device via DirectShow

1.18.11.1.1 System requirements

It is necessary that following drivers and programs are installed on the host device (laptop or PC):

- Windows 7 or higher, 32-bit or 64-bit
- up-do-date VLC Media Player, 32-bit or 64-bit (here: version 2.0.6)
- up-do-date MATRIX VISION driver, 32-bit or 64-bit (here: version 2.5.6)

Note

Using **Windows 10** or **Windows 7**: VLC Media Player with versions 2.2.0 have been tested successfully with older versions of mvIMPACT Acquire. Since version 3.0.0 of VLC at least mvIMPACT Acquire 2.34.0 will be needed to work with devices through the DirectShow interface!

1.18.11.1.2 Installing VLC Media Player

1. Download a suitable version of the VLC Media Player from the VLC Media Player website mentioned below.
2. Run the setup.
3. Follow the installation process and use the default settings.

A restart of the system is not required.

See also

`http://www.videolan.org/`

1.18.11.1.3 Setting up MV device for DirectShow

Note

Please be sure to register the MV device for DirectShow with the right version of **mvDeviceConfigure** (p. 99) . I.e. if you have installed the 32 bit version of the VLC Media Player, you have to register the MV device with the 32 bit version of **mvDeviceConfigure** (p. 99) (*"C:/Program Files/MATRIX VISION/mv↩ IMPACT Acquire/bin"*) !

1. Connect the MV device to the host device directly or via GigE switch using an Ethernet cable.
2. Power the camera using a power supply at the power connector.
3. Wait until the status LED turns blue.
4. Open the tool **mvDeviceConfigure** (p. 99) ,
5. **set a friendly name** (p. 137) ,
6. and **register the MV device for DirectShow** (p. 134) .

Note

In some cases it could be necessary to repeat step 5.

1.18.11.1.4 Working with VLC Media Player

1. Start VLC Media Player.
2. Click on **"Media ->** **Open Capture Device..."** .

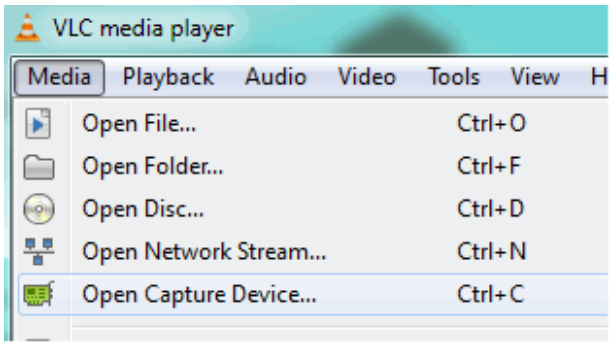


Figure 2: Open Capture Device...

3. Select the tab **"Device Selection"** .
4. In the section **"Video device name"** , select the friendly name of the MV device:

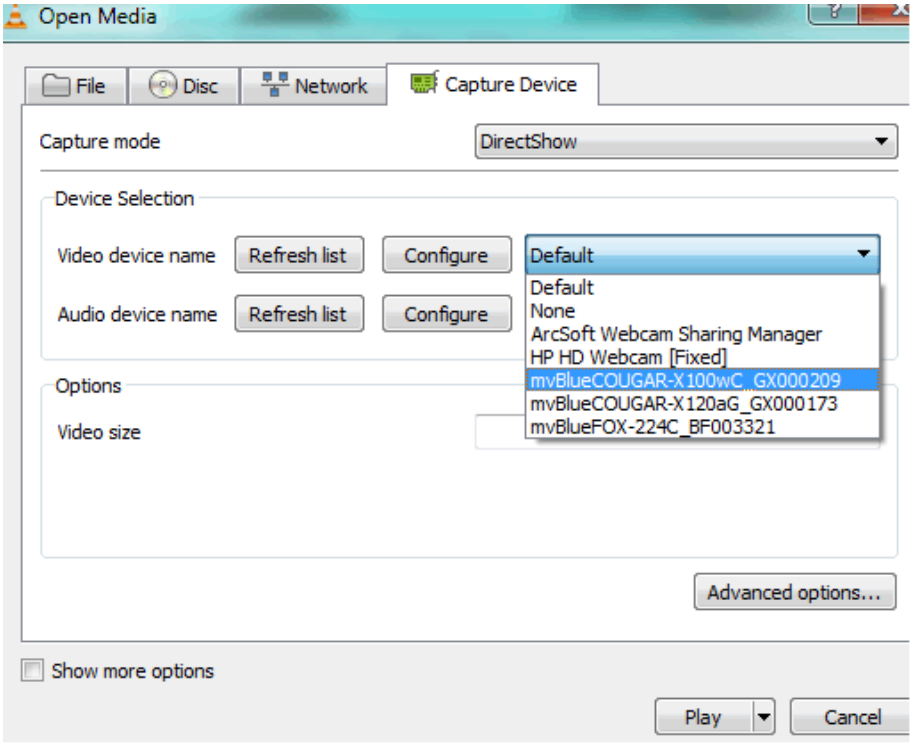


Figure 3: Video device name

5. Finally, click on **"Play"** .
   After a short delay you will see the live image of the camera.

1.18.11.2 Working with ROS (Robot Operating System)

1.18.11.2.1 Recommended ROS Packages If you are planning to use MATRIX VISION GenICam™ devices in a ROS-application, we recommend to use two packages, maintained by the `Roboception GmbH`:

- *rc_genicam_api*, which offers a GenICam™-based access to GenTL™ producers

- *rc_genicam_camera*, which builds on the GenICam™ access provided by *rc_genicam_api* and provides a ROS-node for acquiring images from a camera

Since these packages can be a solid foundation for your ROS project, the following chapters describe how you can use them.

1.18.11.2.2 Initial Setup

1. Make sure to install the mvIMPACT Acquire driver package. Afterwards, the GENICAM_GENTL64_PATH environment variable should point to a directory that contains the mvGenTLProducer.cti library.
2. Install your ROS environment. Please refer to the `ROS Project` if you need more detailed advice.
3. Install or download and build the Roboception packages.
   - For Ubuntu and Debian systems, use the package manager to install:
     (a) **ros-DISTRIBUTION-rc-genicam-api**
     (b) **ros-DISTRIBUTION-rc-genicam-camera** (e.g. `apt install ros-noetic-rc-genicam-api ros-noetic-rc-genicam-camera` if you are using ROS Noetic Ninjemys)
   - If you want to build it yourself, the packages are available on GitHub:
     (a) `rc_genicam_api` like mentioned on the ROS description page `http://wiki.ros.org/rc_genicam_api`
     (b) `rc_genicam_camera`

1.18.11.2.3 Testing The Setup To test your setup, you may use the **gc_info** command line tool, that comes with the *rc_genicam_api* package. This tool will use the **GenICam** (p. 165) Transport Layer that is found within the GENICAM_GENTL64_PATH. A first step might be to list all available devices:

```
gc_info -l
```

The output should look similar to this:



Example of the output from the 'gc_info -l' command

The **GenICam** (p. 165) Transport Layer assigns an individual DeviceID to all devices, which can be used to address the device.

1.18.11.2.4 Streaming From A Device Start the node with the command:
`rosrun rc_genicam_camera rc_genicam_camera _device:=DeviceID_from_step_before`

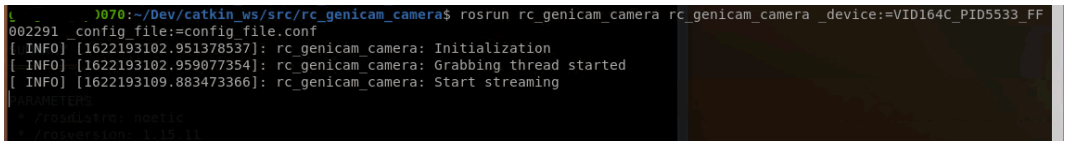
> **Note**
>
> If the *DeviceID* contains one or more colons, it must be preceded by a colon (e.g. :deviceName_MAC:ADDRESS:PART).

With this setup, the node will publish images to the topic */image_raw*. You can now use the tool of your choice to view the image data, for example `RQT`, that comes with the ROS distribution.

1.18.11.2.5 Initial Device Configuration The *rc_genicam_camera* node accepts an optional config file on startup, to launch the device with a defined setup. There is some more information about this file `on the GitHub page of rc_genicam_camera`, but in short it is a list of <property>=<value>, using one line per property. The contents of this file could look like this:

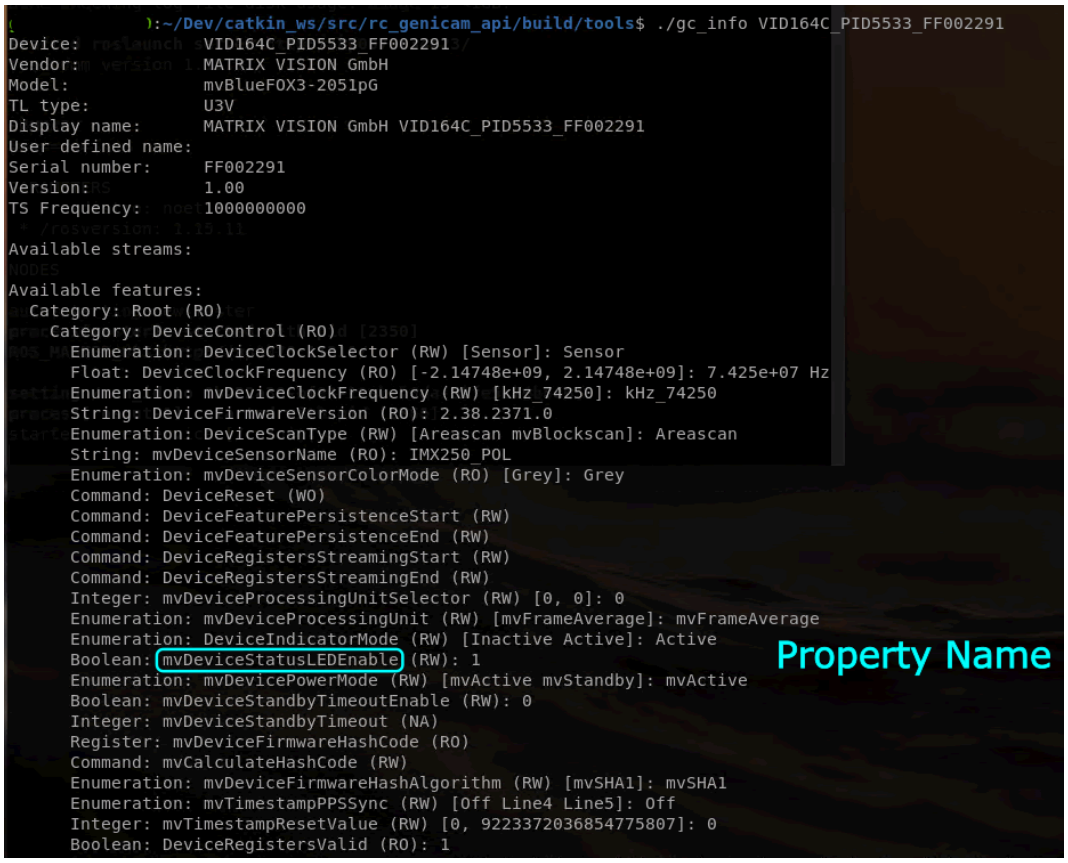
```
# A comment line
Width=720
Height=480
ExposureAuto=Continuous
mvExposureAutoUpperLimit=20000
```

To launch the node with the config file, simply append it to the rosrun call:
`rosrun rc_genicam_camera rc_genicam_camera _device:=DeviceID _config_file:=/path/to/config/file`



Starting the node with DeviceID and config file.

To request a list of all available properties that are accepted by the device, use the *gc_info* tool again. Calling
`gc_info DeviceID`
will return a list, that contains all registered properties:



Listing of available properties from 'gc_info DeviceID'

You may also use **wxPropView** (p. 99) to search for available properties. There, you can find the possible options within the **GenICam** (p. 165) tree. If you are uncertain about valid values for a property, right click on it and choose 'Detailed Feature Information'.

1.18.11.2.6 Device Configuration While Streaming Many properties can be changed while the device is already streaming. This can be accomplished with the *set_genicam_parameter*-service, offered by the *rc_genicam_camera* node. For testing, you may try to use the *rosservice* command. Call *rosservice list* to see all available services.
To set a property, e.g. the *mvExposureAutoUpperLimit*, execute:

```
rosservice call /rc_genicam_camera/set_genicam_parameter 'mvExposureAutoUpperLimit=10000'
```

This will attempt to set the *mvExposureAutoUpperLimit* property to the provided value and return 0 if successful.

Another service, *get_genicam_parameter* is available to read values from properties. E.g. if you want to get the image width, you could execute

```
rosservice call /rc_genicam_camera/get_genicam_parameter 'Width'
```

For a more detailed description of the services, have a look at the `rc_genicam_camera Github page`.

1.18.11.3 Using USB3 Vision™ Devices In A Docker Container

When developing machine vision applications using Docker containers, chances are that you would like to access USB3 Vision™ devices inside the container. With the mvIMPACT Acquire driver stack this can be achieved fairly easy and this chapter will show you how to build a basic Docker container where you can use USB3 Vision™ devices. The current sample Docker container runs on a native Linux machine.

Note

The following chapter is documented only for a native Linux host system.

1.18.11.3.1 Host Preparation

Note

For this demo Docker container the operating system of the host machine is Linux.

Since Docker uses the kernel of the host machine, we first have to **Increasing Kernel memory** (p. 54) of the USB filesystem to make sure that there will be enough temporary buffer for image data transmission at USB3 speed.

1.18.11.3.2 Building A Docker Image The following demo Dockerfile builds a basic Docker image based on a slim version of Debian, where the mvIMPACT Acquire GenTL driver package and its sample programs are installed. This Dockerfile can be used in many ways:

- Use it directly to test your device in a Docker container.
- Use it as a base image for your device applications.
- Use it as an inspiration for building your own Dockerfile.

Before building the Dockerfile, please download the mvIMPACT Acquire GenTL driver installation files from MATRIX VISION GmbH website (`https://www.matrix-vision.com/treiber-software.html`) (user login is required):

- The installation script: *install_mvGenTL_Acquire.sh*
- The installation package: *mvGenTL_Acquire-x86_64_ABI2-*.tgz* (* should be replaced by the version number)

Create a directory called *mvIMPACT_Acquire* (as used in this demo Dockerfile) and move both installation files into this directory. In this example, both files are downloaded into the *Downloads* directory and the *mvIMPACT_Acquire* directory is created inside the *Downloads* directory:

```
$ cd ~/Downloads
$ mkdir mvIMPACT_Acquire
$ mv install_mvGenTL_Acquire.sh mvGenTL_Acquire-x86_64_ABI2-*.tgz mvIMPACT_Acquire/
```

Make the installation script *install_mvGenTL_Acquire.sh* executable:

```
$ cd mvIMPACT_Acquire
$ chmod a+x install_mvGenTL_Acquire.sh
```

Navigate back into the directory where *mvIMPACT_Acquire* resides (e.g. *Downloads*) and create your Dockerfile:

```
$ cd ~/Downloads
$ touch Dockerfile
```

Create the content of your Dockerfile. Our demo Dockerfile (for Linux x86_64) looks as follows:

```
# start with slim version of actual Debian
FROM debian:9-slim

ENV LC_ALL C
ENV DEBIAN_FRONTEND noninteractive

# entrypoint of Docker
CMD ["/bin/bash"]

# set environment variables
ENV TERM linux
ENV MVIMPACT_ACQUIRE_DIR /opt/mvIMPACT_Acquire
ENV MVIMPACT_ACQUIRE_DATA_DIR /opt/mvIMPACT_Acquire/data
ENV GENICAM_GENTL64_PATH /opt/mvIMPACT_Acquire/lib/x86_64
ENV GENICAM_ROOT /opt/mvIMPACT_Acquire/runtime
ENV container docker

# update packets and install minimal requirements
# after installation it will clean apt packet cache
RUN apt-get update && apt-get -y install build-essential && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

# move the directory mvIMPACT_Acquire with *.tgz and *.sh files to the container
COPY mvIMPACT_Acquire /var/lib/mvIMPACT_Acquire

# execute the setup script in an unattended mode
RUN cd /var/lib/mvIMPACT_Acquire && \
    ./install_mvGenTL_Acquire.sh -u && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

Note

In case of ARM architectures, all occurrences of "x86_64" in this demo dockerfile have to be replaced by the correct platform e.g. "arm64" and the install script to use will be *install_mvGenTL_Acquire_ARM.sh* then.

Finally, build a Docker image using this Dockerfile:

```
$ sudo docker build -t [image_name] .
```

Note

Please make sure to call *docker build* from within the directory where the Dockerfile resides. Note that Internet access is required for the *docker build*.

If built successfully, you will be able to see [image_name] being listed when calling:

```
$ sudo docker images
```

1.18.11.3.3 Starting The Docker Container Since the Docker container is isolated from the host system, we have to run it with certain volume mounts and cgroup permissions for it to access USB3 Vision™ devices. In order to avoid running the container in privileged mode, which is not secure, it can be started like this:

```
$ sudo docker run -ti -v /dev:/dev -v /run/udev:/run/udev:ro --device-cgroup-rule 'a 189:* rwm'
 [image_name] /bin/bash
```

Where:

- **-v /dev:/dev**: use volume mount to map the host */dev* directory to the container, so the container will be able to always detect devices also when they get unplugged and re-plugged at any time.
- **-v /run/udev:/run/udev:ro**: volume mount the udev database with read-only permission, so the USB3 Vision™ interfaces can be enumerated correctly in the container.
- **–device-cgroup-rule 'a 189:* rwm'**: with the *–device-cgroup-rule* flag we can add specific permission rules to a device list that is allowed by the container's cgroup. Here in this example, *189* is the major number of the USB bus, * means all minor numbers, and *rwm* are respectively read, write, mknod accesses. By doing so, we will have read, write, mknod accesses to all the USB devices. USB3 Vision™ devices can thus be enumerated successfully.

1.18.11.3.4 Validation After starting the container the correct operation of USB3 Vision™ devices can be validated by running one of the sample programs provided by the mvIMPACT Acquire (e.g. *SingleCapture*):

```
$ cd /opt/mvIMPACT_Acquire/apps/SingleCapture/x86_64
$ ./SingleCapture
```

If the attached USB3 Vision™ device appears in the device list of the program's output, congratulations, you've managed to access USB3 Vision™ devices in the container by using the mvIMPACT Acquire. Now you can use them inside the Docker container for your machine vision applications.

1.19 Working with the temperature sensors

The MATRIX VISION devices offer two different temperature sensors:

- on the sensor board (typically lower)
- on FGPA board (typically higher)

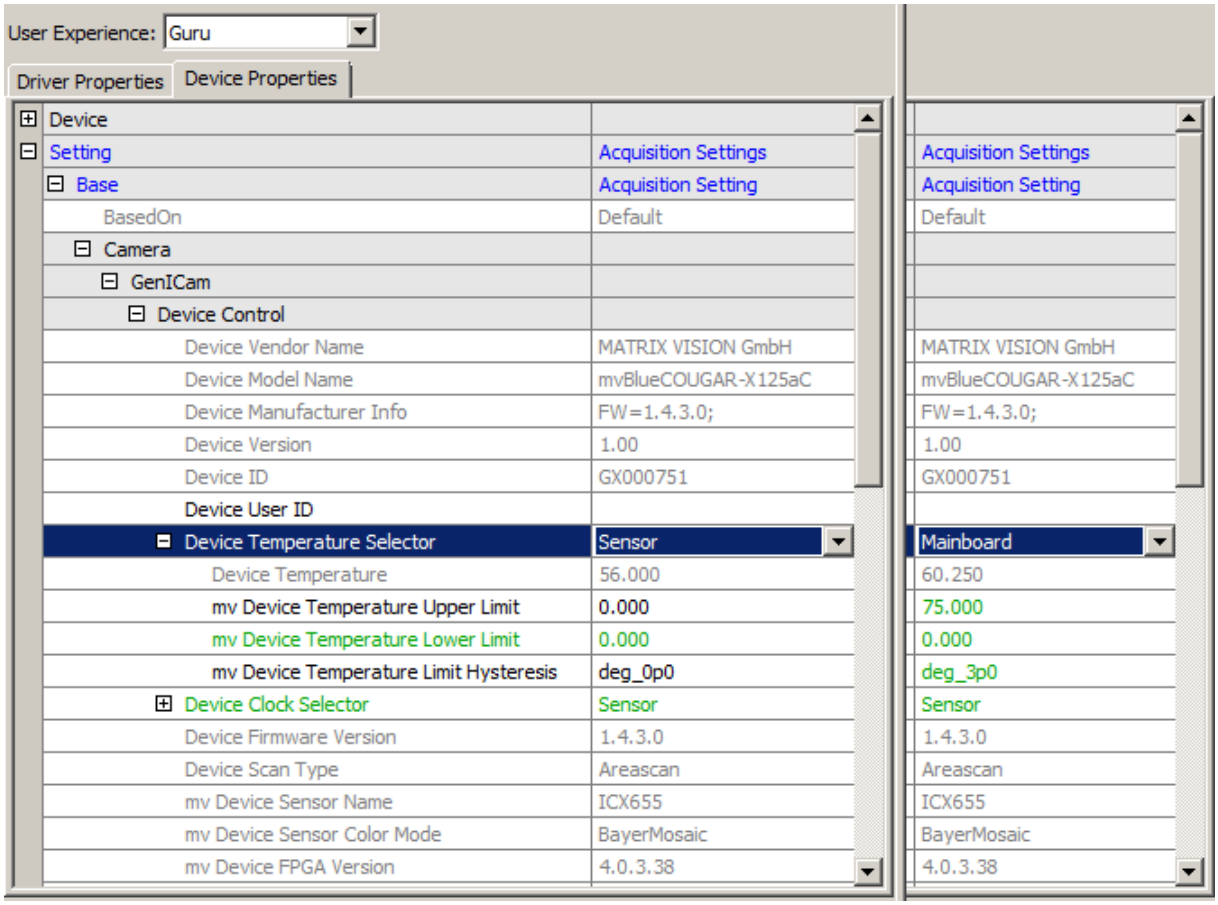


Figure 1: wxPropView - Device Temperature Selector

Note

Avoid temperatures higher than 80 C by lowering thermal resistance of camera housing to connecting structure active cooling means.

It is possible to regulate the temperature of the camera. The limits of this feature are

- upper limit (<= 255 C) and
- lower limit (>= 0 C).

Furthermore, the hysteresis (0,1.5,3,6 C) difference between on/off prevents oscillation. The temperature out of range will set selected output high (see Figure 2). The output can directly drive a fan or a (modest) heating system (but not both due to limit of IC).

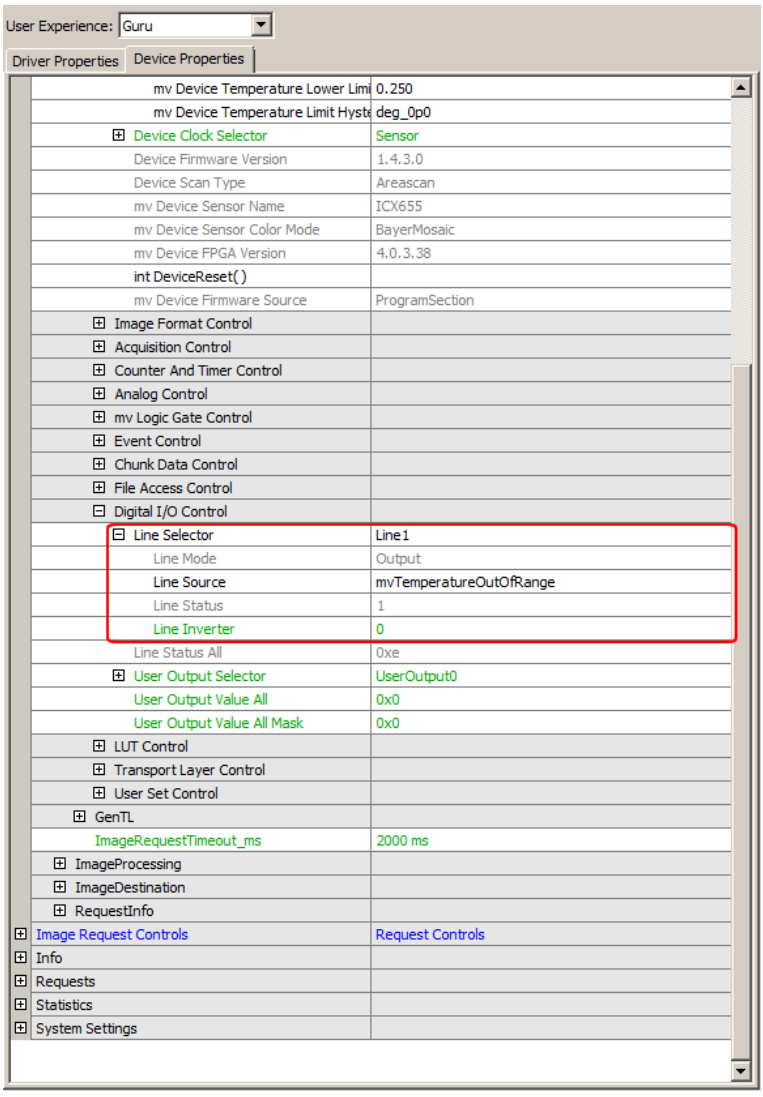


Figure 2: wxPropView - Temperature I/O setting

1.20 Appendix A. Specific Camera / Sensor Data

- **A.1 Pregius CMOS** (p. 355)
- **A.2 Starvis CMOS** (p. 445)
- **A.3 Polarsens CMOS** (p. 459)
- **A.4 CMOS** (p. 463)

1.20.1 A.1 Pregius CMOS

- **Pregius S** (p. 355)
- **Pregius** (p. 376)

1.20.1.1 Pregius S

- **mvBlueFOX3-2051d / BF3-5M-0051D (5.1 Mpix [2472 x 2064])** (p. 355)
- **mvBlueFOX3-2081a / BF3-5M-0081A (8.1 Mpix [2856 x 2848])** (p. 359)
- **mvBlueFOX3-2124d / BF3-5M-0124D (12.4 Mpix [4128 x 3008])** (p. 362)
- **mvBlueFOX3-2162 / BF3-5M-0162A (16.2 Mpix [5328 x 3040])** (p. 366)
- **mvBlueFOX3-2204 / BF3-5M-0204A (20.5 Mpix [4512 x 4512])** (p. 369)
- **mvBlueFOX3-2246 / BF3-5M-0246A (24.6 Mpix [5328 x 4608])** (p. 373)

1.20.1.1.1 mvBlueFOX3-2051d / BF3-5M-0051D (5.1 Mpix [2472 x 2064])

1.20.1.1.1.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX547
Max. frames per second	74.9
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	tbd
DNR (normal / HDR) [dB][1/]	tbd / -
Image size	1/1.8
Number of effective pixels	2472 (H) x 2064 (V)
Unit cell size	2.74µm (H) x 2.74µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.1.1.2 Spectral Sensitivity

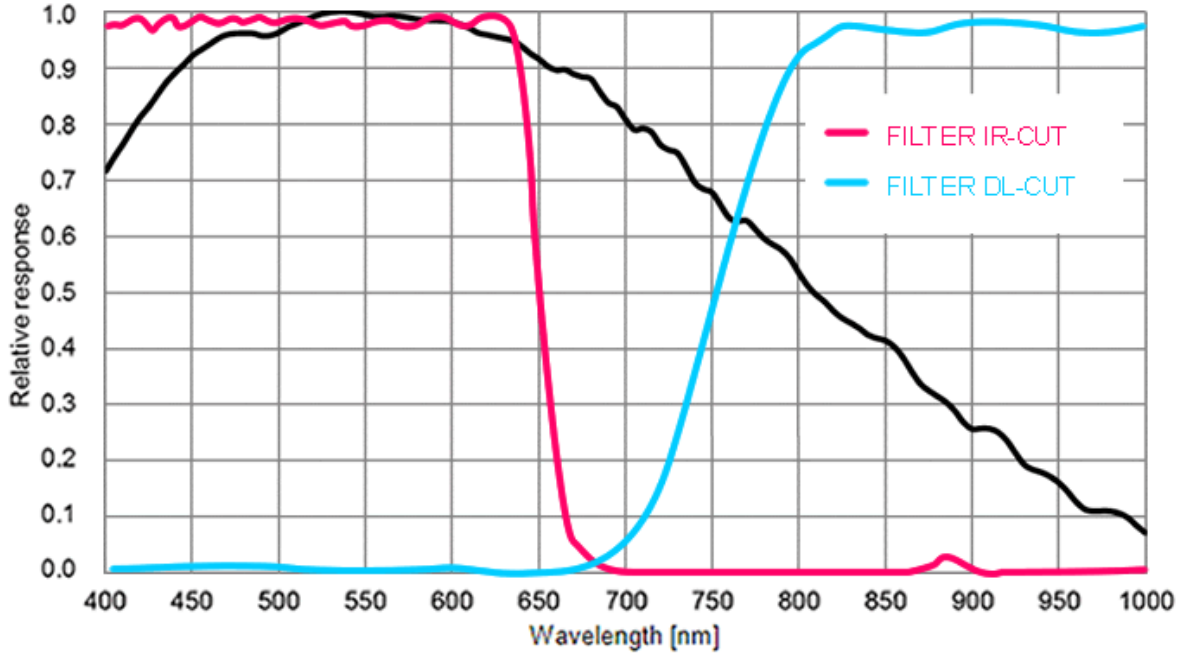


Figure 1: Spectral sensitivity mvBlueFOX3-2051dG

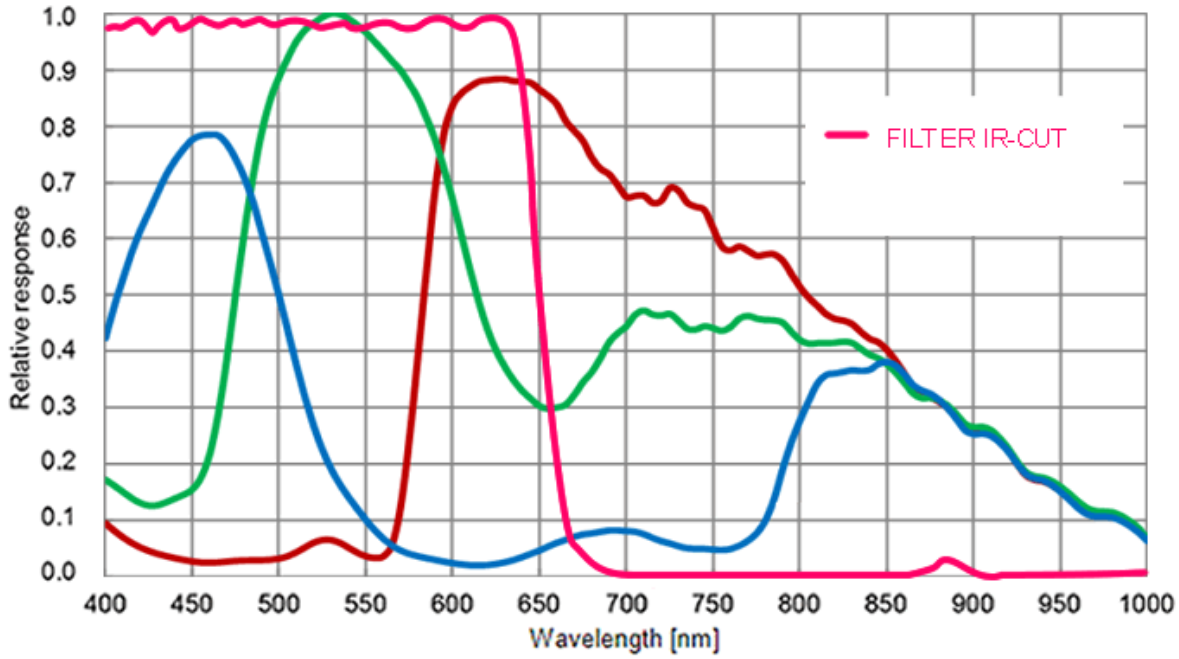


Figure 2: Spectral sensitivity mvBlueFOX3-2051dC

Name	Value
InternalLineLength	450
VerticalBlankLines	139
SensorInClock	74.219 (@74.25 MHz Pixel clock)
NumberOfLVDS	8

1.20.1.1.1.3 Timings

1.20.1.1.1.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 6.06 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 358)

1.20.1.1.1.5 Device Feature And Property List

- **mvBlueFOX3-2051dG / BF3-5M-0051DG Features** (p. 358)
- **mvBlueFOX3-2051dC / BF3-5M-0051DC Features** (p. 358)

1.20.1.1.1.6 mvBlueFOX3-2051dG / BF3-5M-0051DG Features

1.20.1.1.1.7 mvBlueFOX3-2051dC / BF3-5M-0051DC Features

1.20.1.1.2 mvBlueFOX3-2081a / BF3-5M-0081A (8.1 Mpix [2856 x 2848])

1.20.1.1.2.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX546
Max. frames per second	48.7
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	tbd
DNR (normal / HDR) [dB][1/]	tbd / -
Image size	2/3
Number of effective pixels	2856 (H) x 2848 (V)
Unit cell size	2.74µm (H) x 2.74µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.1.2.2 Spectral Sensitivity

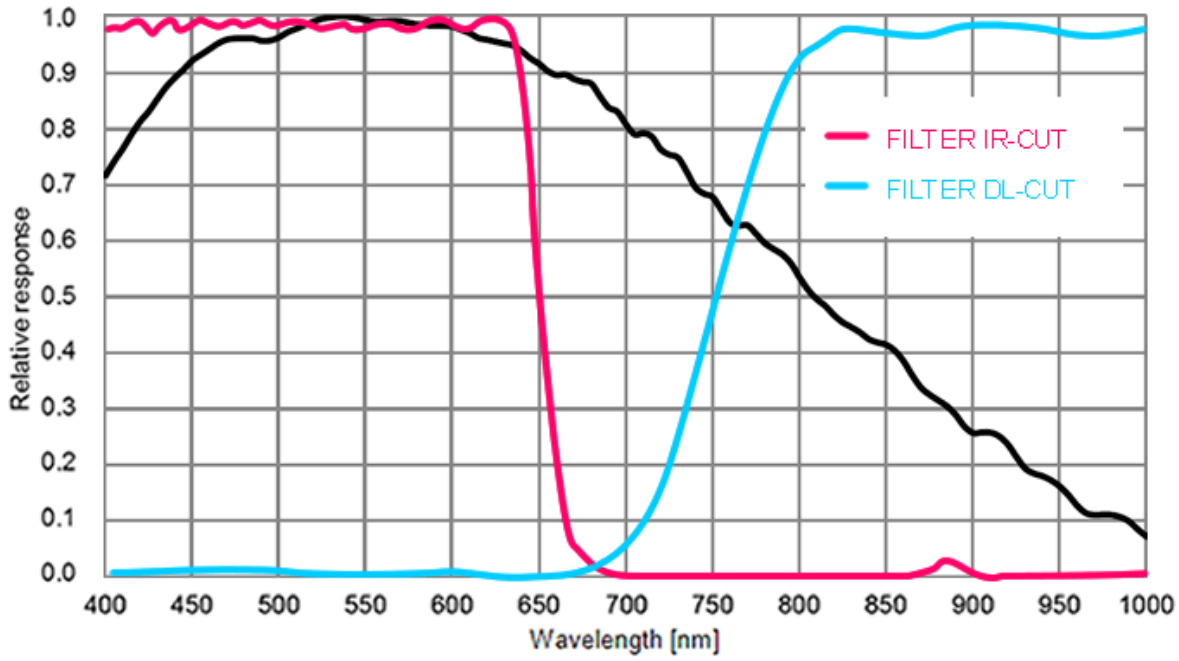


Figure 1: Spectral sensitivity mvBlueFOX3-2081aG

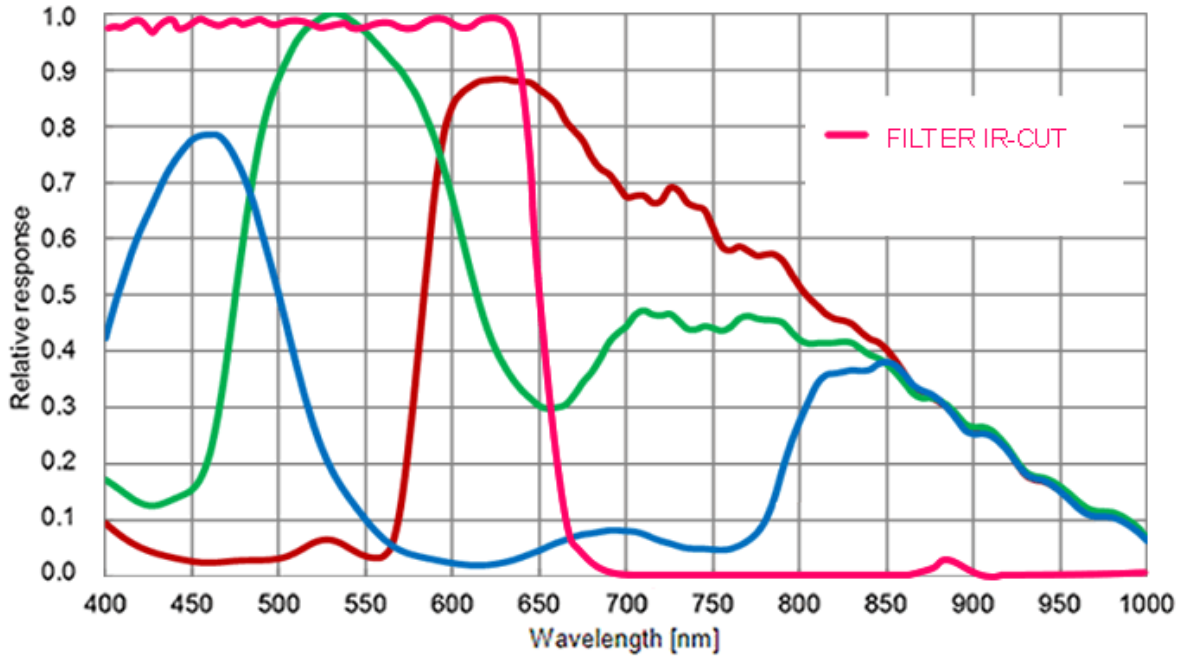


Figure 2: Spectral sensitivity mvBlueFOX3-2081aC

Name	Value
InternalLineLength	510
VerticalBlankLines	139
SensorInClock	74.219 (@74.25 MHz Pixel clock)
NumberOfLVDS	8

1.20.1.1.2.3 Timings

1.20.1.1.2.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 6.87 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 362)

1.20.1.1.2.5 Device Feature And Property List

- **mvBlueFOX3-2081aG / BF3-5M-0081AG Features** (p. 362)
- **mvBlueFOX3-2081aC / BF3-5M-0081AC Features** (p. 362)

1.20.1.1.2.6 mvBlueFOX3-2081aG / BF3-5M-0081AG Features

1.20.1.1.2.7 mvBlueFOX3-2081aC / BF3-5M-0081AC Features

1.20.1.1.3 mvBlueFOX3-2124d / BF3-5M-0124D (12.4 Mpix [4128 x 3008])

1.20.1.1.3.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX545
Max. frames per second	33.5
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	tbd
DNR (normal / HDR) [dB][1/]	tbd / -
Image size	1/1.1
Number of effective pixels	4128 (H) x 3008 (V)
Unit cell size	2.74µm (H) x 2.74µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.1.3.2 Spectral Sensitivity

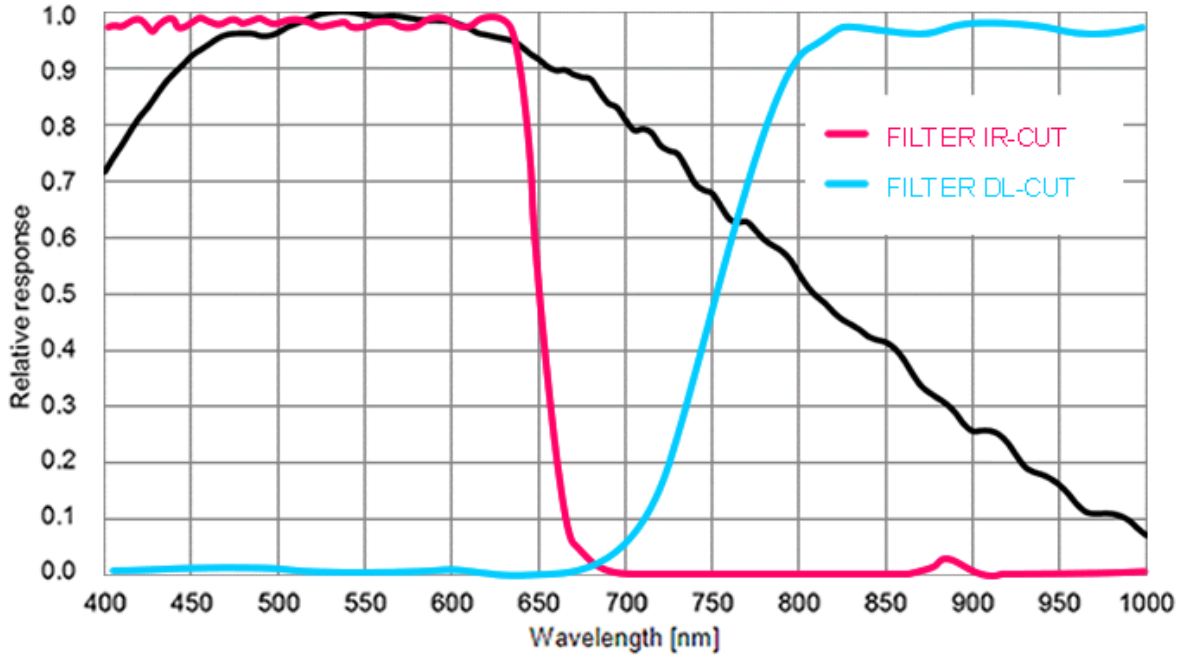


Figure 1: Spectral sensitivity mvBlueFOX3-2124dG

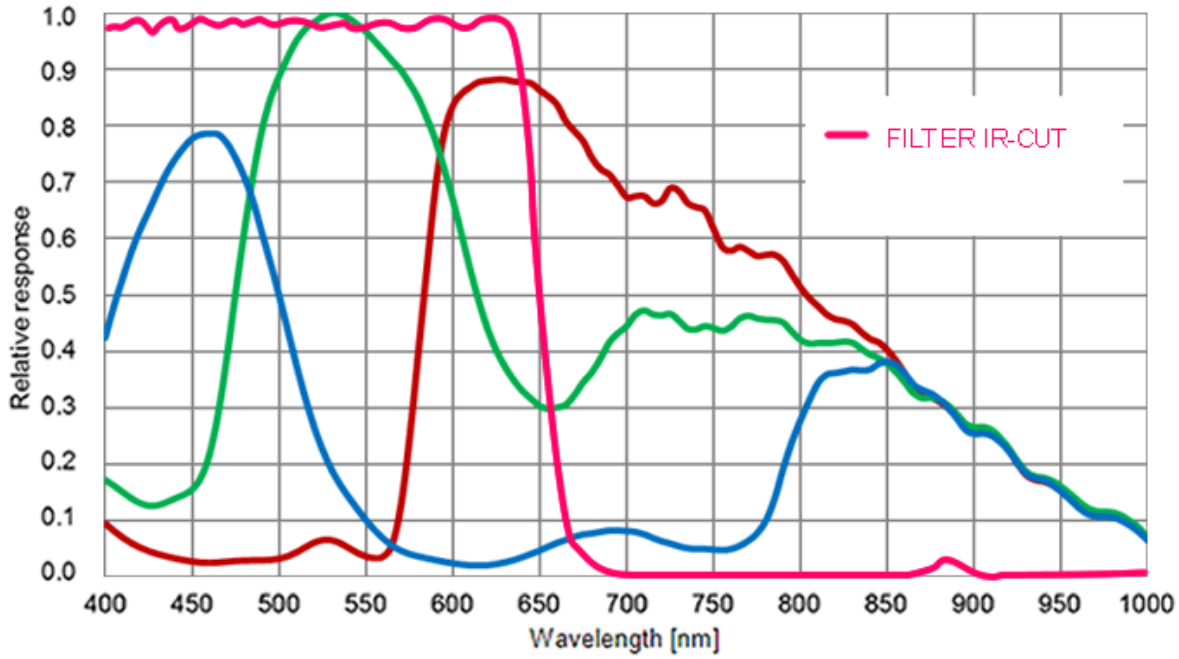


Figure 2: Spectral sensitivity mvBlueFOX3-2124dC

Name	Value
InternalLineLength	704
VerticalBlankLines	139
SensorInClock	74.219 (@74.25 MHz Pixel clock)
NumberOfLVDS	8

1.20.1.1.3.3 Timings

1.20.1.1.3.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 9.49 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 365)

1.20.1.1.3.5 Device Feature And Property List

- **mvBlueFOX3-2124dG / BF3-5M-0124DG Features** (p. 366)
- **mvBlueFOX3-2124dC / BF3-5M-0124DC Features** (p. 366)

1.20.1.1.3.6 mvBlueFOX3-2124dG / BF3-5M-0124DG Features

1.20.1.1.3.7 mvBlueFOX3-2124dC / BF3-5M-0124DC Features

1.20.1.1.4 mvBlueFOX3-2162 / BF3-5M-0162A (16.2 Mpix [5328 x 3040])

1.20.1.1.4.1 Introduction

The sensor is a back-illuminated global shutter, i.e. the image is acquired through the back side without any obstacles like wiring or circuits. Therefore, a wider range of light is collected on the photo-diode.

Feature	Description
Manufacturer	Sony
Sensor name	IMX542
Max. frames per second	26.2
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	39.8
DNR (normal / HDR) [dB][1/]	70.6 / -
Image size	1.1
Number of effective pixels	5328 (H) x 3040 (V)
Unit cell size	2.74µm (H) x 2.74µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.1.4.2 Spectral Sensitivity

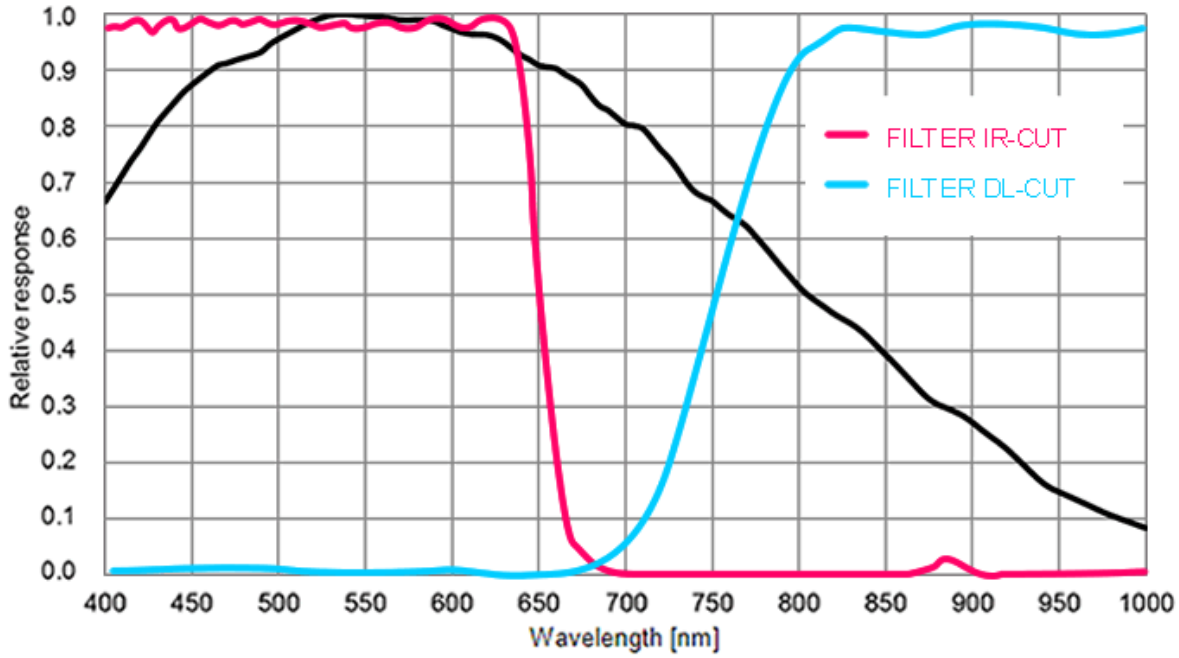


Figure 1: Spectral sensitivity mvBlueFOX3-2162G

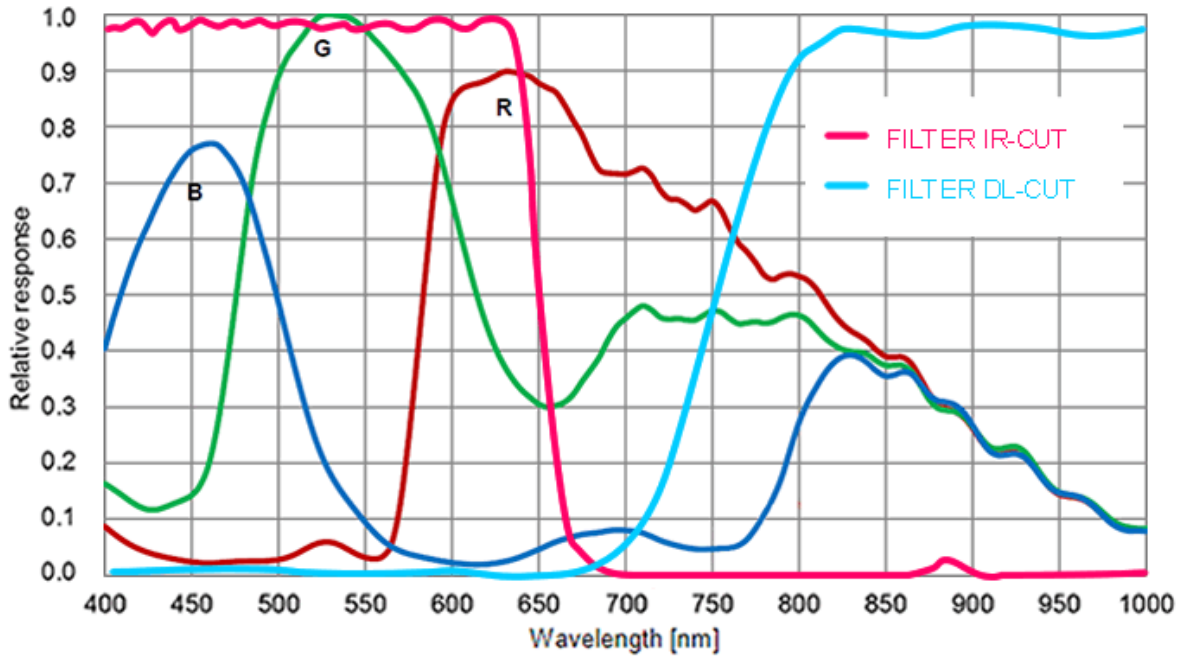


Figure 2: Spectral sensitivity mvBlueFOX3-2162C

Name	Value
InternalLineLength	649 (10 bit) / 776 (12 bit)
VerticalBlankLines	129
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.1.4.3 Timings

1.20.1.1.4.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 12.02 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 369)

1.20.1.1.4.5 Device Feature And Property List

- **mvBlueFOX3-2162G Features** (p. 369)
- **mvBlueFOX3-2162C Features** (p. 369)

1.20.1.1.4.6 mvBlueFOX3-2162G Features

1.20.1.1.4.7 mvBlueFOX3-2162C Features

1.20.1.1.5 mvBlueFOX3-2204 / BF3-5M-0204A (20.5 Mpix [4512 x 4512])

1.20.1.1.5.1 Introduction

The sensor is a back-illuminated global shutter, i.e. the image is acquired through the back side without any obstacles like wiring or circuits. Therefore, a wider range of light is collected on the photo-diode.

Feature	Description
Manufacturer	Sony
Sensor name	IMX541
Max. frames per second	20.9
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	39.8
DNR (normal / HDR) [dB][1/]	70.7 / -
Image size	1.1
Number of effective pixels	4512 (H) x 4512 (V)
Unit cell size	2.74µm (H) x 2.74µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.1.5.2 Spectral Sensitivity

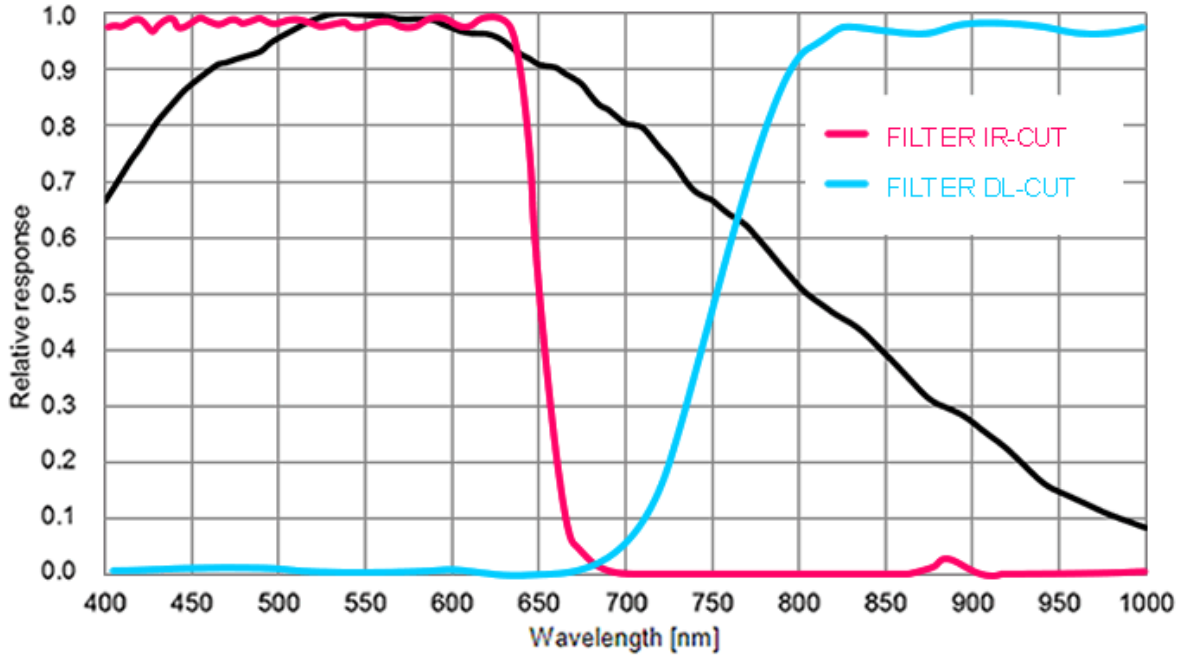


Figure 1: Spectral sensitivity mvBlueFOX3-2204G

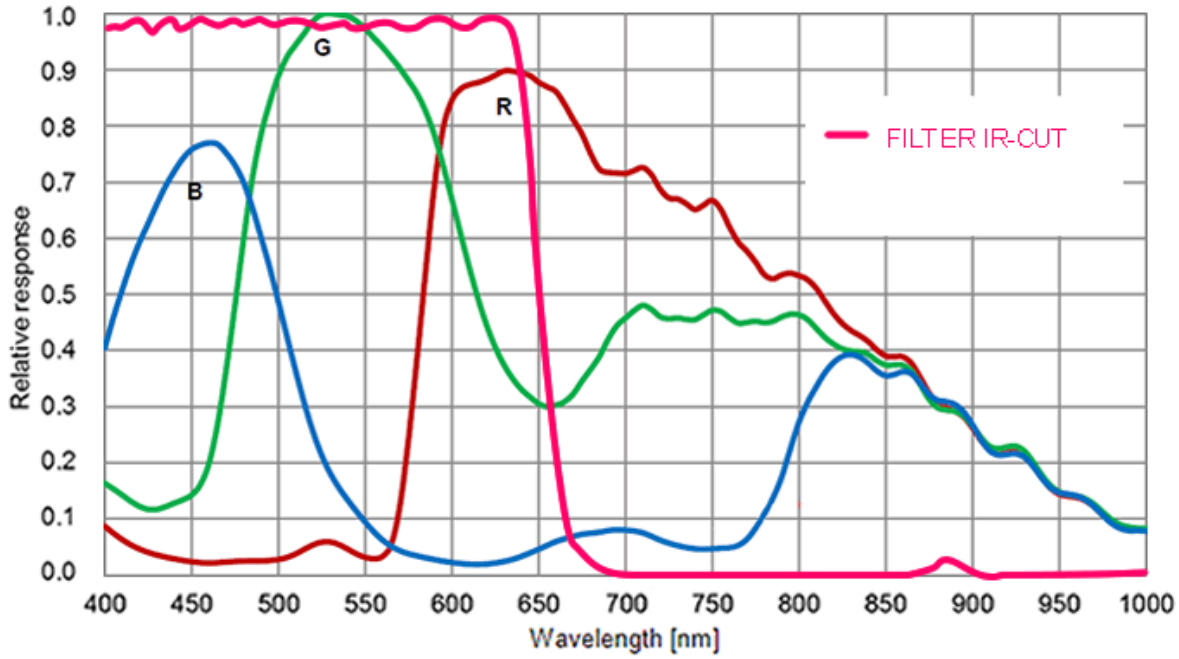


Figure 2: Spectral sensitivity mvBlueFOX3-2204C

Name	Value
InternalLineLength	556 (10 bit) / 665 (12 bit)
VerticalBlankLines	133
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.1.5.3 Timings

1.20.1.1.5.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 10.30 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed"	OnDemand	Image acquisition triggered by command (software trigger).

To trigger one frame execute the *TriggerSoftware@i* command then.

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 372)

1.20.1.1.5.5 Device Feature And Property List

- **mvBlueFOX3-2204G Features** (p. 372)
- **mvBlueFOX3-2204C Features** (p. 372)

1.20.1.1.5.6 mvBlueFOX3-2204G Features

1.20.1.1.5.7 mvBlueFOX3-2204C Features

1.20.1.1.6 mvBlueFOX3-2246 / BF3-5M-0246A (24.6 Mpix [5328 x 4608])

1.20.1.1.6.1 Introduction

The sensor is a back-illuminated global shutter, i.e. the image is acquired through the back side without any obstacles like wiring or circuits. Therefore, a wider range of light is collected on the photo-diode.

Feature	Description
Manufacturer	Sony
Sensor name	IMX540
Max. frames per second	26.2
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	39.7
DNR (normal / HDR) [dB][1/]	70.4 / -
Image size	1.2
Number of effective pixels	5328 (H) x 4608 (V)
Unit cell size	2.74µm (H) x 2.74µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.1.6.2 Spectral Sensitivity

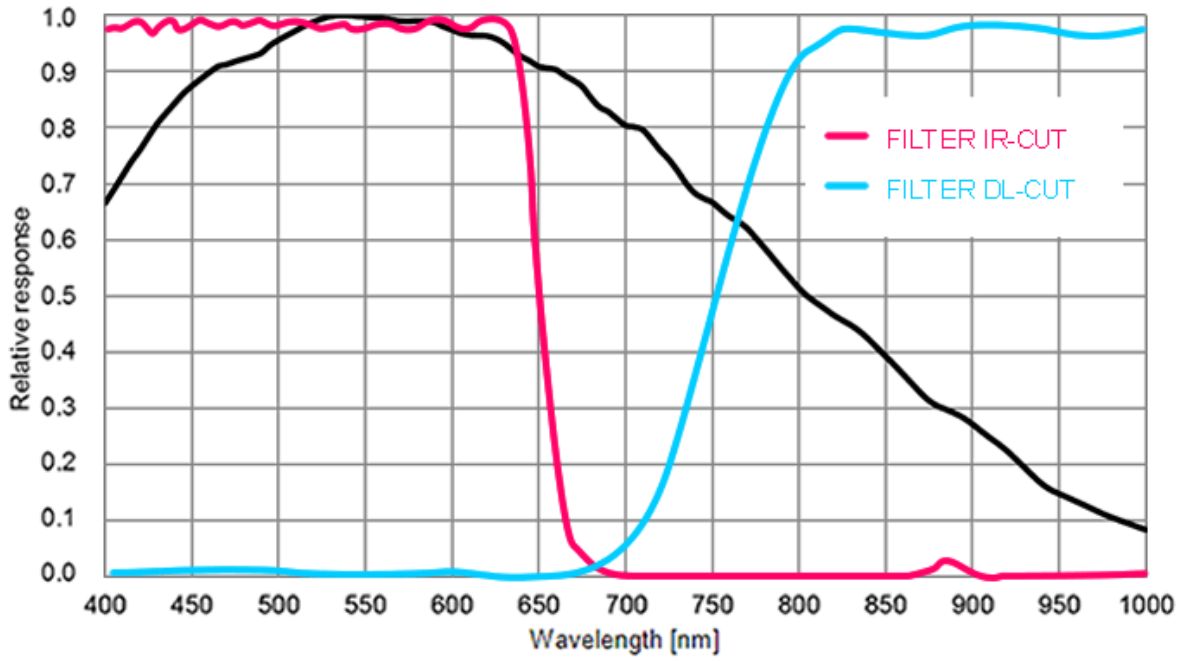


Figure 1: Spectral sensitivity mvBlueFOX3-2246G

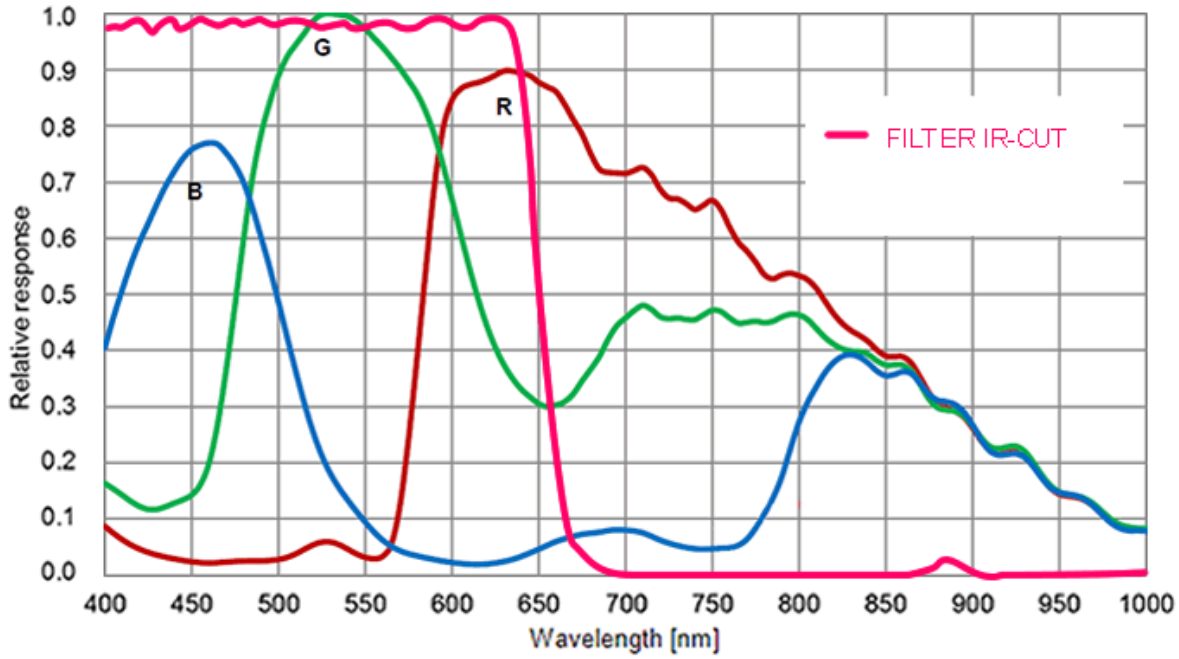


Figure 2: Spectral sensitivity mvBlueFOX3-2246C

Name	Value
InternalLineLength	649 (10 bit) / 776 (12 bit)
VerticalBlankLines	129
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.1.6.3 Timings

1.20.1.1.6.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 12.02 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 376)

1.20.1.1.6.5 Device Feature And Property List

- **mvBlueFOX3-2246G / BF3-5M-0246AG Features** (p. 376)
- **mvBlueFOX3-2246C / BF3-5M-0246AC Features** (p. 376)

1.20.1.1.6.6 mvBlueFOX3-2246G / BF3-5M-0246AG Features

1.20.1.1.6.7 mvBlueFOX3-2246C / BF3-5M-0246AC Features

1.20.1.2 Pregius

- **mvBlueFOX3-2004 / BF3-5M-0004F (0.4 Mpix [728 x 544])** (p. 377)
- **mvBlueFOX3-2016 / BF3-5M-0016Z (1.6 Mpix [1456 x 1088])** (p. 381)
- **mvBlueFOX3-2017 / BF3-5M-0017Z (1.8 Mpix [1600 x 1104])** (p. 384)
- **mvBlueFOX3-2024 / BF3-5M-0024ZG (2.4 Mpix [1936 x 1216])** (p. 387)
- **mvBlueFOX3-2024a / BF3-5M-0024A (2.4 Mpix [1936 x 1216])** (p. 391)
- **BF3-5M-0024B (2.4 Mpix [1936 x 1216])** (p. 395)
- **mvBlueFOX3-2032 / BF3-5M-0032Z (3.2 Mpix [2064 x 1544])** (p. 398)
- **mvBlueFOX3-2032a / BF3-5M-0032A (3.2 Mpix [2064 x 1544])** (p. 402)
- **mvBlueFOX3-2051 / BF3-5M-0024Z (5.1 Mpix [2464 x 2056])** (p. 405)
- **mvBlueFOX3-2051a / BF3-5M-0051A (5.1 Mpix [2464 x 2056])** (p. 409)
- **mvBlueFOX3-2071 (7.1 Mpix [3216 x 2208])** (p. 412)
- **mvBlueFOX3-2071a (7.1 Mpix [3216 x 2208])** (p. 416)
- **mvBlueFOX3-2089 / BF3-5M-0089Z (8.9 Mpix [4112 x 2176])** (p. 419)

- **mvBlueFOX3-2089a / BF3-5M-0089A (8.9 Mpix [4112 x 2176])** (p. 423)
- **mvBlueFOX3-2124 / BF3-5M-0124Z (12.4 Mpix [4112 x 3008])** (p. 427)
- **mvBlueFOX3-2124a / BF3-5M-0124A (12.4 Mpix [4112 x 3008])** (p. 430)
- **BF3-4-0169Z / BF3-5M-0169Z (16.9 Mpix [5472 x 3080])** (p. 434)
- **BF3-4-0196Z / BF3-5M-0196Z (19.6 Mpix [4432 x 4432])** (p. 438)
- **BF3-4-0315Z / BF3-5M-0315Z (31.5 Mpix [6480 x 4856])** (p. 441)

1.20.1.2.1 mvBlueFOX3-2004 / BF3-5M-0004F (0.4 Mpix [728 x 544])

1.20.1.2.1.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX287
Max. frames per second	436.9
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	43.3
DNR (normal / HDR) [dB][1/]	74.2 / -
Image size	1/2.9
Number of effective pixels	728 (H) x 544 (V)
Unit cell size	6.9µm (H) x 6.9µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.1.2 Spectral Sensitivity

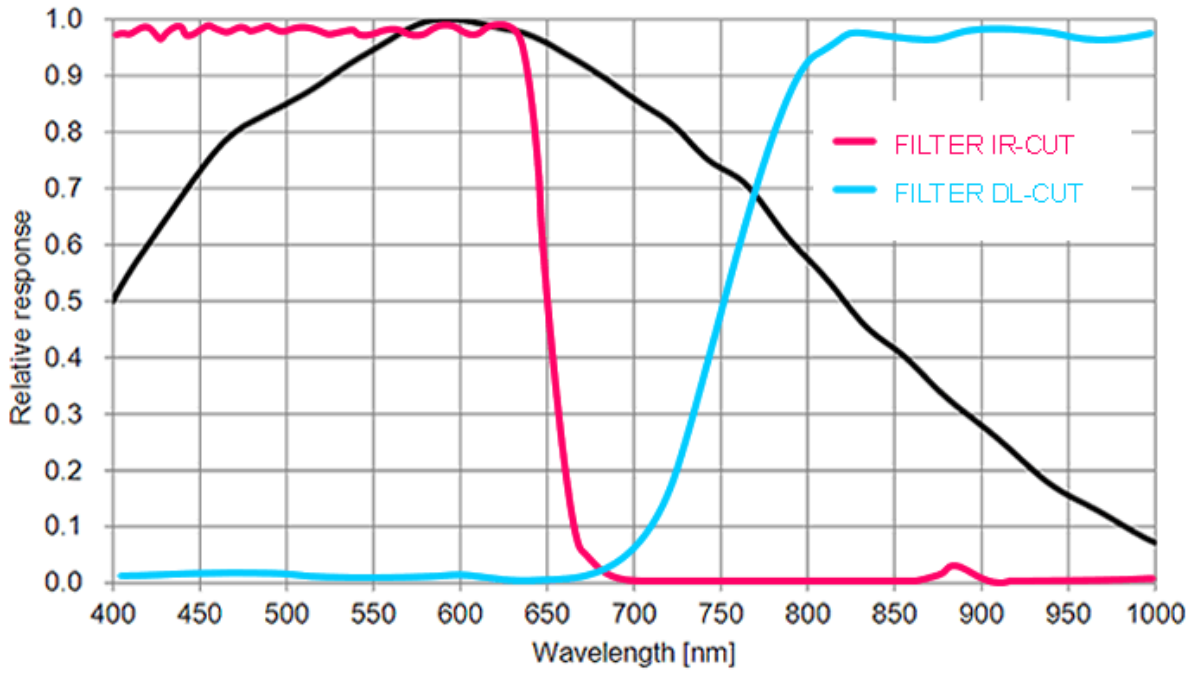


Figure 1: Spectral sensitivity mvBlueFOX3-2004G

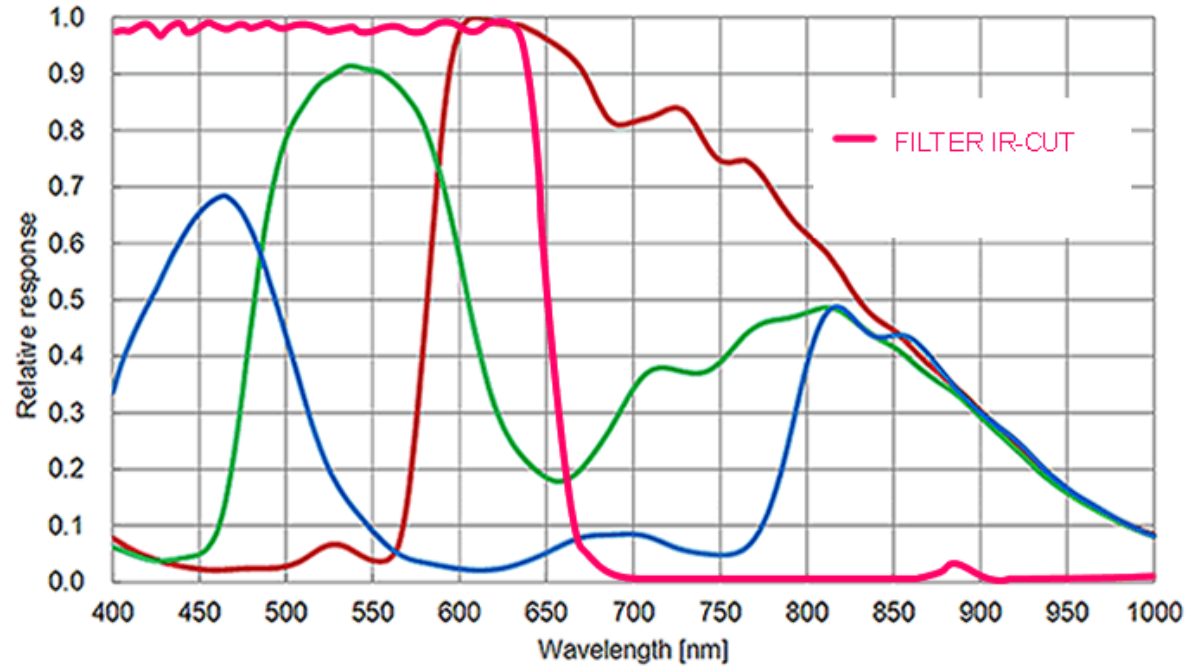


Figure 2: Spectral sensitivity mvBlueFOX3-2004C

Name	Value
InternalLineLength	290 (10 bit) / 396 (12 bit)
VerticalBlankLines	42
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4

1.20.1.2.1.3 Timings

1.20.1.2.1.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.86 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 380)

1.20.1.2.1.5 Device Feature And Property List

- **mvBlueFOX3-2004G / BF3-5M-0004ZG Features** (p. 381)
- **mvBlueFOX3-2004C / BF3-5M-0004ZC Features** (p. 381)

1.20.1.2.1.6 mvBlueFOX3-2004G / BF3-5M-0004ZG Features

1.20.1.2.1.7 mvBlueFOX3-2004C / BF3-5M-0004ZC Features

1.20.1.2.2 mvBlueFOX3-2016 / BF3-5M-0016Z (1.6 Mpix [1456 x 1088])

1.20.1.2.2.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX273
Max. frames per second	226.5
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.4 / -
Image size	1/2.9
Number of effective pixels	1456 (H) x 1088 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.2.2 Spectral Sensitivity

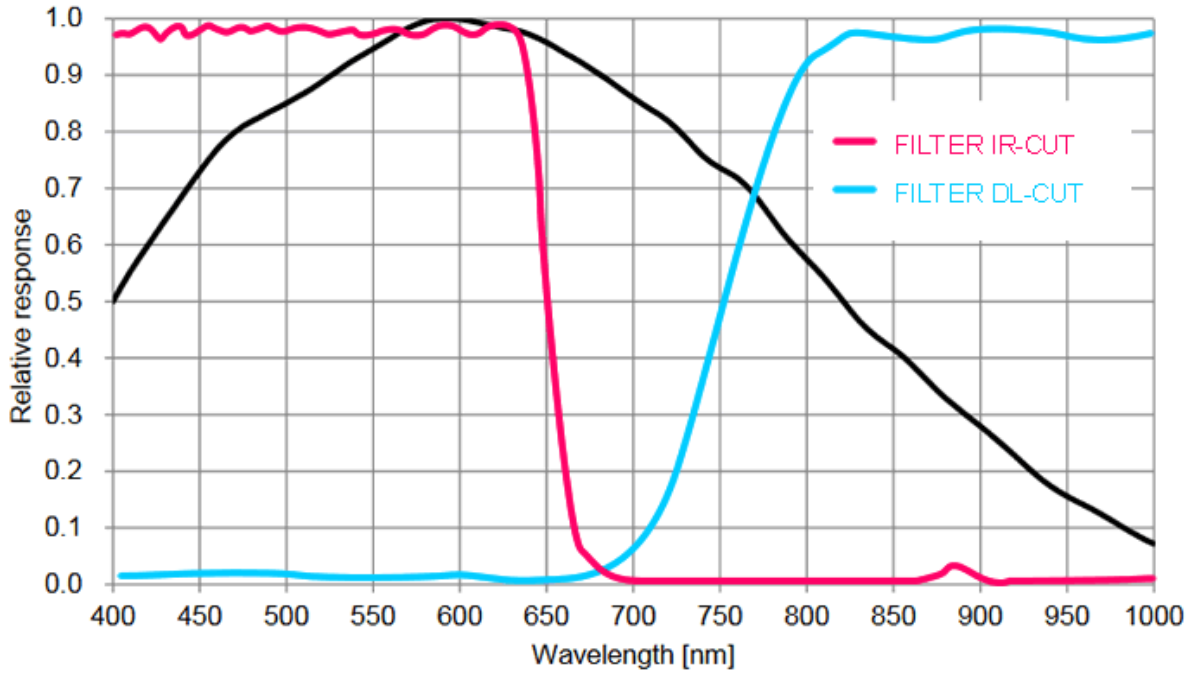


Figure 1: Spectral sensitivity mvBlueFOX3-2016G

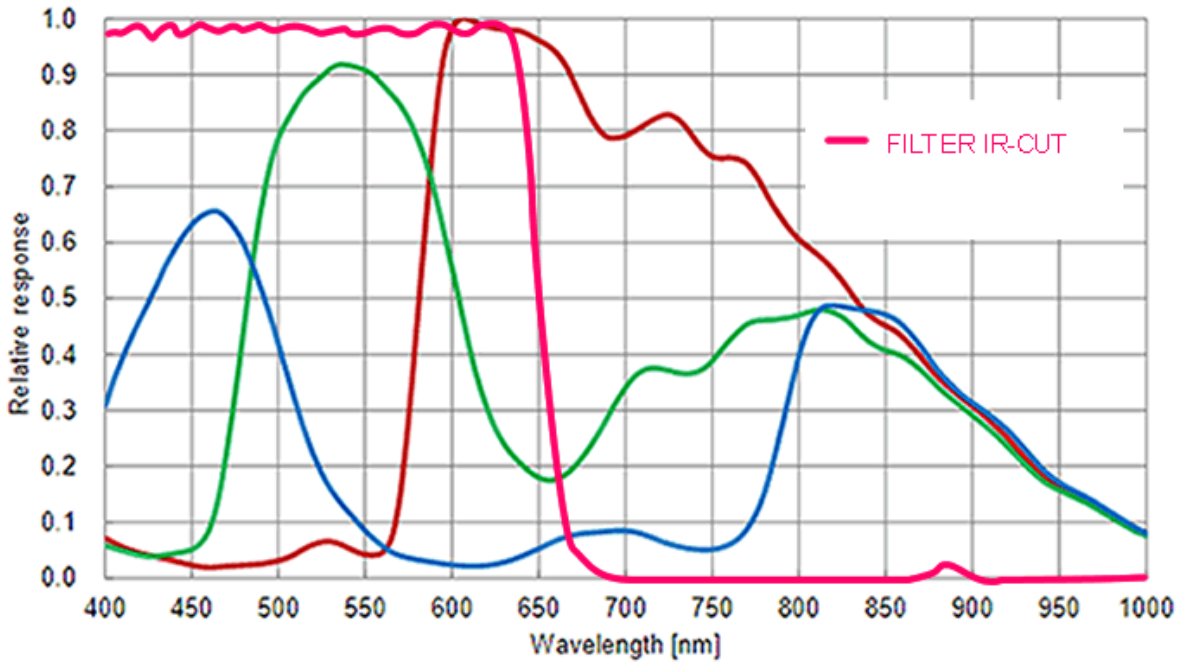


Figure 2: Spectral sensitivity mvBlueFOX3-2016C

Name	Value
InternalLineLength	290 (10 bit) / 396 (12 bit)
VerticalBlankLines	42
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.2.3 Timings

1.20.1.2.2.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.86 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 384)

1.20.1.2.2.5 Device Feature And Property List

- **mvBlueFOX3-2016G / BF3-5M-0016ZG Features** (p. 384)
- **mvBlueFOX3-2016C / BF3-5M-0016ZC Features** (p. 384)

1.20.1.2.2.6 mvBlueFOX3-2016G / BF3-5M-0016ZG Features

1.20.1.2.2.7 mvBlueFOX3-2016C / BF3-5M-0016ZC Features

1.20.1.2.3 mvBlueFOX3-2017 / BF3-5M-0017Z (1.8 Mpix [1600 x 1104])

1.20.1.2.3.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX425
Max. frames per second	212.9
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	tbd
DNR (normal / HDR) [dB][1/]	tbd / -
Image size	1/1
Number of effective pixels	1600 (H) x 1104 (V)
Unit cell size	9µm (H) x 9µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.3.2 Spectral Sensitivity

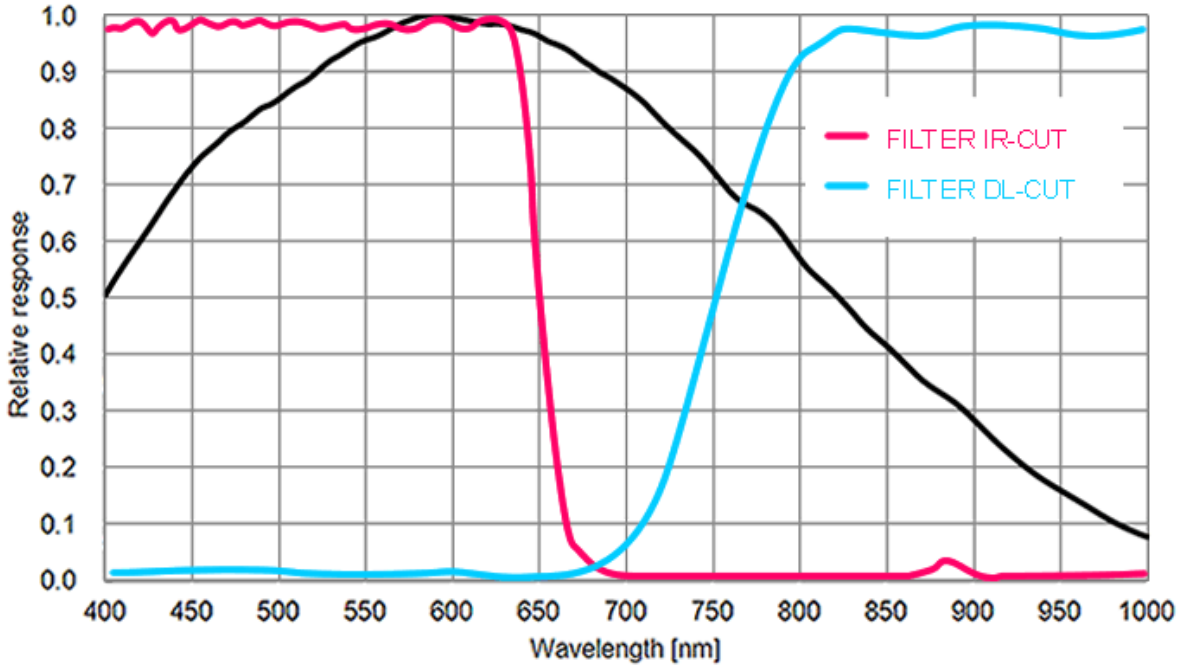


Figure 1: Spectral sensitivity mvBlueFOX3-2017G

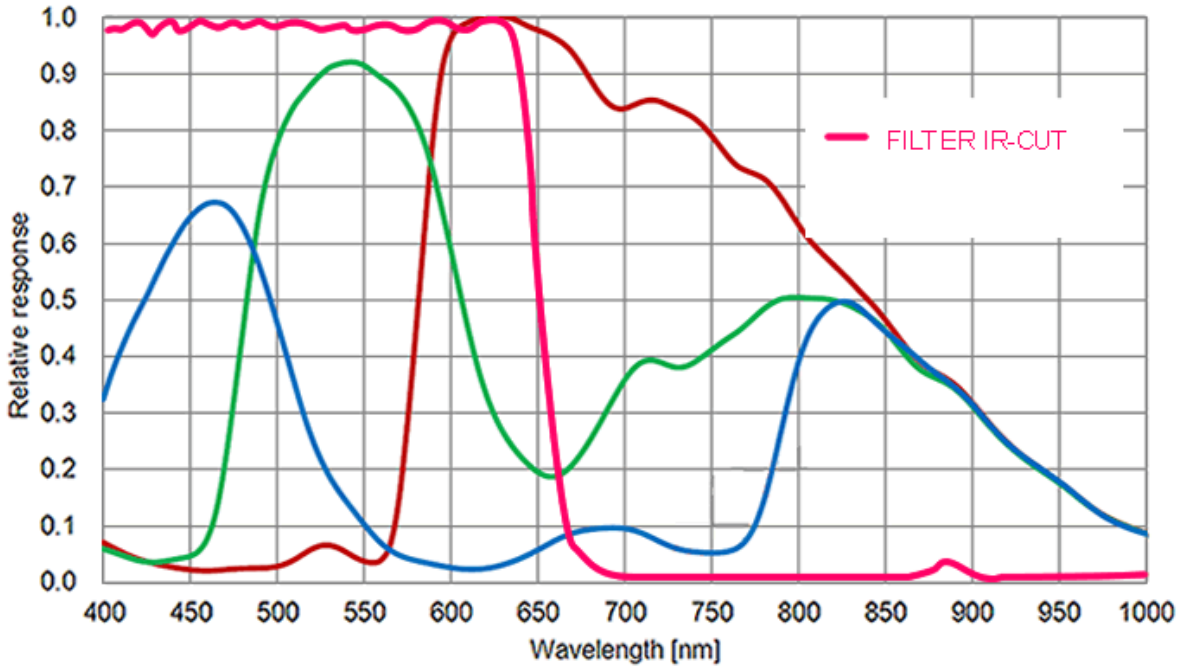


Figure 2: Spectral sensitivity mvBlueFOX3-2017C

Name	Value
InternalLineLength	208
VerticalBlankLines	115
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.2.3.3 Timings

1.20.1.2.3.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.7 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed"	OnDemand	Image acquisition triggered by command (software trigger).
To trigger one frame execute the *TriggerSoftware@i* command then.		

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

mvBF3_subsection_sensors_CMOS_models_2017_DeviceFeatureAndPropertyList

1.20.1.2.4 mvBlueFOX3-2024 / BF3-5M-0024ZG (2.4 Mpix [1936 x 1216])

1.20.1.2.4.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX174
Max. frames per second	164

Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	45.1
DNR (normal / HDR) [dB][1/]	66.4 / -
Image size	1/1.2
Number of effective pixels	1936 (H) x 1216 (V)
Unit cell size	5.86µm (H) x 5.86µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.4.2 Spectral Sensitivity

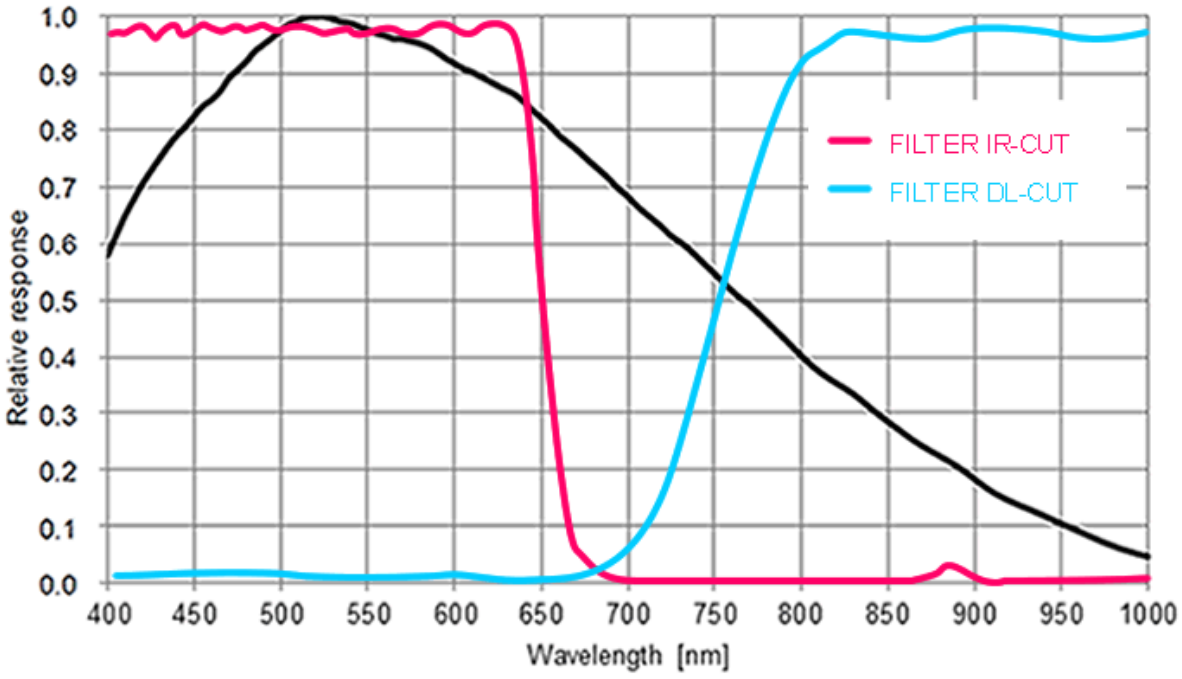


Figure 1: Spectral sensitivity mvBlueFOX3-2024G

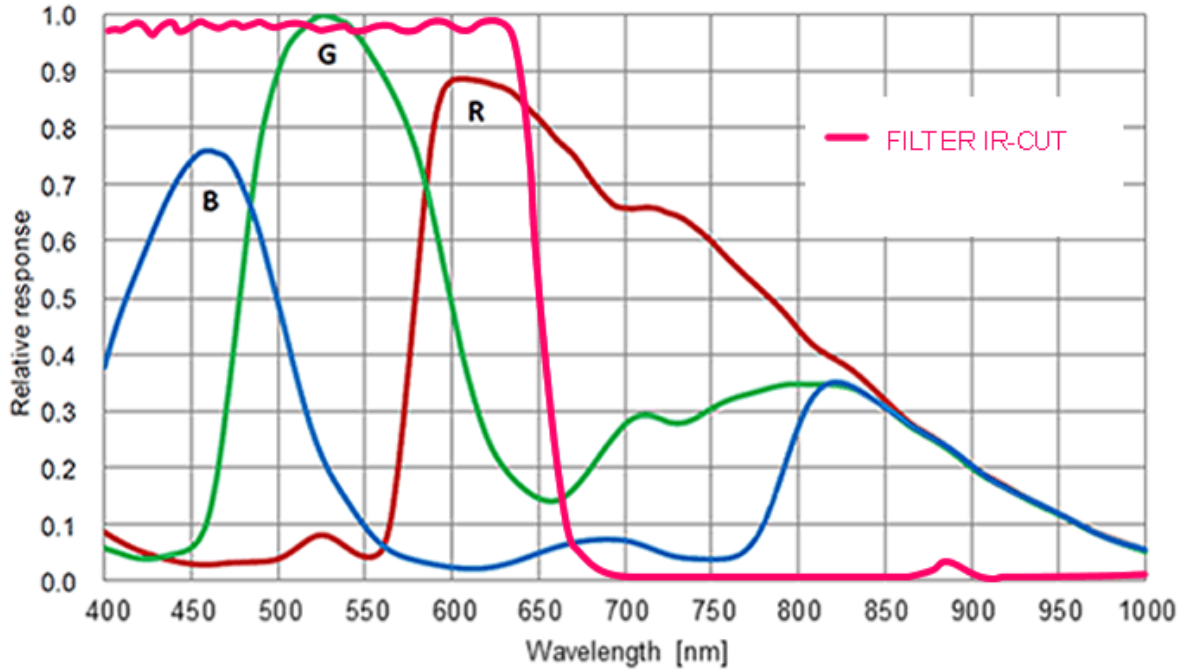


Figure 2: Spectral sensitivity mvBlueFOX3-2024C

Name	Value
InternalLineLength	360 (10 bit) / 462 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.4.3 Timings

1.20.1.2.4.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.86 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 391)

1.20.1.2.4.5 Device Feature And Property List

- **mvBlueFOX3-2024G / BF3-5M-0024ZG Features** (p. 391)
- **mvBlueFOX3-2024C / BF3-5M-0024ZC Features** (p. 391)

1.20.1.2.4.6 mvBlueFOX3-2024G / BF3-5M-0024ZG Features

1.20.1.2.4.7 mvBlueFOX3-2024C / BF3-5M-0024ZC Features

1.20.1.2.5 mvBlueFOX3-2024a / BF3-5M-0024A (2.4 Mpix [1936 x 1216])

1.20.1.2.5.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX249
Max. frames per second	46.9
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	45.1
DNR (normal / HDR) [dB][1/]	73 / -
Image size	1/1.2
Number of effective pixels	1936 (H) x 1216 (V)
Unit cell size	5.86µm (H) x 5.86µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.5.2 Spectral Sensitivity

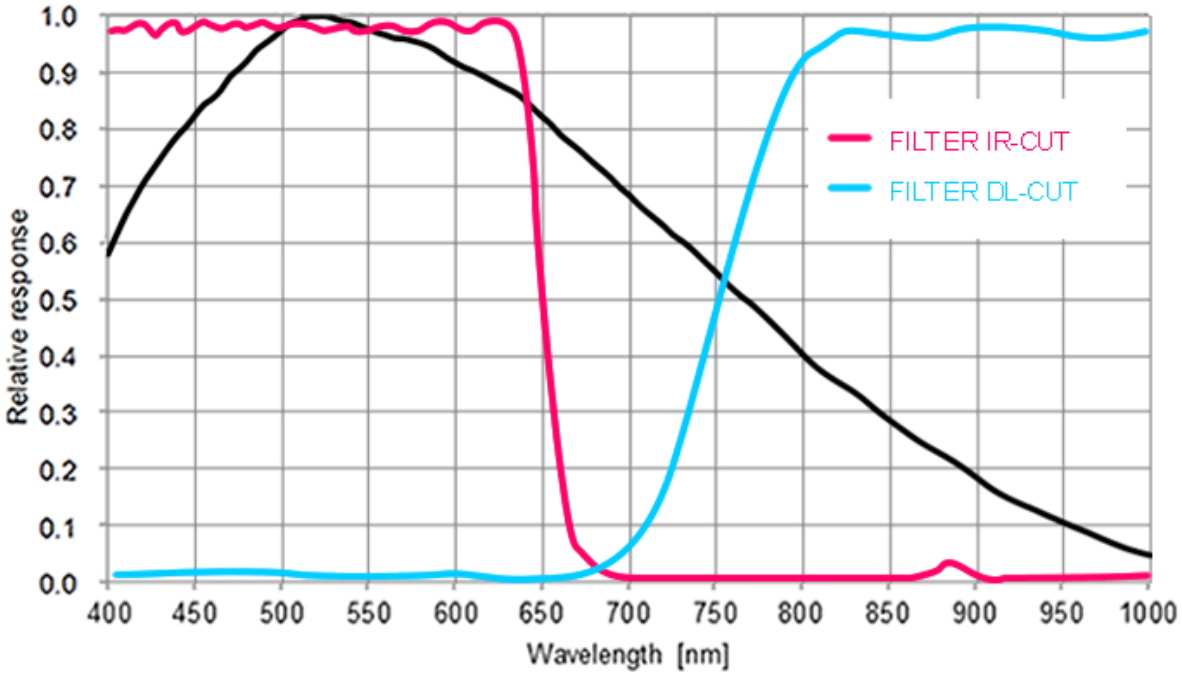


Figure 1: Spectral sensitivity mvBlueFOX3-2024aG

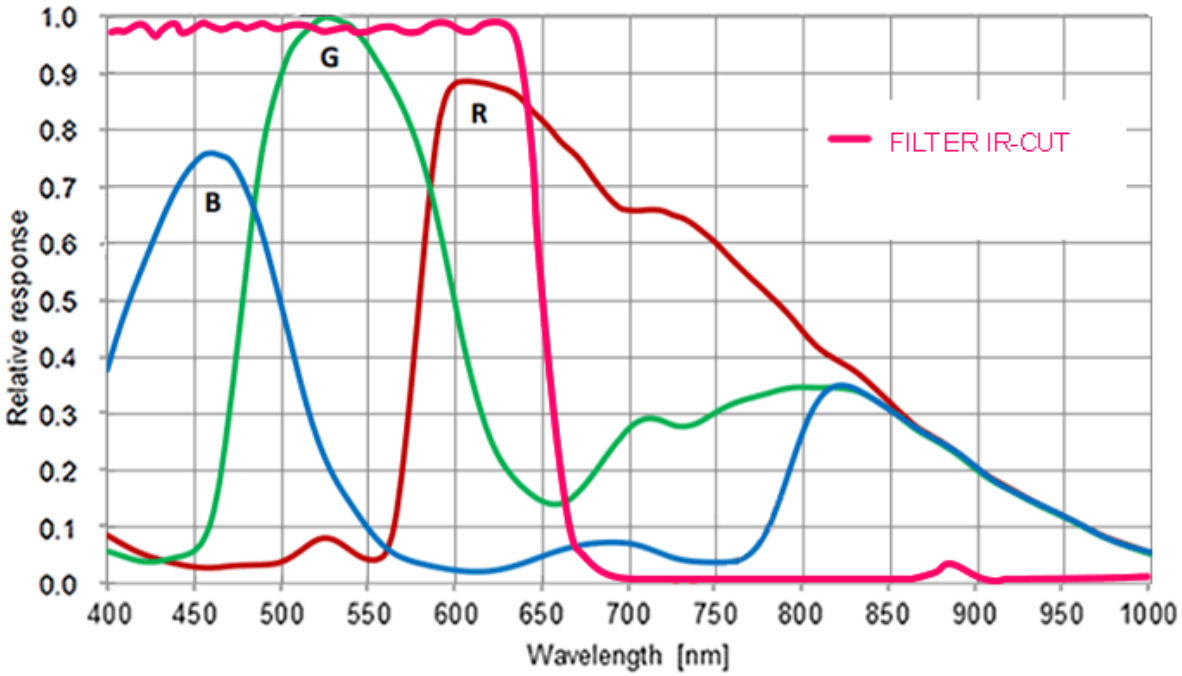


Figure 2: Spectral sensitivity mvBlueFOX3-2024aC

Name	Value
InternalLineLength	462
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	2

1.20.1.2.5.3 Timings

1.20.1.2.5.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.86 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 394)

1.20.1.2.5.5 Device Feature And Property List

- **mvBlueFOX3-2024aG / BF3-5M-0024AG Features** (p. 395)
- **mvBlueFOX3-2024aC / BF3-5M-0024AC Features** (p. 395)

1.20.1.2.5.6 mvBlueFOX3-2024aG / BF3-5M-0024AG Features

1.20.1.2.5.7 mvBlueFOX3-2024aC / BF3-5M-0024AC Features

1.20.1.2.6 BF3-5M-0024B (2.4 Mpix [1936 x 1216])

1.20.1.2.6.1 Introduction The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX392
Max. frames per second	161.4
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.4 / -
Image size	1/2.3
Number of effective pixels	1936 (H) x 1216 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.6.2 Spectral Sensitivity

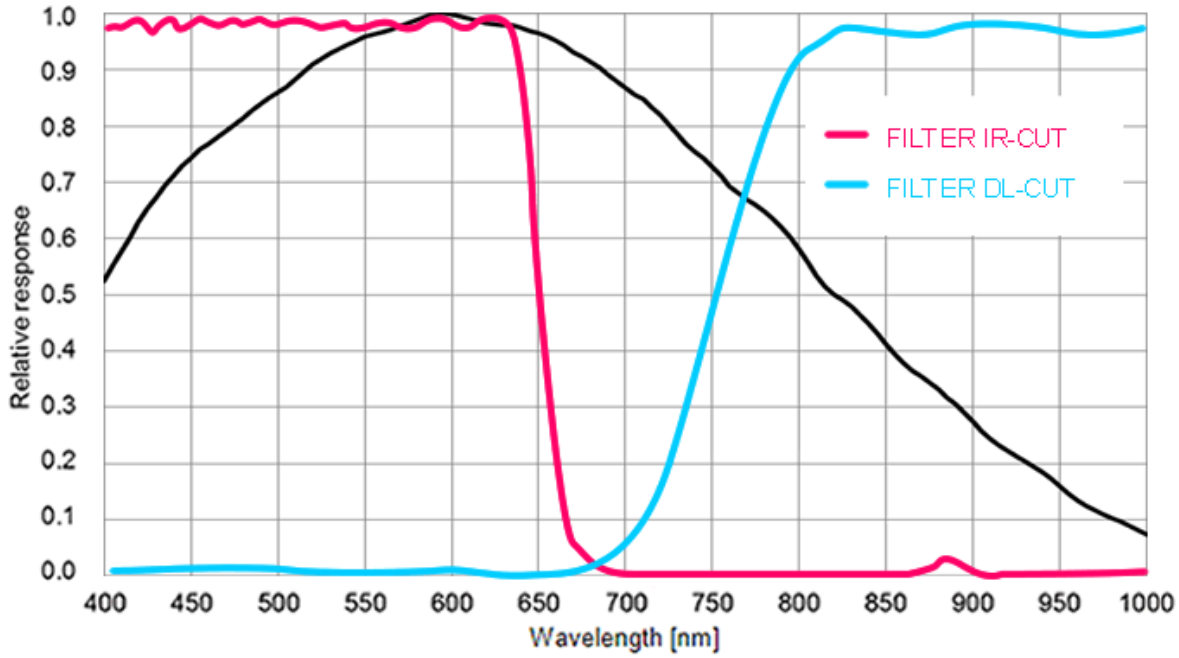


Figure 1: Spectral sensitivity BF3-5M-0024BG

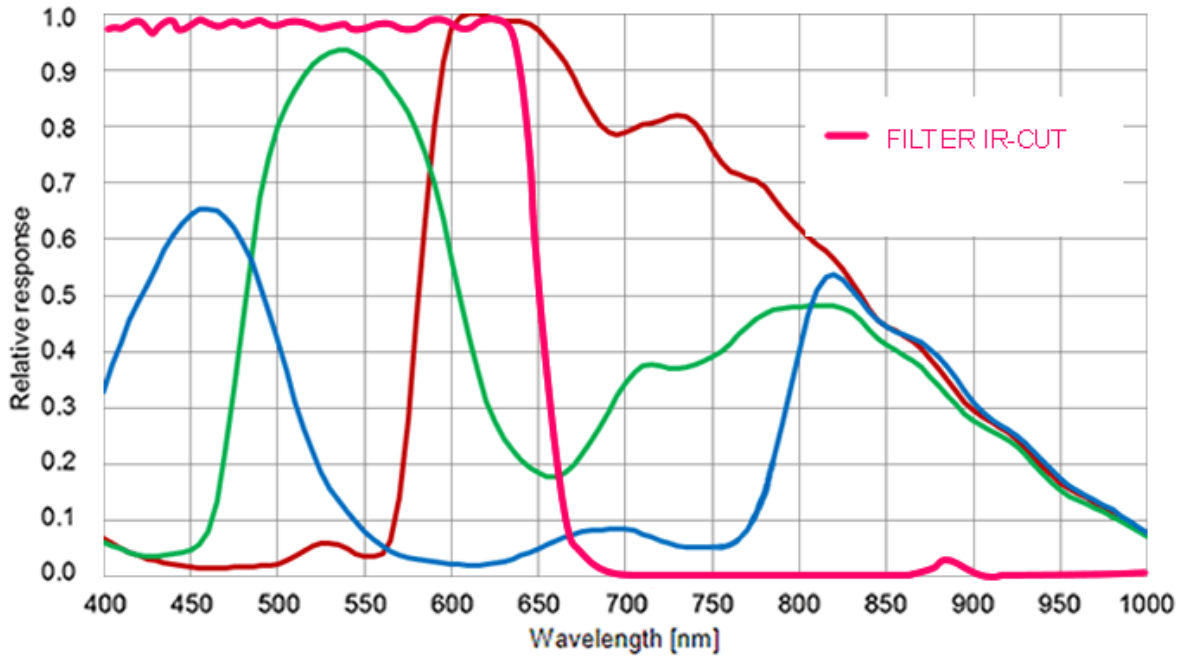


Figure 2: Spectral sensitivity BF3-5M-0024BC

Name	Value
InternalLineLength	355 (10 bit) / 441 (12 bit)
VerticalBlankLines	41
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.6.3 Timings

1.20.1.2.6.4 Free running mode In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.79 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 398)

1.20.1.2.6.5 Device Feature And Property List

- **BF3-5M-0024BG Features** (p. 398)
- **BF3-5M-0024BC Features** (p. 398)

1.20.1.2.6.6 BF3-5M-0024BG Features

1.20.1.2.6.7 BF3-5M-0024BC Features

1.20.1.2.7 mvBlueFOX3-2032 / BF3-5M-0032Z (3.2 Mpix [2064 x 1544])

1.20.1.2.7.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX252
Max. frames per second	123
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.3
DNR (normal / HDR) [dB][1/]	71.1 / -
Image size	1/1.8
Number of effective pixels	2064 (H) x 1544 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.7.2 Spectral Sensitivity

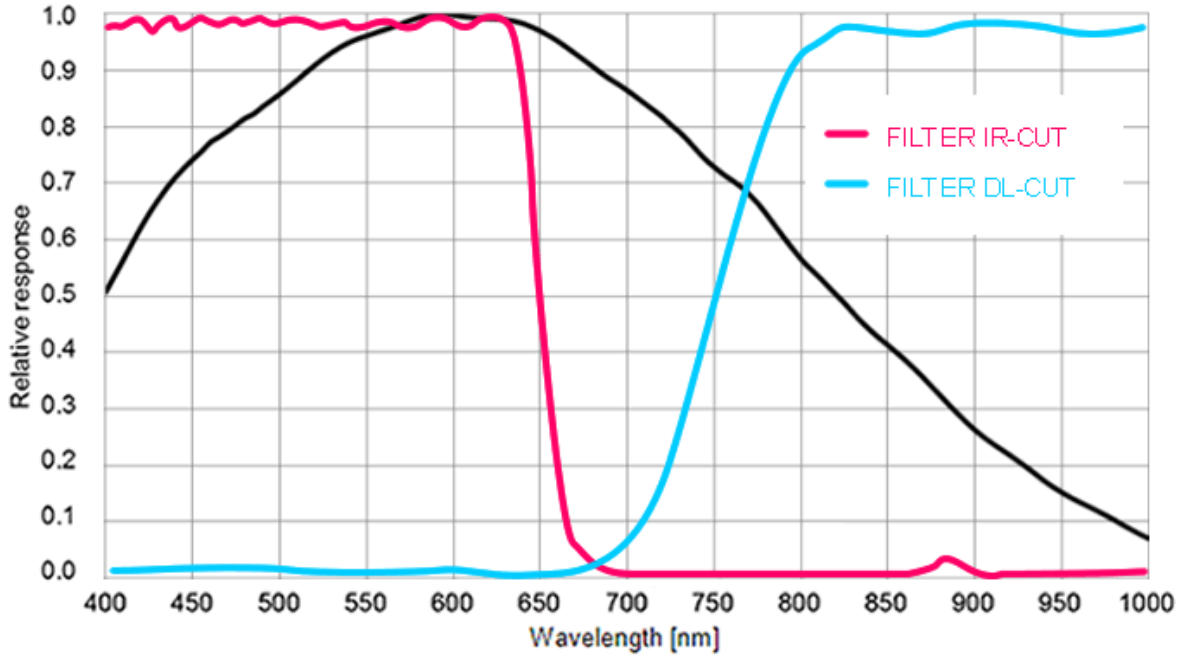


Figure 1: Spectral sensitivity mvBlueFOX3-2032G

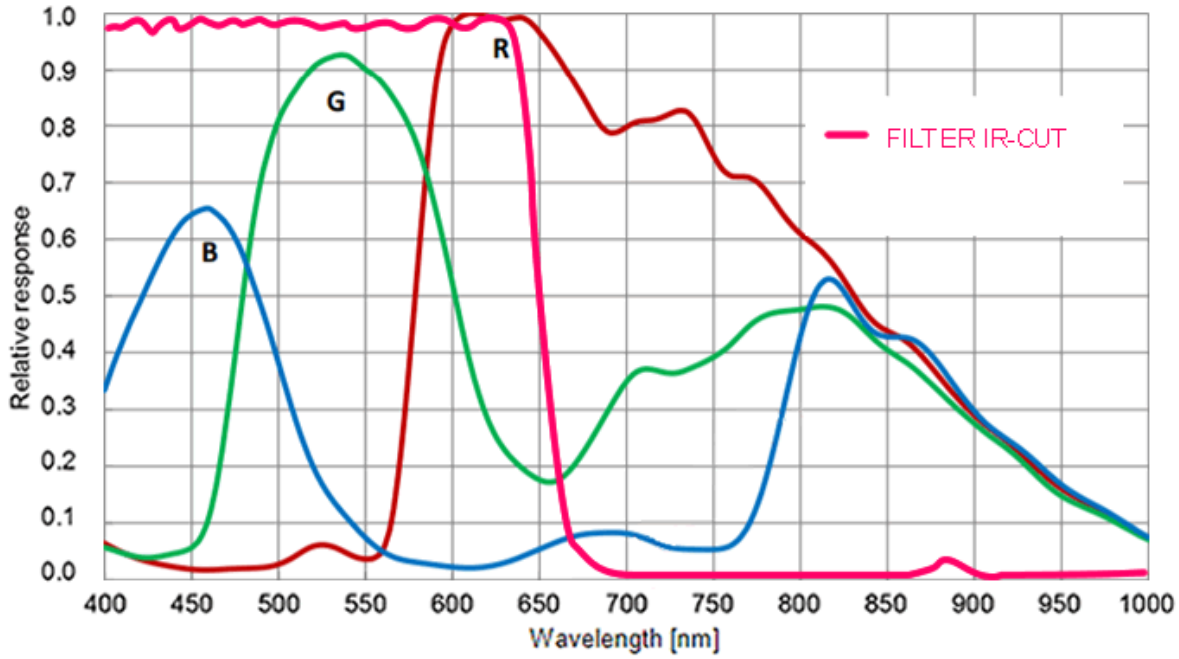


Figure 2: Spectral sensitivity mvBlueFOX3-2032C

Name	Value
InternalLineLength	380 (10 bit) / 444 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.7.3 Timings

1.20.1.2.7.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 5.13 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed"	OnDemand	Image acquisition triggered by command (software trigger).
To trigger one frame execute the *TriggerSoftware@i* command then.		

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 401)

1.20.1.2.7.5 Device Feature And Property List

- **mvBlueFOX3-2032G / BF3-5M-0032ZG Features** (p. 401)
- **mvBlueFOX3-2032C / BF3-5M-0032ZC Features** (p. 401)

1.20.1.2.7.6 mvBlueFOX3-2032G / BF3-5M-0032ZG Features

1.20.1.2.7.7 mvBlueFOX3-2032C / BF3-5M-0032ZC Features

1.20.1.2.8 mvBlueFOX3-2032a / BF3-5M-0032A (3.2 Mpix [2064 x 1544])

1.20.1.2.8.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX265
Max. frames per second	55
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.3 / -
Image size	1/1.8
Number of effective pixels	2064 (H) x 1544 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.8.2 Spectral Sensitivity

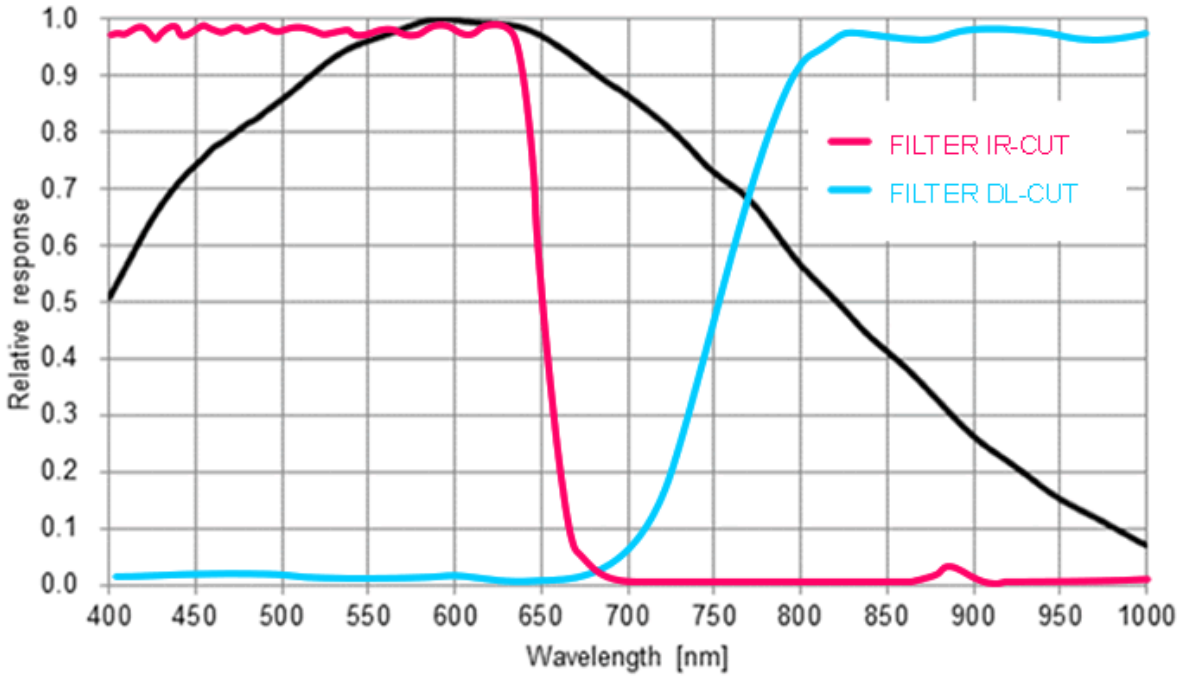


Figure 1: Spectral sensitivity mvBlueFOX3-2032aG

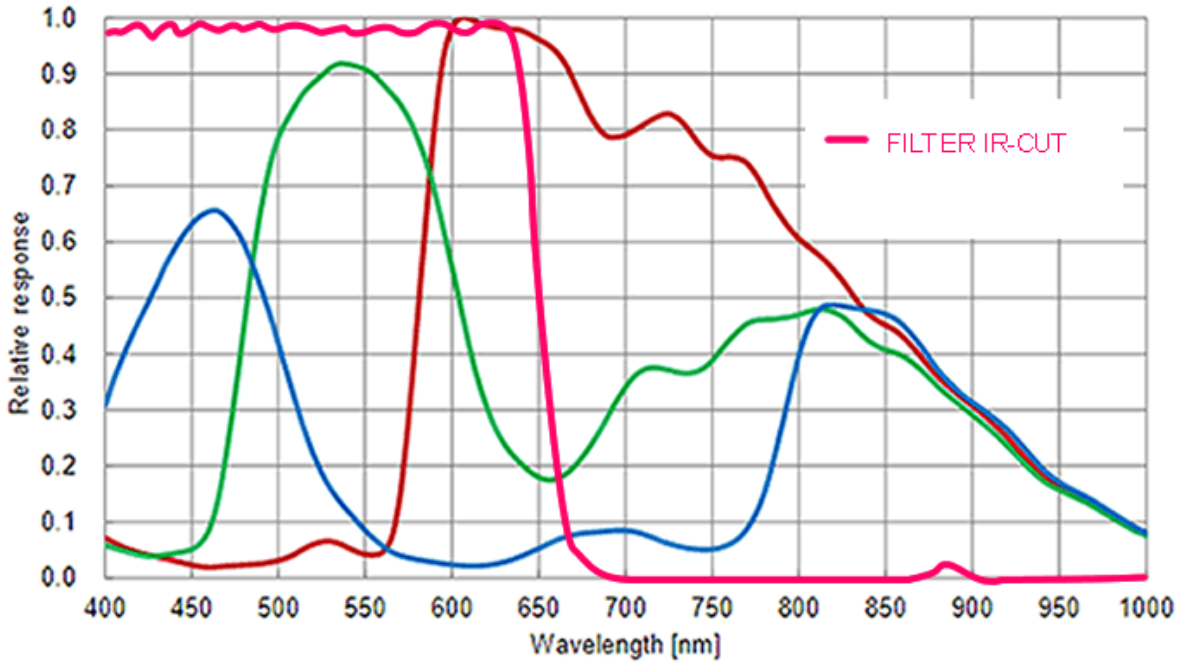


Figure 2: Spectral sensitivity mvBlueFOX3-2032aC

Name	Value
InternalLineLength	423
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4

1.20.1.2.8.3 Timings

1.20.1.2.8.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 11.4 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 405)

1.20.1.2.8.5 Device Feature And Property List

- **mvBlueFOX3-2032aG / BF3-5M-0032AG Features** (p. 405)
- **mvBlueFOX3-2032aC / BF3-5M-0032AC Features** (p. 405)

1.20.1.2.8.6 mvBlueFOX3-2032aG / BF3-5M-0032AG Features

1.20.1.2.8.7 mvBlueFOX3-2032aC / BF3-5M-0032AC Features

1.20.1.2.9 mvBlueFOX3-2051 / BF3-5M-0024Z (5.1 Mpix [2464 x 2056])

1.20.1.2.9.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX250
Max. frames per second	80
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.3
DNR (normal / HDR) [dB][1/]	71.2 / -
Image size	2/3
Number of effective pixels	2464 (H) x 2056 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.9.2 Spectral Sensitivity

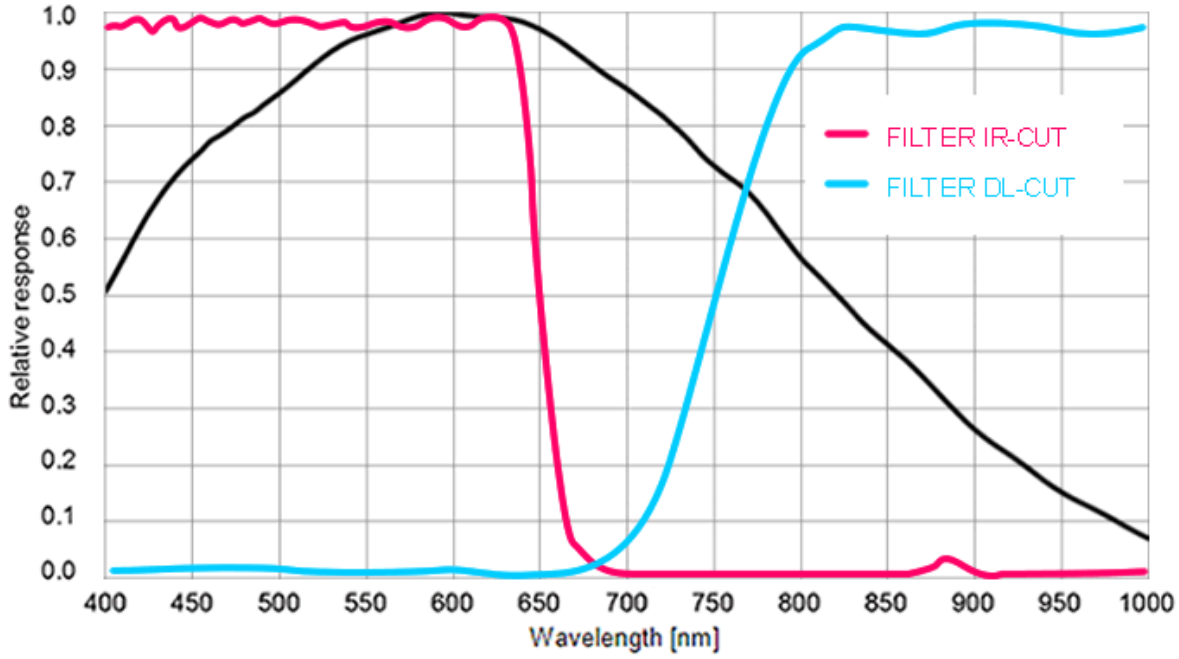


Figure 1: Spectral sensitivity mvBlueFOX3-2051G

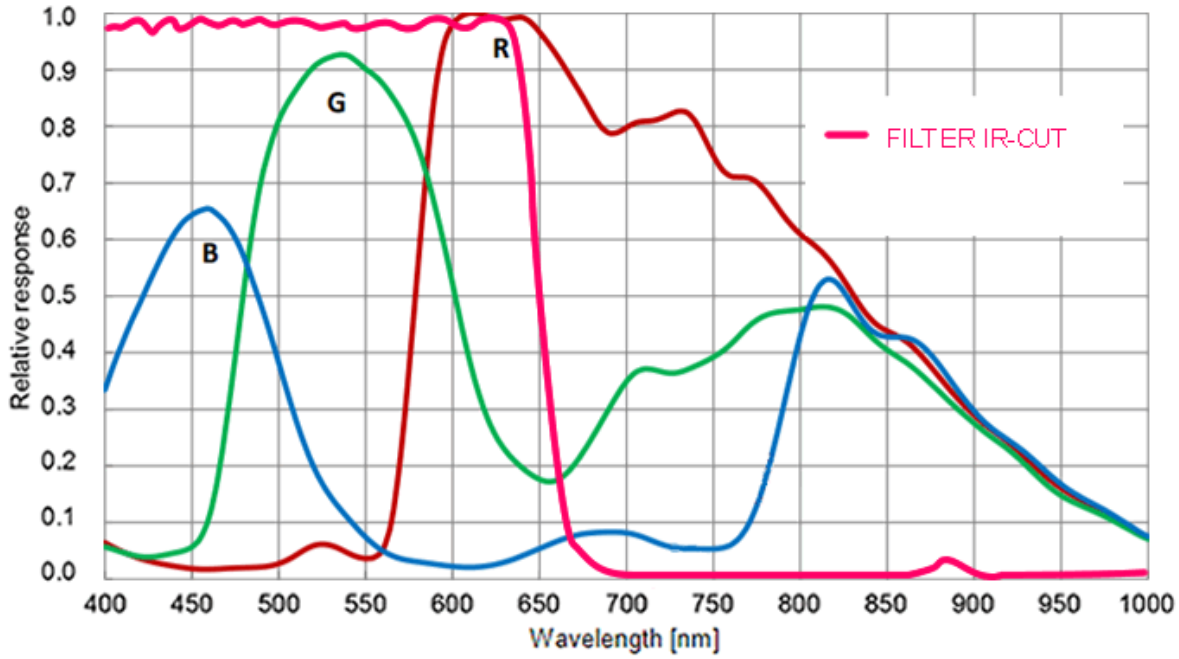


Figure 2: Spectral sensitivity mvBlueFOX3-2051C

Name	Value
InternalLineLength	440 (10 bit) / 519 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	8

1.20.1.2.9.3 Timings

1.20.1.2.9.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 5.94 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 408)

1.20.1.2.9.5 Device Feature And Property List

- **mvBlueFOX3-2051G / BF3-5M-0051ZG Features** (p. 409)
- **mvBlueFOX3-2051C / / BF3-5M-0051ZC Features** (p. 409)

1.20.1.2.9.6 mvBlueFOX3-2051G / BF3-5M-0051ZG Features

1.20.1.2.9.7 mvBlueFOX3-2051C / / BF3-5M-0051ZC Features

1.20.1.2.10 mvBlueFOX3-2051a / BF3-5M-0051A (5.1 Mpix [2464 x 2056])

1.20.1.2.10.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX264
Max. frames per second	35
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.1
DNR (normal / HDR) [dB][1/]	71.3 / -
Image size	2/3
Number of effective pixels	2464 (H) x 2056 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.10.2 Spectral Sensitivity

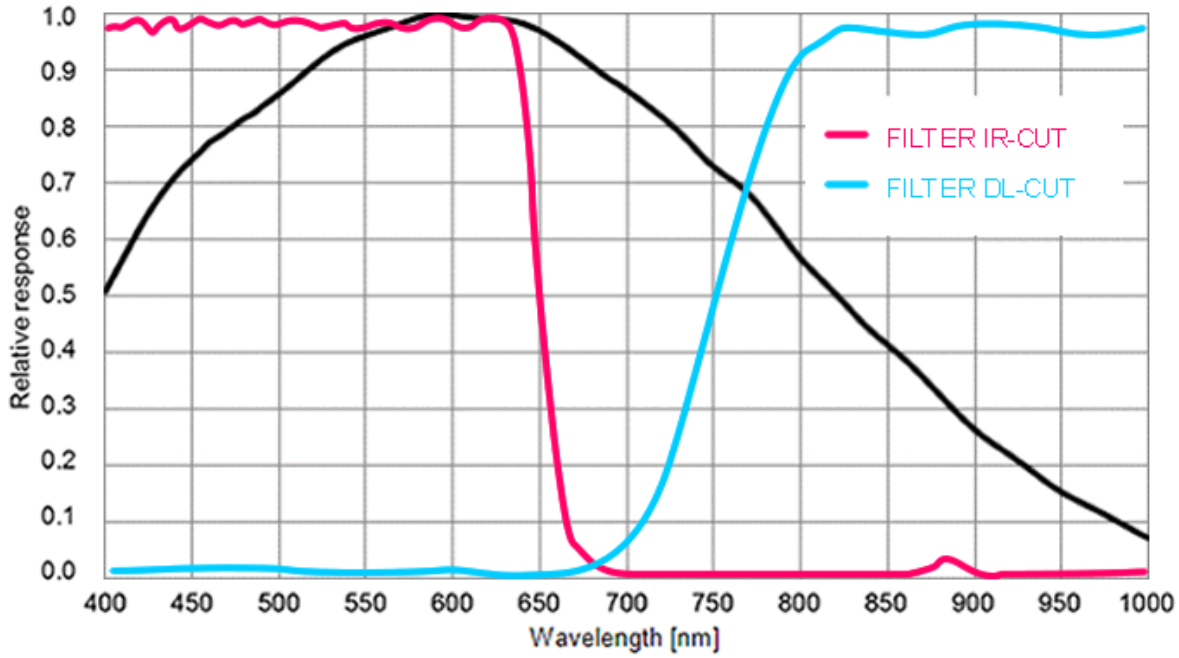


Figure 1: Spectral sensitivity mvBlueFOX3-2051aG

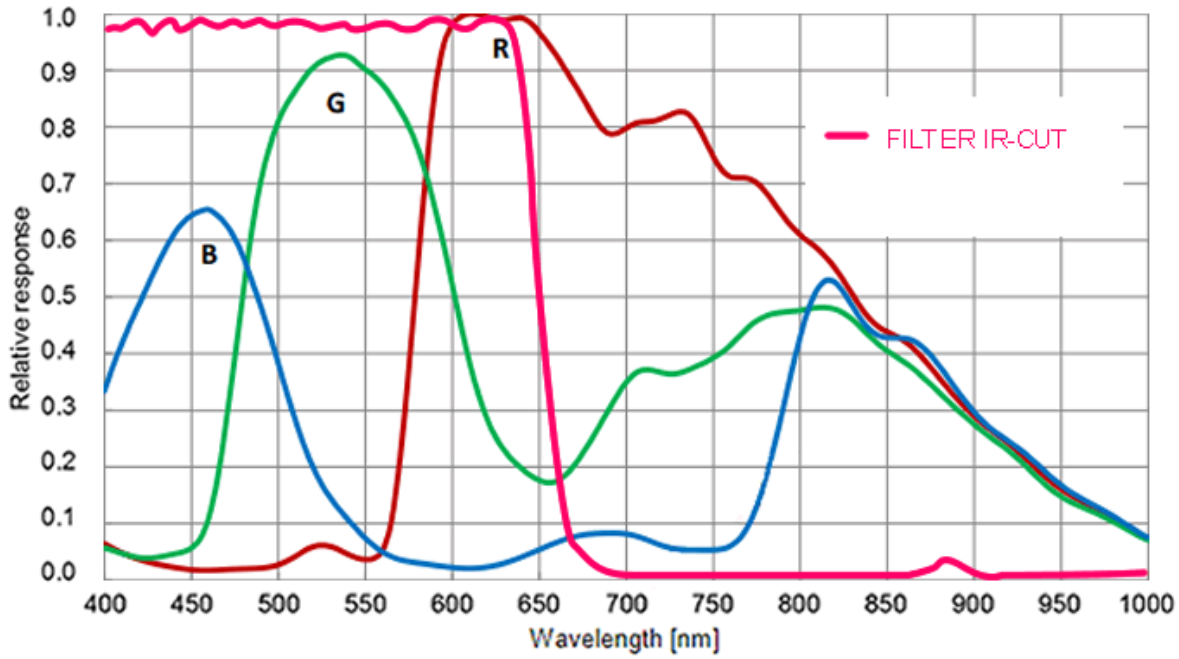


Figure 2: Spectral sensitivity mvBlueFOX3-2051aC

Name	Value
InternalLineLength	498
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.10.3 Timings

1.20.1.2.10.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 13.4 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 412)

1.20.1.2.10.5 Device Feature And Property List

- **mvBlueFOX3-2051aG / BF3-5M-0051AG Features** (p. 412)
- **mvBlueFOX3-2051aC / BF3-5M-0051AC Features** (p. 412)

1.20.1.2.10.6 mvBlueFOX3-2051aG / BF3-5M-0051AG Features

1.20.1.2.10.7 mvBlueFOX3-2051aC / BF3-5M-0051AC Features

1.20.1.2.11 mvBlueFOX3-2071 (7.1 Mpix [3216 x 2208])

1.20.1.2.11.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX420
Max. frames per second	53.5
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	43.8
DNR (normal / HDR) [dB][1/]	71.7 / -
Image size	1.1
Number of effective pixels	3216 (H) x 2208 (V)
Unit cell size	4.5µm (H) x 4.5µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.11.2 Spectral Sensitivity

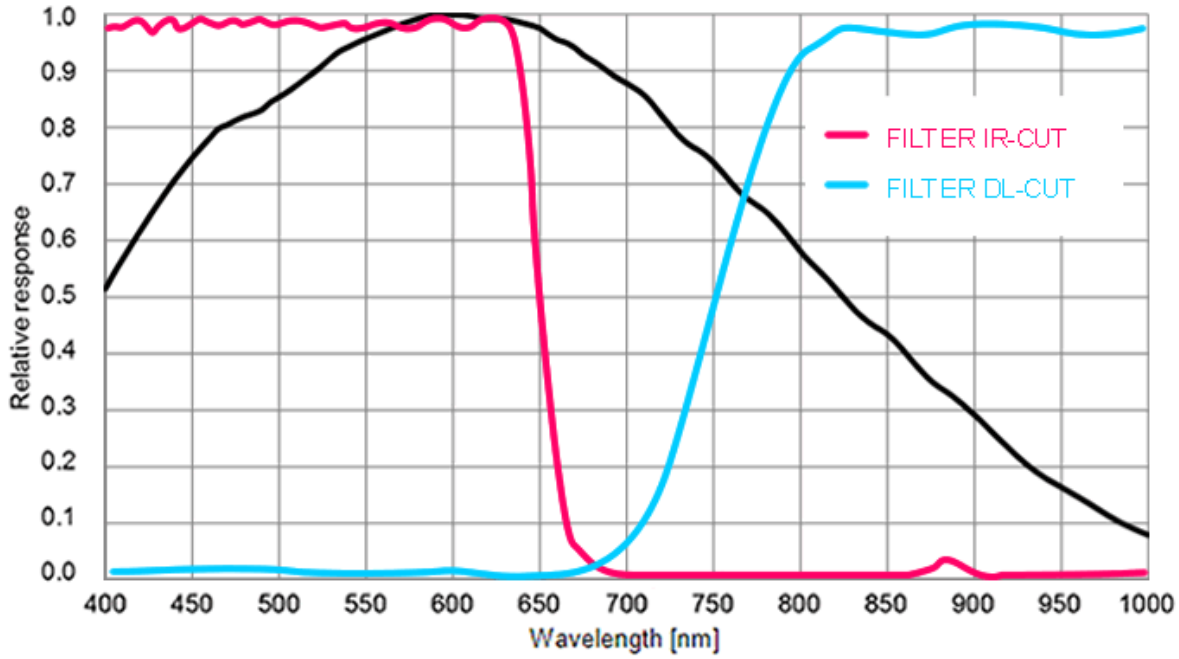


Figure 1: Spectral sensitivity mvBlueFOX3-2071G

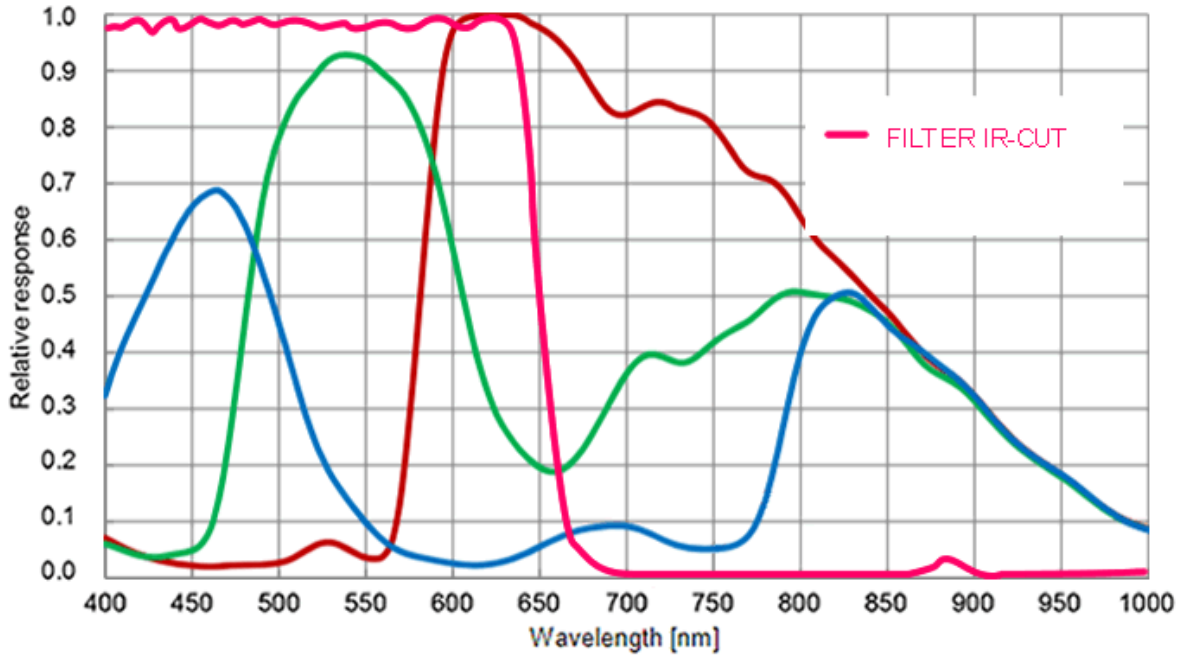


Figure 2: Spectral sensitivity mvBlueFOX3-2071C

Name	Value
InternalLineLength	390
VerticalBlankLines	75
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.2.11.3 Timings

1.20.1.2.11.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 14.02 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 415)

1.20.1.2.11.5 Device Feature And Property List

- **mvBlueFOX3-2071G Features** (p. 416)
- **mvBlueFOX3-2071C Features** (p. 416)

1.20.1.2.11.6 mvBlueFOX3-2071G Features

1.20.1.2.11.7 mvBlueFOX3-2071C Features

1.20.1.2.12 mvBlueFOX3-2071a (7.1 Mpix [3216 x 2208])

1.20.1.2.12.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX428
Max. frames per second	50.7
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	43.9
DNR (normal / HDR) [dB][1/]	71.8 / -
Image size	1.1
Number of effective pixels	3216 (H) x 2208 (V)
Unit cell size	4.5µm (H) x 4.5µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.12.2 Spectral Sensitivity

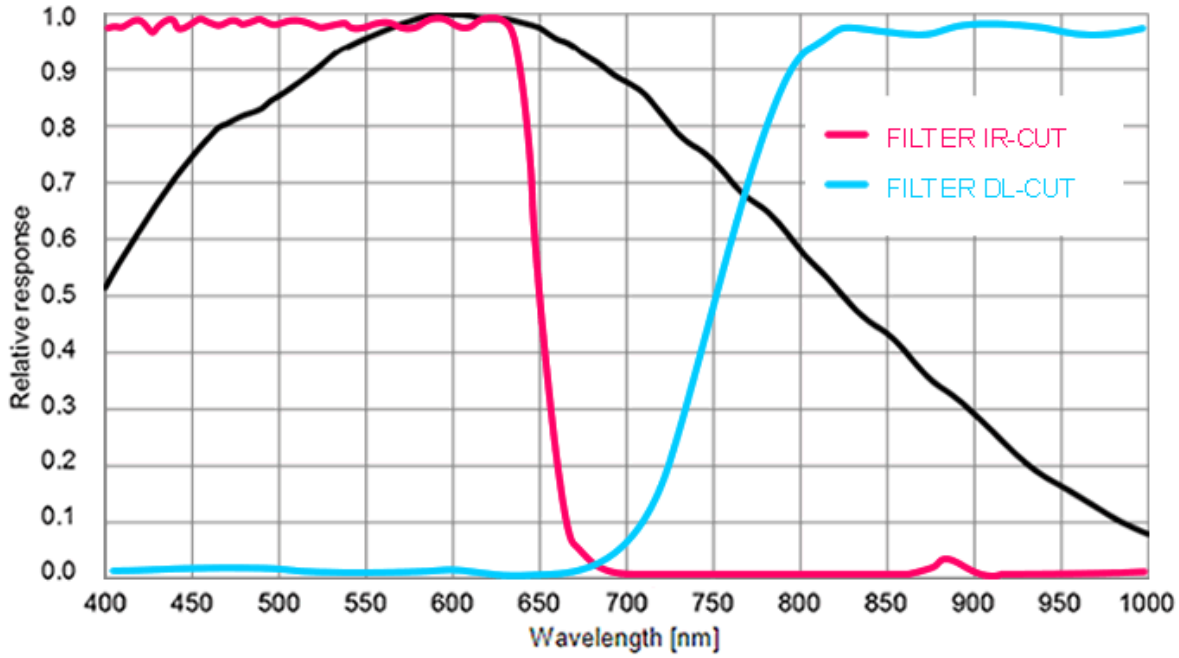


Figure 1: Spectral sensitivity mvBlueFOX3-2071aG

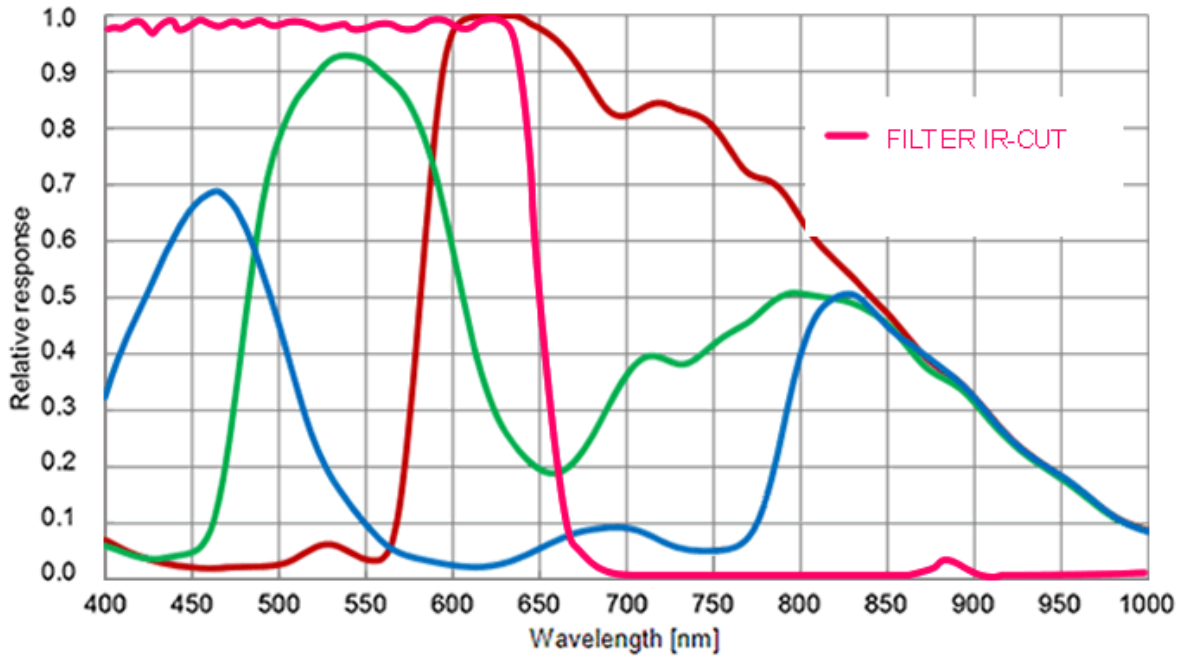


Figure 2: Spectral sensitivity mvBlueFOX3-2071aC

Name	Value
InternalLineLength	466
VerticalBlankLines	75
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.2.12.3 Timings

1.20.1.2.12.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 8.6 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 419)

1.20.1.2.12.5 Device Feature And Property List

- **mvBlueFOX3-2071aG Features** (p. 419)
- **mvBlueFOX3-2071aC Features** (p. 419)

1.20.1.2.12.6 mvBlueFOX3-2071aG Features

1.20.1.2.12.7 mvBlueFOX3-2071aC Features

1.20.1.2.13 mvBlueFOX3-2089 / BF3-5M-0089Z (8.9 Mpix [4112 x 2176])

1.20.1.2.13.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX255
Max. frames per second	47
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.1 / -
Image size	1
Number of effective pixels	4112 (H) x 2176 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Note

Max. AOI width of color version is 4096 pixels.

1.20.1.2.13.2 Spectral Sensitivity

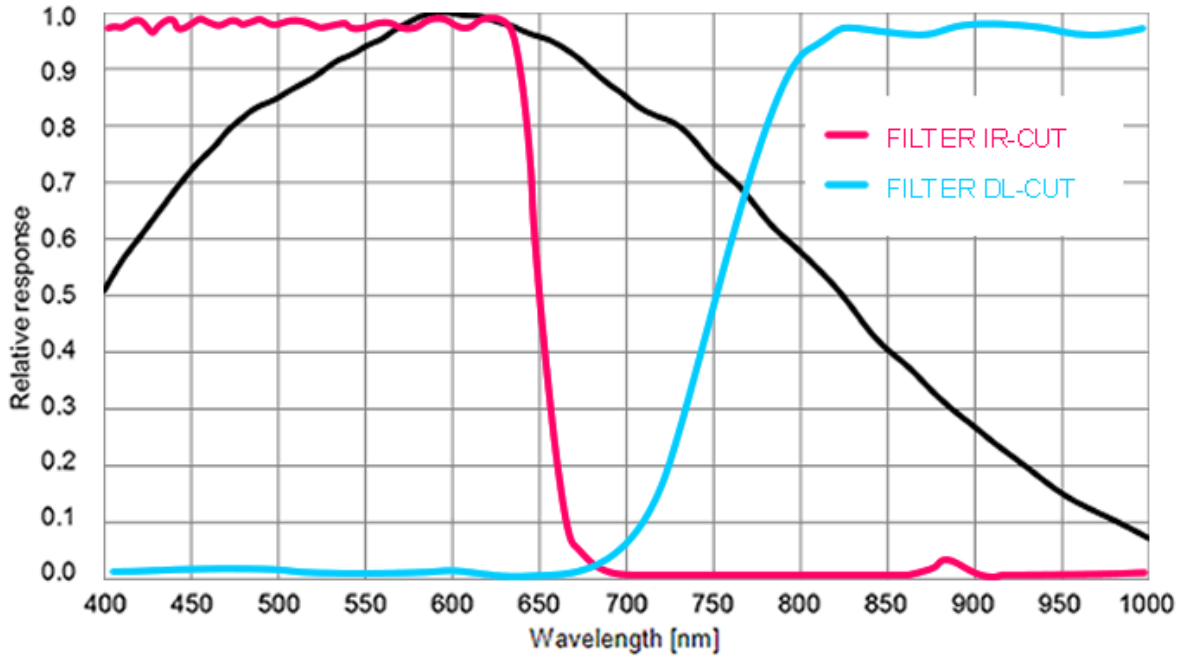


Figure 1: Spectral sensitivity mvBlueFOX3-2089G

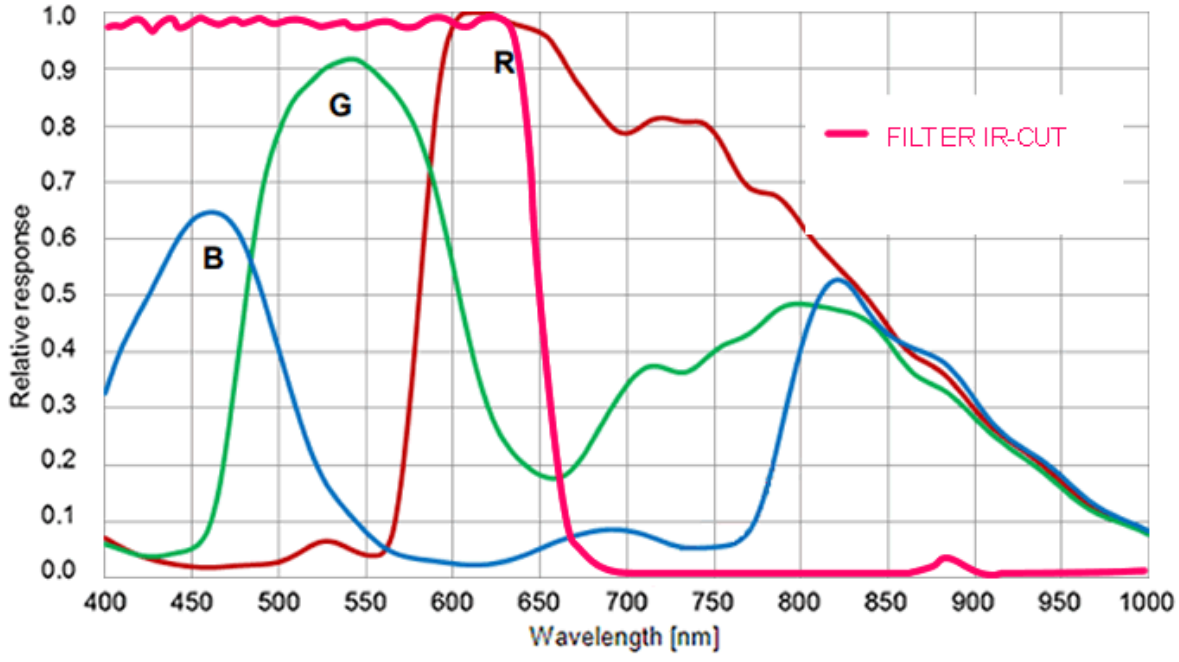


Figure 2: Spectral sensitivity mvBlueFOX3-2089C

Name	Value
InternalLineLength	700 (10 bit) / 831 (12 bit)
VerticalBlankLines	54
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.13.3 Timings

1.20.1.2.13.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 9.45 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Features And Property Lists (p. 423)

1.20.1.2.13.5 Device Features And Property Lists

- **mvBlueFOX3-2089G / BF3-5M-0089ZG Features** (p. 423)
- **mvBlueFOX3-2089C / BF3-5M-0089ZC Features** (p. 423)

1.20.1.2.13.6 mvBlueFOX3-2089G / BF3-5M-0089ZG Features

1.20.1.2.13.7 mvBlueFOX3-2089C / BF3-5M-0089ZC Features

1.20.1.2.14 mvBlueFOX3-2089a / BF3-5M-0089A (8.9 Mpix [4112 x 2176])

1.20.1.2.14.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX267
Max. frames per second	32
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71 / -
Image size	1
Number of effective pixels	4112 (H) x 2176 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Note

Max. AOI width of color version is 4096 pixels.

1.20.1.2.14.2 Spectral Sensitivity

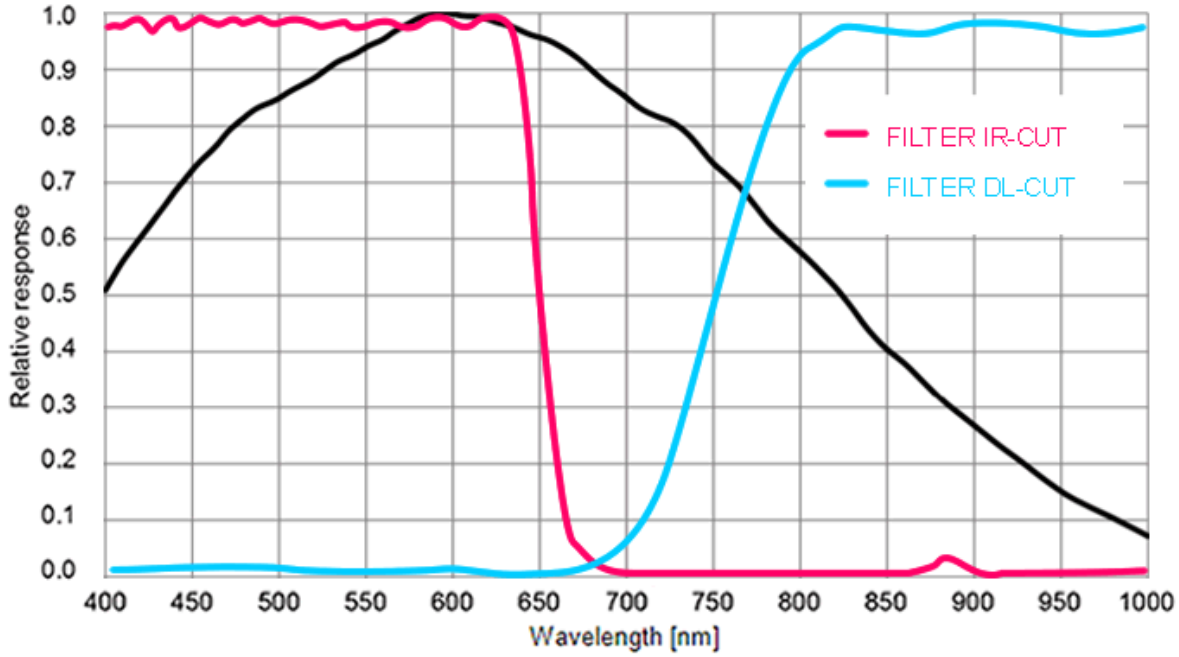


Figure 1: Spectral sensitivity mvBlueFOX3-2089aG

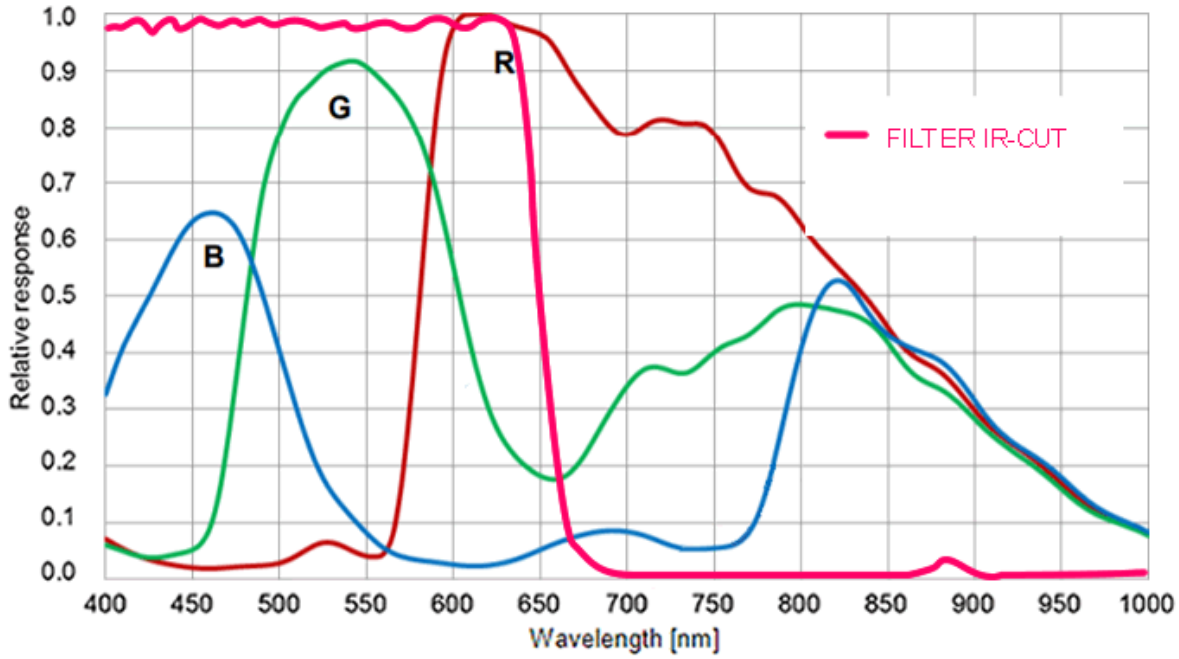


Figure 2: Spectral sensitivity mvBlueFOX3-2089aC

Name	Value
InternalLineLength	1041
VerticalBlankLines	34
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.14.3 Timings

1.20.1.2.14.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 14.05 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 426)

1.20.1.2.14.5 Device Feature And Property List

- **mvBlueFOX3-2089aG / BF3-5M-0089AG Features** (p. 426)
- **mvBlueFOX3-2089aC / BF3-5M-0089AC Features** (p. 426)

1.20.1.2.14.6 mvBlueFOX3-2089aG / BF3-5M-0089AG Features

1.20.1.2.14.7 mvBlueFOX3-2089aC / BF3-5M-0089AC Features

1.20.1.2.15 mvBlueFOX3-2124 / BF3-5M-0124Z (12.4 Mpix [4112 x 3008])

1.20.1.2.15.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX253
Max. frames per second	34.6
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	70.9 / -
Image size	1.1
Number of effective pixels	4112 (H) x 3008 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Note

Max. AOI width of color version is 4096 pixels.

1.20.1.2.15.2 Spectral Sensitivity

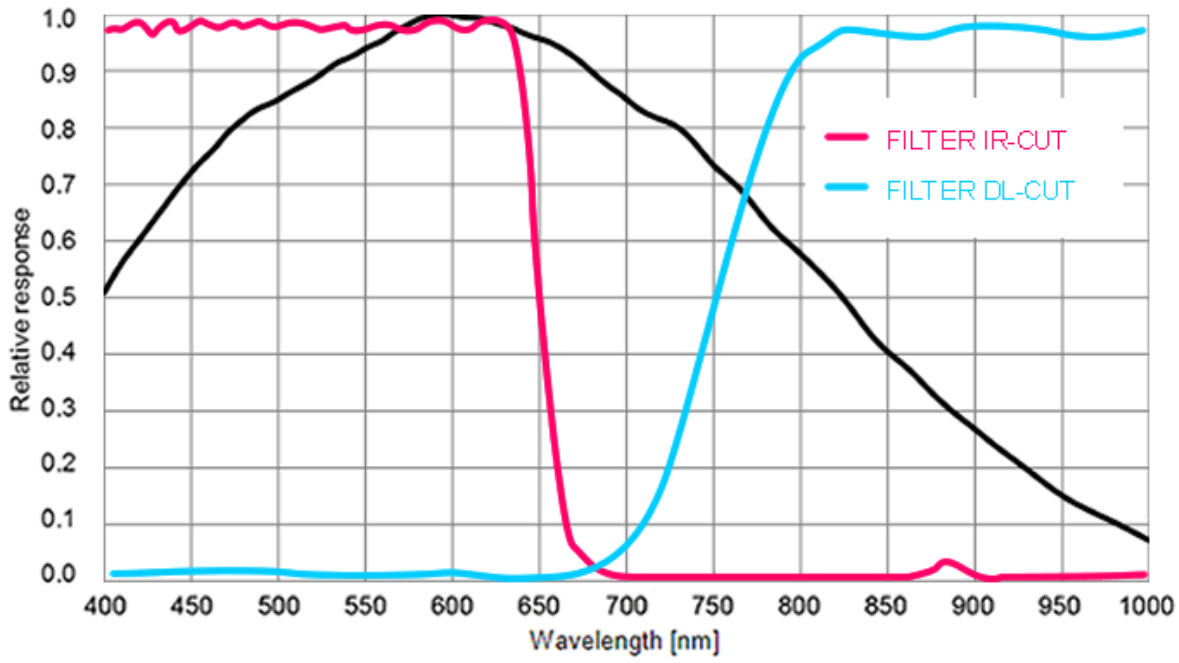


Figure 1: Spectral sensitivity mvBlueFOX3-2124G

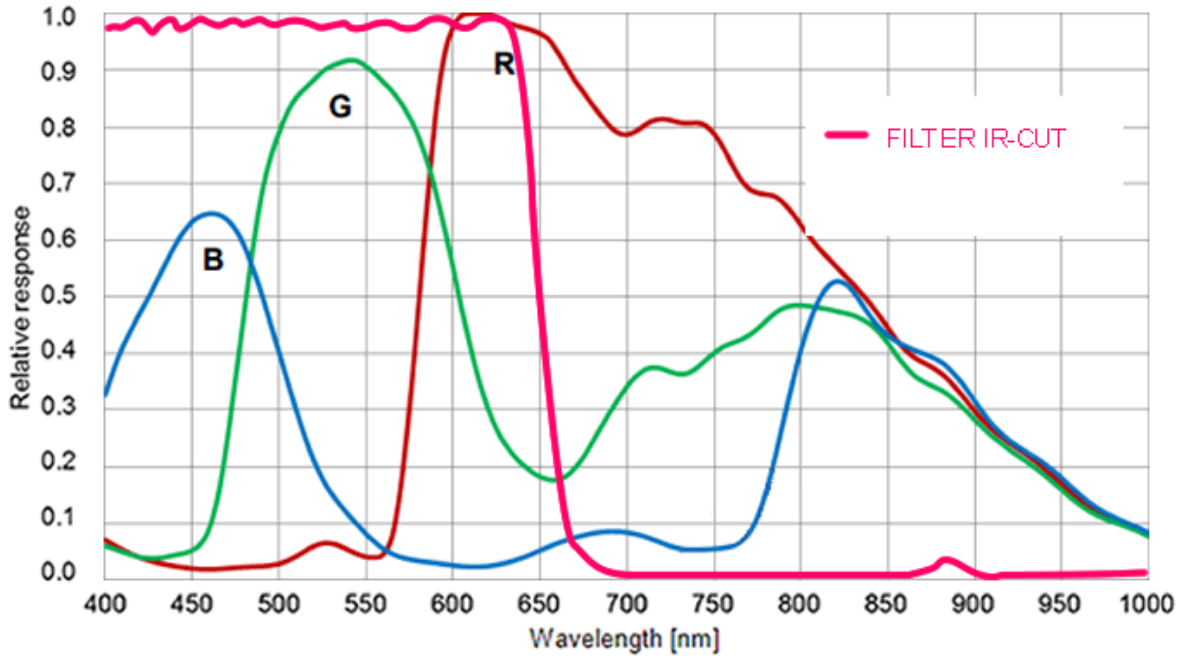


Figure 2: Spectral sensitivity mvBlueFOX3-2124C

Name	Value
InternalLineLength	700 (10 bit) / 831 (12 bit)
VerticalBlankLines	54
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.15.3 Timings

1.20.1.2.15.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 9.45 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 430)

1.20.1.2.15.5 Device Feature And Property List

- **mvBlueFOX3-2124G / BF3-5M-0124ZG Features** (p. 430)
- **mvBlueFOX3-2124C / BF3-5M-0124ZC Features** (p. 430)

1.20.1.2.15.6 mvBlueFOX3-2124G / BF3-5M-0124ZG Features

1.20.1.2.15.7 mvBlueFOX3-2124C / BF3-5M-0124ZC Features

1.20.1.2.16 mvBlueFOX3-2124a / BF3-5M-0124A (12.4 Mpix [4112 x 3008])

1.20.1.2.16.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX304
Max. frames per second	23
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.0 / -
Image size	1.1
Number of effective pixels	4112 (H) x 3008 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Note

Max. AOI width of color version is 4096 pixels.

1.20.1.2.16.2 Spectral Sensitivity

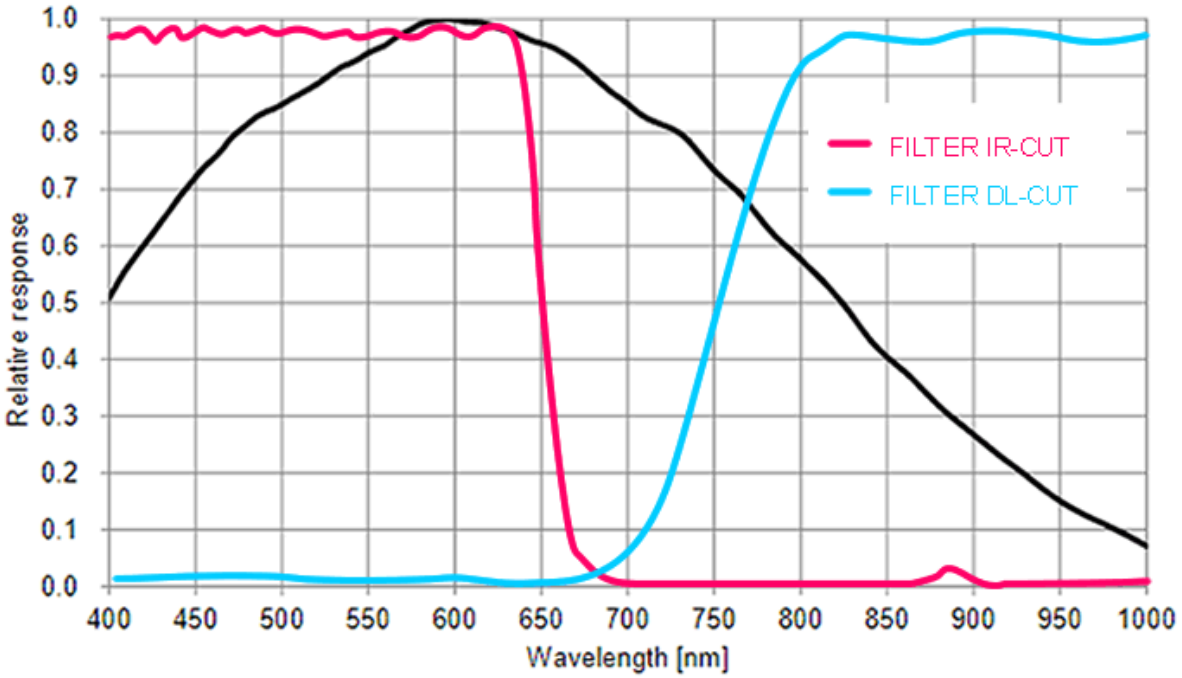


Figure 1: Spectral sensitivity mvBlueFOX3-2124aG

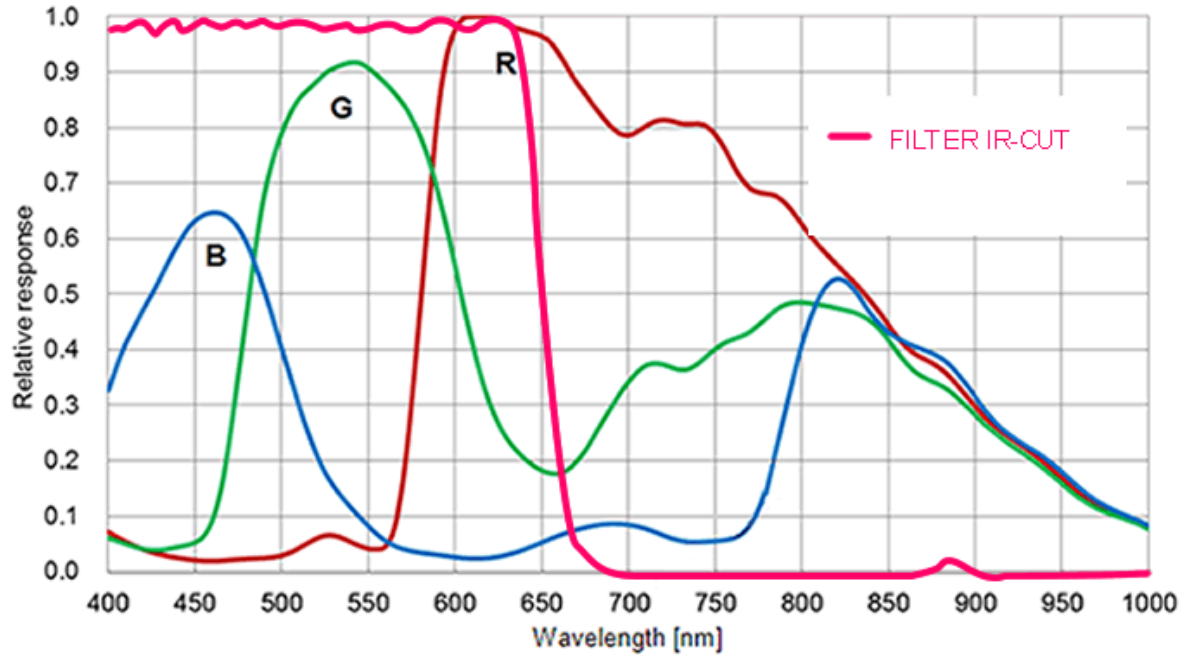


Figure 2: Spectral sensitivity mvBlueFOX3-2124aC

Name	Value
InternalLineLength	1041
VerticalBlankLines	34
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.1.2.16.3 Timings

1.20.1.2.16.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 14.05 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 434)

1.20.1.2.16.5 Device Feature And Property List

- **mvBlueFOX3-2124aG / BF3-5M-0124AG Features** (p. 434)
- **mvBlueFOX3-2124aC / BF3-5M-0124AC Features** (p. 434)

1.20.1.2.16.6 mvBlueFOX3-2124aG / BF3-5M-0124AG Features

1.20.1.2.16.7 mvBlueFOX3-2124aC / BF3-5M-0124AC Features

1.20.1.2.17 BF3-4-0169Z / BF3-5M-0169Z (16.9 Mpix [5472 x 3080])

1.20.1.2.17.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX387
Max. frames per second	22.5
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	70.8 / -
Image size	4/3"
Number of effective pixels	5472 (H) x 3084 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.17.2 Spectral Sensitivity

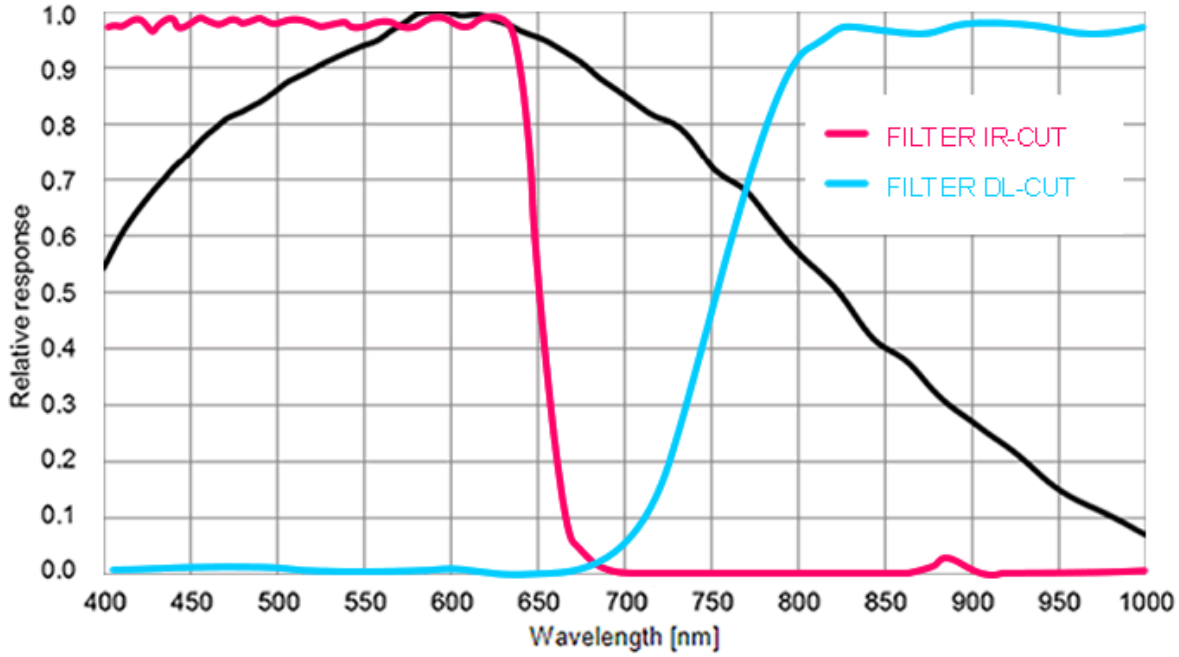


Figure 1: Spectral sensitivity BF3-4-0169Z G

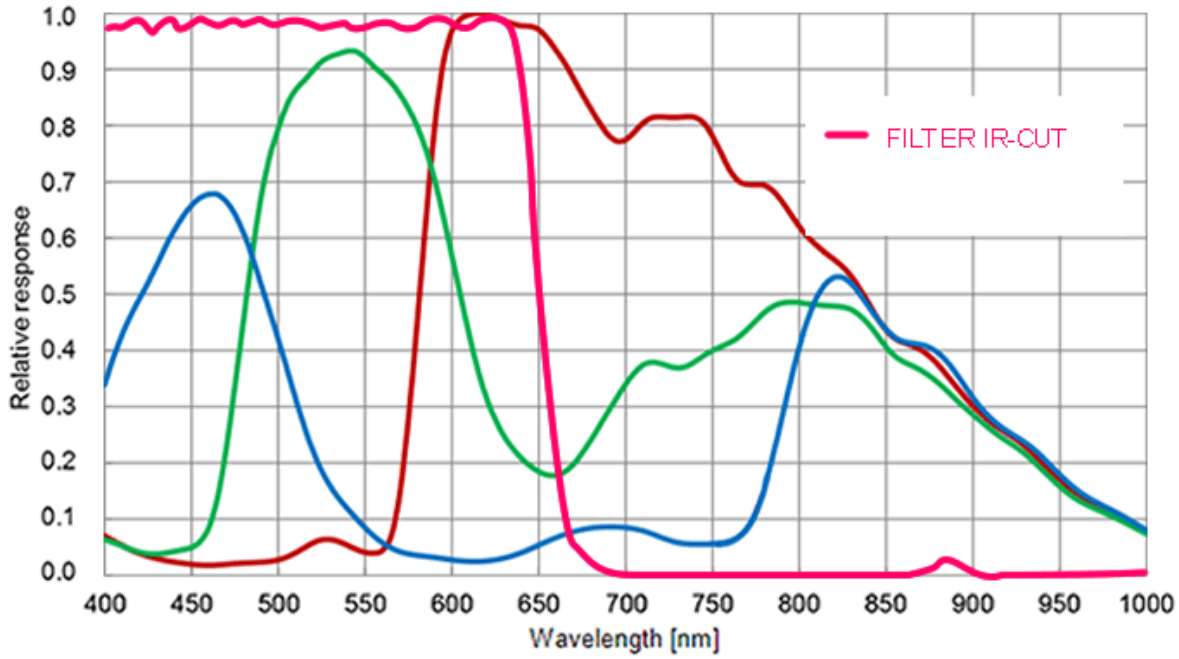


Figure 2: Spectral sensitivity BF3-4-0169Z C

Name	Value
InternalLineLength	648 (10 bit) / 773 (12 bit)
VerticalBlankLines	55
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.2.17.3 Timings

1.20.1.2.17.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 12.01 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 437)

1.20.1.2.17.5 Device Feature And Property List

- **BF3-4-0169ZG / BF3-5-0169ZG Features** (p. 437)
- **BF3-4-0169ZC / BF3-5-0169ZC Features** (p. 437)

1.20.1.2.17.6 BF3-4-0169ZG / BF3-5-0169ZG Features

1.20.1.2.17.7 BF3-4-0169ZC / BF3-5-0169ZC Features

1.20.1.2.18 BF3-4-0196Z / BF3-5M-0196Z (19.6 Mpix [4432 x 4432])

1.20.1.2.18.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX367
Max. frames per second	19.3
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.1
DNR (normal / HDR) [dB][1/]	70.7 / -
Image size	4/3"
Number of effective pixels	4432 (H) x 4436 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.18.2 Spectral Sensitivity

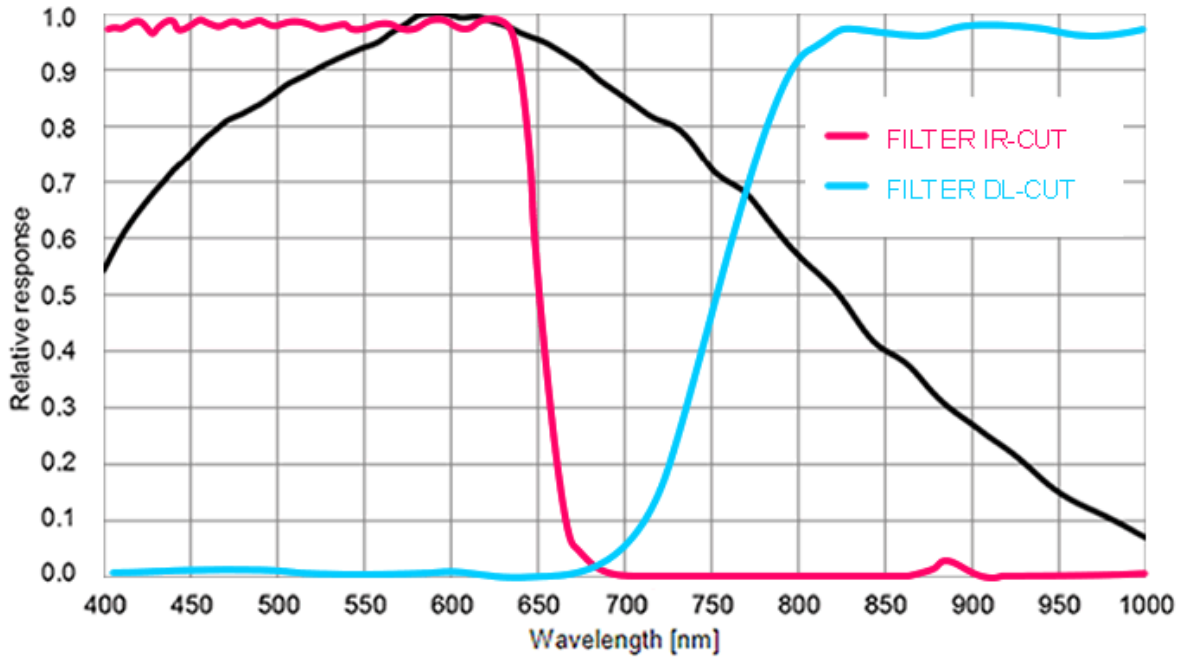


Figure 1: Spectral sensitivity BF3-4-0196Z G

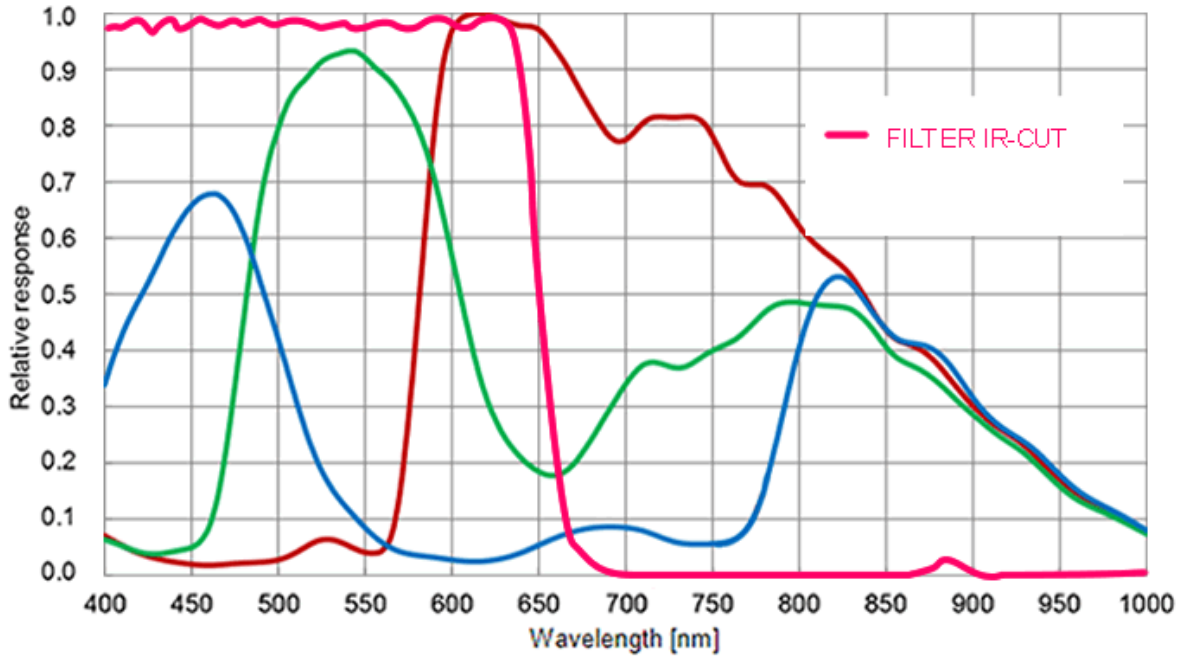


Figure 2: Spectral sensitivity BF3-4-0196Z C

Name	Value
InternalLineLength	528 (10 bit) / 630 (12 bit)
VerticalBlankLines	61
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.2.18.3 Timings

1.20.1.2.18.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 9.78 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 441)

1.20.1.2.18.5 Device Feature And Property List

- **BF3-4-0196ZG / BF3-5-0196ZG Features** (p. 441)
- **BF3-4-0196ZC / BF3-5-0196ZC Features** (p. 441)

1.20.1.2.18.6 BF3-4-0196ZG / BF3-5-0196ZG Features

1.20.1.2.18.7 BF3-4-0196ZC / BF3-5-0196ZC Features

1.20.1.2.19 BF3-4-0315Z / BF3-5M-0315Z (31.5 Mpix [6480 x 4856])

1.20.1.2.19.1 Introduction

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Sony
Sensor name	IMX342
Max. frames per second	12.1
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	39.7
DNR (normal / HDR) [dB][1/]	69.8 / -
Image size	ASP-C
Number of effective pixels	6480 (H) x 4856 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.1.2.19.2 Spectral Sensitivity

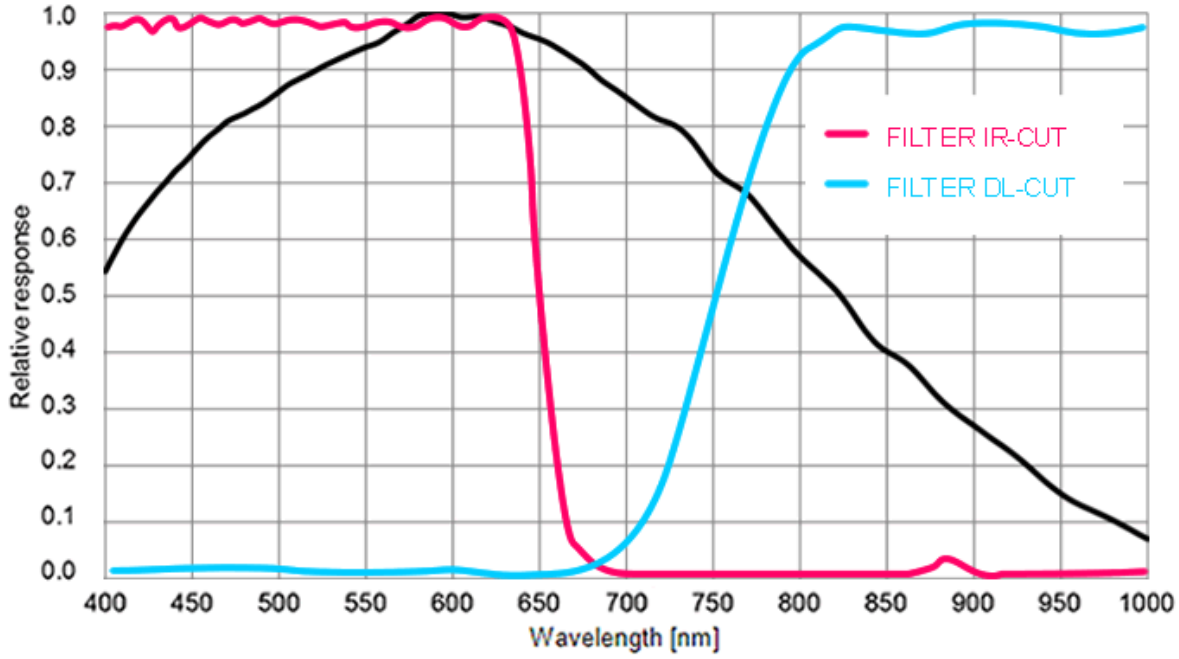


Figure 1: Spectral sensitivity BF3-4-0315Z G

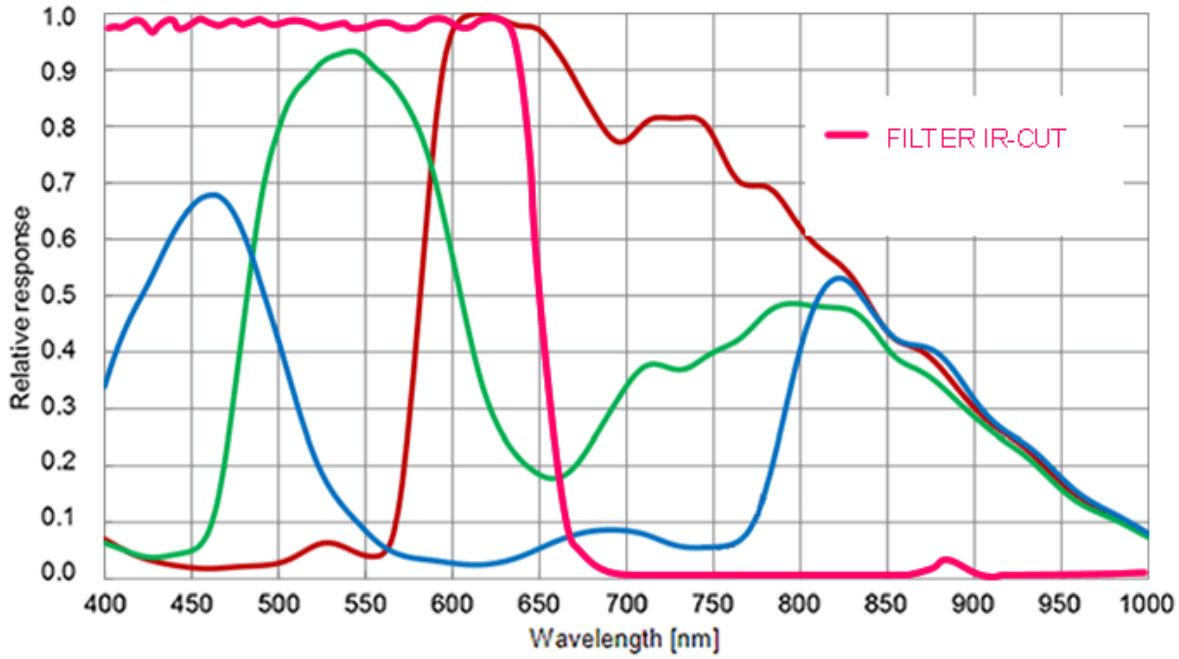


Figure 2: Spectral sensitivity BF3-4-0315Z C

Name	Value
InternalLineLength	762 (10 bit) / 910 (12 bit)
VerticalBlankLines	50
SensorInClock	53.977 (@54 MHz Pixel clock)

1.20.1.2.19.3 Timings

1.20.1.2.19.4 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 16.85 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed"	OnDemand	Image acquisition triggered by command (software trigger).

To trigger one frame execute the *TriggerSoftware@i* command then.

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 444)

1.20.1.2.19.5 Device Feature And Property List

- **BF3-4-0315ZG / BF3-5-0315ZG Features** (p. 444)
- **BF3-4-0315ZC / BF3-5-0315ZC Features** (p. 444)

1.20.1.2.19.6 BF3-4-0315ZG / BF3-5-0315ZG Features

1.20.1.2.19.7 BF3-4-0315ZC / BF3-5-0315ZC Features

1.20.2 A.2 Starvis CMOS

- **mvBlueFOX3-2064 / BF3-3M-0064Z / BF3-5M-0064Z (6.4 Mpix [3096 x 2080])** (p. 445)
- **mvBlueFOX3-2124r / BF3-5M-0124R (12.4 Mpix [4064 x 3044])** (p. 450)
- **mvBlueFOX3-2205 / BF3-5M-0205Z (20.5 Mpix [5544 x 3692])** (p. 454)

1.20.2.1 mvBlueFOX3-2064 / BF3-3M-0064Z / BF3-5M-0064Z (6.4 Mpix [3096 x 2080])

1.20.2.1.1 Introduction

The sensor is a back-illuminated rolling shutter, i.e. the image is acquired through the back side without any obstacles like wiring or circuits. Therefore, a wider range of light is collected on the photo-diode.

Feature	Description
Manufacturer	Sony
Sensor name	IMX178
Max. frames per second	60
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	41.6
DNR (normal / HDR) [dB][1/]	71.6 / -
Image size	1/1.8
Number of effective pixels	3096 (H) x 2080 (V)
Unit cell size	2.4µm (H) x 2.4µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

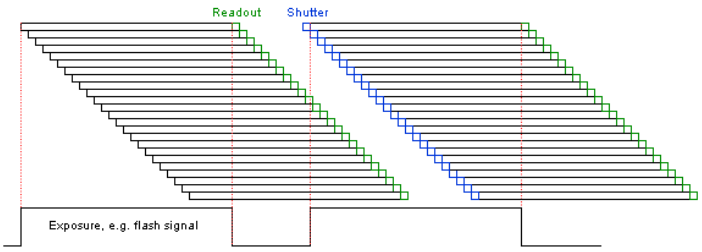
1.20.2.1.1.1 Shutter modes

The sensor offers several shutter modes:

- **rolling shutter** (mvShutterMode = "`mvRollingShutter`"),
- **rolling shutter flash** (mvShutterMode = "`mvRollingShutterFlash`"), and
- **global reset release shutter** (mvShutterMode = "`mvGlobalReset`").

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time.

- The exposure time which is set corresponds to the exposure time of each line.
- The exposure signal corresponds to the exposure of the first line.

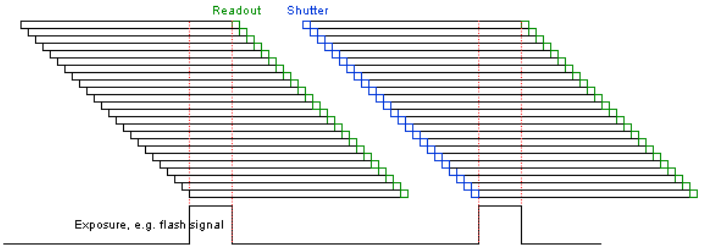


Note

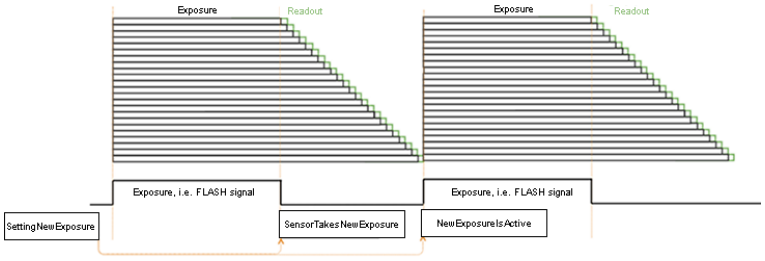
Moving objects together with a rolling shutter can cause a smear in moving objects.

With the **rolling shutter flash**

- The exposure time which is set corresponds to the time where all lines are exposed simultaneously.
- The exposure signal corresponds to the time where the last line starts until the first line ends.



The **global reset release shutter** starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:



Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no extraneous light and the flash duration is shorter or equal to the exposure time.

1.20.2.1.2 Spectral Sensitivity

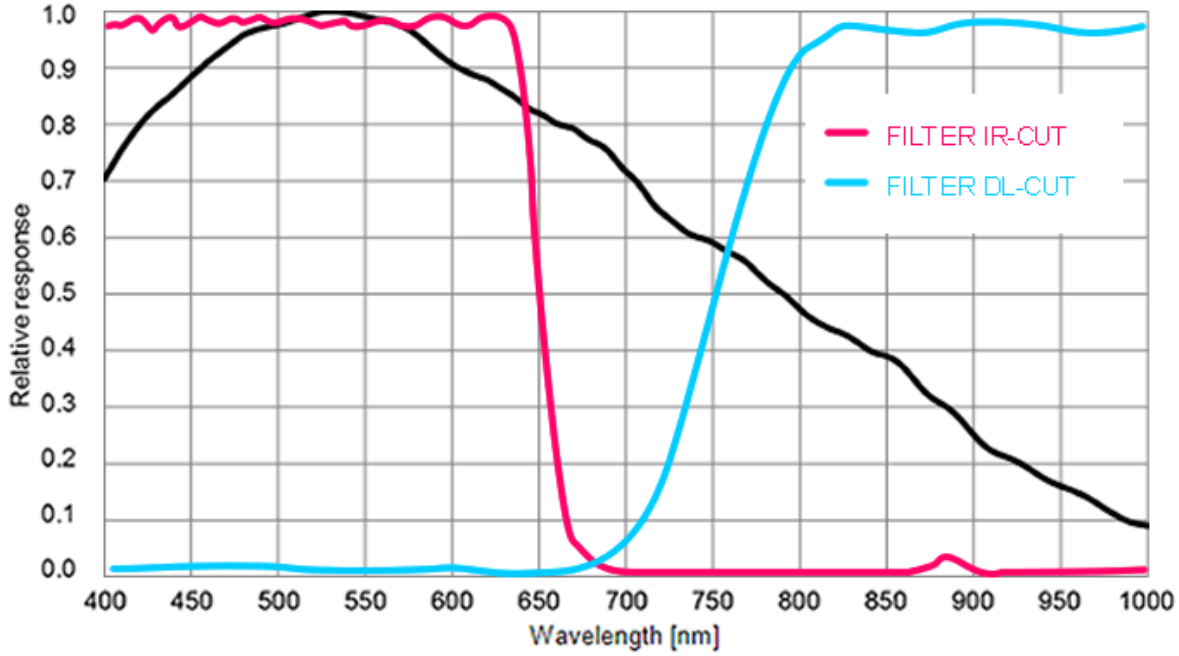


Figure 1: Spectral sensitivity mvBlueFOX3-2064G

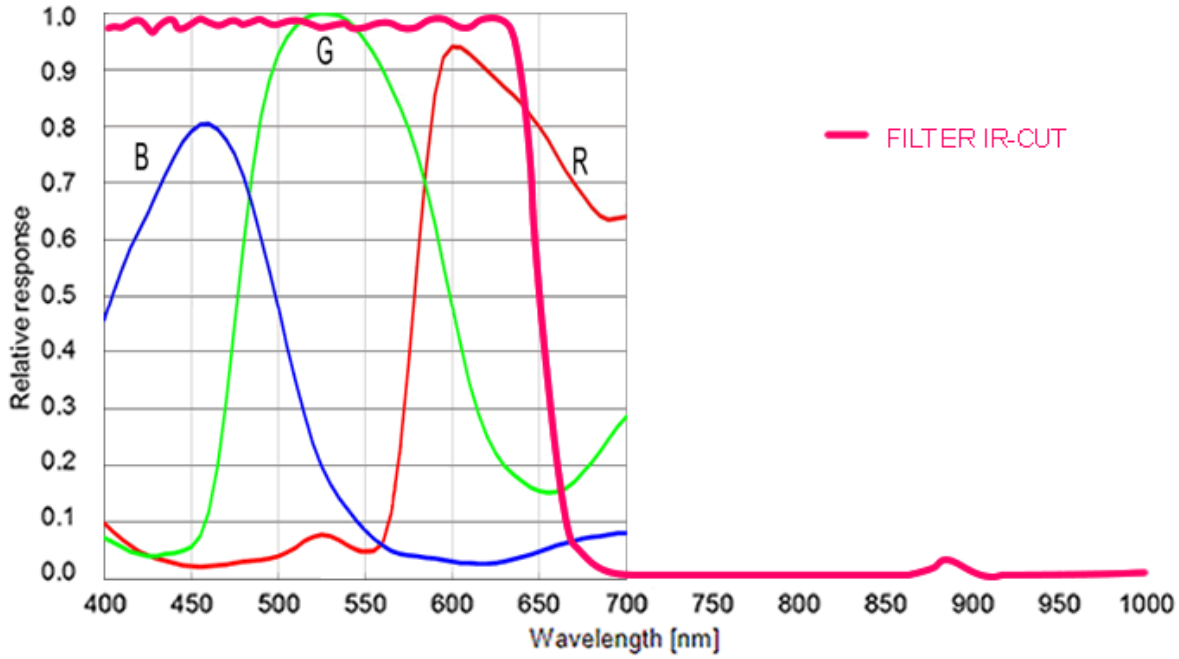


Figure 2: Spectral sensitivity mvBlueFOX3-2064C

Name	Value
InternalLineLength	420 (10 bit) / 495 (12 bit)
VerticalBlankLines	31
SensorInClock	54 MHz
NumberOfLVDS	8

1.20.2.1.3 Timings

1.20.2.1.3.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Connection

Note

The exposure time step width is limited to the sensor's row time of 9.44 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 449)

1.20.2.1.4 Device Feature And Property List

- **mvBlueFOX3-2064G / BF3-3M-0064ZG / BF3-5M-0064ZG Features** (p. 449)
- **mvBlueFOX3-2064C / BF3-3M-0064ZC / BF3-5M-0064ZC Features** (p. 449)

1.20.2.1.4.1 mvBlueFOX3-2064G / BF3-3M-0064ZG / BF3-5M-0064ZG Features

1.20.2.1.4.2 mvBlueFOX3-2064C / BF3-3M-0064ZC / BF3-5M-0064ZC Features

1.20.2.2 mvBlueFOX3-2124r / BF3-5M-0124R (12.4 Mpix [4064 x 3044])

1.20.2.2.1 Introduction

The sensor is a back-illuminated rolling shutter, i.e. the image is acquired through the back side without any obstacles like wiring or circuits. Therefore, a wider range of light is collected on the photo-diode.

Feature	Description
Manufacturer	Sony
Sensor name	IMX226
Max. frames per second	30.7
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.3
DNR (normal / HDR) [dB][1/]	69.2 / -
Image size	1/1.7
Number of effective pixels	4064 (H) x 3044 (V)
Unit cell size	1.85µm (H) x 1.85µm (V)
ADC resolution / output	12 bit → 8/10/(12)

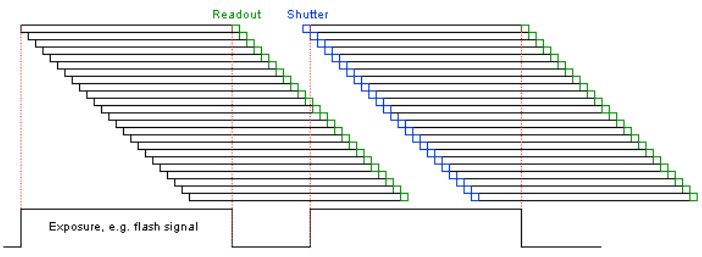
[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.2.2.1.1 Shutter modes The sensor offers several shutter modes:

- **rolling shutter** (mvShutterMode = "`mvRollingShutter`"),
- **rolling shutter flash** (mvShutterMode = "`mvRollingShutterFlash`"), and
- **global reset release shutter** (mvShutterMode = "`mvGlobalReset`").

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time.

- The exposure time which is set corresponds to the exposure time of each line.
- The exposure signal corresponds to the exposure of the first line.

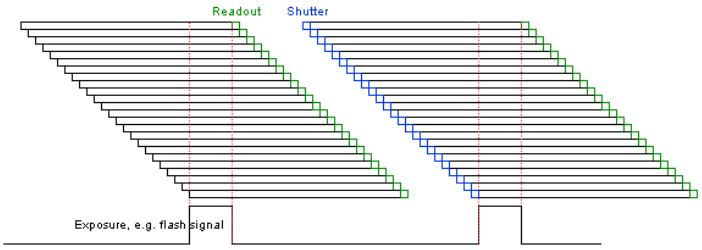


Note

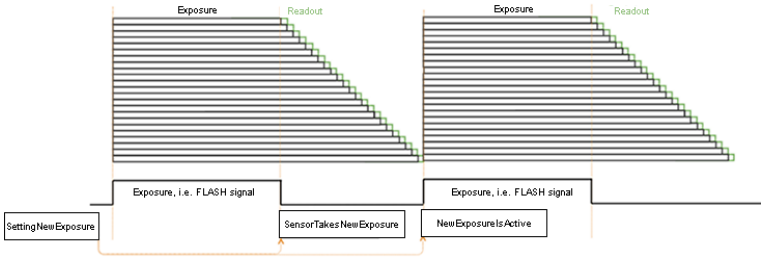
Moving objects together with a rolling shutter can cause a smear in moving objects.

With the **rolling shutter flash**

- The exposure time which is set corresponds to the time where all lines are exposed simultaneously.
- The exposure signal corresponds to the time where the last line starts until the first line ends.



The **global reset release shutter** starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:



Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no extraneous light and the flash duration is shorter or equal to the exposure time.

1.20.2.2.2 Spectral Sensitivity

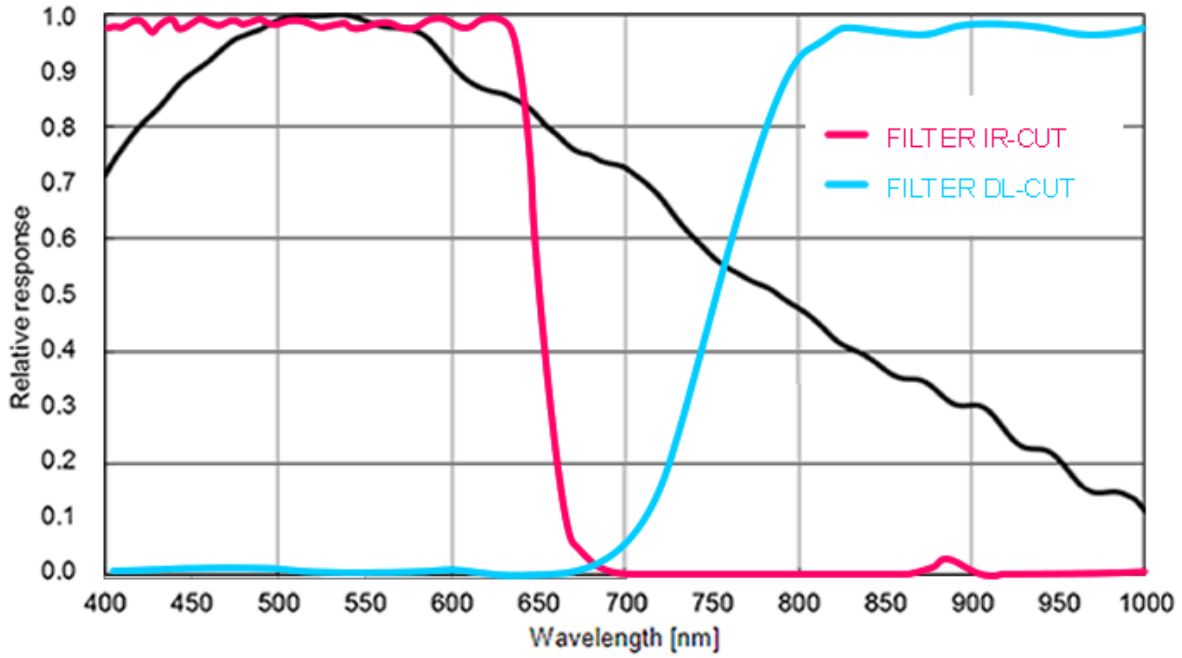


Figure 1: Spectral sensitivity mvBlueFOX3-2124rG

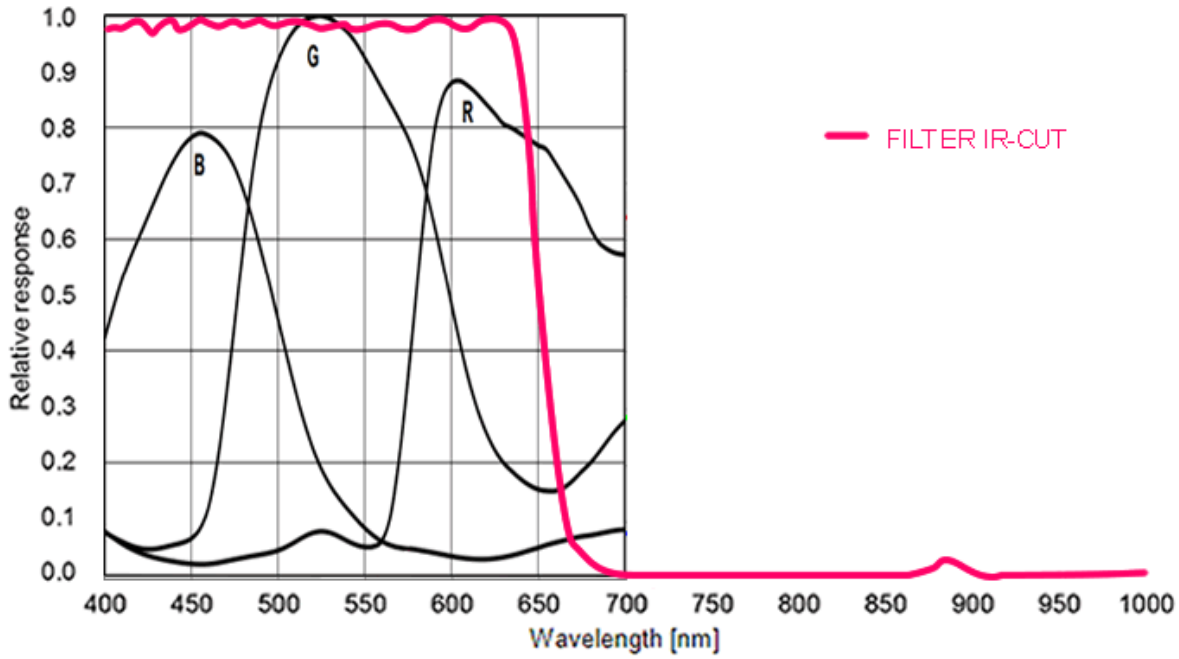


Figure 2: Spectral sensitivity mvBlueFOX3-2124rC

Name	Value
InternalLineLength	660
VerticalBlankLines	38
SensorInClock	72 MHz
NumberOfLVDS	10

1.20.2.2.3 Timings

1.20.2.2.3.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{(InternalLineLength / SensorInClock) * VerticalBlankLines}{1000} + \frac{ImageHeight * (InternalLineLength}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Connection

Note

The exposure time step width is limited to the sensor's row time of 9.17 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 454)

1.20.2.2.4 Device Feature And Property List

- **mvBlueFOX3-2124rG / BF3-5M-0124RG Features** (p. 454)
- **mvBlueFOX3-2124rC / BF3-5M-0124RC Features** (p. 454)

1.20.2.2.4.1 mvBlueFOX3-2124rG / BF3-5M-0124RG Features

1.20.2.2.4.2 mvBlueFOX3-2124rC / BF3-5M-0124RC Features

1.20.2.3 mvBlueFOX3-2205 / BF3-5M-0205Z (20.5 Mpix [5544 x 3692])

1.20.2.3.1 Introduction

The sensor is a back-illuminated rolling shutter, i.e. the image is acquired through the back side without any obstacles like wiring or circuits. Therefore, a wider range of light is collected on the photo-diode.

Feature	Description
Manufacturer	Sony
Sensor name	IMX183
Max. frames per second	22
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	41.6
DNR (normal / HDR) [dB][1/]	71.5 / -
Image size	1
Number of effective pixels	5544 (H) x 3692 (V)
Unit cell size	2.4µm (H) x 2.4µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

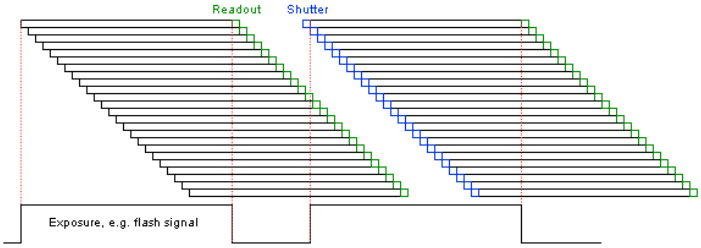
1.20.2.3.1.1 Shutter modes

The sensor offers several shutter modes:

- **rolling shutter** (mvShutterMode = "`mvRollingShutter`"),
- **rolling shutter flash** (mvShutterMode = "`mvRollingShutterFlash`"), and
- **global reset release shutter** (mvShutterMode = "`mvGlobalReset`").

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time.

- The exposure time which is set corresponds to the exposure time of each line.
- The exposure signal corresponds to the exposure of the first line.

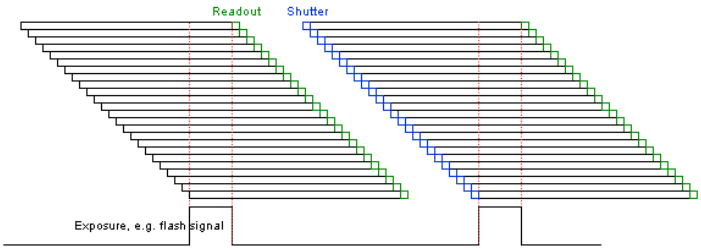


Note

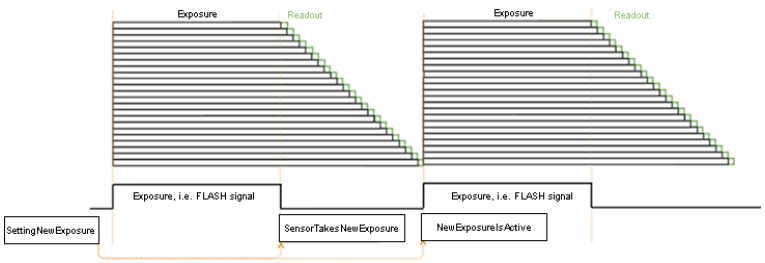
Moving objects together with a rolling shutter can cause a smear in moving objects.

With the **rolling shutter flash**

- The exposure time which is set corresponds to the time where all lines are exposed simultaneously.
- The exposure signal corresponds to the time where the last line starts until the first line ends.



The **global reset release shutter** starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:



Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no extraneous light and the flash duration is shorter or equal to the exposure time.

1.20.2.3.2 Spectral Sensitivity

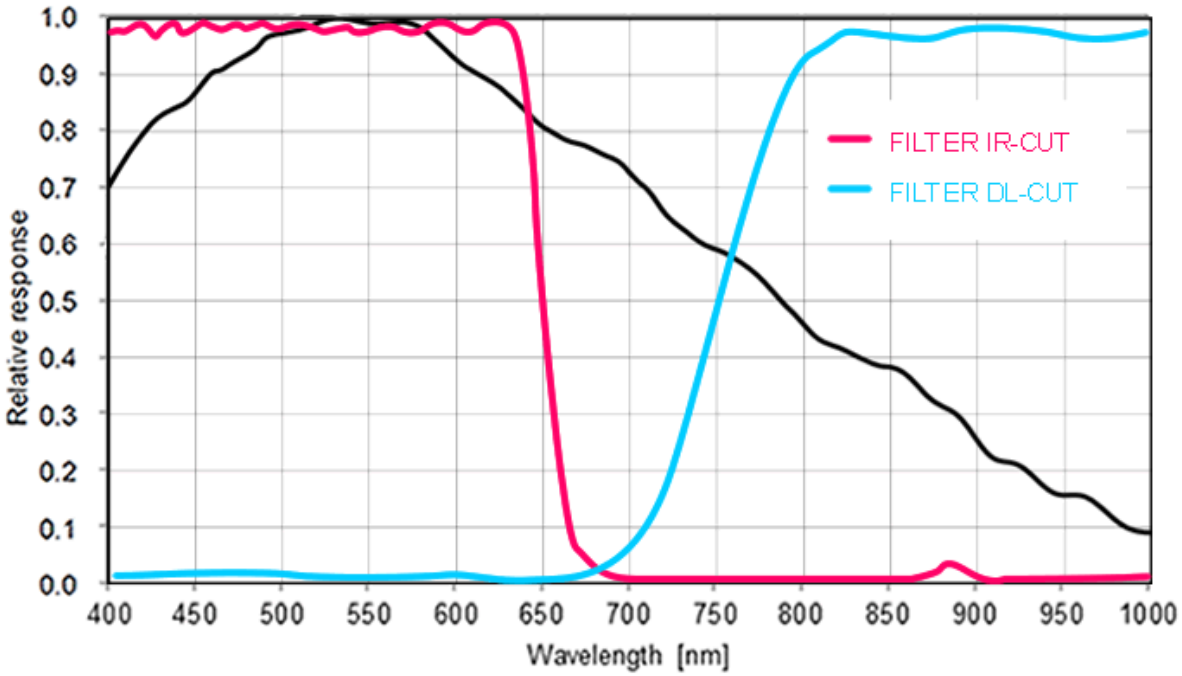


Figure 1: Spectral sensitivity mvBlueFOX3-2205G

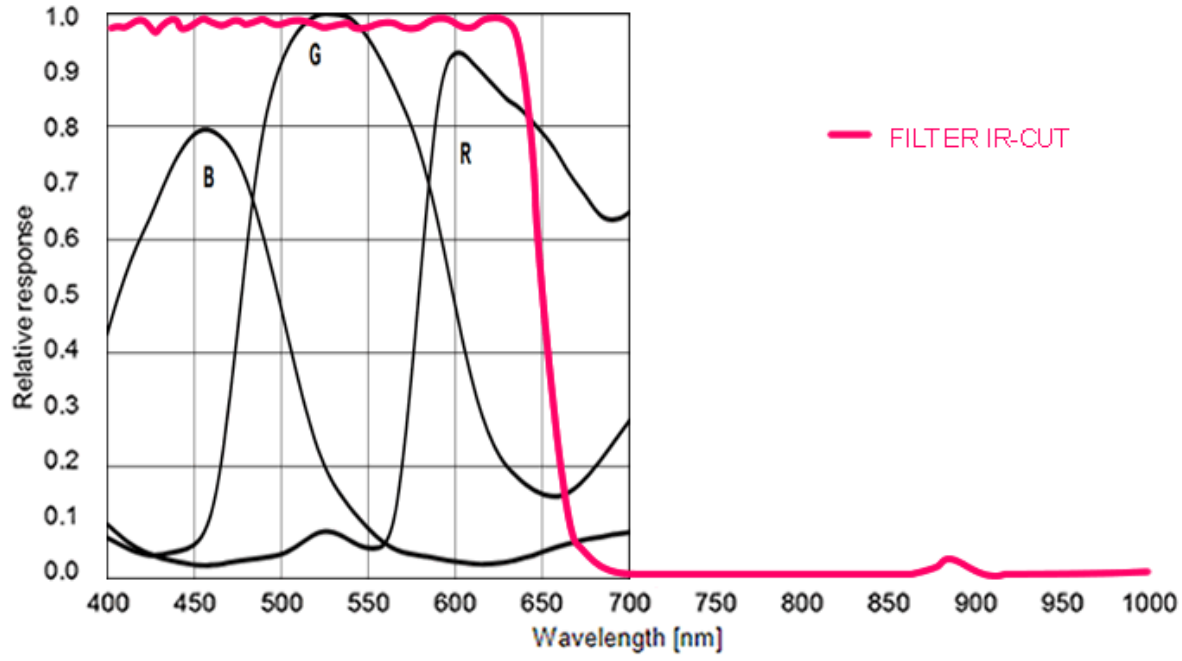


Figure 2: Spectral sensitivity mvBlueFOX3-2205C

Name	Value
InternalLineLength	876
VerticalBlankLines	37
SensorInClock	72 MHz
NumberOfLVDS	8

1.20.2.3.3 Timings

1.20.2.3.3.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{(InternalLineLength / SensorInClock) * VerticalBlankLines}{1000} + \frac{ImageHeight * (InternalLength}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Connection

Note

The exposure time step width is limited to the sensor's row time of 12.16 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 459)

1.20.2.3.4 Device Feature And Property List

- **mvBlueFOX3-2205G / BF3-5M-0205ZG Features** (p. 459)
- **mvBlueFOX3-2205C / BF3-5M-0205ZC Features** (p. 459)

1.20.2.3.4.1 mvBlueFOX3-2205G / BF3-5M-0205ZG Features

1.20.2.3.4.2 mvBlueFOX3-2205C / BF3-5M-0205ZC Features

1.20.3 A.3 Polarsens CMOS

- **mvBlueFOX3-2051p (5.1 Mpix [2464 x 2056])** (p. 459)

1.20.3.1 mvBlueFOX3-2051p (5.1 Mpix [2464 x 2056])

1.20.3.1.1 Introduction

The sensor can acquire a four directional polarization image in one shot. Because the polarizer is under the on-chip lens layer

- reflectances are reduced and
- flare and ghost characteristics are avoided.

The pixel array of the sensor is as follows:

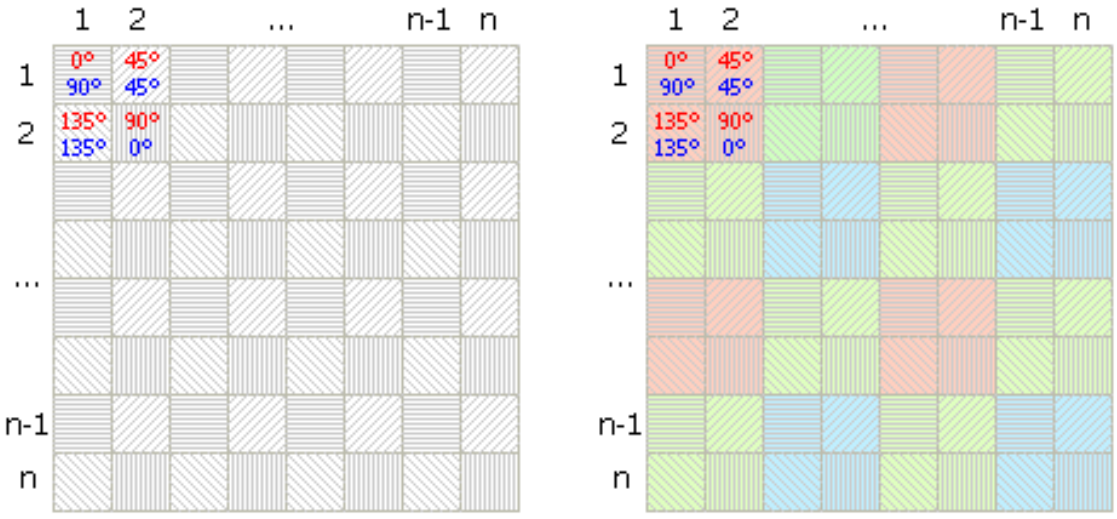


Figure 1: Pixel array gray scale / color

Feature	Description
Manufacturer	Sony
Sensor name	IMX250_POL
Max. frames per second	80
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.2 / -
Image size	2/3
Number of effective pixels	2464 (H) x 2056 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.3.1.2 Spectral Sensitivity

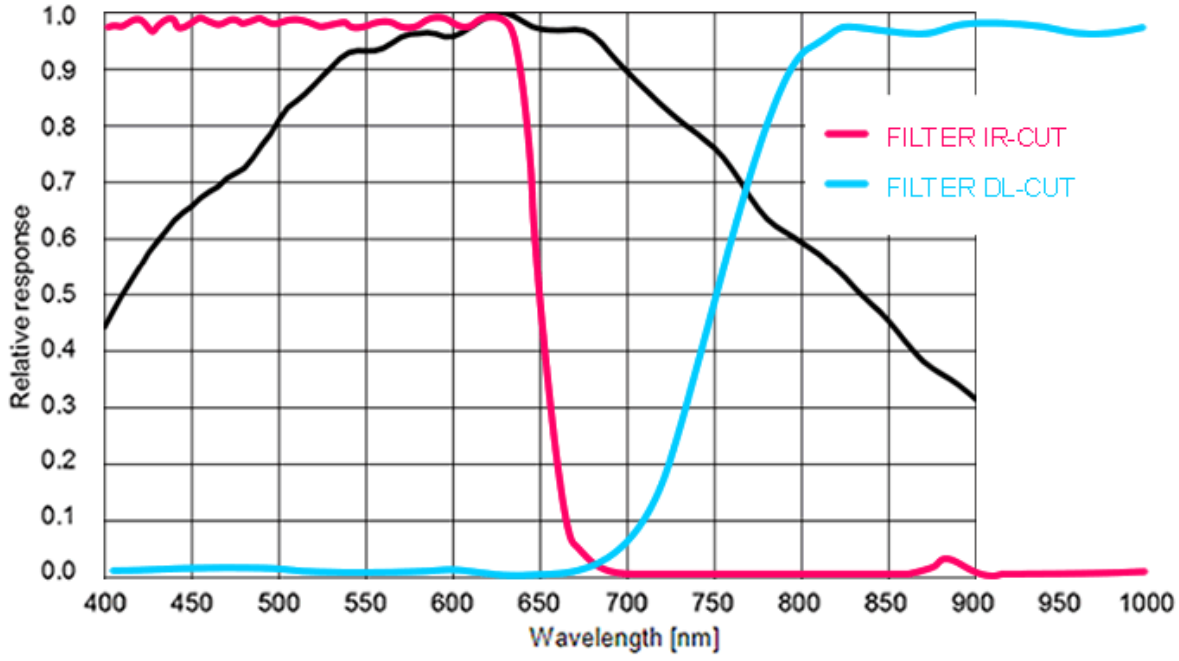


Figure 2: Spectral sensitivity mvBlueFOX3-2051pG

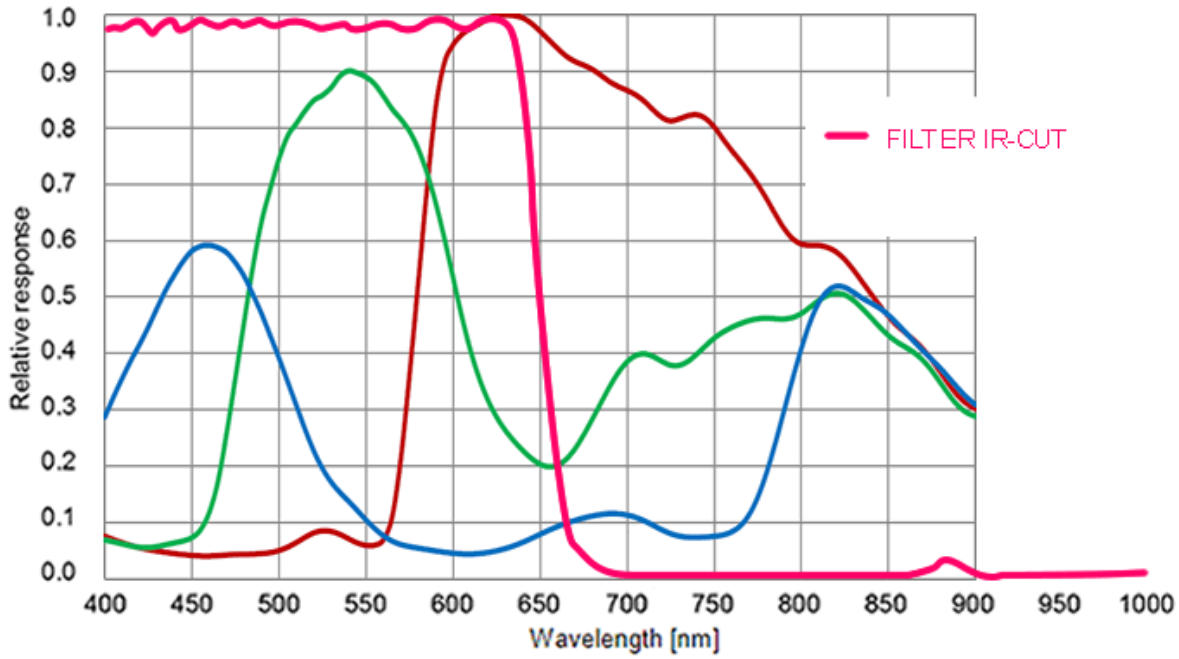


Figure 3: Spectral sensitivity mvBlueFOX3-2051pC

Name	Value
InternalLineLength	440 (10 bit) / 519 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.3.1.3 Timings

1.20.3.1.3.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 5.94 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 463)

1.20.3.1.4 Device Feature And Property List

- **mvBlueFOX3-2051pG Features** (p. 463)
- **mvBlueFOX3-2051pC Features** (p. 463)

1.20.3.1.4.1 mvBlueFOX3-2051pG Features

1.20.3.1.4.2 mvBlueFOX3-2051pC Features

1.20.4 A.4 CMOS

- **mvBlueFOX3-1012b (1.2 Mpix [1280 x 960])** (p. 463)
- **mvBlueFOX3-1012d (1.2 Mpix [1280 x 960])** (p. 466)
- **mvBlueFOX3-1013 (1.3 Mpix [1280 x 1024])** (p. 469)
- **mvBlueFOX3-1020 (1.9 Mpix [1600 x 1200])** (p. 473)
- **mvBlueFOX3-1020a (1.9 Mpix [1600 x 1200])** (p. 476)
- **mvBlueFOX3-1031 (3.2 Mpix [2048 x 1536])** (p. 479)
- **mvBlueFOX3-1100 (11 Mpix [3856 x 2764])** (p. 481)
- **mvBlueFOX3-1140 (14 Mpix [4384 x 3288])** (p. 485)

1.20.4.1 mvBlueFOX3-1012b (1.2 Mpix [1280 x 960])

1.20.4.1.1 Introduction

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	Aptina
Sensor name	MT9M031
Max. frames per second	45.6
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	37.4
DNR (normal / HDR) [dB][1/]	54.3 / -
Image size	1/3
Number of effective pixels	1280 (H) x 960 (V)
Unit cell size	3.75um (H) x 3.75um (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.1.2 Spectral Sensitivity

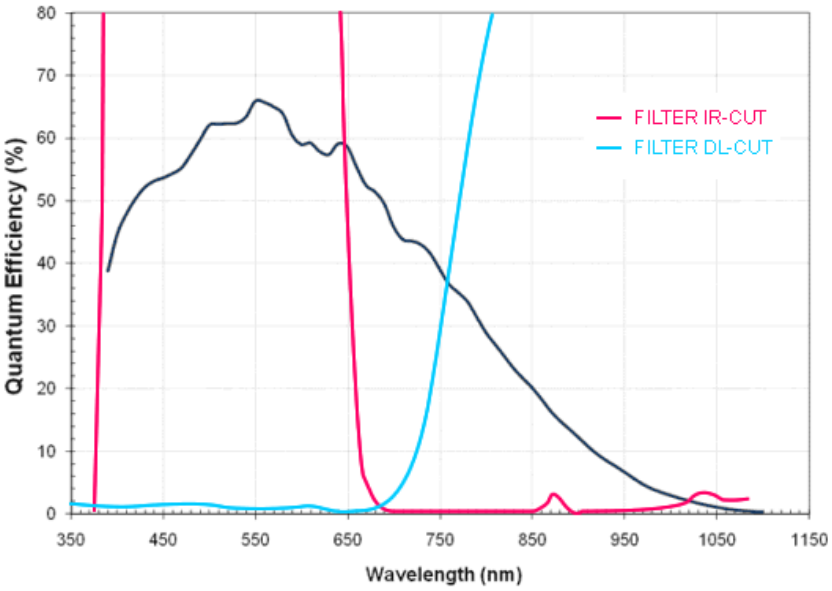


Figure 1: Spectral sensitivity mvBlueFOX3-1012dG

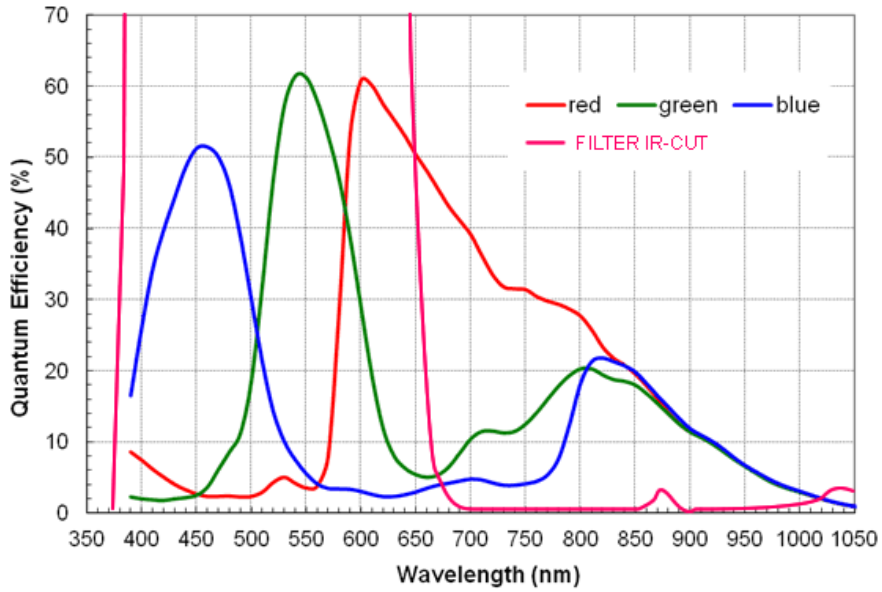


Figure 2: Spectral sensitivity mvBlueFOX3-1012dC

Name	Value
PixelClock	40 & 66
VerticalBlank	25

1.20.4.1.3 Timings

1.20.4.1.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = (ImageHeight * (\frac{1650}{PixelClock})) + ((VerticalBlank+2) * \frac{1650}{PixelClock})$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1000000}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1000000}{ExposureTime}$$

1.20.4.1.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (1650)) + ((VerticalBlank+2) * 1650)}{PixelClock}$$

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 41.3 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

Device Feature And Property List (p. 466)

1.20.4.1.4 Device Feature And Property List

- **mvBlueFOX3-1012bG Features** (p. 466)
- **mvBlueFOX3-1012bC Features** (p. 466)

1.20.4.1.4.1 mvBlueFOX3-1012bG Features

1.20.4.1.4.2 mvBlueFOX3-1012bC Features

1.20.4.2 mvBlueFOX3-1012d (1.2 Mpix [1280 x 960])

1.20.4.2.1 Introduction

The sensor features one acquisition mode:

- rolling shutter.

With the `rolling shutter` the lines are exposed for the same duration, but at a slightly different point in time:

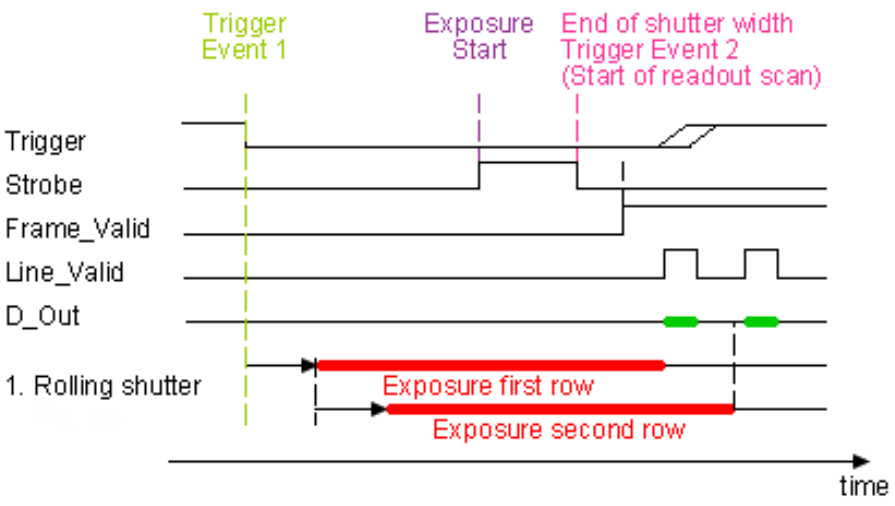


Figure 1: Rolling shutter

Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

Feature	Description
Manufacturer	Aptina
Sensor name	MT9M034
Max. frames per second	45.6
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	37.7
DNR (normal / HDR) [dB][1/]	63.4 / $\geq$ **115** (p. 302)
Image size	1/3
Number of effective pixels	1280 (H) x 960 (V)
Unit cell size	3.75um (H) x 3.75um (V)
ADC resolution / output	12 bit $\rightarrow$ 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.2.2 Spectral Sensitivity

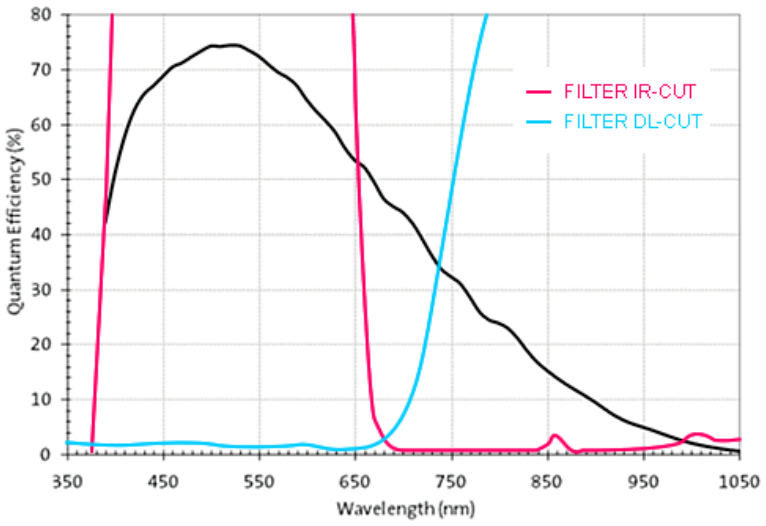


Figure 2: Spectral sensitivity mvBlueFOX3-1012dG

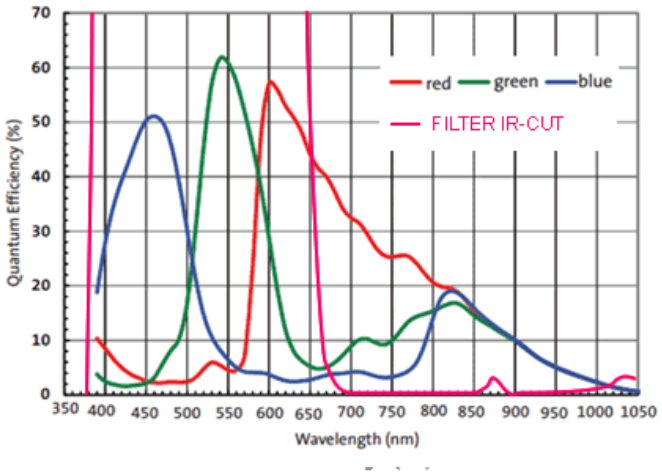


Figure 3: Spectral sensitivity mvBlueFOX3-1012dC

Name	Value
PixelClock	40 & 74,25
VerticalBlank	25

1.20.4.2.3 Timings

1.20.4.2.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (\frac{1650}{PixelClock})) + ((VerticalBlank+2) * \frac{1650}{PixelClock})}{}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1000000}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1000000}{ExposureTime}$$

1.20.4.2.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (1650)) + ((VerticalBlank+2) * 1650)}{PixelClock}$$

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 41.3 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

Device Feature And Property List (p. 469)

1.20.4.2.4 Device Feature And Property List

- **mvBlueFOX3-1012dG Features** (p. 469)
- **mvBlueFOX3-1012dC Features** (p. 469)

1.20.4.2.4.1 mvBlueFOX3-1012dG Features

1.20.4.2.4.2 mvBlueFOX3-1012dC Features

1.20.4.3 mvBlueFOX3-1013 (1.3 Mpix [1280 x 1024])

1.20.4.3.1 Introduction

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	E2V
Sensor name	EV76C560
Max. frames per second	60
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	39
DNR (normal / HDR) [dB][1/]	50.5 / > **100 dB** (p. 303)
Image size	1/1.8
Number of effective pixels	1280 (H) x 1024 (V)
Unit cell size	5.3µm (H) x 5.3µm (V)
ADC resolution / output	10 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.3.2 Spectral Sensitivity

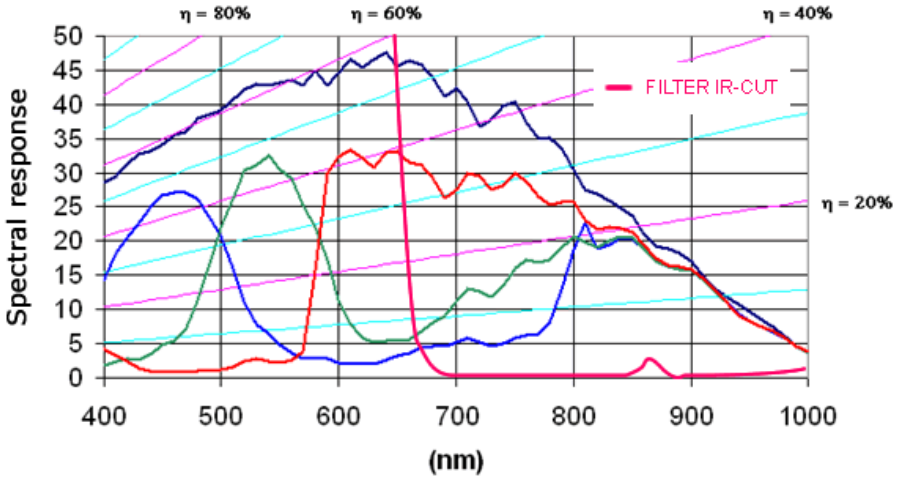


Figure 1: mvBlueFOX3-1013G

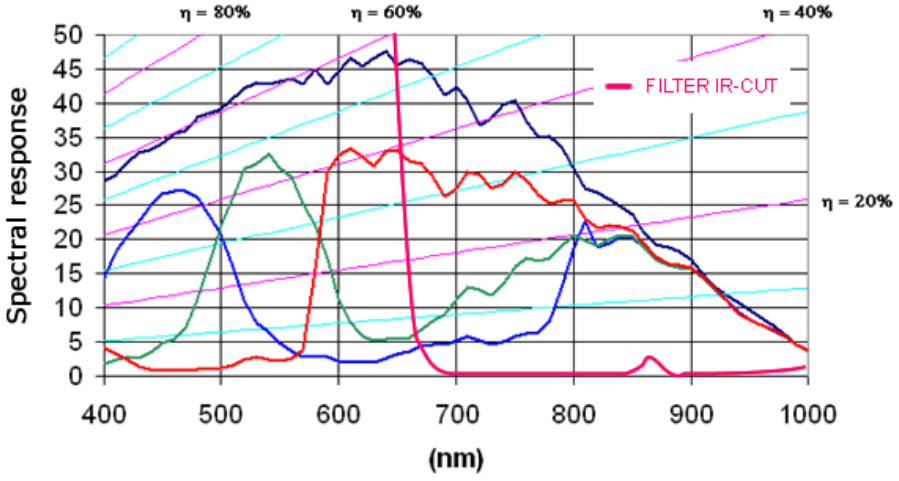


Figure 2: mvBlueFOX3-1013C

1.20.4.3.3 Enhanced version

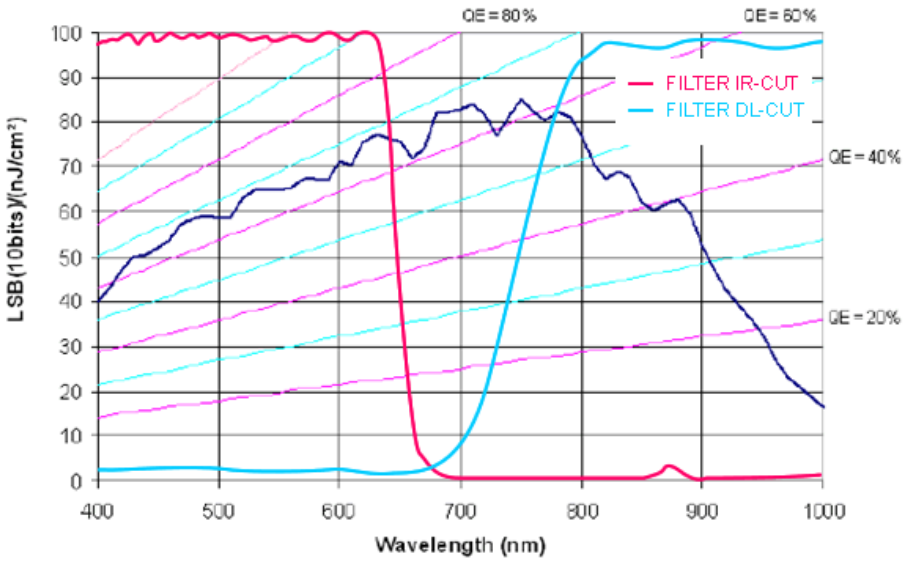


Figure 1: mvBlueFOX3-1013GE

Name	Value
InternalLineLength	944
InternalADCClock	118.75

1.20.4.3.4 Timings

1.20.4.3.4.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$ReadOutTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1000}{ReadOutTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1000}{ExposureTime}$$

1.20.4.3.4.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

```
                                          2             ImageHeight + 10
ReadOutTime =  InternalLineLength * ---------------- * (-----------------)
                                    InternalADCClock          1000


                      1000
FPS_max =  --------------------------
            ReadOutTime + ExposureTime
```

1.20.4.3.4.3 Line scan mode

The sensor offers also a usage as a line scan sensor. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1024 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

The camera then outputs these frames which contain multiples of the same scan line(s) without gaps or interruptions.

To operate in line scan mode, use the following properties:

- In "`Setting -> Base -> Camera -> !GenICam -> Device Control`", please change the `Device Scan Type` to **Linescan**.
- In "`Setting -> Base -> Camera -> !GenICam -> Image Format Control`", please set `Width` and `Height`. *Height* specifies how often the same line(s) will be used to reach the height of the image.
  Use `Offset X` to shift horizontally the starting point of the exposed line.
  Use and `Offset Y` to shift the scan line vertically.

Note

The sensor will not get faster by windowing in x direction.

- Finally, in "`Setting -> Base -> Camera -> !GenICam -> Acquisition Control`", please adapt the `Exposure Time`.
  The exposure time as to be low and max. approx. 60 us in order to achieve the max. line scan rate of 12.6 kHz.

You may use longer exposure times at the expense of scanning frequency.

Note

Using more than one line e.g. 5, is like having an area scan with ImageHeight = 5. In line scan mode, the max. ImageHeight is $\leq$ 16.

You can use either `Continuous` or a trigger mode as `Acquisition Mode` in "`Setting -> Base -> Camera -> GenICam -> Acquisition Control`". However, if an external (line) trigger will have to be used, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible!

Device Feature And Property List (p. 473)

1.20.4.3.5 Device Feature And Property List

- **mvBlueFOX3-1013G Features** (p. 473)
- **mvBlueFOX3-1013GE Features** (p. 473)
- **mvBlueFOX3-1013C Features** (p. 473)

1.20.4.3.5.1 mvBlueFOX3-1013G Features

1.20.4.3.5.2 mvBlueFOX3-1013GE Features

1.20.4.3.5.3 mvBlueFOX3-1013C Features

1.20.4.4 mvBlueFOX3-1020 (1.9 Mpix [1600 x 1200])

1.20.4.4.1 Introduction

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	E2V
Sensor name	EV76C570
Max. frames per second	51
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	38.9
DNR (normal / HDR) [dB][1/]	50.7 / > **100** (p. 303)
Image size	1/1.8
Number of effective pixels	1600 (H) x 1200 (V)
Unit cell size	4.5µm (H) x 4.5µm (V)
ADC resolution / output	10 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.4.2 Spectral Sensitivity

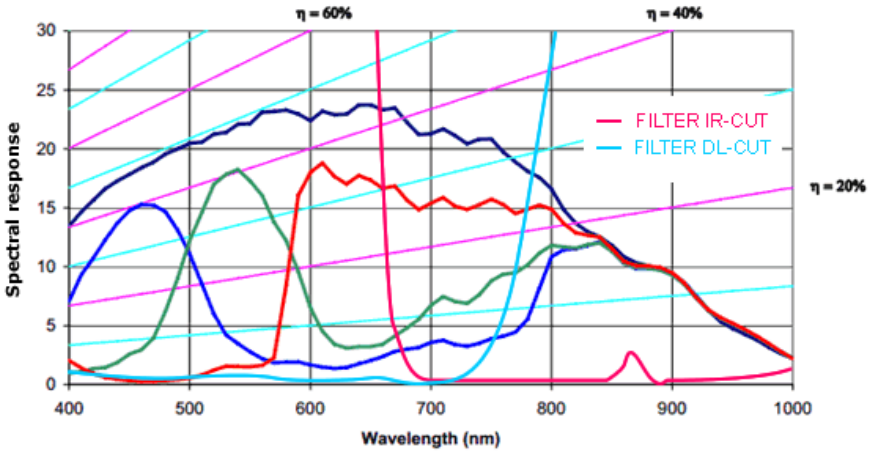


Figure 1: mvBlueFOX3-1020

Name	Value
InternalLineLength	944
InternalADCClock	120

1.20.4.4.3 Timings

1.20.4.4.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$ReadOutTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

If exposure time is lower than read out time:

$$FPS_max = \frac{1000}{ReadOutTime}$$

If exposure time is greater than read out time:

$$FPS_max = \frac{1000}{ExposureTime}$$

1.20.4.4.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$ReadOutTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

$$FPS_max = \frac{1000}{ReadOutTime + ExposureTime}$$

1.20.4.4.3.3 Line scan mode

The sensor offers also a usage as a line scan sensor. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1600 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

The camera then outputs these frames which contain multiples of the same scan line(s) without gaps or interruptions.

To operate in line scan mode, use the following properties:

- In "`Setting -> Base -> Camera -> GenICam -> Device Control`", please change the `Device Scan Type` to **line** scan.
- In "`Setting -> Base -> Camera -> GenICam -> Image Format Control`", please set `Width` and `Height`. *Height* specifies how often the same line(s) will be used to reach the height of the image.
  Use `Offset X` to shift horizontally the starting point of the exposed line.
  Use and `Offset Y` to shift the scan line vertically.

Note

The sensor will not get faster by windowing in x direction.

- Finally, in "`Setting -> Base -> Camera -> !GenICam -> Acquisition Control`", please adapt the `Exposure Time`.
  The exposure time as to be low and max. approx. 60 us in order to achieve the max. line scan rate of 12.6 kHz.

You may use longer exposure times at the expense of scanning frequency.

Note

Using more than one line e.g. 5, is like having an area scan with ImageHeight = 5. In line scan mode, the max. ImageHeight is ≤ 16.

You can use either `Continuous` or a trigger mode as `Acquisition Mode` in "`Setting -> Base -> Camera -> GenICam -> Acquisition Control`". However, if an external (line) trigger will have to be used, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible!

Device Feature And Property List (p. 476)

1.20.4.4.4 Device Feature And Property List

- **mvBlueFOX3-1020G Features** (p. 476)
- **mvBlueFOX3-1020C Features** (p. 476)

1.20.4.4.4.1 mvBlueFOX3-1020G Features

1.20.4.4.4.2 mvBlueFOX3-1020C Features

1.20.4.5 mvBlueFOX3-1020a (1.9 Mpix [1600 x 1200])

1.20.4.5.1 Introduction

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

Feature	Description
Manufacturer	E2V
Sensor name	EV76C570
Max. frames per second	60
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	38.9
DNR (normal / HDR) [dB][1/]	50.5 / -
Image size	1/1.8
Number of effective pixels	1600 (H) x 1200 (V)
Unit cell size	4.5µm (H) x 4.5µm (V)
ADC resolution / output	10 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.5.2 Spectral Sensitivity

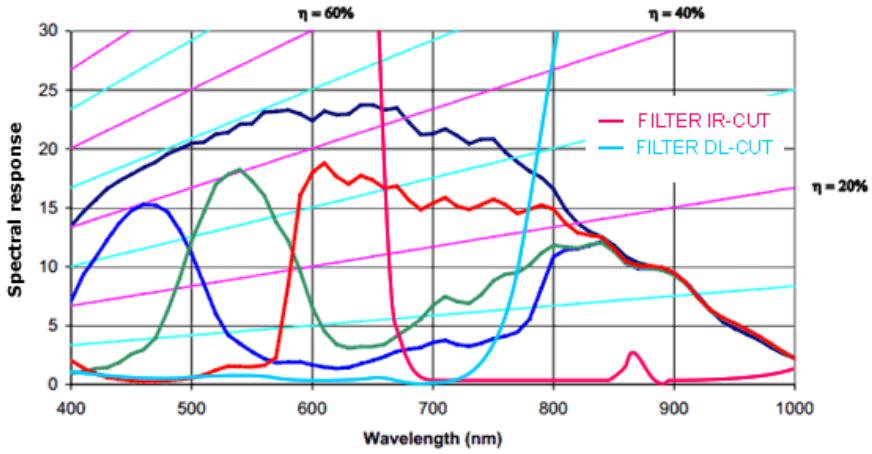


Figure 1: mvBlueFOX3-1020aG

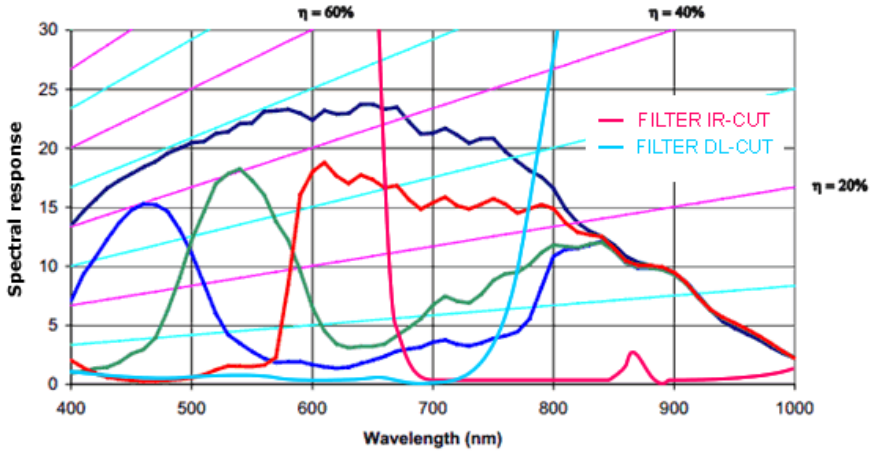


Figure 2: mvBlueFOX3-1020aC

Name	Value
InternalLineLength	984
InternalADCClock	142.↩ 9

1.20.4.5.3 Timings

1.20.4.5.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$ReadOutTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

If exposure time is lower than read out time:

$$FPS_{max} = \frac{1000}{ReadOutTime}$$

If exposure time is greater than read out time:

$$FPS_{max} = \frac{1000}{ExposureTime}$$

1.20.4.5.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$ReadOutTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

$$FPS_{max} = \frac{1000}{ReadOutTime + ExposureTime}$$

1.20.4.5.3.3 Line scan mode

The sensor offers also a usage as a line scan sensor. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1600 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

The camera then outputs these frames which contain multiples of the same scan line(s) without gaps or interruptions.

To operate in line scan mode, use the following properties:

- In "`Setting -> Base -> Camera -> GenICam -> Device Control`", please change the `Device Scan Type` to **line** scan.
- In "`Setting -> Base -> Camera -> GenICam -> Image Format Control`", please set `Width` and `Height`. *Height* specifies how often the same line(s) will be used to reach the height of the image.
  Use `Offset X` to shift horizontally the starting point of the exposed line.
  Use and `Offset Y` to shift the scan line vertically.

Note

The sensor will not get faster by windowing in x direction.

- Finally, in "`Setting -> Base -> Camera -> !GenICam -> Acquisition Control`", please adapt the `Exposure Time`.
  The exposure time as to be low and max. approx. 60 us in order to achieve the max. line scan rate of 12.6 kHz.

You may use longer exposure times at the expense of scanning frequency.

Note

Using more than one line e.g. 5, is like having an area scan with ImageHeight = 5. In line scan mode, the max. ImageHeight is $\leq$ 16.

You can use either `Continuous` or a trigger mode as `Acquisition Mode` in "`Setting -> Base -> Camera -> GenICam -> Acquisition Control`". However, if an external (line) trigger will have to be used, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible!

Device Feature And Property List (p. 479)

1.20.4.5.4 Device Feature And Property List

- **mvBlueFOX3-1020aG Features** (p. 479)
- **mvBlueFOX3-1020aC Features** (p. 479)

1.20.4.5.4.1 mvBlueFOX3-1020aG Features

1.20.4.5.4.2 mvBlueFOX3-1020aC Features

1.20.4.6 mvBlueFOX3-1031 (3.2 Mpix [2048 x 1536])

1.20.4.6.1 Introduction

The sensor features several acquisition modes ("`mv Shutter Mode`"):

- rolling shutter ("`mvRollingShutter`").

Note

"FrameStart" is not available in *"Settings -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector"*

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time:

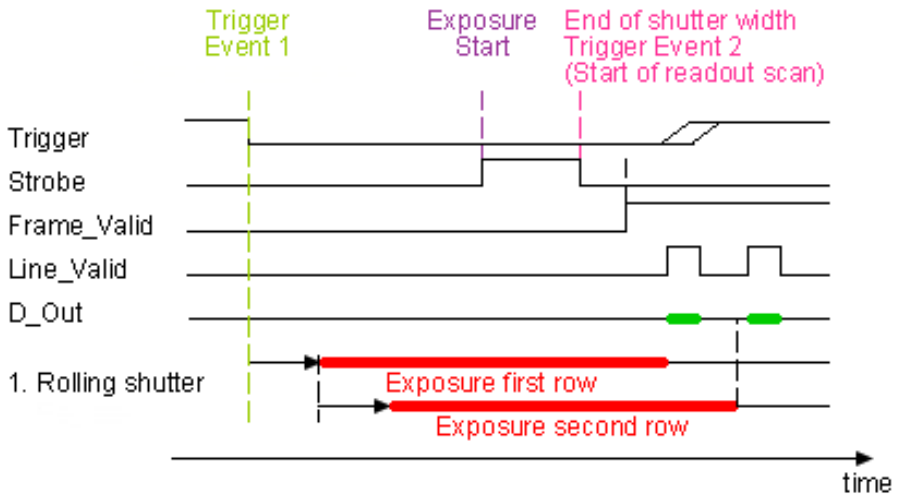


Figure 1: Rolling shutter

In **rolling shutter mode**, a doubling of the exposure time clearly leads to a doubling of the gray value:

$$GrayValueFactor = \frac{ExposureTime2}{ExposureTime1}$$

Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

Feature	Description
Manufacturer	Aptina
Sensor name	AR0331
Max. frames per second	22.2
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	-
DNR (normal / HDR) [dB][1/]	- / -
Image size	1/3
Number of effective pixels	2064 x 1578 (V)
Unit cell size	2.2µm (H) x 2.2µm (V)
ADC resolution / output	12 bit → 8/10/12

[1] Not quantifiable according to EMVA1288

1.20.4.6.2 Spectral Sensitivity

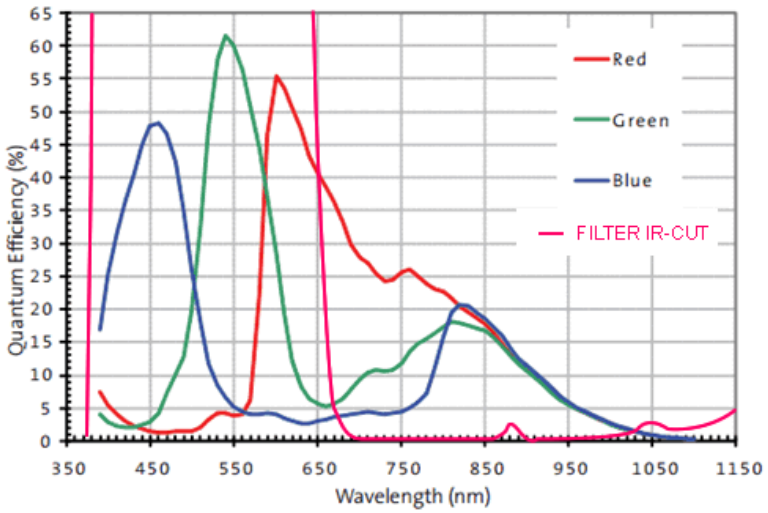


Figure 2: Spectral sensitivity mvBlueFOX3-1031C

Name	Value
PixelClock	74.25
VerticalBlank	15

1.20.4.6.3 Timings

1.20.4.6.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (2176)) + (VerticalBlank * 2176)}{PixelClock} \quad \frac{}{PixelClock}$$

$$FPS_max = \frac{1000000}{FrameTime}$$

1.20.4.6.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (2176)) + (VerticalBlank * 2176)}{PixelClock} \quad \frac{}{PixelClock}$$

$$FPS_max = \frac{1000000}{ExposureTime}$$

Device Feature And Property List (p. 481)

1.20.4.6.4 Device Feature And Property List

- **mvBlueFOX3-1031C Features** (p. 481)

1.20.4.6.4.1 mvBlueFOX3-1031C Features

1.20.4.7 mvBlueFOX3-1100 (11 Mpix [3856 x 2764])

1.20.4.7.1 Introduction

The sensor features several acquisition modes (`"mv Shutter Mode"`):

- rolling shutter (`"mvRollingShutter"`) and
- global reset release shutter (`"mvGlobalReset"`).

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time:

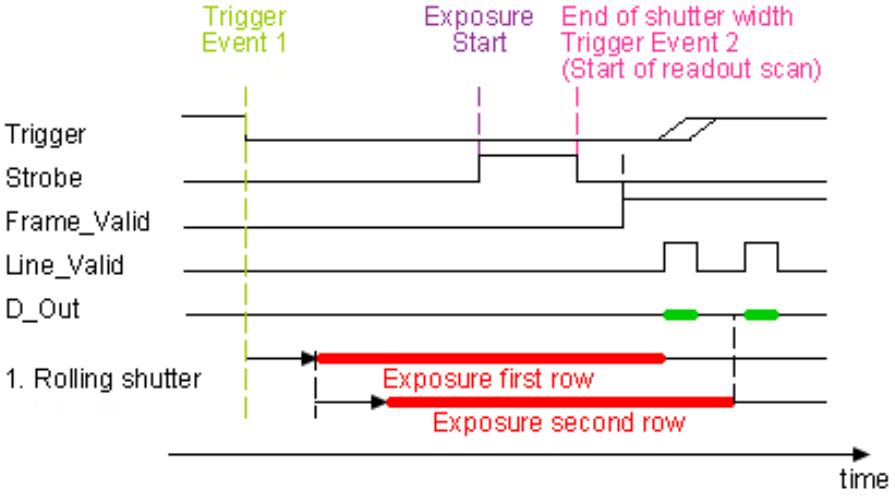


Figure 1: Rolling shutter

In **rolling shutter mode**, a doubling of the exposure time clearly leads to a doubling of the gray value:

$$\text{GrayValueFactor} = \frac{\text{ExposureTime2}}{\text{ExposureTime1}}$$

Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

The **global reset release shutter**, which is only available in triggered operation, starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:

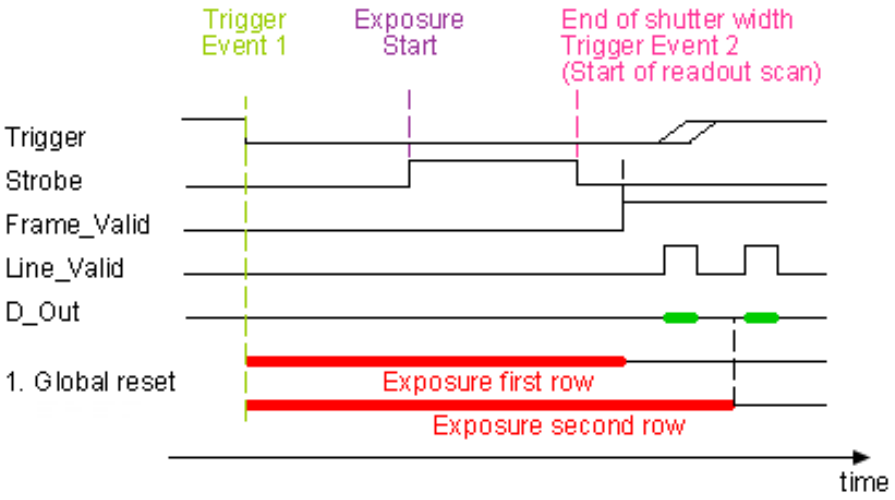


Figure 2: Global reset reslease shutter

Note

The max. exposure time of the `global reset release` mode is 206ms.

Any triggered operation (like software trigger) leads inevitably to the **global reset mode** (even if the shutter mode property is still set to "`mvRollingShutter`").

Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no ambient light and the flash duration is shorter or equal to the exposure time.

Thus, the gray value factor of the first line, which is similar to the one of the rolling shutter, is different to the factor of the last line:

$$\text{GreyValueFactor (last line)} = \frac{\text{ExposureTime2} + \text{FrameTime}}{\text{ExposureTime1} + \text{FrameTime}}$$

Feature	Description
Manufacturer	Aptina
Sensor name	MT9J003
Max. frames per second	7
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	37.2
DNR (normal / HDR) [dB][1/]	56 / -
Image size	1/2.35
Number of effective pixels	3856 (H) x 2764 (V)
Unit cell size	1.67µm (H) x 1.67µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.7.2 Spectral Sensitivity

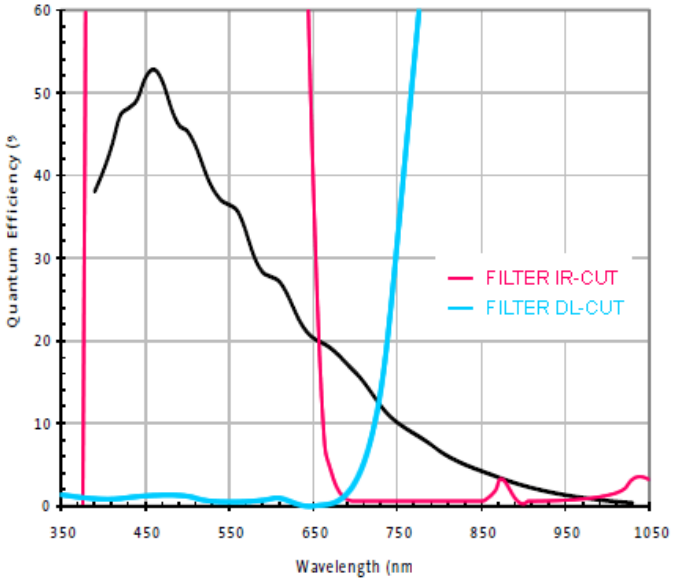


Figure 3: Spectral sensitivity mvBlueFOX3-1100G

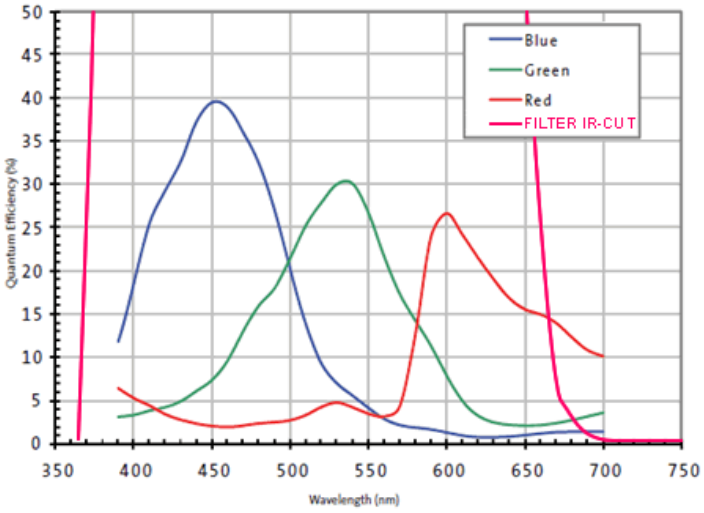


Figure 4: Spectral sensitivity mvBlueFOX3-1100C

Name	Value
PixelClock	81.25
VerticalBlank	15

1.20.4.7.3 Timings

1.20.4.7.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (1648)) + (VerticalBlank * 1648)}{PixelClock} + \frac{1648}{PixelClock}$$

$$FPS_{max} = \frac{1000000}{FrameTime}$$

1.20.4.7.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (1648)) + (VerticalBlank * 1648)}{PixelClock} + \frac{1648}{PixelClock}$$

$$FPS_{max} = \frac{1000000}{ExposureTime}$$

Note

The maximum duration of the signal *ExposureActive* used in *Counter*, *Timer*, or as digital output signal is limited to 207ms (80 MHz pixel clock). This limit also applies to the **chunk data** (p. 113) parameter *ChunkExposureTime*. It does not matter which exposure time is set, the value of this parameter will always be below or equal to 207ms (80 MHz pixel clock). ExposureEndEvents are generated with the help of this signal, too and for this reason they have too small timestamps above this limit.

Device Feature And Property List (p. 485)

1.20.4.7.4 Device Feature And Property List

- **mvBlueFOX3-1100G Features** (p. 485)
- **mvBlueFOX3-1100C Features** (p. 485)

1.20.4.7.4.1 mvBlueFOX3-1100G Features

1.20.4.7.4.2 mvBlueFOX3-1100C Features

1.20.4.8 mvBlueFOX3-1140 (14 Mpix [4384 x 3288])

1.20.4.8.1 Introduction

The sensor features several acquisition modes (`"mv Shutter Mode"`):

- rolling shutter (`"mvRollingShutter"`) and
- global reset release shutter (`"mvGlobalReset"`).

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time:

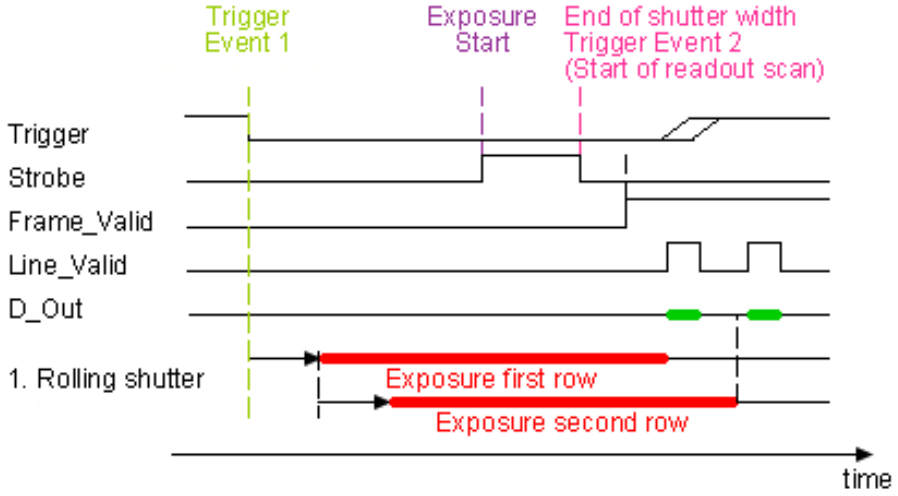


Figure 1: Rolling shutter

In **rolling shutter mode**, a doubling of the exposure time clearly leads to a doubling of the gray value:

$$\text{GrayValueFactor} = \frac{\text{ExposureTime2}}{\text{ExposureTime1}}$$

Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

The **global reset release shutter**, which is only available in triggered operation, starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:

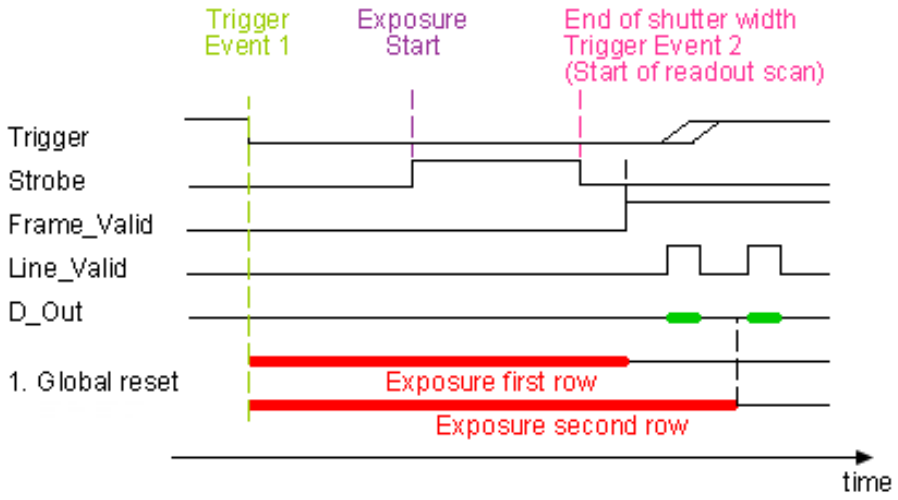


Figure 2: Global reset reslease shutter

Note

The max. exposure time of the `global reset release` mode is 206ms.

Any triggered operation (like software trigger) leads inevitably to the **global reset mode** (even if the shutter mode property is still set to "`mvRollingShutter`").

Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no ambient light and the flash duration is shorter or equal to the exposure time.

Thus, the gray value factor of the first line, which is similar to the one of the rolling shutter, is different to the factor of the last line:

$$\text{GreyValueFactor (last line)} = \frac{\text{ExposureTime2} + \text{FrameTime}}{\text{ExposureTime1} + \text{FrameTime}}$$

With 80 MHz, for example, the `FrameTime` is about 194 ms. I.e, doubling from e.g. 1ms to 2ms won't be visible effectively.

Feature	Description
Manufacturer	Aptina
Sensor name	MT9F002
Max. frames per second	6
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	35.1
DNR (normal / HDR) [dB][1/]	57.3 / -
Image size	1/2.3
Number of effective pixels	4608 (H) x 3288 (V)
Unit cell size	1.4µm (H) x 1.4µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.4.8.2 Spectral Sensitivity

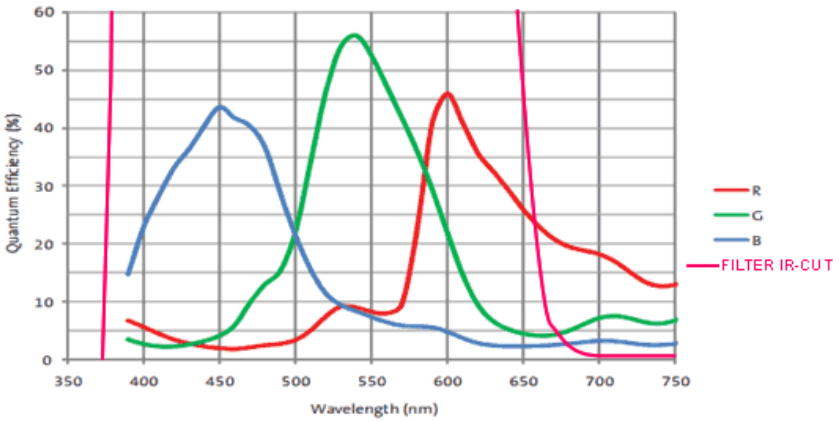


Figure 3: Spectral sensitivity mvBlueFOX3-1140C

Name	Value
PixelClock	96.88
VerticalBlank	15

1.20.4.8.3 Timings

1.20.4.8.3.1 Free running mode

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = (ImageHeight * (\frac{4702}{PixelClock})) + (VerticalBlank * \frac{4702}{PixelClock})$$

$$FPS_max = \frac{1000000}{FrameTime}$$

1.20.4.8.3.2 Snapshot mode

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = (ImageHeight * (\frac{4702}{PixelClock})) + (VerticalBlank * \frac{4702}{PixelClock})$$

$$FPS_max = \frac{1000000}{ExposureTime}$$

Device Feature And Property List (p. 489)

1.20.4.8.4 Device Feature And Property List

- **mvBlueFOX3-1140C Features** (p. 489)

1.20.4.8.4.1 mvBlueFOX3-1140C Features

1.21 Appendix C. Tested ARM platforms

MATRIX VISION devices can run on ARM-based Linux platforms without limitations regarding available feature sets or API functions. However, each platform may have its own limits in terms of achievable data throughput, RAM or bus speeds. Apart from that, each platform may also come with its own specific set of challenges. Therefore, certain modifications may need to be adapted in order to get your devices run at maximum performance.

This chapter contains test results from different ARM platforms, as well as the specific information on each platform, especially changes that need to be applied to achieve better performance.

The following platforms have been tested by MATRIX VISION:

System	ARM-Architecture	Technology Test Results					Approx. price	Performance	Suitable for	More information
		USB2.0	USB3.0	GigE	10GigE	PCIe				
NVIDIA Jetson AGX Xavier (p. 490)	NVIDIA Carmel ARMv8.2						ca. 650-700$		Demanding Applications	nvidia.com
NVIDIA Jetson Xavier NX (p. 493)	NVIDIA Carmel ARMv8.2						ca. 700$		Demanding Applications	nvidia.com
NVIDIA Jetson Nano (p. 496)	ARM Cortex-A57						90-100$		Mid-Range Applications	nvidia.com
NVIDIA Jetson TX2 (p. 499)	NVIDIA Denver 2 and ARM Cortex-A57						ca. 400$		Demanding Applications	nvidia.com
i.MX8M Mini (p. 502)	ARM Cortex-A53								Mid-Range Applications	nxp.com
Raspberry Pi 4 (p. 504)	ARM Cortex-A72						35-75$		Price Sensitive Projects	raspberrypi.org
Raspberry Pi Compute Module 4	ARM Cortex-A72						30-80$		Price Sensitive Projects	raspberrypi.org

 The system delivers good performance with this device.

 The system doesn't work with this device.

 The developer kit doesn't work with this device because it provides no PCI Express interface.

 The system delivers limited performance with this device.

 The system hasn't been tested yet with this device.

Appendices:

- **C.1 ARM64 based devices** (p. 490)
- **C.2 ARMhf based devices** (p. 504)

1.21.1 C.1 ARM64 based devices

- **NVIDIA Jetson AGX Xavier** (p. 490)
- **NVIDIA Jetson Xavier NX** (p. 493)
- **NVIDIA Jetson Nano** (p. 496)
- **NVIDIA Jetson TX2** (p. 499)
- **i.MX8M Mini** (p. 502)

1.21.1.1 NVIDIA Jetson AGX Xavier

CPU	NVIDIA Carmel ARMv8.2 @ 2.26GHz
Cores	8
RAM	32GB
USB2.0 Interfaces	4
USB3.1 Interfaces	3
Ethernet	10/100/1000 MBit
PCIe	1x8 + 1x4 + 1x2 + 2x1 Gen 4.0

1.21.1.1.1 General

Note

The above table describes the specification of the NVIDIA Jetson AGX Xavier Developer Kit.

1.21.1.1.2 Benchmarks

1.21.1.1.2.1 USB3.0 Performance Test



Test setup

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

1.21.1.1.2.2 Additional Settings Applied On The System

Note

Follow the links to find out more about these parameters.

1.21.1.1.2.3 Results The following tests have been performed using different de-Bayering scenarios to achieve the max. FPS while maintaining 0 lost images. The CPU load during the acquisition is also documented below.

Scenarios that have been tested are listed as follows:

1. When de-Bayering is carried out on the camera: The camera delivers RGB8 image data to the host system. This setting results in a lower CPU load but a lower frame rate.
2. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.

3. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	RGB8 (on camera) -> RGB8 (on host)	25	240.93	∼18%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> RGB8 (on host)	119	379.88	∼30%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	119	379.86	∼20%

1.21.1.1.3 Specific Settings

1.21.1.1.3.1 Optimizing USB Performance To use the camera following modifications are necessary in the standard configuration of the NVIDIA Jetson AGX Xavier board:

Note

Depending on the version of the L4T on the device, it might be necessary to update the software to the latest version which can be obtained from the NVIDIA `Developer page`.

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

More specific possibilities to configure the mentioned parameters are described within the following chapters.

Note

The following options to modify usbcore settings allow to setup the device according to the requirements of **"USB3 Vision"** cameras already at boot time. Once applied no changes at runtime will be necessary.

1.21.1.1.3.2 Enabling USB3.0 Support On Some devices it might be necessary to enable the USB3.0 support as it seems as the USB bus runs on USB2.0 speed per default.

Change "usb_port_owner_info=0" to "usb_port_owner_info=2" (this will change the USB port behavior from USB 2.0 to USB 3.0).

To change the USB mode, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usb_port_owner_info=2 root=/dev/mmcblk0p1
```

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.1.3.3 Increasing the allowed USB buffer memory Increase the memory which you have to adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line:

See also

- **Increasing Kernel memory** (p. 54)

Set *"usbcore.usbfs_memory_mb=256"* (this will increase the buffer of the USB bus) E.g.

```
APPEND ${cbootargs}usbcore.usbfs_memory_mb=256 root=/dev/mmcblk0p1
```

1.21.1.1.3.4 Disabling USB Autosuspend In some cases the operating system tries to power down the camera. This can be avoided by disabling the USB autosuspend mode.

To disable the usbcore.autosuspend functionality, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usbcore.autosuspend=-1 root=/dev/mmcblk0p1
```

See also

- **Disabling The Auto-Suspend Mode** (p. 55)

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.2 NVIDIA Jetson Xavier NX

CPU	NVIDIA Carmel ARMv8.2 @ 1.9GHz
Cores	6
RAM	8GB
USB3.1 Interfaces	4
Ethernet	10/100/1000 MBit
PCIe	1x1 + 1x4 Gen 3.0

1.21.1.2.1 General

Note

The above table describes the specification of the NVIDIA Jetson Xavier NX Developer Kit.

1.21.1.2.2 Benchmarks

1.21.1.2.2.1 USB3.0 Performance



Test setup

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

1.21.1.2.2.2 Additional Settings Applied On The System

Note

Follow the links to find out more about these parameters.

1.21.1.2.2.3 Results The following tests have been performed using different de-Bayering scenarios to achieve the max. FPS while maintaining 0 lost images. The CPU load during the acquisition is also documented below.

Scenarios that have been tested are listed as follows:

1. When de-Bayering is carried out on the camera: The camera delivers RGB8 image data to the host system. This setting results in a lower CPU load but a lower frame rate.
2. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
3. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	RGB8 (on camera) -> RGB8 (on host)	25	240.93	∼24%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> RGB8 (on host)	119	379.91	∼48%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	119	379.86	∼27%

1.21.1.2.3 Specific Settings

1.21.1.2.3.1 Optimizing USB Performance To use the camera following modifications are necessary in the standard configuration of the NVIDIA Jetson Xavier NX board:

Note

Depending on the version of the L4T on the device, it might be necessary to update the software to the latest version which can be obtained from the NVIDIA `Developer page`.

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

More specific possibilities to configure the mentioned parameters are described within the following chapters.

Note

The following options to modify usbcore settings allow to setup the device according to the requirements of **"USB3 Vision"** cameras already at boot time. Once applied no changes at runtime will be necessary.

1.21.1.2.3.2 Enabling USB3.0 Support On Some devices it might be necessary to enable the USB3.0 support as it seems as the USB bus runs on USB2.0 speed per default.

Change "usb_port_owner_info=0" to "usb_port_owner_info=2" (this will change the USB port behavior from USB 2.0 to USB 3.0).

To change the USB mode, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usb_port_owner_info=2 root=/dev/mmcblk0p1
```

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.2.3.3 Increasing the allowed USB buffer memory Increase the memory which you have to adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line:

See also

- **Increasing Kernel memory** (p. 54)

Set *"usbcore.usbfs_memory_mb=256"* (this will increase the buffer of the USB bus) E.g.

```
APPEND ${cbootargs}usbcore.usbfs_memory_mb=256 root=/dev/mmcblk0p1
```

1.21.1.2.3.4 Disabling USB Autosuspend In some cases the operating system tries to power down the camera. This can be avoided by disabling the USB autosuspend mode.

To disable the usbcore.autosuspend functionality, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usbcore.autosuspend=-1 root=/dev/mmcblk0p1
```

See also

- **Disabling The Auto-Suspend Mode** (p. 55)

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.3 NVIDIA Jetson Nano

CPU	Cortex-A57 @ 1.43 GHz
Cores	4
RAM	4GB
USB2.0 Interfaces	1
USB3.0 Interfaces	4
Ethernet	10/100/1000 MBit
PCIe	x1/x2/x4 Gen 2.0

1.21.1.3.1 General

Note

The above table describes the specification of the NVIDIA Jetson Nano Developer Kit.

1.21.1.3.2 Benchmarks

1.21.1.3.2.1 USB3.0 Performance Test

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

1.21.1.3.2.2 Additional Settings Applied On The System

Note

Follow the links to find out more about these parameters.

1.21.1.3.2.3 Results The following tests have been performed using different de-Bayering scenarios to achieve the max. FPS while maintaining 0 lost images. The CPU load during the acquisition is also documented below.

Scenarios that have been tested are listed as follows:

1. When de-Bayering is carried out on the camera: The camera delivers RGB8 image data to the host system. This setting results in a lower CPU load but a lower frame rate.
2. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
3. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	RGB8 (on camera) -> RGB8 (on host)	25	240.92	40%

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> RGB8 (on host)	98	312.26	∼90%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	119	379.86	50%

1.21.1.3.3 Remarks

1.21.1.3.3.1 Choose the right power supply The Jetson Nano has 2 power supply possibilities: via the micro-USB connection or via the Barrel Jack connection.

The power (by default 10W) via the micro-USB connector is not sufficient if you want to connect other peripherals (e.g. keyboard, mouse, cameras, etc...) that draw current from the board. So if when powering the USB/USB3 camera through the USB bus (i.e. no external power supply), please supply the board with power through the Barrel Jack connector (4A@5V).

Otherwise the system will throttle due to over-current.

1.21.1.3.4 Specific Settings

1.21.1.3.4.1 Optimizing USB Performance To use the camera following modifications are necessary in the standard configuration of the NVIDIA Jetson Nano board:

> **Note**
>
> Depending on the version of the L4T on the device, it might be necessary to update the software to the latest version which can be obtained from the NVIDIA `Developer page`.

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

More specific possibilities to configure the mentioned parameters are described within the following chapters.

Note

The following options to modify usbcore settings allow to setup the device according to the requirements of **"USB3 Vision"** cameras already at boot time. Once applied no changes at runtime will be necessary.

1.21.1.3.4.2 Enabling USB3.0 Support On Some devices it might be necessary to enable the USB3.0 support as it seems as the USB bus runs on USB2.0 speed per default.

Change "usb_port_owner_info=0" to "usb_port_owner_info=2" (this will change the USB port behavior from USB 2.0 to USB 3.0).

To change the USB mode, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usb_port_owner_info=2 root=/dev/mmcblk0p1
```

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.3.4.3 Increasing the allowed USB buffer memory Increase the memory which you have to adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line:

See also

- **Increasing Kernel memory** (p. 54)

Set *"usbcore.usbfs_memory_mb=256"* (this will increase the buffer of the USB bus) E.g.

```
APPEND ${cbootargs}usbcore.usbfs_memory_mb=256 root=/dev/mmcblk0p1
```

1.21.1.3.4.4 Disabling USB Autosuspend In some cases the operating system tries to power down the camera. This can be avoided by disabling the USB autosuspend mode.

To disable the usbcore.autosuspend functionality, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usbcore.autosuspend=-1 root=/dev/mmcblk0p1
```

See also

- **Disabling The Auto-Suspend Mode** (p. 55)

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.4 NVIDIA Jetson TX2

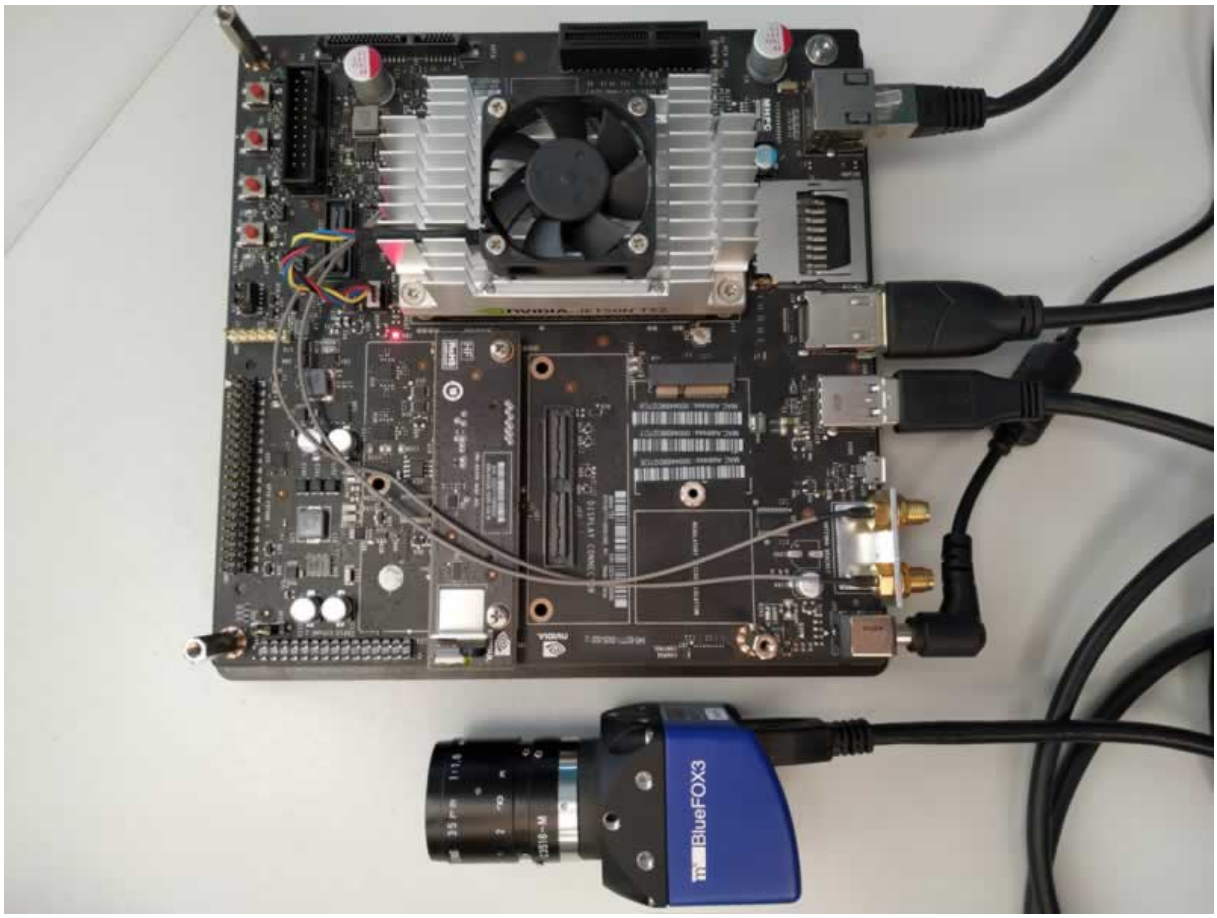
CPU	ARM Cortex-A57 @ 2GHz NVIDIA Denver2 @ 2GHz
Cores	4 2
RAM	8GB
USB2.0 Interfaces	1
USB3.0 Interfaces	1

1.21.1.4.1 General

> **Note**
>
> The above table describes the specification of the NVIDIA Jetson TX2 Developer Kit.

1.21.1.4.2 Benchmarks

1.21.1.4.2.1 USB3.0 Performance Test



Test setup

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

1.21.1.4.2.2 Additional Settings Applied On The System

> **Note**
>
> Follow the links to find out more about these parameters.

1.21.1.4.2.3 Results The following tests have been performed using different de-Bayering scenarios to achieve the max. FPS while maintaining 0 lost images. The CPU load during the acquisition is also documented below.

Scenarios that have been tested are listed as follows:

1. When de-Bayering is carried out on the camera: The camera delivers RGB8 image data to the host system. This setting results in a lower CPU load but a lower frame rate.
2. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
3. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	RGB8 (on camera) -> RGB8 (on host)	25	240.92	∼37%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> RGB8 (on host)	119	379.86	∼88%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	119	379.90	∼40%

1.21.1.4.3 Specific Settings

1.21.1.4.3.1 Optimizing USB Performance To use the camera following modifications are necessary in the standard configuration of the NVIDIA Jetson TX2 board:

Note

Depending on the version of the L4T on the device, it might be necessary to update the software to the latest version which can be obtained from the NVIDIA `Developer page`.

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

More specific possibilities to configure the mentioned parameters are described within the following chapters.

Note

The following options to modify usbcore settings allow to setup the device according to the requirements of **"USB3 Vision"** cameras already at boot time. Once applied no changes at runtime will be necessary.

1.21.1.4.3.2 Enabling USB3.0 Support On Some devices it might be necessary to enable the USB3.0 support as it seems as the USB bus runs on USB2.0 speed per default.

Change "usb_port_owner_info=0" to "usb_port_owner_info=2" (this will change the USB port behavior from USB 2.0 to USB 3.0).

To change the USB mode, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usb_port_owner_info=2 root=/dev/mmcblk0p1
```

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.4.3.3 Increasing the allowed USB buffer memory Increase the memory which you have to adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line:

See also

- **Increasing Kernel memory** (p. 54)

Set *"usbcore.usbfs_memory_mb=256"* (this will increase the buffer of the USB bus) E.g.

```
APPEND ${cbootargs}usbcore.usbfs_memory_mb=256 root=/dev/mmcblk0p1
```

1.21.1.4.3.4 Disabling USB Autosuspend In some cases the operating system tries to power down the camera. This can be avoided by disabling the USB autosuspend mode.

To disable the usbcore.autosuspend functionality, adapt the boot parameters in `/boot/extlinux/extlinux`.conf at the end of the `"APPEND"` line: E.g.

```
APPEND ${cbootargs} usbcore.autosuspend=-1 root=/dev/mmcblk0p1
```

See also

- **Disabling The Auto-Suspend Mode** (p. 55)

Note

Additional parameters e.g. 'usbcore.usbfs_memory_mb' are not shown in the above sample.

1.21.1.5 i.MX8M Mini

CPU	ARM Cortex®-A53 @ 1.6GHz
Cores	4
RAM	1 GB
USB2.0 Interfaces	2
USB3.0 Interfaces	None
Ethernet	1000 MBit
PCIe	1 x 1 Lane Gen 2.0

1.21.1.5.1 General The carrier-board used in this test: MBa8Mx from TQ-Systems GmbH

Note

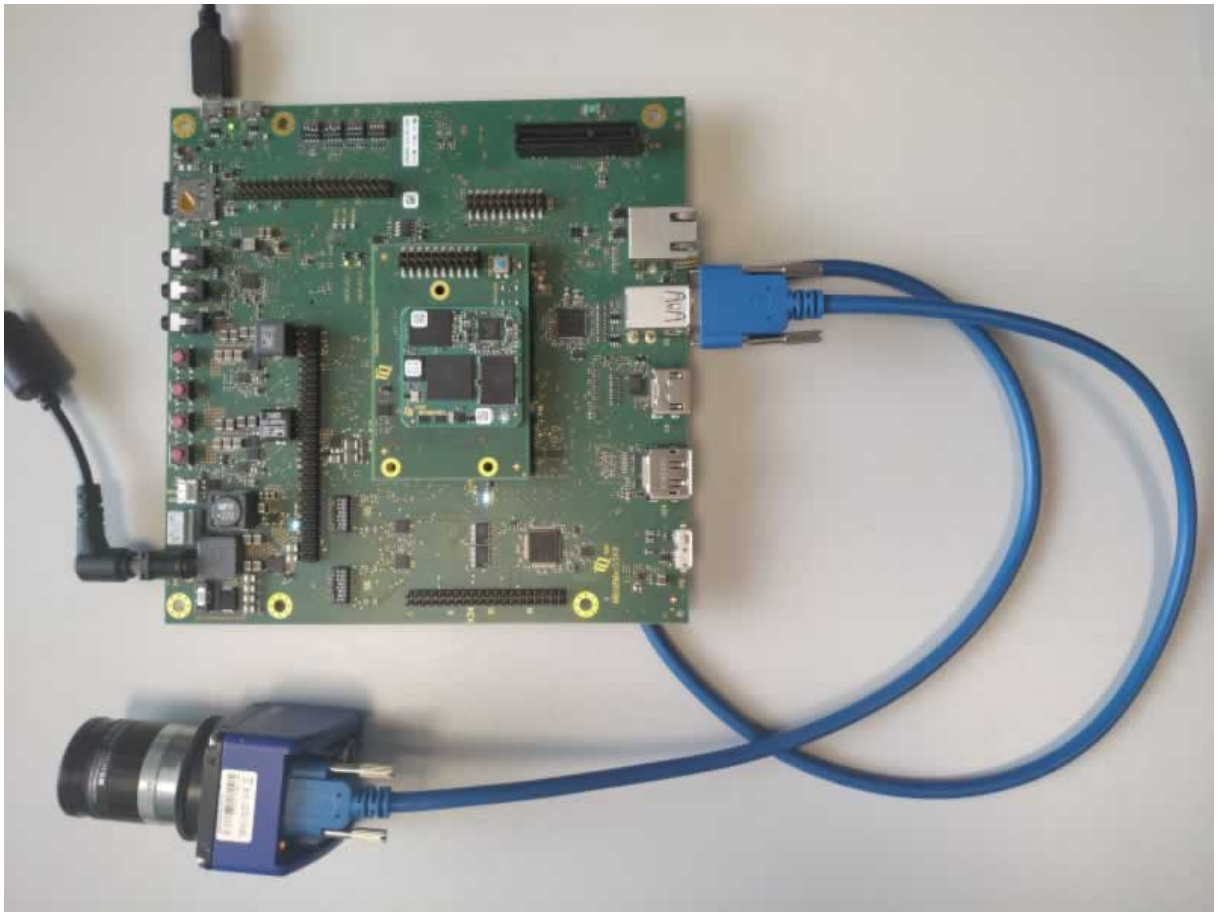
If you are looking for more information and guidance about installing mvIMPACT Acquire driver packages via the Yocto Project, please choose an API-manual suited for your programming language and then go to chapter "Installation From Private Setup Routines -> Embedded Linux -> Yocto Project". All API-manuals can be found under `https://www.matrix-vision.com/manuals/`.

1.21.1.5.2 Benchmarks

1.21.1.5.2.1 USB3.0 Performance Test

Note

The i.MX8M Mini doesn't have a USB3.0 host controller. USB3 Vision™ devices can therefore operate with USB2.0 speed only.



Test setup

1.21.1.5.2.2 Results The following tests have been performed using different de-Bayering scenarios to achieve the max. FPS while maintaining 0 lost images. The CPU load during the acquisition is also documented below.

Scenarios that have been tested are listed as follows:

1. When de-Bayering is carried out on the camera: The camera delivers RGB8 image data to the host system. This setting results in a lower CPU load but a lower frame rate.
2. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
3. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load (averaged over 4 cores)
mvBlueFOX3-2024C	1936 x 1216	RGB8 (on camera) -> RGB8 (on host)	5	35.3	~2.8%
mvBlueFOX3-2024C	1936 x 1216	BayerRG8 (on camera) -> RGB8 (on host)	15.2	35.7	~25%
mvBlueFOX3-2024C	1936 x 1216	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	15.2	35.7	~3.7%

1.21.2 C.2 ARMhf based devices

- **Raspberry Pi 4** (p. 504)

1.21.2.1 Raspberry Pi 4

1.21.2.1.1 General The Raspberry Pi 4 is a well priced platform regarding its performance.

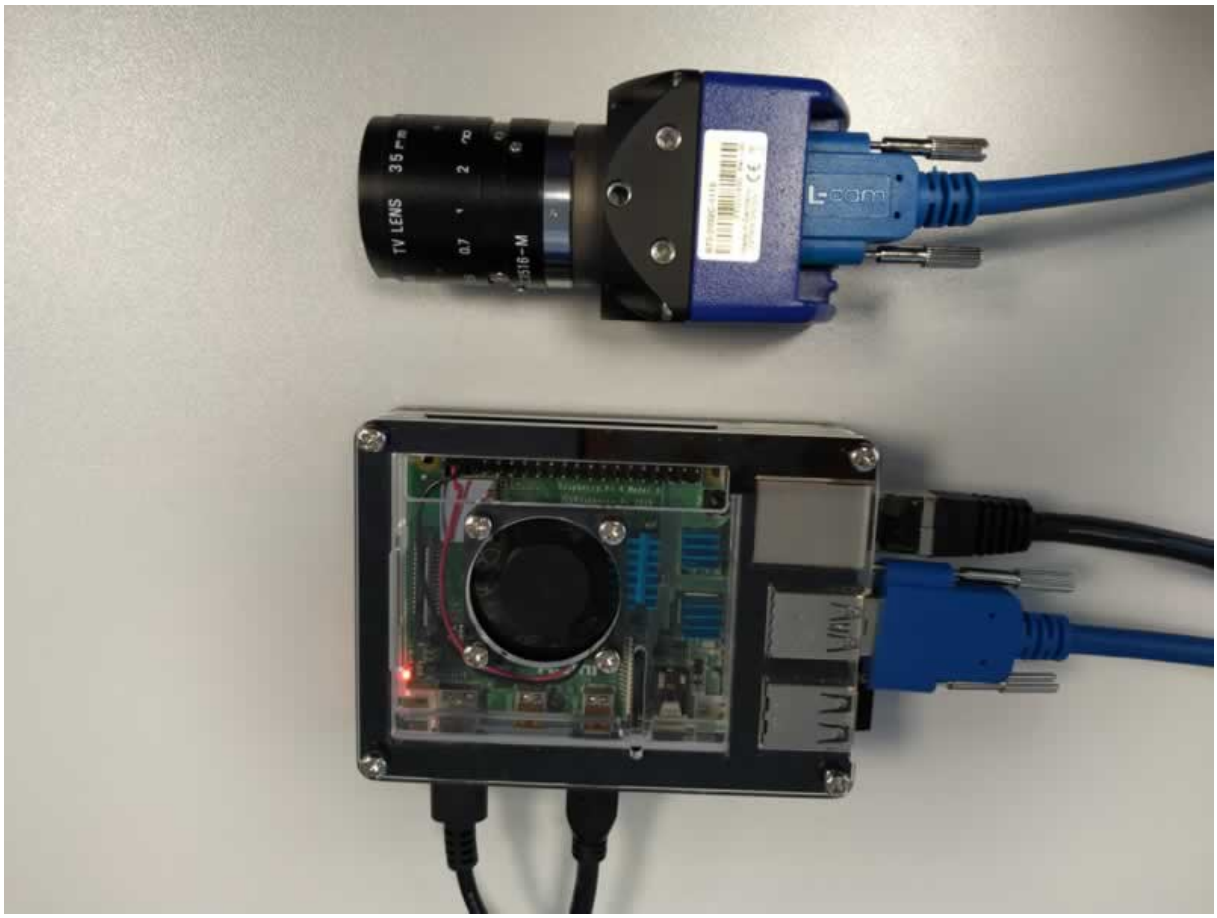
CPU	Cortex-A72 @ 1500MHz
Cores	4
RAM	1/2/4/8 GB
USB2.0 Interfaces	2
USB3.0 Interfaces	2
Ethernet	10/100/1000 MBit

Note

For the following benchmark the 4GB version of the Raspberry Pi 4 with Raspbian OS has been used.

1.21.2.1.2 Benchmarks

1.21.2.1.2.1 USB3.0 Performance Test



Test setup

Setting	Value	Description
usbcore.autosuspend	-1	Disables USB auto-suspend mode. See **Disabling The Auto-Suspend Mode** (p. 55)
usbcore.usbfs_memory_mb	256 MB	Increases the kernel memory for USB traffic. See **Increasing Kernel memory** (p. 54)

1.21.2.1.2.2 Additional Settings Applied On The System

1.21.2.1.2.3 Results The following tests have been performed using different de-Bayering scenarios to achieve the max. FPS while maintaining 0 lost images. The CPU load during the acquisition is also documented below.

Scenarios that have been tested are listed as follows:

1. When de-Bayering is carried out on the camera: The camera delivers RGB8 image data to the host system. This setting results in a lower CPU load but a lower frame rate.
2. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.

3. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-↩ Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	RGB8 (on camera) -> RGB8 (on host)	25	240.92	∼29%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> RGB8 (on host)	40	127.47	∼84%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlueFOX3-2032C	2064 x 1544	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	110	350.58	∼45%